Lecture Notes in Networks and Systems 660

Suchismita Chinara · Asis Kumar Tripathy · Kuan-Ching Li · Jyoti Prakash Sahoo · Alekha Kumar Mishra *Editors*

Advances in Distributed Computing and Machine Learning

Proceedings of ICADCML 2023

Springer

Lecture Notes in Networks and Systems

Volume 660

Series Editor

Janusz Kacprzyk, Systems Research Institute, Polish Academy of Sciences, Warsaw, Poland

Advisory Editors

Fernando Gomide, Department of Computer Engineering and Automation—DCA, School of Electrical and Computer Engineering—FEEC, University of Campinas—UNICAMP, São Paulo, Brazil

Okyay Kaynak, Department of Electrical and Electronic Engineering, Bogazici University, Istanbul, Türkiye

Derong Liu, Department of Electrical and Computer Engineering, University of Illinois at Chicago, Chicago, USA
Institute of Automation, Chinese Academy of Sciences, Beijing, China

Witold Pedrycz, Department of Electrical and Computer Engineering, University of Alberta, Alberta, Canada
Systems Research Institute, Polish Academy of Sciences, Warsaw, Poland

Marios M. Polycarpou, Department of Electrical and Computer Engineering, KIOS Research Center for Intelligent Systems and Networks, University of Cyprus, Nicosia, Cyprus

Imre J. Rudas, Óbuda University, Budapest, Hungary

Jun Wang, Department of Computer Science, City University of Hong Kong, Kowloon, Hong Kong

The series "Lecture Notes in Networks and Systems" publishes the latest developments in Networks and Systems—quickly, informally and with high quality. Original research reported in proceedings and post-proceedings represents the core of LNNS.

Volumes published in LNNS embrace all aspects and subfields of, as well as new challenges in, Networks and Systems.

The series contains proceedings and edited volumes in systems and networks, spanning the areas of Cyber-Physical Systems, Autonomous Systems, Sensor Networks, Control Systems, Energy Systems, Automotive Systems, Biological Systems, Vehicular Networking and Connected Vehicles, Aerospace Systems, Automation, Manufacturing, Smart Grids, Nonlinear Systems, Power Systems, Robotics, Social Systems, Economic Systems and other. Of particular value to both the contributors and the readership are the short publication timeframe and the world-wide distribution and exposure which enable both a wide and rapid dissemination of research output.

The series covers the theory, applications, and perspectives on the state of the art and future developments relevant to systems and networks, decision making, control, complex processes and related areas, as embedded in the fields of interdisciplinary and applied sciences, engineering, computer science, physics, economics, social, and life sciences, as well as the paradigms and methodologies behind them.

Indexed by SCOPUS, INSPEC, WTI Frankfurt eG, zbMATH, SCImago.

All books published in the series are submitted for consideration in Web of Science.

For proposals from Asia please contact Aninda Bose (aninda.bose@springer.com).

Suchismita Chinara · Asis Kumar Tripathy ·
Kuan-Ching Li · Jyoti Prakash Sahoo ·
Alekha Kumar Mishra
Editors

Advances in Distributed Computing and Machine Learning

Proceedings of ICADCML 2023

Springer

Editors
Suchismita Chinara
Department of Computer Science
and Engineering
National Institute of Technology Rourkela
Rourkela, Odisha, India

Kuan-Ching Li
Department of Computer Science
and Information Engineering (CSIE)
Providence University
Taichung, Taiwan

Alekha Kumar Mishra
Department of Computer Science
and Engineering
National Institute of Technology
Jamshedpur, Jharkhand, India

Asis Kumar Tripathy
School of Information Technology
and Engineering (SITE)
Vellore Institute of Technology
Vellore, India

Jyoti Prakash Sahoo
Department of Computer Science
and Information Technology
Shiksha 'O' Anusandhan
Bhubaneswar, Odisha, India

ISSN 2367-3370 ISSN 2367-3389 (electronic)
Lecture Notes in Networks and Systems
ISBN 978-981-99-1202-5 ISBN 978-981-99-1203-2 (eBook)
https://doi.org/10.1007/978-981-99-1203-2

© The Editor(s) (if applicable) and The Author(s), under exclusive license to Springer Nature Singapore Pte Ltd. 2023

This work is subject to copyright. All rights are solely and exclusively licensed by the Publisher, whether the whole or part of the material is concerned, specifically the rights of translation, reprinting, reuse of illustrations, recitation, broadcasting, reproduction on microfilms or in any other physical way, and transmission or information storage and retrieval, electronic adaptation, computer software, or by similar or dissimilar methodology now known or hereafter developed.
The use of general descriptive names, registered names, trademarks, service marks, etc. in this publication does not imply, even in the absence of a specific statement, that such names are exempt from the relevant protective laws and regulations and therefore free for general use.
The publisher, the authors, and the editors are safe to assume that the advice and information in this book are believed to be true and accurate at the date of publication. Neither the publisher nor the authors or the editors give a warranty, expressed or implied, with respect to the material contained herein or for any errors or omissions that may have been made. The publisher remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

This Springer imprint is published by the registered company Springer Nature Singapore Pte Ltd.
The registered company address is: 152 Beach Road, #21-01/04 Gateway East, Singapore 189721, Singapore

Preface

ICADCML-2023

The book *Advances in Distributed Computing and Machine Learning* is a collection of the latest research in the field of distributed computing and machine learning, presented at the 4th International Conference on Advances in Distributed Computing and Machine Learning (ICADCML 2023) held at the National Institute of Technology (NIT), Rourkela, India. The conference, organized by the Department of Computer Science and Engineering at NIT Rourkela, brought together leading researchers, academics, and industry professionals from around the world to share their insights and findings on the latest advances in this rapidly evolving field.

The book includes a wide range of topics, including Cloud Computing, the Internet of Things (IoT), Blockchain technology, Distributed Systems and Algorithms, and their technical applications in distributed environments. The research papers included in the book provide valuable insights into the use of Data Analytics, AI, and Machine Learning to address complex real-world problems, distributed deep learning and reinforcement learning, distributed optimization and its applications, distributed systems security, and the future of distributed computing.

The papers included in this proceedings were selected through a rigorous peer-review process and represent the latest and most innovative research in the field. This book is an essential resource for researchers, academics, and professionals working in the field of distributed computing and machine learning, providing them with a comprehensive understanding of the latest developments in the field. It will also be useful for graduate students and researchers who are looking to learn more about the state of the art in the field.

We would like to express our gratitude to the organizing committee, the keynote speakers, and all the authors and participants who made ICADCML 2023 a success. We also would like to thank the reviewers for their valuable contributions in maintaining the high standard of the conference proceedings. We hope that this book will be a valuable resource for the research community, and will provide inspiration for future research and innovation in the field of distributed computing and machine learning.

Rourkela, India — Suchismita Chinara
Vellore, India — Asis Kumar Tripathy
Taichung, Taiwan — Kuan-Ching Li
Bhubaneswar, India — Jyoti Prakash Sahoo
Jamshedpur, India — Alekha Kumar Mishra

Contents

NS3-Based Performance Assessment of Routing Protocols AODV, OLSR and DSDV for VANETs 1
Madhuri Malakar, Bidisha Bhabani, and Judhistir Mahapatro

A Novel Blockchain-Based Smart Contract for Real Estate Management 15
Ashish Kumar Mohanty and D. Chandrasekhar Rao

A Review on VM Placement Scheme Using Optimization Algorithms 27
Akanksha Tandon, Sudhanshu Kulshresha, and Sanjeev Patel

Use of Blockchain to Prevent Distributed Denial-of-Service (DDoS) Attack: A Systematic Literature Review 39
Md. Rittique Alam, Sabique Islam Khan, Sumaiya Binte Zilani Chowa, Anupam Hayath Chowdhury, S. Rayhan Kabir, and Muhammad Jafar Sadeq

CS-Based Energy-Efficient Service Allocation in Cloud 49
Sambit Kumar Mishra, Subham Kumar Sahoo, Chinmaya Kumar Swain, Abhishek Guru, Pramod Kumar Sethy, and Bibhudatta Sahoo

Small-Footprint Keyword Spotting in Smart Home IoT Devices 59
A. Mohanty, K. Sahu, R. Parida, G. Pradhan, and S. Chinara

Overcoming an Evasion Attack on a CNN Model in the MIMO-OFDM Wireless Communication Channel 71
Somayeh Komeylian, Christopher Paolini, and Mahasweta Sarkar

An Improved Whale Optimization Algorithm for Optimal Placement of Edge Server 89
Rajalakshmi Shenbaga Moorthy, K. S. Arikumar, and B. Sahaya Beni Prathiba

Performance Analysis of LBT Cat4 Based 5G IoT Enabled New Radio in Unlicensed Spectrum 101
Zubair Shaban, Nishu Gupta, Krishan Kumar, Sandeep Kumar Sarowa, and Mohammad Derawi

Lattice Cryptography-Based Geo-Encrypted Contact Tracing for Infection Detection 111
Mayank Dhiman, Nitin Gupta, Kuldeep Singh Jadon, Ujjawal Gupta, and Yashwant Kumar

Beamforming Technique for Improving Physical Layer Security in an MIMO-OFDM Wireless Channel 127
Somayeh Komeylian, Christopher Paolini, and Mahasweta Sarkar

Predictive VM Consolidation for Latency Sensitive Tasks in Heterogeneous Cloud 135
Chinmaya Kumar Swain, Preeti Routray, Sambit Kumar Mishra, and Abdulelah Alwabel

Reporting Code Coverage at Requirement Phase Using SPIN Model Checker 151
Golla Monika Rani, Akshay Kumar, Sangharatna Godboley, and Ravichandra Sadam

Metric-Oriented Comparison of Selective Forwarding Attack Detection Techniques in IoT-Based Systems 163
Nidhi Sinha and Alekha Kumar Mishra

Performance Enhancement of the Healthcare System Using Google Cloud Platform 175
Subhadarshini Mohanty, Alka Dash, Subasish Mohapatra, Amlan Sahoo, and Subrota Kumar Mondal

Front-End Security Analysis for Cloud-Based Data Backup Application Using Cybersecurity Tools 187
S. MD. K. N. U. Affan Ahamed, Vinay Mathew Wilson, Manu Elappila, and Sachin Malayath Jose

Health Insurance Fraud Detection Using Feature Selection and Ensemble Machine Learning Techniques 197
Anuradha Mohanta and Suvasini Panigrahi

Real-Time American Sign Language Interpretation Using Deep Convolutional Neural Networks 209
Arghya Biswasa, Gaurav Sa, Umakanta Nanda, Diksha Sharma, Lakhan Dev Sharma, and Primatar Kuswiradyo

Multi-branch Multi-scale Attention Network for Facial Expression Recognition (FER) in-the-Wild 221
Chakrapani Ghadai and Dipti Patra

Identifying COVID-19 Pandemic Stages Using Machine Learning 231
Shomoita Jahid Mitin, Muhammad Jafar Sadeq, Umme Habiba, Roy D. Gregori Ayon, Md. Sanaullah Rabbi, and S. Rayhan Kabir

A Multi-feature Analysis of Accented Multisyllabic Malayalam Words—a Low-Resourced Language 243
Rizwana Kallooravi Thandil, K. P. Mohamed Basheer, and V. K. Muneer

Machine Learning Based Fruit Detection System 253
Krishnapriya Ajit and S. Sofana Reka

Post hoc Interpretability: Review on New Frontiers of Interpretable AI 261
Ashly Ann Jo and Ebin Deni Raj

SRGAN with 3D CNN Model for Video Stabilization 277
Sunil Kumar Kumawat and Mantosh Biswas

Finding the Source of a Tweet and Analyzing the Sentiment of the User from h(is)er Tweet History 293
Subhadip Mondal, Bilas Ghosh, Uday Dey, Antara Pal, and Alok Ranjan Pal

Bitcoin Price Prediction by Applying Machine Learning Approaches 305
Debachudamani Prusti, Asis Kumar Tripathy, Rahul Sahu, and Santanu Kumar Rath

Social Engineering Attack Detection Using Machine Learning 321
Kesari Sathvik, Pranav Gupta, Saipranav Syam Sitra, N. Subhashini, and S. Muthulakshmi

Corn Yield Prediction Using Crop Growth and Machine Learning Models 333
Audrey B. Moswa, Patrick Killeen, Iluju Kiringa, and Tet Yeap

Deep Learning-Based Cancelable Biometric Recognition Using MobileNetV3Small Model 347
Shakti Maheta and Manisha

Performance-Based Evaluation for Detection and Classification of Breast Cancer in Mammograms 357
Dakshya Prasad Pati and Sucheta Panda

Predictive Maintenance of NASA Turbofan Engines Using Traditional and Ensemble Machine Learning Techniques 369
Dangeti Saivenkat Ajay, Sneegdh Krishnna, and Kavita Jhajharia

Stacking a Novel Human Emotion Recognition Model Using Facial Features 381
Vikram Singh and Kuldeep Singh

RecommenDiet: A System to Recommend a Dietary Regimen Using Facial Features 397
Dipti Pawade, Jill Shah, Esha Gupta, Jaykumar Panchal, and Ritik Shah

Image Transformation Based Detection of Breast Cancer Using Thermograms 409
Vartika Mishra, Shibashis Sahu, Subhendu Rath, and Santanu Kumar Rath

Vehicle Re-identification Using Convolutional Neural Networks 421
Nirmal Kedkar, Kotla Karthik Reddy, Hritwik Arya, Chinnahalli K Sunil, and Nagamma Patil

Evaluation of Federated Learning Strategies on Industrial Time Series Fault Classification 433
Baratam Prathap Kumar, Sameer Chouksey, Madapu Amarlingam, and S. Ashok

Optimized Algorithms for Quantum Machine Learning Circuits 445
Lavanya Palani, Swati Singh, Balaji Rajendran, B. S. Bindhumadhava, and S. D. Sudarsan

Prediction of SOH and RUL for Lithium-Ion Batteries Using Regression Method with Feature of Indirect Related to SOH (FIRSOH) and Linear Time Series Model 457
Aradhna Patel and Shivam Patel

Chatbot for Mental Health Diagnosis Using NLP and Deep Learning 465
Neel Ghoshal, Vaibhav Bhartia, B. K. Tripathy, and A. Tripathy

SincSquareNet: Deep Neural Network-Based Speaker Identification for Raw Speech 477
Banala Saritha, K. Anish Monsley, Rabul Hussain Laskar, and Madhuchhanda Choudhury

RSSI-Based Hybrid Approach for Range-Free Localization Using SA-PSO Optimization 487
Maheshwari Niranjan and Buddha Singh

Multimodal Paddy Leaf Diseases Detection Using Feature Extraction and Machine Learning Techniques 499
P. Kaviya and B. Selvakumar

Quad Mount Fabricated Deep Fully Connected Neural Network Based Logistic Pricing Prediction 509
M. Shyamala Devi, Penikalapati Sai Akash Chowdary, Muddangula Krishna Sandeep, and Yeluri Praveen

Machine Learning and Deep Learning Models for Vegetable Leaf Image Classification Based on Data Augmentation 521
Chitranjan Kumar and Vipin Kumar

Deep Fake Generation and Detection 533
Shourya Chambial, Rishabh Budhia, Tanisha Pandey, B. K. Tripathy, and A. Tripathy

Similarity-Based Recommendation System Using K-Medoids Clustering 545
Aryan Pathare, Burhanuddin Savliwala, Narendra Shekokar, and Aruna Gawade

Towards a General Black-Box Attack on Tabular Datasets 557
S. Pooja and Gilad Gressel

Multi-task System for Multiple Languages Translation Using Transformers 569
Bhargava Satya Nunna

Analysis of Various Hyperparameters for the Text Classification in Deep Neural Network 579
Ochin Sharma

Analysis and Prediction of Datasets for Deep Learning: A Systematic Review 589
Vaishnavi J. Deshmukh and Asha Ambhaikar

Lung Cancer Classification Using Capsule Network: A Novel Approach to Assist Radiologists in Diagnosis 597
S. NagaMallik Raj, Eali Stephen Neal Joshua, Nakka Thirupathi Rao, and Debnath Bhattacharyya

Author Index 605

About the Editors

Suchismita Chinara is currently working as an Assistant Professor in the Department of Computer Science and Engineering, National Institute of Technology, India. She has received her M.E. degree in 2001 and Ph.D. in 2011 from the Department of Computer Science and Engineering, National Institute of Technology, Rourkela. She has authored and co-authored multiple peer-reviewed scientific papers and presented works at many national and international conferences. Her contributions have acclaimed recognition from honorable subject experts around the world. Her academic career is decorated with several reputed awards and funding. Her research interest includes wireless network, ad hoc network and MANET.

Asis Kumar Tripathy is an Associate Professor in the School of Information Technology and Engineering, Vellore Institute of Technology, Vellore, India. He has more than ten years of teaching experience. He completed his Ph.D. from the National Institute of Technology, Rourkela, India, in 2016. His areas of research interests include wireless sensor networks, cloud computing, Internet of things and advanced network technologies. He has several publications in refereed journals, reputed conferences and book chapters to his credit. He has served as a program committee member in several conferences of repute. He has also been involved in many professional and editorial activities. He is a senior member of IEEE and a member of ACM.

Kuan-Ching Li is currently appointed as Distinguished Professor at Providence University, Taiwan. He is a recipient of awards and funding support from several agencies and high-tech companies, as also received distinguished chair professorships from universities in several countries. He has been actively involved in many major conferences and workshops in program/general/steering conference chairman positions and as a program committee member, and has organized numerous conferences related to high-performance computing and computational science and engineering. Professor Li is the Editor-in-Chief of technical publications Connection Science (Taylor & Francis), *International Journal of Computational Science and Engineering* (Inderscience) and *International Journal of Embedded Systems* (Inderscience), and serves as associate editor, editorial board member and guest editor for

several leading journals. Besides publication of journal and conference papers, he is the co-author/co-editor of several technical professional books published by CRC Press, Springer, McGraw-Hill, and IGI Global. His topics of interest include parallel and distributed computing, Big Data, and emerging technologies. He is a Member of the AAAS, a Senior Member of the IEEE, and a Fellow of the IET.

Jyoti Prakash Sahoo is an experienced Assistant Professor, and a Senior Member of IEEE, currently working at the Department of Computer Science and Information Technology, Institute of Technical Education and Research, Siksha 'O' Anusandhan (Deemed to be University). He previously worked as an Assistant Professor with C. V. Raman College of Engineering, Bhubaneswar (now C. V. Raman Global University). He is an active member of several academic research groups including the Scalable Adaptive Yet Efficient Distributed (SAYED) Systems Group at the Queen Mary University of London, the Intelligent Computing and Networking (ICN) Research Group at East China Normal University, and the Modern Networking Lab at National Taiwan University of Science and Technology. He is having expertise in the field of Edge Computing and Machine learning. He is also serving many journals and conferences as an editorial or reviewer board member. He served as Publicity Chair, Web Chair, Organizing Secretary, and Organizing Member of technical program committees for many national and international conferences. Being a WIPRO Certified Faculty, he has also contributed to industry-academia collaboration, student enablement, and pedagogical learning.

Alekha Kumar Mishra received the Ph.D. degree from NIT Rourkela, Rourkela, India. He is currently a faculty member with the National Institute of Technology at Jamshedpur, Jamshedpur, India. He has been actively involved in many major conferences and workshops in program/general/steering conference chairman positions and as a program committee member. He has published several research papers in various international journals and conferences. His current research interests include Internet of Things, WSN, and security. He is a senior member of IEEE.

NS3-Based Performance Assessment of Routing Protocols AODV, OLSR and DSDV for VANETs



Madhuri Malakar, Bidisha Bhabani, and Judhistir Mahapatro

Abstract A self-organized ad hoc network termed as Vehicular Ad hoc NETwork (VANET) allows each vehicle to take part in routing by sending safety-related and non-safety messages to other vehicles. VANETs have captivated many researchers' focus as an emerging research field as it has several challenges to be addressed. The dynamic nature of mobile nodes, network latency because of link failure, and the often changing topology, add challenges while designing a delay-efficient routing protocol in VANETs. Our study analyzes the network performance of AODV, OLSR, and DSDV routing protocol for an urban VANET scenario using the SUMO and NS3 network simulator for our university campus is presented and analyzed. To evaluate the network performance we use Packet Delivery Ratio (PDR), Packet Loss Ratio (PLR), Average Throughput (AT), Average Goodput (AG), Average End-to-End Delay (AEED), and Average End-to-End Jitter (AEEJ) as routing metrics.

Keywords VANET · Routing protocols · AODV · DSDV · OLSR · Performance analysis

1 Introduction

VANETs can provide inter-vehicle communication with or without the aid of infrastructure to guarantee road safety and avoid potential accidents [1] as shown in Fig. 1. VANETs are groups of vehicles that are outfitted with wireless transceivers, known as OBUs, to exchange safety-related or non-safety messages to other vehicles. In VANETs, every vehicle has the capability to interact using Vehicle-to-Vehicle communications (V2V) or another piece of static infrastructure named as Road Side Unit

M. Malakar (✉) · B. Bhabani · J. Mahapatro
National Institute of Technology, Rourkela, Odisha, India
e-mail: malakar.madhuri2018@gmail.com

J. Mahapatro
e-mail: mahapatroj@nitrkl.ac.in

© The Author(s), under exclusive license to Springer Nature Singapore Pte Ltd. 2023

S. Chinara et al. (eds.), *Advances in Distributed Computing and Machine Learning*,
Lecture Notes in Networks and Systems 660,
https://doi.org/10.1007/978-981-99-1203-2_1

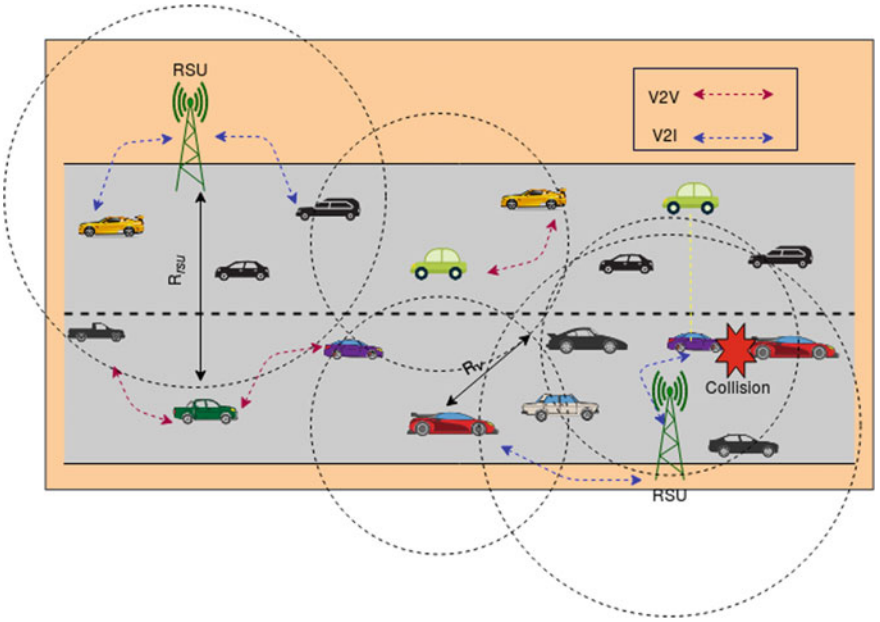


Fig. 1 Architecture of VANET

(RSU) using Vehicle-to-Infrastructure communications (V2I). This architecture provides three types of vehicular communications—V2V, V2I, and hybrid (both V2V and V2I) [2]. The standard for vehicular communication is prescribed by Dedicated Short Range Communication (DSRC) service popularly known as IEEE 802.11p.

Few distinctive features of VANET includes: (i) High dynamic topology—As vehicles move at different speeds, the topology of the VANET rapidly changes. (ii) Frequent network disconnections—Because of the rapid movement of vehicles, VANETs will not always be operational with continuous connectivity. Due to these reasons, finding an efficient routing mechanism for V2V communication is a challenge. For any network, routing defines the dissemination of data following specific and predefined mechanism depending on the network behavior. In case of an ad hoc network, the communication between two vehicle nodes can take place if they are available in each other's radio range and without involving the infrastructure considering the feasibility, availability, and security into account. The primary objective of the routing algorithm is to find a competent route between the transmitting and the recipient vehicle to make a more reliable message delivery. The five available categories of routing protocols [3–7] is presented in Fig. 2. Among which the topology-based routing is divided into: proactive, reactive, and hybrid.

In case of proactive routing, all potential paths are discovered in advance. A vehicle must regularly deliver control messages in order to preserve accurate route information. The key benefit of the proactive method is that each node maintains routes to all potential destination nodes using a routing table. As a result, the path to

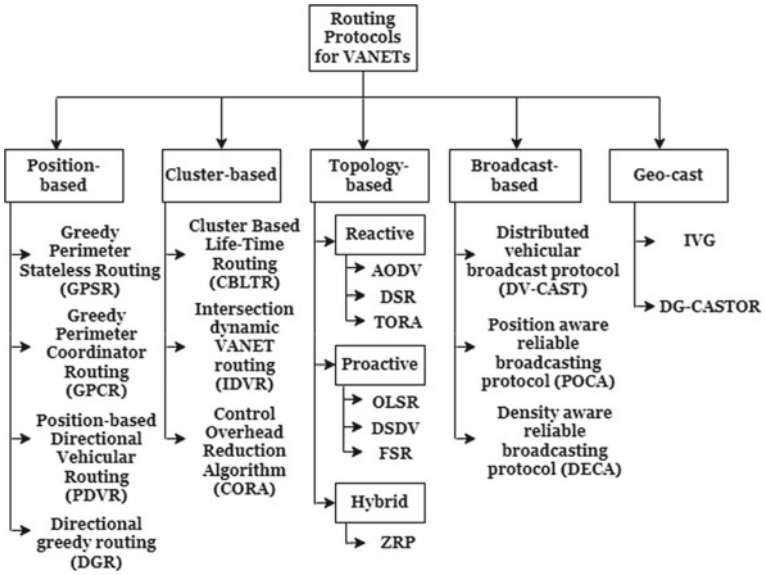


Fig. 2 Classification of routing protocols

destination can be quickly determined. Unfortunately, this results in network overhead and inefficient utilization of bandwidth. Examples of this sort of protocols are Destination-Sequenced Distance-Vector Routing (DSDV) and Optimized Link State Routing (OLSR). On the other hand, reactive routing is also known as on-demand routing. Dynamic Source Routing (DSR) and Ad Hoc On-Demand Distance-Vector (AODV) are examples of reactive routing. Instead of broadcasting all of the vehicles' addresses, it just updates the pertinent nearby vehicle(s).

The hybrid routing protocol separates the network into local and global zones, to lessen the routing overhead and latency brought on by the route discovery process. It does this by fusing together local proactive and global reactive routing methods [8]. We restrict our discussion in this paper to topology-based routing strategies.

The performance of AODV, OLSR, and DSDV in a VANET highway and urban environment is assessed in this paper using NS3 simulation. The remainder of this paper's description is presented and shown below: In Sect. 2, the existing works which are related to our study are briefly outlined. The discussion of routing protocols is covered in Sect. 3. The performance analysis of the utilized routing protocols is shown and discussed in Sect. 4. The work is concluded and future works are discussed in Sect. 5.

2 Related Work

The efficiency of routing methods for VANETs between automatic and manual cars in Madinah city was evaluated by authors in [9] under various traffic conditions. AODV, DYMO, and DSDV are three ad hoc routing protocols that were used in two application scenarios to analyze different traffic distributions and densities. The simulation was based on an extracted map of Madinah city and carried out using SUMO and OMNET++. The output of the simulation parameter average trip time shows that choosing a scenario with fully autonomous cars saves travel time for vehicles by around 7.1% in busy areas. On-demand routing methods also cause the least amount of latency. Thus, authors concluded that automatic vehicles significantly reduce trip time.

Authors in [10] compared AODV, DSDV, DSR, and GPSR by varying the traffic load. Only two simulation metrics have been used for this comparison such as PDR and routing overhead. The simulation outcome shows that AODV has the maximum PDR, whereas, DSR has the least routing overhead as compared to other routing protocols.

Similar kind of comparison is shown in [11] among four routing protocols like AODV, DSDV, OLSR, and DSR considering AT, overhead, transmission power, AEED, PDR, and energy consumption in terms of node mobility and pause time. The outcomes of the simulation are not described properly to understand in which scenario which routing protocol performs better than the others. A comprehensive, in-depth taxonomy of routing protocols in VANETs is proposed in [12], along with the benefits and shortcomings of each category. Additionally, the methods used by routing protocols are defined depending on the location of the vehicles and the network structure. The traffic scenario of Oujda city is simulated using SUMO and NS3 for AODV, OLSR, DSDV, GPSR, and GPCR to study the benefits involved from the perspective of PDR, AT, AEED, and routing overhead.

Authors in [13] used SUMO, and NS-3 to analyze the performances of AODV, DSDV, and DSR in terms of AT by considering two types of packet delivery—TCP and UDP. The simulation outcomes demonstrate that DSDV produces less packet delivery rate. As the network's bandwidth is limited, AODV assists in allowing packets to be sent at a suitable rate. DSR shows the minimum throughput among all.

Using ns-3, the routing protocols OLSR (Proactive) and AODV (Reactive) are compared in [14] under various traffic scenarios. The fundamental premise of the scenario is the intersection of similar geographical topology. They considered two types of mobility densities: high and low. The throughput, PDR, and latency are used for all assessment criteria. The outputs show that for the scenario with less traffic during the course of the simulation, OLSR outperforms AODV in terms of AT, PDR, and AEED by 17.4%, 7%, and 5%, respectively. In dense network, OLSR also outperforms the AODV by 7.9%, 6.5%, and 4%. Additionally, when congestion develops, OLSR is still superior to the other routing mechanisms targeted by this research.

AODV is an enhancement of DSDV but with a completely different insight for route discovery. AODV is on-demand routing protocol. Now, OLSR is an improvement of link state protocol which is also a proactive protocol like DSDV but comparatively a bit advanced one than DSDV. We have taken these three different genres of routing protocols and rigorously analyzed in terms of nearly 7 different performance metrics for comparison.

3 Routing Protocols

For this current study, two primary types of topological routing protocols for VANETs are considered: proactive and reactive. Proactive routing protocols always keep up-to-date routing tables updated by regularly distributing routing information over the entire network. OLSR and DSDV are two different proactive routing protocols that are considered for the performance evaluation. In case of reactive routing protocol, the routing information is not required to be stored in case the nodes need no communication along the way. Path discovery is only required when there's a need of inter-vehicle communication. However, reactive procedures have a large route discovery delay. AODV falls under this kind of routing protocol. Two proactive (OLSR and DSDV) routing protocols and one reactive (AODV) routing protocol are described along with their advantages and disadvantages below.

3.1 AODV

Description It is an improvement of the previously mentioned DSDV algorithm. AODV is an on-demand routing protocol and acts when packet transmission is required. AODV creates routes only when required, thereby reducing the number of broadcasts. The node selection is done only if the node falls into a selected route or else the nodes are not engaged in routing.

The source vehicle starts a path-finding process to find the target vehicle. This path-finding process starts with broadcasting of a route request (RREQ) packet to the available neighbors. These neighbors forward these RREQ packets to its own neighbors and so on until and unless the RREQ packet finds the destination. In response to the RREQ, the target or an intermediary vehicle unicasts a route reply (RREP) packet to the neighbor it initially received from. Vehicles on this path create forward route entries for the vehicle from which the RREP originated in their route tables. The forward route listed in these entries is the one that is currently in use [15].

Advantages Low connection setup latency [16] is a benefit of this protocol, and it has lower overhead than other protocols as the transmitted packets only need to store the information of the destination address. Only the destination IP address and sequence number are transmitted in an AODV RREP. Additionally, because a vehicle

only needs to retain the next hop information, there is reduced memory overhead. The support for multicast that AODV offers is an additional benefit.

Disadvantages The drawback is an increase in control overheads brought on by several route reply messages for a single route request, as noted by the authors in [16]. Additionally, AODV cannot be used where there are asymmetric links since it needs symmetric links between nodes [15].

3.2 OLSR

Description It is a link state routing protocol enhancement. The issue with such protocol is the multiple receptions of the same link state (LS) advertisement, which adds unneeded network overhead. The purpose of OLSR is to prevent the transmission of duplicate LS advertisements.

Because OLSR is proactive, routing tables are updated regularly. Due to the lack of a route discovery procedure, considerable initial latency is not necessary. New routes can quickly determine which offers the best routing efficiency because of constantly updated routing tables. To preserve the routing table information, topology control messages are periodically sent, consuming more network resources. As a result, more bandwidth is used. OLSR is appropriate for large, dense, and highly mobile ad hoc networks. It is even appropriate for applications that are time-sensitive and safety-related [17].

Advantages The benefits of OLSR include optimization over pure link state routing, a decrease of needless LS advertising for retransmission, less initial latency and applications relating to safety, and the ease with which new routes can be found to increase routing efficiency [17].

Disadvantages The following are OLSR's shortcomings: It requires a lot of network resources, creates routing overhead, consumes a lot of network bandwidth, and burdens the network [17].

3.3 DSDV

Description The protocol uses a table-driven algorithm, based on the conventional Bellman-Ford routing algorithm. The elimination of loops in the routing table is one of the enhancements made. Every mobile node in the network keeps track of all potential network destinations and the hop count required to reach them in a routing table. Each item is identified by a sequence number provided by the destination node. The mobile nodes can discriminate between new and old routes thanks to the sequence numbers, preventing the formation of routing loops. The network is routinely updated with routing table modifications to ensure consistency of the tables [15].

Advantages DSDV is one of the early algorithms available. It is a table-driven routing algorithm and appropriate for finite number of vehicles with less velocity. Additionally, the latency for route discovery is low [18].

Disadvantages It requires huge volume of control messages, and a regular update of its routing tables [16].

4 Performance Evaluation

Performance of AODV, OLSR, and DSDV is compared using 7 different performance metrics which are elaborately defined in the following Sect. 4.1. The experiment is simulated jointly using SUMO, NS3, and Open Street MAP (OSM) and a detailed simulation setup is described in Sect. 4.2. The analysis of the outcomes of the simulation is presented in Sect. 4.3.

4.1 Simulation Metrics

The eight different performance metrics and their corresponding equations are presented below. An overview of these comparisons of three routing protocols can be observed in Table 1 for vehicle density as 100.

AT The amount of bits that are delivered successfully to the destination vehicle per unit active network time is termed as AT and is computed using Eq. (1).

$$AT = \frac{\beta_{at}}{\tau_{active}} \times \frac{1}{\vartheta} \tag{1}$$

Table 1 Comparison of routing protocols

Metrics	$AODV$	$OLSR$	$DSDV$
AT (Kbps)	1.86	2.04	1.50
AG (Kbps)	12.73	13.71	10.45
PDR (%)	86	81	51
PLR (%)	13	18	48
AEED (ms)	51	16	30
AEEJ (ms)	36	17	25
Overhead	0.46	0.34	0.46

where, β_{at} is the received number of bits by the receiver vehicle, τ_{active} is the time interval during which network is active, and ϑ is the total number of simulation turn. τ_{active} is computed using Eq. (2).

$$\tau_{active} = \tau_l - \tau_f \tag{2}$$

where τ_l represents the moment when the last bit is received and τ_f represents the moment when the first bit is received.

AG The number of useful bits (excluding retransmitted and overhead bits) transmitted by the source vehicle to the destination vehicle per unit time is referred to as goodput and is computed using Eq. (3).

$$AG = \frac{\beta_{ag}}{\tau_{active}} \times \frac{1}{\vartheta} \tag{3}$$

where, β_{ag} denotes the cumulative received bytes at the receiver vehicle, τ_{active} is the time interval during which network is active, and ϑ is total number of simulation turn.

PDR It is determined by the ratio of the number of data packets successfully received η_r by the destination vehicles by the total number of data packets initiated η_t by the source vehicles and is computed using Eq. (4).

$$PDR = \frac{\eta_r}{\eta_t} \times 100 \tag{4}$$

PLR It is the ratio of the difference between the total packet transmitted by the source vehicle and packet received by the destination vehicle to the total number of packets transmitted. PLR computation is shown using Eq. (5).

$$PLR = \frac{\eta_t - \eta_r}{\eta_t} \times 100 \tag{5}$$

where η_r and η_t is the total number of received and transmitted packets.

AEED It demonstrates the required time needed to transmit the data from source to destination node. It includes queuing delay, propagation delay, transmission delay, MAC retransmission delay, and buffering delay during route identification. AEED is computed using Eq. (6).

$$AEED = \frac{\sum_{i=0}^{n}(\tau_{ri} - \tau_{si})}{n} \tag{6}$$

where τ_{ri} and τ_{si} is the time at which ith packet was received by the receiver vehicle and was sent by the source vehicle. The total number of sent packet is denoted by n.

AEEJ It is a term used to describe the difference in delay between packet flows from source to destination.

$$AEEJ = \frac{\delta(P_n) - \delta(P_{n-1})}{n} \tag{7}$$

$\delta(P_n)$ is the delay occurred for nth packet transmission, $\delta(P_{n-1})$ is the delay occurred for $(n-1)$th packet transmission, and n is the total number of packets sent.

Overhead It is defined as how many extra bits are needed to be transmitted in order to deliver a safety message from the source vehicle to the destination vehicle. It is computed using Eq. (8).

$$Overhead = \frac{\beta_{total} - \beta_m}{\beta_{total}} \tag{8}$$

where β_{total} is the total number of bits transmitted in order to send a safety message which includes control messages, route request, and route reply messages as well. β_m is the number of bits presents in the safety message required to be delivered from source to destination.

4.2 Simulation Setup

The VANET routing protocols were simulated using NS3. In order to predict the road environment and vehicle traffic in the real world, a 3000 m × 1000 m simulation area of our university with two RSUs was derived using Open Street Map (OSM). The mobility model building tool named SUMO and the city road map file are used to create the mobility trace file for the vehicles. Realistic mobility traces are gathered from SUMO and fed as input into NS-3 for simulation. The parameters used for this simulation are outlined in Table 2.

4.3 Result Analysis

The simulation results and analysis of the routing protocols for VANET are presented in this section. The simulation result for PDR is shown in Fig. 3. PDR for DSDV is more as compared to AODV and OLSR with increasing number of vehicle density. Figure 4 presents the simulated results for PLR. The results demonstrate that PLR is more for DSDV whereas, PLR for AODV reduces and PLR for OLSR increases with increasing number of vehicles. Figure 5 presents the simulated results for AT. The results show that the AODV routing protocol shows better AT for less number of vehicles but the AT keeps decreasing with the increasing number of vehicles whereas the AT seems to be quite constant for OLSR and DSDV with the increasing number

Table 2 Simulation metrics

Metrics	Values
Simulation area	3000 m × 1000 m
Number of vehicles	20–100
Number of RSUs	2
Vehicle velocity	0–20 m/s
Packet size	200 bytes
Routing protocol	AODV, DSDV and OLSR
Mobility model	Random Waypoint
Pause time	0 s
MAC	IEEE 802.11p
Loss model	Two-ray ground loss model
Transmit power	20 dBm
Transmission range	145 m
Simulation time	300 s

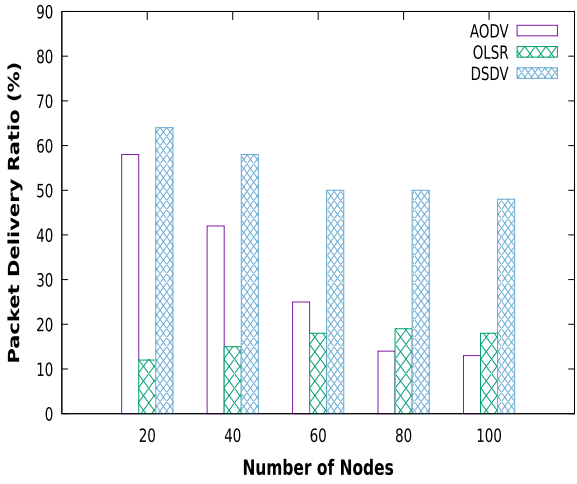


Fig. 3 Impact on PDR with varying vehicle density

of vehicles. Figure 6 presents the simulated results for AG where AODV performs better with the increasing number of vehicles. However, AG is less for DSDV than the other two protocols. Figure 7 presents the simulated results for $AEED$. The results show that AODV routing protocol has an increasing $AEED$ with the increasing number of vehicles.

Figure 8 shows that AODV routing protocol has an increasing $AEEJ$ with the increasing number of vehicles. Figure 9 presents the outputs for the number of vehicles versus MAC/Phy overhead. The results show that DSDV has more overhead

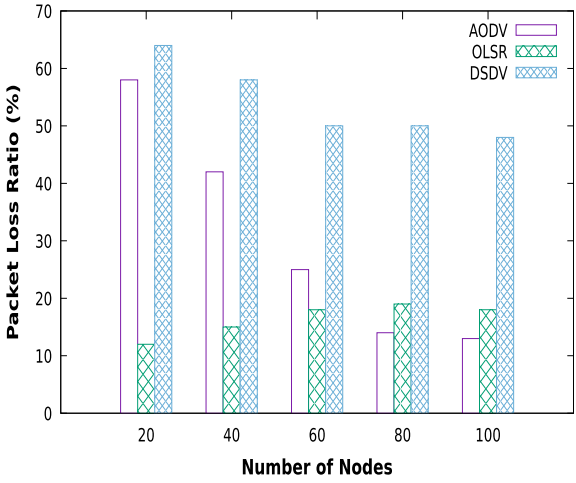


Fig. 4 Impact on PLR with varying vehicle density

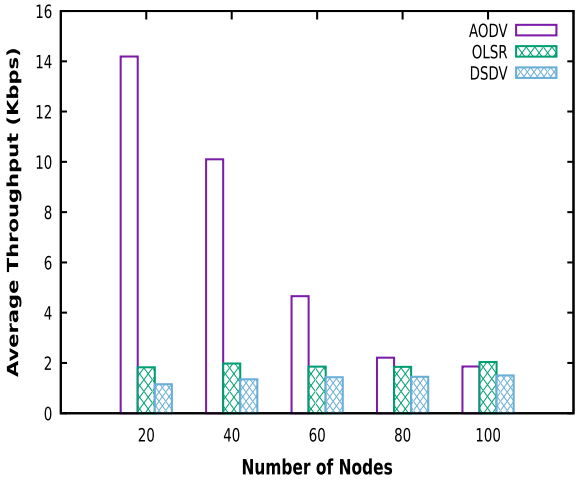


Fig. 5 Impact on AT with varying vehicle density

for sparsely populated vehicles and shows a decrease in overhead for 60 vehicles and then increases slowly with the increasing number of vehicles. AODV shows increasing overhead with the increasing number of vehicles. In this situation, OLSR performs better than AODV and DSDV.

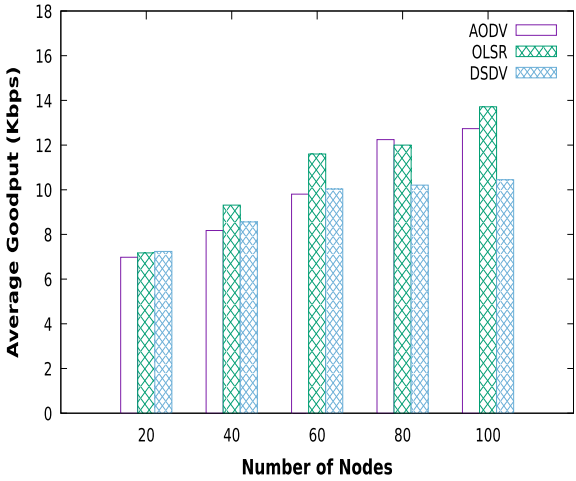


Fig. 6 Impact on AG with varying number of vehicle nodes

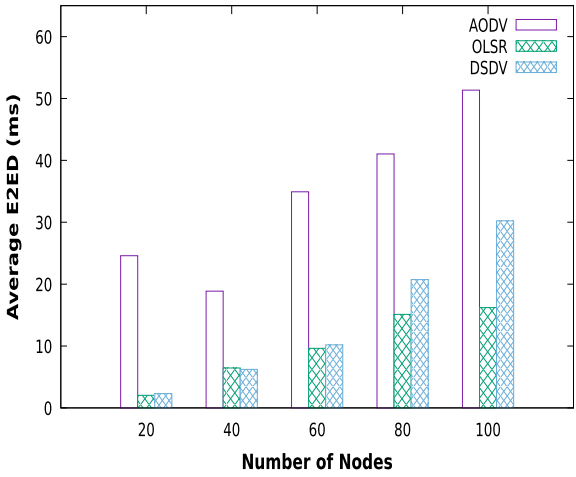


Fig. 7 Impact on AEED with varying vehicle density

5 Conclusion

In this study, the three routing protocols-AODV, OLSR, and DSDV are compared and analyzed in VANET environment simulated using NS3 combined with SUMO and OSM. It is observed that AODV poorly performs with respect to $AEED$ with rise in vehicle density as compared to DSDV and OLSR. AODV may be used for sparse VANETs but does not seem to be good enough for dense network. DSDV has more PLR as compared to OLSR and AODV. The results show that PLR for AODV decreases with the rise in vehicle nodes. In contrast to $AEEJ$, which rises with rise in vehicle nodes, AT for AODV falls as the number of vehicles rises. OLSR

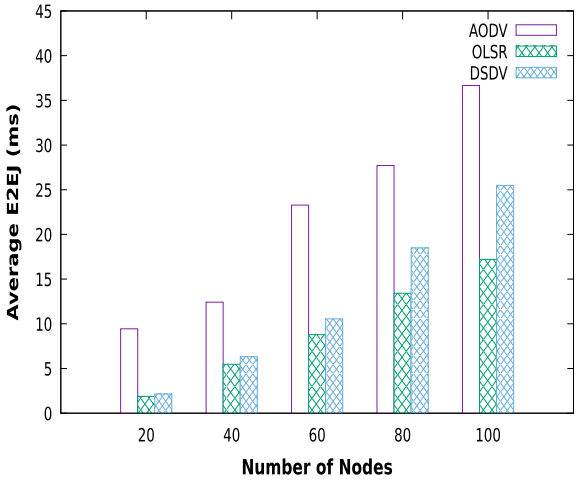


Fig. 8 Impact on AEEJ with varying vehicle density

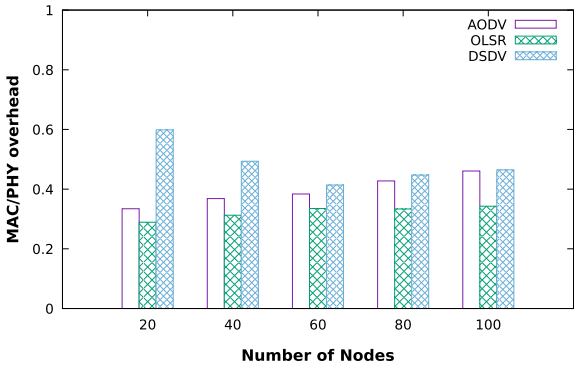


Fig. 9 Number of vehicle nodes versus MAC/Phy overhead

shows better result for *AG* than that of AODV and DSDV. This comparison can be further enhanced by incorporating different mobility models along with the routing protocols and a rigorous analysis can be carried out.

References

1. Bhabani B, Mitra S, Paul A (2018) Efficient and novel broadcasting in VANET using next forwarder selection algorithm. In: 2018 international conference on advances in computing, communications and informatics (ICACCI), pp 273–279
2. Bhabani B, Mahapatro J (2022) RCAPChA: RSU controlled AHP-based prioritised channel allocation protocol for hybrid VANETs. Int J Ad Hoc Ubiquitous Comput (IJAHUC) 40(4):250–266. Inderscience Publication
3. Liu J, Wan J, Wang Q, Deng P, Zhou K, Qiao Y (2016) A survey on position-based routing for vehicular ad hoc networks. Telecommun Syst 62:15–30

4. Abuashour A, Kadoch M (2017) Performance improvement of cluster-based routing protocol in VANET. IEEE Access 5:15354–15371
5. Smiri S, Boushaba A, Ben Abbou R, Zahi A (2018) Geographic and topology based routing protocols in vehicular ad-hoc networks: performance evaluation and QoS analysis. In: 2018 international conference on intelligent systems and computer vision (ISCV), pp 1–8
6. Singh S, Agrawal S (2014) VANET routing protocols: issues and challenges. In: 2014 recent advances in engineering and computational sciences (RAECS), pp 1–5
7. Ghori MR, Sadiq AS, Ghani A (2018) VANET routing protocols: review, implementation and analysis. In: Journal of physics: conference series, vol 1049, no 1, p 012064. IOP Publishing
8. Salman O, Morcel R, Al Zoubi O, Elhajj I, Kayssi A, Chehab A (2016) Analysis of topology based routing protocols for VANETs in different environments. In: 2016 IEEE international multidisciplinary conference on engineering technology (IMCET), pp 27–31
9. Abdeen MAR, Beg A, Mostafa SM, AbdulGhaffar AA, Sheltami TR, Yasar A (2022) Performance evaluation of VANET routing protocols in Madinah city. Electronics 11(5), article 777
10. Liu Y (2021) VANET routing protocol simulation research based on NS-3 and SUMO. In: 2021 IEEE 4th international conference on electronics technology (ICET), pp 1073–1076
11. Rajhi M, Madkhali H, Daghriri I (2021) Comparison and analysis performance in topology-based routing protocols in vehicular ad-hoc network (VANET). In: IEEE 11th annual computing and communication workshop and conference (CCWC), pp 1139–1146
12. Bengag A, Bengag A, Elboukhari M (2020) Routing protocols for VANETs: a taxonomy, evaluation and analysis. Adv Sci, Technol Eng Syst J 5(1):77–85
13. Kang SS, Chae YE, Yeon S (2017) VANET routing algorithm performance comparison using ns-3 and SUMO. In: 2017 4th international conference on computer applications and information processing technology (CAIPT), pp 1–5
14. Basil AK, Ismail M, Altahrawi MA, Mahdi H, Ramli N (2017) Performance of AODV and OLSR routing protocols in VANET under various traffic scenarios. In: 2017 IEEE 13th Malaysia international conference on communications (MICC), pp 107–112
15. Royer EM, Toh C-K (1999) A review of current routing protocols for ad hoc mobile wireless networks. In: IEEE personal communications, vol 6, no 2, pp 46–55
16. Manickam P, Baskar TG, Girija M, Manimegalai D (2011) Performance comparisons of routing protocols in mobile ad hoc networks. Int J Wirel Mob Netw (IJWMN) 3(1):98–106
17. Gupta A, Singh R, Ather D, Shukla RS (2016) Comparison of various routing algorithms for VANETS. In: 2016 international conference system modeling & advancement in research trends (SMART), pp 153–157
18. AL-Dhief FT, Sabri N, Salim MS, Fouad S, Aljunid SA (2018) MANET routing protocols evaluation: AODV, DSR and DSDV perspective. In: Malaysia technical universities conference on engineering and technology (MUCET 2017), MATEC web of conferences, vol 150, article id 06024

A Novel Blockchain-Based Smart Contract for Real Estate Management



Ashish Kumar Mohanty and D. Chandrasekhar Rao

Abstract Blockchain is trending in recent days as it is gaining ground in various technical fields. Smart contracts are the most reliable and secure technology to use when implementing a blockchain-based application. It can be used to maintain an asset record in a blockchain context and can automatically change ownership and attributes when a specific instance or condition is achieved. When it comes to buying or selling properties, real estate is one of the industries where most transactions take place on daily basis. When it comes to tracking real estate assets, blockchain technology seems to be more effective. Transparent ledgers simplify the process of managing inflows and outflows of real estate transactions. Therefore, the application of smart contracts in the real estate industry has the potential to bring about a radical transformation by making it more secure, dependable, and impenetrable. This study aims to develop a theoretical framework for deploying a novel blockchain-based smart contract for hassle-free real estate transactions. By observing this model, a person with little technical understanding can grasp the mechanism and operation of smart contracts in real estate management.

Keywords Blockchain · Smart contract · Web 3.0 · Security · Real estate management

1 Introduction

Due to the significant revenue brought in by real estate intermediary services, real estate-related activities have long been well liked. The power of land and real

A. K. Mohanty
Department of Computer Science & Engineering, CAPGS, Biju Patnaik University of Technology, Rourkela, Odisha 769015, India

D. Chandrasekhar Rao (✉)
Department of Information Technology, VSS University of Technology, Burla, Odisha 768018, India
e-mail: dcrao_it@vssut.ac.in

© The Author(s), under exclusive license to Springer Nature Singapore Pte Ltd. 2023

S. Chinara et al. (eds.), *Advances in Distributed Computing and Machine Learning*, Lecture Notes in Networks and Systems 660,
https://doi.org/10.1007/978-981-99-1203-2_2

estate property is traditionally registered and transferred by dependable third parties. However, the majority of documents are not digitized, are challenging to locate, are much more difficult to update, and occasionally go lost over time are the major concerns in this model. To make it robust, it is required to embed cutting-edge digital technologies. In this connection integrating blockchain technology with real estate management will be beneficial as we are merely transitioning from Web 2.0 to Web 3.0 [1]. This will make legal agreements/bonds/contracts simpler, more trustworthy, secure, free from brokerage, impenetrable, and more informative.

Buying a house or a plot of land is a very drawn-out and time-consuming procedure for both the buyer and the seller. They must do numerous visits to the registrar's office to check their paperwork, make payments, and pay all applicable taxes and fees. After then, there are still several gaps that work to criminals' advantage. Because authentic data records are unavailable to property buyers and the system is opaque, there are an increasing number of fraud cases. The process is complicated and chaotic due to the traditional method of working with multiple nondigitized paper works.

The term "blockchain" was coined more than a decade ago and gained popularity with the introduction of bitcoin in 2009. "Blockchain technology" is a decentralized, distributed ledger that keeps track of a digital asset's provenance in a collection of interconnected blocks that resembles a chain. It is a continuously expanding ledger that maintains an immutable, secure, and chronological record of every transaction that has ever occurred in a decentralized distributed network. It is a ground-breaking technology since it significantly reduces risk factors, middleman intervention, and transparency for a wide range of applications. It is a hybrid technology that combines distributed database systems and cryptography that satisfies the following criteria: verifiable, unchangeable, tamper-proof, and immutable [2, 3]. A smart contract is a piece of code (a contract) that is kept within the blockchain network and can execute by itself if certain conditions are satisfied [4].

Using blockchain-based smart contracts, this study aims to simplify and streamline real estate management so it can be understood even by someone with little technical knowledge. An elaborate theoretical model is presented here and discussed with the help of flow charts and sequence diagrams, as well as pseudo-code. In addition, we discuss how to deploy and run smart contracts in a blockchain environment for real estate management.

The contributions of this study are as follows:

- Proposed a novel sequential smart contract framework for successful real estate transactions on the blockchain.
- Developed a concise and straightforward methodology for designing and operation of smart contacts in the purchase and selling belongs to real estate.

The remaining of this paper is structured as follows. Section 2 archives the survey of the literature. The basics of blockchain and the deployment of the smart contract are presented in Sect. 3. The working of the proposed model is described in Sect. 4 and the performance of the model and its limitation is discussed in Sect. 5. Finally, the study is concluded with future scope in Sect. 6.

2 Literature Survey

The "T4 Labs Inc 2019" forecasted the growth of Global Real Estate Market embedding blockchain technology from the period 2016 to 2025. Figure 1 shows the market value (in trillion dollars) from 2016 and its future trend up to 2025.

Nick Szabo [6] identified the use of decentralized ledgers with smart contracts in 1994. These contracts might be written in code that could be replicated, kept, and monitored by the computer network that makes up the blockchain. Afterward, numerous researchers focused on transaction management with improving the efficacy and regulation of these projects. As the population is increasing rapidly, consequently the demand for residential and commercial property also. Considering this, real estate is one of the most profitable industries in the world, due to that various studies and advances have been undertaken using blockchain-based smart contracts for real estate management.

Muneeb et al. [7] proposed a concept for a Blockchain-Based Architecture of Smart Contracts for Transaction Management. Several smart contract framework components and their execution have been discussed in depth in this study using appropriate use cases. They proposed a model for a blockchain-based smart contracts and transaction management system in this study, and they compared existing smart contract management systems, which can be vital in selecting a specific system for a certain business requirement. An impactful full approach to managing transactions on blockchain has been provided by the team of authors. It can be extended by considering other applications. A theoretical framework for managing real estate in smart cities employing this technology is proposed in [1]. This study concentrated on the potential application of blockchain smart contracts toward real estate management

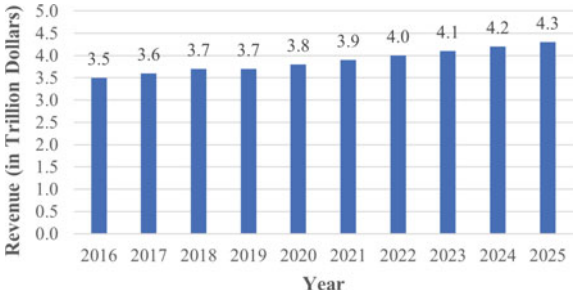


Fig. 1 Global real estate market revenue from 2016 to 2025 employing blockchain [5]

for smart cities. Overall, this report describes how the adoption of blockchain-based smart contracts can assist with real estate management in smart cities. The use of blockchain to prevent fraudulent activities in real estate purchases and sales has been suggested by Bhanushali et al. [8]. In 2020, the authors proposed a model to safeguard property-related transactions in the blockchain ecosystem. This study is based on how blockchain can be used to combat fraudulent practices in the purchasing and selling of property investments. In order to facilitate the buying and selling of real estate, the author deploys a contract, but does not provide a simplified explanation of the process.

The preceding papers, along with a few others, served as the basis for this research. Additionally, we took into account the statistics and survey results offered by a few reliable websites. It is feasible to employ this technology to prevent all sorts of fraud, move all transactions online, enhance transparency, remove any possibility of property rights loss, and eliminate middleman intervention. By using this technique, it can be shown that the established blockchain model is feasible and meets the needs of potential customers. Real estate assets can be tokenized and transacted automatically using smart contracts on the blockchain network. Right now, numerous regional and national policies are distinct from one another. As a result, we need a platform that can operate autonomously, following the stated set of policies. Research, development, and related activity toward Web 3.0 are increasing rapidly. This mixed technology will nurture the much-desired transparency and trust in the system by eradicating security concerns [9, 10].

With respect to the earlier research, the information is concise and there isn't a thorough design and mechanism review so the design and operation of smart contracts used in real estate management will be examined in this research. A systematic review of the literature was undertaken to emphasize a simple yet effective framework of blockchain smart contracts applied to real estate management. By addressing this aspect and keeping it straightforward enough for anyone with adequate technical skills to comprehend, we suggested this theoretical model. The proposed study improves the understanding of tangible resources by integrating physical and digital data in one location using blockchain technology.

3 Blockchain-Based Smart Contract

Smart contracts can be used with a blockchain that records the significant data of all users and transactions in a distributed network [11]. As a result, a decentralized chain is generated and users no longer need to rely on a centralized system for their trades and transactions. Therefore, two buyers or sellers can transact directly between themselves without the intervention of intermediaries [5, 12]. Processes for deploying smart contracts fall into two categories. Regardless of whether the deployment is one-to-one or one-to-many, the approach is essentially the same. Smart contracts do not require a centralized sign-up method that is governed through one centralized server or entity. With this approach, anyone can sign up for several decentralized

applications using a single-user wallet that only its owner can access. The basic methods for deploying a smart contract are listed below:

- It starts with gathering needs and specifications.
- Afterward, the facts and information are validated in accordance with the user's declaration.
- To write the code and implement the smart contract, an appropriate platform, such as Ethereum, needs to be chosen.
- Keys for the deployer and executor must be generated.
- For both the deployer and the executor, private and public keys must be generated.
- To transact in digital currency, a wallet must be created. (Many wallets are accessible; there is no need to develop; simply opening an account on their platform is sufficient.)
- On the chosen platform, write the necessary code in accordance with the requirements.
- Perform the necessary testing.
- Need to sign the smart contract using the deployer's private key.
- The contract then needs to be deployed in the blockchain environment after one final round of verification and validation.

The input data and execution stages of a smart contract must be particular with specific rules and constraints. In other terms, if "A" occurs, then step "B" is executed. As a result, the actual operations performed by smart contracts are quite primitive, such as autonomously shifting ownership from one party to another when specific criteria are fulfilled.

4 Proposed Model

Since blockchain is a relatively new and complicated technology, it might be challenging for the everyday user to comprehend how it works. We are all aware that blockchain will soon disrupt the global economy. We proposed this study to demonstrate a straightforward model for how a smart contract functions in a blockchain framework by considering these sorts of considerations into account.

The upper level steps of a smart contract's operation in real estate management, including the purchasing and selling mechanism, are as follows:

- The information regarding the asset must initially be submitted by the seller.
- The asset data will then be validated and disseminated in the blockchain environment in the form of smart contracts.
- Every time a potential buyer expresses interest, the system will pair that buyer with particular sellers.
- After selecting the property of their choice, the buyer must agree to the terms and conditions and sign them.

- To confirm the seller's agreement to sell, an acceptance request will then be sent to him or her.
- The smart contract will check whether the buyer has the necessary amount of gas or digital currency once the seller has accepted the proposal.
- Then the system will initiate a payment request for the buyer.
- The ownership will automatically switch from the seller to the buyer as soon as the payment has been confirmed and verified.

The concept behind smart contracts is not that complex. They operate based on straightforward logic with some predefined conditions. Figure 2 depicts the sequential operation of smart contracts in the blockchain ecosystem discussed above from deployment to execution.

Initially, the seller (S) and buyer (B) need to register themselves with the required information. There can be m number of buyers and n number of sellers. The buyer needs to specify the requirements of his own and the seller needs to provide the details about the assets intended to sell. Both buyer and seller need to be authorized initially to carry out the transaction(s). Once the authorization process is successful the sellers' declared assets are validated by employing the smart contract. It matches the requirement of the buyer with specifications provided by the seller. Once there is a match, the buyer needs to accept the term conditions then in turn a proposal acceptance request will be sent to the seller. The seller needs to accept the proposal to initiate the payment request by smart contract to the buyer. As soon as the buyer approves the payment request, the deployed smart contract will validate the payment.

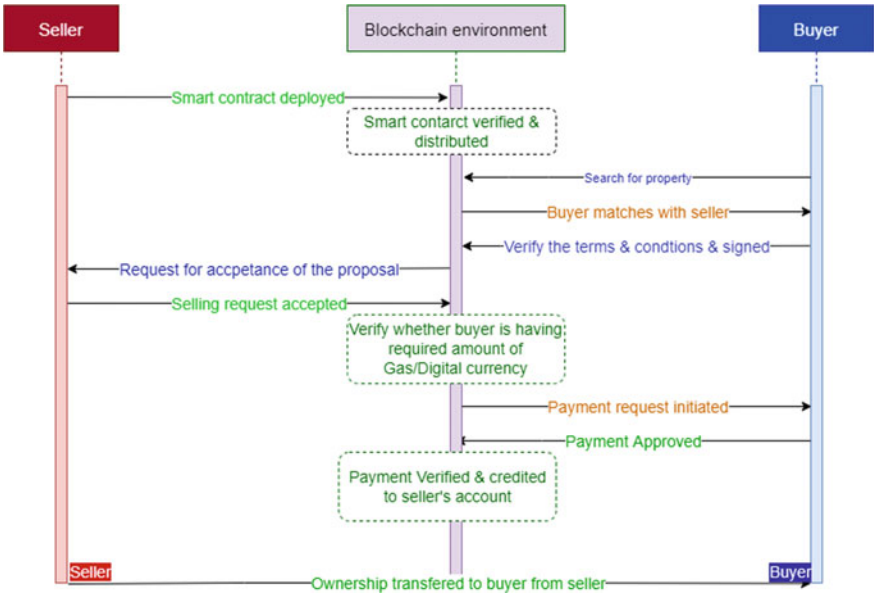


Fig. 2 Simple sequence diagram for buying and selling activities using smart contract

After validation of payment receipt, the ownership transfer between buyer and seller is executed instantly through the smart contract. The complete transaction process is presented in Algorithm 1.

The flow diagram for the smart contract-based buying and selling operation in a blockchain environment is shown in Fig. 3.

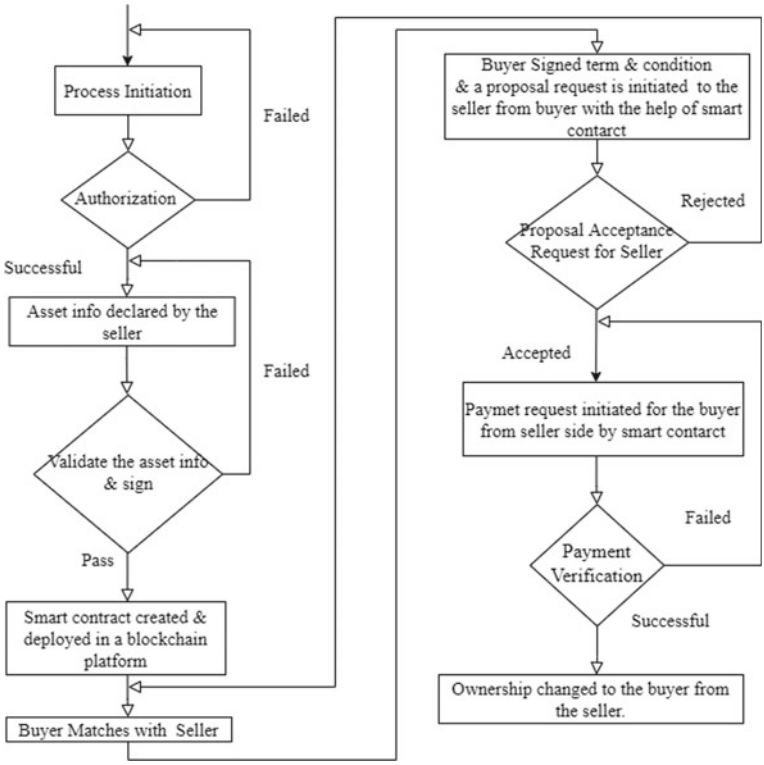


Fig. 3 Flow diagram for smart contract-based buying and selling process

Algorithm 1: Blockchain based Smart contract for Buying / selling of Assets

Input: S← Seller, B ← Buyer, A ← Asset
Output: ownership transfer with verification of transaction

```
1  :  Begin
2  :      Procedure Authorization ();
3  :          If Login successful then
4  :              S will declare the information about A
5  :                  For every asset i
6  :                      Procedure Asset_Validation ();
7  :                          If validation successful then
8  :                              Deploy the smart contract
9  :                          Else
10 :                              Redirect to Asset declaration
                                  page
11 :                          End If
12 :                      End Procedure
13 :                  End For
14 :              B pair with a specific S
15 :              B accepts the T&C
16 :              Proposal acceptance request generated for S
17 :              Procedure Proposal_Acceptance_Request();
18 :                  If approved then
19 :                      Initiate payment request for B
20 :                  Else
21 :                      Redirect to B and S matching page
22 :                  End If
23 :              End Procedure
24 :              Procedure Payment_Verification ();
25 :                  If verification successful then
26 :                      Ownership transfers from B to S
27 :                  Else
28 :                      Redirect to payment request
29 :                  End If
30 :              End Procedure
31 :              Else
32 :                  Redirect to login page
33 :               End If
34 :          End Procedure
35 :  End
```

The pseudo-code for the proposed model is as follows

The pseudo-code below is part of the aforementioned flow diagram. Here, block 2: asset validation phase, block 3: smart contract deployment phase, block 4: buyer seller matching and proposal acceptance phase, block 5: payment verification phase, and block 6: shifting ownership phase.

The pseudo-code for the proposed model

```
func proposal_acceptance_request_seller():
    if approval:
    return True
    else
    return False
    func validation(sign,asset):
    sign1=database(sign)
    asset1=database(asset)
    if sign==sign1 and asset==asset1:
    return True
    else:
    return False
    Func authorization(username,password){
    username1=database(user)
    password1=database(password)
    if username==user1 and password==pwd1:
    return True
    else:
    return False
    }
    Func block2 (){
    Asset info declared by the seller
    if validation(sign=sign, asset=assets):
    block3()
    else:
    block2()
    }
    Func block3(){
    Smart_contract_created_and_deployed
    block4();
    }
    Func block4(){
    Buyer_matches_with_seller
    Buyer_signed_terms_and_condition
    if proposal_acceptance_request_for_seller(approval):
    block5()
    else
    block4()
    }
    Func block5(){
    username=request.session['user1']
    email=request.session['email1']
    Payment request initialized
    if payment_verification sucessed
    Block6()
    else:
    Block5()
    }
    Func block6(){
    Ownership changed to the buyer from the seller
    }
    Void main(){
    authorization = Authorization (username = username, password = password)
    username1=request.session(username)
    email=user.email
    email1=request.session(email)
    if authorization:
    Block2()
    else:
    redirect to login
    }
```

5 Discussion

The World Wide Web has gone from Web 1.0, which was its infancy, to the current level of Web 2.0, which is gradually heading toward the more sophisticated version or forthcoming version of its designated Web 3.0. The global tech sector will be significantly impacted by Web 3.0 [13]. The idea of establishing a completely decentralized and distributed ecosystem is the basis of Web generation 3 or Web 3.0. Also, to establish a decentralized yet secure network where users will safely exchange currency and data without the need for intermediaries or other middlemen's interference.

The real estate sector requires cutting-edge decentralized technology for a safe, transparent, and brokerage-free system since we are just in the beginning phases of the transition from Web 2.0 to Web 3.0. And since blockchain-based smart contracts are completely decentralized and are also preferred for distributed networks, they will

play a significant role in the management of real estate transactions and data for Web 3.0. The smart contracts implementation will play a key role in the evolution of the Web 3.0 economy due to the absence of a central system and distributed potential for users from around the globe. Since the real estate market is one of the most profitable sectors across the globe, employing smart contracts in Web 3.0 will lead to greater financial advantages than using traditional methods. As smart contracts are computer programs that run automatically when certain conditions are achieved, many Web 3.0 apps will rely on blockchain technology's capacity to store and execute them to function. The implementation of smart contracts in Web 3.0 will encourage greater acceptance of this technology because of its trustworthiness.

The benefits of this methodology over the traditional method are as follows:

- No middle man interference or brokerage in real estate deals.
- Time-saving and free of paperwork, which is adding environmental advantages.
- Efficient and secure ownership change with NFT.
- Available of verified data.
- IoT data demand for facilitating rental smart contracts.
- Secure payment and registration
- Relief from a single point of failure like centralized systems.
- Transparency on asset details and payment of fees.

6 Conclusion

The present study focused on a straightforward study of blockchain smart contract applications for real estate management. Smart contracts have only recently begun from a technology standpoint, yet they will gradually play a significant role in real estate management. Information gathering, keeping, processing, and data misuse are the key factors that contribute to the wrong state of land administration systems. Smart Contracts have been used to make possible everything that will take place in Web 3.0. Every entity, whether it is an NFT collection, a cryptocurrency, a land settlement, etc., has a specific smart contract that powers it. Before we can start to consider what Web 3.0 could offer, many things need to be examined and validated. However, there is no doubt that smart contracts will significantly boost real estate management. Blockchain technology's smart contract is currently the most dependable choice for a distributed system. It is worth remembering that as technology advances, smart contracts are likewise getting better every day and will keep the crown of being the most trustworthy and reliable partner for the future.

Future scope:

- A buyer's available digital currency may be shown in the verified stage with auto refresh functionality, eliminating the need to wait for a balance check throughout the transaction.
- To improve security and tackle single-point failure hazards, the PBFT algorithm can be employed.

- For real estate management systems, predictive models can be created employing artificial intelligence.
- With current developing technology, if it can be low code or no code so a person doesn't need to be a technology expert to perform these transactions.

References

1. Ullah F, Al-Turjman F (2021) A conceptual framework for blockchain smart contract adoption to manage real estate deals in smart cities. Neural Comput Appl 1–22
2. Zheng Z, Xie S, Dai H, Chen X, Wang H (2017) An overview of Blockchain technology: architecture, consensus, and future trends. IEEE Int Congr Big Data (BigData Congr) 2017:557–564. https://doi.org/10.1109/BigDataCongress.2017.85
3. Dhaiouir S, Assar S (2020) A systematic literature review of blockchain-enabled smart contracts: platforms, languages, consensus, applications and choice criteria. In: International conference on research challenges in information science. Springer, Cham, pp 249–266
4. Suvitha M, Subha R (2021) A survey on smart contract platforms and features. In: 2021 7th international conference on advanced computing and communication systems (ICACCS), pp 1536–1539. https://doi.org/10.1109/ICACCS51430.2021.9441970
5. Global Real Estate Market Revenue, 2016–2025, Real Estate Market Share (2020). Accessed 30th Aug 2022. https://www.t4.ai/industry/real-estate-market-share
6. Gans JS (2019) The fine print in smart contracts (No. w25443). National Bureau of Economic Research
7. Muneeb M, Raza Z, Haq IU, Shafiq O (2022) SmartCon: a Blockchain-based framework for smart contracts and transaction management. IEEE Access 10:23687–23699. https://doi.org/10.1109/ACCESS.2021.3135562
8. Bhanushali D, Koul A, Sharma S, Shaikh B (2020) BlockChain to prevent fraudulent activities: buying and selling property using Blockchain. In: 2020 international conference on inventive computation technologies (ICICT), pp 705–709.https://doi.org/10.1109/ICICT48043.2020.9112478
9. Laarabi M, Chegri B, Mohammadia AM, Lafriouni K (2022) Smart contracts applications in real estate: a systematic mapping study. In: 2022 2nd international conference on innovative research in applied science, engineering and technology (IRASET), pp 1–8. https://doi.org/10.1109/IRASET52964.2022.9737796
10. Latifi S, Zhang Y, Cheng LC (2019) Blockchain-based real estate market: one method for applying Blockchain technology in commercial real estate market. In: 2019 IEEE international conference on Blockchain (Blockchain). IEEE, pp 528–535
11. Stefanović M, Pržulj Đ, Ristić S, Stefanović D, Nikolić D (2022) Smart contract application for managing land administration system transactions. IEEE Access 10:39154–39176. https://doi.org/10.1109/ACCESS.2022.3164444
12. Khoa Tan V, Nguyen T (2022) The real estate transaction trace system model based on ethereum blockchain platform. In: 2022 14th international conference on computer and automation engineering (ICCAE), pp 173–177. https://doi.org/10.1109/ICCAE55086.2022.9762429
13. Alabdulwahhab FA (2018) Web 3.0: the decentralized web Blockchain networks and protocol innovation. In: 2018 1st international conference on computer applications & information security (ICCAIS), pp 1–4. https://doi.org/10.1109/CAIS.2018.8441990

A Review on VM Placement Scheme Using Optimization Algorithms



Akanksha Tandon, Sudhanshu Kulshresha, and Sanjeev Patel

Abstract In response to the growing popularity of cloud computing, researchers and engineers are striving to improve the efficiency and benefits of cloud computing. Cloud computing is practical when cloud infrastructure is employed efficiently and affordably, allowing organizations of all sizes to become stable. Cloud computing enables users to supply resources on demand and run programs in a way that meets their needs by selecting virtual resources that meet the resource requirements of their application. The duty of accommodating these virtual resources on physical resources falls to cloud resource providers at that point to translate virtual resources into physical resources. Though we consider the providers' optimization goals and resource providers to find some fundamental difficulties in cloud computing. In this paper, we have presented different VM placement techniques based on single-objective and multi-objective optimization algorithms.

Keywords Virtual machine placement · Single-objective optimization · Multi-objective optimization · Nature-inspired algorithm

1 Introduction

Computing technology describes the method of using the Internet to access computing resources. It makes use of a variety of hardware and software platforms to deliver cloud computing services. Cloud service provider (CSP) classifies cloud computing

A. Tandon (✉) · S. Patel
The Department of Computer Science and Engineering, National Institute of Technology Rourkela, Rourkela 769008, Odisha, India
e-mail: akankshatandon97@gmail.com

S. Patel
e-mail: patels@nitrkl.ac.in

S. Kulshresha
Content Quality, Udacity, Inc., New Delhi, India
e-mail: sudhanshu.kulshrestha@udacity.com

© The Author(s), under exclusive license to Springer Nature Singapore Pte Ltd. 2023

S. Chinara et al. (eds.), *Advances in Distributed Computing and Machine Learning*, Lecture Notes in Networks and Systems 660,
https://doi.org/10.1007/978-981-99-1203-2_3

services into three categories based on the customer's business needs, namely, infrastructure as a service (IaaS), software as a service (SaaS), and Platforms as a Service (PaaS). CSPs involve other platforms like Google, deploying data centers in different geographical locations. A cloud data center has a variety of computers or servers with varying speeds, hardware configurations, and capacities. The diversity of servers dramatically influences the performance of cloud applications. It will virtualize them so that several users may use them simultaneously. Physical resources from one or more servers are spread among many executable computers called virtual machines (VMs) for this purpose. Each VM is generally isolated from other VM which is a fully functional environment to perform any services. Cloud computing must be well organized and cost-effective for both cloud-service providers and their clients to be successful. Load balancing is a strategy for dealing with redundancy. A VM encapsulates a physical machine (PM) using processor speed, memory ability, and disk size. Some methods for resolving the VM allocation issue, on a host, selection of the most appropriate host for the VM is known as virtual machine placement (VMP) or simply placement. Estimating the data center's efficiency and efficacy is crucial in the cloud. The constant push to increase data center efficiency and effectiveness has resulted in a disregard for energy usage. Power consumed by data centers was predicted to be 198 Tera-WattHours in 2018. The US Department of Energy accounts for around 1% of world electricity demand. According to Meisner et al. [1], the power demand is expected to quadruple by 2022, when servers are sleeping. There is a considerable reduction in power usage. According to their findings, HP blade server consumes 450 W of electricity while active, 270 W when idle, and 10.4 W in sleep mode. The cost of running data centers has risen with their energy usage. Furthermore, higher energy consumption leads to a rise. At the same time, the idea of energy consumption looks to be increased. Because data centers are diverse, some physical devices are more power efficient than others. To make the most of the data center's resources, VMs are effectively scheduled on more powerful physical machines. In contrast, less powerful and less used physical machines carried out and turned off energy efficiency.

In the following order, our paper has been presented. VMP concepts are addressed in Sect. 2. In the third section, the single-objective optimization algorithms are discussed in the context of VMP mechanism in detail. In the fourth section, multiple-objective optimization algorithms are presented. The conclusion of this research is presented in the last section.

2 Virtual Machine Placement

VMP is the procedure of placing a collection of virtual machines on a group of hosts. A decision-making mechanism is necessary for establishing a one-to-one mapping between the VM and the host. It is possible that the placement of virtual servers is perceived as a scheduling problem. Abbreviation and Descriptions used in our paper are shown in Table 1.

Table 1 Abbreviation used

Abbreviation	Description
ACO	Ant Colony Optimization
BBO	Biological-Based Optimization
CSO	Cuckoo Search Optimization
CSP	Cloud Service Provider
GA	Genetic Algorithm
PM	Physical Machine
VM	Virtual Machine
VMP	Virtual Machine Placement
VMcP	Virtual Machine consolidated Placement
VMPACS	Virtual Machine Placement Ant Colony System
MGGA	Multi-objective Grouping Genetic Algorithm

Kulshrestha et al. [2] have proposed that allocating a collection of m VMs V = V1, V2, . . . , Vm to a set of n computing hosts H = H1, H2, . . . , Hn. There may also be differences in the maximum number of instructions per second (MIPS) and the processing capability of MIPS. If something goes wrong, the values of variables m and n in cloud DC are time dependent. If t represents time, a collection of virtual machines and the accessible host should represent as m(t) and n(t). Scheduling aims to maximize more than one predetermined metrics like processing time, latency, flow time, and overall weight. As a result, the scheduling problem may appear multidimensional, which is NP-hard.

Elbeltagi et al. [3] have compared different evolutionary and nature-inspired algorithms. Five evolutionary algorithms for continuous and discrete optimization were compared. The ant colony, genetic, and particle swarm are few examples of this type of algorithms. Parpinelli et al. [4] have compared ten algorithms based on their sources of inspiration, exploitation methods, investigation, and communication methods which have been included in the reviews which are newly suggested swarm intelligence algorithms. Various search methods and novel meta-heuristic algorithm challenges are inspired by nature. While comparing the behavior, goal function, characteristics, and region of application of the bat, cuckoo search, firefly, and krill herd are carried out by Parpinelli et al. [4] that discusses detailed study of 12 nature-inspired algorithms based on input parameters, evolutionary processes, and relevant application areas.

The major Genetic Algorithm (GA) parameters are the population size, number of generations, crossover probability, mutation probability, and selection operator. The usual control factors are the population size and the number of generations. The control parameters of an algorithm are the crossover probability, mutation probability, and selection operator. Chromosomal length and chromosome encoding techniques

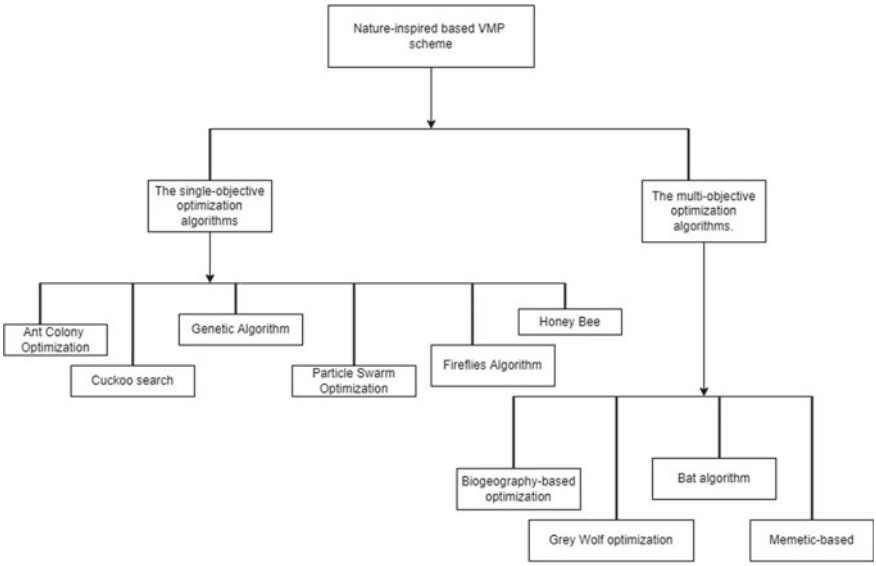


Fig. 1 Classification of the VMP system influenced by biology

are also considered for algorithm-specific characteristics. Because of crossover and mutation, GA has the capacity for simultaneous exploration and exploitation.

The VMP is a type of optimization problem. Optimization is a suitable strategy and algorithm for dealing with and solving this problem. These algorithms come in a variety of forms. The optimization models may be categorized into many significant groups. There are two types of optimization algorithms:

1. Single-objective optimization (SOO).
2. Multi-objective optimization (MOO).

These algorithms focus on a single item while operating with a single goal function. The classification of VMP system influenced by biology is shown in Fig. 1.

3 The Single-Objective Optimization Algorithms

There are many nature-inspired optimization techniques. The notations for variables, constants, and parameters are given in Table 2.

Table 2 Parameters and notation

Parameters	Notation
ϕ_{ij}	Pheromone concentration
d_{ij}	Desirability of same route
S_{ij}	Distance
$\alpha_i \beta$	Influence parameter
g^*	Global best solution
x_i^*	Individual best
β_o	Attractiveness
ϵ_i^t	Gaussian distribution
ϵ	Drawn at random uniform distribution
$\otimes$	Entry-wise product
v_i^t	Velocity
x_*	Current best solution
$m(t)$	Number of VMs at time $t, t \in [0, \tau]$
$n(t)$	Number of hosts at time $t, t \in [0, \tau]$

3.1 Ant Colony Algorithm

Shabeera et al. [5] have developed a mechanism for allocation of VM to PMs using an ACO-based VMP algorithm for data and virtual machines. The PMs must support and provide sufficient resources to the VMs for refining as a host. Nie et al. [6] have proposed the greedy ant colony algorithm to reduce an application's overall execution time in heterogeneous environments to provide scalable computing resources. That can be used in various applications in heterogeneous computing environments. Characteristics of the nature-based VMP schemes are presented in Table 3.

Yang et al. [7] have proposed an improved ant colony optimization algorithm. The researcher's motive is to improve the utilization of cloud computing. Two factors were improved in this algorithm, a pheromone and inspiration based on the existing algorithm. Rai et al. [8] have introduced an algorithm used to solve the large-scale problem in the cloud. Some numerical results also show that it is useful to solve large-scale problems to improve the solution's quality.

3.2 Particle Swarm Optimization

In 1995, Kennedy and Eberhart [9] developed particle swarm optimization (PSO), inspired by natural swarm behavior such as fish and bird schools. The V_i and x_i represent a particle's speed and location which are computed using Eqs. (1) and (2).

$$Vi^{t+1} = Vi^t + \alpha\epsilon_1[g^* - {x_i}^t] + \beta\epsilon_2[x_i^* - x_i^t], \tag{1}$$

Table 3 Characteristics of the nature-based VMP schemes

	Simulator	VMP factors					
Algorithm	Cloud sim	Energy usage	Makespan	No. of VMs	Cost	Memory utilization	SLA
Improved honeybee	✓	✗	✗	✗	✗	✗	✓
Randomized honeybee	✓	✗	✓	✗	✗	✓	✗
Hybrid electro GA	✓	✗	✓	✗	✓	✓	✗
Quantum GA	✗	✗	✓	✗	✗	✓	✗
Johnson's GA	✓	✗	✓	✗	✓	✓	✗
Multi-population GA	✓	✗	✗	✗	✓	✗	✗
Hybrid firefly	✓	✗	✓	✗	✗	✗	✗
Utility firefly	✓	✗	✗	✗	✗	✓	✗
Hybrid BAT and GA	✓	✗	✓	✓	✗	✗	✗
Chaotic BAT swarm	✓	✗	✗	✗	✗	✗	✗
Improved ACO	✓	✗	✗	✗	✓	✗	✗
Map reduced ACO	✓	✗	✗	✗	✗	✗	✓
Greedy ACO	✓	✗	✓	✗	✗	✗	✗
Energy-efficient ACO	✓	✓	✗	✓	✓	✓	✗
Improved PSO	✓	✓	✗	✗	✗	✗	✗
Hybrid PSO	✓	✓	✗	✗	✗	✗	✗
Policy-based PSO	✓	✗	✗	✗	✓	✗	✗
PSO with levy flight	✓	✗	✓	✗	✗	✓	✓
Hybrid PSO	✓	✗	✓	✗	✗	✓	✗
Binary cuckoo	✓	✓	✗	✗	✓	✓	✗
Constrained cuckoo	✓	✓	✗	✗	✗	✗	✗

$$xi^{t+1} = xi^{t} + Vi^{t+1}, \tag{2}$$

where $\epsilon 1$ and $\epsilon 2$ are two random vectors which lie between 0 and 1 for every step. The parameters are acceleration constants or learning parameters which are commonly assumed to be α, $\beta \approx 2$.

The present global best appears to be highly significant, as illustrated in the accelerated PSO, the importance of individual best is not entirely evident. As a result, mutation and selection commonly perform the majority of PSO. As there is no crossover in PSO, particles with a high degree of exploration can have high mobility in PSO. A g^* appears to be used in a highly selective manner which suggests that it may have a dual meaning. It has the advantage of speeding up convergence by pulling toward the current best g^*. However, this may also result in premature convergence.

Gao et al. [10] have proposed an algorithm to improve the overall performance of task execution. Zeyu et al. [11] have proposed an algorithm to optimize particle swarm placement by analyzing the PSO. Kumar et al. [12] have proposed the GA-PSO algorithm to minimize the total execution time using GA. Wu et al. [13] have introduced an improved particle swarm algorithm to reduce the cost of the task scheduling time, or we can say that the convergence effect is far better in this case.

3.3 Fireflies Algorithm

Yang [14] has proposed a firefly algorithm (FA) that is based on tropical fireflies' flashing patterns and behavior. FA is clear-cut, flexible, and easy to use. A firefly's movement for more alluring (brighter) firefly j is presented in Eq. (3):

$$x_i^{t+1} = x_i^t + \beta_0 e^{-\gamma r_{ij}^2}(x_i^t - x_j^t) + \alpha_i^t, \tag{3}$$

where β_0 is the attractiveness at zero distance (r = 0). The second parameter is α randomization, it is a random integer vector drawn randomly from a Gaussian distribution at time t, and it is the randomization parameter. In terms of randomization, ϵ_i^t is easily expanded to other distributions including Levy flights.

Sood et al. [15] have proposed a hybrid task scheduling algorithm also known as hybridized firefly gravitational search algorithm. This algorithm is concerned for improving the processing time, response time, and makespan. Due to diverse cloud architectures, the most significant issues in the cloud are scheduling and load balancing. Tapale et al. [16] have carried out the scheduling, resource imbalance, and load balancing problems for their improvement. The author has also presented a utility-based load balancing method to maximize resource usage and load balancing.

3.4 Honeybee Algorithm

Pham et al. invented honeybee algorithm in 2005 [17]. It imitates honeybee colonies foraging behavior. It is the most simplest version, this approach performs a neighborhood search coupled with global tracking that is used for combinatorial and continuous optimization. Karaboga et al. [18] have proposed an artificial bee colony (ABC) algorithm. In this algorithm, worker bees look for food sources and promote them. The curious observer bees follow their fascinating leader and the scout bees take off independently to locate better food sources.

Rathore et al. [19] have proposed a randomized honeybee algorithm. This algorithm is used to optimize resource utilization and it also appreciates the best task from the overloaded VM.

3.5 Cuckoo Search

Cuckoo Search (CS) algorithm has been invented by Yang et al. in 2009 [20]. It is the most recent meta-heuristic algorithms influenced by nature. Additionally, Levy flights perform better than isotropic random walks. Recent research shows cuckoo search is highly effective than particle swarm algorithm and GA. Vianny et al. [21] have proposed a binary cuckoo search optimization (CSO). Ramkumar et al. [22] have proposed an algorithm-constrained CSO-based protocol, this protocol is a bio-inspired protocol used for routines in the cloud network.

3.6 Genetics Algorithm

Xiong et al. [23] have proposed Johnson's rule-based genetic algorithm for the two-stage task scheduling problem. This algorithm reduces the makespan for the physical machines in cloud centers. Velliangiri et al. [24] have proposed an algorithm that is used to improve the various parameters like makespan, load balancing, resource utilization, and expenses of the multi-cloud. Wang et al. [25] have proposed an algorithm load balancing based on multi-population. It shows lower task completion time, lower processing cost, and load balancing. Belmahdi et al. [26] have proposed the quantum genetic algorithm which provides better workflow scheduling in fog-cloud environments. It compares the algorithm with first come, first serve (FCFS) and classical GA. This algorithm is better in terms of makespan and it adapts appropriate resources.

4 The Multi-objective Optimization Algorithms

MOO function must be minimized or maximized to reduce MOO problem. Different solutions of MOO methods typically have to follow several limitations, just like SOO techniques. A condition of resource allocation known as Pareto efficiency, or Pareto optimally, is one where no one can improve without the case where one other person is getting worst result. Vilfredo Pareto (1848–1923), an economist and engineer from Italy, is remembered by the word. He utilized it in his research on income distribution and economic efficiency. Academic disciplines, including economics, engineering, and the life sciences, may all benefit from the approach. The goal of Pareto optimization is to solve a multifaceted problem. The user picks the Pareto front's best solution from a list of options that work best for them.

4.1 Bat Algorithm

Yang has proposed the meta-heuristic bat algorithm in 2010 [27]. It is influenced by how tiny bats use echolocation. This algorithm is the first of its type. Therefore, frequency tuning should be taken into account. Each bat has an n-dimensional search or solution space, velocity (vt), and location (xt). The x_* denotes the current best solution. To update the preceding three parameters, following Eqs. (4)–(6) will be used:

$$f(i) = f_{min} + (f_{max} - f_{min})\beta \tag{4}$$

$$V_i^t = v_i^{t-1} + (x_i^{t-1} - x_*) f_i \tag{5}$$

$$x_i^t = x_i^{t-1} + V_i^t. \tag{6}$$

A random vector is chosen at random from a uniform distribution $\beta \in [0, 1]$. Fundamentally, this is comparable to the cooling schedule element in simulated annealing. The multi-objective bat algorithm (MOBA) is highly effective. In MOBA, frequency tuning effectively functions as mutation, but selection pressure is maintained steadily using the best solution x_*. Although there is no direct crossover, there are loudness and pulse emission fluctuations. Additionally, when the search draws closer to the global optimally, the differences in loudness and pulsing emission rates also offer a self-zooming capability and make exploitation easier. Jian et al. [28] have proposed a hybrid algorithm; this algorithm is used to explore both the BA and GA benefits by initially addressing the software task placement problem. Further, Nuradis et al. [29] have improved the performance of the bat algorithm.

4.2 Bio-geography-Based Optimization VMP Schemes

Dimensional real-valued functions can be optimized using bio-geography-based optimization (BBO) which differs from traditional optimization techniques like quasi-Newton techniques and gradient descent. It does not depend on the function's gradient and does not require that the process be differentiable. Zheng et al. [30] have developed a VMPMBBO model that uses BBO to reduce resource waste and energy consumption. This method evaluates many parameters and the results are contrasted with existing multi-objective algorithm such as virtual machine consolidated placement (VMcP) systems, multi-objective grouping genetic algorithm (MGGA), and virtual machine placement ant colony system (VMPACS). VMP multi-BBO (VMPMBBO) is more efficient and has more substantial convergence properties when compared to MGGA and VMPACS.

5 Conclusion

The effect of number of PMs will make more influence on data center's energy consumption because PMs are responsible for a significant portion of the energy consumption in data center. The task of VMP in cloud data centers is more difficult in this scenario. As a result, several approaches to the VMP have been presented in recent years that emphasizes bio-inspired VMP schemes. We have presented a detailed study and characteristics of bio-inspired VMP techniques which were suggested for diverse computing settings in this work. It provides the necessary background information regarding the VMP issue. Then, it divides the developed VMP schemes into single-objective and multi-objective branches based on the optimization techniques. Further, it has been classified based on the existing optimization algorithms. The characteristics of VMP schemes are carried out in Table 3 which shows that how different optimization-based VMP schemes use different parameters of VM.

References

1. Meisner D, Gold BT, Wenisch TF (2009) Powernap: eliminating server idle power. ACM SIGARCH Comput Arch News 37(1):205–216
2. Kulshrestha S, Patel S (2021) An efficient host overload detection algorithm for cloud data center based on exponential weighted moving average. Int J Commun Syst 34(4):e4708
3. Elbeltagi E, Hegazy T, Grierson D (2005) Comparison among five evolutionary-based optimization algorithms. Adv Eng Inform 19(1):43–53
4. Parpinelli RS, Lopes HS (2011) New inspirations in swarm intelligence: a survey. Int J Bio-Inspired Comput 3(1):1–16
5. Shabeera T, Kumar SM, Salam SM, Krishnan KM (2017) Optimizing VM allocation and data placement for data-intensive applications in cloud using ACO metaheuristic algorithm. Eng Sci Technol, Int J 20(2):616–628

6. Nie Q, Li P (2016) An improved ant colony optimization algorithm for improving cloud resource utilization. In: 2016 international conference on cyber-enabled distributed computing and knowledge discovery (CyberC). IEEE, pp 311–314
7. Yang Z, Yu Y, Zhang K, Kuang H, Wang W (2017) An improved ant colony algorithm for mapreduce-based fleet assignment problem. In: 2017 IEEE 2nd advanced information technology, electronic and automation control conference (IAEAC). IEEE, pp 104–108
8. Rai H, Ojha SK, Nazarov A (2020) A hybrid approach for process scheduling in cloud environment using particle swarm optimization technique. In: 2020 international conference engineering and telecommunication (En&T). IEEE, pp 1–5
9. Eberhart R, Kennedy J (1995) Particle swarm optimization. In: Proceedings of the IEEE international conference on neural networks, vol 4. Citeseer, pp 1942–1948
10. Gao T, Tang Q, Li J, Zhang Y, Li Y, Zhang J (2022) A particle swarm optimization with lévy flight for service caching and task offloading in edge-cloud computing. IEEE Access 10:76636–76647
11. Zeyu M, Jianwei H, Yanpeng C (2020) Virtual machine scheduling in cloud environment based on annealing algorithm and improved particle swarm algorithm. In: 2020 IEEE international conference on artificial intelligence and information systems (ICAIIS). IEEE, pp 33–37
12. Kumar AS, Parthiban K, Shankar SS (2019) An efficient task scheduling in a cloud computing environment using hybrid genetic algorithm-particle swarm optimization (GA-PSO) algorithm. In: 2019 international conference on intelligent sustainable systems (ICISS). IEEE, pp 29–34
13. Wu D (2018) Cloud computing task scheduling policy based on improved particle swarm optimization. In: 2018 international conference on virtual reality and intelligent systems (ICVRIS). IEEE, pp 99–101
14. Yang X-S, He X (2013) Firefly algorithm: recent advances and applications. arXiv:1308.3898
15. Sood K, Jain A, Verma A (2017) A hybrid task scheduling approach using firefly algorithm and gravitational search algorithm. In: 2017 international conference on energy, communication, data analytics and soft computing (ICECDS). IEEE, pp 2997–3002
16. Tapale MT, Goudar RH, Birje MN, Patil RS (2020) Utility based load balancing using firefly algorithm in cloud. J Data, Inf Manag 2(4):215–224
17. Pham DT, Ghanbarzadeh A, Koç E, Otri S, Rahim S, Zaidi M (2006) The bees algorithm-a novel tool for complex optimisation problems. In: Intelligent production machines and systems. Elsevier, pp 454–459
18. Karaboga D, Gorkemli B, Ozturk C, Karaboga N (2014) A comprehensive survey: artificial bee colony (ABC) algorithm and applications. Artif Intell Rev 42(1):21–57
19. Rathore M, Rai S, Saluja N (2016) Randomized honey bee load balancing algorithm in cloud computing system. Int J Comput Sci Inf Technol 7(2):703–707
20. Yang X-S, Dev S (2010) Cuckoo search. In: Nature-inspired metaheuristic algorithms. Cambridge University Press, pp 105–117
21. Vianny DMM, Aramudhan M, Ravikumar G (2017) An effective binary cuckoo search optimization based cloud brokering mechanism on cloud. In: 2017 international conference on IoT and application (ICIOT). IEEE, pp 1–8
22. Ramkumar J, Vadivel R, Narasimhan B (2021) Constrained cuckoo search optimization based protocol for routing in cloud network. Int J Comput Netw Appl 8(6):795
23. Xiong Y, Huang S, Wu M, She J, Jiang K (2017) A Johnson's-rule-based genetic algorithm for two-stage-task scheduling problem in data-centers of cloud computing. IEEE Trans Cloud Comput 7(3):597–610
24. Velliangiri S, Karthikeyan P, Xavier VA, Baswaraj D (2021) Hybrid electro search with genetic algorithm for task scheduling in cloud computing. Ain Shams Eng J 12(1):631–639
25. Wang B, Li J (2016) Load balancing task scheduling based on multi-population genetic algorithm in cloud computing. In: 2016 35th Chinese control conference (CCC). IEEE, pp 5261–5266
26. Belmahdi R, Mechta D, Harous S, Bentaleb A (2022) Saga: quantum genetic algorithm-based workflow scheduling in fog-cloud computing. In: 2022 international wireless communications and mobile computing (IWCMC). IEEE, pp 131–136

27. Yang X-S (2010) A new metaheuristic bat-inspired algorithm. In: Nature inspired cooperative strategies for optimization (NICSO 2010). Springer, pp 65–74
28. Jian C, Chen J, Ping J, Zhang M (2019) An improved chaotic bat swarm scheduling learning model on edge computing. IEEE Access 7:58602–58610
29. Nuradis J, Lemma F (2019) Hybrid bat and genetic algorithm approach for cost effective SaaS placement in cloud environment. In: 2019 third international conference on I-SMAC (IoT in social, mobile, analytics and cloud)(I-SMAC). IEEE, pp 1–6
30. Zheng Q, Li R, Li X, Shah N, Zhang J, Tian F, Chao K-M, Li J (2016) Virtual machine consolidated placement based on multi-objective biogeography-based optimization. Futur Gener Comput Syst 54:95–122

Use of Blockchain to Prevent Distributed Denial-of-Service (DDoS) Attack: A Systematic Literature Review



Md. Rittique Alam, Sabique Islam Khan, Sumaiya Binte Zilani Chowa, Anupam Hayath Chowdhury, S. Rayhan Kabir, and Muhammad Jafar Sadeq

Abstract A Distributed Denial-of-Service (DDoS) assault overwhelms a server, network, or service with Internet traffic, disrupting regular traffic. As the Internet becomes more accessible, the new attack vector includes insecure, Internet-connected IoT devices, and Denial-of-Service (DoS) vulnerabilities in SDN architecture, such as southbound channel saturation. Blockchain technology holds immutable data in blocks without login or address. A coded username established through a unique mechanism makes hacking the blocks virtually impossible. This paper reports on a systematic literature review (SLR) aimed at identifying and structuring research on the use of blockchain to prevent DDoS attacks. We systematically selected and reviewed articles published in suitable venues in this review. The research outcome provides insight into what constitutes the main contributions of the field, identifies vacancies and opportunities, and purifies several predominant future research directions.

Keywords DDoS attack · Blockchain · Systematic literature review

Md. R. Alam
Department of SWE, Daffodil International University, Ashulia, Dhaka, Bangladesh
e-mail: rittique.swe@diu.edu.bd

S. I. Khan · S. B. Z. Chowa
American International University-Bangladesh, Dhaka, Bangladesh
e-mail: 22-92355-1@student.aiub.edu

S. B. Z. Chowa · A. H. Chowdhury · S. R. Kabir (✉) · M. J. Sadeq
Department of CSE, Asian University of Bangladesh, Ashulia, Dhaka, Bangladesh
e-mail: rayhan923@aub.edu.bd

A. H. Chowdhury
e-mail: anupamhc@aub.edu.bd

M. J. Sadeq
e-mail: jafar@aub.edu.bd

© The Author(s), under exclusive license to Springer Nature Singapore Pte Ltd. 2023

S. Chinara et al. (eds.), *Advances in Distributed Computing and Machine Learning*,
Lecture Notes in Networks and Systems 660,
https://doi.org/10.1007/978-981-99-1203-2_4

1 Introduction

A DDoS attack is a malicious attempt to disrupt the regular traffic of a targeted server, service, or network by overwhelming the target or its surrounding infrastructure with a flood of Internet traffic [1, 8]. The use of blockchain in DDoS attacks is no different. Regular studies are being made, increasing the need for tracking this comprehensive domain. There are multiple ways a study can be completed, but for this topic, we went for the systematic lecture review (SLR) approach [15]. In this case, using blockchain in DDoS attacks is our preferred agenda. We are looking to get results by studying relative works in this field [10–15]. Systematic reviews aim to indicate problems by recognizing, censoriously assessing, and amalgamating the discovery of all relevant, high-quality, independent studies directing one or more research questions. As the definition suggests, this research method suits our research topic as this is a dynamic, fault-predicting evolving research method [16]. On the other hand, blockchain is an essential component of Industry 4.0 and distributed machine learning [7, 9], and its connection with intelligent agents, neural networks, and social media information quality can be the opportunity for potential research scope [25–27]. Besides DDoS attacks, researchers are also concerned about blockchain for centralized databases, IoT, cost monitoring, and healthcare-related issues [28–31].

2 Research Questions and Article Selection

2.1 Research Questions

The questions raised by us cascade within the context of DDoS attacks and their utilization of blockchain. We have accumulated a total of five questions (Table 1). The purpose of posing these questions is to represent a consolidated perspective regarding the use of blockchain technology in DDoS attack research.

2.2 Inclusion Criteria

The subject area focuses heavily on techniques for preventing DDoS attacks, mainly using blockchain technology. Authors must identify the study's objective (e.g., DDoS attack, blockchain, prevent, mitigate, block, IoT) and provide digital evidence of research methodology, an explanation with good examples, and a few case studies. The article must contain a relevant connection to DDoS attacks and explain how to prevent them using blockchain technology. The review includes articles published in conferences and refereed journals. As with most Systematic Literature Reviews, no books are included in this review.

Table 1 Research questions

Category	Question	Main motivation
Target	How to conceal the identity of the user?	To determine if the proposed solution provides users personal information security
	How to restrict botnets from invading devices?	To determine the general security protocols
Approach	What can stop the bot masters from initiating instruction?	To identify which approach best to stop hackers
Target group	Can the incident of running malware codes from unknown devices be stopped?	To determine if the approach has solution to the most basic problems
	Can we stop the server from overloading?	To determine if the approach is suitable for long term uses

2.3 Inclusion Criteria

We have included pertinent articles using Automated Keyword Search from ACM, Google Scholar, IEEE Computer Digital Library, and Easy Chair libraries and databases that were published between January 2010 and January 2020. The following search results list is used here.

2.4 Manual Selection

After conducting a keyword search, numerous irrelevant articles were returned. Due to this issue, the authors manually selected some relevant articles (without key search). If there were any disagreements, they were resolved through group discussion (Table 2).

Table 2 Table for list of inclusion criteria

Terms representing DDoS attack	DDoS, Distributed denial of services, Cybersecurity attack, DDoS attack
Terms representing prevent	Prevent, Block, Mitigate, Reduce
Terms representing blockchain technology	Blockchain, Technology, Blockchain technology

2.5 Final Article Selection

Based on the keyword search and manual selection, the final article selection process yielded 24 articles [1–23] that discussed how to prevent DDoS attacks using blockchain in full or in part.

3 Attribute Framework

3.1 Attribute Identification

The objective of the attribute framework (Phase 3 of Fig. 1) for this SLR paper is to identify the attribute, characterize the selected articles, and answer the research questions (Table 1). The attribute has been set based on two criteria: (a) examining the domain of research; (b) responding to the research questions. We have subdivided the study's eight general attributes (Table 3) into sub-attributes for answering the questions. We have utilized keywords such as Web Services, Cybersecurity, DDoS Attacks, and others to obtain the sub-attributes.

3.2 Characterization of the Articles

Different reviewers may predict different attribute subsets for a particular selected article, as we all have unique interpretations. In order to avoid reviewer bias, each author reviewed each of the articles individually. This verification was performed by comparing the review information of the authors to the reviewed articles. The disagreements have been resolved through discussions between the authors. Using this methodology, we have characterized each of the reviewed articles.

4 Article Assessment and Review Results

In this phase, we merged all the attributes and sub-attributes of the reviewed articles. Here, we will present and interpret the study's findings by discussing the research question responses.

RQ1: How to conceal the identity of the user?

According to our research, there are primarily three types of DDoS attacks:

Volumetric Attacks: Nearly half of all DDoS attacks are volumetric. The user's identity is not concealed, as the hacker obtains all the information.

Protocol Attacks: In this instance, a three-way handshake scenario is created, and information is stolen. This attack is one of the most dangerous DDoS attacks due to

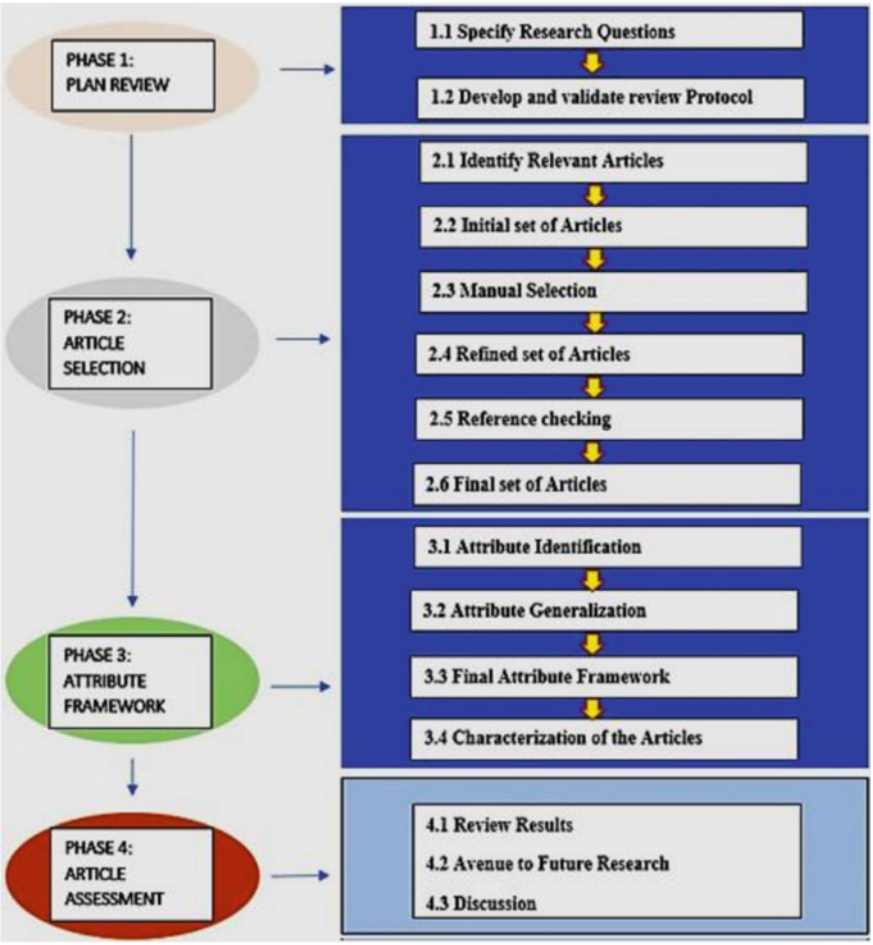


Fig. 1 Overview of the SLR process for this manuscript

the rate at which data is leaked. In this instance, the user's identity is not concealed, resulting in a data leak. According to our research, the blockchain prevents user information leakage in the vast majority of instances, and there is a proven method to prevent it.

Application Attacks: In this method, the hacker repeatedly sends a request to enter a web server, but only half of the request is sent. As a result, no other device can send a request, thereby jamming the server. In this scenario, identity is not revealed, as the attacker cannot access user data.

RQ2: How to restrict botnets from invading devices?

We devised these five DDoS attack methods after reviewing numerous literature reviews and academic papers.

Table 3 Table for list of research question criteria

Attribute	Sub-attribute	Brief overview
General		Review paper type, Publication date
Study type		Survey, Case study, Experiment, Post-data analysis
Study target	Prediction	Study on prevention of the DDoS attack by blockchain
	Study domain	Blockchain, DDoS attack, Cybersecurities, Web services
Case study	DDoS attacks studied	List of DDoS review papers study
	Examination on blockchain	Study on how blockchain prevents DDoS attacks
	Project size	Size measure of project domain
	Project domain	Application of domain of the blockchain projects covered
Data source	Source code	Code base & Github
	Contributions	
	External sources	
Methodology	Methods	Surveys, Case studies, Experiments, Data analysis
	Metrics	Applied metrics
Results	Prediction classification	Other findings
	Review results	
	Summary	
Validation		Validation process for the study

Entropy-based Anomaly Detection: This system is a cloud-based system that detects threats through database matching and threshold matching. It detects bot attacks by detecting different levels of the threshold value [21].

Packet Monitoring Approach: The packet monitoring approach counts the number of hops made by a device and differentiates between authentic and fake users [22].

Cloud Computing and Server: Cloud computing is a system that makes use of third-party applications installed on connected devices, monitors their location, and determines which packet is generating traffic. Furthermore, what actions does this packet perform on each of these devices? Thus, it detects whether or not bots have infiltrated any devices. This system can detect bots, but it is not suitable for large-scale work on a small network [23].

Identifier/Locator Separation: The first method prescribed for addressing the scalability issue in routing. Casting an identifier/locator, an identifier namespace, and a locator namespace to maintain the identities and locations of network nodes, respectively. This method is insufficient to prevent botnet invasions [24].

Blockchain: Several research papers led us to the conclusion that blockchain is a prevalent method. In addition, they all agree that this method is very effective for managing botnets because bots cannot enter the leading network because there are no intermediary devices or mediums.

RQ3: What can stop the bot masters from initiating instruction? Bot masters utilize and manage botnets to commit cybercrimes. Botnets are a combination of "robot" and "network." A single bot cannot cause significant damage and behaves like a single worm or virus; therefore, a hacker or bot master can achieve their goals by coordinating multiple bots. Distributed Denial of Service (DDoS) is an instance of bots cooperating. Botnets are predominantly used for DDoS attacks; in 2016, a significant DDoS attack hits the Internet infrastructure company DYN. A comprehensive strategy is required to prevent botnet infection, which includes safe web browsing and antivirus protection. Moreover, updating the operating system, utilizing a content distribution network (CDN), avoiding clicking on suspicious links, employing antivirus software, and other actions prevent bot masters from issuing instructions. Using blockchain technology, however, we can permanently prevent hackers or bot masters from issuing commands, thereby preventing DDoS attacks. A blockchain is an open, distributed ledger that can efficiently and irreversibly record transactions between two parties. A peer-to-peer network is responsible for inter-node communication and validating new blocks. The benefit of this technology is its ability to maintain security, which enables the next level of all-out attack against the opposition. Therefore, we can conclude that blockchain technology creates an impenetrable obstacle for hackers.

RQ4: Can the incident of running malware codes from unknown devices be stopped? Malicious code consists of unwanted files or programs that can cause damage to a computer or compromise stored data. To prevent these situations, one must install antivirus software, maintain the antivirus software's up-to-date status, run scheduled scans with antivirus software regularly, avoid using open-access Wi-Fi, and avoid visiting websites with questionable content or links. This is likely the most challenging task on the Internet: keeping our personal information secure and using strong passwords. Thus, we can prevent malware code from unknown devices from being executed.

RQ5: Can we stop the server from overloading? First, we must disable the device function that initially activates bot masters. If we want to stop the program that activates bot masters, we must first restart our computer and press the F8 key. When the Boot Menu appears on a computer configured to boot multiple operating systems, we can press the F8 key. When the Windows Advanced Options menu appears, we can also stop this by selecting an option and then pressing enter.

This paper highlights ongoing concerns regarding the use of blockchain technology to prevent DDoS attacks. Currently, there is insufficient information to provide definitive processes, mechanisms, and outcomes, but we are much closer to being able to design studies that will provide better answers.

5 Conclusion

Blockchain is one of the twenty-first century's most revolutionary innovations. There are no limits to this technology's applications. In this study, we examined the performance and efficacy of blockchain technology concerning Distributed Denial-of-Service attacks (DDoS). This review (SLR) paper explained how blockchain could be used to prevent denial-of-service (DDoS) attacks. The attribute diagram was created to classify and organize the chosen papers. A few questions were formulated with the aid of the selected papers and articles. We endeavored to keep it as uncluttered as possible so that readers can quickly find the information they need for future reference. The characterization of the reviewed articles will aid researchers in examining previous studies from the perspectives of metrics, methods, datasets, tool sets, and performance evaluation and validation techniques. Internet connectivity is expanding, as is the number of devices and electronics connected. In the not-too-distant future, even our household electronics and many other standard and industrially used devices will be integrated into the Internet of Things. This is both a cause for celebration and concern, as it would make it easier for unethical people who wish to harm the world and us to obtain their goals. This demonstrates the importance of preventing these individuals from controlling, overloading, and stealing data from users worldwide. This cannot be stressed enough, and the loss may be irreparable if an appropriate solution is not implemented promptly. Consequently, this technology is essential for the contemporary world and its problems.

References

1. Singh R et al (2018) Utilization of blockchain for mitigating the distributed denial of service attacks. Secur Priv 3
2. Jesus EF et al (2018) A survey of how to use blockchain to secure internet of things and the stalker Attack. Secur Commun Netw 2018
3. Kumar S, Paulsen JH (2020) Prevention of DDoS attacks. https://www2.deloitte.com/de/de/pages/technology-media-and-telecommunications/articles/cyber-security-prevention-of-ddos-attacks-with-blockchain-technology.html. Accessed 23 Jun 2022
4. Jung W et al (2017) Preventing DDoS attack in blockchain system using dynamic transaction limit volume. Int J Control Autom Syst 10
5. Patel D (2020) Blockchain technology towards the mitigation of distributed denial of service attacks. Int J Recent Technol 8
6. Mirkin M et al (2020) BDoS: blockchain denial-of-service. In: CCS '20: proceedings of the 2020 ACM SIGSAC conference on computer and communications security. ACM, New York, United States, pp 601–619
7. Hasan MK et al (2022) Evolution of industry and blockchain era: monitoring price hike and corruption using BIoT for smart government and industry 4.0. IEEE Trans Industr Inform
8. Taylor J (2022) What is a DDoS attack?. https://www.cloudflare.com/learning/ddos/what-is-a-ddos-attack/. Accessed 24 Jun 2022
9. Akhtaruzzaman M et al (2020) HSIC bottleneck based distributed deep learning model for load forecasting in smart grid with a comprehensive survey. IEEE Access 8
10. Vanessa RL (2018) A survey of how to use blockchain to secure internet of things and the stalker attack. Secur Commun Netw 2018

11. Ozkanli M (2020) How to prevent DDoS attacks with blockchain technology. https://medium.com/@mrtozkanli/how-to-prevent-ddos-attacks-with-blockchain-technology-5419529cb635. Accessed 25 Jun 2022
12. Kumar S (2020) Prevention of DDoS attacks. https://www2.deloitte.com/de/de/pages/technology-media-and-telecommunications/articles/cyber-security-prevention-of-ddos-attacks-with-blockchain-technology.html. Accessed 24 Jun 2022
13. Casino F et al (2019) A systematic literature review of blockchain-based applications: current status, classification and open issues. Telemat Inform 36
14. Iqbal M, Matuleviius R (2019) Blockchain-based application security risks: a systematic literature review. In: Proper H, Stirna J (eds) Advanced information systems engineering workshops, CAiSE 2019. Lecture notes in business information processing, vol 349. Springer, Cham
15. Taylor PJ et al (2020) A systematic literature review of blockchain cyber security. Digit Commun Netw 6
16. Siddaway AP et al (2019) How to do a systematic review: a best practice guide for conducting and reporting narrative reviews, meta-analyses, and meta-syntheses. Annu Rev Psychol 70
17. Kitchenham B (2004) Procedures for performing systematic reviews. UK
18. Cornelissen B (2009) A systematic survey of program comprehension through dynamic analysis. IEEE Trans Softw Eng 35
19. Xu M, Chen X, Kou G (2019) A systematic review of blockchain. Financ Innov 5:27
20. Mouli VR et al (2006) Web services attacks and security- a systematic literature review. Procedia Comput Sci 92
21. Navaz SSA et al (2013) Entropy based anomaly detection system to prevent DDoS attacks in cloud. Archive
22. Chouhan V, Peddoju SK (2012) Packet monitoring approach to prevent DDoS attack in cloud computing. Comput Sci
23. Jaber A et al (2016) Methods for preventing DDoS attacks in cloud computing. J Comput Theor Nanosci
24. Luo H et al (2013) DDoS attacks by identifier/locator separation. IEEE Netw 27
25. Kabir SR et al (2018) Relative direction: location path providing method for allied intelligent agent. In: Singh M, Gupta P, Tyagi V, Flusser J, ren T (eds) Advances in computing and data sciences, ICACDS 2018. Communications in computer and information science, vol 905. Springer, Singapore
26. Hasan MS et al (2021) Identification of construction era for indian subcontinent ancient and heritage buildings by using deep learning. In: Yang XS, Sherratt RS, Dey N, Joshi A (eds) Proceedings of fifth international congress on information and communication technology, ICICT 2020. Advances in intelligent systems and computing, vol 1183. Springer, Singapore
27. Haque R et al (2018) Modeling the role of C2C information quality on purchase decision in Facebook. In: Challenges and opportunities in the digital era, I3E 2018. Lecture notes in computer science, vol 11195. Springer, Cham
28. Haque R et al (2020) Blockchain-based information security of electronic medical records (EMR) in a healthcare communication system. In: Peng SL, Son LH, Suseendran G, Balaganesh D (eds) Intelligent computing and innovation on data science. Lecture notes in networks and systems, vol 118. Springer, Singapore
29. Sadeq MJ et al (2021) Integration of blockchain and remote database access protocol-based database. In: Yang XS, Sherratt S, Dey N, Joshi A (eds) Proceedings of fifth international congress on information and communication technology. Advances in intelligent systems and computing, vol 1184. Springer, Singapore
30. Sadeq MJ et al (2020) A cloud of things (CoT) approach for monitoring product purchase and price hike. In: Peng SL, Son LH, Suseendran G, Balaganesh D (eds) Intelligent computing and innovation on data science. Lecture notes in networks and systems, vol 118. Springer, Singapore
31. Akhtaruzzaman M et al (2019) A combined model of blockchain, price intelligence and IoT for reducing the corruption and poverty. TIIKM Publishing, Sri Lanka, pp 13–24

CS-Based Energy-Efficient Service Allocation in Cloud



Sambit Kumar Mishra, Subham Kumar Sahoo, Chinmaya Kumar Swain, Abhishek Guru, Pramod Kumar Sethy, and Bibhudatta Sahoo

Abstract Nowadays, cloud computing is growing rapidly and has been developed as an adequate and adaptable paradigm in solving large-scale problems. Since the number of cloud users and their requests are increasing fast, the loads on the cloud data center may be under-loaded or over-loaded. These circumstances induce various problems, such as high response time and energy consumption. High energy consumption in the cloud data center has drastic negative impacts on the environment. Literature shows that scheduling plays a significant role in the reduction of energy consumption. In the recent decade, this problem has attracted huge interest among researchers, and several solutions have been proposed. Energy-efficient service (task) allocation with high Customer Satisfaction (CS) constraint has become a critical problem of a cloud. In this paper, a high CS-based energy-efficient service allocation framework has been designed. This optimizes the energy consumption as well as the CS level in the cloud. The proposed algorithm is simulated in CloudSim simulator and compared with some standard algorithms. The simulation results show in favor of the proposed algorithm.

Keywords Cloud computing · Customer satisfaction (CS) · Energy · Task scheduling

S. Kumar Mishra (✉) · S. Kumar Sahoo · C. Kumar Swain
Department CSE, SRM University-AP, Guntur, India
e-mail: skmishra.nitrkl@gmail.com

A. Guru
KL deemed to be University, Guntur, India
e-mail: abhishekguru0703@kluniversity.in

P. Kumar Sethy
Department of Algorithms and Their Applications, Eötvös Loránd University, Budapest, Hungary
e-mail: pksethy@inf.elte.hu

B. Sahoo
National Institute of Technology Rourkela, Rourkela, India

© The Author(s), under exclusive license to Springer Nature Singapore Pte Ltd. 2023

S. Chinara et al. (eds.), *Advances in Distributed Computing and Machine Learning*,
Lecture Notes in Networks and Systems 660,
https://doi.org/10.1007/978-981-99-1203-2_5

1 Introduction

In the current era, the study of cloud computing is significant and exiting because it enables service providers to share computer resources (such as servers, memory, applications, connection, and services) easily and on demand. The cloud has three different stakeholders: Cloud Developer, Cloud Service Provider (CSP), and End-User [1]. To satisfy the overall quality-of-service (QoS) standards, the cloud developer is situated somewhere between the cloud service provider and the end-user. The Cloud Service Provider provides different infrastructures to the end-user that provides improved security while managing huge data and storing it [2]. Each CSP provides a wide range of services that the End-User uses. The benefit of leveraging affordable computing resources to fulfill complicated business requirements has led to the deployment of a wide variety of commercial cloud platforms. However, for applications like augmented reality, self-driving, and automation or digitalization of technology in the user equipment, computational power and minimal latency are essential. Cloud-based data centers are typically located distant from the user equipment and cannot achieve the computation-intensive and data-intensive applications.

The user's requests, the quantity of resources needed (such as processor, space, and storage) as well as the operating system the user specifies all go into the creation of virtual machines (VM). Since virtualization makes it possible for numerous VMs to run on a single physical server, it is feasible to combine VMs relying on appropriate placement techniques to distribute the most VMs among the fewest physical servers. Integrating virtual machines (VMs) is a very practical technique to accomplish a variety of goals.

The scheduling issue is becoming a major concern throughout the environment, and energy consumption also escalates, as the volume of cloud-based service providers and the traffic upon cloud servers both expand daily. Generally, crashes in the cloud system can happen in four key locations, particularly, among firms that provide services, within providers or suppliers, between a consumer and a supplier, moreover, among consumers. Service providers' shortcomings may result in more energy consumption and also cost. The ceaseless acceleration, increased competition toward cloud services and resource efficiency has set off a significant concern. Another concern is the increase in service requests which directly influence the QoS [3–5]. It has to only be managed efficiently by utilizing the cloud resources properly, i.e., by efficient resource allocation. Overloading resources cause performance to suffer, whereas underloading resources cause resource waste [3, 6–8]. Utilizing distributed resources is the purpose of cloud computing to efficiently attain high performance and throughput while resolving a huge number of size issues.

In this paper, we have designed CS-based energy-efficient service allocation in cloud computing. The proposed method is auto-adaptive. We have elaborate three states:(1) high CS level, (2) medium CS level, and (3) low CS level. The states need to be transferred from one state to another. This paper presents the research on the feasibility of CS-based energy-efficient service allocation in cloud computing. This introductory section presented an overview of the research, containing the motivation

and contributions, the related work found in the literature. The paper follows with the research methodology and how to design a CS-based cloud computing model.

The content in the present study is organized into five portions. The next section narrates the literature work on the allocation problem. The proposed customer-satisfactory-based allocation technique in the cloud is presented in Sect. 3. Section 4 reports research findings, which provide a side-by-side assessment of the existing techniques, in addition to that, it provides a statistical analysis of them. Finally, the paper is concluded in Sect. 5.

2 Related Research Work

In the recent decade, many researchers have studied the energy-efficient approaches within the cloud infrastructure and provided a strong framework for understanding the various angles of this problem. This section describes our reason for delivering this study and provides an overview of earlier survey investigations. The researchers present and investigate several methods or techniques for the allocation of services in the cloud computing environment. In this work, we analyze a few works associated with task scheduling. Pradeep and Jacob [9] proposed a multi-objective-based task scheduling where they used the cuckoo search and harmony search for optimization. They have considered various variables, such as expense, energy utilization, memory use, penalty, and credit. Among the most demanding issues in data centers is resource allocation, i.e., the challenge of ensuring user quality of service and reducing the workflow execution cost.

The authors of [10] have demonstrated a cost-aware workflow scheduling in the cloud, which overcomes workload balancing. The cost-aware challenges for workflow scheduling in the cloud are organized concerning QoS execution, framework functionality, and design of the framework. Shi et al. [11] have performed a scheduling scheme for the execution of deadline-based tasks in the cloud. Their primary objective is to optimize the energy consumed by the local mobile cloud system. Firstly, they stated the mathematical models of mobile and then present the probabilistic algorithm for scheduling purposes. They also developed the multi-objective scheduling method to facilitate mobile devices to manage complex real-time tasks to optimize energy consumption.

The researchers of [12] have investigated two types of load-balancing techniques: dynamic and hybrid methods. They have outlined the key attributes, difficult issues, benefits, and drawbacks of these methods. They have ignored the static techniques. The writers in [13] have explored the objectives, load-balancing strategies, and scheduling strategies that are currently in use. The chosen methods turned out to be divided into four groups: heuristic based, genetic, agent based, and dynamic methods.

A game-theoretical approach was used by [14] to offer resource allocation techniques for all the multi-cloud systems. They choose to address the issue of cost minimization associated with the distribution of virtual machines across various infras-

tructures. Even though job distribution is based on clearly specified SLAs, Farokhi has thoroughly examined the cloud system agreement [15]. He has described the SLA violation level in a multi-cloud environment during service delivery and also noted various challenges. To enhance several characteristics (such as makespan, overall cloud usage, benefit, and penalty cost) in a multi-cloud context, Panda and Jana suggested two SLA-based task scheduling algorithms [16]. The matrix's anticipated computation time is utilized to transmit the anticipated duration of each task across all virtual machines [17].

Our study and observations show that there is no comprehensive study regarding the prevailing methods for allocating energy-based services in the literature. Therefore, we make an effort to close this gap by taking a systematic approach. To cover the gap, this paper seeks to provide an orderly and in-depth analysis on energy-based service allocation methods, highlighting the best research in the area and comparing them.

3 Proposed Service Allocation Technique in Cloud

Many factors affect the energy consumption of the server, such as CPU, memory, hard disk, bandwidth, etc. Figure 1 gives an overview of the proposed CS-based energy-efficient service allocation framework. The cloud users put their jobs into the local queue of the service provider (CSP). Based on the resource requirement of each task, the CSP is inserted into different queues. There are three queues: (1) high CS level, (2) medium CS level, and (3) low CS level. In the first group of tasks, the customer satisfaction level is high, which means those tasks are going to be executed in high-end Virtual Machines (VMs). Therefore, here, there is a need for a reduction of energy consumption for the execution of tasks. In the second group of tasks, the CS level is medium, which means those tasks are going to be executed in VMs with medium capacity. Therefore, here, there is a demand for a reduction in energy consumption and also enhance the CS level. In the third group of tasks, the CS level is low, which means those tasks are going to be executed in low-end virtual machines. Therefore, here, there is a demand to improve the CS level. The service manager is responsible for the allocation of tasks to the cloud resources, i.e., VMs. The cloud has several data centers, and in Fig. 1, we have shown only one data center. Each data center has several physical hosts. In the figure, we have shown a cloud data center with k hosts (H_1, H_2, ..., H_k). Each host has a different number of virtual machines. The jth host H_j has m_j number of VMs. Here, V_{ij} represents the jth virtual machine of ith host.

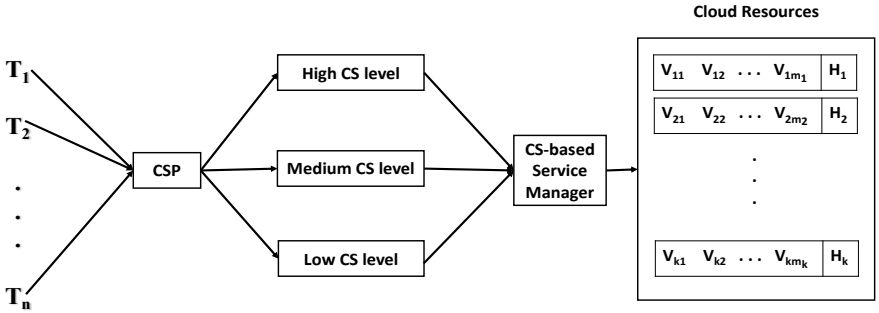


Fig. 1 CS-based energy-efficient service allocation framework

3.1 Service (Task) Model

The cloud user submits their tasks to the service provider for execution. In this work, we have followed the Expected Time to Compute (ETC) task model. Each service has tuples such as {Service_ID, Service Length, Deadline, Resource Requirement, CS_Level}. Here, the CS_Level can have values among {1, 2, 3}. CS_Level = 1 means the service is of high CS Level, CS_Level = 2 means the service is of medium CS Level, and CS_Level = 3 means the service is of low CS Level. Based on the CS level, the tasks are allocated to appropriate VMs with different objectives, as specified above.

The Service-Level Agreement (SLA) matrix is shown in Fig. 2. This matrix outlines the customer satisfaction level when executing their tasks in various VMs. $SLA_{i,jk}$ represents the SLA value or CS level when ith task executed in kth VM of jth host.

The Energy Consumption (EC) matrix is shown in Fig. 3. This matrix draws energy consumption when executing a task in various VMs. $EC_{i,jk}$ represents the energy consumption value when ith task executed in kth VM of jth host. These two matrices

$$SLA = \begin{array}{c} \\ T_1 \\ T_2 \\ T_3 \\ \vdots \\ T_n \end{array} \begin{array}{c} \overbrace{\qquad\qquad\qquad\qquad}^{H_1} \qquad\qquad\qquad\qquad \overbrace{\qquad\qquad\qquad\qquad}^{H_k} \\ \begin{array}{cccc} V_{11} & V_{12} & \cdots & V_{1m_1} \end{array} \quad \cdots \quad \begin{array}{cccc} V_{k1} & V_{k2} & \cdots & V_{km_k} \end{array} \\ \begin{bmatrix} SLA_{1,11} & SLA_{1,12} & \cdots & SLA_{1,1m_1} & \cdots & SLA_{1,k1} & SLA_{1,k2} & \cdots & SLA_{1,km_k} \\ SLA_{2,11} & SLA_{2,12} & \cdots & SLA_{2,1m_1} & \cdots & SLA_{2,k1} & SLA_{2,k2} & \cdots & SLA_{2,km_k} \\ SLA_{3,11} & SLA_{3,12} & \cdots & SLA_{3,1m_1} & \cdots & SLA_{3,k1} & SLA_{3,k2} & \cdots & SLA_{3,km_k} \\ \vdots & \vdots & \vdots & \vdots & & \vdots & \vdots & \vdots & \vdots \\ SLA_{n,11} & SLA_{n,12} & \cdots & SLA_{n,1m_1} & \cdots & SLA_{n,k1} & SLA_{n,k2} & \cdots & SLA_{n,km_k} \end{bmatrix} \end{array}$$

Fig. 2 Representation of SLA matrix

$$
EC = \begin{array}{c} \\ T_1 \\ T_2 \\ T_3 \\ \cdot \\ \cdot \\ \cdot \\ T_n \end{array}
\begin{array}{c}
\overbrace{\phantom{EC_{1,11}\ EC_{1,12}\ \ldots\ EC_{1,1m_1}}}^{H_1} \quad \ldots \quad \overbrace{\phantom{EC_{1,k1}\ EC_{1,k2}\ \ldots\ EC_{1,km_k}}}^{H_k} \\
\begin{array}{cccc} V_{11} & V_{12} & \ldots & V_{1m_1} \end{array} \quad \ldots \quad \begin{array}{cccc} V_{k1} & V_{k2} & \ldots & V_{km_k} \end{array} \\
\left(\begin{array}{cccc}
EC_{1,11} & EC_{1,12} & \ldots & EC_{1,1m_1} \\
EC_{2,11} & EC_{2,12} & \ldots & EC_{2,1m_1} \\
EC_{3,11} & EC_{3,12} & \ldots & EC_{3,1m_1} \\
\cdot & \cdot & \cdot & \cdot \\
\cdot & \cdot & \cdot & \cdot \\
\cdot & \cdot & \cdot & \cdot \\
EC_{n,11} & EC_{n,12} & \ldots & EC_{n,1m_1}
\end{array}\right) \quad \ldots \quad \left(\begin{array}{cccc}
EC_{1,k1} & EC_{1,k2} & \ldots & EC_{1,km_k} \\
EC_{2,k1} & EC_{2,k2} & \ldots & EC_{2,km_k} \\
EC_{3,k1} & EC_{3,k2} & \ldots & EC_{3,km_k} \\
\cdot & \cdot & \cdot & \cdot \\
\cdot & \cdot & \cdot & \cdot \\
\cdot & \cdot & \cdot & \cdot \\
EC_{n,k1} & EC_{n,k2} & \ldots & EC_{n,km_k}
\end{array}\right)
\end{array}
$$

Fig. 3 Representation of EC matrix

(i.e., SLA and EC) represent the task heterogeneity and machine heterogeneity of the cloud system. In this proposed system, SLA is mainly based on deadline and the CS level.

3.2 Proposed Algorithm

We have proposed an algorithm "CS-based Energy-Efficient Service Allocation" (*CSEESA*) for the allocation of services to optimize the energy consumption as well as the customer satisfaction level. The services (tasks) with their constraints (SLAs) and some different required pieces of information are provided to the Algorithm-1 as input. The Algorithm-1 computes the total energy consumed for the execution of all tasks.

Algorithm 1 : $CSEESA$: CS-based Energy-Efficient Service Allocation

Input: T: Set of tasks ($T = \{T_1, T_2, ..., T_n\}$), SLA: SLA Matrix; EC: Energy Consumption Matrix; $MIPS_{jk}$: Speed of kth VM of jth host;
Output: E: Total Energy Consumption.

1: Classify T into T_{HCS}, T_{MCS}, T_{LCS};
2: Allocate T_{HCS} to the VMs with less resource, where SLA should be maintained;
3: Allocate T_{MCS} to the appropriate VMs for reduction of Energy consumption and improve the CS level;
4: Allocate T_{LCS} to the VMs with high resource;
5: Calculate E_i for each task execution;
6: $E = \sum_{i=1}^{n} E_i$;
7: return E;

The tasks are classified into three groups based on the CS level. T_{HCS} represents the services with high CS level, T_{MCS} represents the services with medium CS level, and T_{LCS} represents the services with low CS level. So, for T_{HCS}, the energy consumption needs to be optimized by allocating these tasks into VMs with fewer resource capacities. For T_{MCS}, the energy consumption, as well as CS level, needs to be optimized by allocating these tasks into appropriate VMs. For T_{LCS}, the CS level needs to be increased by allocating these tasks into VMs with high speed. In this work, for any mapping between the task and VM, the SLA is maintained.

4 Simulations and Results

The experiment is executed on the CloudSim simulation platform for the cloud environment. The experimental environment is Windows 10, 64-bit operating system, Intel Core i5-7500 CPU, and 8GB memory. In this simulation, we performed many results to estimate the effectiveness of the suggested approach in cloud computing. Each host has a distinct overall resource capacity, and each host has a distinct number of virtual machines (VMs) with diversified resources. Each physical host in the system serves as the hypervisor for the stationing of virtual machines using "Xen". Every incoming task does have a unique resource demand which is assigned a unique task length (measured in Million Instructions (MI)), and is given a distinct deadline for the execution of the concerning service.

Here, the physical host information is not exhibited because it remains unchanged in comparison to the ideal state energy consumption of VM which is not taken into account. To evaluate the performance of the proposed algorithm ($CSEESA$), we have compared the algorithm with the random allocation algorithm, and the First-Come-First-Served (FCFS)-based allocation algorithm. The simulation for each model was performed 20 times and for each of these decisions, the CS level and the average energy consumption were evaluated. In Situation 1, the quantity of services is ranged between 100 and 1000 in 100-unit increments, with 80 VMs connected to the cloud network. In Scenario-2, the quantity of services is rooted, i.e., 500, but in the period of 20 s, the quantity of VM increases from 20 to 140. The three aforementioned algorithms have been used to determine the energy consumed by the cloud system. Figures 4, 5, 6, and 7 display the outcomes of an aggregate for 20 runs.

The random algorithm represents the average optimally of the service allocation problem. As demonstrated in Figs. 4 and 6, the energy parameter decreases more with the $CSEESA$ method than for the existing approaches including both scenarios. And, also the CS level is more for the proposed algorithm as compared to others, as shown in Figs. 5 and 7.

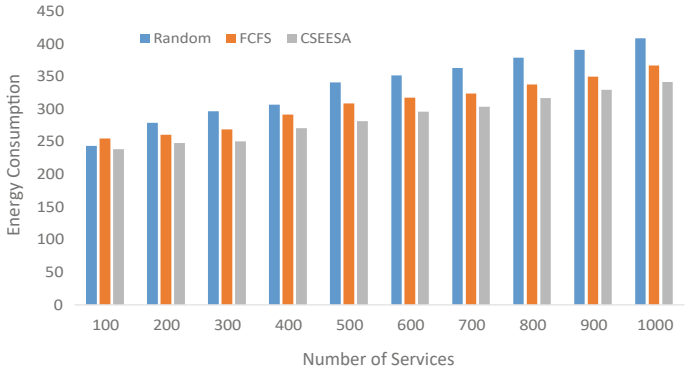


Fig. 4 Comparison of energy consumption for Scenario-1

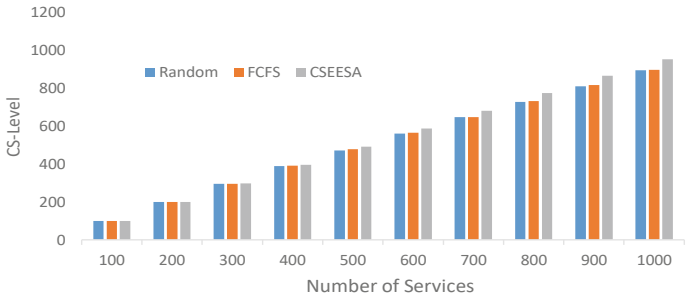


Fig. 5 Comparison of CS level for Scenario-1

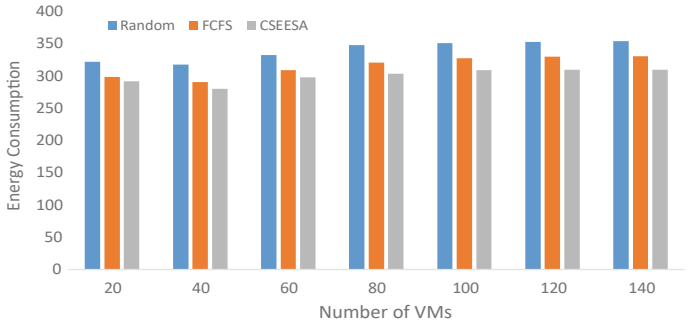


Fig. 6 Comparison of energy consumption for Scenario-2

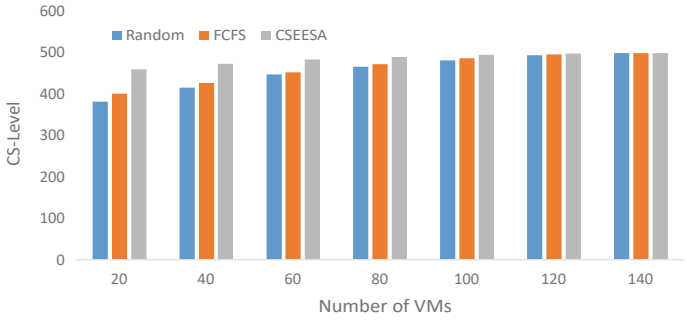


Fig. 7 Comparison of CS level for Scenario-2

5 Conclusion

There are many CS-based states or regions in cloud computing, and these states could transform from one state to another. In this study, we have proposed a CS-based (or SLA based) and auto-adaptive scheduling framework for energy efficiency and offer resources flexible in the cloud system. According to the system's current state, three possible models will be automatically chosen to accomplish the goals. The outcome reports that our proposed approach CSEESA can perform distinct environments in different situations of the cloud system. Two metrics have been designed to identify the status of the system, and the results show that the proposed method performs better for service allocation in our simulations.

References

1. Li H, Zhu G, Zhao Y, Dai Y, Tian W (2017) Energy-efficient and QoS-aware model based resource consolidation in cloud data centers. Clust Comput 20(3):2793–2803. https://doi.org/10.1007/s10586-017-0893-5
2. Chen J, Li K, Tang Z, Bilal K, Yu S, Weng C, Li K (2016) A parallel random forest algorithm for big data in a spark cloud computing environment. IEEE Trans Parallel Distrib Syst 28(4):919–933. https://doi.org/10.1109/TPDS.2016.2603511
3. Mishra SK, Puthal D, Sahoo B, Jena SK, Obaidat MS (2018) An adaptive task allocation technique for green cloud computing. J Supercomput 74(1):370–385. https://doi.org/10.1007/s11227-017-2133-4
4. Mishra SK, Sahoo S, Sahoo B, Jena SK (2019) Energy-efficient service allocation techniques in cloud: a survey. IETE Tech Rev :1–14. https://doi.org/10.1080/02564602.2019.1620648
5. Dai X, Wang JM, Bensaou B (2015) Energy-efficient virtual machines scheduling in multi-tenant data centers. IEEE Trans Cloud Comput 4(2):210–221. https://doi.org/10.1109/TCC.2015.2481401
6. Mishra SK, Sahoo B, Parida PP Load balancing in cloud computing: a big picture. J King Saud Univ Comput Inf Sci 32(2):149–158. https://doi.org/10.1016/j.jksuci.2018.01.003

7. Puthal D, Sahoo BP, Mishra S, Swain S (2015) Cloud computing features, issues, and challenges: a big picture. IEEE Int Conf Comput Intell Netw :116–123. https://doi.org/10.1109/CINE.2015.31
8. Mishra SK, Khan MA, Sahoo S, Sahoo B (2019) Allocation of energy-efficient task in cloud using DVFS. Int J Comput Sci Eng 18(2):154–163. https://doi.org/10.1504/IJCSE.2019.097952
9. Pradeep K, Jacob TP (2018) A hybrid approach for task scheduling using the cuckoo and harmony search in cloud computing environment. Wirel Pers Commun 101(4):2287–2311. https://doi.org/10.1007/s11277-018-5816-0
10. Alkhanak EN, Lee SP, Khan SUR (2015) Cost-aware challenges for workflow scheduling approaches in cloud computing environments: Taxonomy and opportunities. Futur Gener Comput Syst 50:3–21. https://doi.org/10.1016/j.future.2015.01.007
11. Shi T, Yang M, Li X, Lei Q, Jiang Y (2016) An energy-efficient scheduling scheme for time constrained tasks in local mobile clouds. Pervasive Mob Comput 27:90–105. https://doi.org/10.1016/j.pmcj.2015.07.005
12. Milani AS, Navimipour NJ (2016) Load balancing mechanisms and techniques in the cloud environments: systematic literature review and future trends. J Netw Comput Appl 71:86–98
13. Tiwari PK, Joshi S (2016) A review on load balancing of virtual machine resources in cloud computing. In: Proceedings of first international conference on information and communication technology for intelligent systems, vol 2. Springer, Cham, pp 369–378
14. Ardagna D, Ciavotta M, Passacantando M (2017) Generalized nash equilibria for the service provisioning problem in multi-cloud systems. IEEE Trans Serv Comput 10(3):381–395. https://doi.org/10.1109/TSC.2015.2477836
15. Farokhi S (2014) Towards an SLA-based service allocation in multi-cloud environments. In: 14th IEEE/ACM international symposium on cluster, cloud and grid computing (CCGrid), pp 591–594. https://doi.org/10.1109/CCGrid.2014.62
16. Panda SK, Jana PK (2017) SLA-based task scheduling algorithms for heterogeneous multi-cloud environment. J Supercomput 73(6):2730–2762. https://doi.org/10.1007/s11227-016-1952-z
17. Mishra SK, Puthal D, Sahoo B, Sharma S, Xue Z, Zomaya AY (2018) Energy-efficient deployment of edge dataenters for mobile clouds in sustainable IoT. IEEE Access 6:56587–56597. https://doi.org/10.1109/ACCESS.2018.2872722

Small-Footprint Keyword Spotting in Smart Home IoT Devices



A. Mohanty, K. Sahu, R. Parida, G. Pradhan, and S. Chinara

Abstract When it comes to automatic speech recognition, keyword spotting plays a very important role in making human–machine interaction effective. Keyword spotting is a technique of detecting predefined keywords from a continuous audio stream. This project aims to design a system that can spot keywords with high accuracy and low latency without the need for a huge memory footprint or huge computation power. This thesis implements a keyword spotting system with the help of different neural network models. It also gives us an idea of the effect of different feature extraction techniques on the performances of such models. The model is finally deployed in a lightweight edge device to perform various applications in a smart home setting. The thesis primarily explains the pipeline of exporting the models built on PyTorch to Raspberry Pi and finally running inference using TensorFlow-Lite runtime and techniques to handle the continuous audio stream given as input. Results obtained from training suggested that the Log-Mel Filter Bank Energy Delta feature extraction method and the depthwise separable convolutional neural network performed better than the rest of its competitors. Multiple real-time keywords could be detected with minimum latency when these methods are implemented in an edge device.

Keywords Keyword spotting · Feature extraction · Raspberry Pi · TensorFlow-Lite · Internet of things · Deep learning

1 Introduction

If one of the significant-tech developments in the 2000s had been smartphones, the next big step in the 2020s was the development of smart home systems. We could tap into high-tech functionality and luxury that was not possible in the past with smart

A. Mohanty (✉) · K. Sahu · R. Parida · S. Chinara
National Institute of Technology, Rourkela, India
e-mail: bunty.arijeet04@gmail.com

G. Pradhan
Vellore Institute of Technology, Chennai, India

© The Author(s), under exclusive license to Springer Nature Singapore Pte Ltd. 2023

S. Chinara et al. (eds.), *Advances in Distributed Computing and Machine Learning*,
Lecture Notes in Networks and Systems 660,
https://doi.org/10.1007/978-981-99-1203-2_6

home systems. The rise of IoT and advancements in machine learning and deep learning have led to a new era of IT technologies where systems are more modern and intelligent. This progress led the big tech companies to focus their attention on carrying out processes with the help of voice commands the user speaks. For example, Google Home and Amazon Echo can perform many tasks using their voice assistant system. Even our laptops and smartphones come with Cortana and Siri to ease the tasks.

Traditional systems used to run every time, constantly transmitting the voice data to the server side. The voice data are processed in the cloud with the help of a speech recognition algorithm, and the output is sent back. A single person can transmit 94 MBs of data per day, making power and network usage costly. Moreover, it is not feasible to process 100s of petabytes of data per day on the server side. These technicalities can further give rise to congestion problems in network and cloud services, increased latency, and privacy concerns for users. To tackle such problems, researchers came up with the idea of activating the systems using specific keywords.

When it comes to automatic speech recognition, keyword spotting plays a critical role in effective human–machine interaction. Keyword spotting is a technique of finding specific keywords in continuous audio data. This system can reduce the load on the server-side processing and the network traffic. At the same time, it can protect user privacy and provide higher positioning. Keyword spotting aims to achieve high accuracy and low latency without increasing the cost of storage and processing power.

The main idea behind this technique is to recognize speech in offline mode with the help of an algorithm running inside a dedicated MCU that wakes up the application processor whenever a keyword is detected. A significant disadvantage of these kinds of system is that it has to be in the always-listening mode. It would run on an MCU, requiring a minimal footprint and low computation cost and low power consumption. To provide a natural experience to the user, it has to accurately determine multiple keywords in real time.

This paper aims to design a system that can spot keywords with high accuracy and low latency without needing a vast memory footprint or enormous computation power.

2 Background

2.1 State of Art

Over the past decade, there has been substantial research on KWS, but most are not suitable for low-latency applications. A common and competitive technique was proposed where Hidden Markov Model (HMM) was used but it was often computationally expensive as it required Viterbi decoding and not feasible for low-latency applications [1]. Other works included techniques like large margin formulation and

recurrent neural networks. Nevertheless, these often required processing entire utterances of the sentence, thus increasing latency [2, 3].

Chen et al. (2014) proposed a DNN-based KWS system that yielded better search performance over the standard HMM KWS system and led to a more straightforward implementation [4]. Furthermore, Nakkiran et al. (2015) proposed a method of compressing DNN using a rank-constrained topology. This optimized the model to use less memory and computation cost. But the disadvantages of using DNN were that the model did not recognize local temporal and speech correlation in the data, making the model inefficient for speech recognition [5]. To overcome this issue, CNN was used to capture local features to achieve higher accuracy [6]. However, CNNs could not capture global features. So, later Sun et al. (2016) proposed an LSTM, which could recognize the global temporal features, thus achieving better performance in speech processing over sequenced data [7]. However, this failed to detect the spectral features. Arık et al. (2017) introduced a convolutional recurrent neural network (CRNN), which was a combination of both CNN and RNN to exploit local and long-range contexts [8]. The depthwise separable convolution neural network (DSCNN) is a modern variation of CNN developed by Zhang et al. (2017), which performed classic 3D convolution by splitting the channels in depthwise and then performing pointwise convolutions to aggregate the channels [9].

Shahnawaz et al. (2018) give a comparison between different neural networks used by comparing how different parameters of feature extraction techniques can affect the model accuracies and model size. Out of all the models, DSCNN performed the best in terms of ROM requirement, whereas GRU performed the best in terms of RAM requirement. LSTM also required minimal RAM [10]. Later, Wei et al. (2021) introduced a new variant of CRNN called EdgeCRNN. It showed an accuracy of 97.89% with only 14.54M parameters, proving that the model is lightweight [11].

2.2 Feature Extraction Techniques Explored in This Work

Two popular feature extraction techniques used for keyword spotting are Mel-Frequency cepstral coefficient (MFCC) and Log-Mel Filter Bank Energies Delta (LFBE-Delta).

2.3 Neural Networks Explored in This Work

We found out from literature review that there are some models which can be both lightweight and can spot keywords with a good accuracy. Such models are discussed below.

LSTM [7] An important element of an LSTM is cell state that runs horizontally through the architecture. The input can pass straight through the cell state without any

modification and hence we can understand the cell condition of LSTM by comparing it with a conveyor belt. LSTM consists of structures in their architecture known as gates that allow the addition or deletion of information to the cell state. The responsibility of the gate is to decide whether or not to let the information pass through. Gates are made up of sigmoid neural net with a pointwise multiplication operation.

DSCNN [9] DSCNN divides an n-dimensional input tensor into separable channels. A two-dimensional filter is used to convolve the input for each channel. The n-dimensional tensor is then constructed by stacking the outputs of each channel. In the end, the n-channels are merged to generate x-channels. After feature extraction from audio samples, we have 39 dimensions of input in our scenario. These 39 channels are isolated and convolved separately before merging to create a single channel.

EdgeCRNN [11] The EdgeCRNN model consists of an EdgeCRNN block followed by two base blocks after the first layer. Two branches make up the EdgeCRNN block. EdgeCRNN used the architecture of DSCNN by employing depthwise convolution and pointwise convolution. The outputs of each branch are concatenated after the input is passed through both branches. One branch just forwards the input while the network is passed via pointwise and depthwise convolutions on the other branch. The output of both branches is combined to produce output. The output of the EdgeCRNN block is fed via another convolution after three stages. Flattening is done after the final convolution and fed into a FC layer after passing through an LSTM. The final output contains the logits for each audio label.

TCN [12] Temporal Convolutional Network or TCN combines aspects of both RNN and CNN. The TCN architecture consists of a single-dimensional fully connected network. Each of the hidden layer has the same length as the input. There is zero padding used along with kernel of size 1 in order to get the length of the next layers same as that of the previous layer. The architecture also consists of dilated convolutions that widen the reception field to an exponential magnitude. Finally there are residual blocks that are responsible for concatenating the input with the output of the hidden layer.

3 Training

3.1 Dataset and Experimental Setup

The dataset used to train the model is Google Speech Commands Dataset [13], a very common dataset for KWS systems. Each audio data is stored in a one second wav file sampled at a rate of 16 kHz. Our task is to discriminate among 10 classes "left," "no," "off," "on," "one," "right," "three," "two," "yes," and "zero." The dataset is then trained on all models in two ways—one by using MFCC and another by using

LFBE-Delta. The accuracy of both cases is recorded, and the result is obtained. After training, we found out that the training time of the model is high. So, we took the help of cyclic LR to reduce the training time [14].

4 Deployment

4.1 Conversion of PyTorch Model to TensorFlow Lite

TensorFlow-Lite is optimized specially for computationally constrained devices with a focus on speed, model size, and power consumption [15]. It has a runtime module that consists of an interpreter which is extremely lightweight (not more than 300 KB) and is responsible for running the final optimized model. It also has a converter which converts the model in raw format into the TF-Lite format, which the interpreter may utilize for inference at runtime.

4.2 Open Neural Network Exchange (ONNX)

Our current implementation of all of the above models mentioned is in PyTorch. Since TF-Lite is written with the TensorFlow model as a reference point, there is no direct way to convert the PyTorch model to the TF-Lite model. ONNX is an open format built to represent machine learning models [16]. ONNX defines a common set of operators, the building block of machine learning and deep learning models and a common file format which enables AI developers to use models with a variety of framework, tools, runtimes, and compilers.

4.3 Post-training Quantization

Quantization is one of the most useful optimizations used by TF-Lite. With little loss in model accuracy, post-training quantization is a conversion approach that can minimize model size while decreasing CPU and hardware accelerator latency. Deep learning stores weights and biases as 32-bit floating-point numbers to enable for high-precision computations during model training. To decrease latency, this conversion is done only once and cached.

4.4 Handling of Multiple Keywords

The whole process of training and inferencing up to this point on includes audio files with just one spoken word. Our use case will include waking up the device on the utterance of a particular keyword and upon waking up listening for more commands and performing actions accordingly. Both of these cases would require handling audio files with multiple spoken keywords. The user will not be constrained to just speaking one word hence we need to handle audio files with multiple spoken word. To obtain this we use the following strategies.

Splitting on Silence To automate the process of recognizing distinct words from an audio stream, we used an open-source Python library called Pydub which accepts an AudioStream Object, splits it on silence, i.e., when its quieter than a particular threshold (−25 dbFS) and finally returns an array of recognized words in the form of .wav files. These .wav files being all single word commands are ultimately fed to the DSCNN model for inference. Although the splitting was crisp and clear but the inference could not be in real time as it only recorded a single audio file (Figs. 1 and 2).

Sliding Window The idea behind the sliding window technique was to slide a window of a predefined length across an infinitely long audio stream with an appropriate offset. This way it would give the result in almost real time and the offset made sure that no word is missed. The window length was chosen to be 1 s with a sampling rate of 16kHz and a stride of 0.5 s was used. Every 0.5 s the inference callback function

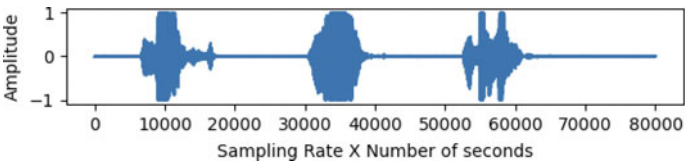


Fig. 1 Waveform

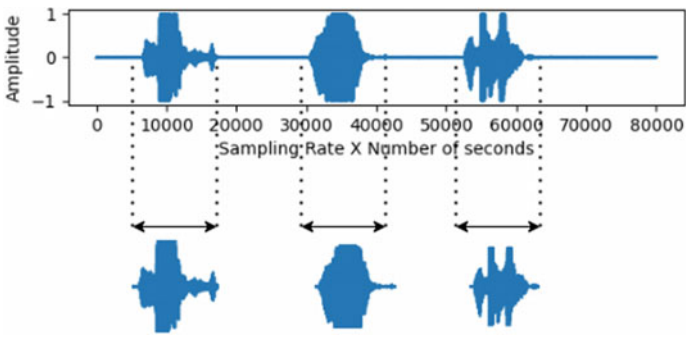


Fig. 2 Splitting audio based on silence

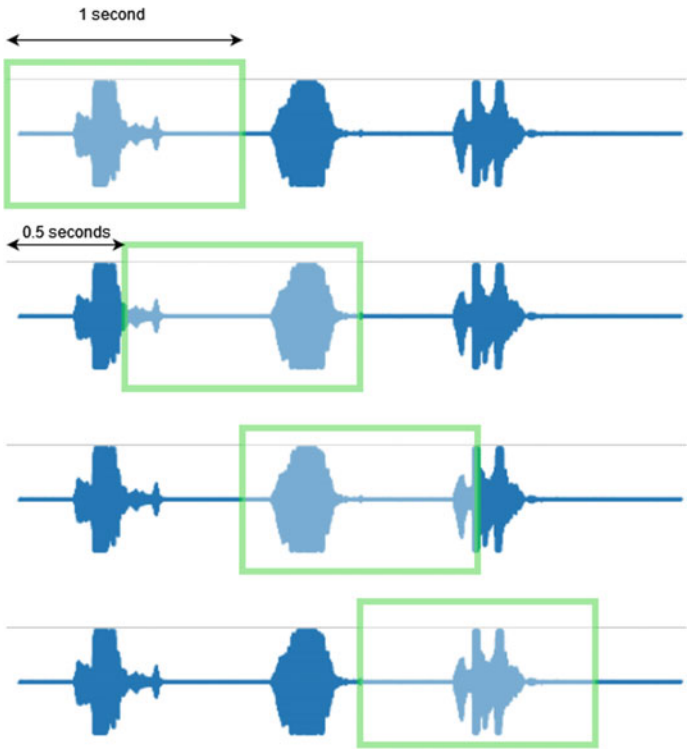


Fig. 3 Sliding window waveform

is called which takes a window of 1 s as input and predicts the keyword with a mere latency of 0.3 s. The problem with this approach was multiple unnecessary inferences were made in each window which also led to large number of false positives as an appropriate threshold was difficult to set for each trained word. The latency was arguably high as the inference callback functions were queued up as the window progressed adding up to the total delay (Fig. 3).

Splitting on Silence in Sliding Windows In order to have the detection engine running all the time we combined the sliding window technique with Pydub. A sliding window of length 5 s with a stride of 4 s was moved across an infinitely running audio stream. Every 4 s an audio file that is 5 s long is split on silence into distinct words and every word is fed into the model for inference as shown in the figure below (Figs. 4 and 5).

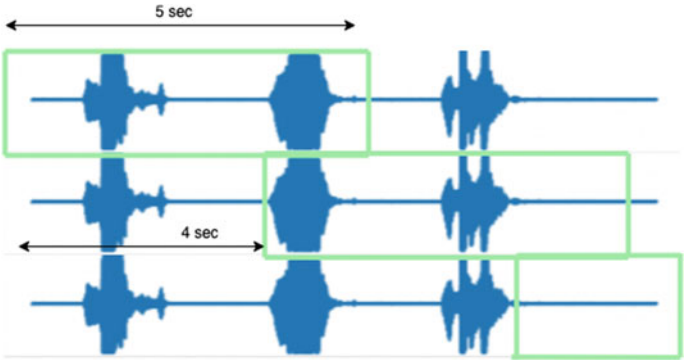


Fig. 4 Waveform representation of splitting on silence in sliding windows

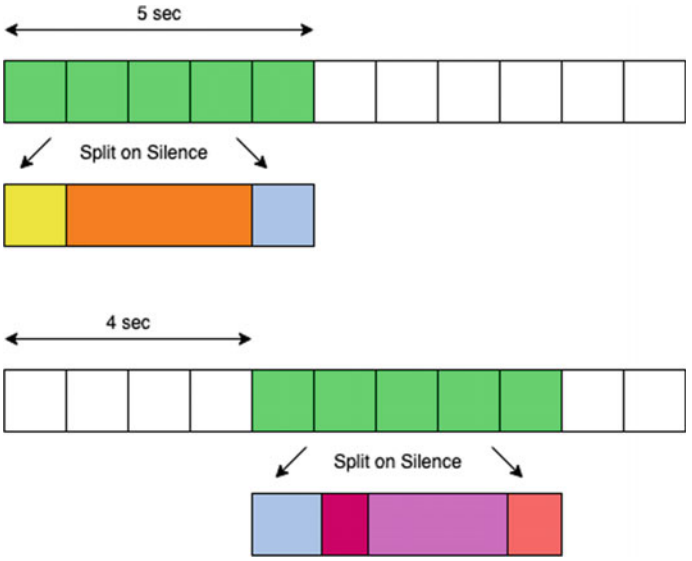


Fig. 5 Timeblock representation of splitting on silence in sliding windows

4.5 Inference in Raspberry Pi

The Raspberry Pi 3 Model B+ was utilized in this project to implement our model on the edge device as we needed an underpowered and lightweight device. The TF-Lite model is loaded into the device, and then the audio stream is given as input through a microphone. The device preprocesses the audio signals by applying LFBE-Delta and then passes it to the model for inference. The model, in turn, returns the word scores of all the classes on which it is trained. For our inference task, we needed four classes, i.e., "on," "off," "zero," and "one." The inference engine runs on a callback

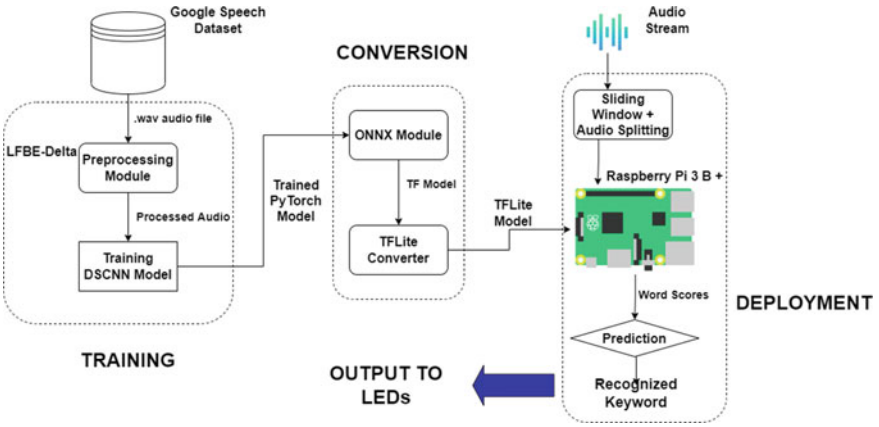


Fig. 6 Overall pipeline

function which is called every 5 s to infer results from the audio stream. This method ensures that our system is real time and can detect multiple keywords at a time. The word scores are then sent to the prediction module for further inference. The word scores obtained from the model are passed through a softmax layer to compute the relative probabilities between keywords. The relative probabilities are compared, and the maximum probability is chosen provided that the probability is higher than a predefined probability threshold. After repeated trials we found out that the standard 50% threshold works perfectly fine for our use case. All predictions with less than 50% confidence were discarded as "unknown" or "background noise" (Fig. 6).

5 Results

First, we evaluate our models with respect to the feature extraction techniques (LFBE-Delta and MFCC), which is depicted in Table 1. It shows that the accuracy obtained using LFBE-Delta is always better than MFCC. Thus, we can say that it is always beneficial to use LFBE-Delta rather than MFCC for keyword spotting systems.

Table 1 Comparison of accuracies when we compare MFCC to LFBE-Delta trained for 10 classes for 15 epochs

Model name	MFCC accuracy	LFBE-del accuracy
TCN(8 layers)	90.88	91.5
EdgeCRNN(width multiplier 1x)	89.61	94.78
DSCNN	96.17	96.66
LSTM	74.81	78.86

If we talk about keyword spotting systems in particular, MFCC can only extract the envelope information on the spectrum and thus loses some sound details in the process. If the dataset involves utterances of large words or complete sentences, it might be suitable to use MFCC in that case. But our dataset involves smaller keywords and recognition of particular keywords, so LFBE on such a dataset can contain more features such as low frequency and spectral details. Therefore, it is quite safe to assume that LFBE-Delta will always perform better than MFCC in keyword spotting systems. We then evaluate the performance of all the models when LFBE-Delta is used as the feature extraction technique and obtain the results shown in Table 2.

From the table, it is quite sure that except LSTM, all other models performed good in terms of accuracy. The accuracy achieved in case of EdgeCRNN is also very promising. The model size varies a lot if we move from one model to another. If we use a width multiplier of 0.5x, we find that the model size decreases to 660 KB, which is a 65% reduction in model size from the EdgeCRNN model with a width multiplier of 1x. There was a slight decrease in accuracy but not large enough to make a significant effect in the keyword spotting. This model size is also light enough to be used on edge computing devices.

As DSCNN showed the best results in training, we moved on to deploy it in the RPi, but to do that, we need to convert it to a TensorFlow-Lite model. We first converted it to ONNX and then to TensorFlow, followed by using the TF-Lite Converter module to convert it into a TF-Lite model. There is a clear reduction in model size with very little change in accuracy. This is depicted in Table 3.

Table 2 Comparison of accuracies for different neural networks on Google Speech dataset with 10 classes for 15 epochs

Model name	#Epochs	Accuracy	Per epoch time	Model size
TCN(8 layers)	15	91.5	50 s	320 KB
EdgeCRNN(width 0.5x)	15	93.15	65 s	660 KB
EdgeCRNN(width 1x)	15	94.78	75 s	1900 KB
DSCNN	15	96.66	535 s	252 KB
LSTM	15	78.86	18 s	988 KB

Table 3 Comparison of accuracies achieved with PyTorch, TF-Lite int8 Model, and TF-Lite float16 Model

Properties	PyTorch model	TF-Lite int8 model	TF-Lite float16 quantized model
Accuracy (in %)	96.66	95.77	96.5
Model size (in KB)	252	80	124
Inference time (in secs)	–	0.74	0.34

6 Conclusion

After training all the models with MFCC and LFBE-Delta as feature extraction methods, we find equal or better accuracies with LFBE-Delta in all cases. The results obtained suggested that LFBE-Delta is better at capturing the features from audio samples than MFCC. Training all the different models with the LFBE-delta shows that the EdgeCRNN model gives good accuracy. The 1x width multiplier variant of EdgeCRNN has a large size, but if we consider the 0.5x variant, we find it suitable for IoT devices. The DSCNN model has the highest accuracy and has the minimum model size. In contrast, the LSTM model gives us low accuracy, suggesting that it is not suitable for this use case.

After training and implementing the DSCNN model in the Raspberry Pi, we found that when we consider the TF-Lite models, we see a very high reduction in model size, and the accuracy reduction is minimal. We observe a 68.2% reduction in model size in the int8 version of the TF-Lite model with just a 1% reduction in accuracy. The inference time for our model is 0.34 s for the TF-Lite float16 model and 0.74 s for the TF-Lite int8 model. This latency is practical for the detection of a keyword. Our system is also able to detect multiple keywords in real time. The proposed system met the conditions provided in the project's objective, i.e., minimal model size (small footprint), low latency, and low power consumption.

References

1. Rose RC, Paul DB (1990) A hidden Markov model based keyword recognition system. Int Conf Acoust, Speech, Signal Process 1:129–132. https://doi.org/10.1109/ICASSP.1990.115555
2. Keshet J, Grangier D, Bengio S (2009) Discriminative keyword spotting. Speech Commun 51(4):317–329. https://doi.org/10.1016/j.specom.2008.10.002
3. Fernández S, Graves A, Schmidhuber J (2007) An application of recurrent neural networks to discriminative keyword spotting. Lecture notes in computer science, vol 4669, pp 220–229. https://doi.org/10.1007/978-3-540-74695-9_23
4. Chen G, Parada C, Heigold G (2014) Small-footprint keyword spotting using deep neural networks. In: 2014 IEEE international conference on acoustics, speech and signal processing (ICASSP), pp 4087–4091. https://doi.org/10.1109/ICASSP.2014.6854370
5. Nakkiran P, Alvarez R, Prabhavalkar R, Parada C (2015) Compressing deep neural networks using a rank-constrained topology. Interspeech 2015, pp 1473–1477. https://doi.org/10.21437/interspeech.2015-351
6. Sainath TN, Parada C (2015) Convolutional neural networks for small-footprint keyword spotting. Interspeech 2015. https://doi.org/10.21437/interspeech.2015-352
7. Sun M, Raju A, Tucker G, Panchapagesan S, Fu G, Mandal A, Matsoukas S, Strom N, Vitaladevuni S (2016) Max-pooling loss training of long short-term memory networks for small-footprint keyword spotting. In: 2016 IEEE spoken language technology workshop (SLT), pp 474–480. https://doi.org/10.1109/slt.2016.7846306
8. Arık SÖ, Kliegl M, Child R, Hestness J, Gibiansky A, Fougner C, Prenger R, Coates A (2017) Convolutional recurrent neural networks for small-footprint keyword spotting. Interspeech 2017. https://doi.org/10.21437/interspeech.2017-1737
9. Zhang Y, Suda N, Lai L, Chandra V (2017) Hello edge: keyword spotting on microcontrollers. https://doi.org/10.48550/arXiv.1711.07128

10. Shahnawaz M, Plebani E, Guaneri I, Pau D, Marcon M (2018) Studying the effects of feature extraction settings on the accuracy and memory requirements of neural networks for keyword spotting. In: 2018 IEEE 8th international conference on consumer electronics - Berlin (ICCE-Berlin), pp 1–6. https://doi.org/10.1109/ICCE-Berlin.2018.8576243
11. Wei Y, Gong Z, Yang S, Ye K, Wen Y (2021) EdgeCRNN: an edge-computing oriented model of acoustic feature enhancement for keyword spotting. J Ambient Intell HumIzed Comput 13(3):1525–1535. https://doi.org/10.1007/s12652-021-03022-1. Mar
12. Lea C, Flynn MD, Vidal R, Reiter A, GD Hager (2016) Temporal convolutional networks for action segmentation and detection. https://arxiv.org/abs/1611.05267
13. Warden P (2018) Speech commands: a dataset for limited-vocabulary speech recognition. https://doi.org/10.48550/arXiv.1804.03209
14. Smith LN (2017) Cyclical learning rates for training neural networks. In: 2017 IEEE winter conference on applications of computer vision (WACV), pp 464–472. https://doi.org/10.1109/wacv.2017.58
15. TensorFlow Lite | ML for Mobile and Edge Devices (2022) TensorFlow. https://www.TensorFlow.org/lite (Accessed 07 Mar 2022)
16. ONNX | Home (2022) Onnx.ai. https://onnx.ai/ (Accessed 07 Mar 2022)

Overcoming an Evasion Attack on a CNN Model in the MIMO-OFDM Wireless Communication Channel



Somayeh Komeylian, Christopher Paolini, and Mahasweta Sarkar

Abstract Implementation of deep learning (DL)-based algorithms for wireless communication channels can drastically enhance the discrepancy between features and interference. However, the performance of DL-based algorithms can be degraded by multiple different attackers, whereby a DL-based model is subjected to adversarial input to purposefully and maliciously fool or defeat the DL-based model. To overcome an evasion attack, this work contributes toward implementing a channel coding technique, specifically convolutional channel coding, for improving a CNN model's performance in a multi-input multi-output-orthogonal frequency-division multiplexing (MIMO-OFDM) wireless channel. In the preliminary stage, our CNN architecture has been trained by the mini-batch gradient descent algorithm, and we have achieved approximately 98% accuracy. In the presence of an evasion attack, we have deployed convolution channel coding for a CNN model in an MIMO-OFDM wireless channel. The effectiveness and performance of the model has been evaluated using the metrics BER, classification accuracy, physical layer security, and reliability, in the presence of an evasion attack. In the proposed model, a significant reduction in BERs has been obtained, and also a significant enhancement in wireless security in the presence of an evasion attack, which is accompanied by a robust and accurate wireless communication channel.

Keywords MIMO · OFDM · Convolutional coding · Deep learning

S. Komeylian · C. Paolini (✉) · M. Sarkar
Electrical and Computer Engineering, San Diego State University, San Diego, CA, USA
e-mail: paolini@engineering.sdsu.edu

S. Komeylian
e-mail: skomeylian81@gmail.com

M. Sarkar
e-mail: msarkar2@sdsu.edu

© The Author(s), under exclusive license to Springer Nature Singapore Pte Ltd. 2023

S. Chinara et al. (eds.), *Advances in Distributed Computing and Machine Learning*,
Lecture Notes in Networks and Systems 660,
https://doi.org/10.1007/978-981-99-1203-2_7

1 Introduction

Important technological advancements in the design and characterization of next-generation wireless communication systems have been focused on increasing network capacity and link throughput (bit rate). Performance enhancements in link throughput and network capacity can be increased by employing multiple antennas at both transmitter and receiver sides, especially over highly fading channels. Techniques of multiple-transmitter and multiple-receiver antennas, or MIMO, involve performing space-time coding (STC) and space division multiplexing (SDM) [1–4]. The STC technique allows for an improvement in the performance of wireless communications systems by coding over different transmitter branches. The SDM technique allows for achieving a higher throughput by transmitting data streams over independently different transmitter branches, simultaneously, and also at equal carrier frequencies. Although STC and SDM techniques well represent the practical implementation of the wireless communication channel, fundamental constraints, including the inadequacy of model accuracy, and lack of global optimality and scalability are present in MIMO-OFDM systems. Furthermore, the main disadvantage of the MIMO-OFDM technique consists in eliminating the effect of orthogonality from an undesired signal due to multi-path propagation. To overcome this limitation, channel coding techniques [1–4] have been employed for wireless digital communications to minimize errors in data transmission.

The Shannon capacity theorem, Eq. 1, describes the maximum amount of data, or data capacity, which can be transmitted or received over a noisy wireless channel (or other media),

$$C = B\, log_2\left(1 + \frac{S}{N}\right), \tag{1}$$

where C refers to the channel capacity in bits per second, B is transmission bandwidth in Hertz, and S and N are measures of data (or signal) and noise power, respectively, in Watts. In this work, to boost the channel coding performance, we have proposed a deep learning technique using a CNN model in a MIMO-OFDM wireless communication channel [5, 6]. DL-based techniques can be leveraged to train a mapping model between the omnidirectional radiation pattern of antennas and a wireless communication channel.

The main advantage of deep learning techniques over conventional machine learning algorithms, in the framework of wireless security, is its capability in capturing physical signal characteristics efficiently. Indeed, optimizing hyperparameters relies on enhancing the deep learning performance along with the wireless security requirements. This practical interest has been further fostered and enhanced by a proper combination of the MIMO-OFDM technique and DL-based techniques. This combination allows a further increase in the discrepancy between training and testing datasets, ensuring that a practical implementation is robust against interference, fading, noise, and adversarial learning [7].

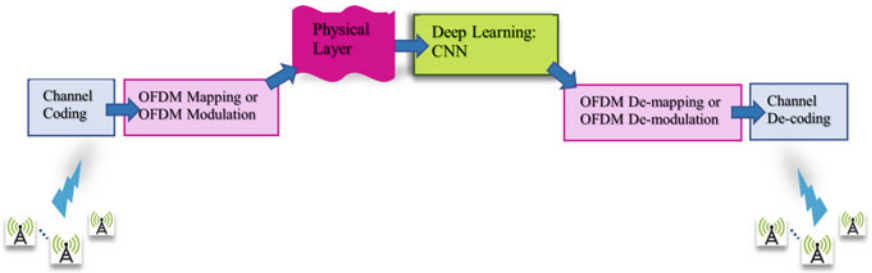
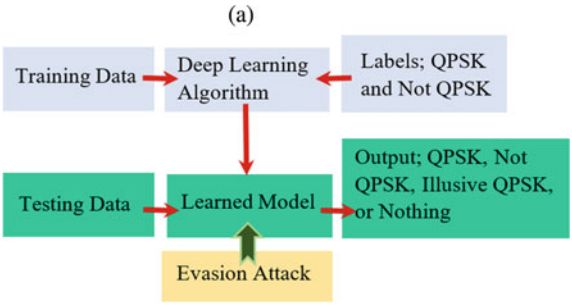


Fig. 1 Schematic of the proposed wireless communication channel. It is assumed an identical number of antennas is placed in the input and output, which, in this work, is 5

Fig. 2 **a** Demonstration of the evasion attack to the CNN model. **b** MATLAB codes for simulating the evasion attack on the proposed deep learning-based model



(b)

MATLAB Codes for Evasion Attack Implementation

```
EvsionAtt = phased.BarrageJammer ('ERP',.0001);
EvsionAtt.SamplesPerFrameSource='Input port',
EvsionAttack= step (EvsionAtt, length(rxWaveform));
rxWaveform=rxWaveform + noise+EvsionAttack;
```

However, the performance of DL-based techniques can still be significantly degraded by several different attack models, including evasion, exploratory (inference), poisoning (causative), and Trojan attacks, as discussed in [8–23], and thereby become vulnerable. Most of the studies and implementations have strongly considered the evasion attack model in the DL-based framework due to its greater harmful effects. In this work, we have focused on overcoming the destructive effect of an evasion attack on the CNN model by performing MIMO-OFDM and channel coding techniques, as demonstrated in Fig. 1.

As demonstrated in Fig. 2, an evasion attack manipulates the testing dataset such that the classification model predicts an incorrect application or removes an applica-

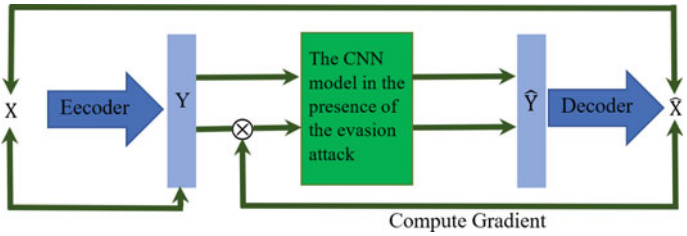


Fig. 3 The channel coding implementation for the CNN in the presence of the evasion attack

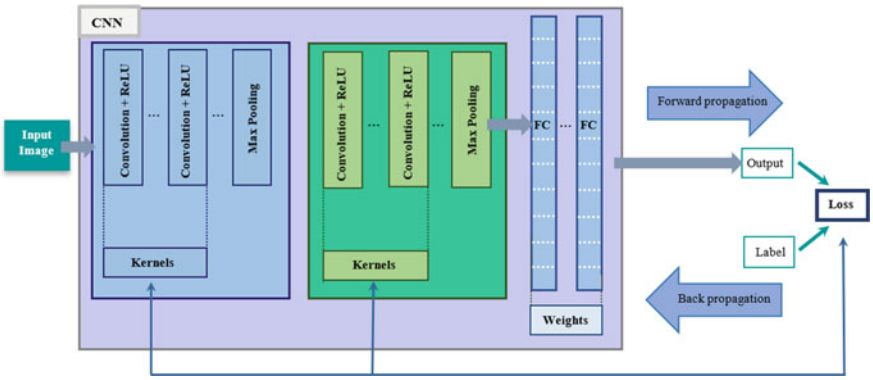


Fig. 4 Our proposed CNN architecture [25, 26]

tion. In other words, in the classification decision process, an evasion attack, with a minimum evasion cost, can maximize the total loss of the classification.

In this work, we have employed the open-source dataset of RML2016.10A [24]. However, we disabled all modulations except for BPSK, so this work can focus only on evaluating the implementation of MIMO-OFDM and channel coding for overcoming an evasion attack on the CNN performance and functionality.

Furthermore, the CNN deployment can significantly improve a lack of a parallelization capability of the channel coding especially in the MIMO-OFDM wireless channel, which involves transferring a high rate of data, and processing a large volume of data, as demonstrated in Fig. 3 (Figs. 4 and 5).

- Advantages of the MIMO-OFDM and channel coding for the CNN model: We have significantly increased the classification accuracy and reduced BERs by deploying the channel coding and MIMO-OFDM for the CNN optimization model even in the presence of the evasion attack.
- The CNN model can overcome the challenging requirements due to the fact that wireless communications involve high-computational complexities and very large volumes of data.

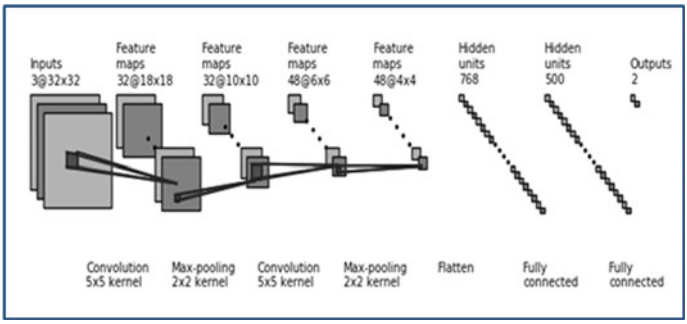


Fig. 5 The proposed CNN architecture for MIMO-OFDM wireless communication [1]

- Error detection or correction coding in the channel coding technique allows for detecting and correcting bit errors in a wireless communication channel, thus enhancing the channel's robustness against attackers.
- An effectiveness of the CNN model for channel coding: on the channel coding (encoding) side, the mutual information is maximized between the codeword and data, while the channel decoding (decoder) is performed within an adversarial framework. However, the performance of the channel coding degrades significantly due to a high decoding complexity and incapability to parallelize the computing processes. Hence, the CNN implementation presents a highly parallelized structure to reduce the computational complexity of channel coding.

A contribution of this work has only focused on mitigating the effect of the evasion attack on the CNN performance by implementing the channel coding and MIMO-OFDM techniques.

Consequently, in addition to the introduction section, this work is organized into three sections: Sect. 2 presents a full evaluation for the proposed CNN model. Section 3 consists of evaluating the effect of an evasion attack on a CNN model together with the channel coding technique, compared to a model without any channel coding technique in the context of the MIMO-OFDM. We have comprehensively evaluated the performance of the integrated model in terms of BERs, automatic modulation classification (AMC), physical layer security and reliability, and success rate of the evasion attack. The conclusion of this work has been presented in Sect. 4.

2 CNN Implementation for a Wireless Communication Channel

Important technological advancements in the design and characterization of next-generation wireless communication systems have been strongly focused on increasing network capacity and link throughput (bit rate). CNN optimization methods

consist of a class of deep learning methods, which has potential applications in the area of image processing, signal processing, and computer vision tasks. A contribution of this work is to deploy a CNN model to improve wireless security in the presence of the evasion attack.

Feature extraction allows for reducing the amount of redundant data from the dataset. However, manual feature extraction from a wireless signal is a time-consuming task. CNN techniques bring the merit of extracting important features automatically for signal processing without any human supervision. Furthermore, weight sharing, as another advantage of CNN models, allows for reducing the number of parameters for the purpose of extracting robust features. Hence, CNNs [12, 25–28] provide a trade-off between accuracy and compression.

The discussion in this section has focused on evaluating the implementation of an unsupervised CNN for enhancing wireless security in the presence of an evasion attack.

2.1 Details of the Deep CNN Architecture

A CNN architecture is composed of stacking several building blocks, such as (1) convolutional layers, (2) pooling layers, and (3) fully connected (FC) layers [1, 2]. Through a forward propagation procedure, a model is trained using the training dataset and kernels, and a loss value is calculated. Convolutional kernels and weights are updated according to the estimated loss value through the backpropagation procedure using a gradient descent algorithm.

Generally, a convolution layer in a CNN architecture, including convolution operations and activation functions, allows for performing automatic feature extraction [12, 25–28]. In this work, an input signal consists of 11×1 arrays, as reported in Table 1. A pooling layer allows for a down-sampling operation. It is worth mentioning that the pooling layers do not consist of any trainable parameters. Convolutional filter size, stride, and padding are considered as hyperparameters of pooling operations. Through a down-sampling procedure, the height and width of layers change, but their depth remains unchanged [12, 25–28]. Fully connected (FC) layers map the final convolution or pooling layer into one-dimensional vectors, as shown in Fig. 2.

The proposed CNN was trained by the mini-batch gradient descent method, in which multiple training examples (but less than the whole training dataset) with no overlap between them are entered into the training process at every pass [12, 25–28]. CNN parameter updating is performed by gradient descent computations on each mini-batch. We have used a mini-batch with a size of 32.

Table 1 The input, hidden, and output layers of our CNN architecture

Layer	11 × 1 layer array with layers		
	Layer name	Type	Dimension
1	signal input	Signal input	612×14×1 signals
2	$conv_1$	Convolution	64 9×9×1 convolutions
3	$relu_1$	ReLU	ReLU
4	$conv_2$	Convolution	64 5×5×64 convolutions
5	$relu_2$	ReLU	ReLU
6	$conv_3$	Convolution	64 5×5×64 convolutions
7	$relu_3$	ReLU	ReLU
8	$conv_4$	Convolution	32 5×5×64 convolutions
9	$relu_4$	ReLU	ReLU
10	$conv_5$	Convolution	1 5×5×32 convolutions
11	Regression output	Regression output	Mean-squared-error with response "Response"

2.2 Performance Evaluation of the CNN Architecture

The discussion in the previous section has focused on demonstrating the proposed CNN architecture. In this section, we have reported the performance evaluation of the deep CNN model using the metrics of accuracy, loss, and root mean squared error (RMSE) (Fig. 6).

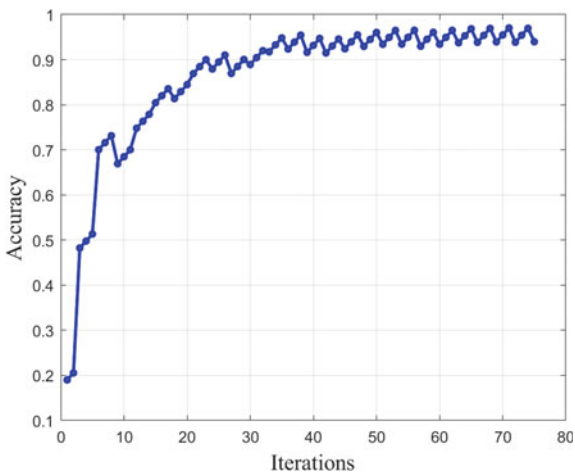


Fig. 6 Accuracy of the proposed CNN, Fig. 2, trained by the mini-batch gradient descent with a size of 32

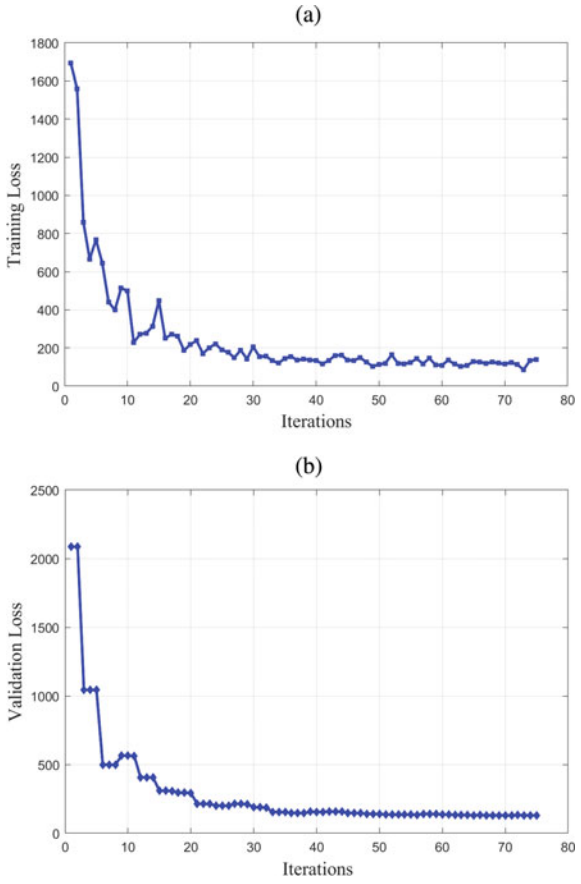


Fig. 7 Training (**a**) and validation (**b**) loss versus iterations

The loss function estimates the loss of the obtained model such that weights can be updated to decrease the loss on the next pass.

$$Loss\left(Y_{pre}, Y_{true(orlabel)}\right) = -\frac{1}{N}\sum_{onN}\left(\left(Y_{pre} log Y_{label}\right) + \left(1 - Y_{pre}\right) log\left(1 - Y_{label}\right)\right), \quad (2)$$

where N is equal to the number of classes. Figure 7a, b shows losses of the proposed CNN, Fig. 2, trained by the mini-batch gradient descent with a size of 32.

RMSE refers to the standard deviation of the residuals in the same units. Residuals show a measure of how far apart the regression line data points are. In other words, RMSE represents a concentration of the data around the line of best fit. Figure 8 (top) and (bottom) shows Root Mean Square Errors (RMSEs) of the proposed CNN, Fig. 2, trained by the mini-batch gradient descent with a size of 32, for the training dataset and the validation dataset, respectively.

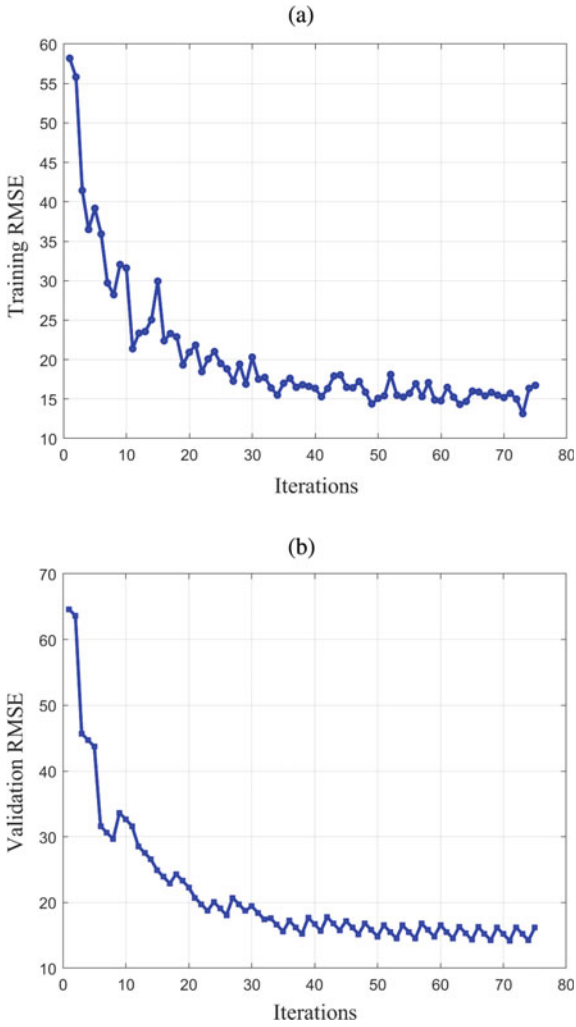


Fig. 8 Root mean square error for training (a) and validation (b) versus iterations

3 Evaluation of the Proposed Configuration to Variations of the Wireless Channel

In this section, Fig. 1 has been deployed using the deep CNN model in the previous section. The effectiveness and performance of the proposed wireless channel have been evaluated in the presence of the evasion attack for the combination of MIMO-OFDM and the deep CNN technique assisted without, and with, convolutional channel coding.

3.1 Estimation of BERs

Figure 6 demonstrates that with deploying channel coding for the combination of the MIMO-OFDM and the CNN model, a drastic reduction occurs in BERs of the wireless channel in the presence of evasion attack, even at low values of SNRs, which is accompanied by a significant enhancement in the channel performance. In other words, since the channel coding technique could significantly reduce BERs, the wireless channel becomes robust against attackers (Figs. 9 and 10).

3.2 Automatic Modulation Classification

In each wireless channel model, modulated signals are subjected to AWGN (additive white Gaussian noise), whose power is estimated by $\frac{E_s}{N_0}$ and varied from 0 dB to 20 dB uniformly. Hence, the evasion attack to noise ratio is defined by the following equation:

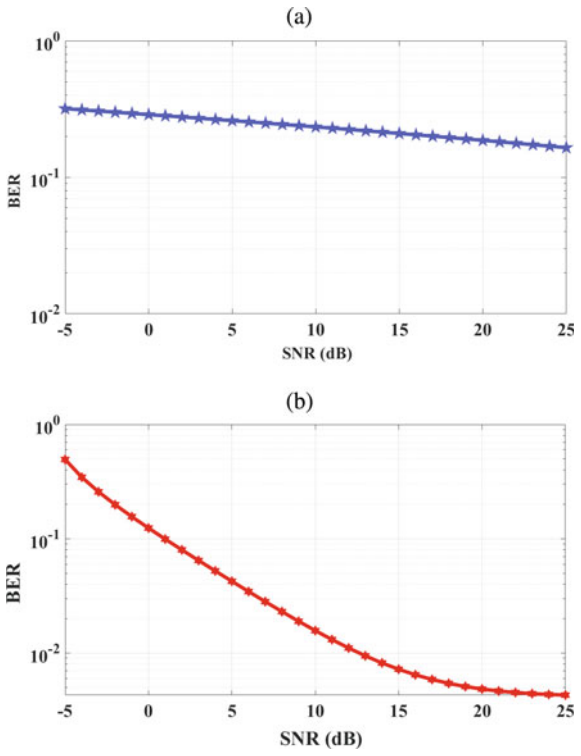


Fig. 9 Variation of BERs in terms of SNRs in the wireless channel with the techniques of the combination of the MIMO-OFDM and deep CNN optimization in the presence of evasion attack (a) and with assisted channel coding (b)

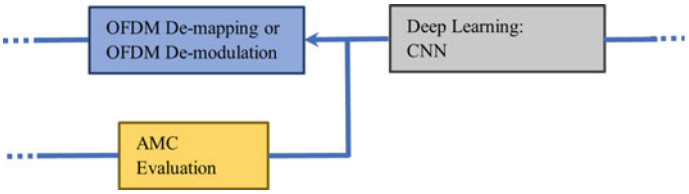


Fig. 10 Demonstration of the AMC evaluation, consistent with Fig. 1

$$\frac{E_e}{N_0} = \frac{E_s/N_0}{E_s/E_e} = \frac{E_s}{N_0}(dB) - \frac{E_s}{E_e}(dB). \tag{3}$$

Although an increase in the input size of the proposed DNN can drastically improve the accuracy in non-adversarial scenarios, on the other hand, the proposed adversarial DNN becomes more susceptible to the values of $\frac{E_s}{E_e}$. The question of how the presence of the evasion attack effects on the channel functionality in Fig. 1 has been rigorously addressed by the concept of automatic modulation classification. Automatic modulation classification (or recognition) refers to an evaluation metric for identifying the modulation type of the received signals, as well as for performing appropriate demodulations, as demonstrated in Fig. 7 consistent with Fig. 1. Since features are preliminary trained by the CNN algorithm, the received signal can be classified directly without any feature extraction. In other words, the AMC technique using the CNN algorithm can automatically learn features. In order to evaluate the effectiveness and performance of the adversarial deep learning on the raw-IQ-based AMC, we have obtained a classification accuracy in terms of $\frac{E_s}{N_0}$ on the dataset trained by the CNN of the previous section. A source modulation class is assumed to be BPSK.

Two features can be outlined in Fig. 11: (1) approximately in the lower powers of AWGNs, the receiver cannot recognize the modulation type of the received signal and (2) with performing channel coding, the accuracy of classifying the received signal increases. Signal classification systems do not address any information about the transmission procedure. Hence, we have to obtain a baseband for each classification procedure. This may turn out to have errors. S. C. Hauser et. al in [29] verified that the effect of errors appears in the center frequency estimation, and thereby results in the frequency offset signals. Furthermore, they have validated that the raw-IQ-based AMC is only generalized over the training dataset and thereby the frequency offsets and the classification accuracy still can be degraded by other sources. In order to achieve full accuracy, the concept of the estimation frequency offset should be extended for other negative effects in the open wireless channels. In this work, we have considered additional frequency offsets of ± 0.1 during the training procedure due to the air affect in the wireless channel.

The implementation results of the classification accuracy in terms of frequency offsets for the proposed wireless channel have been rigorously reported in Fig. 12. The effectiveness of the proposed wireless channel is noticeable in Fig. 12, where

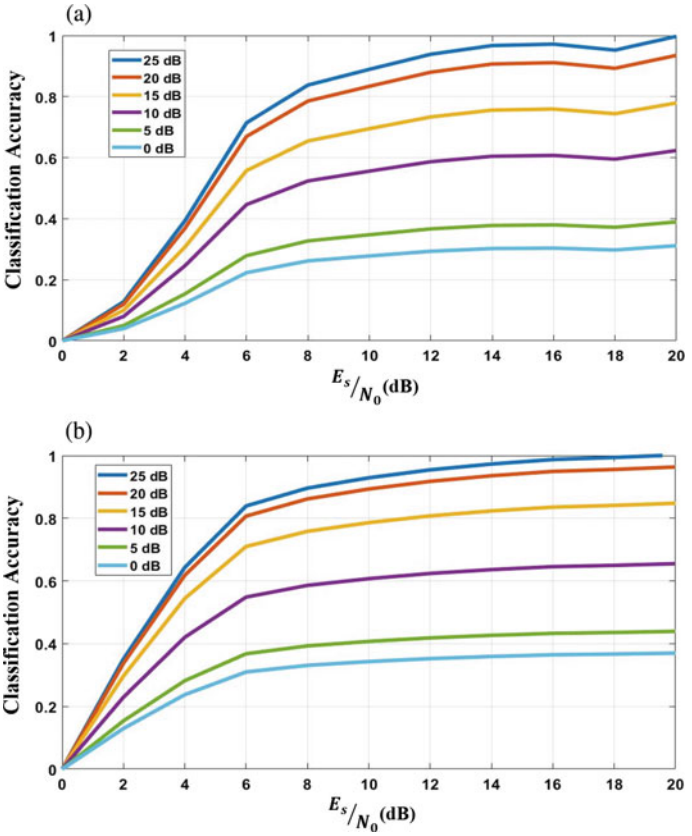


Fig. 11 Classification accuracy of the received signal in terms of the AWGN power of $\frac{E_s}{N_0}$, where E_s refers to the average energy per symbol of the modulated signal, for (top) the combination of the MIMO-OFDM and deep CNN optimization in the presence of the evasion attack, and (bottom) and channel coding assisted with the combination of the MIMO-OFDM and deep CNN optimization in the presence of evasion attack. The source modulation class is supposed to be BPSK. Furthermore, the legend of the figure shows the variation of $\frac{E_s}{E_e}$, where E_e refers to the average energy of the evasion attack

$E_s/E_e = 0$ (dB) = 1 or $E_s = E_e$, the classification accuracy is still identical to 0.4. In other words, in the worst scenario, in which the average energy per symbol of the modulation signal and the evasion attack are equal, the proposed wireless channel, consistent with Fig. 1, still exhibits a good classification accuracy of 0.4 for reliable transmission bandwidth. Following the framework of the signal classification system, in which we aim to estimate the frequency bins of the present signals as well as the start and stop times of the transmission, we have demonstrated the classification accuracy in terms of time offset in samples in Fig. 13. Sample time offsets allow for producing discrete deep learning examples by a rectangular windowing function as well as avoiding any alignment between crafting adversarial perturbation and signal

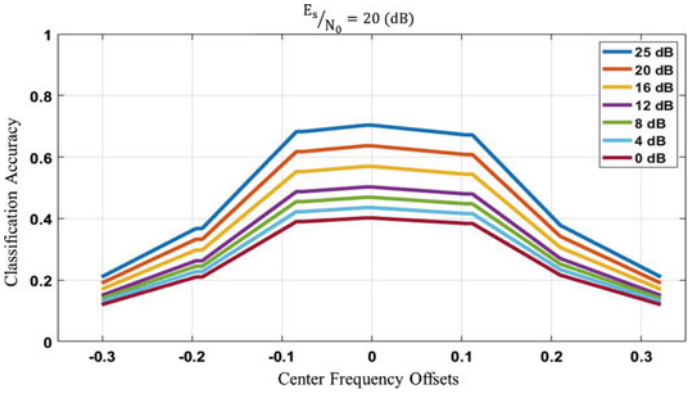


Fig. 12 Classification accuracy versus center frequency offset for investigating the effectiveness of the technique of channel coding assisted with the combination of the MIMO-OFDM and deep CNN optimization in the presence of evasion attack for the different ratio of $\frac{E_s}{E_e} = 0$, 4, 8, 12, 16, 20, and 25 (dB), where E_e refers to the average energy of the evasion attack. The dataset has been trained using the proposed CNN of the previous section. The source modulation class is supposed to be BPSK

classifications. An alternative way to estimate the stop and start times of transmission consists of employing an energy detection algorithm. The concept of energy detection allows for measuring the energy of N received samples or, in other words, it takes an average of the squared root of FFT over the N samples [30–35]. The obtained energy is compared to the threshold for computing the sensing decision. A high threshold refers to the presence of a lag in estimating start time. On the other hand, a low threshold exhibits the presence of a high false alarm rate. Generally, any shift in the starting index during the procedure of either non-overlapping (non-consecutive) windowing or slicing the signal for evaluating the signal classification performance can be modeled as the time offset. Analogously, Fig. 13 reports that the classification accuracy has resulted from the proposed channel in Fig. 1, in terms of the time offsets for the different signal-to-evasion ratios.

3.3 Physical Layer Security and Reliability

Inherent characteristics of wireless communication channels such as superposition, broadcast, and openness bring challenges and difficulties for ensuring secure and reliable communications. The ubiquitous presence and growing capacity of wireless communication can significantly worsen privacy and security of users [1–4]. This section has focused on investigating the effectiveness and performance of the proposed technique in wireless channel security using two metrics of security rate and success rate. The simulation results of Figs. 11 and 12 have been obtained assuming

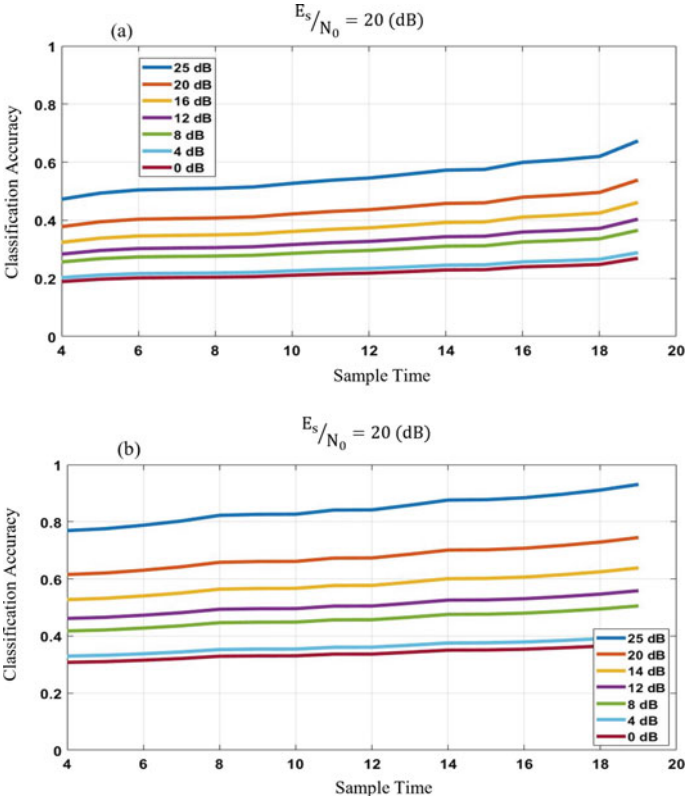


Fig. 13 Classification accuracy versus time windowing offsets for investigating the effectiveness of the technique of channel coding assisted with the combination of the MIMO-OFDM and deep CNN optimization in the presence of evasion attack for the different ratio of $E_s/E_e = 0, 4, 8, 12, 16, 20$, and 25 (dB), where E_e refers to the average energy of the evasion attack. The dataset has been trained using the proposed CNN in the previous section. The source modulation class is supposed to be BPSK. (a) without the coding channel, and (b) with the coding channel

that the evasion attack is equivalent to eavesdropping in the wiretap channel [1–4]. Figure 14 reports the significant effectiveness of the proposed CNN algorithm in increasing wireless security. Compared to Fig. 14 (bottom), we have achieved a high security rate in the MIMO-OFDM wireless communication, in Fig. 14 (top), with performing the deep CNN algorithm, even in the presence of the lower values of SNRs.

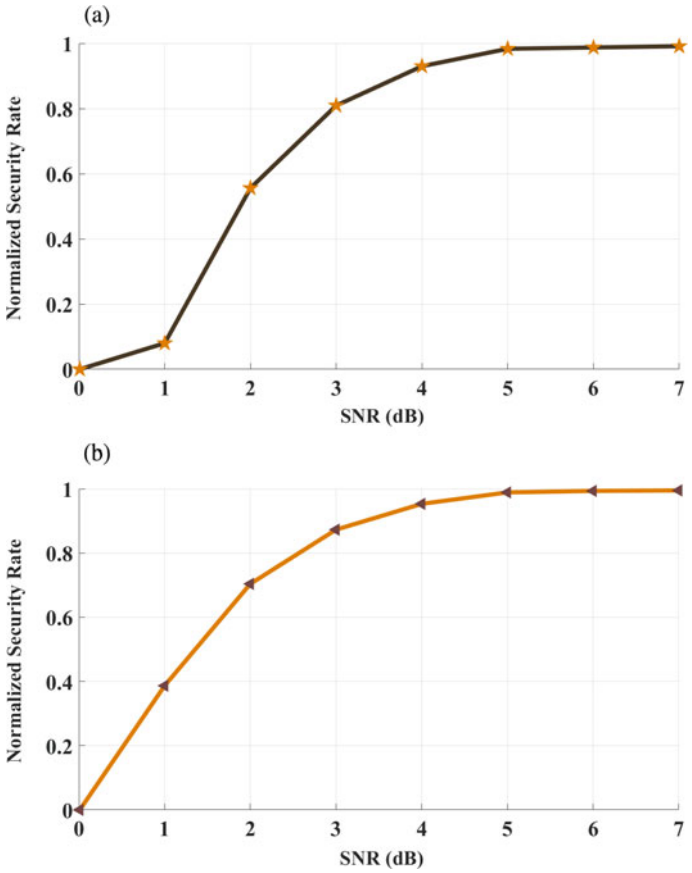


Fig. 14 Normalized security rate of the wireless channel in terms of SNRs, (a) the combination of the MIMO-OFDM and deep CNN optimization in the presence of evasion attack, and (b) and channel coding assisted with the combination of the MIMO-OFDM and deep CNN optimization in the presence of evasion attack

3.4 Success Rate of the Evasion Attack

The performance of the given evasion attacker has been evaluated using the concept of the success rate in Fig. 15. The success rate of the evasion attacker becomes zero in the lower SNRs with performing the deep CNN technique compared to the scenario without performing the CNN technique. In addition, a further effectiveness of the proposed CNN architecture can be accompanied by an increase in SNRs. In other words, the success rate of the given evasion attacker decreases significantly when SNRs increase.

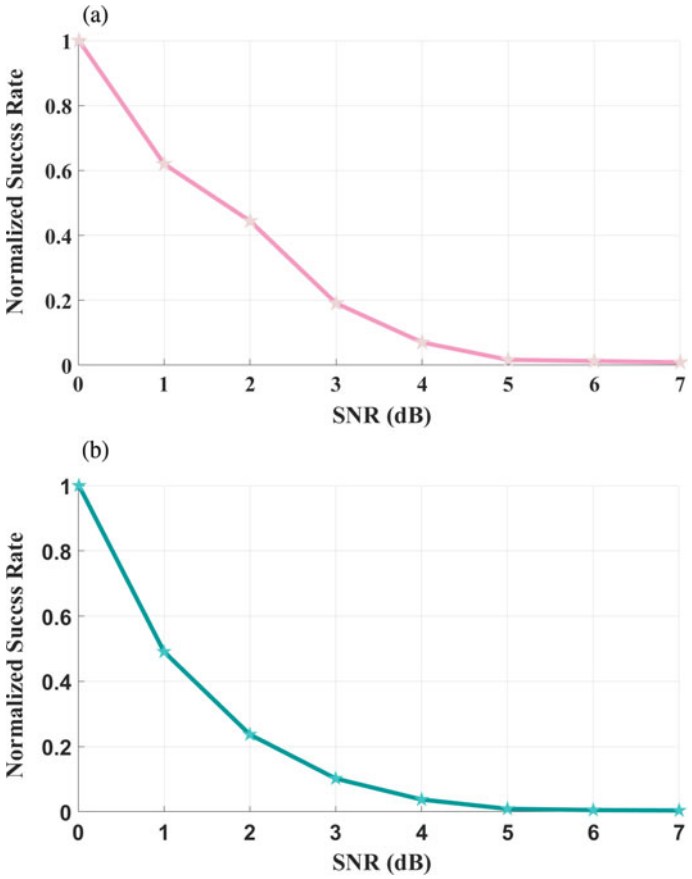


Fig. 15 Variation of the success rate of the given evasion attacker versus SNR (in dB), (a) the combination of the MIMO-OFDM and deep CNN optimization in the presence of evasion attack, and (b) and channel coding assisted with the combination of the MIMO-OFDM and deep CNN optimization in the presence of evasion attack

4 Conclusion

To conclude, in this work, we have proposed and implemented the deep CNN for the MIMO-OFDM wireless channel. Approximately, 98% accuracy has been obtained for the deep CNN architecture. In an ideal scenario, in which there is no inference and no attackers, the CNN model could drastically improve the performance of the wireless channel in terms of reliability and robustness. However, in general, it may be challenging to implement the CNN model for wireless channels due to the presence of several different attackers. Channel coding assisted with the combination

of the MIMO-OFDM and the CNN model could strongly address the aforementioned challenges. Indeed, this technique has significantly enhanced the robustness and security of the wireless channel in the presence of the evasion attack.

References

1. Komeylian S (2021) Deep neural network modeling of different antenna arrays; analysis, evaluation, and application. IEEE Can J Electr Comput Eng 44(3):261–274
2. Bianco S, Cadene R, Celona L, Napoletano P (2018) Benchmark analysis of representative deep neural network architectures. IEEE Access 6:64 270–64 277. https://doi.org/10.1109/access.2018.2877890
3. Agarwal A, Mehta S (2018) Development of mimo-ofdm system and forward error correction techniques since 2000s. Photonic Netw Commun 35:02
4. Chen Z, Bai P, Li Q, Hu S, Huang D, Li Y, Gao Y, Li X (2017) A novel joint optimization of downlink transmission using physical layer security in cooperative 5g wireless networks. Int J Distrib Sens Netw 13
5. Komeylian S, Komeylian S (2020) Deploying an ofdm physical layer security with high rate data for 5g wireless networks. In: 2020 IEEE Canadian conference on electrical and computer engineering (CCECE), pp 1–7
6. Komeylian S, Paolini C, Sarkar M (2023) Beamforming technique for improving physical layer security in a mimo-ofdm wireless channel. In: International conference on advances in distributed computing and machine learning (ICADCML) 2023, India, pp 1–9
7. Gruber T, Cammerer S, Hoydis J, ten Brink S (2017) On deep learning-based channel decoding. CoRR . arXiv:abs/1701.07738
8. Sadeghi M, Larsson EG (2019) Adversarial attacks on deep-learning based radio signal classification. IEEE Wirel Commun Lett 8:213–216
9. Flowers B, Buehrer RM, Headley WC (2020) Evaluating adversarial evasion attacks in the context of wireless communications. IEEE Trans Inf Forensics Secur 15:1102–1113
10. Hameed M, György A, Gündüz D (2019) Communication without interception: Defense against deep-learning-based modulation detection. arXiv:1902.10674
11. Flowers B, Buehrer RM, Headley WC (2019) Communications aware adversarial residual networks for over the air evasion attacks. In: MILCOM 2019 - 2019 IEEE military communications conference (MILCOM), pp 133–140
12. Kim B, Sagduyu Y, Davaslioglu K, Erpek T, Ulukus S (2020) Over-the-air adversarial attacks on deep learning based modulation classifier over wireless channels. In: Conference: 2020 54th annual conference on information sciences and systems (CISS), pp 1–6
13. Kokalj-Filipovic S, Miller R (2019) Adversarial examples in rf deep learning: detection of the attack and its physical robustness. arXiv:abs/1902.06044
14. Kokalj-Filipovic S, Miller R, Chang N, Lau CL (2019) Mitigation of adversarial examples in rf deep classifiers utilizing autoencoder pre-training. arXiv:abs/1902.08034
15. Shi Y, Erpek T, Sagduyu YE, Li JH (2019) Spectrum data poisoning with adversarial deep learning. arXiv:abs/1901.09247
16. Sagduyu YE, Shi Y, Erpek T (2019) Iot network security from the perspective of adversarial deep learning. arXiv:abs/1906.00076
17. Sagduyu YE, Shi Y, Erpek T (2021) Adversarial deep learning for over-the-air spectrum poisoning attacks. IEEE Trans Mob Comput 20(2):306–319
18. Sadeghi M, Larsson EG (2019) Physical adversarial attacks against end-to-end autoencoder communication systems. arXiv:abs/1902.08391
19. O'Shea T, Hoydis J (2017) An introduction to deep learning for the physical layer. IEEE Trans Cogn Commun Netw 3(4):563–575

20. Shi Y, Sagduyu Y, Grushin A (2017) How to steal a machine learning classifier with deep learning. In: 2017 IEEE International Symposium on Technologies for Homeland Security (HST), pp 1–5
21. Shi Y, Sagduyu YE, Erpek T, Davaslioglu K, Lu Z, Li JH (2018) Adversarial deep learning for cognitive radio security: jamming attack and defense strategies. In: 2018 IEEE international conference on communications workshops (ICC Workshops), pp 1–6
22. Erpek T, Sagduyu YE, Shi Y (2018) Deep learning for launching and mitigating wireless jamming attacks. arXiv:abs/1807.02567
23. Shi Y, Sagduyu YE, Davaslioglu K, Li JH (2018) Active deep learning attacks under strict rate limitations for online api calls. arXiv:abs/1811.01811
24. O'Shea T, West N (2016) Radio machine learning dataset generation with gnu radio. Proc GNU Radio Conf 1(1). https://pubs.gnuradio.org/index.php/grcon/article/view/11
25. Yamashita R, Nishio M, Kinh Gian Do R, Togashi K (2018) Convolutional neural networks: an overview and application in radiology. Insights Imaging 9
26. Dong C, Loy CC, Tang X (2016) Accelerating the super-resolution convolutional neural network. arXiv:abs/1608.00367
27. Albawi S, Mohammed TA, Al-Zawi S (2017) Understanding of a convolutional neural network. In: 2017 International conference on engineering and technology (ICET), pp 1–6
28. Duan S, Zheng H, Liu J (2019) A novel classification method for flutter signals based on the cnn and stft. Int J Aerosp Eng 2019:1–8
29. Hauser SC, Headley WC, Michaels AJ (2017) Signal detection effects on deep neural networks utilizing raw iq for modulation classification. In: MILCOM 2017 - 2017 IEEE military communications conference (MILCOM), pp 121–127
30. Anurag AR, Lakha B (2016) Design and analysis of spectrum sensing in cognitive radio based on energy detection. In: 2016 international conference on signal and information processing (IConSIP), pp 1–5
31. Khan RT, Islam MI, Zaman S, Amin MR (2016) Comparison of cyclostationary and energy detection in cognitive radio network. In: 2016 international workshop on computational intelligence (IWCI), pp 165–168
32. Manesh MR, Apu MS, Kaabouch N, Hu W-C (2016) Performance evaluation of spectrum sensing techniques for cognitive radio systems. In: 2016 IEEE 7th annual ubiquitous computing. electronics and mobile communication conference (UEMCON), pp 1–7
33. Tandra R, Sahai A (2008) Snr walls for signal detection. IEEE J Sel Top Signal Process 2(1):4–17
34. Giweli N, Shahrestani S (2016) Selection of spectrum sensing method to enhance qos in cognitive radio networks. Int J Wirel Mob Netw 8:02
35. Komeylian S, Paolini C (2022) Implementation of a three-class classification ls-svm model for the hybrid antenna array with bowtie elements in the adaptive beamforming application. arXiv:abs/2210.00317

An Improved Whale Optimization Algorithm for Optimal Placement of Edge Server



Rajalakshmi Shenbaga Moorthy, K. S. Arikumar, and B. Sahaya Beni Prathiba

Abstract The rapid growth of the Internet of Things makes a vast number of smart devices perform operations via the Internet which leads to depletion of resources like network bandwidth and also increases latency. Edge computing offers a promising solution to the traditional way of centralized computing, which places the operation at the edge close to the devices. As the operations are close to the devices, there will be a significant improvement in latency and energy consumption. But the question of optimal placement of edge servers is potential research that needs to be addressed carefully. In this research, the authors propose Edge Server Placement (ESP) algorithm based on the Improved Whale Optimization Algorithm (IWOA). The traditional Whale Optimization Algorithm (WOA) falls in local optima which leads to premature convergence. To overcome that, improvement is made in WOA for effective convergence toward a global optimal solution. Thus, the proposed ESP based on IWOA provides an optimal solution for the placement of edge servers thereby minimizing delay and energy consumption. The proposed ESP is compared with WOA and Particle Swarm Optimization (PSO) and the experimental results reveal that the proposed ESP is better than traditional algorithms for optimal placement of edge servers.

Keywords Edge server · Edge computing · Whale optimization algorithm · Particle swarm optimization algorithm

R. S. Moorthy (✉)
Sri Ramachandra Institute of Higher Education and Research, Chennai, Tamil Nadu, India
e-mail: srajiresearch@gmail.com

K. S. Arikumar
VIT–AP University, Amaravati, India

B. S. B. Prathiba
Vellore Institute of Technology (VIT) Chennai Campus, Chennai, India

© The Author(s), under exclusive license to Springer Nature Singapore Pte Ltd. 2023

S. Chinara et al. (eds.), *Advances in Distributed Computing and Machine Learning*,
Lecture Notes in Networks and Systems 660,
https://doi.org/10.1007/978-981-99-1203-2_8

1 Introduction

Edge computing is a scalable and promising solution for the potential problems of centralized cloud computing. In recent days, a vast amount of data gets generated by IoT devices which when processed by cloud servers incurs huge delays and also consumes more bandwidth. Also, when the data is processed in the cloud server, the physical location of the server, monitoring the status of the server, and handling the data remotely are challenging tasks in this IoT era. Edge computing introduces a computational layer over the network devices. The edge devices situated between the cloud layer and the device layer perhaps reduce the burden of transferring the data to the cloud for processing. Various edge computing platforms include (i) cloudlet—a small-scale data center situated at the edge to provide resources for computation with minimum latency, particularly for mobile applications [2]. Mobile Edge Computing (MEC) intends to keep resources nearer to 4G and 5G Radio Access Networks. Further, it brings the capabilities of cloud computing to the edge which provides an environment with low latency [3]. (iii) Fog Computing is an architecture where the computation and storage of data happen in edge devices [4].

As the edge devices are nearer to the IoT devices they are having the potential to process the applications which require minimum latency. Also, the processing of data in edge devices provides privacy for user data and also reduces the burden of transferring the data to the cloud [1]. The edge computing environment aims to move the computation to edge devices thereby minimizing the congestion and delay which leads to unexpected results. The edge server placement and allocation are considered challenging tasks in the edge computing environment. These two problems are interrelated as they may incur latency while processing the request. The problem of edge server placement should be addressed when the number of user devices increases. Various constraints imposed by the user devices including minimum latency and energy consumption should be considered for optimally placing the edge server across the geographical location. The constraints for placing the edge server in the optimal location include

- Edge server placement has a serious impact on the latency for processing the request from the device. The distance between the edge server and the device originating the request has a performance impact on the Quality of Service.
- Edge server placement also plays a major role in consuming the energy of any device originating the request. Poor placement of edge servers may increase energy consumption thereby reducing user satisfaction.

This research work focuses on the placement of the edge server to the device which is an NP-hard problem. A novel edge server placement algorithm, called ESP, has been designed based on the Improved Whale Optimization Algorithm. The proposed ESP can be able to allocate the devices to the edge server thereby minimizing the latency and energy consumption. The proposed ESP includes exploration and exploitation which is governed by the convergence factor. The convergence factor

is tuned in such a way for providing optimal allocation of the edge server. The novelty of the research work includes

- Designing optimal edge server placement algorithm based on Improved Whale Optimization Algorithm.
- Comparing the proposed ESP with WOA and PSO in terms of delay and energy consumption.
- Experimentation is carried out by increasing the number of edge servers to 15, 20, 25, and 30.

The remaining of this paper is organized as Sect. 1 describes the introduction of edge computing and the need for edge server placement. Section 2 briefly describes the various mechanisms available for edge server placement. Section 3 describes the system design of the proposed method. Section 4 describes the working of the proposed ESP algorithm. Section 5 describes the experimental results. Finally, Sect. 6 concludes the work with future scope.

2 Related Work

Edge Server Placement Problem was formulated as a multi-objective optimization problem that was solved by mixed integer programming. The objectives are taken into account where the workload of the edge server and the delay between the edge user and mobile user. Experimentation was carried out on Shanghai Telecom's Base Station and the designed method was compared with other algorithms like K-Means, and Top-K, and the designed technique outperforms in terms of delay and workload balancing [5]. An algorithm called PACK was designed for edge server placement which minimizes the distance between edge server and access points and also performed workload balancing. The algorithm was evaluated with high- and low-capacity edge servers [1]. The problem of placement of edge server was solved by EPMOSO which is the integration of Genetic Algorithm (GA) and Particle Swarm Optimization (PSO). The benefits are given by GA which is finding the global optimal solution and the benefit of PSO which is faster convergence is taken into account for finding the optimal location for the edge server. The EPMOSO was compared with GA and PSO in terms of delay, energy consumption, variance in load, and also server utilization [6]. The Profit Model had been developed considering access delay and energy consumption. PSO algorithm had been used for optimizing the profit by introducing q-value. Shanghai Telecom base station dataset had been used for measuring the performance of designed QPSO with other algorithms like PSO, Greedy in terms of delay, and energy consumption [7]. The combinatorial optimization problem of edge server placement was solved by designing an edge server placement algorithm based on a genetic algorithm (EPG). Improvement was made in the traditional genetic algorithm by recording the best gene at each iteration. The designed EPG was evaluated at the Shanghai Telecom base station [8]. Edge server placement was done using a metaheuristic algorithm (ESH-GM) designed with the aid to access the

services remotely with minimum delay and high bandwidth. K-means algorithm is used with ant colony optimization algorithm together with a pheromone feedback mechanism and the taboo table for improving the convergence speed. The ESH-GM was evaluated on the Shanghai Telecon base station dataset based on the parameters such as latency and load balancing without compromising the quality of service [9]. Integer Linear Programming was used to solve the edge server placement by using Column Generation (CG) technique. The designed method was compared with Farthest Node Shortest Path (FNSP) in terms of balancing the network traffic and it was observed that CG was better than FNSP [10]. Edge Servers are prone to various attacks and thus it might lead to failure. The users when connected with the failed edge server require to access the cloud for performing the task. To overcome this, Robustness-oriented k Edge Server Placement (RkESP) Integer Linear Programming-based method called OPT and Approx based on O(k) approximate approach was designed. The experimentation was carried out on the EUA dataset which consists of details of the city of Melbourne, Australia [11]. The problem of deploying the edge servers cost-effectively to minimize the number of edge servers without violating the delay was addressed by using a greedy approach. Greedy-based Minimum Extended Dominating Set Algorithm (GMEDS) was designed to place the edge server in an appropriate location. Further, the Simulated Annealing-based Minimum Extended Dominating Set Algorithm (SAMEDS) was used to improve GMEDS from falling into local optima [12]. Energy-aware edge server placement problem has been solved using particle swarm optimization which reduced the energy consumption by 10% on the Shanghai Telecom dataset [13]. The genetic algorithm was used to find the optimal location for the edge server to minimize the delay and the cost. The performance of the genetic algorithm was evaluated on the Shanghai Telecom dataset and compared against local search algorithms like hill climbing and simulated annealing [14]. Edge server and service placement were solved by using a two-step method. The first method includes the clustering algorithm and the second method includes nonlinear programming. As the service placement depends on the edge server location, to keep eye on service price and request rate without compromising the profit, these two problems are coupled and solved by a two-step method [15].

From the survey, it has been observed that there are a lot of metaheuristic algorithms that exist for solving the problem of edge server placement. To resolve the premature convergence, the authors intend to propose an improved whale optimization algorithm for optimal edge server placement.

3 System Design

Edge computing (EC), where the computing happens at the edge rather than in a centralized server as in cloud computing. The idea behind EC is to bring the computation nearer to the user devices with the aid to minimize the latency (L) and Energy Consumption (EC). In the real-world environment, many IoT devices produce a tremendous amount of data. The data needs to be processed for making the optimal decision. Due to the low storage and processing capability of IoT devices, the request generated by IoT devices is forwarded to Edge Servers (ES) in edge computing. It is, therefore, essential to locate the edge server in an optimal location. The request from the IoT device has to be forwarded to the edge server which tends to minimize latency and energy consumption. The representation of IoT devices and edge servers in edge computing is represented in Figure 1. The problem of forwarding the request generated by the IoT device to the edge server is viewed as NP-Hard problem. Such NP-Hard problem of optimal allocation of edge server to the user request is solved by using the proposed improved whale optimization algorithm. The improved whale optimization algorithm is a population-based metaheuristic algorithm where each solution is represented as an edge server. The algorithms aim to choose the edge server which incurs minimum latency and energy consumption for processing the user request. To achieve the objective of choosing the optimal edge server, the following assumptions are taken into account:

- The edge servers are homogeneous which means they are having same storage and processing capability.
- Each device is connected to the edge server. At any point, no device is connected to two different edge servers which are represented as $\varnothing_{ES_i} \cap \varnothing_{ES_j} = \{\}$.

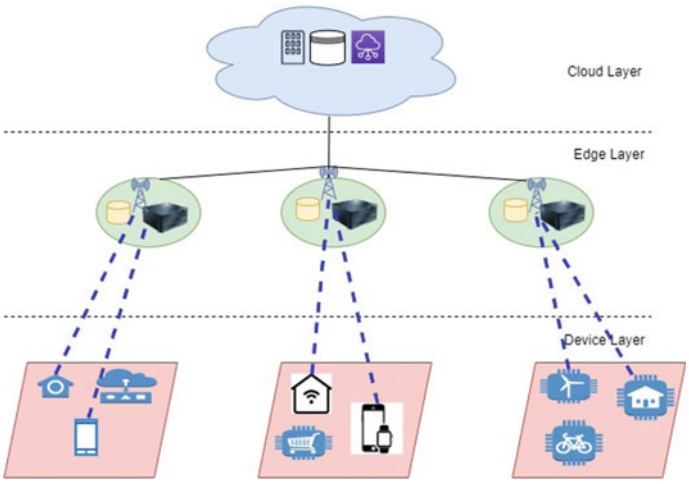


Fig. 1 Representation of edge server placement problem

Table 1 Notations and description

Notations	Description
n	Number of edge servers
$E \leftarrow \{ES_1, ES_2, \ldots, ES_n\}$	Set of edge servers
ES_i	ith edge server
$D \leftarrow \{d_1, d_2, \ldots, d_n\}$	Set of user devices
d_j	jth user device
$\varnothing_{ES_i}$	set of user devices that are connected to the ith edge server
$L_{ES_i d_j}$	Latency incurred between ith edge server and jth user device
$EC_{ES_i d_j}$	Energy consumed between ith edge server and jth user device
W_1 and W_2	Represents the weight assigned to the objective function latency and energy consumption, respectively

Various notations used in this paper are represented in Table 1. Thus, the problem focused in this paper is edge server placement which is optimally solved by using the proposed improved whale optimization algorithm. The underlying fitness function used by IWOA for optimal allocation of edge servers is specified in Eq. (1).

$$Minf(ES_i) \leftarrow W_1 * L_{ES_i d_j} + W_2 * EC_{ES_i d_j} \tag{1}$$

subject to

(a) Only one edge server is chosen for processing each request

$$\sum_{j=1}^{n} x_{ES_i d_j} = 1 \tag{2}$$

(b) *Sum* of the weight assigned for latency and energy consumption must be equal to 1

$$W_1 + W_2 = 1 \tag{3}$$

(c) *i*th edge server will be chosen if and only if the latency and energy consumption are less than any other edge servers in the environment

$$L_{ES_i d_j} < L_{ES_k d_j} \&\& EC_{ES_i d_j} < EC_{ES_k d_j} \tag{4}$$

The proposed IWOA solves the fitness function specified in Eq. (1) with maximum server utilization.

4 Proposed Improved Whale Optimization Algorithm-Based Edge Server Placement (ESP)

The edge server placement algorithm is designed using an improved whale optimization algorithm. The conventional whale optimization algorithm suffers from the problems such as premature convergence and thereby trapping in the local optimal solution. In this research work, the exploration and exploitation capability of WOA is improved by introducing a switch between linear reduction and nonlinear reduction of convergence factor a. The linear reduction of a tends to do exploration at the initial stage which tends to slow down the convergence while in the latter stage, it does exploitation by increasing the convergence speed which results in trapping local optimal solution. Therefore, this research tends to do exponential reduction at the beginning stage and linear reduction at the latter stage as specified in Eqs. (5) and (6), respectively.

$$a = 2^{(1-t/T_{\max})} \; if \; t < 0.5T_{\max} \tag{5}$$

$$a = 2\frac{2.t}{T_{\max}} \; if \; t \geq 0.5T_{\max} \tag{6}$$

The working of the edge server allocator is shown in Algorithm 1. The algorithm initially started with N number of whales where each whale represents the edge server. At each iteration, each whale is evaluated against the fitness as specified in Eq. (1). Any metaheuristic algorithm tends to have the principle of doing exploration and exploitation for finding the best optimal solution. During the exploitation phase, each whale tends to move toward the position of the best whale which is also termed encircling the prey. The behavior of encircling the prey is mathematically modeled as in Eq. (7):

$$\overrightarrow{ES_i(t+1)} \leftarrow \overrightarrow{ES_*(t)} - \overrightarrow{A}.\left|\overrightarrow{C} * \overrightarrow{ES_*(t)} - \overrightarrow{ES_i(t)}\right| \tag{7}$$

where $\overrightarrow{ES_*(t)}$ represents the position of the best edge server at iteration t. $\overrightarrow{A}$ and $\overrightarrow{C}$ are coefficient vectors that can be computed using Eqs. (8) and (9), respectively.

$$\overrightarrow{A} \leftarrow 2.\overrightarrow{a}.\overrightarrow{r} \tag{8}$$

$$\overrightarrow{C} \leftarrow 2.\overrightarrow{r} \tag{9}$$

The random variable $\overrightarrow{r}$ is initialized in the interval [0, 1]. The bubble net attacking method of whales is also used to update the position of the edge server at each iteration. The updation of position using the bubble net attacking method or by encircling the prey is done by the probability of 0.5. The mathematical modeling of the bubble net attacking method is shown in Eq. (10):

$$\overrightarrow{ES_i(t+1)} \leftarrow \overrightarrow{ES_*(t)} - \overrightarrow{ES_i(t)} - \left| e^{bl}.\cos 2\pi l + \overrightarrow{ES_*(t)} \right| \tag{10}$$

where l is a random value generated in the interval $[-1, 1]$ and b is a constant. Enough exploration is required to avoid trapping in the local optimal solution. Thus, while doing exploration, a random edge server is chosen, and based on it, the position of other edge servers will be updated. The mathematical modeling for exploration is specified in Eq. (11):

$$\overrightarrow{ES_i(t+1)} \leftarrow \overrightarrow{ES_{rand}(t)} - \overrightarrow{A}.\left| \overrightarrow{C} * \overrightarrow{ES_{rand}(t)} - \overrightarrow{ES_i(t)} \right| \tag{11}$$

Algorithm	EdgeServerPlacement(E, D)
Input	Set of Edge Servers E, Set of Edge Devices D
Output	Each device is assigned to Edge Server
	Initialize population of whales as $ES_1, ES_2, \ldots, ES_n$
	For each device $d_j \in D$
	For each Whale ES_i
	Compute fitness using Equation (1)
	End For
	While $(t \leq Max_Iter)$
	For each Whale ES_i
	Compute a, A, C, l, p
	If $p < 0.5$ then
	If $\lvert A \rvert < 1$then
	Update the position of the Edge Server using Equation (7)
	Else
	Update the position of the Edge Server using Equation (10)
	End If
	Else
	Update the position of the Edge Server using Equation (11)
	End If
	End For
	Update the best Edge Server ES_* as the one with minimum fitness
	End While
	Return ES_*

5 Experimental Results

The efficiency of the proposed ESP is analyzed by conducting experiments by varying the number of edge servers. The experimentation is simulated in Python with Intel(R) Core (TM) i7-8565U CPU @ 1.80GHz 1.99 GHz processor and 16 GB RAM. For experimentation, the number of IoT devices is fixed at 200 and the edge servers are increasing from 15, 20, 25, and 30. The size of the user request is considered in the range of 40–100 MB. In each experiment, the proposed ESP is compared with conventional WOA-based placement and PSO-based placement of edge servers. The experimentations are conducted to compare the delay and energy consumption of proposed ESP with WOA- and PSO-based placement of edge servers. Also, the convergence of algorithms is plotted to claim that the ESP does not strike in local optima as compared to other traditional algorithms.

5.1 Comparison of Delay

Figure 2 shows the comparison of delay between the proposed ESP, WOA, and PSO. It has been observed that, when the number of edge servers is increasing, the delay in processing the request decreases. This is because the request has been allocated to the nearest edge server which thereby minimizes the time taken for forwarding the request and responding to the result. Also, it has been observed that the proposed ESP achieves minimum delay when compared to other placement algorithms. The reason behind this is that the proposed algorithm has a balance between exploration and exploitation and also it is governed by the convergence factor which paves for both linear and exponential reduction for finding the global solution. When the edge servers are 15, ESP achieves a 10, and 15% reduction in delay when compared to WOA and PSO, respectively. Also, when the number of edge servers is kept at 30, WOA achieves a 22.22% increased delay and PSO achieves a 31.14% increased delay than the proposed ESP.

5.2 Comparison of Energy Consumption

Energy consumption increases when the number of IoT devices increases. Energy consumption represents the energy consumed by the IoT device while transferring the request to the edge server. When the number of edge servers increases, the energy consumed by the devices decreases as the edge servers are situated nearer to IoT devices and the transmission time between the device and the edge server decreases thereby reducing the energy consumption. Figure 3 represents the energy consumption of IoT devices while varying the number of edge servers. It has been observed that when the edge servers are kept at 20, ESP achieves 24.19% and 35.61%

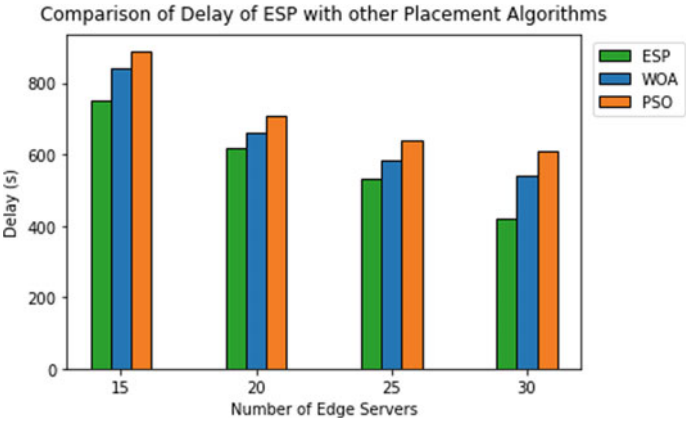


Fig. 2 Comparison of delay

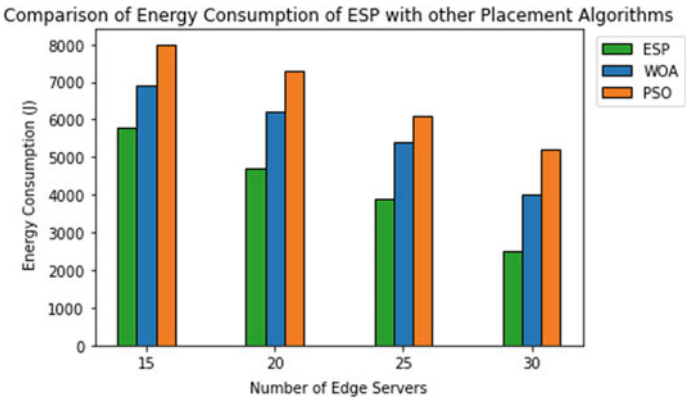


Fig. 3 Comparison of energy consumption

reduced energy consumption than WOA and PSO, respectively. Also, the energy consumed by the proposed ESP is 20.51% less when the number of edge servers is kept as 30 instead of 25.

5.3 Comparison of Convergence

Figure 4 represents the convergence of fitness of various edge server placement algorithms. From Fig. 4, it is evident that the proposed ESP achieves a globally optimal solution when compared to WOA and PSO. As the proposed ESP is governed by the convergence factor, it promotes a good level of exploration and exploitation when compared to WOA and PSO. The traditional WOA switches between exploration

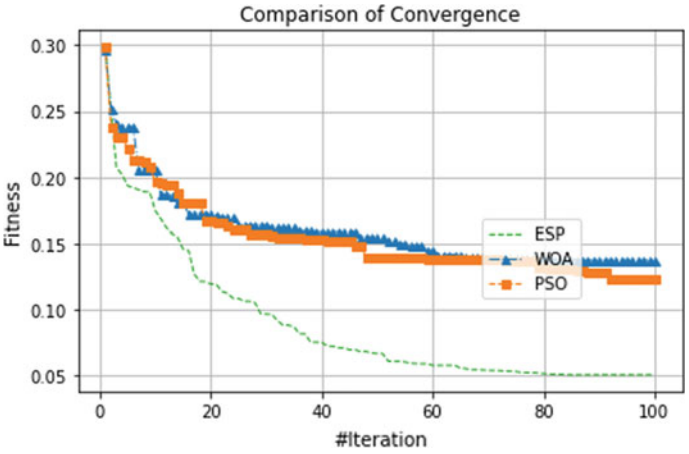


Fig. 4 Comparison of fitness

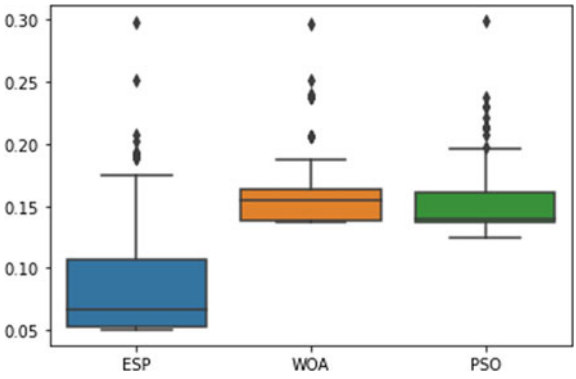


Fig. 5 Boxplot representation of characteristics of fitness value

and exploitation with a probability of 0.5, always governed by linear convergence speed. Also, the PSO does the exploration and exploitation by considering the best and random particle tends to converge in the local optimal solution. Figure 5 represents the summary of the best fitness obtained across each iteration. The minimum value of ESP is 0.050 which is less than WOA (0.137) and PSO (0.124).

6 Conclusion

To solve the problem of the traditional way of forwarding the request to the cloud which incurs huge delays and consumes more energy, edge computing is used for doing the computation nearer to the user devices. The problem of edge server placement has been addressed using the proposed ESP algorithm based on IWOA. The

proposed algorithm outperforms the traditional placement algorithms like WOA and PSO in terms of metrics like delay and energy consumption. The proposed ESP uses exponential reduction at the initial stage and linear reduction at a later stage for promoting exploration and exploitation. The experimentation is carried out with a different number of edge servers from 15, 20, 25, and 30. When the number of edge servers is increased from 25 to 30, it has been observed that 56% of energy is observed in the proposed ESP. Also, the proposed ESP achieves a 9.4 and 17.18% minimum delay over WOA and PSO, respectively.

References

1. Lähderanta T, Leppänen T, Ruha L, Lovén L, Harjula E, Ylianttila M, Riekki J, Sillanpää MJ (2021) Edge computing server placement with capacitated location-allocation. J Parallel Distrib Comput 1(153):130–49
2. Schneider P, Xhafa F (2022) Chapter 7—IoT, edge, cloud architecture and communication protocols: Cloud digital ecosystem and protocols. In: Schneider P, Xhafa F, Detection A (eds) Complex event processing over IoT data streams. Academic Press, pp 129–148. ISBN 9780128238189. https://doi.org/10.1016/B978-0-12-823818-9.00018-3
3. Shahzadi S, Iqbal M, Dagiuklas T, Qayyum ZU (2017) Multi-access edge computing: open issues, challenges, and future perspectives. J Cloud Comput 6(1):1–3
4. Moorthy RS, Pabitha P (2021) Design of wireless sensor networks using fog computing for the optimal provisioning of analytics as a service. In: Machine learning and deep learning techniques in wireless and mobile networking systems 13 Sep 2021. CRC Press, pp 153–173
5. Wang S, Zhao Y, Xu J, Yuan J, Hsu CH (2019) Edge server placement in mobile edge computing. J Parallel Distrib Comput 1(127):160–8
6. Ma R (2021) Edge server placement for service offloading in internet of things. Secur Commun Netw 30:2021
7. Li Y, Zhou A, Ma X, Wang S (2021) Profit-aware edge server placement. IEEE Internet Things J 9(1):55–67
8. Hu Z, Xu X, Chen J (2021) An edge server placement algorithm based on genetic algorithm. In: ACM turing award celebration conference-China (ACM TURC 2021) 30 July 2021, pp 92–97
9. Guo F, Tang B, Zhang J (2021) Mobile edge server placement based on meta-heuristic algorithm. J Intell Fuzzy Syst 40(5):8883–97
10. Gupta D, Kuri J (2021) Optimal network design: edge server placement and link capacity assignment for delay-constrained services. In: 2021 17th international conference on network and service management (CNSM) 25 Oct 2021. IEEE, pp 111–117
11. Cui G, He Q, Xia X, Chen F, Jin H, Yang Y (2020) Robustness-oriented k edge server placement. In: 2020 20th IEEE/ACM international symposium on cluster, cloud and internet computing (CCGRID) 11 May 2020. IEEE, pp 81–90
12. Zeng F, Ren Y, Deng X, Li W (2018) Cost-effective edge server placement in wireless metropolitan area networks. Sensors 19(1):32
13. Li Y, Wang S (2018) An energy-aware edge server placement algorithm in mobile edge computing. In: 2018 IEEE international conference on edge computing (EDGE) 2 July 2018. IEEE, pp 66–73
14. Kasi SK, Kasi MK, Ali K, Raza M, Afzal H, Lasebae A, Naeem B, Ul S, Rodrigues JJ (2020) Heuristic edge server placement in industrial internet of things and cellular networks. IEEE Internet Things J. 8(13):10308–17
15. Zhang X, Li Z, Lai C, Zhang J (2021) Joint edge server placement and service placement in mobile edge computing. IEEE Internet Things J 13 Nov 2021

Performance Analysis of LBT Cat4 Based 5G IoT Enabled New Radio in Unlicensed Spectrum



Zubair Shaban, Nishu Gupta, Krishan Kumar, Sandeep Kumar Sarowa, and Mohammad Derawi

Abstract 5G Internet of Things (5G IoT) which is currently under development by the 3rd Generation Partnership Project (3GPP) envisages the way for a diversity of connecting devices to the IoT via cellular networks. As the amount of licensed spectrum is scarce in nature, New Radio Unlicensed (NR-U) is seen as a favorable solution for meeting the exponential growth in traffic demands. 5G NR-U combines the features from License Assisted Access (LAA) and 5G NR to support global cellular operations over entire possible unlicensed bands. As spectrum efficiency is an important metric in 5G communication networks, there is a need to operate in unlicensed spectrum efficiently. In this paper we adhere to Listen Before Talk (LBT) mechanism as defined by the 3GPP. Using this mechanism, NR-U will allow network terminals to use unlicensed spectrum with fair coexistence with Wi-Fi, which is an incumbent wireless technology in unlicensed spectrum. Furthermore, we propose collision avoidance technique among NR-U and other incumbent radio access technologies, better channel efficiency and high Jain's Fairness index algorithm.

Keywords New radio unlicensed · Energy detection · LTE-LAA · Channel occupancy time · LBT · CSMA/CA · AIFS · CCA · Internet of things

Z. Shaban
Department of Electronics and Communication Engineering, IIIT Delhi, New Delhi, India

N. Gupta (✉) · M. Derawi
Department of Electronic Systems, Faculty of IES, Norwegian University of Science and Technology (NTNU) in Gjøvik, Gjøvik, Norway
e-mail: nishugupta@ieee.org

Z. Shaban · K. Kumar
Department of Electronics and Communication Engineering, National Institute of Technology Hamirpur, Hamirpur, India

S. K. Sarowa
Scientist at Cert-In, MeitY, Government of India, New Delhi, India

© The Author(s), under exclusive license to Springer Nature Singapore Pte Ltd. 2023

S. Chinara et al. (eds.), *Advances in Distributed Computing and Machine Learning*,
Lecture Notes in Networks and Systems 660,
https://doi.org/10.1007/978-981-99-1203-2_9

1 Introduction

Due to the exponential increase in mobile traffic on licensed spectrum, implementation of 5G on unlicensed spectrum, known as NR-U (New Radio in Unlicensed spectrum) has been decided as a favorable solution for network operators to enhance their limited licensed spectrum. NR-U which will use unlicensed spectrum which is based on the NR air interface has been given an approval by the 3rd Generation Partnership Project (3GPP) for Release 16 [1, 2]. The unlicensed Long-Term Evolution (LTE) operation has been standardized by 3GPP under the name Licensed Assisted Access (LAA) [3]. In LTE-LAA, as the name suggests, access to unlicensed spectrum is via licensed assistance only which is known as carrier aggregation (CA), while in NR-U there is a standalone mode where there is no need of licensed assistance and also supports a non-standalone mode.

The Internet of Things (IoT) has emerged as a big booster to the telecommunication technology and has become an indispensable part of our daily life with applications in smart cities, smart health, smart logistics, smart industries, and home automation. Mobile IoT, such as narrowband IoT (NB-IoT) and LTE for machines (LTE-M), are important advancements in this rapidly evolving field of IoT technology. As the data users are increasing continuously with the widespread deployment of 5G IoT enabled technologies, modern wireless networks are in need of demand for high throughput, connectivity at large scale and minimum delay in transmission. Wireless communication technologies such as 6LoWPAN Bluetooth Low Energy (BLE) and ZigBee have been widely used for IoT solutions in smart cities, but their limited coverage is a major concern [4]. 5G network is expected to support challenging requirements for the wireless access connection, targeting the throughput capacity will be targeted at 1000-times more as compared to the current 4G network and less than 1 ms of return latency. To establish a connection between an IoT device and the cellular network, either a direct 3GPP connection narrowband IoT (NB-IoT) is suitable or an indirect non-3GPP connection. [5, 6]. For achieving this goal there is a need to design new transmission technologies to achieve the expected throughput and to minimize the cost of deployment. But due to the overloading data traffic, there arises the problem of licensed spectrum shortage because of overloading in traffic of data. On the other hand, as the deployment expenses are low, Wi-Fi becomes a prominent network which is dominant in the bands 2.4 and 5 GHz which are unlicensed [7]. To overcome this problem, recently 3GPP started working on the exploration of unlicensed spectrum, which is said to be a prominent enhancement to licensed spectrum. One such work has been discussed for LTE-LAA for multiple carriers in [8]. The major contributions of the article are:

1. First, we define some fundamentals of the NR-U based (LBT) listen-before-talk, such as modes of LBT, random back-off, contention window, energy detection (ED) threshold, and different settings of parameters for four priority access classes.

2. We then propose LBT mechanism to improve the successful transmission probability, decrease collision probability and improve Jain's fairness index.
3. We evaluate the performance of the proposed mechanism on different parameters.

1.1 NR-U Scenarios

Scenario	Description
Scenario A (NR/ NR-U LAA)	A UE is supported in the CA mode by a licensed carrier through a 5G NR cell and an unlicensed carrier through an NR-U cell
Scenario B (LTE/ NR-U DC)	A UE is supported in the DC mode by a licensed carrier through an LTE cell and an unlicensed carrier through an NR-U cell
Scenario C (NR-U Standalone)	A UE is supported primarily by an NR-U cell. This scenario is useful for operating private networks
Scenario D (NR/ NR-U UL/DL)	A UE is supported by a licensed carrier through an NR cell for UL communication, and by an unlicensed carrier through an NR-U cell for DL communications
Scenario E (NR/ NR-U DC)	A UE is supported in the DC mode by a licensed carrier through an NR cell and an unlicensed carrier through an NR-U cell

1.2 Channel Access Categories

For accessing the channel in NR-U, following four categories, known as LBT categories, have been defined by 3GPP, which are same as that of LTE-LAA.

CAT-1-LBT (category 1 LBT)	NR-U/LTE-LAA device transmits immediately after 16 micro-seconds switch gap (without performing LBT)
CAT-2-LBT (category 2 LBT)	NR-U/LTE-LAA device performs LBT skipping random back-off where CCA (Clear Channel Assessment) period is predefined (fixed at 25 micro-seconds)
CAT-3-LBT (category 3 LBT)	NR-U/LTE-LAA device is performing LBT with back-off randomly with fixed size of contention window where eCCA (extended CCA) period is randomly chosen from contention window
CAT-4-LBT (category 4 LBT)	NR-U/LTE-LAA device will perform LBT with random back-off where size of contention window is not fixed but variable. Here CCA extension period is chosen randomly from variable contention window

2 Coexistence Mechanism of Listen Before Talk (LBT) for NR-U

In order for achieving harmonious coexistence between different network devices, LBT mechanism is applied when different operators operating independently are sharing similar unlicensed spectrum. For avoiding collisions between devices accessing the channel, every node of transmission is directed to continuously check if the unlicensed channel is busy or idle with the help of CCA.

2.1 Modes of Transmission

While studying LBT we come across two main types of LBT [9] which are frame-based and load-based LBT's, described below with appropriate figures.

Frame-Based LBT: Before transmission of packets, a node is mandated to go through a mechanism called CCA wherein it checks for the availability of channel. When the node perceives the channel as idle through CCA only then it can start delivering the packets of data but only for the period called as COT. During this COT also there needs to be at least 5% of COT preserved for an idle period. If this process is not satisfied transmission is suspended and deferred for the following period of fixed frame (Fig. 1).

Load-Based LBT: Here while performing the CCA, its initial period is used to detect the channel availability. Transmission from the node starts only when channel is detected idle. If not, then node has to apply an eCCA, the duration of which is decided by the product of random factor and observation time of CCA (Fig. 2).

When we compare load-based and frame-based LBT, it is seen that frame-based LBT has low cost of deployment and is in need of a smaller number of changes under the current framework of LTE. But, load-based LBT has low negative effect on incumbent networks and in addition because of its compatibility towards existing LBT mechanism it gives efficient coexistence performance metrics.

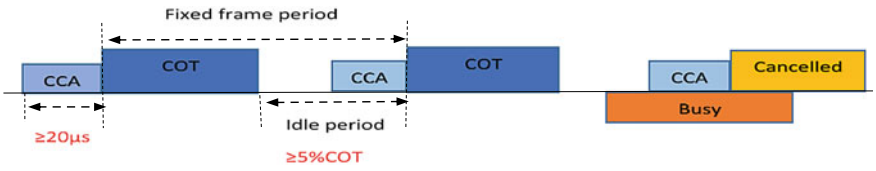


Fig. 1 System model showing spectrum handoff in NEMO based CR vehicular networks

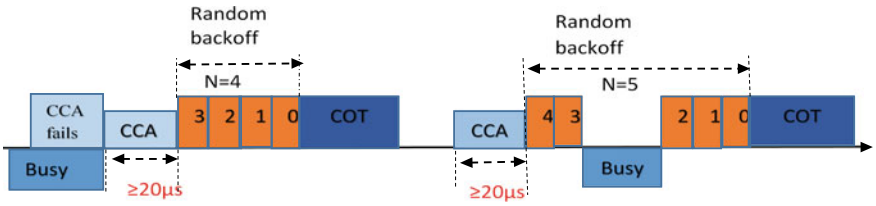


Fig. 2 System model showing access network selection in single non-safety service call/ multiple non-safety services call simultaneously in CR vehicular networks

2.2 Energy-Detection-Threshold

In order to avoid channel collisions and to increase successful transmission probability ED (Energy Detection), based on CCA, is applied prior to channel accessing for transmission of data. The transmitting node refrains from transmission if receives a higher energy level than the threshold energy because the channel is determined as busy; otherwise, the transmitting node starts transmitting data as it takes the channel in idle state. If we set a higher threshold for ED, more packets of data get transmitted which in turn increases the probability of collisions and causes higher packet dropping probability. Therefore, ED threshold must be set appropriately which mainly relies on the deployment environment.

2.3 Contention Window-Size

In the release 16 (Rel-16) of NR two kinds of settings are studied for the contention window size. The two types are LBT algorithms which are based on LBT Category 3 (Cat3) and LBT Category 4 (Cat4) where contention window is fixed in the former case and variable in later case. In LBT Cat3 case, initialization of back-off timer is random from a contention window of fixed size whereas in LBT Cat4, back-off timer gets initialized randomly which is drawn from a contention window of dynamic nature. The size of LBT Cat4 contention window increases exponentially when the collision occurs. Once there is a successful transmission or when contention window size passes maximum set back-off stage, it has to readjust to the minimum size. This process makes the behaviour of back-off better as it decreases the collision probability. Thus, LBT Cat4 with contention window of variable size is efficient in order to achieve fair and harmonious coexistence when operating on unlicensed spectrum.

2.4 Back-Off

As can be seen in Fig. 1, back-off mechanism is the fundamental differing point in the two types of LBT. When we compare LBT with back-off with LBT without back-off, we see that random back-off LBT is more efficient for decreasing the probability of collisions as it introduces random nature. At the start, contention window of W size is taken, then a back-off timer randomly initialized which is drawn uniformly between 1 and W. Then it starts decrementing until it reaches 0 to access the idle. During this process if the channel is sensed busy, NR release 16 specifies that back-off timer to be frozen and suspends the process of countdown as it improves the performance of coexistence in LBT mechanism in NR-U. The process of countdown resumes once the channel becomes idle. Thus, the back-off algorithm in NR-U LBT is more efficient.

2.5 Channel Access Priority Classes

3GPP has defined 4 access priority classes with different parameter settings for NR-U LBT, which are mentioned in Table 1.

3 Wi-Fi NR-U Co-Existence

Coordinating access in unlicensed spectrum channels is one of the challenges faced in achieving fair coexistence. Therefore, most of wireless standards use CSMA/CA algorithm (Carrier Sense Multiple Access with Collision Avoidance) where the transmitter backs off randomly prior to gaining the access in unlicensed channel. CSMA/CA algorithm has various versions. In CSMA/CA with exponential back-off CSMA/CA defines that the device senses the channel at the start of transmission for fixed time period of AIFS (Arbitration Inter Frame Space) which is called as defer period in NR-U/LTE-LAA. During AIFS when the channel gets idle the device

Table 1 NR-U LBT parameters settings

Access priority class	Defer period	Initial CW size	Maximum CW size	Maximum Back-off stage	Maximum COT
#1	25 or 34 μs	4	8	1	2 ms
#2	25 or 34 μs	8	16	1	3 or 4 ms
#3	43 μs	16	64	2	6,8 or 10 ms
#4	79 μs	16	1024	6	6,8 or 10 ms

randomly backs off for q slots, here q is defined between $\{0, \ldots, W_{min}-1\}$ and also the minimum contention window size (CW_{min}) is given by W_{min}. When the channel is sensed busy, back-off gets frozen until the channel becomes free again. Once the backing off process is finished the device can start the transmission for a duration called as TXOP (Transmission Opportunity) which is known as COT (Channel Occupancy Time) in NR-U and LTE-LAA. If there are some remaining frames to transmit, the device starts back-off process again. If there is a transmission failure or a collision occurs, the contention window size gets doubled and the contention begins again for new access and q gets a new value again:

$$q \in \left\{0, \ldots, \min\left\{2^{l} W_{\min}, W_{\max}\right\} - 1\right\}$$

Here l defines the number of retransmitting attempts and the maximum contention window size is given by W_{max} (CW_{max}). The procedure stops when it reaches the limit of maximum retransmission.

3.1 Coexistence Performance Evaluation

The main target of NR-U is to provide harmonious coexistence among incumbent RATs and NR-U, e.g., Wi-Fi, and also between NR-Us which operate on the same unlicensed spectrum. Evaluation results for NR-U and Wi-Fi are provided in the below section.

3.2 Evaluation Methodology

We have tested the performance evaluation for 10 gNBs (5G base stations) using channel access priority class 3. As performance metrics, Average collisions, Average COT, Average Normalized Airtime, Total collision probability, Total channel efficiency and Jain's Fairness index are considered, where Average collisions gives an average of collisions between incumbent RAT and NR-U, which in turn gives total collision probability, average COT gives the average time a base station occupies a channel which also gives us total channel efficiency, Normalized airtime defines what percentage of channel is under use and what percentage is free and finally Jain's fairness f(X) index [10] gives the fairness measure of the coexistence given by Eq. 1.

$$f(x) = \frac{\left[\sum_{i=1}^{n} x_i\right]^2}{n \sum_{i=1}^{n} x^2} \tag{1}$$

where x gives the number of resources and n gives the number of users with same resource.

$$\text{where } 0 \leq f(x) \leq 1.$$

3.3 Coexistence Performance

As performance metrics for coexistence, several parameters are simulated for priority class#3. At first, Gap-period is inserted before back-off with reservation signal (RS) as True and hence metrics are evaluated. Then for comparison reservation signal is changed to False, Gap-period taken after back-off and again parameters are simulated. The simulation results are given below:

As shown in Fig. 3, it can be seen that with the increase in number of gNB's, gap-period before back-off and with Reservation signal has a better collision probability performance than gap-period after back-off without RS. Also, the graph for collision probabilities depicts that gap-period before back-off behaves more favorably than that of gap-period after back-off.

In Fig. 4 it can be seen that channel utilization is better when gap-period is performed before back-off as compared to gap-period after back-off.

Figure 5 shows that gap-period before back-off has better performance metrics when normalized airtime and Jain's fairness are considered.

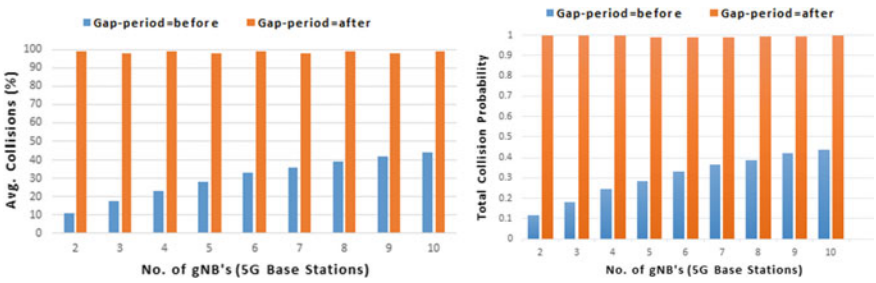


Fig. 3 Average collisions and total collision probability

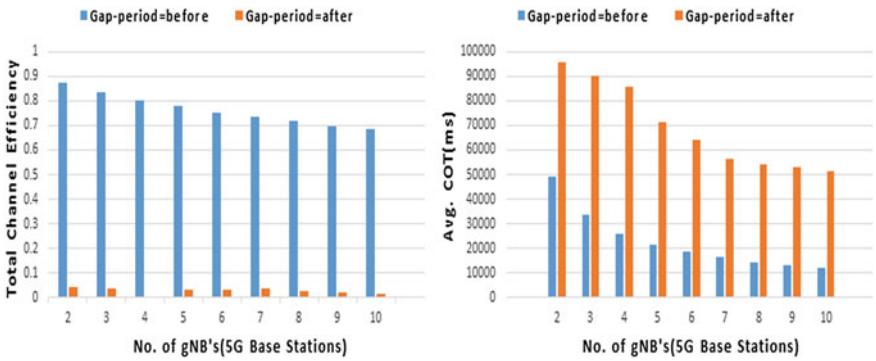


Fig. 4 Average channel occupancy time and total channel efficiency

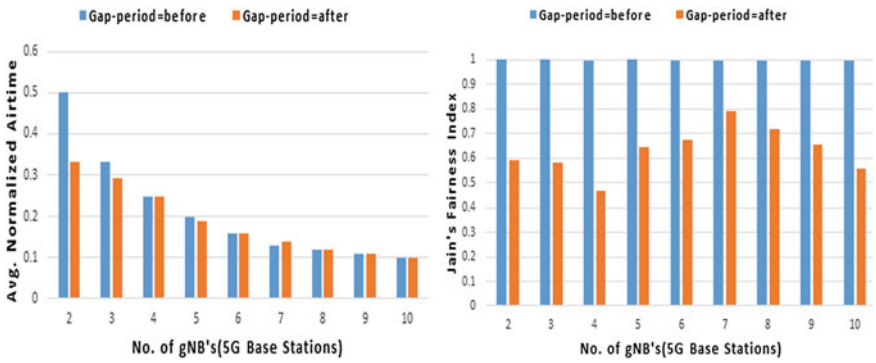


Fig. 5 Average normalized airtime and Jain's fairness index

4 Conclusion

In this paper, fundamental concept of aggregation between licensed bands and unlicensed bands has been presented. In the 5G wireless technology era, NR-U is the main aggregation technology operating in unlicensed spectrum. Therefore, fundamentals of LBT based NR-U such as ED threshold, transmission modes, contention window random back-off and four priority access classes have been discussed. We proposed two back-off mechanisms for NR-U LBT for unlicensed carriers in order to make transmission behaviour and coexistence performance efficient. It has been shown that in NR-U LBT mechanism, gap-period before back-off with reservation signal (RS) gives efficient performance metrics such as collision probability, channel efficiency, airtime, Jain's fairness index than that of gap-period after back-off without reservation signal (RS) for class#3 access priority parameter settings.

References

1. RP-181339 (2018) Study on NR-based access to unlicensed spectrum. 3GPP RAN Meeting #80
2. RP-191575 (2019) NR-based access to unlicensed spectrum. 3GPP RAN Meeting #84
3. Hirzallah M, Krunz M, Xiao Y (2019) Harmonious cross-technology coexistence with heterogeneous traffic in unlicensed bands: analysis and approximations. IEEE Trans Cogn Commun Netw 5(3):690–701. https://doi.org/10.1109/TCCN.2019.2934108
4. Bouzidi M, Dalveren Y, Cheikh FA, Derawi M (2020) Use of the IQRF technology in internet-of-things-based smart cities. IEEE Access 8:56615–56629. https://doi.org/10.1109/ACCESS.2020.2982558
5. Froytlog A et al (2019) Ultra-low power wake-up radio for 5G IoT. IEEE Commun Mag 57(3):111–117. https://doi.org/10.1109/MCOM.2019.1701288
6. 3GPP TR 22.861 (2016) Feasibility study on new services and markets technology enablers for massive Internet of Things; Stage 1 14.1.0
7. Pirayesh H, Zeng H (2022) Jamming attacks and anti-jamming strategies in wireless networks: a comprehensive survey. IEEE Commun Surv Tutor
8. Zheng B, Wen M, Lin S, Wu W, Chen F, Mumtaz S, Ji F, Yu H (2019) Design of multi-carrier LBT for LAA & WiFi coexistence in unlicensed spectrum. IEEE Netw 1–8. https://doi.org/10.1109/MNET.2019.1900172
9. ETSI EN 301 893 V1. 7.2 (2014) Broadband Radio Access Networks (BRAN); 5 GHz High Performance RLAN; Harmonized EN Covering the Essential Requirements of Article 3.2 of the R&TTE Directive
10. Wang Y, Ma W, Zhang* M, Liu Y, Ma S (2022) A survey on the fairness of recommender systems. ACM J ACM (JACM)

Lattice Cryptography-Based Geo-Encrypted Contact Tracing for Infection Detection



Mayank Dhiman, Nitin Gupta, Kuldeep Singh Jadon, Ujjawal Gupta, and Yashwant Kumar

Abstract The world has already witnessed many epidemic diseases in the past years, like H1N1, SARS, Ebola, etc. Now, Covid-19 has also been added to list, which is declared as pandemic by World Health Organization. One of the most commonly used methods to tackle the spread of such diseases is using mobile applications to perform contact tracing of the infected person. However, contact tracing applications involve transmitting sensitive location-based data of the infected person to the government servers. Therefore, recently this has raised a lot of concerns regarding the privacy of the infected persons. This work proposes a lightweight and secure encryption scheme, based on location-based encryption which can be used to transfer the location data to the server without compromising its security. The main aim of the work is to design an algorithm in such a way that the encrypted transferred data can only be decrypted at the server and in-between data leakage can be prevented. This work proposes to use location-based encryption combined with Learning with Errors problems in Lattices, which can provide a solution to privacy concerns in contact tracing, which will even be applicable in the post-quantum period.

Keywords COVID-19 · Lattice based cryptography · Contact tracing · Location data · Security and privacy

1 Introduction

In the recent past, the world has observed many infectious diseases. However, the Coronavirus disease (COVID-19), seems to be the most fatal among all. COVID-19 is an infectious disease caused by SARS-CoV-2, was first believed to originated from Wuhan, China in December 2019 [1, 2]. The World Health Organization also declared this outbreak a pandemic on March 11, 2020. More than 28.4 million cases of

M. Dhiman · N. Gupta · K. Singh Jadon (✉) · U. Gupta · Y. Kumar
Department of Computer Science and Engineering, National Institute of Technology Hamirpur, Himachal Pradesh, India
e-mail: kuldeep788@gmail.com

© The Author(s), under exclusive license to Springer Nature Singapore Pte Ltd. 2023

S. Chinara et al. (eds.), *Advances in Distributed Computing and Machine Learning*, Lecture Notes in Networks and Systems 660,
https://doi.org/10.1007/978-981-99-1203-2_10

COVID-19 have been reported in more than 188 countries and territories, resulting in more than 9,15,000 deaths, while approximately 20.4 million people have also been recovered [3].

Like other contagious diseases, it is believed that the COVID-19 infection spreads when a healthy person comes in close contact with the already COVID-19 infected person with symptoms [4, 5]. Although work to develop a potential vaccine has already begun, however, till now major breakthrough in this area has not been achieved. Till any vaccine arrives which can be used to cure COVID-19, one of the possible preventive solutions is to track and isolate the infected person. Further, all the persons which are primary contact of that person are also required to be identified [6, 7]. This is not a new technique, humans have already used manual contact tracing during previous pandemics, notably during Spanish Flu in 1918 [8]. However, with large population of the globe, it is not feasible to do contact tracing manually.

To achieve this objective of contact tracing, mostly all the countries have developed contact tracing apps for a smart phone. These apps are used for tracing and identifying the COVID-19 effected person [9]. Government of India has also developed 'Aarogya Setu' application [10], which reached 50 million users in just 13 days. Government of Singapore uses 'TraceTogether' application [11], which relies on Bluetooth technology, for doing contact tracing. Honk-Kong Government has developed 'StayHomeSafe' [12] application, which pairs with wristbands to perform contact tracing. Other countries have also developed their indigenous contact tracing applications and many have mandated their citizens to install the application on their smartphone.

Although contact tracing has proved to be effective in tackling COVID-19, however, many privacy advocates and security researchers have raised their concerns about the privacy problems associated with these applications. Most contact tracing applications in use today, involve collecting location data from the user and sending it to the government servers. Since sensitive location data is transmitted, one of the major concerns is data leakage on the way to the government servers. Due to the data security concerns, many persons are not willing to install the contact tracing app to share their private data [13]. Therefore, a secure and privacy-preserving software application is highly required to build the confidence of a contact tracing application user.

The main aim of this work is to secure contact tracing application. All contact tracing applications mostly have similar underlying technology [14]. As mentioned above, mostly all the contact tracing applications send the user's private data to the remote server and prone to the in-between data leakage. The objective of this work is to ensure that data obtained from the contact tracing application can only be decrypted at the specified server. To achieve this objective, the proposed work introduces *Learning with Errors* (LWE) problem of Lattices with location-based encryption scheme to prevent data leakage in contact tracing applications. Moreover, the proposed lattice-based encryption is a strong candidate for post-quantum cryptography [15]. Therefore, the proposed scheme will be a promising approach even in the post-quantum period. Further, a geo-encryption scheme is implemented by incorporating the location parameters in the key generation process of standard

LWE-based encryption. It is shown through analysis that the proposed encryption algorithm is lightweighted and fast enough to be effectively used in contact tracing applications.

The rest of the paper is organized as follows. In Sect. 2, previous works which have been done in this field are discussed. In Sect. 3, brief background about LWE problem and geo-encryption is provided. In Sect. 4, the proposed approach with LWE-based cryptography system is discussed followed by describing the various ways in which location parameters in the key generation process can be incorporated. Further, various methods of generating key are also discussed. Finally, in Sect. 5, the performance of the proposed scheme is evaluated followed by the conclusions.

2 Related Work

Contact tracing applications typically involve sending the location-based data of the infected person to a server, so that the data can be analyzed and the people who have been in contact with him or her can be notified about the potential risk. In recent times, there have been some works to analyze what kind of privacy concerns may arise and how they could be tackled. Authors in [16] discussed various types of contact tracing applications and their privacy concerns. The authors have highlighted data leakage problem in this work also. James Bell et al. in [17] analyzed the security concerns about 'TraceTogether' application. The authors analyzed the use of Additive Homomorphic encryption as an effective measure to secure contact tracing. Ni Trieu et al. [18] considered the application where random tokens are exchanged between users' smart phones when they come within proximity of each other. These tokens then, get stored locally on users' phones. These tokens are used to learn if any user has recently been in the contact of an infected person. The authors proposed a two-party private set intersection cardinality-based algorithm where private information is not exchanged between phones. Authors in [19] discussed an approach to secure contact tracing, by using multi-party computation. This approach allows a group of participants to evaluate a particular function. An individual can only learn the final result and could not see the private input of the users. This approach however has a larger run time. Thamer Altuwaiyan et. al. in [20] attempted to secure the information of the user using matching techniques over the encrypted content, with enhancements using the weight-based-matrix. Authors in [21, 22] proposed to collect and store users' data based on proximity-based protocols, which enables two users to match their profile without disclosing any personal information.

Although all of these methods provide secure lightweight encryption, we believe these methods will cease to be productive after the post-quantum period. That is, once the Quantum Computers take over in the near future, these cryptographic schemes will not prove to be a valid solution.

Contribution of the work: To the best of knowledge, the proposed lattice-based Geoencryption scheme has not been used till now for securing data in the contact tracing applications. The proposed lattice-cryptosystem-based scheme can prove to

be an effective answer even in the post-quantum period. The main objective of the work is to design a decryption algorithm that ensures that data is decryptable only at the specific location. Furthermore, this work also proposes a few methods to make the proposed scheme lightweighted which may be used to increase the speed of proposed algorithm, and so that can be effectively used in these applications.

3 Background

Before proceeding to the explanation of the proposed lattice-based algorithm for contact tracing, some key techniques used in the proposed scheme are discussed in this section.

LWE problem introduced by Oded Regev is a versatile basis for cryptographic constructions [23], and the cryptographic constructions based on it are claimed to be secure. The LWE problem asks to recover a secret $s \in Z_n^q$ given a sequence of 'approximate' random linear equations on s, and each equation is correct up to some small additive error. Recovering s from these equations would have been quite easy with Gaussian Elimination Algorithm [24] if there was no additive error, however, introduction of errors makes this problem significantly difficult. Further, the best known algorithms for the lattice problems require $O(2^n)$ time [25, 26]. Since there is no polynomial time quantum algorithm, even quantum computers won't be able to solve this problem. This hardness of LWE problem set the basis for many cryptographic constructions. However, LWE problem can be reduced to many easier problems. This flexibility of creating variants of LWE problem is one of the reasons for its a large number of applications in cryptography. In addition to this, LWE can be implemented efficiently as it involves low-complexity operations (mainly additions).

Next, the main component of the proposed work is *Geoencryption*. Logan Scott and Dorothy E. Denning in [19] have discussed the general mechanism of Geoencryption. Location-based-Encryption is referred to a method of encryption using which an encrypted text is decryptable at the specific location only. Usually, standard encryption algorithms like AES or RES have been used a lot in location-based encryption. Location-based-encryption typically involves, taking a standard cryptography algorithm and *trying* to incorporate the location parameter in it's key. The receiver using its location data to generate a secret key. The decryption is possible only if the "key generated by the receiver" is same as "key sent by the sender". Hence adding an *extra layer of security* in the algorithm. One of the applications of location-based encryption is the effective movie distribution, where the movie is available at only those theaters, which have actually paid for the movie. Further, the main part of the location-based-encryption algorithm is the key generation process where the location is incorporated in the key, however, it should be difficult to retrieve the location back from the key.

Let us understand LWE problem with an example. There is a secret vector $S = (s_1, s_2, s_3, s_4)^T \in Z_{13}^4$. Following equations are correct up to some small additive error.

$$5s_1 + 5s_2 + (-3)S_3 + 7s_4 \approx 6(mod 13)$$

$$(-1)s_1 + 1s_2 + 2S_3 + (-5)s_4 \approx -4(mod 13)$$

$$(-3)s_1 + 3s_2 + 7S_3 + 4s_4 \approx 2(mod 13)$$

$$5s_1 + 4s_2 + (-1)S_3 + 2s_4 \approx -5(mod 13)$$

$$(-4)s_1 + 6s_2 + 3S_3 + (-2)s_4 \approx 5(mod 13)$$

$$(-2)s_1 + 3s_2 + 1S_3 + 6s_4 \approx -3(mod 13)$$

Using these equations, secret vector S is to be solved. In Fig. 1, Challenger is having the secret S and generates LWE samples (A, b) from the LWE distribution $A_{s,n,q,\chi}$. For secret vector $S \in Z_q^n$ and distribution χ, LWE distribution $A_{s,n,q,\chi}$ generates samples $(a,\ b) \in Z_q^n \times Z_q$ where a sampled uniformly from Z_q^n and $b = \langle a, s \rangle + e$ where $e \leftarrow \chi$. Challenger passes LWE samples (A, b) to the adversary and then the adversary tries to find secret $S \in Z_q^n$

Next, a brief introduction of LWE-based encryption and decryption is discussed [23]. For an encryption using LWE, one private key (secret key S), and two public keys (A, B) are required, where a private key is used for decryption of the data and public keys are used for encryption of the data. The various parameters in LWE cryptosystem are as follows:

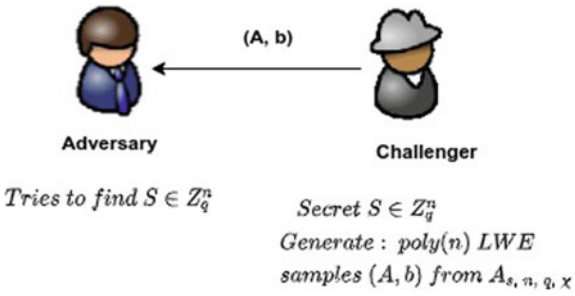


Fig. 1 Learning with errors problem (search)

1. Security parameter - n
2. Number of equations - m
3. Modulus - q
4. Noise parameter - α (real number)

For having both security and correctness, q needs to be a prime number between n^2 and $2n^2$, $m = 1.1 \cdot nlogq$, and $\alpha = \frac{1}{\sqrt{n}\ log_2 n}$.

Key generation, encryption, and Decryption in general Learning with Errors-based Cryptosystem is described as (all additions are performed modulo q)

- *Private Key:* It is vector s chosen uniformly from Z_q^n.
- *Public Key:* The public key consists of m samples $(a_i, b_i)_{i=1}^m$ from the LWE distribution with secret s, modulus q, and error parameter α.
- *Encryption:* For each bit of the message, select a random set S uniformly among all 2^m subsets of $[m]$. The encryption is $(\sum_{i \in S} a_i, \sum_{i \in S} b_i)$, if the bit is 0 and $(\sum_{i \in S} a_i, \left\lceil \frac{q}{2} \right\rceil + \sum_{i \in S} b_i)$, if the bit is 1.
- *Decryption:* The decryption of a pair (a, b) is 0 if $b - \langle a, s \rangle$ is closer to 0 than to $\left\lceil \frac{q}{2} \right\rceil$ modulo q, and 1 otherwise.

4 Proposed Lattice Cryptosystem

Let us consider an user *Alice* comes in contact with an infected person *Bob*. A contact tracing application sends *Alice's* information to a server, say *Servy*. Based on the contact tracing application, the *Servy* then notifies *Alice* about the contact. However, this data may be attacked by a middle-man or hacker, say *Trudy*. The example scenario is shown in Fig. 2. The main objective of the proposed work is to make the location data only available to *Servy*. The data should not be visible to any person who is not in the same *location* as of *Servy*. The proposed work considers an LWE-based scheme to encrypt the data sent by the *Alice* to the *Servy*. The main idea is that the data is only encrypted using a key derived from the location data of *Servy*, thus it can only be decryptable at the *Servy* itself.

The main components of the proposed scheme, the key generation, encryption, and decryption are discussed next.

4.1 Secret Key Generation Using Location Parameters

As mentioned above, the main objective of the work is that the encrypted data can only be decrypted at a particular location (at server). The considered location (or encryption) parameters in the proposed scheme are latitude, longitude, and tolerance Distance of the server. The tolerance distance represents the radius around a particular

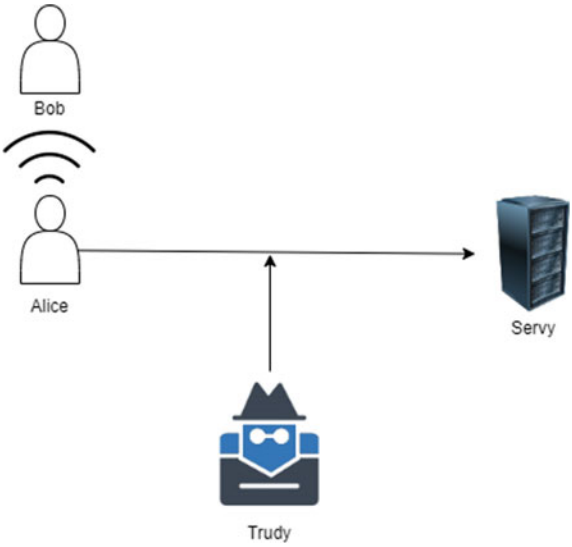


Fig. 2 Overview of contact tracing apps

value of latitude and longitude in which the data can be decrypted. Further, while discussing standard LWE-based encryption, a secret key S is required. Depending on the number of considered parameters, S is a vector of dimensions $1 \times s$. To show the complexity involved in different dimensions, four cases are considered where s can be either 1, 2, 3, or in general n.

1. $[1 \times 1]$: For different locations, different keys must be generated to make the system more secure. Therefore, some hash functions can be used to generate a secret key using latitude, longitude, and tolerance distance.

$$S = (u \times P_1 + v) \times P_2 + t, \tag{1}$$

   where u is latitude, v is longitude and t is the tolerance distance. Further, P_1 and P_2 are selected as some big prime numbers such that chances of collision are minimized.
2. $[1 \times 2]$: Consider three constants or weights a, b, and c, which are multiplied with u, v, and t respectively to get the normalized results. For, the first value of S, S_1 is defined as

$$S_1 = \frac{au}{au + bv}, \tag{2}$$

   and S_2 is defined as

$$S_2 = \frac{bv}{au + bv}. \tag{3}$$

3. $[1 \times 3]$: Here the above process is extended by including tolerance to generate three values. For, the first value of S, S_1 is defined as

$$S_1 = \frac{au}{au + bv + ct}, \tag{4}$$

S_2 is defined as,

$$S_2 = \frac{bv}{au + bv + ct}, \tag{5}$$

similarly, S_3 is defined as,

$$S_3 = \frac{ct}{au + bv + ct} \tag{6}$$

This process can be combined with standard key generation algorithms like Password-Based Key Derivation Function 2 (PBKDF2)[27].

4. $[1 \times n]$ *or Generalized Secret Key Generation*: To generate a secret key matrix of size $1 \times n$, n random values from the Normal Distribution are selected. The mean of this normal distribution function is the weighted average of latitude, longitude, and tolerance distance. Further, standard deviation will be a constant. Generation of secret key is summarized in Algorithm 1.

Algorithm 1 GenerateSecretKey(u, v, t, n)

Assumptions:

- *Random Class is a Class for generating random variables.*
- *Random class also contains functions for generating numbers using some probability distributions like Normal Distribution.*
- *A seed value can be set for an instance of a random class such that the same pseudo-random numbers may be generated again, if required in future.*
- *Implementation of this class may vary in different programming languages.*

string *u: Latitude of the location*
string *v: Longitude of the location*
string *t: Tolerance of the location*
int *n: Length of the secret key vector*

1: Begin
2: Initialize **r** with some random value
3: Initialize Standard Deviation **S.D** for normal distribution with a constant value
4: Calculate **mean** as weighted average of latitude, longitude and tolerance:

$$mean = \frac{au}{au + bv + ct},$$

where a,b,c are weights
5: Get an Instance of Random class: **rand = new Random()**
6: Set seed of instance as **r** so that same random numbers can be generated again if required in future: **rand.seed(r)**
7: Call Normal Distribution of Random class with parameters mean, S.D, n: **S = rand.normal(mean, S.D, n)**
8: **Output:** Secret Key, S (a vector of size $n \times 1$)

4.2 Generating Public Key

After the secret key $S_{n\times 1}$ of size $n \times 1$ is generated, public key $A_{m\times n}$ of size $m \times n$ needs to be generated. In the proposed scheme, public key $A_{m\times n}$ is created by generating a random matrix of $m \times n$. Then, public key $A_{m\times n}$ is multiplied with secret key $S_{n\times 1}$ and the result is added with error $e_{m\times 1}$ to get public key $B_{m\times 1}$. The error to be added is a matrix of size $m \times 1$ which follows distribution χ.

$$B = (A \times S + e)\,\%q \tag{7}$$

4.3 Encryption

Data that needs to be transmitted is first converted into a binary format. Then binary data is encrypted using different keys. Two public keys A and B are generated, where A and B are matrices. Further, a value q is selected which is a prime number used for modulus operation. Secret key S (a private key) is generated using geo-encryption that considers three parameters latitude, longitude, and tolerance. Then using A, B, and q each bit is converted to (u, v) pairs, and hence, encryption is completed.

The public key consisting of m samples $(a_i,\ b_i)_{i=1}^{m}$, has 2^m possible subsets. For every bit of the message, a random set F is selected uniformly among all the 2^m subsets. For generating (u, v) pair for every bit, following LWE scheme as discussed in [23] is followed (in which all additions are performed modulo q):

1. *When bit value is 0*: For every (a_i, b_i) pair in the set F, summation of all a_i is done to generate u and summation of all b_i is done to generate v.

$$(u, v) = \left(\sum_{i\in F} a_i, \sum_{i\in F} b_i\right). \tag{8}$$

2. *When bit value is 1*: For every (a_i, b_i) pair in the set F, summation of all a_i is done to generate u. For generating v, summation of all b_i is added with $\left\lceil \frac{q}{2} \right\rceil$, where q is a modulus number.

$$\left(\sum_{i\in F} a_i, \left\lceil \frac{q}{2} \right\rceil + \sum_{i\in F} b_i\right). \tag{9}$$

After each bit is converted to (u, v) pair, encryption is completed and this encrypted data is then, ready to be transmitted from *Alice* to the *Servy*. Further, the seed value used for secret key generation, modulo number q, and location tolerance value are also required to be send to the *Servy* along with the encrypted data. The proposed encryption process is summarized in Algorithm 2.

Algorithm 2 Encryption()

1: Begin
2: Generate secret key S using location parameters.
3: Generate public key A using Random sampling.
4: Select modulo number q.
5: Generate public key B, as given in (7).
6: Convert data to binary format.
7: Convert each bit of binary data to (u, v) pair using encryption scheme.
8: **OUTPUT:** Encrypted Data.

4.4 Decryption

Alice (sender) sends data to the *Servy*(receiver) in the encrypted form. For decryption to happen on the *Servy's* side, *Servy* also receives some parameters mentioned above, through the secure channel. One trivial mathematical operation which can be used to make data decryptable only at *Servy's* location is using XOR operation and is shown in Fig. 3. In the proposed approach, a decryption scheme discussed below is followed for converting each (u, v) pair in the encrypted data to its binary form. After conversion of whole encrypted data into the binary format, conversion is done to retrieve the original text/data. The decryption scheme used to check if the data can be decrypted at location (u,v) is as follows:

1. Location of *Servy* is fetched using a location device. Let's consider this location value to be (x, y), where x is the latitude and y is the longitude.
2. The location of the *Servy* (x, y), is then added with location tolerance value (t) (received from *Alice*) to generate say P different samples of the locations such that $(x, y) + t = (x_i, y_i)_{i=1}^{P}$.
3. For every location $(x_i, y_i)_{i=1}^{P}$, $[(x_i, y_i), t]$ pair is used to generate a secret key $S^{'}$ using the same scheme employed on the *Alice's* side.
4. The following scheme is used to generate a secret key S' using (x_i, y_i) [23]:
   – For secret key S' and pair (u, v) in the encrypted data, calculate $v - \langle u, S' \rangle$

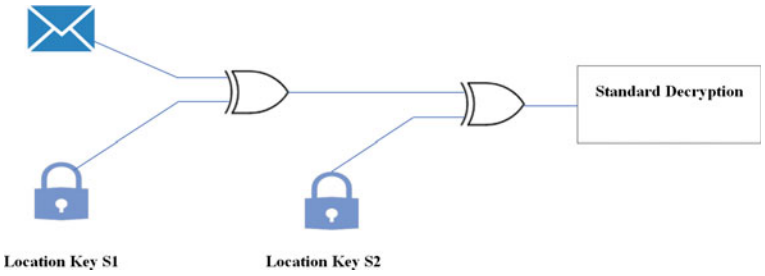


Fig. 3 Standard location-based encryption using XOR gate

- If $v - \langle u, S' \rangle$ is closer to 0 than to $\left\lfloor \frac{q}{2} \right\rfloor$ modulo q, the pair (u, v) is decrypted as 0.
- If $v - \langle u, S' \rangle$ is closer to $\left\lfloor \frac{q}{2} \right\rfloor$ modulo q, the pair (u, v) is decrypted as 1.

5. This secret key $S^{'}$ is used for decryption of the encrypted data. If decryption fails for $S^{'}$ then, other secret keys generated using $(x_i, y_i)_{i=1}^{P}$ is used. If *Servy* is in correct desired position, data would get decryptable for some S' generated using (x_i, y_i). If data doesn't get decrypted for any of the generated secret keys, that means the location of the *Servy* is invalid, i.e., *Servy* is not at correct desired location for decryption of the data to take place.

Summary of proposed decryption algorithm is shown in Algorithm 3.

Algorithm 3 Decryption()

```
Begin
Fetch Servy's location (x, y).
Set (x, y) + t = (x_i, y_i)_{i=1}^{P}.
Set i = 1.
for i ≤ P do
    S' = (x_i, y_i) + t .
    Try Decryption using S'.
    if Success then
        BREAK
    end if
end for
if Still Decryption fails then
    Invalid Location of Servy.
else
    Convert each (u, v) pair in encrypted data into binary format.
    Convert to Original data.
end if
OUTPUT: Encrypted Data.
```

5 Performance Analysis

In this section, analysis of the proposed algorithm on various parameters is done.

5.1 Time Complexity Analysis

The performance speed of the algorithm relies on two major parts, first is the key generation part, and second is matrix or vector creation, which is further used for

subsequent calculations with other matrices A and B. Both of these scenarios are described separately below, and the final complexities are inferred.

Key Generation Phase In Sect. 4.1, various ways of generating key for the proposed algorithm like generating a $[1 \times 1]$, $[1 \times 2]$ or $[1 \times N]$ key have been discussed. Overall, the complexity varies as the N varies. The method, of generating keys for $N \leq 3$ requires O(1) time, however, as the value of N increases so does the time complexity of key generation algorithms. In the method, for $N > 3$, taking the help of a normal distribution, generating key for specific values could take up to O(N).

Calculations After Key Generation Once the key is generated, matrix calculations at encryption and decryption stages are required to be performed. It is observed that this T varies with a cubic factor as N increases, or as $T \propto N^3$. The reason of dependency is that as the N increases, so does the dimensions of the corresponding matrices. The reason being that the standard matrix generation algorithms take a time and space complexity of $O(N^3)$.

5.2 Security Analysis

Security on the Basis of Structure of Lattice In lattice cryptography, considered lattice is a regularly spaced grid of points or vectors, and also can be extended to infinity. However, since there is a memory limit, *lattice basis* [28] (collection of vectors that can be used to reproduce any point in grid that forms lattice) is used in the proposed work to define lattice, which produces the final grid. Moreover, to increase the security of the proposed algorithm the dimensions of the *lattice basis* can be increased as much as possible to enhance security. However, increasing too much dimensions will also increase the required number of mathematical operations, which will finally increase the complexity. Therefore, there is a trade-off between dimension and security and one has to make the balance.

On Basis of Secret Key For generating a secret key, location parameters are used in the proposed work, which includes latitude, longitude, and tolerance discussed in Sect. 4.1. Further, location is converted into a binary format, and then PBKDF2 is applied, which is a simple cryptographic key derivation function. This function takes parameters as binary data, salt or seed, and counts. Salt is randomly generated bytes which is generally 16, 32, 64, or 128 bytes long. The proposed work considers 32 bytes salt, and this size can be further increased to enhance the security as total number of possible combinations will also be increased. However, it will also increase the computation time slightly. Further, count parameter which denotes the number of iterations, to generate the key can also be manipulated accordingly.

Compromise Between Speed and Security One of the objectives of the algorithm is to provide a good and efficient mechanism for contact tracing, which is used in mobile applications. Therefore, it is of utmost importance that the algorithm should be lightweight, at the same time since location data is involved security should not

be compromised. With some experimentation, it is found that the equilibrium point for both to work efficiently is, by using a key of size $N \in [3, 10]$. It can be deduced that the key is generated for $N \leq 5$ using various $O(1)$ methods. However, as the size for N increases it is better to use some uniform probability distributions, like Normal distribution, as discussed in the previous section.

5.3 Possible Optimization

For the decryption process, two possible cases can be considered. One without considering the tolerance distance and the other with the tolerance distance. In the former, the region or space in 3D, where the decryption is possible is determined by the accuracy of the GPS. However, this is not the same in every scenario, as one GPS may vary from the other.

In the other case where tolerance distance is considered, each point in the hypothetical sphere or circle (if only two dimensions are considered) formed by the tolerance distance is searched. Further, certain optimization can be made here. Instead of dividing the circle, into fixed size squares, and searching in it; a lot of computational time may be saved, if the size of squares is less, near the center, and increases as we move outwards from the center. Thus, there will be more density of search points at the center and this will decrease as we move out.

Another factor that can decide the density of the squares is the accuracy of the receiver. If the considered GPS has low accuracy, it is better to search more rigorously in a larger part, otherwise, it is sufficient to search in the areas closer to the center (Fig. 4).

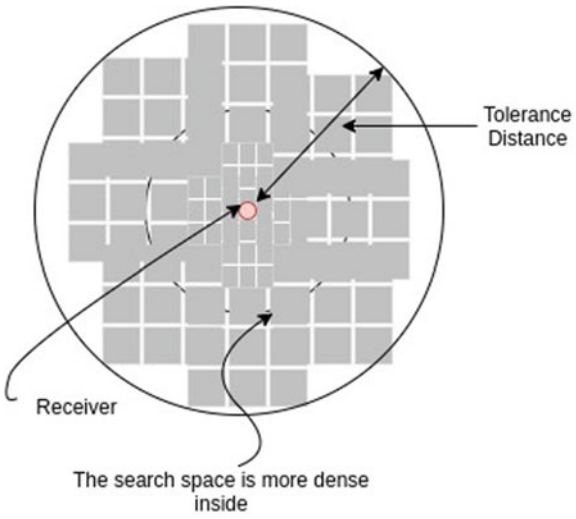


Fig. 4 Example of dynamic search space. The square size is low and thus density high inside the inner circle. The inner circle radius can be varied

6 Conclusions

In the Covid-19 pandemic, one of the effective ways to tackle the infection is to identify the infected person and to alert all the nearby persons who could possibly come in the contact with that person. A large number of contact tracing applications are available worldwide to meet this objective. In this work, a secure cryptographic algorithm for contact tracing applications has been discussed. The proposed method can effectively be used even in the post-quantum period. The proposed scheme can be effectively applied to all the contact tracing applications where private data of the user is sent to the server for contact tracing of an infected person. The analysis shows that the proposed scheme is lightweighted and effective in preserving privacy as data of the user can only be decrypted at the server's location and hence data leakage in between can be well avoided. For the future work, mobile server can be considered, where location data is directly transferred between two mobile devices. Hence, location data and coordinates need to be fetched in real time. Further, multiple server scenarios may be considered, data fragments are sent to different servers for parallel processing. This would require an altogether different mathematics for key generation as the keys have to be different for different servers and yet should contain some information about the fragment being sent to that server. Although there are numerous schemes that entirely skip the server, and thus are more secure. Yet we developed the algorithm in this form so that it can secure those applications (like Indian application, Aarogya Setu) which already involve sending data to the server, and entirely skipping server is not a viable option. The main effort of this work is to help mankind fight the pandemic, which had already taken many lives, and is not showing any signs of decrease in numerous regions.

References

1. Zu ZY, Jiang MD, Xu PP, Chen W, Ni QQ, Lu GM, Zhang LJ (2020) Coronavirus disease 2019 (covid-19): a perspective from china. Radiology, p 200490
2. Rohila VS, Gupta N, Kaul A, Sharma DK (2021) Deep learning assisted covid-19 detection using full ct-scans. Internet Things 14:100377
3. A J H University (2020) Covid-19 dashboard by the center for systems science and engineering (csse) at johns hopkins university (jhu)
4. Centre For Disease Control and Prevention (2020) Symptoms of coronavirus
5. Rohila VS, Gupta N, Kaul A, Ghosh U (2021) Towards framework for edge computing assisted covid-19 detection using ct-scan images. In: ICC 2021-IEEE international conference on communications. IEEE, pp 1–6
6. Azad MA, Arshad J, Akmal A, Abdullah S, Ahmad F, Imran M, Riaz F (2020) A first look at contact tracing apps. arXiv:2006.13354
7. Ish P, Agrawal S, Goel AD, Gupta N (2020) Covid19 contact tracing: unearthing key epidemiological features of covid-19. In: SAGE open medical case reports, vol 8, p 2050313X20933483
8. Contact tracing at the times of spanish flu. https://csahq.org/news/blog/detail/csa-online-first/2020/04/06/history-and-covid-19-part-i-lessons-from-1918
9. Ng PC, Spachos P, Plataniotis KN (2021) Covid-19 and your smartphone: Ble-based smart contact tracing. IEEE Syst J 15(4):5367–5378

10. Aarogya setu by Indian government. https://www.mygov.in/aarogya-setu-app/
11. Trace together by Singapore goverment. https://www.tracetogether.gov.sg/
12. Stayhomesafe application by Honk kong government. https://www.coronavirus.gov.hk/eng/stay-home-safe.html
13. Horvath L, Banducci S, James O (2020) Citizens' attitudes to contact tracing apps. J Exp Polit Sci, pp 1–27
14. Legendre F, Humbert M, Mermoud A, Lenders V (2020) Contact tracing: an overview of technologies and cyber risks. arXiv:2007.02806
15. Micciancio D, Regev O (2009) Lattice-based cryptography. In: Post-quantum cryptography. Springer, pp 147–191
16. Cho H, Ippolito D, Yu YW (2020) Contact tracing mobile apps for covid-19: privacy considerations and related trade-offs. arXiv:2003.11511
17. Bell J, Butler D, Hicks C, Crowcroft J (2020) Tracesecure: towards privacy preserving contact tracing. arXiv:2004.04059
18. Trieu N, Shehata K, Saxena P, Shokri R, Song D (2020) Epione: lightweight contact tracing with strong privacy. arXiv:2004.13293
19. Scott L, Denning DE (2003) A location based encryption technique and some of its applications. ION Natl Tech Meet 2003:730–740
20. Altuwaiyan T, Hadian M, Liang X (2018) Epic: efficient privacy-preserving contact tracing for infection detection. In: IEEE international conference on communications (ICC), pp 1–6
21. Zhang R, Zhang J, Zhang Y, Sun J, Yan G (2013) Privacy-preserving profile matching for proximity-based mobile social networking. IEEE J Sel Areas Commun 31(9):656–668
22. Li M, Yu S, Cao N, Lou W (2013) Privacy-preserving distributed profile matching in proximity-based mobile social networks. IEEE Trans Wirel Commun 12(5):2024–2033
23. Regev O (2009) On lattices, learning with errors, random linear codes, and cryptography. J ACM (JACM) 56(6):1–40
24. Grcar JF (2011) Mathematicians of Gaussian elimination. Not AMS 58(6):782–792
25. Ajtai M, Kumar R, Sivakumar D (2001) A sieve algorithm for the shortest lattice vector problem. In: Proceedings of the thirty-third annual ACM symposium on theory of computing, pp 601–610
26. Micciancio D, Voulgaris P (2013) A deterministic single exponential time algorithm for most lattice problems based on voronoi cell computations. SIAM J Comput 42(3):1364–1391
27. Josefsson S (2011) Pkcs# 5: Password-based key derivation function 2 (pbkdf2) test vectors. In: Internet engineering task force (IETF), RFC Editor, RFC, vol 6070
28. Peikert C (2016) A decade of lattice cryptography. Found Trends® Theor Comput Sci 10(4):283–424

Beamforming Technique for Improving Physical Layer Security in an MIMO-OFDM Wireless Channel



Somayeh Komeylian, Christopher Paolini, and Mahasweta Sarkar

Abstract The open nature of the wireless medium leads to the growing technical literature on solutions and techniques for increasing the privacy and security of a wireless communication channel. This work focuses on implementing the beamforming technique incorporated with a combination of MIMO-OFDM techniques for increasing physical layer security in the downlink of a wireless communication channel. The rectangular antenna array has been employed at the Tx side of an MIMO-OFDM channel with two orthogonal polarizations. This results in an improvement in the downlink performance due to a reduction in a duration of the channel sounding procedure compared to its uplink counterpart. We have used the WINNER II module from MathWorks' MATLAB to simulate a realistic wiretap channel including Alice (to represent a sender), Bobs (to represent users), and Eve (to represent the information leakage). We have numerically estimated the success rate of Eve and physical layer security with performing a beamforming technique, compared to the available techniques without any beamforming technique, in an MIMO-OFDM wireless channel. Although physical layer security is affected by power, we have demonstrated that the beamforming technique with the technique of assigning subcarriers for Bobs can significantly improve the physical layer security and thereby overcome information leakage in the wiretap channel.

Keywords MIMO · OFDM · Physical layer security · Beamforming and beamsteering · Wiretap channel

S. Komeylian
Electrical and Computer Engineering, Concordia University, Montreal, QC, Canada
e-mail: skomeylian81@gmail.com

C. Paolini (✉) · M. Sarkar
Electrical and Computer Engineering, San Diego State University, San Diego, CA, USA
e-mail: paolini@engineering.sdsu.edu

M. Sarkar
e-mail: msarkar2@sdsu.edu

© The Author(s), under exclusive license to Springer Nature Singapore Pte Ltd. 2023

S. Chinara et al. (eds.), *Advances in Distributed Computing and Machine Learning*,
Lecture Notes in Networks and Systems 660,
https://doi.org/10.1007/978-981-99-1203-2_11

1 Introduction

The open nature of the wireless communication channels leads to their vulnerability and susceptibility in being sensitive to the leakage of information. Various interesting and exciting solutions and techniques have been employed for addressing challenges in privacy and security in wireless networks [1–11]. Conventional techniques can be susceptible to technological imperfections, over-exposed to eavesdroppers, and dependent on complicated mathematics, leading to failure for implementing a secure wireless communication channel. Some other approaches and solutions for increasing security and privacy in a wireless communication channel have relied on cryptographic techniques at the upper layers in the open systems interconnection (OSI) model of the wireless channel [4–8]. Physical layer methods allow for distributing secret keys and thereby supplying location security and secrecy [5–8] rather than implementing any additional wireless channel. The implementation of physical layer security provides more constraints for eavesdroppers to decipher transmitted information.

In this work, we have deployed a beamforming technique for achieving a higher rate of transferring data and improving the quality and performance of MIMO-OFDM wireless communication channels, as well as supporting a large number of simultaneous users [11–15]. Since individual users are efficiently targeted by beamforming and beamsteering techniques, the overall capacity of the wireless channel has been increased. Beamforming technique computes the scalar product between measured data and the steering vector of a rectangular array [16–19] to compute the position and power of the source, as illustrated in Fig. 1. Although the security capacity is constrained by the transmission power, channel parameters, and noise, we have verified that the beamforming technique has a significant improvement in the security capacity and a drastic reduction in the leakage of information. In other words, since the

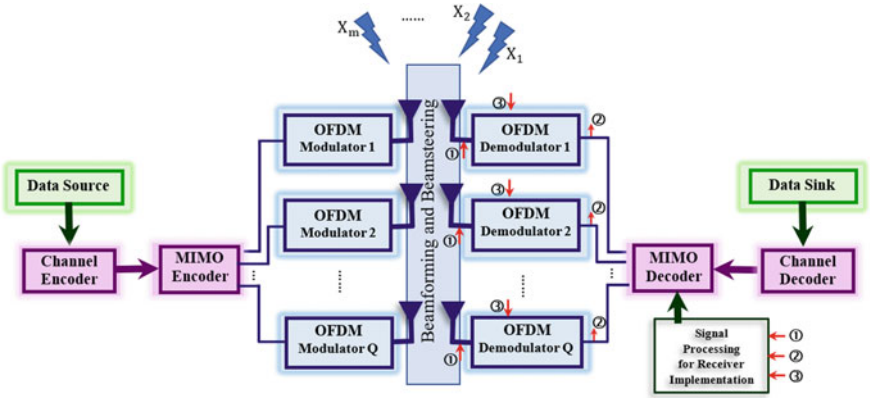


Fig. 1 The MU-MIMO and OFDM model [1–4, 20]. In this work, the rectangular antenna arrays consist of 32 transmitters and 16 receivers

MIMO technique employs rectangular antenna arrays, the beamforming technique is adopted to enhance the received signal-to-noise ratio which, in turn, decreases the bit error rate (BER).

Extensive numerical simulations are required to implement and estimate the proposed techniques in a realistic wireless communication channel. The wireless channel model of WINNER II in the MATLAB programming platform enables us to simulate line-of-sight (LoS) and non-line-of-sight (NLoS) propagation conditions.

Consequently, in addition to the introduction, this work is organized into the four following sections: we have proposed a beamforming technique incorporated with the physical MIMO-OFDM channel in Sect. 2. In Sect. 3, we have implemented the proposed model in the previous section to overcome eavesdropping leakages by illegal users. Section 4 presents the performance evaluation of the proposed model in the previous sections in terms of the physical layer security and the success rate of the eavesdropper. The conclusions of this work are presented in Sect. 5.

2 Theoretical and Methodological Backgrounds

The discussion in this section focuses on providing the theoretical framework of implementing the beamforming for a MIMO-OFDM wireless channel. The effective channel for estimating receivers is a combination of the beamformer and the physical MIMO channel in the frequency domain when the beamforming technique is performed.

The beamforming technique is employed for the direction of arrival estimation in the following expression:

$$F(\alpha) = h(X)^H \overset{\wedge}{\sum} h(X), \tag{1}$$

where $h(X)$ refers to the steering vector of the rectangular antenna array for a set of positions of users. Also, $\sum^{\wedge}$ includes a sample of the covariance matrix. $F(\alpha)$ becomes maximum at the actual location of the source. In practice, the measured signals are obtained by computing the short-time Fourier transform, which we call *snapshots*. Therefore, the covariance matrix of the data can be expanded in terms of m snapshots of P_m by the given equation,

$$\overset{\wedge}{\sum} = \frac{1}{m} \sum_{m=1}^{M} P_m P_m^H. \tag{2}$$

The proposed MIMO-OFDM wireless channel in the frequency domain consists of L_t transmitter antennas, L_r receiver antennas, and D subcarriers. The discrete-

frequency signal model of the effective MIMO channel after beamforming at the kth subcarrier and lth time interval can be expressed by

$$\bar{Y}(k,l) = \mathbf{C}(k)\mathbf{H}(k)\bar{X}(k,l) + \bar{N}(k,l), \tag{3}$$

where $C(k)$ and $H(k)$ represent the frequency domain of the MIMO channel and beamformer, respectively. $N(k,l)$ refers to noise. The term $C(k)H(k)$ shows the effective MIMO channel after performing the beamforming technique.

OFDM should be rigorously incorporated into antenna arrays at both transmitter and receiver sides to boost the system capacity on time-variant and frequency-selective channels. The multicarrier system of OFDM can efficiently perform time-domain modulation and demodulation using an inverse FFT (or IFFT) and an FFT, respectively. Hence, outputs of the IFFT operation consist of time-domain samples of the transmitted waveform, while the transmitted data consists of frequency-domain coefficients. Stuber, Gordon L. et al. in [20] have described in detail the theoretical and mathematical background of the MIMO-OFDM model. OFDMA divides a channel into multiple resource units (RUs), which are employed for different stations, simultaneously. Resource dimensions include time, frequency, and space. OFDMA determines the division length of the channel in the frequency domain, for instance, each 20 MHz-CH. It is worth mentioning that MU-MIMO spatial streams are placed on OFDMA RUs rather than on the whole bandwidth of the operation. In other words, the MU-MIMO OFDMA should be loaded into the wireless channel using frequency/spatial streams. Hence, within a certain observation time, an associated number of spatial streams is placed on each RU in a division length of the wireless communication channel. Downlink MU-MIMO allows the access point (AP) to convey spatial streams on the same frequency and to serve 16 stations per RU, simultaneously.

In this work, the MU-MIMO technique consists of rectangular antenna arrays on the transmitter and receiver sides with 32 transmitters and 16 receivers. The proposed channel allows the AP to collect the beamforming information of the locations of stations as well as to transmit the spatial/frequency streams toward the accurate direction of receivers.

3 Wireless Channel Model

In this section, we aim at deploying the proposed wireless channel in the previous section to overcome leakages of illegal eavesdroppers. Channel state information (CSI) provides information about how signals propagate from the transmitter to the receiver under realistic conditions of the wireless communication channel including power decay with distance, fading, and scattering. In Fig. 2, Alice (sender) intends to convey the message of X to the legitimate Bobs (users) through the wireless communication channel, $V : X \to Y$. The illegal Eve (or eavesdropper) can have access to the message that Alice wants to transmit through the wiretap channel,

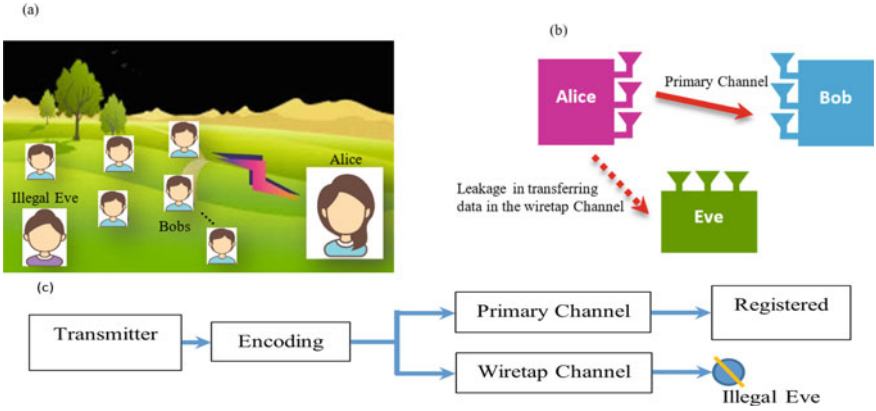


Fig. 2 (**a**) The legitimate transmitter (Alice) and receivers (Bobs) communicate over a discrete memoryless wireless channel. Alice is interested in transmitting a message to Bobs reliably, and in keeping the message confidential from any Eve, (**b**) members of the primary and wiretap channels, and (**c**) demonstration of the principle of the physical layer security of wireless communication channel [4]

$W : X \rightarrow Z$. Alice should keep the message hidden from the eavesdropper while maximizing the rate of the transmitted information to Bobs. The secrecy capacity is expressed by

$$C_s = sup(I(X;Y) - I(X;Z)) \; over \; P(x) \in \mathcal{P}, \tag{4}$$

where $\mathcal{P}$ refers to the class of mass functions of a random variable of X. Hence, the channel capacity is defined by

$$I(X;Y) = \frac{1}{2} log(det(I + H^{'} \sum H)), \tag{5}$$

where H and $\sum$ refer to the channel gain and the covariance matrix of the input signals of X, respectively. Equations 1 and 2 should be expanded for the combination of MIMO-OFDM, techniques, as discussed in detail in [1, 4].

In this work, we have employed the MATLAB programming platform to model practical scenarios of the wireless communication channel. Since MathWorks functions of the MATLAB programming platform have been extensively tested, evaluated, and verified based on IEEE standards and criteria, thereby simulation results obtained through the MATLAB programming provide a very high level of realistic accuracy. Hence, we have implemented the WINNER II channel model with the scattering type from the MathWorks' platform for the MU-MIMO OFDMA technique. Indeed, the wireless channel is modeled with a fixed seed so as to keep the similar channel realization between sounding and data transmission. In reality, the channel would evolve between the two stages.

The evolution of this wireless channel is modeled by prepending the sounding signal to the data signal to prime the channel to the same valid state for data transmission, and then ignoring the preamble portion from the channel output. The channel computations and calculations have been fulfilled in an ideal practical scenario, in which there is line-of-sight and non-line-of-sight propagation.

4 Physical Layer Security and Success Rate of the Eve

The discussion in this section has been focused on improving and implementing physical layer security by performing the DL MU-MIMO and OFDM techniques. Since the success rate to decode is significantly affected by the power, electrical beamforming techniques alone cannot overcome the leakage of the information by eavesdroppers in the wiretap channel. Hence, in this work, to control the power, and thereby enhance the physical layer security, we have rigorously performed the beamforming and beamsteering technique for the rectangular antenna arrays in addition to the technique of assigning subcarriers to Bobs. Then, these two techniques have been implemented for the proposed MU-MIMO and OFDMA channel model by the numerical solutions in the MATLAB simulation platform. Compared to the simulation results in [6], Fig. 3 reports better performance in terms of the security rate with performing beamsteering technique and allocating subcarriers to Bobs for the proposed MU-MIMO and OFDMA wireless channel. The success rate of the eavesdropper in terms of the normalized SNRs has been demonstrated in Fig. 4, which shows a better performance to overcome the leakage of the information by the Eve with performing the beamforming technique, compared to the simulation results in [6] without performing any beamforming technique.

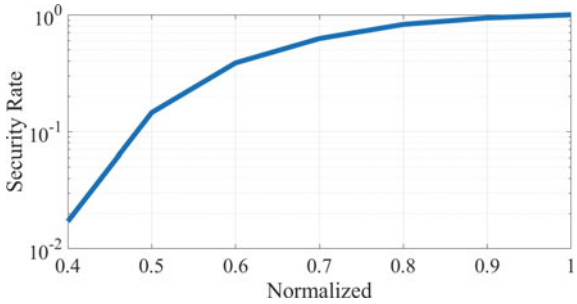


Fig. 3 The variation of the physical layer security in terms of the normalized SNRs. Simulation results have been obtained with performing beamforming techniques for rectangular antenna arrays and assigning subcarriers to Bobs for the MU-MIMO and OFDMA wireless communication channel, consistent with Figs. 1 and 2 and Table 1. $SNR_{\text{normalized}} = \frac{\text{a SNR value normalized}}{\text{sum of all SNR values normalized}} = \frac{SNR_i}{\sum_i SNR}$

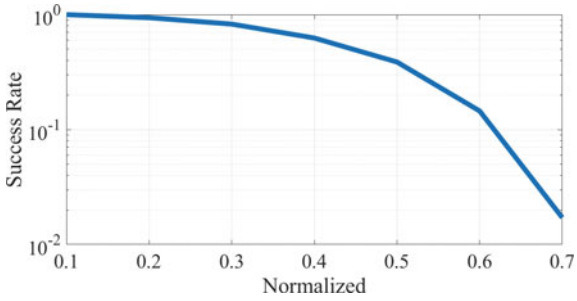


Fig. 4 The variation of the success rate of the eavesdropper in terms of the normalized SNRs. Simulation results have been obtained with performing beamforming techniques for rectangular antenna arrays and assigning subcarriers to Bobs for the MU-MIMO and OFDMA wireless communication channel, consistent with Figs. 1 and 2 and Table 1. $SNR_{\text{normalized}} = \frac{\text{a SNR value normalized}}{\text{sum of all SNR values normalized}} = \frac{SNR_i}{\sum_i SNR}$

Table 1 Channel parameters for the DL MU-OFDM MIMO technique

Channel rate	100×10^6
Carrier frequency	100×10^7
FFT length	256
Cyclic prefix length	64
Number of carriers	234
Bit per subcarrier	4
Data symbols	10
Pilot tons per OFDM symbol	8

5 Conclusion

To conclude, the beamforming technique is incorporated with the combination of the MIMO-OFDM techniques to boost the security and privacy in wireless communication channels. Extensive numerical simulations in the MATLAB programming software have been performed for implementing the proposed technique for the wiretap communication channel. Simulation results of the physical layer security and the success rate of the Eve for the channel model have been shown that we can achieve better performance in terms of overcoming the leakage of information by performing the beamforming technique compared to the methods without implementing any beamforming method.

References

1. Marzetta T, Hochwald B (1999) Capacity of a mobile multiple-antenna communication link in rayleigh flat fading. IEEE Trans Inf Theory 45(1):139–157
2. Foschini GJ, Gans MJ (1998) On limits of wireless communications in a fading environment when using multiple antennas. Wirel Pers Commun 6:311–335
3. Telatar E (1999) Capacity of multi-antenna gaussian channels. Eur Trans Telecommun 10(6):585–595. https://onlinelibrary.wiley.com/doi/abs/10.1002/ett.4460100604
4. Komeylian S, Komeylian S (2020) Deploying an ofdm physical layer security with high rate data for 5g wireless networks. In: 2020 IEEE Canadian conference on electrical and computer engineering (CCECE), pp 1–7
5. Wyner AD (1975) The wire-tap channel. Bell Syst Tech J 54(8):1355–1387. https://onlinelibrary.wiley.com/doi/abs/10.1002/j.1538-7305.1975.tb02040.x
6. Chen Z, Bai P, Li Q, Hu S, Huang D, Li Y, Gao Y, Li X (2017) A novel joint optimization of downlink transmission using physical layer security in cooperative 5g wireless networks. Int J Distrib Sens Netw 13:155014771773822
7. Munisankaraiah S, Kumar AA (2016) Physical layer security in 5g wireless networks for data protection. In: 2016 2nd international conference on next generation computing technologies (NGCT), pp 883–887
8. Moosavi H, Bui FM (2016) Delay-aware optimization of physical layer security in multi-hop wireless body area networks. IEEE Trans Inf Forensics Secur 11(9):1928–1939
9. Komeylian S, Komeylian S (2020) Deploying an ofdm physical layer security with high rate data for 5g wireless networks. In: 2020 IEEE Canadian conference on electrical and computer engineering (CCECE), pp 1–7
10. Komeylian S, Paolini C, Sarkar M (2023) Overcoming an evasion attack on a cnn model in the mimo-ofdm wireless communication channel. In: International conference on advances in distributed computing and machine learning (ICADCML) 2023, India, pp 1–5
11. Komeylian S (2021) Deep neural network modeling of different antenna arrays; analysis, evaluation, and application. IEEE Can J Electr Comput Eng 44(3):261–274
12. Komeylian S (2020) Performance analysis and evaluation of implementing the mvdr beamformer for the circular antenna array. In: 2020 IEEE radar conference (RadarConf20), pp 1–6
13. Komeylian S (2020) Implementation and evaluation of ls-svm optimization methods for estimating doas. In: 2020 IEEE Canadian conference on electrical and computer engineering (CCECE), pp 1–8
14. Komeylian S (2021) Optimization modeling of the hybrid antenna array for the doa estimation. Int J Electron Commun Eng 15(2):72–77. https://publications.waset.org/vol/170
15. Komeylian S, Paolini C (2022) Implementation of a three-class classification ls-svm model for the hybrid antenna array with bowtie elements in the adaptive beamforming application. https://arxiv.org/abs/2210.00317
16. Behar VP, Kabakchiev CA, Gaydadjiev G, Kuzmanov GK, Ganchosov P (2009) Parameter optimization of the adaptive mvdr qr-based beamformer for jamming and multipath suppression in gps/glonass receivers. In: Proceedings of 16th Saint Petersburg international conference on integrated navigation systems
17. Capon J (1969) High-resolution frequency-wavenumber spectrum analysis. Proc IEEE 57(8):1408–1418
18. Rader C (1996) Vlsi systolic arrays for adaptive nulling [radar]. IEEE Signal Process Mag 13(4):29–49
19. Loan CFV (1999) Introduction to scientific computing: a matrix-vector approach using MATLAB, 2nd edn. Prentice Hall
20. Stuber G, Barry J, McLaughlin S, Li Y, Ingram M, Pratt T (2004) Broadband mimo-ofdm wireless communications. Proc IEEE 92(2):271–294

Predictive VM Consolidation for Latency Sensitive Tasks in Heterogeneous Cloud



Chinmaya Kumar Swain, Preeti Routray, Sambit Kumar Mishra, and Abdulelah Alwabel

Abstract Virtualization technology plays a crucial role for reducing the cost in a cloud environment. Efficient virtual machine (VM) packing method that focuses on compaction of hosts such that most of its resources are used when it serves the user requests. Here our aim is to reduce the power requirements of a cloud system by focusing on minimizing the number of hosts. We propose a predictive scheduling approach considering the deadline of a task request and make flexible decisions to allocate the tasks to hosts. Experimental results show that the proposed approach can save around 5 to 10% power consumption than the standard VM packing methods in most scenarios. Even when the total power consumption requirements remain the same as that of standard methods in some scenarios, the average number of hosts required in the cloud environment are reduced and thereby reducing the cost.

Keywords Cloud computing · Heterogeneous · Virtual machine · Consolidation · Power consumption

1 Introduction

With large-scale adoption of cloud system, the essential infrastructure is needed in the form of data centers. As reported by Mastelic et al. [1], the total energy consumption of the cloud infrastructure is up to 1.5% of total energy used in the world. Out of the total energy used by the cloud infrastructure, 6% to 12% of energy

C. Kumar Swain (✉) · P. Routray · S. Kumar Mishra
Department of CSE, SRM University, Andhra Pradesh, India
e-mail: chinmayakumar.s@srmap.edu.in

S. Kumar Mishra
e-mail: sambitkumar.m@srmap.edu.in

A. Alwabel
Department of Computer Sciences, College of Computer Engineering and Sciences, Prince Sattam bin Abdulaziz University, Al-Kharj 11942, Saudi Arabia
e-mail: a.alwabel@psau.edu.sa

© The Author(s), under exclusive license to Springer Nature Singapore Pte Ltd. 2023

S. Chinara et al. (eds.), *Advances in Distributed Computing and Machine Learning*, Lecture Notes in Networks and Systems 660,
https://doi.org/10.1007/978-981-99-1203-2_12

used to provide power to cloud servers for the computation purpose and rest of the electricity was simply wasted [2, 3]. To reduce the power consumption and better resource utilization, virtualization technology is being used. In virtualization, a single physical machine (PM) can deploy a number of virtual machines (VMs) to run concurrently. In cloud computing, many user requests can be handled by a single PM by using virtualization in which each user request is mapped to a VM. Virtualization also provides security and isolation to different user requests, which is considered as an integral part of cloud infrastructure. However, if virtualization results in sparse allocation of user requests onto physical machines, then it will result in more number of physical machines to be used and that leads to high power consumption and low resource utilization.

In heterogeneous environment, more number of active PMs results in more power consumption, the focus was to minimize the use of active PMs. This can be done by making active PMs at any instant as compact as possible which will result in less power usage thereby saving energy. For compaction, the system needs to allocate the VMs to PMs efficiently and migrates the VMs at run time to improve the compaction. VM migration helps in VM packing methods, where a VM can be shifted to a different PM during its execution[4, 5]. The VM packing problem, which is a variant of the bin-packing problem, is NP-hard in strong sense [6]. So there is a need of designing heuristics to solve the VM packing problem in real-time.

The most challenging aspect of the cloud environment is that the future user requests are not known in advance. So, that causes the wastage of resources by deploying large number of PMs. So the scheduling approach which can predict the future resource requirement will be able to minimize the resource wastage. Current state-of-the-art approaches for VM packing focus either on minimizing the number of PM deployments [7] or on minimizing the number of VM migrations [8]. Both the cases the objective is to reduce the cost and energy.

The major contributions of the proposed work are as follows:

1. Resource prediction model to predict the future resource requirement based on the previous resource usage, which will helps to deploy the required number of PMs for future VM allocation.
2. In this VM packing method, we design efficient heuristic where VMs are not immediately allocated to PMs rather consider future demand of task requests and take appropriate decision for final allocation of VMs to PMs.
3. Slack aware heuristics are proposed to efficiently allocate the latency sensitive tasks in heterogeneous cloud environment.

2 Related Work

VM consolidation is well-known optimization problem in cloud environment. Wu et al. [9] proposed the approach to reduce the energy consumption and migration in their work. They used genetic algorithm based approach to reduce the number of

VM migrations. In [10] Ashraf et al. proposed the ant-colony optimization algorithm to reduce the number of VM migrations in heterogeneous cloud environment. Beloglazov et al. [11] proposed threshold-based VM migration when a cloud server is overloaded with more number of requests. Once the server utilization goes beyond the threshold level, then the VM will be migrated to other servers which are underloaded.

Farahnakian et al. [12] proposed threshold-based VM migration based on the server utilization. They use linear regression-based approach to predict the CPU utilization and allow the overloaded CPU to migrate some the VMs to reduce the load and SLA violations. In the work [13], Haghshenas and Mohammadi proposed new regression technique to predict the CPU utilization and allocate the VMs to servers with higher utilization. Suhib et al. [14] proposed Markov prediction model to predict the server utilization and status about overloaded servers. That prediction model is being used to make efficient VM migration strategy in cloud system.

A new iterative budget-based algorithm was proposed by Laili et al. [15], which reduces the cost of migration and communication for the overloaded servers. The heuristic strategy uses multistage approach for efficient scheduling of VMs on servers. Sharma et al. [16] proposed the VM consolidation approach to handle the unreliable servers. They consider the real-time monitoring of the tasks and servers allocations for VM consolidation. In the paper Swain et al. [17, 18] proposed the scheduling of bag of real-time tasks to handle the failure of the cloud servers. They propose the efficient heuristic to improve the reliability through replication and repetition of the task allocation to servers. Jheng et al. [20] proposed a prediction model to predict the CPU utilization of the server, but their model unable to guarantee the prediction accuracy due to fluctuations in workload. ARIMA model was proposed by Chehelgerdi-Samani and Saf-Esfahani [19], where they developed to detect the overloaded servers during VM allocation and make proper VM consolidation to reduce the number of VMmigrations.

The previous work on VM packing methods worked on algorithms that would try to allocate a VM request directly when it arrives and not take into account the deadline which can provide us a window of timeline where the allocation can be done. Also, VM migrations are allowed which moves VM from one host to another. In our work, we do not allow VM migrations at all and instead focus on the slack time available to us to make allocations in the future and also modify the future allocations if a larger VM request comes in the near future that can replace the currently allocated request.

3 Problem Formulation

3.1 System Model

In this work, we consider m number of PMs available to serve n number of VM requests. The set of PMs $M = \{M_1, M_2, M_3, \ldots, M_m\}$ are heterogeneous in their configuration. The configuration of each PM specify the resources available for

usage, i.e., CPU, memory, network bandwidth, disk space, etc. Here we consider only two types of resources, i.e., CPU and memory. So the capacity tuple for any PM, M_j is represented by $C_j = \langle C_j^{cpu}, C_j^{mem} \rangle$, where C_j^{cpu} represents number of available CPUs and C_j^{mem} represents RAM capacity of the PM. For simplicity, we build our model by taking two resources; however our approaches can be applicable for any number of resources.

The set of task requests that arrived to the system can be represented as $T = \{T_1, T_2, T_3, \ldots, T_n\}$. Each task from the task set is assigned to a dedicated VM which is represented by its memory and CPU requirement. The set of VMs generated corresponding to each task from the task set can be represented as $V = \{V_1, V_2, V_3, \ldots, V_n\}$. It is assumed here that VMs are independent of each other and do not share the allocated resources. Each VM can be represented by a tuple Q and for VM_i, the request tuple is $Q_i = \langle a_i, e_i, d_i, u_i, m_i \rangle$, where a_i represents the arrival time, e_i represents the worst case execution time, d_i represents the deadline of the task, u_i represents the CPU requirement, and m_i represents the memory requirement. The actual execution time, CPU usage, and memory usage of VM may be less than the requested.

Here our assumption is that a VM instance is dynamically created based on the resource request of each task, which may not be the same as predefined VM instances like proportionate requirements of CPU and memory (For example, EC2 instances like a1.medium, a1.large, a1.xlarge, and etc. [21]). At any point of time, only a fixed number of PMs will be active, known as active set of machines ($\mathbf{M}_a$). $\mathbf{M}_{na}$ denotes the set of inactive machines. The inactive set of machines will be made active whenever active machines cannot handle the new incoming VM requests.

In the cloud system, the power consumption mostly depends on the collective power consumption of its active PMs. The power consumption model of each physical machine can be represented with two components, i.e., static component and dynamic component. Static component of the power consumption is same for each PM and that is independent of how much resources are used by that PM. The resources usage pattern affects the dynamic power consumption of the server. In this work, we use the power consumption model defined in [22]. The power consumption of jth machine P_j is defined as

$$P_j = P_j^s + P_j^d \tag{1}$$

where P_j^d and P_j^s are considered as the dynamic and static component of the power consumption model, respectively. P_j^d used in our the model defined in [23].

$$P_j^d = \left(\kappa_1 . U_j^{cpu} + \kappa_2 . U_j^{mem}\right) . P_j^{dmax} \tag{2}$$

where U_j^{cpu} and U_j^{mem} represents the fraction of total CPU and memory used respectively. The terms κ_1 and κ_2 are positive constants and $\kappa_1, \kappa_2 \leq 0.5$. The P_j^s is not negligible, and is between 40% and 70% of maximum power consumption of PM. The term P_j^{dmax} represents the maximum dynamic power consumption of jth PM and $P_j^d \leq P_j^{dmax}$. As our considered cloud system consists of number of heteroge-

neous machines, the values of κ_1, κ_2, and P_j^{dmax} are different for different types of machines. The power consumption variations in a PM are due to difference in compute power of the system and other configurations of the system. The summation of the power consumption of individual PMs leads to total power consumption of the cloud system at a particular time.

3.2 Problem Statement

Consider a cloud system with set of heterogeneous PMs and a sequence of VM requests, the proposed approach will allocate the VM requests PMs which will satisfy the following constraints.

(i) Capacity constraint: A VM request i can be assigned to PM j, if the PM has sufficient amount of available resources like memory and CPU throughout the execution of the VM_i on PM_j, i.e.,

$$U_j^{cpu} + u_i \leq C_j^{cpu} \tag{3}$$

$$U_j^{mem} + m_i \leq C_j^{mem} \tag{4}$$

(ii) A VM request is assigned to a PM, if the VM can complete its execution before its deadline.

(iii) VMs are allocated to PMs in a non-preemptive manner, i.e., VM will continue its execution till its completion with the same PM. Here we will not allow the VM migrations and preemptions. However, this constraint can be considered for future improvement of our proposed work.

(iv) The objective of this work is to deploy minimum number of PMs to be active by making them as compact as possible. Hence that will consume less power and improve the energy efficiency of the whole system.

4 Methodology

4.1 System Architecture

We consider the cloud system with heterogeneous physical machines. Our aim is to allocate online aperiodic VM requests (tasks) to PMs so as to minimize the total power consumption of all the PMs. The architecture of our proposed work is shown in Fig. 1. The n number of task requests are arrived to the system, and each task request is mapped to a VM. The size of these VMs varies according to the resource requests of the concerned task. Based on the resource request of the set of tasks for a time interval, we predict the future resource requirement of the incoming tasks for

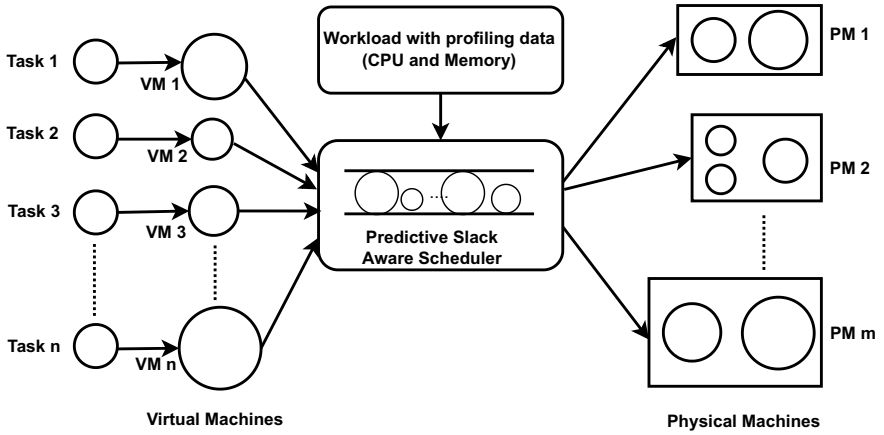


Fig. 1 Overall architecture of the system

the next time interval. This prediction model is used to predict the number of PMs required to be activated in future time.

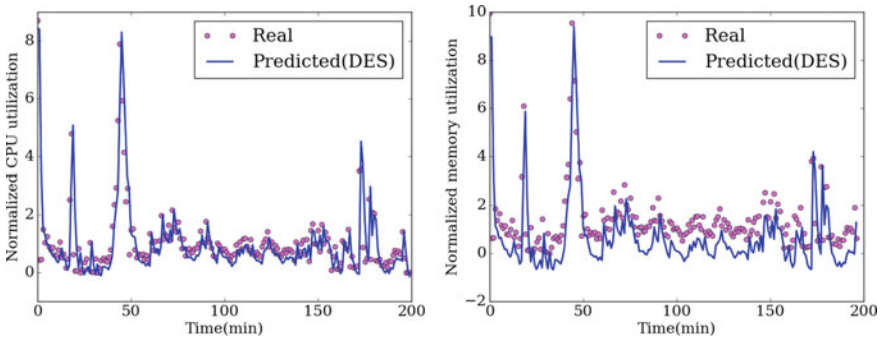
In this approach, we will not promptly schedule the VM to PM rather postpone the decision for some time so as to make PM as compact as possible using the resource prediction model. We made the tentative VM allocation decision which is used for final allocation when task requests arrive to the system dynamically. The tentative allocation gets final allocation, when the VM tentative start time equal to the current time, and once a VM starts its execution it cannot be migrated or preempted.

4.2 Resource Prediction

In the proposed approach, one of the key features is to predict the resource need in future based on the resource usage pattern of the task set of previous time interval. Based on that prediction number of PMs are activated. Here we use double exponential smoothing (DES) to predict the CPU and memory utilization in future time interval [24, 25]. The DES works as follows:

$$E(t) = \begin{cases} O(1) & \text{if } t \leq 2 \\ \alpha \times O(t) + (1-\alpha) \times (E(t-1) + B(t-1)) & \text{if } t > 2 \end{cases} \tag{5}$$

$$B(t) = \begin{cases} O(1) - O(0) & \text{if } t \leq 2 \\ \beta \times (E(t) - E(t-1)) + (1-\beta) \times B(t-1) & \text{if } t > 2 \end{cases} \tag{6}$$



(a) CPU usage (b) Memory usage

Fig. 2 Resource prediction using DES with $\beta = 0.3$ and $\alpha = 0.4$

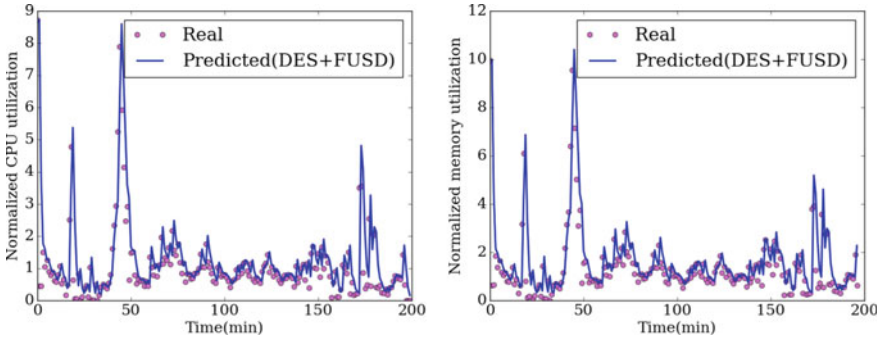
where $O(t)$ represents the observed CPU/memory usage and $E(t)$ represents the estimated CPU/ memory usage at time t. The value of $B(t)$ represents the estimation of trend, α represents the data smoothing factor, $0 < \alpha < 1$, and β is the trend smoothing factor, $0 < \beta < 1$.

Based on the prediction model, the CPU and memory utilization were calculated for every five minutes interval. Figure 2a, b reports the results for CPU utilization and memory utilization respectively with $\alpha = 0.4$ and $\beta = 0.3$. The median error calculated as a percentage of the observed usage value: $|E(t) - O(t)|/O(t)$ and it turn out to be 7.72% for CPU usage prediction and 6.01% for memory usage prediction of a portion of Google cluster data (GCD) [26].

The prediction model reports resource prediction satisfactorily but lack in handling the irregular resource usage pattern. For the same we use Fast Up and Slow Down (FUSD) mechanism (as defined in [27]) along with DES model to handle irregular resource usage pattern. The transformed mathematical model uses the difference between expected and observed resource usage from Eq. 5 and defined as follows:

$$E^{'}(t) = O(t-1) + \gamma \cdot |E(t-1) - O(t-1)|, 0 < \gamma < 1, \tag{7}$$

where $E^{'}(t)$ the new estimated resource usage, $E(t)$ old estimated resource usage (using Eqs. 5 and 6) and $O(t)$ represents the observed resource usage. The value of $E(t)$ and $O(t)$ was collected from Eq. 5 and used with the transformed Eq. 7. Figure 3a, b reports the resource usage prediction of CPU and memory respectively using Eq. 7, i.e., double exponential smoothing cascaded with FUSD [28]. The new model handles the irregular changes of increasing and decreasing of resource usage over time.



(a) CPU usage (b) Memory usage

Fig. 3 Resource prediction using modified DES with $\beta = 0.3$, $\alpha = 0.4$, and $\gamma = 0.4$

4.3 Energy Efficient Resource Management

The prediction model uses the resource utilization of set of tasks in current time interval (t) and predicts resource requirement for the next time interval (t' or t''). The number of active PMs may either increase or decrease based on the prediction value, which is shown in Fig. 4.

Scale Up Phase For the case of increase in resource requirement, the scheduling system increases the number of PMs which can be done by switching on the new PMs satisfying the resource requirement. The amount of CPU and memory required for the time interval $t + 1$ can be formulated as defined in Eqs. 8 and 9.

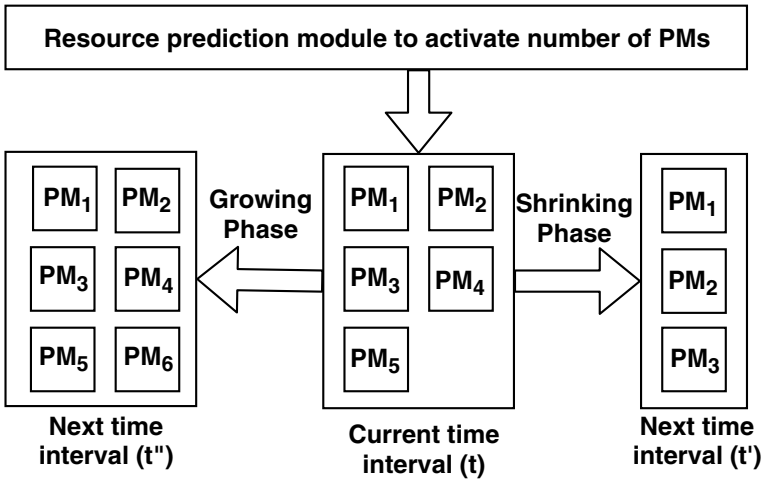


Fig. 4 Scale Up and Scale Down process

$$R^{cpu}(t+1) = U^{cpu}(t+1) - U^{cpu}(t) \tag{8}$$

$$R^{mem}(t+1) = U^{mem}(t+1) - U^{mem}(t) \tag{9}$$

Here $U^{cpu}(t+1)$ and $U^{mem}(t+1)$ represents the future (at time $t+1$) CPU and memory requirements and $U^{cpu}(t)$ and $U^{mem}(t)$ represents the current (at time t) CPU and memory usage of VMs. The terms $R^{cpu}(t+1)$ and $R^{mem}(t+1)$ represents the extra resources required in terms of CPU and memory at time $t+1$. Based on this requirement, the system deploys additional number of PMs for VM allocation. The new PMs are selected from the set of unused PMs (M_{na}), where the workload make the PM as compact as possible. The PMs in the set M_{na} are sorted based on decreasing order of its capacity and the compaction ratio of each PM is calculated with respect to the workload of the considered time interval. The PM with high compaction ratio is considered for the workload allocation because that minimize the number of PMs used for the VM allocation. As our objective is to minimize the power consumption, so we pick up the energy efficient PMs (using formulation given in Eq. 10) from the inactive PM set which are already sorted based on their compaction ratio and that can be formally defined as follows.

$$Minimize \sum_{j \in M_{na}} P_j \tag{10}$$

Subject to (a) $\sum_{v_i \in M_j} u_i \leq C_j^{cpu}$ and (b) $\sum_{v_i \in M_j} m_i \leq C_j^{mem}$, where (a) specifies the CPU capacity constraint and (b) specifies the memory capacity constraint of a PM.

Scale Down Phase But for the case, where the resource requirement decrease (shrinking phase) we need to reduce the number of active PMs from the set of active PMs (M_a). Once the PMs targeted to be switched off are identified, the system will not assign any further VMs to those machines. Based on the characteristics of VM requests, the system will decide whether to gracefully release the PMs or migrate the VMs of the targeted PM to other PMs and then switch off the PM. If a PM is allocated with VMs of bigger tasks for small duration then we will gracefully release the PM after completion of the assigned tasks. However, if a PM is assigned with smaller tasks with longer duration then migration will be beneficial. Smaller task migration will incur less migration overhead than bigger tasks. Here the big and small tasks are categorized based on their resource requirements. Bigger tasks need higher amount of computational resources than smaller tasks. For the case of migration, we need a migration planning module to handle this type of situation. For migration, we had modified Eqs. 3 and 4, so that it will handle the load balancing and power management.

$$\lambda'_{cpu}.C_j^{cpu} \leq U_j^{cpu} \leq \lambda_{cpu}.C_j^{cpu} \tag{11}$$

$$\lambda'_{mem}.C_j^{mem} \leq U_j^{mem} \leq \lambda_{mem}.C_j^{mem} \tag{12}$$

where λ_{cpu} and λ_{mem} represent the upper threshold for CPU and memory usage. The terms λ'_{cpu} and λ'_{mem} represent the lower threshold for CPU and memory usage. For example if $\lambda_{cpu} = 0.9$, then at a point of time the CPU utilization of a PM must not exceed 90% of the entire accessible CPU resource. Similarly, if $\lambda'_{cpu} = 0.2$ (that is the total resource usage is 20% of the CPU resource), then the VM allocated to that PM must be migrated because of low resource utilization and high power consumption. The value of λ_{cpu} and λ_{mem} is set to migrate the VMs from heavily loaded PMs to get optimal resource utilization, maximize workload throughput, and avoid the overload. The values λ'_{cpu} and λ'_{mem} differentiate the lightly loaded PM from heavily loaded PM and that drives for the VMs allocated to very lightly loaded PMs must be migrated for better resource utilization and power management. The migration planning module picks up the VMs of under utilized PMs and placed those to PMs, which have least number of VMs running and also the compaction ratio will be the most within the constraints λ_{cpu} and λ_{mem}.

4.4 Solution Approach

The solution of bin-packing problem can be used to solve the standard VM packing problem, where all PMs can be considered as bins where VMs are going to be scheduled. Each VM request is submitted to the system with their size represented as tuple, as defined previously. So the VM packing problem can be viewed as the multidimensional bin-packing problem considering two resources (CPU and memory). The VMs are fitted with the PM so that compaction ratio will improve, i.e., more number of VMs per PM. The compaction ratio for a PM is to compare the compactness of a PM when a VM request is considered for allocation. Higher the compactness ratio, better for the allocation. Compaction ratio for a PM M_j is defined as follows:

$$CR_j = \frac{U_j^{cpu}}{C_j^{cpu}} + \frac{U_j^{mem}}{C_j^{mem}} \tag{13}$$

where U_j^{cpu} represents the CPU utilization and C_j^{cpu} represents total amount of CPU available with the machine M_j and similarly U_j^{mem} represents the memory utilization and C_j^{mem} represents the total amount of memory available with the machine M_j. The CR_j value gives us an idea of better compaction as the value increases. Here our objective is to deploy less number of servers making the system energy efficient.

For the above stated problem we design heuristic approaches to solve. The proposed approaches use a term w_i, weight of a VM i, and w_i can be defined as follows.

$$w_i = (u_i + m_i) \cdot e_i \tag{14}$$

The terms e_i, u_i, and m_i represent execution time, CPU usage, and memory usage of the VM V_i, respectively. The other term used by our approach is the slack time (s_i) of a VM and can be defined as

$$s_i = d_i - e_i - t \tag{15}$$

where t is the current time. Here we consider the slack aware scheduling approach where we consider the slack time by which a VM can be delayed without missing the deadline requirement. In this work, we discuss different variants of slack aware and non-slack aware approaches to explore the efficacy of the different models.

4.5 Non-slack Aware Approaches (Non-predictive)

First Fit (FF) In this approach, we are not considering the slack time for VM allocation. The VM requests are scheduled serially in first come first serve basis to the available active PMs. The system must ensure that the constraints like capacity and deadline must not be violated. If system does not have the sufficient active servers then new servers are activated for allocation of VM requests.

Best Fit (BF) This approach is similar to FF approach. Here we consider the compaction ratio (using Eq. 13) of the servers for VM allocation. The PMs are sorted in descending order of their compaction ratio and allocation of the VM to PM is made based on that order. The new VM request which is allocated to a PM increases the compaction ratio of the PM.

4.6 Slack Aware Approaches (Predictive)

Slack Aware First Fit (FF-SA) and Best Fit (BF-SA) These approaches are similar to the baseline FF and BF methods. These approaches differ from the baseline approaches whenever no PM is found suitable for allocation at current time. The new requests are put into the waiting queue. The waiting time for a VM request must not exceed the slack time (as defined in Eq. 15) of that VM request. If a VM request is not considered within its slack time window, then that request is dropped because that will violate the deadline constraint of the VM request. The pseudo-code of the slack aware approach is presented through Algorithm 1. In Algorithm 1 the tasks are segregated into different batches based on the time window. The resource requirement of the subsequent batches is predicted using the resource consumption of the previous batch and the required number of PMs are activated accordingly. If more number of resources are required then new PMs are switched on else switch off the PMs which will not be used further. The PM for a VM is selected based on the compaction ratio and that results in less resource consumption leading to minimization of power consumption.

5 Experimental Evaluation

This section describes the experimental settings of our proposed strategy in heterogeneous cloud setup. The extensive simulations are conducted to evaluate the performance of slack aware approaches against non-slack aware approach and results are analyzed. The experimental goal is to reduce the deployment of number of physical machines and total power consumption. The simulation was performed using custom built software written in C++ and run on 3.20 GHz Intel Quad-Core processor 64 bit system. The simulation setup accepts server configuration and stream of VM requests as input and schedules the VMs to PMs with maximum compaction. Based on the VM allocations the scheduler generates several metrics for result analysis (e.g., total power consumption, average number of physical machines used, average weight of jobs served by each PM, and others). These metrics are used to compare the performance of different allocation strategies. The proposed approaches are evaluated using real world traces (Google cluster data [26]).

Algorithm 1 Predictive Slack Aware VM Packing (FF and BF)

procedure VM- PACKING
  for each new VM request v_i **do**
    Add v_i into NQ
  Sort the VM requests in NQ based on their non-decreasing order of deadlines
  for all the time windows under consideration **do**
    Create batch at each time interval (W_i) and store in TQ
    Predict the number of active machines based on the current resource requested by VMs and store it in $PMList$ (Using Eq. 7)
  Allocate the VM requests in TQ to PMs using FF/BF (Called as active set of PMs (M_a)) //Initial time interval requests
  for all the time windows under consideration **do**
    if $|PMList| > |M_a|$ **then**
      Add the energy efficient PMs to M_a
    else
      Switch off the lightly loaded PMs from M_a after efficient migration of VMs
    for each time stamp t in W **do**
      for each VM request v_i in TQ **do**
        $isValid \leftarrow$ CHECK- IF- VALID- REQUEST(v_i, t) // Resource and deadline constraints to be checked
        if $EST(v_i) \geq t$ and $isValid$ **then**
          $M_t, t_{pst} \leftarrow$ CHECK- FOR- TARGET- PM(M_a, v_i, t) // PM with high compaction ratio is selected
          Update the $PMList$
        else
          No suitable PM available
    Update the scheduling decision and update the TQ for the next time window
  return Set of PMs used, and total power consumption of all PMs

5.1 Experiment with Google Cluster Data

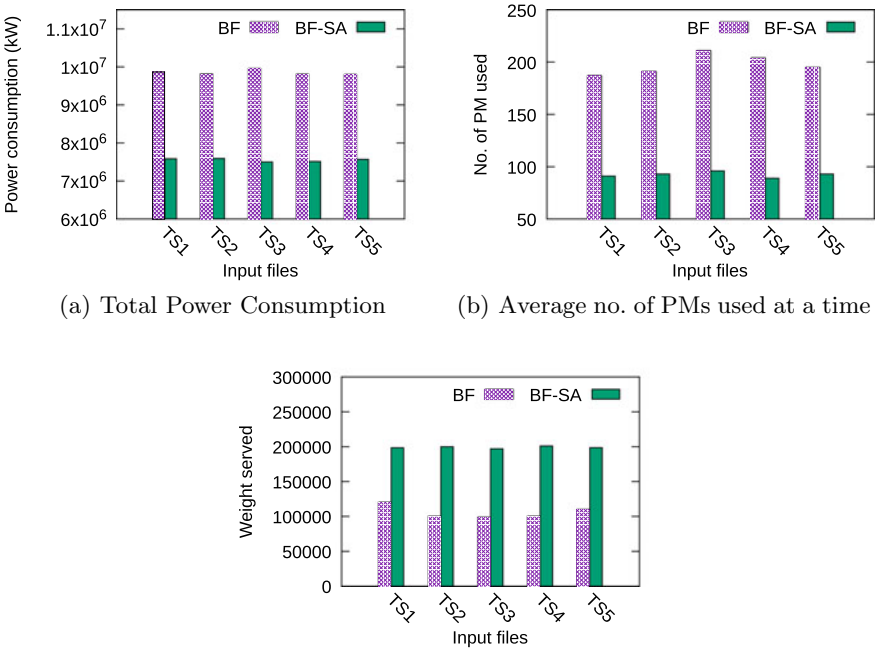
We used Google cluster data [26], which was collected over one month period in May 2011. The data set contains over 650 thousand jobs across over 12000 heterogeneous PMs. We had extracted data from the tables (Machine events, Task events, and Task usage) of the Google cluster data for our evaluation purpose. For this experiment, we segregate the data into multiple sets. For a single test, we took 10 sets of 4000 VMs and 300 PMs. The set of 300 PMs contain the equal proportion of different PMs with 10 different configurations. The considered PMs contain either 2, 4, 8, or 16 number of CPUs with the memory (RAM) capacity 2GB, 4GB, 8GB, and 16GB. The configurations of each PM is set in such a way that number of CPU must be less than equal to the amount of memory in GB. For example 4 CPU core case, the memory capacity would be either 4GB, 8GB, or 16GB. This makes a group of 10 different PM configurations with 30 in each group. These configurations presented in Table 1 are taken only for experimental purpose, whereas our system can incorporate other types of PM configurations with their power consumption metrics. The values of P_s and P_{dmax} are set such that P_s is 40% and P_{dmax} is 60% of the total power consumption of the considered PM. The time taken to complete all VM requests depend upon the arrival and execution time of each VM. The VM request parameters are collected from Google cluster data except the deadline. We set the deadline of the task as

$$d_i = a_i + e_i + baseDeadline \tag{16}$$

where $baseDeadline$ controls the task deadline and uniformly distributed, i.e., $U(baseTime, \mu \times baseTime)$ and we set $baseTime = 10000$ and $\mu = 8$. The values of κ_1 and κ_2 defined in Eq. 2 are taken as 0.5. The results are discussed in the following sections.

Table 1 PM configurations with their power consumption metrics

PM configurations in <CPU(units), MEM(GB)>	P_s	P_{dmax}	PM configurations in <CPU(units), MEM(GB)>	P_s	P_{dmax}
<2, 2>	60	90	<4, 8>	120	180
<2, 4>	70	105	<4, 16>	130	195
<2, 8>	80	120	<8, 8>	140	210
<2, 16>	90	135	<8, 16>	150	225
<4, 4>	110	165	<16, 16>	160	240



(a) Total Power Consumption

(b) Average no. of PMs used at a time

(c) Average weight served by one PM

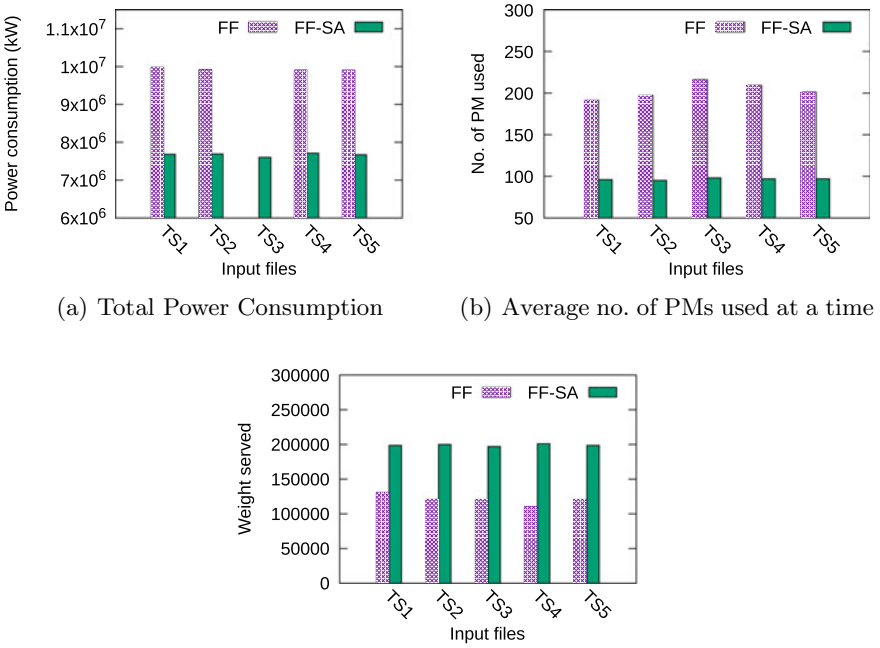
Fig. 5 Best Fit variations—Google Cluster Data

5.2 Result Analysis for Google Cluster Data

Figures 5 and 6 report the results of the BF and FF approaches based on their power consumption, average number of PMs at a time, and average of weight of tasks served by each PM. It is being observed that the slack aware approaches perform better in these parameters. The total power consumption decreases in slack aware approaches with increase in compactness of the PMs. The similar pattern is being observed for the cases of first-fit and best-fit approaches. The slack aware approaches improve the performance by delaying the decision of allocation and increasing the compactness.

6 Conclusion and Future Work

In this work, we designed and analyzed the solution approach for a variant of VM packing problem using slack aware principle. The proposed approach handles heterogeneous machines and slack time of the incoming tasks into consideration for robust VM packing. The work considers different types of VM instances and experimentally reported that slack aware-based approach performs better than its base



(a) Total Power Consumption

(b) Average no. of PMs used at a time

(c) Average weight served by one PM

Fig. 6 First Fit variations—Google Cluster Data

variants. Experimentation with other type of resources like disk space and network bandwidth will be the possible extension of this work. Since our approach did not allow VM migrations or preemption, we can extend the work on employing them in our existing model.

References

1. Mastelic T, Oleksiak A, Claussen H, Brandic I, Pierson J-M, Vasilakos AV (2014) Cloud computing: survey on energy efficiency. ACM Comput Surv 47, 2, Article 33 (January 2015), 36 pp. https://doi.org/10.1145/2656204
2. T. n. y. times, the cloud factories: Power, pollution and the internet. [online], posted at http://www.nytimes.com/2012/09/23/technology/ data-centers-waste-vast-amounts-of-energy-belying-industry-image.html (05 2012)
3. Ye K, Wu Z, Wang C, Zhou BB, Si W, Jiang X, Zomaya AY (2015) Profiling-based workload consolidation and migration in virtualized data centers. IEEE Trans Parallel Distrib Syst 26(3):878–890. https://doi.org/10.1109/TPDS.2014.2313335
4. Farahnakian F, Pahikkala T, Liljeberg P, Plosila J, Hieu NT, Tenhunen H (2018) Energy-aware vm consolidation in cloud data centers using utilization prediction model. In: IEEE transactions on cloud computing

5. Huang D, Ye D, He Q, Chen J, Ye K (2011) Virt-lm: a benchmark for live migration of virtual machine. SIGSOFT Softw Eng Notes 36(5):307–316. https://doi.org/10.1145/1958746.1958790
6. Song W, Xiao Z, Chen Q, Luo H (2014) Adaptive resource provisioning for the cloud using online bin packing. IEEE Trans Comput 63(11):2647–2660
7. Rampersaud S, Grosu D (2017) Sharing-aware online virtual machine packing in heterogeneous resource clouds. IEEE Trans Parallel Distrib Syst 28(7):2046–2059
8. Vm instances of EC2 (2019) posted at https://aws.amazon.com/ec2/instance-types/
9. Wu Q, Ishikawa F, Zhu Q, Xia Y (2016) Energy and migration cost-aware dynamic virtual machine consolidation in heterogeneous cloud datacenters. IEEE Trans Serv Comput 12(4):550–563
10. Ashraf A, Porres I (2018) Multi-objective dynamic virtual machine consolidation in the cloud using ant colony system. Int J Parallel Emergent Distrib Syst 33(1):103-120. Taylor & Francis
11. Beloglazov A, Buyya R (2012) Managing overloaded hosts for dynamic consolidation of virtual machines in cloud data centers under quality of service constraints. IEEE Trans Parallel Distrib Syst 24(7):1366–1379
12. Farahnakian F, Liljeberg P, Plosila J (2013) LiRCUP: linear regression based CPU usage prediction algorithm for live migration of virtual machines in data centers. In: 2013 39th Euromicro conference on software engineering and advanced applications. IEEE, pp 357–364
13. Haghshenas K, Mohammadi S (2020) Prediction-based underutilized and destination host selection approaches for energy-efcient dynamic VM consolidation in data centers. J Supercomput 76(12):10240–10257. Springer
14. Melhem SB, Agarwal A, Goel N, Zaman M (2018) Markov prediction model for host load detection and vm placement in live migration. IEEE Access 6:7190–7205
15. Laili Y, Tao F, Wang F, Zhang L, Lin T (2018) An iterative budget algorithm for dynamic virtual machine consolidation under cloud computing environment. IEEE Trans Serv Comput 14(1):30–43
16. Sharma Y, Si W, Sun D, Javadi B (2019) Failure-aware energy-efcient vm consolidation in cloud computing systems. Futur Gener Comput Syst 94:620–63
17. Swain CK, Saini N, Sahu A (2020) Reliability aware scheduling of bag of real time tasks in cloud environment. Computing 102:451–475. https://doi.org/10.1007/s00607-019-00749-w
18. Swain CK, Sahu A (2022) Reliability-ensured efficient scheduling with replication in cloud environment. IEEE Syst J 16(2):2729–2740. https://doi.org/10.1109/JSYST.2021.3112098. June
19. Chehelgerdi-Samani M, Saf-Esfahani F (2021) PCVM. ARIMA: predictive consolidation of virtual machines applying ARIMA method. J Supercomput 77(3):2172–2206. Springer
20. Jheng J-J, Tseng F-H, Chao H-C, Chou L-D (2014) A novel VM workload prediction using Grey Forecasting model in cloud data center. In: The international conference on information networking 2014 (ICOIN2014). IEEE, pp 40–45
21. Vm instances of EC2 (2019) posted at https://aws.amazon.com/ec2/instance-types/
22. Dayarathna M, Wen Y, Fan R (2016) Data center energy consumption modeling: a survey. IEEE Commun Surv Tutor 18(1):732–794
23. Alan I, Arslan E, Kosar T (2014) Energy-aware data transfer tuning. In: 2014 14th IEEE/ACM international symposium on cluster, cloud and grid computing, pp 626–634
24. Chang V (2014) The business intelligence as a service in the cloud. Future Gener Comput Syst 37(C):512–534
25. Garg SK, Toosi AN, Gopalaiyengar SK, Buyya R (2014) Sla-based virtual machine management for heterogeneous workloads in a cloud datacenter. J Netw Comput Appl 45(C):108–120
26. Wilkes J (2011) More Google cluster data, Google research blog, posted at http://googleresearch.blogspot.com/2011/11/more-google-cluster-data.html
27. Xiao Z, Song W, Chen Q (2013) Dynamic resource allocation using virtual machines for cloud computing environment. IEEE Trans Parallel Distrib Syst 24(6):1107–1117
28. Swain CK, Sahu A (2022) Interference aware workload scheduling for latency sensitive tasks in cloud environment. Computing 104:925–950

Reporting Code Coverage at Requirement Phase Using SPIN Model Checker



Golla Monika Rani, Akshay Kumar, Sangharatna Godboley, and Ravichandra Sadam

Abstract We automatically generate code coverage in the verification phase with the help of PROMELA (a language used by Spin checker). PROMELA is a process metalanguage of the Spin model checker. We are presenting a method that automatically generates from Abstract State Machine (ASM) specifications test which fulfils the desired coverage. ASM is used to predict the expected output under the test. A prototype tool that implements the proposed method. The experiments show comparison of code coverage and random test generation. We discuss the benefits and restrictions of the spin model checker. This work aims to reduce the number of features selected without compromising on delivering comparable accuracy and also performs the task in less time.

Keywords Code coverage · Model checking · Requirement phase · Testing · Test cases

1 Introduction

The Spin model checker [2, 7, 8, 16] is used for verifying the correctness of the concurrent software design model and tracing the design error. It uses a high-level language called PROMELA [5]. PROMELA stands for process metalanguage. The tools check logical consistency of specification. It supports random, interactive and

G. Monika Rani · A. Kumar · S. Godboley (✉) · R. Sadam
National Institute of Technology, Warangal, India
e-mail: sanghu@nitw.ac.in

G. Monika Rani
e-mail: gm720080@student.nitw.ac.in

A. Kumar
e-mail: kumar_mc19152@student.nitw.ac.in

R. Sadam
e-mail: ravic@nitw.ac.in

© The Author(s), under exclusive license to Springer Nature Singapore Pte Ltd. 2023

S. Chinara et al. (eds.), *Advances in Distributed Computing and Machine Learning*, Lecture Notes in Networks and Systems 660,
https://doi.org/10.1007/978-981-99-1203-2_13

guided simulation and includes everything possible and partial proof technique. The tool works easily without interruption with problem sizes and are even designed to handle very large problem sizes.

Spin uses LTL (Linear time temporal logic) model checking [14] that supports the all correctness requirements. Most of the properties can be verified by using Assertion instead of LTL. A property quickly verifies the program that can compile and then executed. Each PROMELA has an *init* process where it represents the entry point for the simulation and verification [9] of a model. All these are executed at the same time and it contains only one *init* process.

The spin model checker verifies the process metalanguage which supports the asynchronous distributed algorithm model [1, 12] as nondeterministic automata. Spin acts as a simulator and follows the execution path through the system and presents the execution trace to the user.

So, the Spin model checker has roots in the verification of the protocol system. The main purpose of this verifier is to provide effective tools for solving problems of practical significance. The predecessor supported the safety properties and liveness properties only.

The spin model [10] translates the process into finite automata. The behaviour of the concurrent system can be obtained by the asynchronous interleaving of automata products. The interleaving product is referred to as state space because it can be represented as a graph easily. It is sometimes also referred to as the global reachability graph.

For verification the spin takes correctness claims as a temporal logic formula and it converts the formula into the Buchi automation [17] for computing the asynchronous product of the claim. Automation represents the global state space. If the language is accepted by the automation and if it is empty, it means that the original claims are not satisfied by the system. If the language is nonempty it means it satisfied the temporal logic formula. The Spin Model checker accepts only correctness properties only in the form of Linear Temporal Logic (LTL) [4]. LTL can be translated into Buchi automation.

Flat Translation: The first method uses one process *init* for the execution of the model. After the Enumeration declaration, constant declaration and variables declaration the init process starts. It contains do cycle to perform monitored variables updating, execute the rules and update the current values of the variables. **Chan Translation**: This is used to process each rule and execute it asynchronously. Synchronization is possible only through the rendezvous channel. **Comparison**: We compared two translation method specifications. PROMELA execution specification obtained by Chan translation requires more memory than the flat translation. So, Chan is slightly faster than flat translation.

As part of this research work, we answer the questions like how the spin model checker tools can be used to generate the code coverage at the requirement phase, how the spin model checker verifies the code, how to implement the assert statement by PROMELA expression in the program and how to trap violation in the spin.

2 Related Work

Holzmann et al. [11] shows how we can verify the larger applications without giving too much time, and it should be corrected and reliable. Because when if there are complex problems in our software, we need to explore that problems with the help of the software tester. They may take more time to fix the issues. By avoiding this type of problem, an automated extraction model [13] can be used. In this paper, the author used the top-down method. At the time of model extraction from the code, the tester checks the design requirement and then implementation. After checking these things the tester starts the verification [6, 9] during the code development, and the tester starts designing and implementing it by satisfying the correctness. After verification, they move the data and control it, which has some code like what syntax has been used ex. *if, while, until, and goto* statements. In Spin the specification language, i.e PROMELA sends and receives the message by message queue through the channel. So operation used here is the message parameter with the correctness properties of the applications. In this paper, model extraction tools are used where it has source code and move to the catch format and then move to PROMELA where it lookup table and model template. The spin tools check the property and then trace.

Wei et al. [15] implemented the Model checking tools concept by using three methods. The first is the modelling phase where it generates the model for the system, the second is the running phase where it checks the validation of the properties and the third one is the Analysis phase where it checks the satisfaction of the property. Spin tools using PROMELA for finding bugs like System file bugs, Concurrency bugs, Crash consistency bugs, Logic bugs, Memory bugs and Specification Violation bugs like that. A role to produce high-quality products. Software needs to be tested perfectly without any coding error to assess its reliability and dependability to satisfy the customers that software performance is acceptable. Specification-based testing provides an opportunity to reduce costs. So spin model checker can support random and interactive with partial proof technique. It can scale smoothly with large problem sizes and is designed to handle large problems. It uses an LTL model checking system [4] for checking all correctness requirements. Many properties can be expressed and verified by using assertion instead of LTL. Each PROMELA has its *init* process which is an entry point of the model for simulation and verification. The other process can be executed by the main process. All these processes are executed asynchronously.

Castillo et al. [3] observed that the software development for control systems such as the software control application for aeronautics and medical devices is a more costly, time-consuming and error-vulnerable process. So verification and validation are more important for controlling this type of stuff and it consumes 50% to 70% of the software development of the process. So, if the deriving test case for validation and verification could be automated and required based it can reduce time and cost also. This paper presents an automatically generated test case for structural code coverage. **System model**: System model has finite and infinite data components. So, we do not differentiate between the control and data components of the system. **Transition relation**: It is specified as a Boolean predicate value of the variable

defining the state space. **Structural Code coverage**: In structural code coverage, we need to formalize the notation of the test suite and test cases. **Test case generation**: In this approach, we can generate test cases by using the model checker. The property called trap property is generated and the spin model checker tools is verifying the property one by one.

3 Proposed Approach

In the proposed methodology, we are trying to generate code coverage by using a relational operator with the help of an assert statement. The assert statement is always executable and has no other effect on the state of the system. It will cover the entire code of the process. The execution flow of the spin checker is depicted in Fig. 1.

Code coverage of the program can be derived from the hash factor. It can be executed by using the analyzer. The coverage of conventional analysis goes down rapidly. If the states may be twice as many there are more states in the entire state space than we can store.

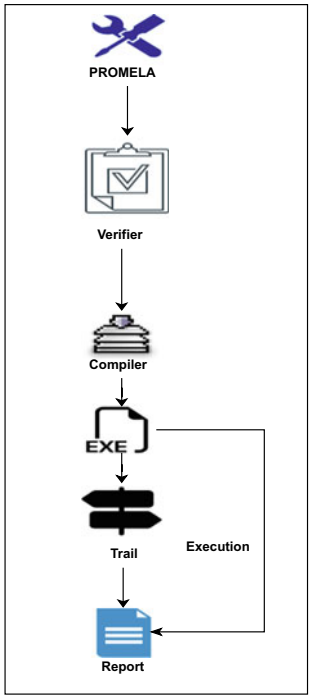


Fig. 1 Working of SPIN checker

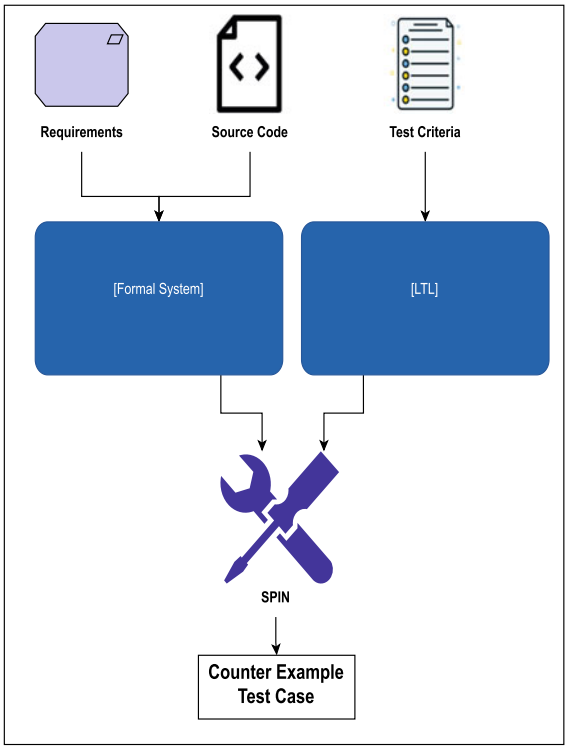


Fig. 2 Test generation

The property that we want to generate is expressed in Logical Temporal Logic (LTL) which can be handled by the spin model checker and SMV. A trap property can produce only one counterexample so that it can handle the condition that measures multiple test cases. The test case generation framework is illustrated in Fig. 2. The workflow diagram to produce code coverage is shown in Fig. 3.

The main challenge of this approach is to find the abstraction technique for reducing the size and addressing of state space explosion and infinite state space in the software model checking artefacts. The working area of abstraction technique has the advantage to create a model manageable to model checking. The main issue is the loss of details in the abstraction technique that makes the instantiation of test sequences decidable.

Hash factor is nothing but it is a state, which means 32 million bits are equal to states. The hash factor is equal to the maximum no of states divided by the actual no of states. A state is a block of code in a different process. A large Hash factor means it has more accuracy and has no error and coverage of 99% or 100%. If the hash factor is 1, the coverage may be 0%. It can be easily calculated by the memory requirement of analysis in full state space.

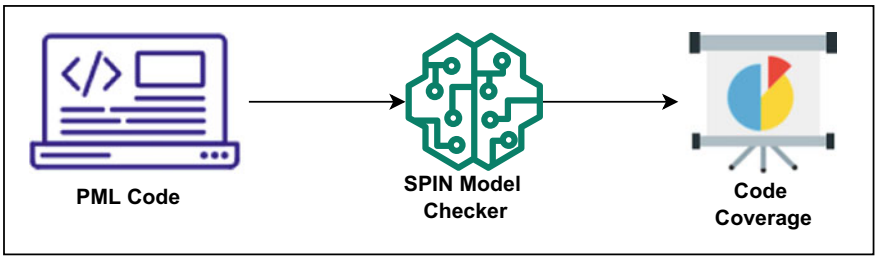


Fig. 3 Work flow diagram

But our aim of testing is to discover faults in the code and produce quality software. When we try to use the relational operator in process one and process two and so on in the program, it will trap the code violation in every part of the program. The trapping may be thousand of codes. We can't use the assert statements anywhere in the program for generating coverage. It might give an error. It can be used in large programs to trap violations and errors. The spin Model checker is identifying the correctness and shows the result at the end.

Here, we discuss how the process works. We take any PML program for generating code coverage and put it into the spin model checker tool. The spin model checker verifies the code for error and then we try for verification. Then, we run the code by selecting depth-first search [13] mode for invalid and assertion violations. We can create the state tables for checking how the process is working. We can set the parameter for *physical Memory*, *Estimated state space size*, *Maximum depth search*, *Size for minimize automation* and *No. of the hash function in Bit state mode*.

We set advanced Error trapping for stops at error number 1 or don't stop when more than one error is in a program. In Depth First Search (DFS) mode we can select report unreachable code. We can set the number to the amount of physical (not virtual) memory in our system, in MegaBytes, and leave it there for all runs. To avoid trashing, the verification is halted when the limit is reached. For the size of the estimated state space: The size of the hashtable is calculated using this option. It leads to the selection of a numeric parameter for the verifier's $-w$ *flag*. If you set it too high, you can get an out-of-memory error when zero states are reached (meaning that the verification could not be started). If you set it too low, hash collisions can cause inefficiencies. No of the hash function determines how many bits are set per state when in Bitstate verification mode. The default is 3, but we can use any number greater than or equal to 1. At the end of a Bitstate verification run, the verifier can issue a recommendation for a different set of this parameter (specified with the -*k flag*), if this can improve coverage.

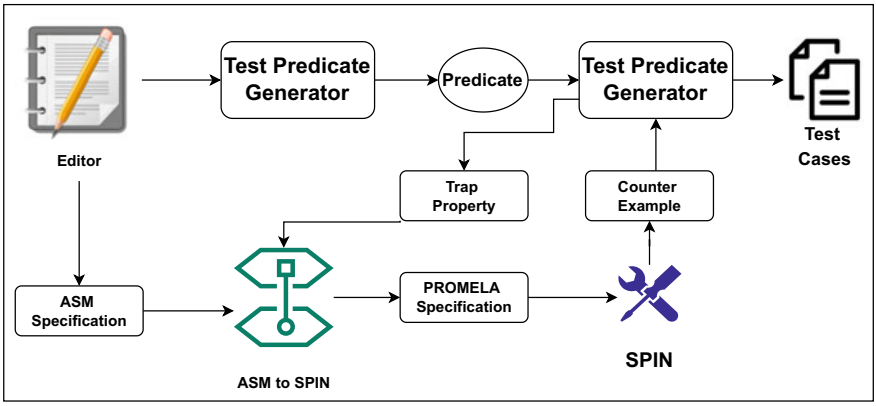


Fig. 4 Tool architecture

Listing 1.1: Assertion Statement by PROMELA expression

```
....
statement 1
assert(min==x)
statement 2
........
or
.........
if
y1   ->   statement 3
assert(a>b)
y2  -> statement 4
.........
```

Figure 4 shows the functionality of the spin model checker tools. So model checker tools are the best way of finding a counterexample for correctness. It does not try to prove the properties. Spin is simulating the model and generates the verifier. Generating code coverage with the help of an Assertion Statement by PROMELA expression: The expression is always in bool type and can be any PROMELA expression and it can appear anywhere in the PROMELA. An example of an assert statement is shown in Listing 1.1. The Assert statement has no effect if the expression is true but if the expression is false then it will trigger the error message. So an example of a Lynch algorithm for generating the code coverage is by putting the Assert statement in between the PROMELA Program. Figure 5 depicts the coverage report of the Lynch algorithm.

Here we observed that there are so many violations happening that are triggered by the spin model checker tools. We can simply find out the errors. The tools clearly show the line-by-line errors for verification. The state table shows where the program is started for a run and how the work is taking place. It will show the entire part of the core functionality so that we can understand it.

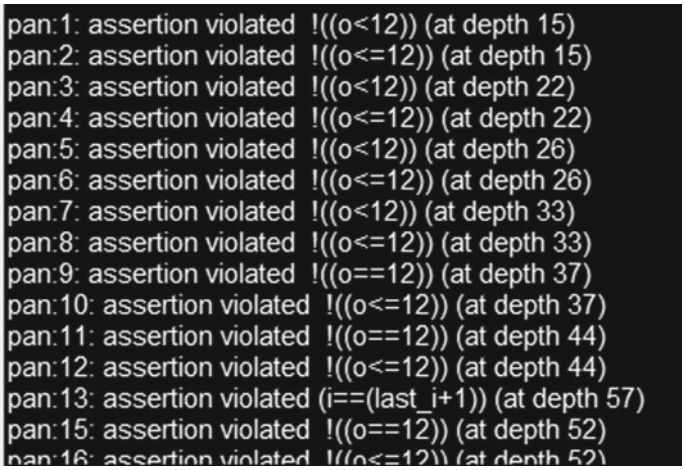


Fig. 5 Coverage of lynch

4 Experimental Study

To illustrate our work, let us consider the sample program[1] which has three processes. The three processes have different control flow graphs which are shown in Figs. 6, 7 and 8. It will show how the process will work when we use a relational operator in each process, it will trap all possible violations, and it will show error trapping also. In spin model checker tools the program has variable values, statement merging and queues.

In the above program we observed that the transfer process, channel process and right process start with a state table with some transaction id and source with statements. It shows how the program is working in the process with the help of variables used in the program. Each process has shown its statements and source. Here only three processes have been used so it shows only three processes. It may be more than three also. After state tables, we try to verify the code using the relational operator, the program generates the coverage line by line where the assertion is happening until the entire part of the code. The spin model checker tools will generate *state Vector*, *Depth reached*, *Error*, *states stored*, *states matched*, *transitions*, *atomic steps*, *Hash conflicts*, *Equivalent memory usage for states*, *memory used for hash table and DFS stack*, and *total actual memory usage*.

The results corresponding to Sorting, Dijkstra's, and Lynch algorithms are depicted in Tables 1, 2, and 3, respectively.

The main aim of the experiment is to provide evidence that our method is effective. Code coverage of a single program has been generated below where we can see that the coverage may be many lines. It can be executed by using the analyzer.

[1] https://figshare.com/articles/software/src_pml/20731522.

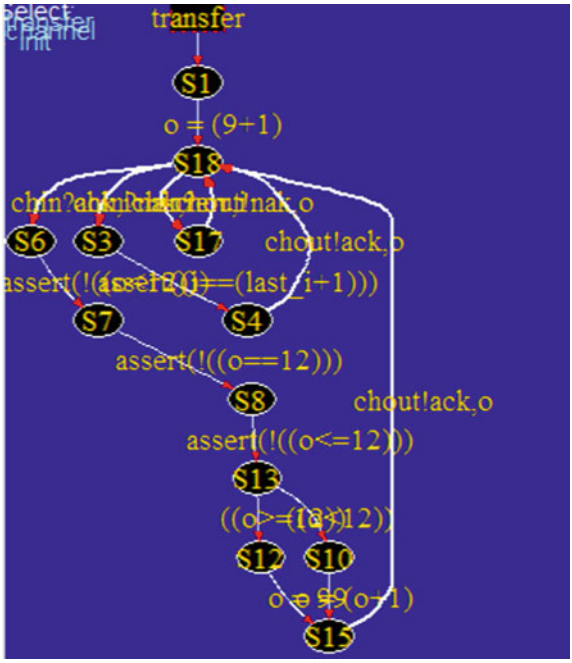


Fig. 6 CFG of transfer process

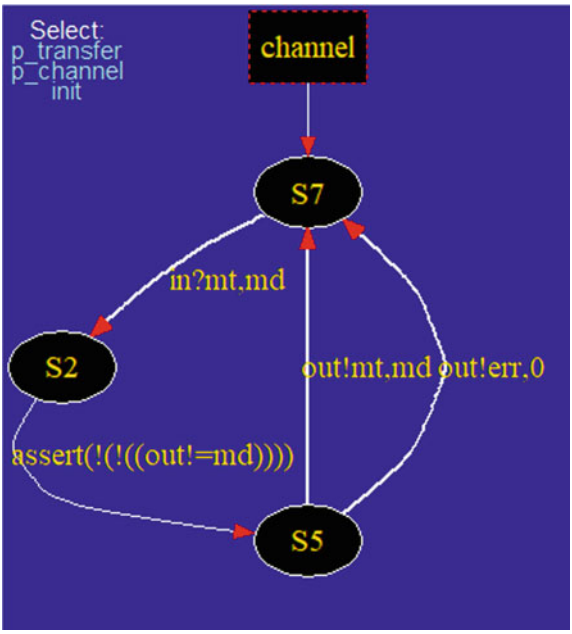


Fig. 7 CFG of channel process

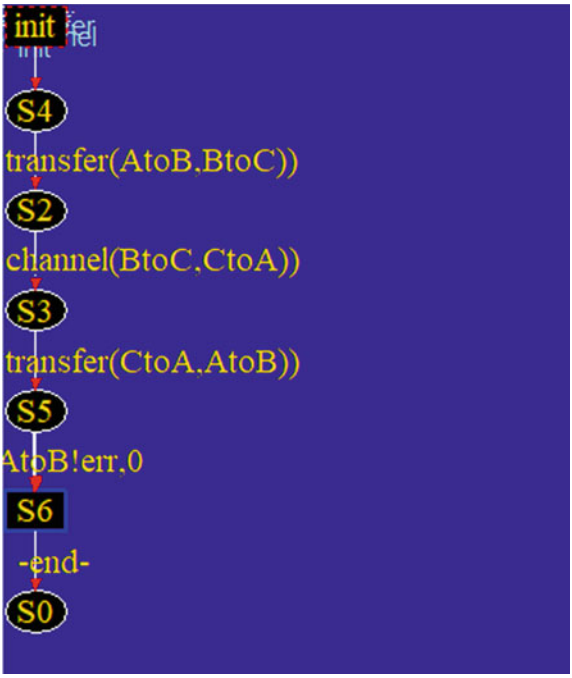


Fig. 8 CFG of init process

Table 1 State table of sorting algorithm

Proc name	From state	Trans id	To state	Statement
Left	1	[23]	13	counter=0
Left	13	[24]	10	out!seed
Left	10	[25]	16	counter==7
Middle	1	[12]	2	counter=(7-procnum)
Middle	2	[13]	2	inp?myval
Right	1	[10]	2	inp?biggest
Right	2	[11]	0	end

The coverage of conventional analysis goes down rapidly if the states may be twice as many states in the full state space than we can store. But our aim of testing is to discover faults in the code and produce quality software. When we try to use a relational operator in process one and process two and so on in the program it will trap the code violation in every part of the program. The trapping may be thousand of codes. We can't use assert statements anywhere in the program for generating coverage, it might give an error. We try to generate the code coverage of five different programs by using the relational operators. The report of these programs is depicted in Table 4.

Table 2 State table of dijkstra's algo

Proc name	From state	Trans id	To state	Statement
Dijkstra	1	[11]	8	assert(!count==1)))
Dijkstra	8	[12]	3	assert((count==1))
Dijkstra	8	[15]	6	assert(count==0))
Dijkstra	3	[15]	4	sema!0
Dijkstra	4	[14]	8	assert(count=0))
User	1	[3]	2	sema?0)
User	2	[4]	3	sema?1)

Table 3 State table of lynch algo

Proc name	From state	Trans id	To state	Statement
Transfer	1	[13]	18	o=(9+1)
Transfer	18	[14]	3	chin?nak,i
Transfer	18	[17]	6	chin?err,i
Transfer	3	[15]	4	assert((i==(last_i+1))
Channel	7	[8]	2	in?mt,md
Channel	2	[9]	5	assert(!((out!=md)))

Table 4 Code coverage generation

Code	Relational operator	Output (Coverage)
Dijkstra	==	3770
Sorting	>= ==	12032
lynch	<=	6000
critical section program	==	1203
program 5	! =	7233

5 Conclusion

In this work, a spin model checker technique is used to generate code coverage by using the relational operator. It covers the entire code of large programs very quickly without skipping any part of the code and gives sound results with high accuracy. Additionally, it adopts sophisticated techniques to compute and explore the state space in efficient ways using hashing [6] technique and symbolic execution. The code coverage at the requirement phase in spin there are so many researchers explored actively. So, we need to implement more expression on it so that we generate the code easily. If we find code coverage at Requirement Phase, then we can save huge time during the testing phase.

References

1. Aiswarya C, Bollig B, Gastin P (2018) An automata-theoretic approach to the verification of distributed algorithms. Inf Comput 259:305–327. 22nd international symposium on temporal representation and reasoning
2. Barros J-P, Gomes L (2022) Towards an integrated tool support for the analysis of iopt nets using the spin model checker. In: 2022 IEEE 31st international symposium on industrial electronics (ISIE), pp 239–244
3. Del Castillo G, Winter K (2000) Model checking support for the asm high-level language. In: Proceedings of the 6th international conference on tools and algorithms for construction and analysis of systems: held as part of the european joint conferences on the theory and practice of software, ETAPS 2000, TACAS '00. Springer, Berlin, Heidelberg, pp 331–346
4. Gerth R, Peled D, Vardi MY, Wolper P (1996) Simple On-the-fly automatic verification of linear temporal logic. Springer US, Boston, MA, pp 3–18
5. Holzmann GJ (1993) Design and validation of protocols: a tutorial. Comput Netw ISDN Syst 25(9):981–1017. Protocol specifications, testing and verification
6. Holzmann GJ (1996) An analysis of bitstate hashing. Springer US, Boston, MA, pp 301–314
7. Holzmann GJ (1997) The model checker spin. IEEE Trans Softw Eng 23(5):279–295
8. Holzmann GJ (2000) Logic verification of ansi-c code with spin. In: Proceedings of the 7th international SPIN workshop on SPIN model checking and software verification. Springer Berlin, Heidelberg, pp 131–147
9. Holzmann GJ, Peled D (1995) An improvement in formal verification. In: Proceedings of the 7th IFIP WG6.1 international conference on formal description techniques VII. Chapman; Hall, Ltd, GBR, pp 197–211
10. Holzmann G, Puri A (1999) A minimized automaton representation of reachable states. Int J Softw Tools Technol Transf (STTT) 2
11. Holzmann G, Margaret S (2001) Software model checking: extracting verification models from source code. Softw Test Verif Reliab 11:65–79
12. Jiang X, Zeng X, Sun J, Chen J (2022) Distributed synchronous and asynchronous algorithms for semi-definite programming with diagonal constraints. In: IEEE transactions on automatic control, pp 1–1
13. Laarman A, Langerak R, van de Pol J, Weber M, Wijs A (2011) Multi-core nested depth-first search. In: Bultan T, Hsiung P-A (eds) Automated technology for verification and analysis. Springer Berlin Heidelberg, Berlin, Heidelberg, pp 321–335
14. Li Yongming, Wei Jielin (2021) Possibilistic fuzzy linear temporal logic and its model checking. IEEE Trans Fuzzy Syst 29(7):1899–1913
15. Su W, Liu Y, Ganesan G, Holzmann G, Smolka S, Zadok E, Kuenning G (2021) Model-checking support for file system development. In: Proceedings of the 13th ACM workshop on hot topics in storage and file systems, HotStorage '21. Association for Computing Machinery, New York, NY, USA, pp 103–110
16. Tekik O, Surer E, Betin Can A (2022) Verifying maze-like game levels with model checker spin. IEEE Access 10:66492–66510
17. Zhang Y, Song Y, Zhang H, Jin Y (2021) Stability analysis of reentry constrained switched system based on büchi automata. In: 2021 36th youth academic annual conference of chinese association of automation (YAC), pp 338–343

Metric-Oriented Comparison of Selective Forwarding Attack Detection Techniques in IoT-Based Systems



Nidhi Sinha and Alekha Kumar Mishra

Abstract The Internet of Things (IoT) is an innovative technology nowadays due to its broad variety of applications in industrial and consumer markets. Secure and timely packet transmission in IoT is the basic requirement of the network. The selective forwarding attack possesses a higher level of threat to the process of packet transmission. Here, attacker compromises its trustworthiness and decide to discard packets selectively instead of dropping all of them like a blackhole node. This results in packet loss, latency, link failures, data inconsistency, and collision. In this paper, we provide a characteristic level analysis of the selective forwarding attack in an IoT environment. We have enlisted and analyzed the approaches and metrics commonly used to detect a selective forwarding attack.

Keywords IoT · Security attacks · Selective forwarding attack · Attack analysis

1 Introduction

The Internet of Things (IoT) is a brand-new paradigm that provides various new services for the upcoming wave of technological developments [2]. IoT applications are virtually endless and allow for seamless connectivity of the physical and digital world. It was developed by Kevin Ashton in 1999 [14]. Integration of multiple processes, including identification, sensing, networking, and computation, is the backbone of the IoT. IoT applications come in a wide range as shown in Fig. 1 and can be categorized into a number of different sectors such as health, traffic, transportation, retail, farming, smart buildings, smart metering, monitoring systems, and automation systems [5]. The deployment of IoT networks is still facing a lot

N. Sinha · A. Kumar Mishra (✉)
Department of Computer Science and Engineering,
National Institute of Technology Jamshedpur, Jamshedpur, India
e-mail: alekha.cse@nitjsr.ac.in

N. Sinha
e-mail: nidhi10nitp@gmail.com

© The Author(s), under exclusive license to Springer Nature Singapore Pte Ltd. 2023

S. Chinara et al. (eds.), *Advances in Distributed Computing and Machine Learning*,
Lecture Notes in Networks and Systems 660,
https://doi.org/10.1007/978-981-99-1203-2_14

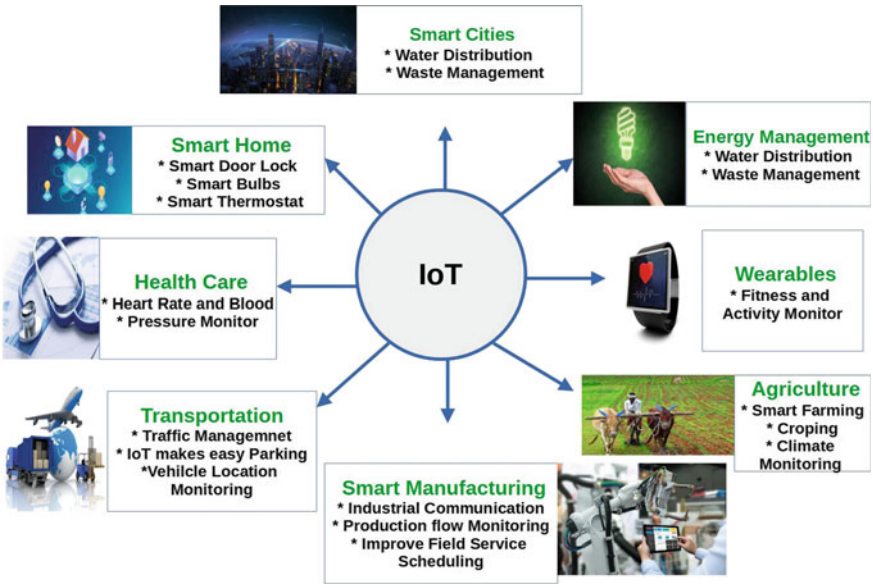


Fig. 1 Application of Internet of Things

of challenges such as security and privacy, scalability, interoperability, and heterogeneity [19]. These challenges need to be considered from various perspectives, including enabling technologies, applications, business models, social impacts, and environmental effects.

The routing of packets using a multi-hop routing process is the key to collect data from a wide range of IoT devices to their respective data centers. The routing over lower powered and lossy networks (LLN) [7] is different from standard routing mechanisms in terms of resource constraints on a device and on the communication links [17]. However, the packet transmission process in IoT is vulnerable to a number of threats such as jamming, wormhole, blackhole, Sybil, selective forwarding attack, and sinkhole attack [12]. In this paper, we have initially provided the description and characteristics of all possible attacks in an IoT Network Communication Infrastructure. Then, we particularly focused on the selective forwarding attack and its recent detection techniques provided in the literature. Finally, we provide an analysis of various selective forwarding attack detection techniques comparing their detection metrics, the underlying technique, and evaluation criteria for experiments.

The remaining paper is organized in the following sections. Section 2 describes the possible attacks on IoT networks. Section 3 describes the attack characteristics of a selective forwarding attack. The literature survey on selective forwarding attack detection mechanism is provided in Sect. 4. The analysis of the mechanism is discussed in Sect. 5 followed by conclusions in Sect. 6.

2 Security Threats in IoT

Most of the IoT devices operate in three layered architecture of IoT, namely, the perception layer, communication layer, and application layer. The characteristics of the threats vary with the service layers of IoT. The possible threats [18] for each layer are discussed in the following subsections.

2.1 Perception Layer Attacks

1. **Capture of a Node Attack**: By capturing a node, an attacker can get complete control of a node and can instruct it to perform any malicious operation in the perception layer. It leaks the information stored in the captured device's memory.
2. **Eavesdropping Attack**: Eavesdropping attack is a form of passive attack where attackers can sniff wireless traffic and steal users' valuable information.
3. **Exhaustion**: Exhaustion is a form of battery drainage, where a node is prevented to go to energy-saving mode or going to sleep by bombarding it with a huge number of legal requests.
4. **Malicious Data Injection Attack** [20]: Here, an attacker may launch a device to insert falsified messages or flood radio channels with false messages generating inconsistency among the network data.
5. **Sybil Attack**: In this attack, a malicious node may present many identities by fooling other nodes or creating new fake identities. The overall goal of an attacker in this attack is to take the control of various parts of the network without using any additional physical devices.
6. **Replay Attack**: In this type of attack, intruder listens to the conversation allying a sender and a receiver and at a later part, it repeats some selected conversations in order to access confidential data or to corrupt legitimate data.
7. **Side-Channel Attacks**: A side-channel attack is a type of security threat that tries to steal secrets from a chip or system. This can be done by measuring or examining various physical parameters including supply current, execution time, and electromagnetic emission.
8. **Unauthorized Access**: In this attack, the attacker gains unauthorized access to either a device or communication channel to perform malicious actions.

2.2 Communication Layer Attack [16]

Most of the attacks of communication layer target the underlying routing protocol used to send and receive packets over IoT networks. The list of possible attacks in the communication layer is presented as follows:

1. **Man-in-The-Middle (MITM)**: MITM attack is a cross-layer attack, where a malicious entity secretly eavesdrops on the communication and can alter the communication between the two entities, while the two entities think that they have direct communication between them.
2. **Storage Attack**: In a storage attack, an attacker can attack both the cloud storage and storage devices and tamper the contents of the information.
3. **Exploit Attack**: Any illegal or unethical attacks in the structure of software, pieces of data, or a stream of instructions is called an exploit attack.
4. **Distributed Denial-of-Service (DoS) Attack**: Through a denial-of-service attack, the legitimate users are prevented from using their systems or other network resources. It is frequently practised by flooding targeted devices with multiple requests and overflow the buffer.
5. **Sinkhole Attack**: A sinkhole attacker [11] attracts all the network traffic toward itself by advertising itself as the closest node to the sink. As a result, the attacker drops all the incoming data packets in the network.

2.3 Application Layer Attacks

Most of the attacks in application layers are code-based attack targeting the device software and script. A few of them are listed below:

1. **Cross-site Scripting**: Cross-site scripting is a type of injection attack which allows an attacker to deploy client scripts, such as JavaScript to trusted websites that other users can access to ultimately gain access to confidential data.
2. **Malicious Code Attack**: Malicious code is a type of software that is implemented as a set of instructions that are executed on IoT devices and can be designed to achieve a variety of different results.
3. **Attack on the ability to deal with large volume of data**: In this threat, a large volume of data is created or communicated by a big number of IoT devices. It test the limits of the data collectors and mostly causes network disruption and data loss.

3 Selective Forwarding Attack (SFA)

Normally, a node transmits data packets to its next-hop toward the destination assuming that it would be forwarded by the next-hop node. However, a selective forwarding node would choose only a few packets to forward toward its destination dropping all the remaining packets [13]. Figure 2 demonstrates the behavior of a typical selective forwarding attacker node with D as attacker. As a result, most of the destination nodes do not receive the packets intended for them. The selectively dropping of packets by an attacker has two variations: first, where the attacker drops packets from a spe-

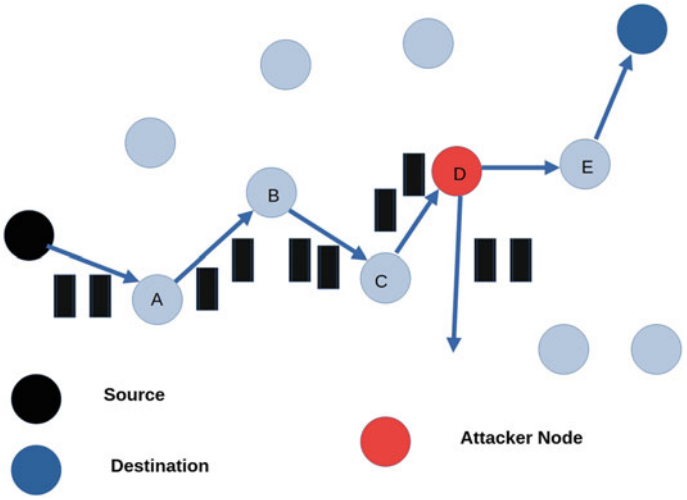


Fig. 2 Selective forwarding attack

cific group of devices and second, where the attacker drops a packet of a particular protocol type.

There are several detection techniques for selective forwarding attacks. The most generic mitigation strategies include (i) mechanisms for identifying the selective forwarding nodes and removing them from routing data based on acknowledgment and (ii) mechanisms of identifying attackers based on neighborhood knowledge. Some of the basic needs of a selective forwarding detection [21] mechanism are (i) secure communication between the devices in the network and (ii) the packet drop ratio of the attacker must be higher than usual drops due to unreliable links and this margin should be significant to distinguish an attacker.

4 Literature Survey

In this section, we briefly discussed the selective forwarding detection mechanisms as reported in the literature recently.

Mathur et al. [9] have proposed a technique for detecting selective forwarding attacks and blackhole attacks caused by routing traffic between sensor nodes and access points. They have used cryptographic hashes and digital signature for this purpose. The proposed mechanism has claimed to detect the attack with an accuracy of 96%.

Airehrour et al. [1] have proposed a trust-based routing protocol that addresses the selective forwarding attack as well as blackhole attack. The trust-based approach

has been implemented in RPL to improve its capability to prevent blackhole and selective forwarding attacks.

Yin et al. [21] have proposed an MPSR routing protocol that is highly secure and depend on a multiple attribute decision-making model for a scale-free network. In this proposed model, the parameters such as load, packet loss rate, and energy transmission efficiency are added for monitoring the behavior of packet forwarding of each node. MPSR protocol has been used for secure routing. MATLAB has been used for the simulation. Experimental results show that the proposed algorithm can prevent selective forwarding attacks and as well as optimize the network energy and increase network lifetime.

Srinivas and Manivannan [15] have proposed the detection and prevention mechanism to detect both the attack black hole and selective forwarding by using the bait process with a packet validation. The proposed model's primary objective is to determine the shortest path for node communication, which optimizes the network lifespan of networks and sensor nodes. As a result, a trustworthy reliable routing architecture with attack discernment and impediment techniques for secure communication is established in clustered IoT-WSN.

Jiang and Liu [6] proposed a state-of-the-art SFA with a variety of behavior against the RPL network. The attacker node not only selects the types of packets to let go but besides dynamically controls the packet forward rate. This model is a novel centralized trust model. The model incorporates two trust values: a tree-based descendent trust value that makes use of the RPL network architecture to disregard badmouthing attacks, and a node's self-trust value that represents the trustworthiness of its packet forwarding nature.

Kim et al. [8] have done a brief comparative analysis on the countermeasures in defending against Sybil attacks. In this paper, two schemes have been discussed about the countermeasure of sybil attack that is (i) Entropy Impurity Observation Scheme and (ii) Artificial Bee Colony Scheme.

Huang et al. [4] have proposed a danger model for detecting the selective forwarding and grayhole attack. To increase the detection accuracy and decrease the complexity of the algorithm, they have proposed a screen-confirm scheme, which is based on two phases: (i) screening phase, (ii) confirmation phase using SVM (support vector machine) model.

Ding et al. [3] have proposed a selective forwarding attack model of smart malicious node with a reinforcement learning algorithm. DT-DPC (Double-Threshold Density Peak Clustering) algorithm is used to detect the attack effectively in a harsh environment.

5 Comparison and Analysis

In this section, we have provided the analysis of various selective forwarding detection mechanism discussed in Sect. 4. Table 1 summarized the techniques, and detection metrics used by the detection mechanisms. It is observed that most of the tech-

Table 1 Analysis of selective forwarding attack detection mechanisms

Authors	Techniques	Detection metrics	Type
Kim et al. [8]	Artificial Bee colony scheme, Gini	Entropy impurity	Distributed
Patel and Jinwala [13]	Reputation propagation based	Packet lost and sent	Distributed
Yin et al. [21]	Decision model based	Energy efficiency, delay, and packet drop rate	Distributed
Srinivas and Manivannan	Bait process and packet validation, D-DHOA	Trust, latency, or packet delay and packet drop	Distributed
Jiang and Liu [6]	Trust based	Packet received and lost	Centralized
Airehrour et al. [1]	Trust based	Rank value, number of transmission count, reliability	Centralized
Mathur et al. [9]	Cryptographic hashes and threshold	Network traffic	Centralized
Huang et al. [4]	Missed detection rate and false detection rate	Network throughput	Distributed
Ding et al. [3]	Double-threshold density peaks clustering (DT-DPC)	Detection accuracy and throughput	Distributed

niques for detection of selective forwarding attack are reputation or trust based. In other words, based on the behavior of a neighboring devices in the network, a node measures its trust level toward successfully forwarding the packet toward the destination. However, a few detection mechanism also uses the cryptographic technique, artificial bee scheme, and Gini entropy impurity. This indicates that the detection of this attack may also need a complex model to design a detection system. It is also inferred that the majority of the techniques uses packet dropped, packet lost, or successfully sent as a detection metric to either detect an attacker directly or to compute the trust value of a node to measure its legitimacy. Since the characteristics of a selective forwarding attacker node is determined by the way it handles the incoming packets from various sources, the metrics such as the packet dropping rate and the number of packet lost help to determine the trustworthiness of a node. The last column of the table shows the type of detection mechanism. It is observed that there is a nearly equal number of distributed and centralized mechanisms proposed recently for detecting selective forwarding attack. From Fig. 3, it is observed that the underlying protocol for most of the detection technique is RPL. This is because, RPL is considered at the standard routing protocol for communication layer of IoT. Therefore, devising detection mechanism over the RPL is expected to be widely acceptable.

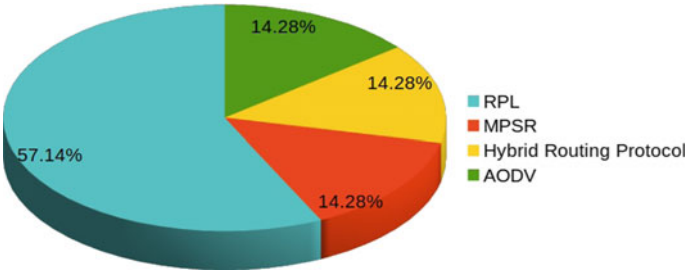


Fig. 3 Pie chart graph for underlying protocol

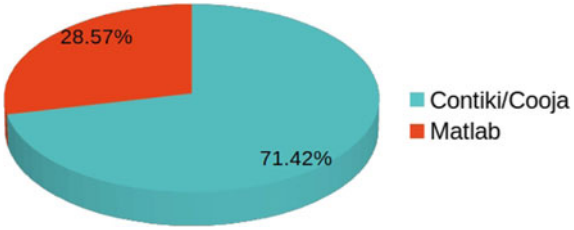


Fig. 4 Pie chart graph for simulator used for attack detection

Next, we analyze the parameters related to the experiment performed by the detection techniques as reported in the literature. From Fig. 4, it is observed that the Contiki/Cooja is widely used for implementing selective forwarding attack detection in IoT due to its popularity as well as flexible structure. Figure 5 shows the comparison of network size range used by various detection mechanisms. It is observed that this figure varies from 20 to 400 nodes. This indicates that the various platform have tested the mechanism on different size of network. However, the technique with higher network size for evaluation has the advantage over the techniques that use smaller network size. Finally, Table 2 shows the comparison of performance metrics used and claimed throughput. It is observed that the most common performance evaluation metrics are detection rate and packet loss rate. This is because the detection rate evaluates the performance of a proposed mechanism for its ability to distinguish an attacker from a genuine node in the network, whereas packet loss rate evaluates the ability of the proposed mechanism to recover the network statistics under the attack of selective forwarding. Only a few uses the power consumption as metric, nevertheless almost all metrics ultimately depend on the underlying power consumption model of the network [10]. The claimed throughput shows the throughput achieved by various detection mechanism in their respective attack scenarios. It is observed that Airehrour et al. have claimed to achieve 96% of throughput followed by the mechanism proposed by Huang et al. [4] with 88%.

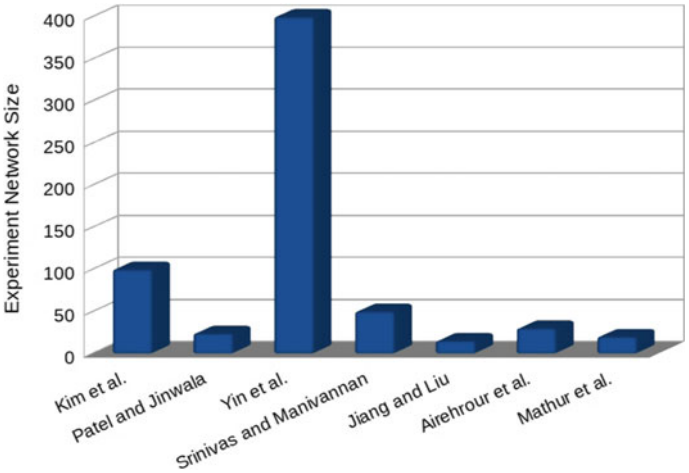


Fig. 5 Comparison of network size used in experiment

Table 2 Analysis of performance metrics used and claimed throughput

Authors	Throughput	Performance metrics
Kim et al. [8]	NA	Detection rate and average waiting time for better performance
Patel and Jinwala [13]	2.7 to 5.3 kbps	Packet loss rate and throughput
Yin et al. [21]	NA	Energy utilization and network lifetime.
Srinivas and Manivannan	NA	Normalized energy, latency, and shortest path.
Jiang and Liu [6]	44%	Receiver Operating Characteristic (ROC) curve, detection rate, false alarm rate and power consumption
Airehrour et al. [1]	96%	Throughput and packet loss rate
Mathur et al. [9]	83%	Neighborhood watch and energy/power consumption, network latency
Huang et al. [4]	88%	Throughput, detection accuracy rate, false detection rate, missed detection rate
Ding et al. [3]	85%	Detection rate, throughput, false detection rate, missed detection rate

6 Conclusion

There are different types of attacks in IoT networks. Selective forwarding is one of them. An attacker node would choose only a few packets to forward to its destination and dropping all the remaining packets. Different detection and prevention techniques have been analyzed in this paper based on different criteria used in several research papers such as performance evaluation metric, the routing protocol used,

network size, simulators, and detection criteria. This study will assist the researchers in understanding the shortcomings of the earlier work done and it might be useful in resolving them in the future.

References

1. Airehrour D, Gutierrez J, Ray S (2017) A trust-aware rpl routing protocol to detect blackhole and selective forwarding attacks. Aust J Telecommun Digit Econ 5. https://doi.org/10.18080/ajtde.v5n1.2
2. Čolaković A, Hadžialić M (2018) Internet of things (iot): a review of enabling technologies, challenges, and open research issues. Comput Netw 144:17–39. https://doi.org/10.1016/j.comnet.2018.07.017
3. Ding J, Wang H, Wu Y (2022) The detection scheme against selective forwarding of smart malicious nodes with reinforcement learning in wireless sensor networks. IEEE Sens J 22(13):13696–13706. https://doi.org/10.1109/JSEN.2022.3176462
4. Huang X, Wu Y (2022) Identify selective forwarding attacks using danger model: promote the detection accuracy in wireless sensor networks. IEEE Sens J 22(10):9997–10008. https://doi.org/10.1109/JSEN.2022.3166601
5. Jeevanantham S, Rebekka B (2022) Hierarchical stable election protocol for WSN-based IoT inhabitant and environmental monitoring applications. Int J Commun Syst, p e5301. https://doi.org/10.1002/dac.5301
6. Jiang J, Liu Y (2022) Secure iot routing: selective forwarding attacks and trust-based defenses in rpl network
7. Kamgueu P, Nataf E, Djotio T (2018) Survey on rpl enhancements: a focus on topology, security and mobility. Comput Commun 120. https://doi.org/10.1016/j.comcom.2018.02.011
8. Kim JD, Ko M, Chung JM (2022) Novel analytical models for sybil attack detection in ipv6-based rpl wireless iot networks. In: 2022 IEEE international conference on consumer electronics (ICCE), pp 1–3. https://doi.org/10.1109/ICCE53296.2022.9730349
9. Mathur A, Newe T, Rao M (2016) Defence against black hole and selective forwarding attacks for medical wsns in the iot. Sensors 16(1) . https://doi.org/10.3390/s16010118, https://www.mdpi.com/1424-8220/16/1/118
10. Mishra AK, Samantara MK (2016) A dynamic energy-efficient chain formation scheme for pegasis in wireless sensor networks. In: 2016 2nd international conference on computational intelligence and networks (CINE), pp 41–46. https://doi.org/10.1109/CINE.2016.15
11. Mishra AK, Sinha M, Tripathy A (2020) A sinkhole prevention mechanism for rpl in iot. Int J Comput Sci Eng 23:262. https://doi.org/10.1504/IJCSE.2020.111435
12. binti Mohamad Noor M, Hassan WH (2019) Current research on internet of things (IoT) security: a survey. Comput Netw 148:283–294
13. Patel A, Jinwala D (2022) A reputation-based rpl protocol to detect selective forwarding attack in internet of things. Int J Commun Syst 35(1):e5007. https://onlinelibrary.wiley.com/doi/abs/10.1002/dac.5007
14. Radoglou Grammatikis P, Sarigiannidis P, Moscholios I (2018) Securing the internet of things: challenges, threats and solutions. https://doi.org/10.1016/j.iot.2018.11.003
15. Srinivas T, Manivannan S (2021) Black hole and selective forwarding attack detection and prevention in iot in health care sector: hybrid meta-heuristic-based shortest path routing. J Ambient Intell Smart Environ 13:1–24. https://doi.org/10.3233/AIS-210591
16. Stoyanova M, Nikoloudakis Y, Panagiotakis S, Pallis E, Markakis EK (2020) A survey on the internet of things (IoT) forensics: challenges, approaches, and open issues. IEEE Commun Surv Tutor 22(2):1191–1218

17. Tawalbeh L, Muheidat F, Tawalbeh M, Quwaider M (2020) Iot privacy and security: challenges and solutions. Appl Sci 10(12). https://doi.org/10.3390/app10124102, https://www.mdpi.com/2076-3417/10/12/4102
18. Tournier J, Lesueur F, Mouël FL, Guyon L, Ben-Hassine H (2021) A survey of iot protocols and their security issues through the lens of a generic iot stack. Internet Things 16:100264
19. Verma A, Ranga V (2020) Security of RPL based 6LoWPAN networks in the internet of things: a review. IEEE Sens J 20(11):5666–5690. https://doi.org/10.1109/JSEN.2020.2973677
20. Wang Q, Tai W, Yi T, Ni M (2019) A review of the false data injection attack against the cyber physical power system. IET Cyber-Phys Syst Theory Appl 4. https://doi.org/10.1049/iet-cps.2018.5022
21. Yin R, Zhang F, Xu Y, Liu L, Li X (2020) A security routing algorithm against selective forwarding attacks in scale-free networks. Procedia Comput Sci 174:543–548. 2019 international conference on identification, information and knowledge in the internet of things. https://doi.org/10.1016/j.procs.2020.06.151, https://www.sciencedirect.com/science/article/pii/S1877050920316793

Performance Enhancement of the Healthcare System Using Google Cloud Platform



Subhadarshini Mohanty, Alka Dash, Subasish Mohapatra, Amlan Sahoo, and Subrota Kumar Mondal

Abstract Recent technology's ongoing evolution is unexpectedly altering many industries. A widely held idea states that the combination of scientific Internet of Things (IoT) devices would cause a significant shift, particularly in the healthcare sector. In the implementation of vital medically accepted hardware, software, and connections, the usage of scientific IoT devices has repercussions. This report seeks to identify upcoming standard performance challenges by studying current IT cloud services. As a result, there are now new community performance demands, and there is a need for brand-new community performance standards and recommendations that are focused on network-related, scientific IoT devices, their users, and the associated cloud services in integrated home care. The Internet of Things (IoT) produces plenty of statistics. Scientific sensor and actuator statistics can be remotely saved and analyzed using distributed servers thanks to cloud computing. The cloud version for data storage and analysis is becoming increasingly unable to handle the data avalanche, and processing is being pushed out to the edge of the network where the data-producing devices are. Region computing and fog computing are data management architectures that can unload data from the cloud, a method close to the patient, and communicate data from tool to tool or from tool to person in seconds or milliseconds. Fog computing (with nearby nodes) and location computing can be used to process sensor statistics close to the detecting and actuating devices (withinside the sensing devices). Fog computing and location computing, when compared to cloud computing, offer five advantages: faster data transmission, much less reliance on constrained bandwidths, more privacy and security, and greater control over data generated in foreign nations where judicial recommendations may also moreover

S. Mohanty · A. Dash · S. Mohapatra (✉) · A. Sahoo
Odisha University of Technology and Research, Bhubaneswar, Odisha, India
e-mail: smohapatra@outr.ac.in

S. Mohanty
e-mail: sdmohantycse@cet.edu.in

S. K. Mondal
Macau University of Science and Technology, Av. Wai Long, Contai, Macao
e-mail: skmondal@must.edu.mo

© The Author(s), under exclusive license to Springer Nature Singapore Pte Ltd. 2023

S. Chinara et al. (eds.), *Advances in Distributed Computing and Machine Learning*,
Lecture Notes in Networks and Systems 660,
https://doi.org/10.1007/978-981-99-1203-2_15

limit the use or allow unwanted governmental access, and lower costs because more sensor-derived data is used.

Keywords Cloud computing · Clustering · Internet of things · Google cloud platform · Performance enhancement · K-means · K-Nearest neighbor

1 Introduction

With the speedy growth of the Internet of Things healthcare, a couple of home medical equipment, which incorporates infrared thermometers, blood pressure monitors, and coronary heart rate monitor, is already pretty often utilized by human beings in ordinary lives [1]. Therefore, outpatients can gather personal scientific diagnoses by uploading their physical health data acquired from IoT scientific gadgets to acquire professional healthcare concepts from the cloud. Furthermore, the forms of personal provider scientific gadgets have become more portable, accurate, and specialized with the fast growth of data technology [2]. Like e-commerce, it's far going to be incredibly accessible for outpatients to acquire custom-designed and professional diagnostic files every time and anywhere, especially with the popularity of IoT healthcare.

Smart IoT healthcare in cloud environments is not possible to withstand and promising future style inside facet of the medical field. Firewalls manage visitors and separate networks with one-of-a-kind protection requirements. They are used by organizations to protect their community reasserts from unauthorized access. They look at community visitors (packets) and carry out movements based absolutely on the rules, and the movements awesome therein within the method [3]. An agency should have as many as masses of firewalls scattered around its community, and consequently, the location of firewalls and their method play a key characteristic of their effectiveness. In exercising network, directors decide the vicinity and configuration of firewalls primarily based totally on community safety layout standards and first-rate practices [4]. However, this approach is at risk of error, particularly for big networks. Therefore, the right approach is wanted to attain and assure a steady community layout.

2 Literature Survey

Zhang et al. proposed high-performing IoT devices using algorithms like MQTT, Sliding window algorithm, Token bucket algorithm, and resource demand estimation algorithm. Its drawback is securely obtaining private medical diagnosis data without affecting performance isolation and service quality is still a challenge [5].

Mansour et al. proposed a model for diabetes and heart disease diagnosis using IoT and AI convergence techniques. The model goes through several phases, including data collection, preprocessing, classification, and parameter adjustment. Wearables

and sensors that are part of the Internet of Things make it possible to collect data easily, and AI algorithms use that data to diagnose diseases. Using Cascaded Long Short-Term Memory-based Crow Search Optimization techniques, for example (CSO-CLSTM). It was shown that there was a significant likelihood of entering local optima and a slow search precision [6]. Mukherjee et al. proposed an enhanced KNN researching IoT-cloud-based COVID-19 detection while looking for the best value for the k. The Enhanced K-Nearest Neighbor method is the algorithm employed (eKNN). The paper's flaw is the slight increase in computation time brought on by feature selection implementation [7].

Shah et al. proposed that The development and rising popularity of the cloud and IoT have emerged as the key enablers of seamless healthcare applications that affect our daily lives. The integration of cloud and IoT technologies into healthcare systems is highly motivated by the need for effective computing infrastructures, an unlimited data warehouse, optimal network performance, and availability employing IoT and cloud computing technologies. The challenge is to achieve faultless and perfect convergence of cloud-based and Internet of Things-based healthcare applications [8].

M. Anuradha et al. proposed that This research project's major goal is to increase the security and usability of accessing cancer patient information saved in the cloud using deep learning tools like Alexnet, VGG 16, and GoogleNet. Input is encrypted using the AES algorithm. NetBeans, CloudSim [9]. Milovanovic et al. proposed that being able to include a wide range of devices and services into an architecture for the Internet of Things (IoT) is a challenging task. Data collectors and wireless sensors for healthcare applications, such as ZigBee-based WSNs and body sensor networks, must meet stringent standards for system dependability, quality of service (QoS), and security (BSN). The disadvantages include access control, data confidentiality, location privacy, outsourced data mining that protects privacy, and the enormous amount of computing complexity that still must be researched [10].

Kumar et al. discussed the big data model in cloud IoT and used the cuckoo search algorithm, PSO algorithm, and ABC algorithm to propose an intelligent architecture of HIS also they compared the efficiency rate among the three algorithms, but the disadvantage is that the task scheduling process by using the different optimization techniques and environments, especially data processor used in a limited amount is lacking [11]. Santos et al. said that, by the use of cloud, fog, and, edge computing, they have also used stochastic Petri nets, then Sensitivity analysis is done and then they modeled an e-health monitoring system, then they analyzed the throughput of the system, service time taken by the model, after that prototyping of e-health monitoring system is done but the disadvantage is there are fewer cloud servers and redundancy is more [12].

Motivation. There are many interesting topics under cloud computing and these days fog and edge computing are used more often. And, these days medical IoT devices are developed using these technologies, also challenges are there in this technology can be the future scope. Mainly, we can enhance the performance and security of IoT devices using cloud computing. We can integrate almost everything into the cloud for better performance and high security. This technology is the most

used these days. Thus, many MNCs have launched their cloud platforms like Azure launched by Microsoft, AWS launched by Amazon, GCP launched by Google, and so on.

Objectives. The major contribution or the objectives of this paper is noted as follows:

- To evaluate the performance of IoT healthcare devices in normal scenarios.
- To propose a hybrid technique for faster and more efficient communication between IoT healthcare devices and the cloud.
- To compare the existing communication technologies with the proposed technique.

3 Proposed Methodology

The act or process of improving performance at a task or activity by increasing one's physical and mental endurance for it through exercise, encouraging self-talk, using imagery, and other methods.

Performance enhancement in Cloud Services. According to performance research from 2020, more than 90% of firms employ cloud services, and 67% of performance departments have expressed concerns about the lack of transparency in their cloud infrastructure and compliance. An incorrect setup for cloud formation has been identified as the root cause. Additionally, the number of Internet of Things (IoT) active connections will rise globally from 8.74 million in 2020 to 25.44 million in 2030. The advantages of new technologies and cloud services will boost innovation, especially in the healthcare industry, but they will also make attacks more likely. Particularly sensitive information is handled and stored in the healthcare sector. It is quite evident that the healthcare industry is under pressure to convert to new telecommunications technologies and use them more quickly in light of the current COVID-19 outbreak. Given how swiftly the business sector is digitizing, it may already be clear that producers of medical IoT gadgets are keen to enhance their present hardware and add more cloud services.

This trend raises several IT performance challenges.

- With the improvement of medical IoT devices, producers are seeking to increase their commercial enterprise services with the aid of using growing virtual services.
- As a result of financial restrictions, scientific healthcare facilities will integrate their outdated scientific inventory into their current IT infrastructure.
- This Medical Device Directive serves as the foundation for ongoing research into how to enforce IT efficiency in the field of network-connected medical devices.
- It is crucial to keep in mind that these devices were not designed to operate in an IoT network and consequently lack the necessary performance architecture to operate in this environment.

The goal of the International Medical Device Regulators Forum's (IMDRF) current work item, the "Medical Device Cyber overall performance Guide," is to "promote a globally harmonized approach to medical tool cyber overall performance that at a fundamental stage guarantees the protection and overall performance of medical devices and encouraging information." The article "Cyber Performance Requirements for Network-Connected Medical Devices" offers makers special assistance in figuring out how to lessen known performance concerns. The location (such as a private home, clinic, or nursing home) in which medical IoT devices may be incorporated, the focused consumer's technical skills, and the specific IT landscape into which the devices are integrated are all key distinctions to make.

Performance Enhancement in IoT Devices. The performance of IoT primarily based networks typically degrades because the device starts capabilities because of numerous reasons especially because of the presence of heterogeneity; want to move the information from layer to layer in both of the instructions to transport the information from sensors to the storage of the identical on a remote server and to reply queries concerning the functioning of sensing and actuating mechanisms; the queries initiated from a remote place with the aid of using the customers through a cloud computing interface.

The overall performance of IoT networks significantly suffers because of many problems which need to be well mitigated to make sure that the overall performance of an IoT network is maintained as designed. One of the major worries is to mitigate the latency that exists in each of the layers of the IoT-primarily based totally network.

Proposed Model

See (Fig. 1).

Proposed Model.

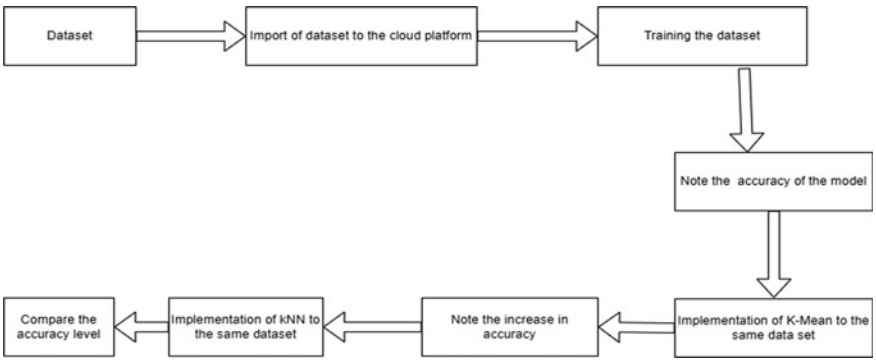


Fig. 1 Flowchart of the proposed methodology

3.1 Collection of Data

Data related to healthcare or healthcare devices like insulin value, blood pressure, and so on are recorded and collected from different sources or repositories.

3.2 Import Dataset to the Cloud Platform

The collection of datasets is imported to the cloud to train evaluate and use those to see the accuracy of the raw data taken.

3.3 Training Dataset

The collection of data is trained for further uses and then it is modeled to give the accuracy of the raw data.

3.4 Note the Accuracy

The accuracy of the raw dataset is noted and the next aim is to increase it for the machine or system to perform it better.

3.5 Implementation of K-means Clustering Algorithm

The K-means clustering algorithm is implemented in the collection of datasets after training and scaling. We are supposed to see an increase in accuracy and better performance than the raw data imported to the cloud.

3.6 Note the Increase in Accuracy

The accuracy we get after applying the k-mean being scaled is more than the raw data when imported to the cloud.

3.7 Implement of K-NN Classification Algorithm

The K-NN clustering algorithm is implemented in the same collection of datasets after training and scaling we are supposed to see an increase in accuracy and better performance than the K-Mean clustering algorithm.

3.8 Compare the Accuracy Level

We compare the accuracy of the raw dataset, k-mean, and k-nearest neighbors algorithm (k-NN) and found that k-NN gives the best accuracy here.

4 Simulation Environment

Integration with the Cloud. We have taken a collection of datasets and integrated them into the cloud to train evaluate and test that collected dataset. Here, we have used Google Cloud Platform (GCP) for cloud services.

Google Cloud Platform. A group of Google's cloud computing services is known as the Google Cloud Platform (GCP). The internal architecture is the same as what Google uses for its consumer products, such as Google Search, Gmail, Drive, and YouTube. Google Cloud Platform provides serverless computing environments, platform as a service, and infrastructure as a service.

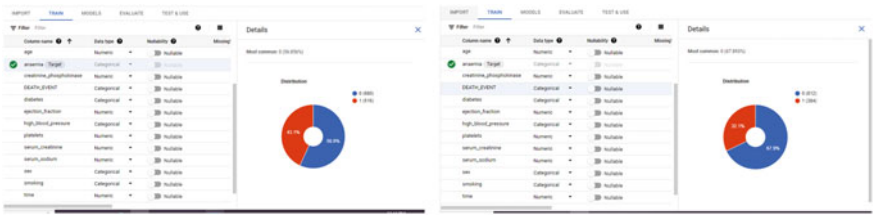
Dataset Importing to the GCP

See (Fig. 2).

Training of Dataset

See (Fig. 3).

Dataset Importing to the GCP.



(a) **(b)**

Fig. 2 **a–b** Importing of the categorical dataset into the google cloud platform (GCP)

Training of Dataset.

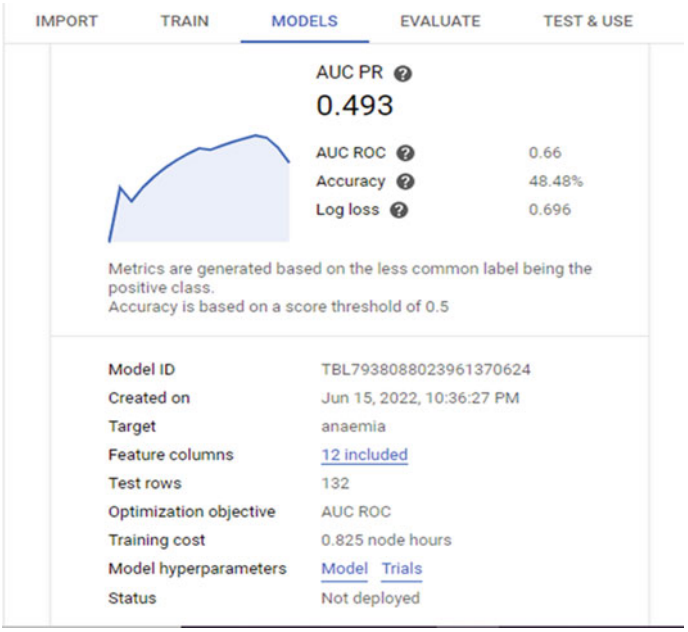


Fig. 3 Training of the imported dataset in the google cloud platform (GCP)

Evaluation In Cloud Environment

See (Fig. 4).

Evaluation In Cloud Environment.

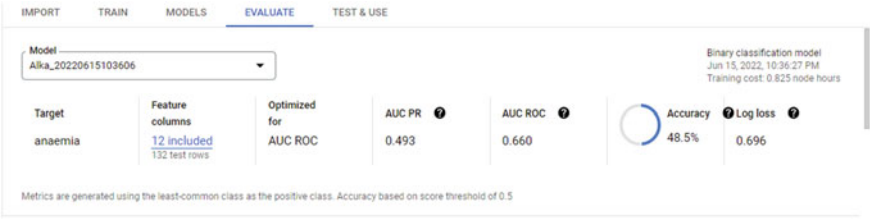


Fig. 4 Evaluation model in GCP

5 Result and Discussions

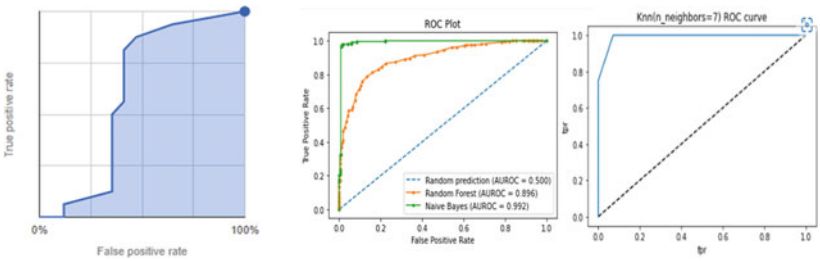
We found that, after importing the collected dataset, the data get trained, then evaluation takes place, then we can model the dataset, and further test it and use it. We can check the details of each attribute by clicking on it as shown in the figures above, this happens while training the dataset. Then the dataset is modeled, here we can view a graph that shows the accuracy of the dataset, we can also see the model ID, the date on which it is created, the target attribute, and so on. We can see this in figure in the training of the dataset. Then the dataset is evaluated, we can see the results in the figure in the evaluation of the dataset. In testing, predictions take place such as batch prediction, and online prediction, then we can export the model (Figs. 5 and 6).

Comparison of the Applied Techniques

When we integrate the raw data into the GCP, we see an accuracy of about 49.8%, where the data gets trained and evaluated, then to get a better accuracy we applied the k-means algorithm. Here, we see an accuracy of 87% after training the collection of datasets here we get an accuracy of 87% after this the accuracy kept on decreasing then we applied the KNN algorithm. Here, we get a better accuracy of about 96% (Fig. 7).

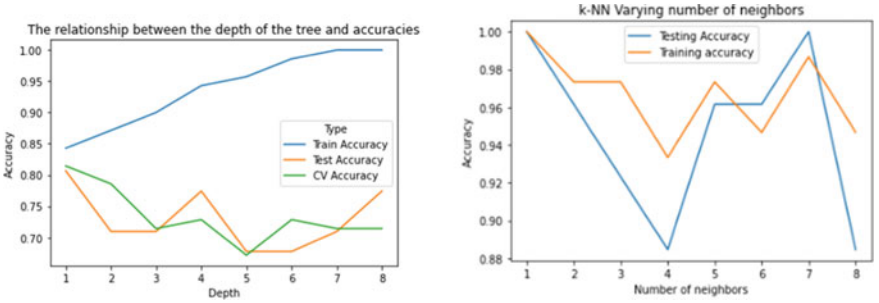
Depth	Train Accuracy	Test Accuracy	CV Accuracy
1	0.8429	0.8065	0.8143
2	0.8714	0.7097	0.7857
3	0.9000	0.7419	0.7143
4	0.9429	0.7419	0.7286
5	0.9571	0.7097	0.7000
6	0.9857	0.6774	0.7286
7	1.0000	0.7097	0.6857
8	1.0000	0.7419	0.7000

Fig. 5 Accuracy table of K-means



(a) (b) (c)

Fig. 6 **a** RoC curve of cloud dataset; **b** RoC curve of K-means; **c** RoC curve of KNN



(a) (b)

Fig. 7 Training and testing accuracy of **a** K-means; **b** KNN

6 Conclusion and Future Scope

Healthcare devices are now have been a part of everyone's life, and have become necessary for emergency conditions. So, to make life easier, we have compared the enhancement of performance in an IoT-cloud-based environment. In this paper, we have compared the accuracy of the raw data imported to the cloud without scaling and then the collection of datasets is scaled and the k-means clustering algorithm is implemented, we see an improvement in the accuracy. Again, we implement the KNN classification algorithm for better accuracy and got the best accuracy possible.

This paper is based on the enhancement of performance after integrating the clustering algorithms into the cloud. Here, we have used the k-means clustering algorithm and KNN classification algorithm. And, we have seen that KNN is better. Now, we can see that KNN is the best for enhancing performance using cloud-IoT.

In the future, we can work on the security of the KNN algorithm and make it even more efficient for performing better with less time and space. And, c-means will also be used to check if it will give better accuracy and security to the data or information given by the user.

References

1. Unnisa A et al (2020) A study on review and analysis of cloud, fog and edge computing platforms. IJISRT 5(11). https://ijisrt.com/a-study-on-review-and-analysis-of-cloud-fog-and-edge-computing-platforms
2. Mondragón-Ruiz G et al (2021) An experimental study of fog and cloud computing in CEP-based real-time IoT applications. J Cloud Comput 10(1). https://doi.org/10.1186/s13677-021-00245-7
3. Gleißner M, Dotzler J, Hartig J, Aßmuth A, Bulitta C, Hamm S (2021) IT security of cloud services and IoT devices in healthcare

4. Klonoff DC (2017) Fog computing and edge computing architectures for processing data from diabetes devices connected to the medical internet of things. J Diabetes Sci Technol 11(4):647–52. https://doi.org/10.1177/1932296817717007
5. Zhang Y et al (2021) High-performance isolation computing technology for smart IoT healthcare in cloud environments. IEEE Internet Things J 8(23):16872–16879. https://doi.org/10.1109/jiot.2021.3051742
6. Mansour RF et al (2021) Artificial intelligence and internet of things enabled disease diagnosis model for smart healthcare systems. IEEE Access 9:45137–45146. https://doi.org/10.1109/access.2021.3066365
7. Mukherjee R et al (2021) IoT-cloud based healthcare model for COVID-19 detection: an enhanced k-nearest neighbour classifier based approach. Computing. https://doi.org/10.1007/s00607-021-00951-9
8. Shah J, Bhat H, Khan A (2020) Integration of cloud and IoT for smart e-healthcare. https://doi.org/10.1016/B978-0-12-819664-9.00006-5
9. Anuradha M et al (2021) IoT enabled cancer prediction system to enhance the authentication and security using cloud computing. Microprocess Microsyst 80:103301. https://doi.org/10.1016/j.micpro.2020.103301
10. Milvanovic D, Zoran B (2017) Cloud-based IoT healthcare applications: requirements and recommendations. Int J Internet Things Web Serv. www.iaras.org/iaras/filedownloads/ijitws/2017/022-0009(2017).pdf
11. Yang H, Liang S, Ni J, Li H, Shen XS (2020) Secure and efficient *k* NN classification for industrial internet of things. IEEE Internet Things J 7(11):10945–10954. https://doi.org/10.1109/jiot.2020.2992349
12. Kumar P, Silambarasan K (2022) Enhancing the performance of healthcare service in IoT and cloud using optimized techniques. IETE J Res 68(2):1475–1484
13. Santos GL, Takako Endo P, Ferreira da Silva Lisboa Tigre MF, Ferreira da Silva LG, Sadok D, Kelner J, Lynn T (2018) Analyzing the availability and performance of an e-health system integrated with edge, fog and clou,d infrastructures. J Cloud Comput *7*(1). https://doi.org/10.1186/s13677-018-0118-3
14. Javaid M, Khan IH (2021) Internet of Things (IoT) enabled healthcare helps to take the challenges of COVID-19 Pandemic. J Oral Biol Craniofacial Res 11(2):209–214. https://doi.org/10.1016/j.jobcr.2021.01.015
15. Abdali-Mohammadi F, Meqdad MN, Kadry S (2020) Development of an IoT-based and cloud-based disease prediction and diagnosis system for healthcare using machine learning algorithms. IAES Int J Artif Intell (IJ-AI) 9(4):766. https://doi.org/10.11591/ijai.v9.i4.pp766-771
16. Shirley DMAJ (2020) A cloud IoT based smart patient health monitoring system. Adalya J 9(1):1301–2746. http://adalyajournal.com/
17. SR (2020) Tele-health in population health management using IoT devices in cloud based environment. J Adv Res Dyn Control Syst 12(SP7):212–221.https://doi.org/10.5373/jardcs/v12sp7/20202100
18. Kumar P, Silambarasan K (2019) Enhancing the performance of healthcare service in IoT and cloud using optimized techniques. IETE J Res 68(2):1475–1484. https://doi.org/10.1080/03772063.2019.1654934
19. Enhancement of IoT based smart salvation and monitoring devices for agriculture (2018) Int J Recent Trends Eng Res 204–209. https://doi.org/10.23883/ijrter.conf.02180328.034.6fsd3
20. Elrawy MF, Awad AI, Hamed HFA (2018) Intrusion detection systems for IoT-based smart environments: a survey. J Cloud Comput 7(1). https://doi.org/10.1186/s13677-018-0123-6
21. Mohapatra S, Sahoo A, Mohanty S, Patra PK (2022) Ubiquitous healthcare system using recent ICT. In: Intelligent systems. Springer, Singapore, pp 81–91
22. Dabiri S, Azizi S, Abdollahpouri A (2022) Optimizing deadline violation time and energy consumption of IoT jobs in fog–cloud computing. Neural Comput Appl. https://doi.org/10.1007/s00521-022-07596-5

23. Artificial intelligence-enabled wearable medical devices, clinical and diagnostic decision support systems, and Internet of Things-based healthcare applications in COVID-19 Prevention, Screening, and Treatment (2021) Am J Med Res 8(2):9. https://doi.org/10.22381/ajmr8220211
24. Paul M, Das A (2017) Service level agreements for smart healthcare in cloud. Int J Grid High-Perform Comput 9(4):57–70. https://doi.org/10.4018/ijghpc.2017100104

Front-End Security Analysis for Cloud-Based Data Backup Application Using Cybersecurity Tools



S. MD. K. N. U. Affan Ahamed, Vinay Mathew Wilson, Manu Elappila, and Sachin Malayath Jose

Abstract In this challenging, demanding, daunting, and competitive business world, the rise, and growth of cybercrimes are very high. With the proliferation of Cloud Computing techniques, usually in industrial arenas, business information and important clients' data are stored and managed using cloud platforms. Application programs are developed to handle such valuable information assets of the organizations. Cloud backups are provided for these client data where security is the most concerning aspect. There are many vulnerabilities in the current scenario where intruders can cause havoc. Destruction of the product can happen by exploiting vulnerabilities that can put the company and the product in jeopardy. It may create a bad impression about the organization among the customers, competitors, and the public world. This paper shows the work done by a cyber security team whose main objective is to run vulnerability analysis and mitigate threats on an application that backs up the client's data to the cloud. Cyber Security is an important aspect in all types of businesses because it protects all categories of data such as fragile data, private information, intellectual property data, and other data including governmental and industrial information systems from theft and damage which concludes in huge financial loss and loss of client data.

Keywords Cyber security · Cloud computing · Data backup application · Front-end security analysis

S. MD. K. N. U. Affan Ahamed · V. Mathew Wilson · M. Elappila (✉)
CHRIST (Deemed to be University), Bengaluru, India
e-mail: manu.elappila@christuniversity.in

S. MD. K. N. U. Affan Ahamed
e-mail: u.affan@btech.christuniversity.in

V. Mathew Wilson
e-mail: vinay.mathew@btech.christuniversity.i

S. Malayath Jose
University of Liverpool, Liverpool L69 3BX, United Kingdom
e-mail: sgsjose@liverpool.ac.uk

© The Author(s), under exclusive license to Springer Nature Singapore Pte Ltd. 2023

S. Chinara et al. (eds.), *Advances in Distributed Computing and Machine Learning*,
Lecture Notes in Networks and Systems 660,
https://doi.org/10.1007/978-981-99-1203-2_16

1 Introduction

In this competitive business world, cybercrimes are increasing by exponential magnitude. With the rapid increase of Cloud Computing techniques in industrial domains, business information and important data from clients are managed and backup provisions provided using cloud platforms. Organizations develop specific Application Programs for handling such valuable information and assets. Here, security is the most concerning aspect where cloud backups are used to handle the client data. In the current scenario many vulnerabilities are there, and intruders can cause havoc. For example, intruders can create destruction of the product of the company by exploiting vulnerabilities, which can pull the company and the product into jeopardy. This will generate a bad impression about the organization among competitors and in the public world. The primary focus of this work is on the security aspects of the Cloud backup application, and to find out vulnerabilities that may lead to possible attacks in the application, and the ways to mitigate them. A vulnerability analysis of the front end of the cloud backup application is presented in this work. There are many problems in Cloud-based backup systems in the current market scenario. There are many flaws in the aspect of security and in many situations they are vulnerable. So, to understand the vulnerabilities we need to perform different attacks and act as an attacker to figure out the vulnerabilities. In the given scenario many of the vulnerabilities are on the front-end part of the application so gathering all the information on the vulnerabilities is reported to the development team as it's their role to correct the mistake which resulted in the vulnerability. As we understand that most of the cloud-based backup platforms are in the form of web applications, hence it is needed to consider the vulnerability aspects of a Web application. So, the first thing that pop-ups on anyone's mind as a vulnerability analyst is the Login verification. Firstly, it is needed to consider the login verifications as they are the first ones to get targeted by sniffing attacks, which especially happens in the case where the intruder manipulates POST requests [1]. The main objective of a cloud backup system is to back up the client's data in a safe location and make it available to their clients whenever they need it from anywhere around the world.

Network safety is of utmost significance due to the fact that it inculcates everything that associates with safeguarding our data from digital assailants who are required to gather the client's information and make use of it by wrong means. This might potentially indicate testy information, legislative and industrial data, individual data, recognizable data (PII), licensed innovation, and safeguarded wellbeing data (PHI).

As this field gained ground and advanced, it is essential to digitally guard projects and components; it is vital as a bedrock should be established to safeguard information and utilize it to everybody's greatest advantage. On a daily basis, it is known that most in the public eye are contingent on foundations, such as medical care organizations, fiscal aid projects, and sustainable power plants. It is our objective to keep our general public active. There are many problems with respect to cyber security. In this aspect, the login page plays a vital role in authentication for the front-end

dashboard, from which anyone will be able to read or sniff requests and respond with a simple man-in-the-middle attack. As stated, we will be performing different attacks and will be analyzing the cloud-based backup systems.

2 Related Works

A few research papers [2–5] were audited so as to encourage a progressively guided condition for the development stage. The papers referred to in this process have shed light on the best practices and approaches to be considered. These papers were of tremendous help in finishing up the tasks. They were helpful in making us understand the importance of vulnerability analysis and helped in focusing on many core issues and its related aspects these papers have all shed light on mitigation and remediation works that must be focused on to build a top-notch product with very strong security features that can withstand attacks.

Before explaining the intricacies of this work and it is important to have a clear understanding of the current cyber world. This is to understand the working routine, the analysis benchmarks kept, the filters applied on different metrics, and the herculean tasks associated with keeping data safe and secure. After all the primary aim of every client is to enhance safety and increase the ease of use as it is very important for all the people as well as the organization [6]. This starts right from understanding the expectation of the clients.

There have been many attacks that happened at various points in time and the OWASP top 10 vulnerabilities were always in our concern. As a product security team, one should be in the work to figuring out ways to protect the system and mitigate the vulnerabilities [7]. There were other studies on access tokens and refresh tokens client's backup request and response channels and how they should work in different environments.

3 Methodology of Experimentation

The research methodology that is adopted for this work is to study how targeted system works and create a document on how we carried out those attacks we had to use and understand different software such as Burp Suite, Fiddler, Microsoft attack surface analyzer, Microsoft threat analyzer, Snyk, and Microsoft threat modeling tool. Set it up, run these, and create reports of the vulnerabilities that we found and discuss ways to mitigate them. So, the research methodology overall adopted was the observation of the system, initiating attacks to figure out how the system responded, and finally finding ways to crack in and if its success finds ways to mitigate those security threats. The research methodology for this case study was to analyze the backup security software, by attacking the software with different tools and analyzing the reports of vulnerabilities that were released from the respective tools which were

used for the analysis. Using these results, we try to understand the software regarding how it responds to the given situation in hand and find out the ways to mitigate those security threats.

We need to look into different factors when we are trying to create a proper security system for the clients' workspace as it is our responsibility to take care of it. We need to understand different vulnerability points that can cause a security issue. We need to closely monitor our security issues as well from a backup security provider's perspective. We can make use of an already existing security system or we can develop our own procedures and techniques for a secured workspace by making a proper difference in what is lacking in the existing one. So here to understand the existing vulnerabilities we look into the OWASP top 10 vulnerability list and try to relate the list with the threats which are possible in the software. The next step was to perform those attacks and check if any of those vulnerabilities are still applicable to our software. The Security issues which were examined are SQL injection, XXS, etc. The procedure for these was taken from various open sources.

As we are mainly focusing on the Front-End vulnerabilities for a UI dashboard, we are going to use the tool Burp suit which is mostly used for Penetration testing of Web Applications. As mentioned earlier we carry out attacks on various aspects of the software and verify the work with the development team to make the necessary changes with respect to the Sentinel UI and implement it. Most of the mitigation is automated and has to create or run these tests in different vector points or use different tools to find vulnerabilities in them [8]. As the software holds most of the client data as a provider nobody wants to risk it.

3.1 Tools Considered for Cyber Analysis

Following are the methods that are used for building different threat analysis

Attack Surface analyzer: It analyzes changes made to the attack surface of the operating system and tell particular changes causing threats. This software helps us to do a comparative scan before versus after doing the necessary changes to the system and gives a report on changes between the two scans. Where an unauthorized user can enter/retrieve data, keeping the attack surface as small as possible is a basic security measure.

Microsoft threat modeling tool: Sanctioned as the core component of the Microsoft Security Development Lifecycle, Microsoft threat modeling is a tool that enables one to innovate the designs. It is an engineering technique that can be used by any developer to achieve any one or all of the following: Convey about their system's security design. Through an authorized approach, inspect those designs for prospective security vulnerabilities. The proposition as well as governing of security mitigation.

HCL AppScan: The intention of this tool is to analyze on-premise as well as web applications for security vulnerabilities throughout the development process.

DevSecOps has an edge since it pinpoints application vulnerabilities, and permits flexible remediation in every phase of the development lifecycle.

On its own, the mentioned product scrutinizes the functioning of each application, whether an off-the-shelf application or internally developed, and develops a program intended to test all of its functions for both common and application-specific vulnerabilities. SAST, DAST, IAST as well as Mobile Analysis converse to the user's source code; In addition, it will keep checking for vulnerabilities. All these functionalities can be efficiently performed by this group of the family.

Burp Suite: Popularly known as Burp Suite, or Burp is a congregation of tools. Mainly used for purposes such as web application penetration testing. It is created by The Port swigger company. Their objective is to offer a compact or unified toolkit, and its features can be expanded in folds by installing BApps, which are its add-ons. In terms of application, professional researchers in the field of web application security and bug bounty hunters use it the most.

Spider: It's a web crawler or spider that maps the target web application. The map ping aims to create a list of endpoints that can be examined for functionality and potential vulnerabilities.

Proxy: Burp Suite features an intercepting proxy that helps the user access and change request and response contents while in transit. It also allows the user to submit the under-monitored request/response to another appropriate Burp Suite tool, eliminating the need for copy-paste. The proxy server may be programmed to use a particular loop-back address and port. Unique forms of request-response pairs may be filtered out using the proxy.

Intruder: This is used to pass a series of values through a single input point. The values are run, and the success/failure and size of the content are evaluated. Anomalies usually result in a difference in response code or response content length. For payload position, Burp Suite supports brute-force, single values, and dictionary files. The intruder is used for the following purposes:

1. Pin forms, Password forms, as well as other forms are vulnerable to brute-force attacks.
2. The dictionary attack on password types, which are vulnerable to SQL injection or XSS.
3. On the web app, rate limiting is being tested and attacked.

Repeater: Repeater allows a person to submit requests continuously when making manual changes.

Sequencer: The sequencer is an entropy checker that ensures that tokens created by the web server are random. Anti-CSRF and Cookies tokens are examples of these tokens, which are often used for the authentication of sensitive operations. These tokens should preferably be generated in a truly random way, with the likelihood of each potential character occurring at each position shared uniformly. This can be accomplished both in terms of bits and characters. An entropy analyzer verifies that

this concept is valid. It functions like this: the tokens are considered to be random at first. The tokens are then placed into a series of checks to see whether they follow those requirements. A significant point is a minimum value of probability that a token would exhibit for an attribute, such that if the token's characteristic probability is less than the significance level, the argument that the token is random is dismissed. This method can be used to identify and count vulnerable tokens.

Decoder: Some of the methods listed in Decoder are URL, HTML, Base64, Hex, and other popular encoding methods. This approach is useful for discovering datasets in parameter or header values. It's also used to build payloads for a number of vulnerability types. It's used to find the most common IDOR and session hijacking events.

Extender: To amplify the potential of the tool suite's capability, External modules can be implemented into Burp Suite. External modules are abbreviated as BApps. However, there is no change in its functionality, that is, it functions in the same fashion as browser extensions do. These functions can be retrieved, updated, mounted, and uninstalled. While some of them can be used for the free community version, others include the paid professional or technical version.

Scanner: There is a lack of a Scanner in the group version. It impulsively hunts through the website for various or specific vulnerabilities and records them, in addition with factors regarding the degree of trust in each discovery and the difficulty of exploitation. It's re-evaluated on a daily basis so as to make an addition of recent and lesser-known flaws.

Snyk: Snyk is a cyber security and vulnerability analysis tool that lets you find major threats in your code. Snyk runs vulnerability analysis on your code to produce output. Discover as well as innately alleviate vulnerabilities in your code; with the objective to find other vulnerabilities such as open-source dependencies, containers, and framework as code. Snyk lets continuous scans of containers and your code it can be easily integrated into your IDE and run the packet manager to find out vulnerabilities in DLL files.

Fiddler: Fiddler is a debugger that may also be used to study request and response patterns and how websites work. Capturing Traffic also provides certain security features, such as establishing rules to affect how a website works and decoding HTTP/HTTPS traffic to discover weaknesses. Log, inspect, and change HTTP(s) network requests and server responses successfully. In record time, go from error to repair. Import/export requests in a variety of formats and eliminate the guesswork. Create custom scripts to help you debug faster. Debug quickly and build web communication code more quickly, freeing more time for other things.

4 Results and Analysis

The work which was going on and all the results were obtained from a Cloud-based data backup application that helped the clients to back up the data in situations where the data is being lost in any form be it because of natural calamity or some sort of Cyber Attack [9]. So, the main motto here was to establish a proper security threat handling mechanism and to remove any type of existing vulnerabilities. Initially, there were no such protocols established. So below are the results from the individual tools which we have used for testing purposes.

1. **Burp Suite**: The use of burp suite for penetration testing has been the best in my experience throughout this work. We have tested out on many common OWASP vulnerabilities. Using Burp we ensured that there were no such vulnerabilities present in the sentinel dashboard. SQL and XSS scripting were getting blocked. In the initial phase using burp's proxy, it has been acted as the man in the middle where it is able see the react request and response. This was used to find out how the system behaves and reacts. Screenshot of BurpSuit is shown in Fig. 1.
2. **Nessus**: A vulnerability scanner was used to cross-check the results and we have successfully tackled most of the errors. Screenshot of BurpSuit is shown in Fig. 2.
3. **Auth0**: It is one of the most leading protection platforms that provide authentication and logging which has multi-factor authentication and a single-entry logging system with state-of-the-art technology so that no one else can access your folders unless authorized.
4. We had in-depth research and study on content security policy (CSP). This prevents hackers from doing XSS, Injection, and CSRF attacks as the contents or the actions the clients enter into the system will be sanitized these commands so that scripting attacks will not work. CSP lets you define certain rules in your system.
5. Microsoft Threat Analyzer tools were used in between particular intervals to figure out any change in the system before v/s after installation of the Rebit desktop and the changes it had made were observed and rectified and ports opened were in adjourning to our development team's directions.
6. The cyber security and vulnerability issues were rectified and cleared and mitigation plans were attached and forwarded to the system development team for rectification in code, patching up vulnerabilities, and providing extra layers of CSP protection.
7. We also discussed the importance of access tokens and refresh tokens implemented in Rebit with the management team and the importance of them and their duration were discussed and rectified.
8. **Snyk** was our last analysis to find vulnerabilities with respect to our core code infrastructure. I had to connect Snyk to our IDE which was a visual studio and ran the scans to find out associated vulnerabilities with respect to dependency files or DLL files. And the vulnerabilities were mitigated by upgrading the packages to the latest forms.

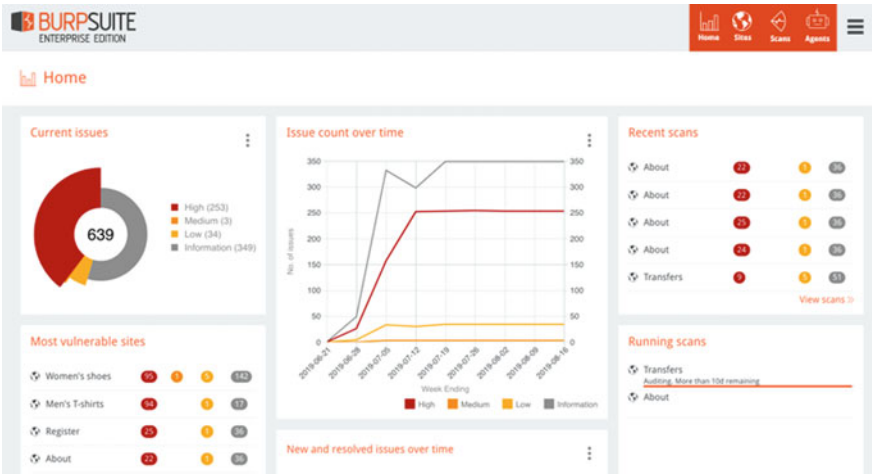


Fig. 1 Screenshot of Burp Suit

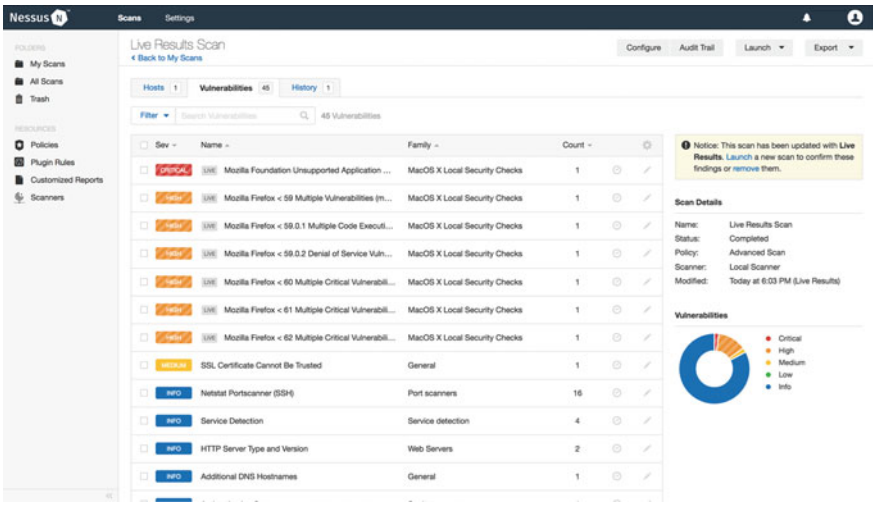


Fig. 2 Screenshot of Nessus

4.1 Discussion

The vulnerabilities which were found were patched and most of the vulnerabilities were reported and mitigated. The process of security analysis goes on until new bugs and lacuna open up in the system as it is impossible to say that 100 percent security has been achieved. The process of threat detection and vulnerability assessment will go on forever to depreciate the changes of an attack or to loosen up the impact of an attack.

5 Conclusions

The application was executed remembering the necessities and benchmark principles looked for by the organization. The previously mentioned assignments required a genuine measure of difficult work, devotion, earnestness, and responsibility. The capabilities and farfetched usage were already seen. Although numerous complexities were involved the entire set of tasks ranging from finding errors to mitigation were thoroughly relished upon. The countless number of things learned in this period is just not quantifiable. Not only has it triggered an unsurpassable interest in the domain but it also has kept me excited for my future ventures in the company. In particular, a presentation about the quality norms of the business and its work culture was earned.

Scope for Future Work The future scope for the work that has been carried out is immensely bright. Since the threat is always inevitable. It is the duty of all the white hat ethical hackers, Cyber security, Professionals, and Product security officers to make sure that the chances of an attack are always minimal and low. They are responsible for the protection of organizations, institutions, etc. that should be protected and maintained. The work which is displayed here is just the basics of vulnerability analysis as it was our aspect to showcase the vulnerabilities and to protect our organization from the larger rough world of Cyber Crimes. The escalation in the number of Cyber Crimes has increased dramatically and is at all new heights. There should be proper checks, accountability, and system checks in place when we discuss the future scope of vulnerability assessments. The vulnerability assessments will go on as per schedule until new threats pop up and the project will welcome the utmost changes to provide a more secure and reliable experience to its users.

References

1. Goel JN, Mehtre BM (2015) Vulnerability assessment & penetration testing as a cyber defence technology. Procedia Comput Sci 57:710–715
2. Lodh AA, Dalave CV, A study on types of cyber crimes and cyber attacks today
3. Fan-Osuala O, A note on web vulnerabilities. Muma Case Rev 3:001–007
4. Lyne J (2012) Eight trends that are changing network security. A Sophos article, 04.12 v1. dNA 2
5. Godbole N, Belapure S (2015) Cyber security-understanding cyber crimes. Comput Forensics Leg Perspect :14
6. Carey-Smith MT (2011) Improving information security management in nonprofit organisations. Diss. Queensland University of Technology
7. Corrons L (2013) A look back at cyber-security in 2012. Panda Labs
8. Reddy GN, Reddy GJ (2014) Study of cloud computing in healthcare indus-try. arXiv preprint arXiv:1402.1841
9. Bloomfield Robin, Lala Jay (2013) Safety-critical systems: the next generation. IEEE Secur & Priv 11(4):11–13
10. Cho JH, Xu S, Hurley PM, Mackay M, Benjamin T, Beaumont M (2019) Stram: measuring the trustworthiness of computer-based systems. ACM Comput Surv (CSUR) 51(6):1–47

11. Cho JH, Hurley PM, Xu, S (2016) Metrics and measurement of trustworthy systems. In: MILCOM 2016-2016 IEEE Military Communications Conference. IEEE, pp 1237–1242
12. Knowles W, Baron A, McGarr T (2016) The simulated security assessment ecosystem: does penetration testing need standardisation? Comput & Secur 62:296–316
13. Yaqoob I, Hussain SA, Mamoon S, Naseer N, Akram J, ur Rehman A (2017) Penetration testing and vulnerability assessment. J Netw Commun Emerg Technol (JNCET) 7(8). https://www.jncet.org/
14. Enoch SY, Huang Z, Moon CY, Lee D, Ahn MK, Kim DS (2020) HARMer: cyber-attacks automation and evaluation. IEEE Access 8:129397–129414

Health Insurance Fraud Detection Using Feature Selection and Ensemble Machine Learning Techniques



Anuradha Mohanta and Suvasini Panigrahi

Abstract Frauds existing in the healthcare system have led to additional expenses as well as caused degradation of the quality of services being provided to patients. In this research, we have proposed a Sequential Forward Selection (SFS) method and SMOTE oversampling approach to healthcare insurance fraud detection. We employed K-Nearest Neighbors (KNN), Artificial Neural Network (ANN), Linear Discriminant Analysis (LDA), Gradient Boosting Machine (GBM), Bagging classifier, and stacking meta-estimator for the classification purpose. We preferred Stacking aggregator over others in fusion with SFS as there is a statistically significant difference between them. This work categorizes, compares, and summarizes almost all published technical and review articles in healthcare fraud detection over the past 10 years. It defines the professional fraudster, formalizes the main types and subtypes of known fraud, and presents the nature of data evidence collected within affected industries. Within the business context of mining the data to achieve higher cost savings, this research presents methods and techniques together with their problems. The accuracy is best obtained when feature selection is applied to the Stacking classifier which is 97.19%. To demonstrate the effectiveness of the proposed methodology, we have experimentally evaluated our proposition using a real-world healthcare insurance fraud dataset taken from the literature.

Keywords SMOTE · Sequential forward selection · K-nearest neighbor · Artificial neural network · Adaboost · Linear discriminant analysis · Gradient boosting machine · Bagging · Stacking

A. Mohanta (✉) · S. Panigrahi
Department of CSE, Veer Surendra Sai University of Technology, Burla, India
e-mail: anuradhamohanta5@gmail.com

S. Panigrahi
e-mail: spanigrahi_cse@vssut.ac.in

© The Author(s), under exclusive license to Springer Nature Singapore Pte Ltd. 2023

S. Chinara et al. (eds.), *Advances in Distributed Computing and Machine Learning*,
Lecture Notes in Networks and Systems 660,
https://doi.org/10.1007/978-981-99-1203-2_17

1 Introduction

Fraud can be defined as anything wrong or having a criminal deception which leads to financial loss or personal gain to an entity. According to the National Health Care Anti-Fraud Association, health care fraud is an intentional craftiness made by a person, or an entity that could result in some unauthorized benefits to him or her accomplices [1]. Fraud causes difficulties in our day-to-day activities and also at times incurs great losses to insurance companies. Healthcare has emerged as one of the important aspects of human lives, but at the same time, it involves high expenditure which abstains the patients from taking up the services. Many government-funded healthcare insurance policies rent out money to the needy and reimburse the amount at a later period.

Due to healthcare abuse and fraud, a country's economy faces a huge financial loss. It is estimated that 10% of medical system expenditure is wasted due to fraud, making it an alarming concern. A report suggests that the health care industry in India is losing upto ₹600–₹800 crores on fraudulent claims annually [2]. A safe and reliable environment should be built for efficient fraud detection and for this purpose, the very step is the identification of the fraud and then applying suitable approaches to detect and eradicate the bug from the system should be established.

The healthcare fraud detection can be either anomaly-based or misuse-based detection system where is the first approach, the model is trained with normal claims profiles and any deviation from the normal behavior is labeled as intrusive. On the contrary, in the misuse-based detection scenario, a misuse signature is built and any incident claim when matched with the misuse pattern is flagged as a fraud. Data mining is an efficient tool which helps in exploring new variants of the useful information from a huge raw data. It helps in decision making. Computerized statistical tools are used with integration with data mining approach. The two distinct analysis tools that can be enumerated are- supervised and unsupervised learning. The supervised learning works on labeled datasets whereas the unsupervised approach is applied to unlabeled datasets. Another improvisation of machine learning is the application of ensemble methodologies to several base learners to develop a final efficient predictor. Instead of repeatedly executing the base models to improve the performance ensemble methods can be used to boost up the performance.

Recent times have witnessed health insurance fraud as a serious threat. Health insurance companies lose an enormous money as a part of these fraudulent activities. Even service providers and at times patients also manipulate the policies and carry out fraud to gain unauthorized financial benefits. Hence, healthcare insurance detection has become crucial. Such scenarios at times cause harassment to legitimate clients and does not allow the company to deliver the required sum of money to the needy patients at the right time. Hence, lowering the effectiveness and working of the health insurance companies.

The main goal of the research is to devise an efficient model that could detect health insurance fraud, that is it should be able to detect fraudulent claims accurately, attempt to increase the performance metrics and minimizing the misclassification rate. It would basically help to improvise the existing health insurance company's system to detect anomalies accurately and in less time. According to the current worldwide economy, an efficient insurance company is one that fulfills its customer's needs and demands and safeguards its patient's documents without any ambiguity. Most insurance companies use obsolete techniques to find abusive claims and this makes way for frauds to creep into the system. The primary task of a health insurance company is to identify the fraud claims and restore the losses incurred by the customer. It is obligatory to devise a system to trace the customer's records which would reduce fraud happening in the system and deliver quality health care services.

1.1 Categories of Health Insurance Frauds

The frauds which are encountered in the healthcare industries can be under three categories. Firstly, the one which is carried out by service providers which includes billing services which have not been done, unbundling i.e., billing more costly services which are not required, upcoding which to bill costly services which were not part of the treatment and performing unnecessary medical services which are not needed. In the second place, come the frauds perpetuated by insurance subscribers that is the customers by deceiving the eligibility or employment criteria in order to gain premiums at lower rate, proclaiming medical claims which are actually not received and at time identity theft is also done. The third category is insurance carrier fraud where the insurance companies forge reimbursements, rewards and service statements.

1.2 Our Contributions

The significant contribution of the paper illustrates:

1. Investigation of existing data mining and machine learning methodologies of Artificial Neural Network (ANN), Linear Discriminant Analysis (LDA), Gradient Boosting Machine (GBM), Adaptive Boosting (Adaboost), K-Nearest Neighbor (KNN), Bagging Classifier and Stacking Classifier;
2. Designing and developing the efficient feature subset through Sequential Feature Selection (SFS) technique and then applying the learners and ensembles
3. And then comparing the results of all the classifiers and finding out the best fit for our approach and the dataset.

The remaining work in this paper is organized as follows; Sect. 2 has the literature review of various research papers on the health insurance fraud detection domain,

Sect. 3 discusses the methodologies used, Sect. 4 enumerates the experimental analysis and Sect. 5 says about the conclusion and future work.

2 Related Work

In the literature, various methodologies have been proposed to detect fraud in the health insurance domain. Richard A. Bauder and Taghi M. Khoshgoftaar propped a novel work where they drew a comparison between supervised, unsupervised and hybrid learners on 2015 Medicare Provider Utilization Payment Dataset. A total of 10 techniques were used and results out that the hybrid model having the combination of unsupervised autoencoder and supervised neural network shows better accuracy [3]. An approach enumerated by Rawte and Srinivas [4] drew the advantages of both techniques of supervised and unsupervised techniques and incorporated them into a hybrid approach where Evolving Clustering Method (ECM) is used to cluster dynamic data and a Support Vector Machine is used for training and classification. A work by Ayushi Verma, Anu Taneja and Anuja Arora was designed by integrating Association rule mining, clustering and outlier detection where the claims were divided into period-based claims and disease-based claims and it was successfully finding out claims from the existing data [5]. D. Thornton and et al. created a multi-dimensional data model for classifying the frauds based on the seven-level proposed by Sparrow [6]. The seven levels proposed by Sparrow encompasses several frauds like upcoding, unbundling, phantom billing, duplicate billing, bill padding, excessive or unnecessary services and kickbacks. The work done by Wan-Shiou Yang, San-Yih Hwang is a process mining framework where clinical pathways is used and a sensitivity of 64% and specificity of 67% [7].

Pallavi Pandey et al. proposed a detection and analyses approach where logistic regression, decision tree and neural network where the neural network accuracy is 99% [8]. A big data fraud detection using supervised learning approaches where logistic regression was found to be better by given by Matthew Harland et al. [9]. Roshan et al. proposed the Markov model and improvised Markov model with gradient boosting which improved Markov model by 97.10% when applied on health insurance [10]. The novel work proposed by Namrata Ghuse, Pranli Pawar and Amol Potgantwar [11] deep learning algorithms are applied in the form of neural networks as part of unsupervised learning vs the supervised learning of logistics regression and random forest. The accuracy of random forest outperformed. Ananthi Sheshasaayee and Surya Susan Thomas used health care data to which supervised learners of naïve bayes, linear regression, decision trees, random forest, logistic regression and support vector machine [12]. Melih Kirilidog and Cuneyt Asuk [13] proposed an anomaly detection technique which applied to a data obtained from a Turkish insurance company. Lu et al. have made use of heterogenous graph and graph neural network to devise fraud detection model called MHAMFD which focuses on patients' regular doctor visits [14]. A detailed taxonomy has been discussed by K. Kapadiya

et al. where a survey is presented of all health insurance fraud detection mechanisms that are carried out using AI and blockchain enabled systems [15].

3 Proposed Methodology

The dataset [16] is obtained by merging three datasets and the merged dataset had a class imbalance problem which is dealt by using SMOTE oversampling technique to sample the minority fraudulent cases. Then, machine learning models are applied to the dataset without including any feature selection algorithm. Next, the classifiers are operated in a feature selection algorithm and the performances are compared. Feature selection aims at reducing the dimensionality of the dataset by identifying the input variables which are required to establish a strong relationship with the target output variables which helps in improving the performance of the model, improves data compatibility in the model and computational statistics are reduced.

3.1 About the Dataset

The dataset basically focuses on the details of the inpatient, outpatient and beneficiary details of each healthcare provider. It is an amalgamation of three datasets namely, Outpatient, Inpatient and Beneficiary datasets.

3.2 Data Preprocessing

The data quality is although very good, but it is seen that some improvements are required to make it robust and efficient.

The values of beneficiary dataset especially in the ChroniCond_. So, 0 and 1 values are imputed where 0 depicts no chronic conditions and 1 for yes i.e., the patient has chronic conditions. All numeric values are imputed with zero. Further, for improving the efficiency ‘WhetherDead’, ‘AdmitForDays’ and ‘Age’ attributes. After merging, the final dataset has 57 attributes and the shape of the test data is 135392 and that of training dataset is 558211.

3.3 Algorithms and Models used for Model Building

3.3.1 Sequential Feature Selection

Sequential Feature Selection is a wrapper method that enables us to add or remove features to form a feature subset based on feature importance in a greedy fashion. At every stage, the estimator either adds or removes the best feature based on the cross-validation score of the estimator. The technique contains two basic components; an objective criterion which is used to minimize the overall features of the dataset to enhance the performance metric. And a sequential search algorithm which either increases or decreases the candidate feature. It can be of two types; Sequential Forward Selection (SFS) where the features are sequentially added to a null set till the additional feature does not reduce the objective criterion and the second variant is Sequential Backward Selection (SBS) where all the input data is merged and united into a set and goes on removing the feature till the extraction does not hamper the criterion.

3.3.2 Machine Learning Techniques Used for Classification

The supervised learning paradigms are used to distinguish fraud and non-fraud scenarios. The base learners like K-Nearest Neighbor (KNN), Linear Discriminant Analysis (LDA) and Artificial Neural Network (ANN) is used. KNN is a straightforward approach which works on the principle of finding out the k nearest neighbors by using a distance parameter. LDA is a rectilinear model where the number of features in the feature subset are maximized in order to gain a better classification. ANN are basically computational methods that are inspired by the human nervous system and brain which is composed of neurons.

Ensemble techniques of Adaboost, Gradient boost, Bagging and Stacking has been used. Boosting is a type of ensemble learning algorithm which aims to increase the efficiency of weak learners. Adaptive boosting, or AdaBoost classifier commences by adjusting a classifier on the original dataset and then fits the extra copies of the classifier on the same datasets where the incorrect weights are adjusted. In the case of Gradient Boosting Machine, the base learner is typically decision tree but at times regression can also be used. The construction of base learners is achieved in a way that they are maximally correlated to a negative gradient of loss function. Bootstrapping aggregator, or a bagging classifier is an ensemble meta-estimator that fits into a base learning classifier randomly of the original dataset and then used voting or averaging to get the final prediction. Stacking classifier where predictions are made using multiple classifiers namely, level-one classifier and a meta-estimator. The baseline models predict the output of test datasets and the second layer has a meta-classifier or regressor which uses up the predictions of the baseline models.

In the proposed model, a modified dataset is built by removing the unbalanced factor from the original dataset which contained 10% of the minority class which disabled the learners to perform well. An oversampling technique called SMOTE (Synthetic Minority Over-Sampling Technique) is applied to generate samples of the minority cases. The modified dataset is given to the Sequential Forward Selection technique to select features based on importance and go on adding the features until further addition does not affect the objective criterion. It is then randomly sampled to obtain training and testing datasets. The training dataset goes through the machine learning techniques of KNN, LDA, ANN, AdaBoost, GBM, Bagging and Stacking. The testing dataset is evaluated to determine the intrusive cases. Lastly, the performance metrics of the classifiers are obtained to get the best learning technique suitable for our dataset for fraud detection.

The proposed model is implemented on a Windows 10 Education 64-bit operating system with an Intel core i-3-6006U processor and 8gb RAM on Python 3.7 environment.

3.4 Performance Metrics

The learners which are implemented in this paper are compared based upon some performance metrics to uplift the qualities are discussed below;

Accuracy

It is defined as the fraction of all the true outcomes including both true positives and true negatives to the sum of all cases under examination.

$$Accuracy = TP + TN/(TP + TN + FP + FN) \tag{1}$$

Sensitivity

The proportion of correctly predicted positive observations to the observations. It is also known as recall or true positive rate.

$$Sensitivity = TP / (TP + FN) \tag{2}$$

Specificity

True negative rate defined as the proportion of false positive to all the predicted observation.

$$Specificity = FP / (TN + FP) \tag{3}$$

MCC

Mathew's Correlation Coefficient is basically a balanced performance measuring parameter and is also known as phi-coefficient (ϕ) which is the estimate of the association between two classes in machine learning.

$$MCC = (TP * TN - FP * TN)/\sqrt{((TP + FP)(TP + FN)(TN + FP)(TN + FN))} \tag{4}$$

G-mean

G-mean is defined as the squared mean of sensitivity and specificity.

$$G - mean = \sqrt{Sensitivity * Specificity} \tag{5}$$

4 Results and Observations

A comparison is drawn out by evaluating the performance metrics of each classifier by first taking all the features and then by applying SFS. The results are tabulated below (Tables 1 and 2, Fig. 1).

Table 1 Comparison table of K-nearest neighbor, Linear Discriminant Analysis (LDA) and Artificial Neural Network (ANN)

Classifier	Performance metric	Using all features	Using SFS
K-Nearest Neighbor	Accuracy	0.9193	0.9379
	Sensitivity	0.9391	0.9525
	Specificity	0.6082	0.7152
	MCC	0.4453	0.5641
	G-mean	0.7558	0.8254
LDA	Accuracy	0.9261	0.9433
	Sensitivity	0.9412	0.9572
	Specificity	0.6739	0.7988
	MCC	0.4882	0.6203
	G-mean	0.7964	0.8723
ANN	Accuracy	0.9316	0.9366
	Sensitivity	0.9410	0.9508
	Specificity	0.7531	0.7322
	MCC	0.5188	0.5777
	G-mean	0.8418	0.9290

Table 2 Comparison table of Gradient Boosting Machine (GBM), AdaBoost, bagging and stacking classifier

Classifiers	Performance metric	Using all features	Using SFS
GBM	Accuracy	0.9162	0.9283
	Sensitivity	0.9658	0.9638
	Specificity	0.5421	0.6089
	MCC	0.5605	0.5905
	G-mean	0.7236	0.7660
Adaboost	Accuracy	0.9360	0.9440
	Sensitivity	0.9415	0.9535
	Specificity	0.8353	0.7994
	MCC	0.5771	0.6240
	G-mean	0.8868	0.8731
Bagging	Accuracy	0.9421	0.9525
	Sensitivity	0.9509	0.9630
	Specificity	0.8021	0.8175
	MCC	0.6097	0.6946
	G-mean	0.8733	0.8873
Stacking	Accuracy	0.9601	0.9719
	Sensitivity	0.9379	0.9813
	Specificity	0.8482	0.8333
	MCC	0.7302	0.7768
	G-mean	0.8919	0.9043

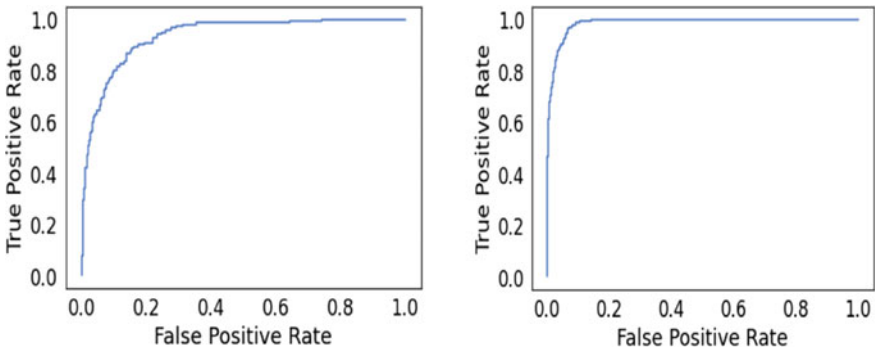


Fig. 1 ROC curve of stacking meta-estimator using all features (left) and using sequential feature selection-SFS (right)

Table 3 Performance evaluation between existing work and proposed approach

	Performance metrics				
	Accuracy	Sensitivity	Specificity	MCC	G-Mean
CatBoost [17]	0.9304	0.9426	0.7191	0.5171	0.8233
Proposed SFS based on stacking classifier	0.9719	0.9813	0.8333	0.7768	0.9043

In the literature, drawing out a comparative analysis between our proposed work and [17], we can deduce the following results (Table 3).

5 Conclusion and Future Scope

The experimental analysis of the mentioned method demonstrates that when feature selection technique is not applied then Stacking meta-estimator outperforms the other classifiers in all the metrics. When Sequential Feature Selection (SFS) technique is applied to the classifiers, then also Stacking meta-estimator shows better performance than the other illustrated classifiers. So, to generalize Stacking generalization classifier when applied to a balanced and feature selected dataset, it shows better performance as compared to other classifiers and takes no iterative loops and less computational time.

The research work has potential future scope if we incorporate optimization technique Differential Evolution and Particle Swarm Optimization to further enhance the performance metrics by applying them to select important features of the dataset and also by reducing the search space. As a part of future prospects, the proposed approach can be applied to various other insurance datasets like automobile, financial, bank, etc. to establish a versatile model.

References

1. Guidelines to health care fraud (1991) Guidelines to health care fraud. National Health Care Anti-Fraud Association (NHCAA), NHCCA Board of Governors. http://www.nhcaa.org
2. Dr. Bardhan B Frauds in health insurance. http://healthcare.financialexpress.com/200711/market13.shtml
3. Bauder RA, Khoshgoftaar TM (2017) Medicare fraud detection using machine learning methods. In: 2017 16th IEEE international conference on machine learning and applications (ICMLA). IEEE, pp 858–865
4. Rawte V, Anuradha G (2015) Fraud detection in health insurance using data mining techniques. In: 2015 international conference on communication, information & computing technology (ICCICT). IEEE, pp 1–5
5. Verma A, Taneja A, Arora A (2017)Fraud detection and frequent pattern matching in insurance claims using data mining techniques. In: 2017 tenth international conference on contemporary computing (IC3). IEEE, pp 1–7

6. Thornton D, Mueller RM, Schoutsen P, Van Hillegersberg J (2013) Predicting healthcare fraud in medicaid: a multidimensional data model and analysis techniques for fraud detection. Procedia Technol 9:1252–1264;Verma A, Taneja A, Arora A (2017) Fraud detection and frequent pattern matching in insurance claims using data mining techniques. In: 2017 tenth international conference on contemporary computing (IC3). IEEE, pp 1–7
7. Yang W-S, Hwang S-Y (2006) A process-mining framework for the detection of healthcare fraud and abuse. Expert Syst Appl 31(1):56–68
8. Pandey P, Saroliya A, Kumar R (2018) Analyses and detection of health insurance fraud using data mining and predictive modeling techniques. In: Soft computing: theories and applications. Springer, Singapore, pp 41–49
9. Herland M, Khoshgoftaar TM, Bauder RA (2018) Big data fraud detection using multiple Medicare data sources. J Big Data 5(1):1–21
10. Gupta RY, Mudigonda SS, Baruah PK, Kandala PK (2021)Markov model with machine learning integration for fraud detection in health insurance. arXiv:2102.10978
11. Ghuse N, Pawar P, Potgantwar A (2017) An improved approach for fraud detection in health insurance using data mining techniques. Int J Sci Res Netw Secur Commun 5(5):28–32
12. Sheshasaayee A, Thomas SS (2018) A purview of the impact of supervised learning methodologies on health insurance fraud detection. In: Information systems design and intelligent applications. Springer, Singapore, pp 978–984
13. Kirilidog M, Asuk C (2012) A fraud detection approach with data mining in health insurance. Procedia Soc Behav Sci 62:989–994
14. Lu J, Lin K, Chen R, Lin M, Chen X, Lu P (2022). Health insurance fraud detection by using an attributed heterogeneous information network with a hierarchical attention mechanism
15. Kapadiya K, Patel U, Gupta R, Alshehri MD, Tanwar S, Sharma G, Bokoro PN (2022) Blockchain and AI-empowered healthcare insurance fraud detection: an analysis, architecture, and future prospects. IEEE Access 10:79606–79627
16. https://www.kaggle.com/rohitrox/healthcare-provider-fraud-detection-analysis
17. Hancock J, Khoshgoftaar TM (2020)Medicare fraud detection using catboost. In: 2020 IEEE 21st international conference on information reuse and integration for data science (IRI). IEEE, pp 97–103

Real-Time American Sign Language Interpretation Using Deep Convolutional Neural Networks



Arghya Biswasa, Gaurav Sa, Umakanta Nanda, Diksha Sharma, Lakhan Dev Sharma, and Primatar Kuswiradyo

Abstract In spite of being the 4th most commonly used language in the United States, sign language is actively used by only 10–14% of the members of the speech and hearing impairment community and continues to be one of the most understudied areas. In compliance with recent advances in deep learning, this paper explores the possibility of using deep convolutional neural network to interpret American Sign Language in real-time. The paper aims to develop a CNN from scratch and train it using a dataset patterned to closely match the format of the classic MNIST dataset (28×28 pixel images with pixel values ranging from 0 to 255). The evaluation of the network shows that it outperforms all previous implementations surrounding this task with 89.6% accuracy during real-time user testing (99.7% accuracy on validation set). Such high accuracy measure and fast converging time can be attributed to modelling the training data as a dataframe of pixel values rather than using traditional images and including batch normalization, concepts which have not been employed in earlier implementations to the best of our knowledge. To make the solution as accessible as possible, we refrained from using sophisticated hardware like motion-tracking gloves and depth-sensing cameras and deployed the trained model as a multi-platform mobile application.

Keywords American sign language · Hearing impairment · Deep learning · Deep convolutional neural networks · Flutter development

A. Biswasa · G. Sa · U. Nanda (✉) · L. D. Sharma
VIT-AP University, Amaravati, Near Vijayawada, Andhra Pradesh, India
e-mail: uk_nanda@yahoo.co.in

D. Sharma
Department of Nanoscience and Technoloagy, Central University of Jharkhad, Ranchi, India

P. Kuswiradyo
Department of Electrical Engineering, Brawijaya University, Malang, Indonesia

© The Author(s), under exclusive license to Springer Nature Singapore Pte Ltd. 2023

S. Chinara et al. (eds.), *Advances in Distributed Computing and Machine Learning*,
Lecture Notes in Networks and Systems 660,
https://doi.org/10.1007/978-981-99-1203-2_18

1 Introduction

According to the National Institute on Deafness and Other Communication Disorders (NIDCD), approximately 7.5 million people in the United States have speech and hearing impairment [1]. However, only 0.75 to 1 million people actively use sign language as a mean of communication [2]. This can be attributed to the fact that 98% of the people from the deaf-and-dumb community do not receive any education pertaining to the use of sign language in daily life and face intense social and cultural distaste. In order to bridge this communication gap, break the social stigma towards sign language and enable dynamic communication, we present a real-time American Sign Language (ASL) interpretation tool that uses a deep convolutional neural network (DCNN) to translate ASL finger-signs into text.

The goals of this research paper are:

- Interpret ASL finger-signs in real-time using a deep CNN
- Deploy the model in a multi-platform mobile application
- Make the mobile application as accessible and user-friendly as possible (no additional hardware or internet required).

While neural networks have been employed for ASL finger-signs interpretation in the past that consistently guarantee accuracies over 80% [3–9, 15, 16], many of them either require complex hardware like 3D image capturing with motion-tracking gloves or a Microsoft Kinect depth-sensing camera [15, 16], or, don't provide real-time interpretation, or, have not been deployed such that it can be used by general masses [3–8]. The restrictions imposed by these excess requirements hinder the feasibility and practicality of these solutions.

Our solution presents a pipeline that enables the user to point his/her mobile camera at a person communicating in ASL. The mobile application then simultaneously extracts individual frames of the video and feeds it to the CNN model to predict the letter represented by the corresponding video frame (excluding j and z as they require motion of fingers). The results are appended to a string and a language model is used in order to output the most likely word to the user. We were able to achieve 99.7% accuracy on the validation set and 89.6% accuracy during real-time user testing with our application which was developed using the Flutter framework.

2 Related Works

ASL recognition and interpretation is not an obscure computer vision problem. Over the better part of the recent years, researchers came up with various algorithms to address this issue. We can group the implementations of the past studies broadly into linear classifiers and deep neural networks.

Linear classifiers are more user-friendly due to their simplicity, but they require complex preprocessing methods and feature extraction to be successful. Using

Karhunen–Loeve Transforms, an accuracy of 96% is found by [3] on 10 classes of gestures of a single hand. They translated and rotated the axes of the images to determine new coordinates depending on the data variance. However, they used a linear classifier to discern hand gestures including thumbs-up, index finger pointing in a direction and numbers, but not ASL signs. Sharma et al. used piece-wise classifiers (Support Vector Machines and k-Nearest Neighbours) to identify each colour channel after background removal and noise elimination [4]. Their novelty came from using a contour trace which produced an efficient depiction of the hand gesture. Using an SVM on the segmented colour channel model, they attained an accuracy of 62.3%.

Some neural network models have been used to tackle ASL interpretation in the past too. The ability of deep learning methodologies to learn the most important classification features is arguably the most crucial advantage. However, training them requires substantially more amount of data and time. Till date, most of the neural networks implemented are comparatively shallow. Videos of ASL gestures are classified into text using advanced feature extraction and a 3-layer neural network [5]. Two categories of features were extracted by them which are movement and hand position. In [6], Ethiopian Sign Language letters were categorized with an accuracy of 98.5% employing a feed-forward neural network. The most relevant work to date is the application of CNN's to classify 20 Italian gestures from the ChaLearn 2014 [7]. They used a Microsoft Kinect camera on full-body images of people performing the gestures and achieve a cross-validation accuracy of 91.7%. As in the case with the 3D gloves, the Kinect enables them to capture depth features in space, which aids substantially in interpreting the ASL signs.

Bheda and Radpour very recently used a 9-layer deep CNN to interpret and classify ASL letters and numbers without the assistance of any sophisticated hardware [8]. Though they attained a decent accuracy of 82.5% on the validation set using the NZ ASL dataset, their model struggled to produce a good result on self-generated data (62% on the validation set).

It is worth noting that none of the above models was deployed to be used by general masses, an issue addressed by Garcia and Viesca from Stanford University [9]. They used transfer learning from the GoogLeNet architecture pre-trained on the 2012 ILSVRC dataset and attained an accuracy of 72% on the validation set. They developed a web-app to deploy their model.

3 Approach and Method

The fundamental hurdle in working with video is that each video frame is characterized into a 2-D matrix, instead of a single 1-D vector. Each pixel is then a separate feature and hence the neural network will not emphasis on finding patterns with respect to its neighbouring pixels in the image. Hence to overcome this problem, we have utilized a DCNN architecture. This architecture is highly efficient to handle 2-D input data.

3.1 Network Architecture

Training of the network here is preferred to be done from scratch [9]. The few instances where researchers have trained their network from scratch, the usual architecture has been the fairly common CNN architecture of groups of convolution, pooling and dropout layers followed by a final classification layer [8]. We decided to go down the same road, but with an important modification.

The kernel which is convolved with an image, holds the pattern which is required to be extracted from the image. This process brings us a different image as the output. This extracted image contains higher (white) and lower (black)-value pixels in areas of the original image. White corresponds to the existence of the pattern and black represents the non-existence of the pattern.

The convolution of an input image x with a convolution kernel k as computed by Jarret et al. is:

$$(x * k)_{ij} = \sum_{p,q=0}^{r-1} (x_{i+p,j+q} k_{r-p,r-q})$$

where x is the input image in the current layer and a feature map in the subsequent layers [10], k is a square matrix. In our architecture, 75 filters were used, each with a 3 × 3 kernel size and a stride of 1 which led to 75 planes each of dimension 28 × 28. In the later stages, we kept diminishing the number of filters to 50 and then 25 with the same kernel size and stride. We used ReLU as an activation function for all the filters (Fig. 1).

After each convolution layer, we have included a batch normalization layer which has not been done before by any researcher in implementations surrounding this task to the best of our knowledge (Fig. 1). Batch normalization layer automatically normalizes the feature maps generated by the preceding convolution layer so that it resembles a normal distribution (zero mean and a unitary variance). For our implementation, this simple addition dramatically accelerated the training process of the network (bringing it down to only ~2 s per epoch), and also improved the performance of the model via a modest regularization effect.

Following the batch normalization layers, a max-pooling operation was performed. We adopted the standard 2 × 2 max-pooling window (Fig. 1). Max-pooling was performed so that only the high-intensity pixels get propagated forward while reducing the size of the feature maps. In this way, we kept the important information of extracted patterns while removing all redundant pixels in the feature maps.

We used a dropout (randomly setting the outgoing edges of hidden units to 0) layer with 50% probability to prevent a model from overfitting (Fig. 1). It is done at each update of the training phase which acts as a regularization method that mimics training a large number of neural networks parallelly with different architectures.

The final part of the architecture aimed to perform the classification using a fully connected network. The 25-dimensional feature vectors with a matrix of size 4 × 4

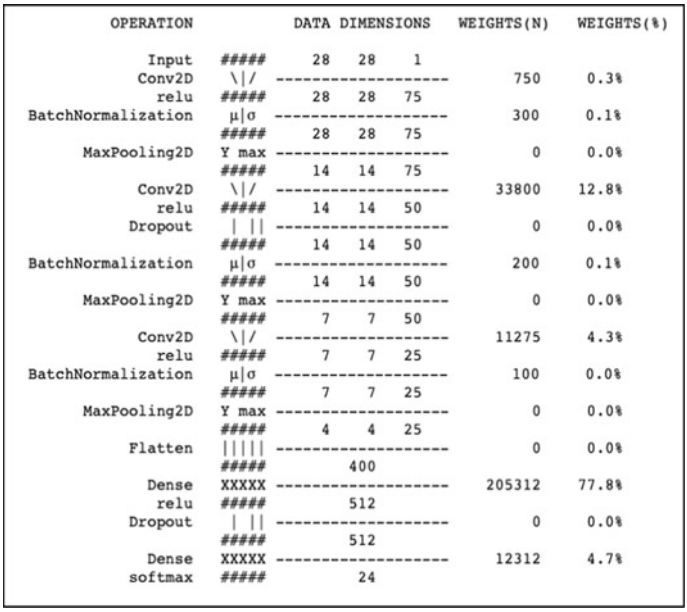


Fig. 1 The neural network architecture

were reshaped to a single 400-dimensional vector and used as input to a two-layer network with 512 nodes in the hidden, followed by a 0.5 probability dropout layer and then finally 24 class nodes, representing each English letter (Fig. 1).

3.2 Optimization and Loss Function

The activation function used in the final layer of our network was softmax. This function guaranteed that all the output nodes had values ranging from 0 to 1 and the sum of all the output node values was always equal to 1. The formula for softmax [11] is as follows:

$$Softmax(y_i) = \frac{e^{y_i}}{\sum_{i=0}^{n} e^{y_i}}$$

The goal of softmax is to ensure that one value is extremely high (that is, close to 1) and all other values are extremely low (that is, close to 0). Softmax employs exponential function to make this happen. The loss function that we used for our network is categorical cross entropy. Once we had the outputs of the final layer utilizing the softmax activation function, we applied log loss on each output node

with respect to its respective target value and then we found the sum of this across all output nodes.

For optimization function, Adam was an obvious choice. First, we calculated our m value, which corresponds to the momentum at the current point [12].

$$m_{new} = \beta_1 * m_{old} - (1 - \beta_1) * \frac{\partial(Loss)}{\partial(W_{old})}$$

Next, we computed the accumulated cache:

$$cache_{new} = \beta_2 * cache_{old} + (1 - \beta_2) * \left(\frac{\partial(Loss)}{\partial(W_{old})}\right)^2$$

The final update formula resulted into:

$$W_{new} = W_{old} - \frac{\alpha}{\sqrt{cache_{new}} + \varepsilon} * m_{new}$$

We were accumulating the gradients by calculating momentum and also constantly changed the learning rate by using the cache. Due to these two features, Adam usually performs better than any other optimizer out there and was preferred for our implementation. In the research paper for Adam optimizer, the recommended parameters are 0.9 for β_1, 0.99 for β_2 and 1e-08 for ε [13].

4 Data

One of the most important choices that we made while developing this project was to go with a dataset that most would consider somewhat "unconventional". Most of the previous works related to interpreting ASL using deep learning techniques fed images of finger-characters to train and evaluate the network. However, for our network, we decided to use a dataset patterned to closely match the format of the classic MNIST. Each image of an ASL finger-character was 28 pixels in height and width and a total of 784 pixels. Each pixel was represented as a single pixel value between 0 and 255 associated with it, indicating the intensity of that pixel, with higher numbers meaning darker. Each training and test example consisted of a label (0–25) representing a one-to-one mapping of each alphabetic characters A-Z (no example for J = 9 or Z = 25 because of motion gestures).

Fortunately for us, we found a public dataset in Kaggle compiled by Kaggle user 'tecperson' that served our requirements and we decided to use the same for our model [14]. The training data (27,455 cases) and test data (7172 cases) are approximately half the size of the standard MNIST but otherwise similar with 785 columns of data (the first column for label signifying the corresponding ASL letter and the rest of

the columns containing the pixel values of the associated image) which represent a single 28 × 28 pixel image with grayscale values between 0 and 255.

To generate new data, an image pipeline was employed based on Image Magick and involved cropping to hands-only, grey-scaling, resizing and then producing at least 50 + variations to enlarge the quantity of data. The modification and expansion strategy employed was filtering ('Mitchell', 'Hermite', 'Robidoux', 'Spline', 'Catrom'), accompanied with 5% random pixelation, 15% brightness/contrast adjustments and finally 3° random rotation [14].

4.1 Data Preprocessing

The first and fundamental thing we did was to check the count of images of each class in the dataset. In several supervised learning tasks, the classes are not balanced and thus in the case of anomaly detection, there is a large difference between the positive and negative classes. However, our dataset seemed to be well balanced for each training class, with at least 900 training examples existing for each class (Fig. 2).

The label for each tuple in the dataset consisted of a single integer value between 1 and 24 corresponding to the ASL letter represented by that tuple. The final output layer in our network contained 24 nodes as a multiclass classifier. So, we used the Label Binarizer package from sklearn. preprocessing to encode each integer in a binary array of size 24 with the corresponding label being 1 and all other labels are 0.

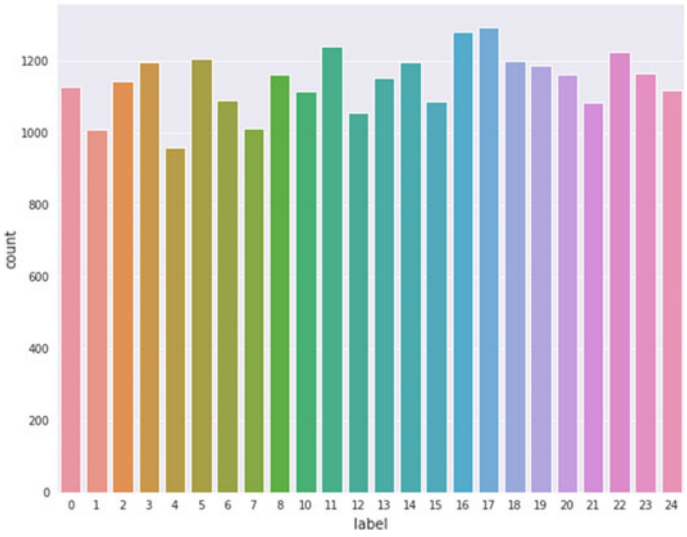


Fig. 2 Class wise training data distribution

As mentioned before, each pixel value was an integer ranging from 0 to 255. This range is too high and makes the network converge slower. So, the best approach was to normalize the data. In our case, as the pixel value was in the known range it was sufficient to conduct min–max normalization, i.e., to scale the pixel values in the range of [0, 1] by simply dividing the array by 255.

4.2 Data Augmentation

Deep neural networks are usually very data-hungry affairs and lack mechanisms to learn abstractions from a limited amount of data. In order to avoid overfitting, we need to have a large amount of data in our training dataset. However, obtaining huge amounts of quality data is often easier said than done and resource-intensive. Data augmentation is a strategy that enables researchers to considerably improve the diversity of the available data for training models by applying small transformations, without actually collecting new data. For our implementation, we opted to randomly rotate an image by 10°, randomly zoom by 10%, randomly shift images horizontally by 10% of the total width and randomly shift images vertically by 10% of the total height.

5 Evaluation and Model Analysis

We evaluated two metrics in order to compare our model performance with respect to those of other research papers. The most popular criterion is validation set accuracy, that is, the percentage of correctly classified examples. We also generated a confusion matrix to visualize the performance of the classification model and help us identify which characters are the most misclassified and draw insights for further development (Fig. 2). For additional insights, we also calculated the precision, recall and f1-score for each class (Fig. 3).

We observed a 99.7% accuracy on the validation set when using the ASL MNIST dataset (Fig. 4). In terms of time complexity, the model converged in approximately 4 min for 20epochs (~8 s per epoch). Such high accuracy measure and fast completion time can be attributed to the important choices we made throughout the development process. Modelling the data to be fed to the network in MNIST format rather than using traditional images and including batch normalization in the network greatly boosted the performance of our model. We observed a lower accuracy measure of 89.6% while conducting real-time user testing which we accounted largely to the skin tone, lighting and image quality variation from the training dataset images. However, compared to previous implementations by fellow researchers, our model performed extremely well considering that Dong, Leu and Yin were using both colour gloves and depth-sensing Kinect camera [15, 16]. Our network performed much better than

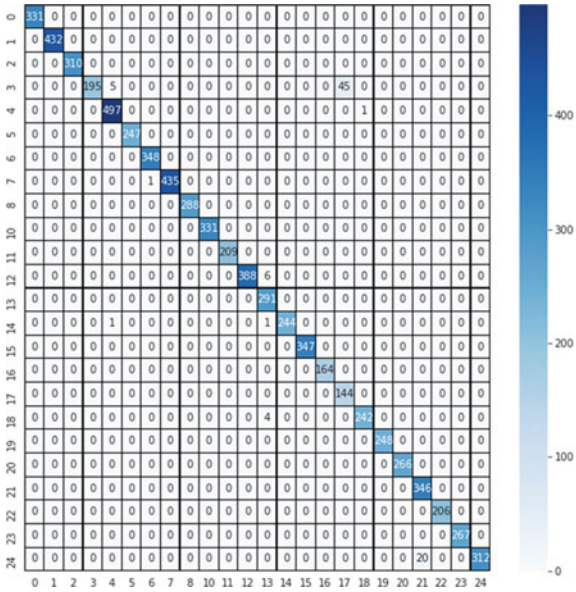


Fig. 3 Confusion matrix for our model

the models designed by researchers at Stanford University and the State University of New York [8, 9] (Table 1).

6 Mobile Application

Most studies surrounding this task in the past have just provided us with a classification model, but no real implementation that can be utilized by general masses [3–8]. Researchers at Stanford were the only group to address this issue and deploy their classification model through a web-app [9]. However, the web-app relies on the user to manually start and stop the interpretation window. Besides, carrying around laptops for the sake of interpretation is unrealistic and infeasible. Considering this in mind, we deployed our model through a Flutter application (Fig. 5). This enables our application to have multi-platform support (both iOS and Android) and also enables us to seamlessly integrate our model as a tflite file since our model was built using the Tensor Flow framework (Fig. 6).

Fig. 4 Class wise evaluation of our model through precision, recall and f1-score

	precision	recall	f1-score	support
Class 0	1.00	1.00	1.00	331
Class 1	1.00	1.00	1.00	432
Class 2	1.00	1.00	1.00	310
Class 3	1.00	0.80	0.89	245
Class 4	0.99	1.00	0.99	498
Class 5	1.00	1.00	1.00	247
Class 6	1.00	1.00	1.00	348
Class 7	1.00	1.00	1.00	436
Class 8	1.00	1.00	1.00	288
Class 10	1.00	1.00	1.00	331
Class 11	1.00	1.00	1.00	209
Class 12	1.00	0.98	0.99	394
Class 13	0.96	1.00	0.98	291
Class 14	1.00	0.99	1.00	246
Class 15	1.00	1.00	1.00	347
Class 16	1.00	1.00	1.00	164
Class 17	0.76	1.00	0.86	144
Class 18	1.00	0.98	0.99	246
Class 19	1.00	1.00	1.00	248
Class 20	1.00	1.00	1.00	266
Class 21	0.95	1.00	0.97	346
Class 22	1.00	1.00	1.00	206
Class 23	1.00	1.00	1.00	267
Class 24	1.00	0.94	0.97	332
accuracy			0.99	7172
macro avg	0.99	0.99	0.99	7172
weighted avg	0.99	0.99	0.99	7172

Table 1 Model accuracy comparison of recent research works

Model	Accuracy (real-world performance) (%)
Our model	89.6
Dong, et al. (colour gloves + Kinect cameras) [15]	90
The State University of NY (deep CNN) [8]	82.5
Stanford University (GoogLeNet trained on ILSVRC) [9]	72

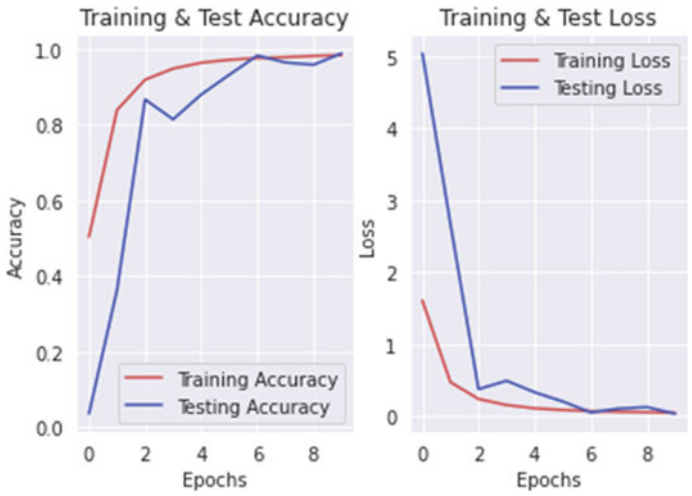


Fig. 5 Training and validation set accuracies and losses

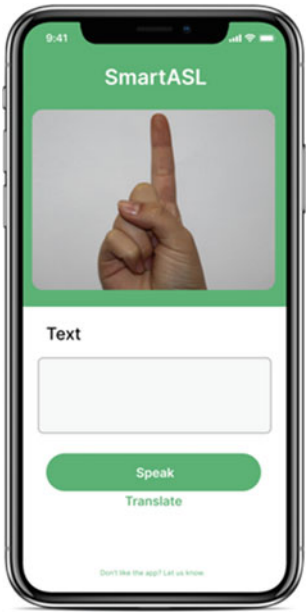


Fig. 6 Screenshot of our mobile application

7 Conclusion

The primary purpose of this paper was to provide the general masses with an accessible mobile application that interprets ASL in real-time with no additional hardware required. Considering the performance of the mobile application in real-time user

testing scenarios, it would be safe to say that we have achieved our goals. However, this does not imply that there is no room for betterment. With colour gloves and Microsoft Kinect camera having depth sensors, our model would definitely perform better than all existing models. However, such sophisticated hardware and technology are not widely accessible and often too expensive. Thus, we refrained from utilizing such hardware for our implementation. Our method proves to have the potential to solve the global challenge of bridging the communication gap using only a smartphone. With more and more people gaining access to smartphones, our model would only get better with time as more user-data is generated and contribute to a scalable solution.

References

1. Statistics on Voice, Speech, and Language, NIDCD (2020). https://www.nidcd.nih.gov/health/statistics/statistics-voice-speech-and-language
2. Mitchell R (2004) How many people use ASL? and other questions without good answers
3. Singha J, Das K (2013) Hand gesture recognition based on Karhunen-Loeve transform. arXiv:1306.2599
4. Sharma R et al (2013) Recognition of single handed sign language gestures using contour tracing descriptor. In: Proceedings of the world congress on engineering, vol 2
5. Mekala P et al (2011) Real-time sign language recognition based on neural network architecture. In: 2011 IEEE 43rd Southeastern symposium on system theory. IEEE
6. Admasu Y, Raimond K (2010) Ethiopian sign language recognition using artificial neural network. In: 2010 10th international conference on intelligent systems design and applications. IEEE
7. Pigou L et al (2014) Sign language recognition using convolutional neural networks. In: European conference on computer vision 6–12 Sept 2014
8. Bheda V, Radpour N (2017) Using deep convolutional networks for gesture recognition in American sign language. arXiv:1710.06836
9. Garcia B, Viesca S (2016) Real-time American sign language recognition with convolutional neural networks. Convolutional Neural Netw Vis Recognit 2:225–232
10. Jarrett K et al (2009) What is the best multi-stage architecture for object recognition? In: 2009 IEEE 12th international conference on computer vision. IEEE
11. Bommana H (2020) Loss functions explained. Deeplearningdemystified.com. https://www.deeplearningdemystified.com/article/fdl-3. Accessed 28 June 2020
12. Bommana H (2020) Understanding optimizers. Deeplearningdemystified.com. https://www.deeplearningdemystified.com/article/fdl-4. Accessed 28 June 2020
13. Kingma D, Ba J (2014) Adam: a method for stochastic optimization. arXiv:1412.6980
14. Kaggle.com (2017). Sign Language MNIST. https://www.kaggle.com/datamunge/sign-language-mnist?select=sign_mnist_test. Accessed 28 June 2020
15. Dong C, Leu M, Yin Z (2015) American sign language alphabet recognition using microsoft kinect. In: Proceedings of the IEEE conference on computer vision and pattern recognition workshops
16. Agarwal A, Thakur MK (2013) Sign language recognition using Microsoft Kinect. In: 2013 sixth international conference on contemporary computing (IC3), Noida, pp 181–185. https://doi.org/10.1109/IC3.2013.661218

Multi-branch Multi-scale Attention Network for Facial Expression Recognition (FER) in-the-Wild



Chakrapani Ghadai and Dipti Patra

Abstract The challenges in facial expression recognition (FER) are mostly caused by high intra-class variations, subtle inter-class visual changes, and smaller datasets. The intra-class and inter-class variations suffer big from the pose, illumination, or partial occlusion in the real world, which degrade the performance of FER significantly. Multi-scale and attention-based networks are widely used to address these challenges. In most of the previous approaches, lower level features at smaller scale progress toward higher levels to construct features at larger scales, or convolutions at different resolutions are used for multi-scale feature representations. The used methods have increased depth, but lacked width and are inadequate in representing features at granular levels to precisely capture important facial expression features. Here, we introduce a novel multi-branch multi-scale attention network (MSA-Net) for FER. MSA-net is a deeper and wider network and it extracts multi-scale features at different receptive fields in a parallel network structure. Moreover, to improve the effective receptive field and extract diverse features, different kernel sizes in each parallel branch are used. Further, to focus on important regions and make the features more discriminating multi-scale features are passed through attention networks. MSA-Net can extract sufficiently diverse attention-enhanced multi-scale features from different parallel paths, this can lessen the effect of intra-class and inter-class variations due to external factors. Further, features at different receptive fields from each parallel path are combined together to reduce the effect of pose and partial occlusion. The experimental findings reveal that the suggested method achieves competitive results on widely used in-the-wild public datasets.

Keywords Facial expression recognition · CNN · Multi-scale · Attention · Receptive field · kernel size

C. Ghadai · D. Patra (✉)
National Institute of Technology, Rourkela, India
e-mail: dpatra@nitrkl.ac.in

C. Ghadai
e-mail: 520ee7014@nitrkl.ac.in

© The Author(s), under exclusive license to Springer Nature Singapore Pte Ltd. 2023

S. Chinara et al. (eds.), *Advances in Distributed Computing and Machine Learning*,
Lecture Notes in Networks and Systems 660,
https://doi.org/10.1007/978-981-99-1203-2_19

1 Introduction

The facial expression conveys one's emotional state of mind, and it is one of the most powerful nonverbal communication methods for social interactions. The research in FER is growing due to its broad applications in many domains, e.g., human–computer interaction [25], modern driver aid systems [28], medical diagnosis [1], etc. FER system assigns a category to a still image or a video out of many basic emotions. FER from still images is called static FER and FER from a video is called dynamic FER. Further, FER can be divided into controlled and in-the-wild depending upon the dataset used. Recently, FER on laboratory-controlled datasets has achieved significant results due to all frontal images taken under controlled laboratory conditions. However, FER on in-the-wild datasets is challenging due to large intra-class variations, very small inter-class visual differences, and complexity in the representation of subtle local variations caused by muscle movements of dynamic face regions.

Over the years, researchers followed different approaches to address the above challenges in FER. Traditional methods used Gabor filter [21], histograms of oriented gradients (HOG) [4], scale-invariant feature transform (SIFT) [23], etc. for FER. These methods use shallow learning for feature extraction, are subject to underfit, and the performance of these methods are not encouraging. Recently, much work used deep learning especially convolutional neural network (CNN) [19, 22] in FER which significantly improved performance over traditional shallow learning methods. The methods used deeper CNNs, which have better generalization capability, but can be easily overfitted in presence of external factors, e.g., pose, illumination, occlusion, etc. To focus on emotion-related regions many methods used attention networks in CNN backbone [15] for FER in-the-wild. Further, some methods used DenseNet, a variant of inception [11], and pyramid [24] as multi-scale feature extractors, and multi-scale features serve as context to attention networks for learning more discriminating features. However, many of the existing methods either use convolutions at different resolutions in parallel paths or use a single path for multi-scale feature extraction. In single path multi-scale networks, lower level features at smaller receptive fields progress toward higher levels to construct features at a larger receptive field. Both the single path-based and multi-resolution-based methods lack diversity and a larger effective receptive field in feature learning. To overcome some of the above challenges a novel multi-branch multi-scale attention network (MSA-Net) for FER in-the-wild is proposed.

The main contributions are summarized as follows:

1. Multi-branch multi-scale attention network is constructed to learn features at different receptive fields and the feature maps of each branch are fused together, which can reduce the effect of pose and partial occlusion.

2. The multi-scale block used extracts multi-scale features by combining features at a different receptive fields in a hierarchical fashion within a single multi-scale block, which can lessen the FER system's vulnerability to inter-class and intra-class variations due to external factors.

3. The attention module is used in the deep layer of each branch of MSA-Net to focus on important regions and make the expression-related features more discriminating.

The remaining part of this paper is structured as follows. Section 2 provides an overview of the proposed MSA-Net. Section 3 discusses the experimental details, results, and discussion. In Sect. 4, we finally give concluding remarks.

2 Proposed Method

2.1 A Framework Overview

Figure 1 shows the overall framework of the proposed MSA-Net. MSA-Net consists of feature pre-extractor, parallel multi-scale, and attention network blocks. The feature pre-extractor extracts middle-level feature maps of size 128 x 28 x 28 through the use of one 2D convolution layer, a max pooling layer, and two layers of ResNet-18 [12]. Then, middle-level feature maps are passed through three parallel branch networks; each of the parallel branches consists of a multi-scale module and an attention module. Inside the multi-scale module, two multi-scale blocks are connected in series. 7 x 7, 5 x 5, and 3 x 3 convolutions are used in multi-scale blocks of the first, second, and third branches, respectively, for finding multi-scale features at different receptive fields. Finally, the feature maps from three branches are fused at the feature level to obtain the FER results.

2.2 Multi-scale Module

Two multi-scale blocks connected in series in each parallel path constitute a multi-scale module. The designed multi-scale block is inspired by Res2Net [9]. Each multi-

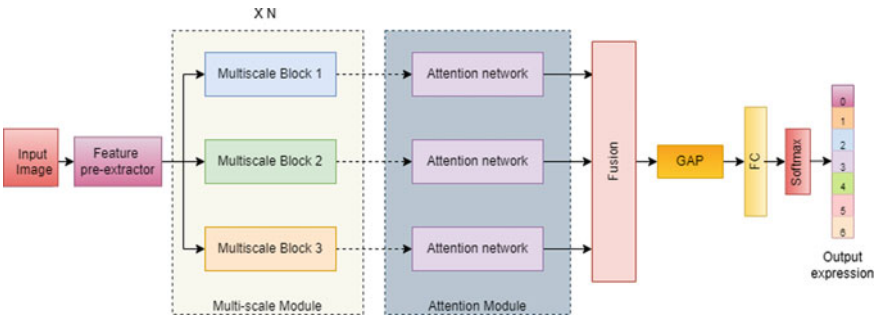


Fig. 1 The block diagram of MSA-net showing different components

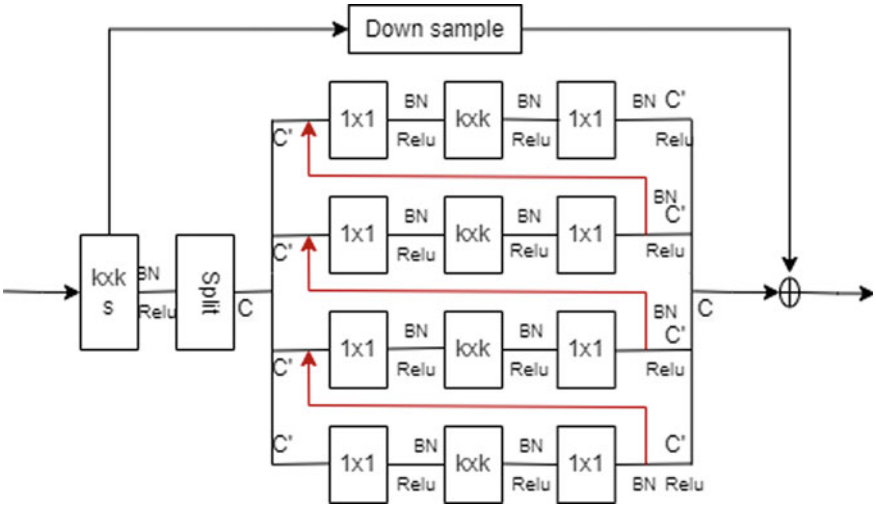


Fig. 2 Multi-scale block, k=7, 5, 3 for multi-scale block 1, 2, 3, respectively

scale block uses bottleneck blocks in a hierarchical-like manner within a single residual block to learn features at different receptive fields. Moreover, convolution with higher kernel sizes is biased toward shape, and convolution with smaller kernel sizes is biased toward texture [6]. So, multi-scale blocks in each branch use different kernel sizes to make the multi-scale feature maps diverse. Figure 2 shows the basic multi-scale block, 1 x 1 convolution layers are used for downsampling and upsampling, and the number of input and output channels remain the same in the bottleneck convolution layer. This arrangement reduces the computation overhead due to the larger kernel size. Let X be the feature maps obtained after k $\times$ k convolution. The feature map X is split into n equal feature map subsets along the channel axis. Let X_i be the feature map subsets, where $i \in \{1, 2, ..., n\}$. Hence, the feature map subsets $X_i \ \forall \ i$ have the same spatial size, and the number of channels of is $\frac{1}{n}$th of the total number of channels of X. Moreover, the feature map subsets $X_i \ \forall \ i$ are processed in three parallel multi-scale blocks. Let the m $\times$ m convolution process applied on each of X_i is denoted by $P_i^p(.)$,where $p \in \{1, 2, 3\}$, p denotes the position of the multi-scale block. Let Y_i^p denote the output of $P_i^p(.)$. Therefore, the output Y_i^p for each feature map subset X_i can be written as

$$Y_i^p = \begin{cases} P_i^p(X_i), & i = 1 \\ P_i^p(X_i) + Y_(i-1)^p, & 1 < i \leq n \end{cases} ; 1 < p \leq 3 \quad (1)$$

Let's consider the first multi-scale block, i.e., $p = 1$, notice that each 7 x 7 convolutional operator $P_i^p(.)$ receives feature information from all feature subsets $X_j : j < i$. Each time a feature subset X_i passes through a 7 x 7 convolutional operator, the out-

put feature map attains a larger receptive field than X_j. Moreover, the second and third multi-scale blocks have 5 x 5 and 3 x 3 convolutional operators, respectively. Hence, the first multi-scale block has a higher effective receptive field than the second multi-scale block, and the second multi-scale block has a higher effective receptive field than the third multi-scale block.

2.3 Attention Module

Let the feature maps of each parallel path obtained after the multi-scale module be denoted as $F^p \in \mathbb{R}^{H \times W \times C}$, where $p \in \{1, 2, 3\}$. Then, each of the feature maps is passed through an attention network. A convolutional block attention module (CBAM) [29] is used as the attention network. In CBAM, attention feature maps are extracted sequentially along the channel and spatial dimensions, then the refined adaptive feature maps are obtained by multiplying attention maps with the input feature maps. For each parallel path, the attention network computes channel attention map $M_c^p \in \mathbb{R}^{1 \times 1 \times C}$ and spatial attention map $M_s^p \in \mathbb{R}^{H \times W \times C}$. Therefore, the final refined output of attention networks from each path can be formulated as $F_r^p = M_s^p(F) \circledast (M_c^p(F) \circledast F)$ here $\circledast$ denotes element-wise multiplication. The channel and spatial attention maps are scaled along spatial and channel directions, respectively, for conducting multiplication. Finally, the attention feature maps from the parallel branches are fused and let it be denoted as F_r.

2.4 Fusion Strategy and Loss Function

The feature maps from the parallel branches are fused using simple concatenation operations along the channel axis. The global average pooling (GAP) operation is performed on the final fused attention feature maps to obtain a vector, v. The end-to-end model is trained by the cross-entropy loss function as given below:

$$L = -\frac{\beta}{N} \sum_{0}^{N-1} \log \frac{e^{w_i^T v_i + b_i}}{\sum_0^{C-1} e^{w_j^T v_j + b_j}}, \tag{2}$$

where N is the input batch size; C is the number of expression categories; W and b are the weight and bias terms of the FC layer; v_i is the input to the FC layer at i^{th} sample; and β is the hyperparameter of the loss function.

3 Results and Discussion

3.1 Datasets and Implementation Details

Two popular FER in-the-wild datasets, RAF-DB [16] and Affectnet [20], are used for conducting the experiments. RAF-DB contains 29672 images annotated with basic or compound, 12264 as training samples, and 3061 as testing samples from basic images taken. In the Affectnet dataset, 283,901 images as training data and 2992 images as test data are selected. We conducted an experiment for seven basic expression categories. In both datasets, the officially aligned data samples are used directly, and then they are resized to 224 x 224 pixels. Simple data augmentation techniques are employed to extract the random crops (central, corner, and horizontal flips). We trained the model in NVIDIA Geforce RTX 3060 GPU using an SGD optimizer with a momentum of 0.9, batch size of 32, and initial learning rate of 0.1 decayed by a factor of 10 for every 10 epochs. Hyper-parameter β is empirically set as 0.6. Further, we used four numbers of workers in multi-process data loading to speed up the training process.

3.2 Comparison of Results

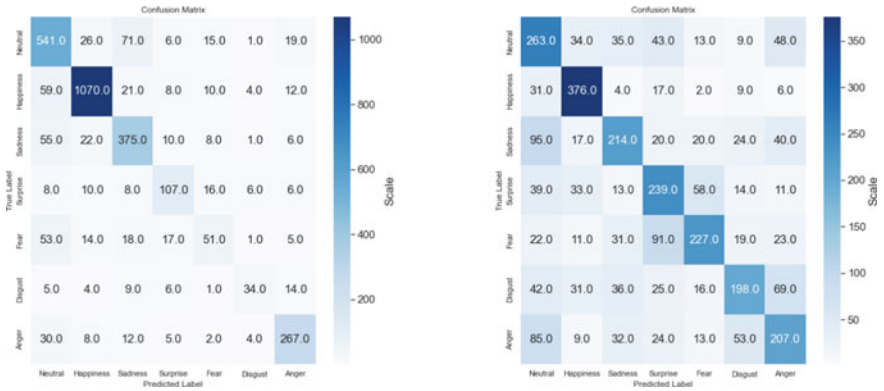
We present the experiment results in Tables 1 and 2 for RAF-DB and Affectnet datasets, respectively. The proposed MSA-net method outperforms some of the state-of-the-art methods with a recognition accuracy of 86.32 and average accuracy of 79.88 on the RAF-DB dataset. Similarly, it achieves a recognition accuracy of 63.61 and average accuracy of 57.63 on the Affectnet dataset. Figure 3 presents the confusion matrices of the RAF-DB and AffectNet.

Table 1 Comparison of results on RAF-DB dataset

Method	Backbone	Year	Acc. (%)
RLPS [14]	10-layer DCNN	2020	72.89
Sadeghi & Raie [21]	–	2019	76.23
SCN [26]	ResNet-18	2020	78.31
pACNN [18]	Densenet	2018	83.27
ALT [8]	–	2019	84.50
gACNN [17]	VGG-16	2019	85.07
Proposed method	Manually designed	2022	86.32

Table 2 Comparison of results on Affectnet dataset

Method	Backbone	Year	Acc. (%)
pACNN [18]	Densenet	2018	55.33
gACNN [17]	VGG-16	2019	58.78
LDL-ALSG [2]	ResNet-50	2020	59.35
VGG-FACE [13]	VGG-16	2020	60.00
FMPN [3]	Inception-V3	2019	61.52
OADN [5]	Manually designed	2020	61.89
DDA-Loss [7]	DCNN	2020	62.34
LLHF [10]	VGG	2018	63.31
Proposed method	Manually designed	2022	63.61



(a) RAF-DB (b) AffectNet

Fig. 3 Confusion matrices

3.3 Ablation Analysis

Ablation analysis on RAF-DB, Affectnet, and Pose-AffectNet [27] datasets are performed to evaluate the effectiveness of each component used in the proposed method. Table 3 shows the analysis of the effectiveness of the multi-scale module and attention module in one branch network structure, Resnet-18 is considered as one branch baseline, and it's also observed that MSA-net gives the best result with three parallel branch structures.

Table 3 Evaluation of each component on RAF-DB, Affectnet, and Pose-Affectnet

Methods	RAF-DB	Affectnet	Pose-AffectNet≥ 30°	Pose-AffectNet ≥ 45°
Baseline	82.59	59.4	53.55	52.39
MS	84.33	61.23	54.85	53.66
AS	85.36	62.11	55.77	54.81

Multi-scale Structure (MS)

The multi-scale structure in one branch network of MSA-Net without attention module is studied. It is observed from Table 3 that MS improves the baseline by 2.1%, 3.08%, 2.61%, 2.43% on RAF-DB, AffectNet Pose-AffectNet (30°), and Pose-AffectNet (45°), respectively.

Attention Structure (AS)

The attention module is added to a multi-scale structure in one branch network. It is clear that the attention network embedded with a multi-scale module significantly improves accuracy by 1.22%, 1.46%, 1.68%, and 2.14% on RAF-DB, AffectNet Pose-AffectNet (30°), and Pose-AffectNet (45°), respectively. Hence, the multi-scale module extracts diverse multi-scale features and the attention network further makes features more discriminating.

4 Conclusion

A novel multi-branch multi-scale attention network (MSA-Net) for in-the wild facial expression recognition is proposed to address the issues of both intra-class and inter-class variations due to external factors like occlusion and pose. The multi-scale module learns features at different receptive fields, which can reduce the effect of pose and partial occlusions in the inference process. The attention module can focus on the important part, neglect other parts, and extract features at granular levels which can further reduce the susceptibility of the network toward subtle expression-related variations. The proposed method MSA-Net has achieved competitive accuracy with 86.32% and 63.61% on RAF-DB and AffectNet, respectively. MSA-Net presents a coarse feature learning model focusing mainly on large intra-class and inter-class variations due to pose and partial occlusions. In future work, we will investigate subtle intra-class visual differences based on feature diversification and adaptive learning to improve the performance of the model.

References

1. Canal FZ, Müller TR, Matias JC, Scotton GG, de Sa Junior AR, Pozzebon E, Sobieranski AC (2022) A survey on facial emotion recognition techniques: a state-of-the-art literature review. Inf Sci 582:593–617
2. Chen S, Wang J, Chen Y, Shi Z, Geng X, Rui Y (2020) Label distribution learning on auxiliary label space graphs for facial expression recognition. In: Proceedings of the IEEE/CVF conference on computer vision and pattern recognition. pp 13984–13993
3. Chen Y, Wang J, Chen S, Shi Z, Cai J (2019) Facial motion prior networks for facial expression recognition. In: 2019 IEEE visual communications and image processing (VCIP). IEEE, pp 1–4
4. Dahmane M, Meunier J (2011) Emotion recognition using dynamic grid-based hog features. In: 2011 IEEE international conference on automatic face & gesture Recognition (FG). IEEE, pp 884–888
5. Ding H, Zhou P, Chellappa R (2020) Occlusion-adaptive deep network for robust facial expression recognition. In: 2020 IEEE international joint conference on biometrics (IJCB). IEEE, pp 1–9
6. Ding X, Zhang X, Han J, Ding G (2022) Scaling up your kernels to 31x31: revisiting large kernel design in cnns. In: Proceedings of the IEEE/CVF conference on computer vision and pattern recognition. pp 11963–11975
7. Farzaneh AH, Qi X (2020) Discriminant distribution-agnostic loss for facial expression recognition in the wild. In: Proceedings of the IEEE/CVF conference on computer vision and pattern recognition workshops. pp 406–407
8. Florea C, Florea L, Badea MS, Vertan C, Racoviteanu A (2019) Annealed label transfer for face expression recognition. In: BMVC. p 104
9. Gao SH, Cheng MM, Zhao K, Zhang XY, Yang MH, Torr P (2019) Res2net: a new multi-scale backbone architecture. IEEE Trans Pattern Anal Mach Intell 43(2):652–662
10. Georgescu MI, Ionescu RT, Popescu M (2019) Local learning with deep and handcrafted features for facial expression recognition. IEEE Access 7:64827–64836
11. Hardjadinata H, Oetama RS, Prasetiawan I (2021) Facial expression recognition using xception and densenet architecture. In: 2021 6th international conference on new media studies (CONMEDIA). IEEE, pp 60–65
12. He K, Zhang X, Ren S, Sun J (2016) Deep residual learning for image recognition. In: Proceedings of the IEEE conference on computer vision and pattern recognition. pp 770–778
13. Kollias D, Cheng S, Ververas E, Kotsia I, Zafeiriou S (2020) Deep neural network augmentation: generating faces for affect analysis. Int J Comput Vis 128(5):1455–1484
14. Li H, Xu H (2020) Deep reinforcement learning for robust emotional classification in facial expression recognition. Knowl Based Syst 204:106172
15. Li J, Jin K, Zhou D, Kubota N, Ju Z (2020) Attention mechanism-based cnn for facial expression recognition. Neurocomputing 411:340–350
16. Li S, Deng W, Du J (2017) Reliable crowdsourcing and deep locality-preserving learning for expression recognition in the wild. In: Proceedings of the IEEE conference on computer vision and pattern recognition. pp 2852–2861
17. Li Y, Zeng J, Shan S, Chen X (2018) Occlusion aware facial expression recognition using cnn with attention mechanism. IEEE Trans Image Process 28(5):2439–2450
18. Li Y, Zeng J, Shan S, Chen X (2018) Patch-gated cnn for occlusion-aware facial expression recognition. In: 2018 24th international conference on pattern recognition (ICPR). IEEE, pp 2209–2214
19. Mohan K, Seal A, Krejcar O, Yazidi A (2020) Facial expression recognition using local gravitational force descriptor-based deep convolution neural networks. IEEE Trans Instrum Meas 70:1–12
20. Mollahosseini A, Hasani B, Mahoor MH (2017) Affectnet: a database for facial expression, valence, and arousal computing in the wild. IEEE Trans Affect Comput 10(1):18–31

21. Sadeghi H, Raie AA (2019) Human vision inspired feature extraction for facial expression recognition. Multimed Tools Appl 78(21):30335–30353
22. Sadeghi H, Raie AA (2022) Histnet: histogram-based convolutional neural network with chi-squared deep metric learning for facial expression recognition. Inf Sci 608:472–488
23. Soyel H, Demirel H (2010) Facial expression recognition based on discriminative scale invariant feature transform. Electron Lett 46(5):343–345
24. Vo TH, Lee GS, Yang HJ, Kim SH (2020) Pyramid with super resolution for in-the-wild facial expression recognition. IEEE Access 8:131988–132001
25. Wang HH, Gu JW (2018) The applications of facial expression recognition in human-computer interaction. In: 2018 IEEE international conference on advanced manufacturing (ICAM). IEEE, pp 288–291
26. Wang K, Peng X, Yang J, Lu S, Qiao Y (2020) Suppressing uncertainties for large-scale facial expression recognition. In: Proceedings of the IEEE/CVF conference on computer vision and pattern recognition. pp 6897–6906
27. Wang K, Peng X, Yang J, Meng D, Qiao Y (2020) Region attention networks for pose and occlusion robust facial expression recognition. IEEE Trans Image Process 29:4057–4069
28. Wilhelm T (2019) Towards facial expression analysis in a driver assistance system. In: 2019 14th IEEE international conference on automatic face & gesture recognition (FG 2019). IEEE, pp 1–4
29. Woo S, Park J, Lee JY, Kweon IS (2018) Cbam: Convolutional block attention module. In: Proceedings of the European conference on computer vision (ECCV). pp 3–19

Identifying COVID-19 Pandemic Stages Using Machine Learning



Shomoita Jahid Mitin, Muhammad Jafar Sadeq, Umme Habiba, Roy D. Gregori Ayon, Md. Sanaullah Rabbi, and S. Rayhan Kabir

Abstract Although the severity of COVID-19 has decreased in most of the countries, it remains a threat that the world continues to tackle. Consequently, this paper presents a classification of pandemic stages and a method to determine the current severity of the pandemic using machine learning. Parameters for determining pandemic severity include economy, education, and physical and mental health data of Bangladesh. The analysis has been done using K-Means Clustering algorithm, Decision Tree, Naive Bayes, Random Forest and Decision Tree boosted by AdaBoost. The best performance is achieved by the latter. This technique can be applied to future pandemic data to develop pandemic comparisons and also to develop more robust models that can be applied to pandemics caused by other diseases.

Keywords COVID-19 · Pandemic · Machine learning

S. J. Mitin · M. J. Sadeq · U. Habiba · R. D. G. Ayon · Md. S. Rabbi · S. R. Kabir (✉)
Asian University of Bangladesh, Ashulia, Dhaka, Bangladesh
e-mail: rayhan923@aub.edu.bd

S. J. Mitin
e-mail: shomoitajahid@aub.edu.bd

M. J. Sadeq
e-mail: jafar@aub.edu.bd

U. Habiba
e-mail: ummehabiba1331241@aub.edu.bd

R. D. G. Ayon
e-mail: ayon19@aub.edu.bd

Md. S. Rabbi
e-mail: srrabbi01@aub.edu.bd

© The Author(s), under exclusive license to Springer Nature Singapore Pte Ltd. 2023

S. Chinara et al. (eds.), *Advances in Distributed Computing and Machine Learning*,
Lecture Notes in Networks and Systems 660,
https://doi.org/10.1007/978-981-99-1203-2_20

1 Introduction

COVID-19 was declared a global pandemic on 11 March 2020 by World Health Organization (WHO). The SARS-CoV-2 virus that causes Covid-19 spreads from person to person through droplets released into the air when an infected person breathes, talks, laughs, sings, coughs or sneezes, making it a highly contagious disease that is even now being controlled by tests, quarantines and unprecedented application of health habits such as wearing of masks, social distancing, washing hands and the use of disinfectants.

Middle income countries have been severely affected by COVID-19. For instance, Bangladesh has suffered due to overpopulation, large number of refugees and lack of equipment, skilled human resources, facilities, funds and public awareness [7].

The first COVID-19 case was detected in Bangladesh on March 8, 2020. Since then, a total of 1.97 million infected cases have been officially reported with 29,145 deaths. Additionally, the pandemic has caused a great amount of harm in people's lives including socio-economic aspects, mental health, employment, and education [2, 4–6, 10, 11, 14, 16]. The country's economy has also suffered through a slowdown of economic activities including foreign trade [1].

Due to vaccination efforts, the severity of the pandemic in Bangladesh is decreasing. This paper identifies three stages of the pandemic in Bangladesh, and, using machine learning, identifies which stage of the pandemic is being faced based on personal and national factors in four sectors: education, economy, physical and mental health. Five machine learning techniques were applied: K-Means Clustering, Decision Tree, Naive Bayes Classifier, Random Forest and AdaBoost Decision Tree.

The remainder of this paper is organized as follows. Section 2 presents some previous works related to this topic. Section 3 explains the method and dataset of this research. Section 4 describes the results. Finally, Sect. 5 concludes the paper.

2 Related Work

Machine learning has been used in various ways with regard to data of the COVID-19 pandemic. For instance, univariate logistic regression was used to determine the factors affecting youth mental health during the pandemic [10]. Countries that are similar, and therefore likely to be similarly-affected by the pandemic, were identified using K-means Clustering [9]. Random Forest boosted by AdaBoost has been used to predict the severity of single patients' infections [8]. In contrast, this paper attempts to predict the severity of a country. The same combination of Random Forest and AdaBoost was tweaked to classify pandemic growth rates in various regions of Italy based on various geographical factors and containment policies [15]. Mathematical models have been used to predict the current pandemic risk given recent data [3]. In contrast, the current paper uses a machine learning model to determine the risk (by identifying the pandemic stage) using all previous data on the pandemic, so that

it may be able to learn from previous behavior of the pandemic. Researchers also focused on the COVID-19 impact of Facebook commerce for Bangladesh regions [22] where researchers used a customer-to-customer information quality model [23].

In [7] the authors aimed to examine psychological problems due to COVID-19 and explored the factors affecting the mental health of youth groups. They used a cross-sectional design and univariate logistic regression analysis. In [10] the authors proposed three empirical indexes to estimate the state of the pandemic. These indexes quantify both the propagation and the number of estimated cases, allowing us to accurately determine the real risk of a country. They calculated these indexes' evolution for several European countries. In [3] the authors showed an assessment of the effects of the COVID-19 pandemic on the mental health of college going students. The authors analyzed data using quantitative and qualitative methods. According to their analysis, stress and anxiety increased by 71% due to the COVID-19 outbreak. Rather, multiple stressors were identified that influence the increased levels of stress, anxiety, and depressive thoughts among students. In [15] the authors summarized the influence of COVID-19 on health and proposed alternative nutrients that can help infected people strengthen their immune systems. Nowadays, machine learning is a trending area of the fourth industrial revolution [18] that broadly used in distributed computing, data prediction, computer vision, etc. areas [19, 20] and this trending field has been awaited in IoT, cloud platform, and blockchain [21, 24].

3 Pandemic Stage Identification

3.1 Three Pandemic Stages

Three stages were identified in the pandemic:

1. Embryonic Stages (2020): This is the beginning of the COVID-19 pandemic in Bangladesh. In this year, the first case of COVID-19 was identified on 8 March in 2020 and after that, a lockdown followed with strong containment measures. The crisis had a massive impact in this year.
2. Intervening Stages (2021): In this year, the government allowed all public and private offices, courts, restaurants, shopping malls, recreation centers, religious centers, etc. to carry out regular activities without any supervision. All educational institutes also reopened in the middle of October.
3. Terminal Stages (2022): During this period, a majority of the people were vaccinated and they went back to their normal life. Infected rate is less than before and the government has been relaxing all restrictions in all sectors including economy, education, and physical and mental health.

These three stages may be assigned severity values based on its impact. Therefore, if it is possible to identify the year of the pandemic based on various features, it will also be able to identify the stage of a pandemic in the future if similar values for

features exist. In other words, it may be possible to identify if a pandemic is reentering a severe stage.

3.2 Preparation of Dataset

The dataset consists of some data taken from existing sources [6, 13] and surveys conducted specifically for this purpose from the years 2020–2022. The aggregate data for 2022 (such as number of deaths) was proportionally increased to get an estimate for the year. In total, 23 factors that were affected by the pandemic were identified, covering economy, education, and physical and mental health in Bangladesh. From the 23 features, 15 were selected, as listed below. (Program, Dataset, and GitHub Link: [17]).

1. Age
2. Gender
3. Dropout from educational institution
4. Loneliness
5. Anxiety
6. Education gap
7. Income
8. Test cases
9. Infected cases
10. Recovery cases
11. Death cases
12. Depression
13. Sleep disturbance
14. Expense
15. Remittance.

From these features, there were 1.65% missing values. The missing values were filled with feature average values. The dataset was split so that 75% of the dataset was used as the training dataset and 25% was used as the test dataset. To construct the dataset, we collected information on the Covid-19 epidemic in the country of Bangladesh from the Owid dataset, World Health Organization (WHO), Dhaka Tribune and BRAC's Covid-19 survey (Table 1).

The above features were used to predict the year of the pandemic (2020, 2021, 2022), which was extracted from the date field. The year of the pandemic is used here as an identifier for the pandemic stage.

Table 1 A sample from the dataset

No	Date	New cases	New deaths	Positive rate	Median age	GDP Per capita	Age 65 older	Total mental issue	…
1	2020–03-08	3.0	1.0	2.0	27.5	3523.984	5.098	0.0	…
2	2020–03-09	2.0	0.0	1.0	23.2	3523.984	3.126	0.0	…
3	2020–03-10	6.0	3.0	4.0	26.9	3523.984	7.069	0.01	…
4	2020–03-11	8.0	2.0	5.0	21.7	3523.984	9.143	0.01	…
5	2020–03-12	7.0	1.0	4.0	28.3	3523.984	5.963	0.02	…

3.3 Dataset Extraction

In our work, we extracted a sample of 1,000 data instances along with 15 features which are Test cases, Infected cases, recovery cases, death cases, loneliness, depression, sleep disturbance, expense, Age, Gender, dropout, anxiety, education gap, income, remittance (Fig. 1).

Figure 1 shows the frequency of age effects on the patient confirmation and death cases. Figures 2, 3 and 4 show the frequency of patients' mental health, economy and education on covid-19. Table 1 shows an uncomplicated sample of the dataset that we utilized in this paper. The authors have done a comparison between covid-19

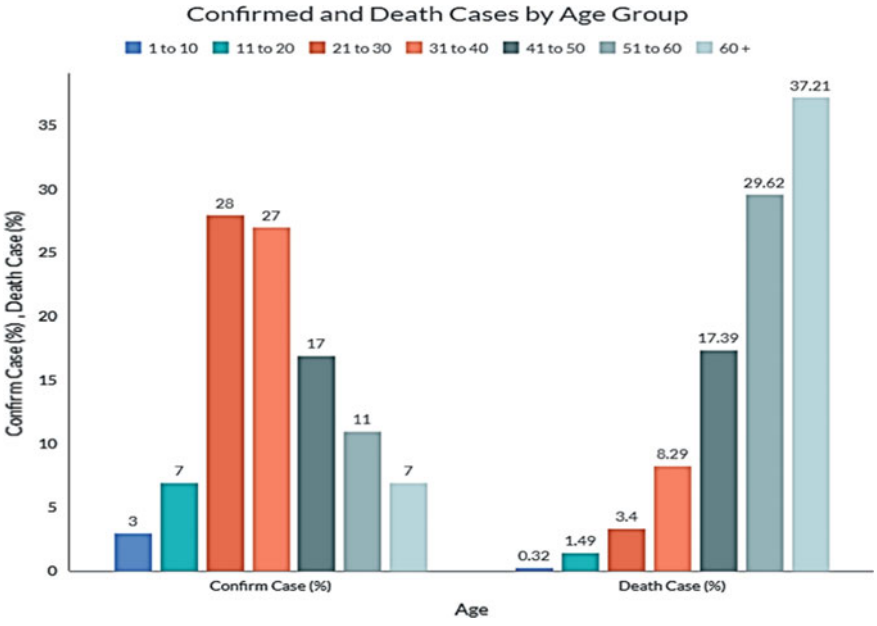


Fig. 1 Covid-19 confirmed and death cases by age group

confirmed and death cases based on age that is shown in Fig. 1. Based on the result, it is emphasized that though 21–30 age groups are infected mostly, the people of 60 plus are dying more which is 37.21%. It's seen that in 2020, the death rate is comparatively higher than the confirmed cases which makes this year more severe than other two years. In Fig. 2, the authors emphasized some common factors that affected mental health [10] of a person during this covid-19 such factors are loneliness, depression, anxiety and sleep disturbance (Figs. 2, 3, 4).

Among those factors, depression has the highest increasing rate and the second highest rate is sleep disturbance. Then we analyzed the impact of covid-19 pandemic based on some common factors in the economy sector. In Fig. 3, we observed that during the pandemic, most of the employees were ejected from their jobs, which is

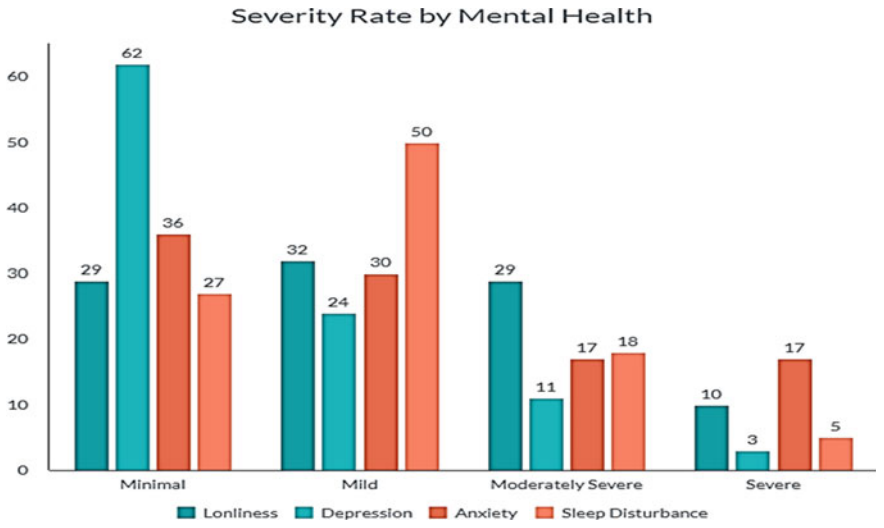


Fig. 2 Impact of covid-19 pandemic on mental health

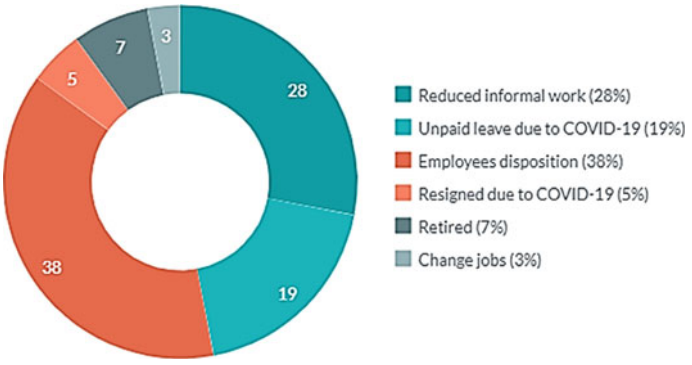


Fig. 3 Impact of covid-19 pandemic economy

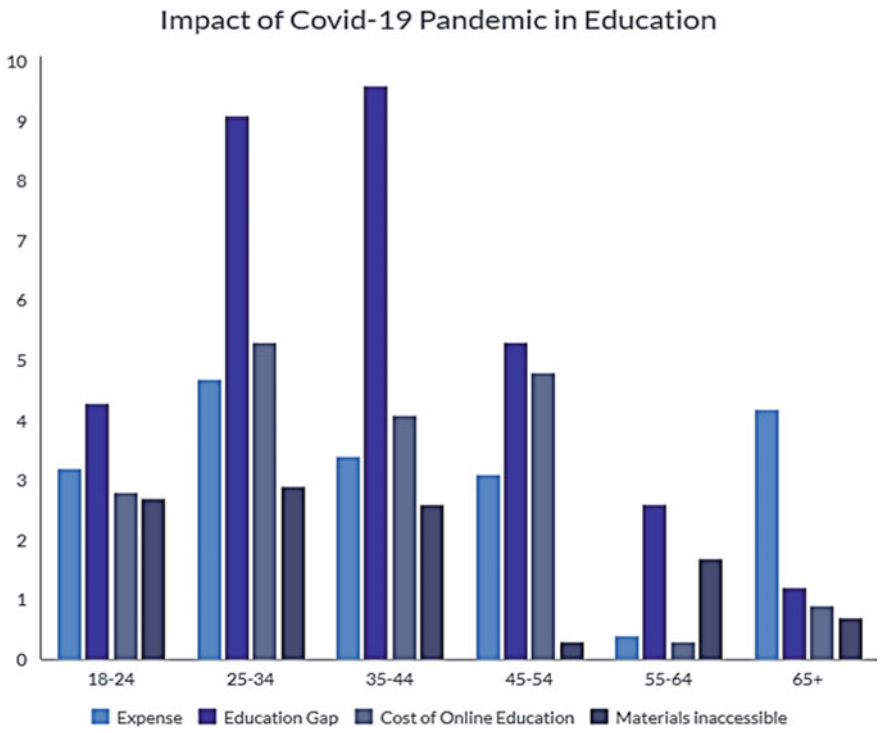


Fig. 4 Impact of covid-19 on Education

38% and 28% reduced informal work during this pandemic [15]. In Fig. 4, we have found the impact of covid-pandemic on the education sector considering expense, education gap, cost of online education and material inaccessible.

3.4 Machine Learning Algorithms

The supervised and unsupervised learning approaches that have been used in this research are K-Means Clustering, Decision Tree, Naïve Bayes, Random Forest and AdaBoost Decision Tree, which are described below.

1. Decision Tree: For classification problems, the decision tree [15], a supervised algorithm, predicts the attribute of the expected variable by learning decisions extrapolated from the data and creating a model from that. Initially, a root node is designated for the forming of this model based on the best attribute collected by the gain approach and the sub-nodes are then initiated on the basis of the decision extracted in relation to the status of quality selected at each node. These courses

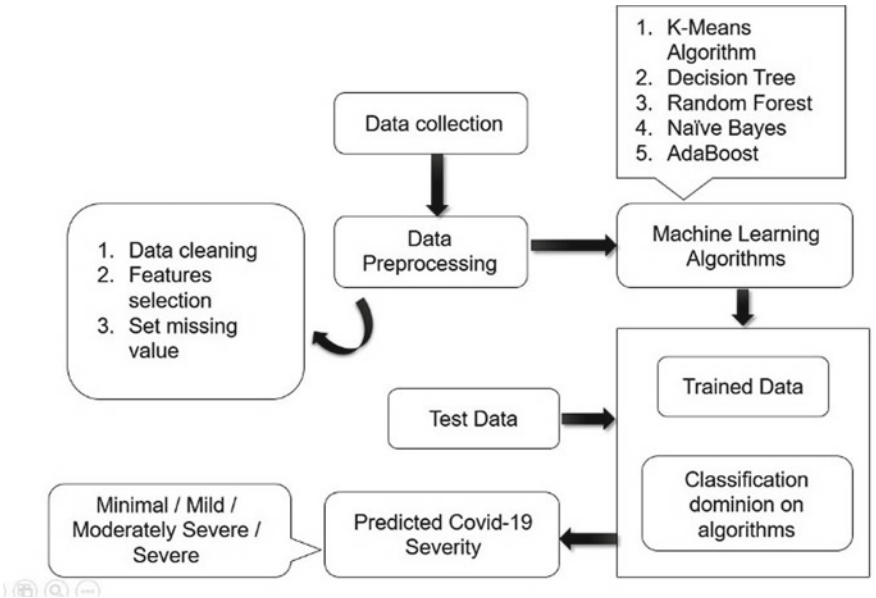


Fig. 5 Overall process of pandemic stage identification

of action proceed iteratively until a class prediction is achieved at the end of each node. This node is called a leaf (Fig. 5).

2. Random Forest: First of all, n groups of random records are taken from the total dataset. Individual decision trees are constructed for each group. Each decision tree will generate a prediction. The final prediction is considered based on Majority Voting or Averaging for Classification and regression respectively [8].
3. K-Means Clustering: K-Means Clustering [9] is an unsupervised learning algorithm to group an unlabeled dataset into different clusters. K number of predefined clusters are created. The algorithm finds the best value for K center points or centroids by an iterative process. Firstly, centroids for these clusters are initiated at random. Each data point is assigned to their closest centroid, forming the K clusters. Then the centroid is recalculated from the cluster, and the process is repeated, until two subsequent iterations result in no change in clusters.
4. AdaBoosted Decision Tree: AdaBoost is a boosting algorithm. Every instance in the training dataset is weighted. At first, the weight is set to,

$$w(x, y) = \frac{1}{N} \tag{1}$$

where w(x, y) is the weight of the *x*th training instance's *y*th feature, and N is the number of training instances. The weighted samples are used to produce an initial classifier. Wherever an incorrect prediction is made, the weights for those

samples are increased. Conversely, weights for correctly predicted samples are reduced [17]. In this paper, AdaBoost was used with decision tree.

5. Naïve Bayes: This is a classification technique based on Bayes theorem. It predicts the conditional probability of different classes based on several aspects and allocates the new class to the maximal probability. Probabilities are calculated as follows:

$$P(H|E) = \frac{P(E|H) * P(H)}{P(E)} \tag{2}$$

where P(H) is the prior probability. P(E) is the evidence of probability, P(E | H) is the probability of the evidence given that the hypothesis is true, P(H | E) is the probability of the hypothesis given that there is evidence.

These algorithms are applied to determine the pandemic stage. The overall process is presented in Fig. 5.

4 Results

After running the five different machine learning techniques, it was found that AdaBoosted Decision Tree achieved the best performance with an accuracy of 82% (Table 2).

We applied ML algorithms on the extracted dataset of COVID-19 patients to find the stages of the symptoms. We used Python programming language to implement ML algorithms which are K-Means Clustering, Decision Tree, Naive Bayes, Random Forest and AdaBoosted Decision Tree to classify the dataset. We utilized precision, recall, f1 score, and accuracy to show the effectiveness and trustworthiness of each technique as it's shown in Table 2. Where precision is the fraction of true positive to the summation of true positive and false positive. Recall is the fraction of true positive to the summation of true positive and false negative; whereas, F1 score is the mean of recall and precision. We obtained sufficient outcomes from those algorithms in classifying patients' cases depending on their chronic diseases. Table 2 shows the details of each algorithm showing their outcome of precision, recall, F1 score, and

Table 2 Performance of the five techniques

Algorithm	Precision	Recall	F1 score	Accuracy
K-Means	82	76	79	79
Decision tree	73	65	69	74
Naïve bayes	77	69	73	72
Random forest	81	71	76	78
AdaBoosted decision tree	83	77	80	82

accuracy. In our experiment, we have determined the performance of each algorithm, for predicting the year of the pandemic, which is used to represent pandemic stages. AdaBoost achieves the highest accuracy among these five approaches, and its accuracy is 82% (Table 2). By overall performance, AdaBoost gives the best result because of its iterative classification on decision trees.

5 Conclusion

The COVID-19 pandemic has taken a dramatic toll on human life including loss of livelihood, food insecurity, domestic violence, increasing suicide rate, disruption of education and other changes in lifestyle. If a country could predict the direction of a pandemic, it would be able to mobilize resources to tackle the problem. This paper presents a method for identifying the stage of a pandemic currently being faced by a country based on historical data of the pandemic. It demonstrates that it is possible to identify the year of the pandemic with reasonable accuracy given various statistics. Hence, it will also be possible to identify when a pandemic is moving from stage to stage given current statistics. It is hoped that this will help improve the preparedness of countries in facing COVID-19 escalations in the future, and that it will be useful for facing future pandemics.

References

1. Anwar S, Nasrullah M, Hosen MJ (2020) COVID-19 and Bangladesh: challenges and how to address them. Front Public Health 8. https://doi.org/10.3389/fpubh.2020.00154
2. Bodrud-Doza M, Shammi M, Bahlman L, Islam ARMT, Rahman MM (2020) Psychosocial and socio-economic crisis in bangladesh due to COVID-19 pandemic: a perception-based assessment. Frontiers Public Health 8. https://doi.org/10.3389/fpubh.2020.00341
3. Català M, Marchena M, Conesa D, Palacios P, Urdiales T, Alonso S, Alvarez-Lacalle E, Lopez D, Cardona PJ, Prats C (2021) Monitoring and analysis of COVID-19 pandemic: the need for an empirical approach. Frontiers Public Health 9. https://doi.org/10.3389/fpubh.2021.633123
4. Cullen W, Gulati G, Kelly BD (2020) Mental health in the COVID-19 pandemic. QJM: Int J Med 113(5):311–312. https://doi.org/10.1093/qjmed/hcaa110
5. Gautam S, Setu S, Khan MGQ, Khan MB (2022) Analysis of the health, economic and environmental impacts of COVID-19: the bangladesh perspective. Geosystems Geoenvironment 1(1):100011. https://doi.org/10.1016/j.geogeo.2021.100011
6. iMMAP: COVID-19 impacts in bangladesh: Nationwide survey on livelihoods, nutrition, education and health. resreport, iMMAP (2022)
7. Islam T, Talukder AK, Siddiqui N, Islam T (2020) Tackling the covid-19 pandemic: the bangladesh perspective. J Public Health Res 9(4):389–397. https://doi.org/10.4081/jphr.2020.1794
8. Iwendi C, Bashir AK, Peshkar A, Sujatha R, Chatterjee JM, Pasupuleti S, Mishra R, Pillai S, Jo O (2020) COVID-19 patient health prediction using boosted random forest algorithm. Frontiers Public Health 8. https://doi.org/10.3389/fpubh.2020.00357

9. Kurniawan R, Abdullah SNHS, Lestari F, Nazri MZA, Mujahidin A, Adnan N (2020) Clustering and correlation methods for predicting coronavirus COVID-19 risk analysis in pandemic countries. In: 2020 8th international conference on cyber and IT service management (CITSM). IEEE. https://doi.org/10.1109/citsm50537.2020.9268920
10. Liang L, Ren H, Cao R, Hu Y, Qin Z, Li C, Mei S (2020) The effect of COVID-19 on youth mental health. Psychiatr Q 91(3):841–852. https://doi.org/10.1007/s11126-020-09744-3
11. Miyah Y, Benjelloun M, Lairini S, Lahrichi A (2022) COVID-19 impact on public health, environment, human psychology, global socioeconomy, and education. Sci World J 1–8. https://doi.org/10.1155/2022/5578284
12. Ritchie H, Mathieu E, Rod´es-Guirao L, Appel C, Giattino C, Ortiz-Ospina E, Hasell J, Macdonald B, Dattani S, Roser M (2022) Coronavirus pandemic (covid19). https://ourworldindata.org/coronavirus
13. Saha P, Gulshan J (2021) Systematic assessment of COVID-19 pandemic in Bangladesh: effectiveness of preparedness in the first wave. Frontiers Public Health 9. https://doi.org/10.3389/fpubh.2021.628931
14. Singer G, Marudi M (2020) Ordinal decision-tree-based ensemble approaches: the case of controlling the daily local growth rate of the COVID-19 epidemic. Entropy 22(8):871. https://doi.org/10.3390/e22080871
15. Son C, Hegde S, Smith A, Wang X, Sasangohar F (2020) Effects of COVID-19 on college students' mental health in the United States: interview survey study. J Med Internet Res 22(9):e21279. https://doi.org/10.2196/21279
16. Wang R (2012) AdaBoost for feature selection, classification and its relation with SVM, a review. Phys Procedia 25:800–807. https://doi.org/10.1016/j.phpro.2012.03.160
17. Kabir SR, Mitin SJ, Sadeq MJ, Habiba U, Ayon RDG, Rabbi MS (2022) COVID-19 pandemic stages identification using machine learning. Zenodo. https://doi.org/10.5281/zenodo.7317553
18. Hasan MK et al (2022) Evolution of industry and Blockchain era: monitoring price hike and corruption using BIoT for smart government and industry 4.0. IEEE Trans Ind Inform 18(12):9153–9161
19. Hasan MS et al (2021) Identification of construction era for Indian subcontinent ancient and heritage buildings by using deep learning. In: Yang XS, Sherratt RS, Dey N, Joshi A (eds) Proceedings of fifth international congress on information and communication technology. ICICT 2020. Advances in intelligent systems and computing, vol 1183. Springer, Singapore. https://doi.org/10.1007/978-981-15-5856-6_64
20. Akhtaruzzaman M et al (2020) HSIC bottleneck based distributed deep learning model for load forecasting in smart grid with a comprehensive survey. IEEE Access 8:222977–223008
21. Sadeq MJ et al (2020) A cloud of things (CoT) Approach for monitoring product purchase and price hike. In: Peng SL, Son LH, Suseendran G, Balaganesh D (eds) Intelligent computing and innovation on data science. Lecture notes in networks and systems, vol 118. Springer, Singapore. https://doi.org/10.1007/978-981-15-3284-9_39
22. Alam S, Rahman MM (2022) COVID-19 impact on Facebook-based social commerce in Bangladesh. Int J Electr Comput Eng 12(2):1636–164 (2022). https://doi.org/10.11591/ijece.v12i2.pp1636-1649
23. Haque R et al (2018) Modeling the role of C2C information quality on purchase decision in Facebook. In: Al-Sharhan SA et al (eds) Challenges and opportunities in the digital era. I3E 2018. Lecture notes in computer science, vol 11195. Springer, Cham. https://doi.org/10.1007/978-3-030-02131-3_22
24. Sadeq MJ et al (2021) Integration of Blockchain and remote database access protocol-based database. In: Yang XS, Sherratt S, Dey N, Joshi A (eds) Proceedings of fifth international congress on information and communication technology. Advances in intelligent systems and computing, vol 1184. Springer, Singapore. https://doi.org/10.1007/978-981-15-5859-7_53

A Multi-feature Analysis of Accented Multisyllabic Malayalam Words—a Low-Resourced Language



Rizwana Kallooravi Thandil, K. P. Mohamed Basheer, and V. K. Muneer

Abstract Accented Automatic Speech Recognition (AASR) is an active area of research in all languages. Even though the research has advanced far in the area of Automatic Speech Recognition (ASR) in languages like English, Arabic, and Chinese; the research is still in its infancy in the area of AASR. Malayalam is a language that is rich with many varieties of accents spoken in different parts of Kerala. Here in this work, the authors discuss the various experiments conducted on the accented features and propose a novel approach to better represent the accented speech data for the Malayalam language. There is no benchmark data available for the accented Malayalam speech and hence authors have collected data across five districts to represent five different accents from people of all age groups. The data collected includes multisyllabic isolated utterances recorded under a natural recording environment using crowdsourcing methods. The data collected have been properly annotated and then underwent a series of experiments to find the best feature extraction method that would best represent the accented speech signals. In every experiment, we used different features to build machine learning models using different machine learning approaches. And finally, we combined all the feature extraction methods and constructed a deep learning model using Long Short-Term Memory—Recurrent Neural Networks (LSTM-RNN).

Keywords Accented Malayalam speech · Accented automatic speech recognition · Acoustic modeling · Speech feature extraction

1 Introduction

The accent is a variation in the pronunciation of languages across different geographical regions. Speech data attributed to different accents pose a great challenge in constructing the ASR for any language. The pitch, modulation, and background noise

R. K. Thandil (✉) · K. P. Mohamed Basheer · V. K. Muneer
Sullamussalam Science College, Areekode, Kerala, India
e-mail: ktrizwana@gmail.com

© The Author(s), under exclusive license to Springer Nature Singapore Pte Ltd. 2023

S. Chinara et al. (eds.), *Advances in Distributed Computing and Machine Learning*, Lecture Notes in Networks and Systems 660,
https://doi.org/10.1007/978-981-99-1203-2_21

of the speech signals, and the speed of utterance, age, and gender of the speaker are some other factors that make the task of ASR complicated. Malayalam is a Dravidian language spoken in the Indian state of Kerala and the Lakshadweep Islands. Malayalam has different varieties of accents spoken throughout Kerala based on the different regions/districts, religions, and castes. The authors here propose a novel approach to construct a unified accent model for the Malayalam language.

Since there are no benchmark datasets available for accented Malayalam speech, the authors constructed a dataset that includes native speakers of five districts to conduct the experiments. The paper discusses the experiments conducted on the different feature selection methods for modeling Malayalam accented speech and proposes a novel approach for feature selection.

The whole system pipeline can be described in this section. The authors primarily worked on Mel Frequency Cepstral Coefficient algorithm (MFCC) to extract the audio features from the speech data and constructed machine learning models adopting different machine learning methods, i.e., Decision Tree Classifier, Multi-Layer Perceptron (MLP), Support Vector Machine (SVM), Stochastic Gradient Descent Classifier (SGD), K Nearest Neighbor (KNN) Classifier, Random Forest Classifier, and Ensembled learning methods. The authors then used Short Term Fourier Transform (STFT) to extract the low-frequency features from the audio signals which are then further used to construct machine learning models using all the above methods. The authors then worked with the combined feature set obtained using MFCC and STFT and constructed the machine learning models as mentioned above. The authors then extracted the tempogram features from the audio data to extract the accent features from the speech data and repeated the above experiment by constructing the machine learning models. Later, the MFCC is combined with the tempogram features and machine learning models were constructed as mentioned above. Finally, a Neural Network LSTM-RNN model was constructed using the combined feature set obtained by MFCC, STFT, and tempogram methods.

2 Related Work

Research on Accented Speech Recognition for the Malayalam language is advancing at a very slow pace. The unavailability of a benchmark dataset makes the task more challenging for the language. AASR is an active area of research for many languages across the globe. The factors that cause the deviation from the standard accents have to be properly identified for each language. Various studies are being conducted for identifying the accent of a language and also identifying the utterances in a particular accent of a language. A uniform model to identify all the accents of a language would be ideal when considering AASR for any language. This session discusses some of the relevant works that have happened in the area of accented speech recognition.

Keqi Deng et al. [1] proposed self-supervised pretrained methods for accent identification of audio data. They proposed an accent-dependent ASR for accented speech

recognition. They used the mean value of the frame-level vectors for accent identification rather than the sentence-level vectors. Huang et al. [2] proposed a system for accent identification where they collected 160 h data from non-native speakers from eight different countries and used a pretrained model for better results. They used the phone posteriorgram method for extracting the accent features, and the experiment yielded better results with a reduced WER.

Hyeong-Ju Na et al. [3] proposed a Domain Adversarial Neural Network (DANN) for modeling accented speech. They could considerably reduce the deviations in the accented speech from the standard speech. They constructed the ASR using three different subnets such as feature extraction, domain prediction, and accent classification. The outcome of their results proved that the DANN method could reduce the distribution differences between accented speech and standard speech. Nilaksh Das et al. [4] proposed an approach by combining transfer learning and adversarial learning methods. With this approach, they could considerably reduce the WER as compared to the baseline approaches. The dataset they chose contained annotated samples from only one standard accent.

Chen et al. [5] proposed a model for unified accent recognition that could identify speech across multiple accents. They experimented on generative adversarial nets for multi-accent ASR. They proposed an accent invariant method combined with attention-based encoder décor for multi-accent ASR. The experiment also resulted in a considerable decrease in the WER rate. Song Li et al. [6] worked on accented English data from eight countries and proposed two unsupervised architectures for modeling accented data. The proposed methodology could map different accents to a unified model for modeling multi-accent speech data.

3 Proposed Methodology and Design

The primary objective of this experiment is to find a better feature extraction approach to represent the accented Malayalam speech. The experiment conducted used accented data from five accent-speaking districts of the northern part of Kerala. The data collected contained multisyllabic isolated utterances recorded under a natural recording environment. The authors have considered speech input from donors belonging to all age groups of male and female genders.

The speech signals undergo a series of feature extraction mechanisms, and the experiments are conducted to model the accented speech data using machine learning and deep learning approaches. The accent variants are used to construct a unified accented speech model for the Malayalam language. The word-level vectors are used for the study. Every accented speech is annotated with a standard accent for better mapping of the accented speech with the standard accent. Separate experiments are conducted for each feature set extracted using different approaches. The results of individual experiments are then compared against each other to find the better feature extraction technique. The flow of the proposed methodology is shown below (Fig. 1).

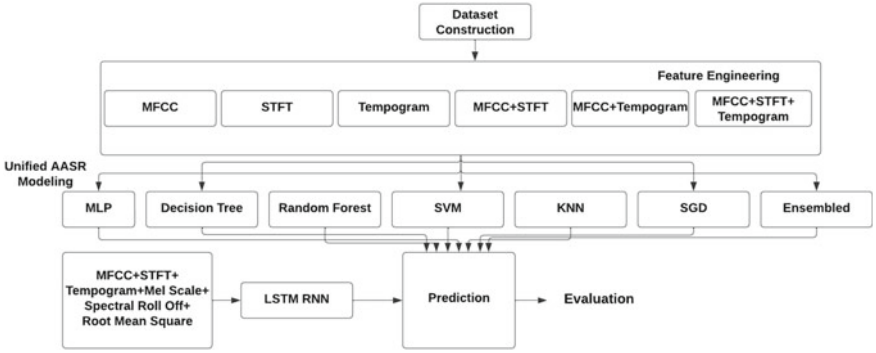


Fig. 1 Flow of the proposed methodology

3.1 Dataset Construction

A speech corpus of about 1.17 h has been constructed for this study. Data collected includes isolated words with multiple syllables. Some words are of only two syllables while others range from three to five syllables. The individual utterances range from two seconds to five seconds in the dataset. Speech recordings from forty different speakers are collected, i.e., from twenty male and twenty female speakers. Speech samples from five different districts have been collected from speakers of the age groups that range from five to eighty. The data we have considered for this experiment is from the Kasaragod, Kannur, Kozhikode, Wayanad, and Malappuram districts of Kerala. The accents are highly influenced by the languages that are spoken in the neighboring states with which these districts share the border.

We have collected the majority of the samples from the Kozhikode district where the accent is comparatively closer to the standard accent. Every speech signal is annotated with the text with the standard accent. The accented words are vectorized and compared against the vectors of the standard accented similar words. The data collected are then converted into .wav format and sampled to a frequency of 16 kHz. Table 1 contains the district-wise data collection statistics.

Table 1 District-wise data collection statistics

District	No. of audio samples
Kasaragod	1360
Kannur	1360
Kozhikode	1690
Malappuram	1360
Wayanad	1300
Total	7070

3.2 Experiment with MFCC

The speech samples that are collected are vectorized and labeled accordingly. Initially, we used the Mel Frequency Cepstral Coefficients to extract the speech features from the audio input. Forty features from every uttered word are extracted and annotated for the study. A triangular filter bank linearly spaced on the Mel frequency scale is used to obtain the MFCC coefficients by de-correlating the output log energies. De-correlation of speech is typically carried out using a discrete cosine transform (DCT) since it is the best approximation of the Karhunen–Loeve Transform (KLT) [7, 8].

The original speech frequency is measured in Hz, whereas the subjective pitch can be measured in mel scale. These word-level vectors are then used to model the accented speech using MLP, Decision Tree, Random Forest, KNN, SGD, SVM, and Ensembled methods.

3.3 Experiment with STFT

The signal is decomposed, and the relative strength of the signal is analyzed using the Fourier analysis method. Here in this experiment, we have divided the signal into small overlapping frames where a fast Fourier transform is applied to each frame [9]. Twelve spectral features are extracted from each speech signal, and a dataset is constructed for modeling the accented speech using the machine learning algorithms mentioned in the above experiments.

3.4 Experiment with Combined MFCC and STFT

Here, the authors combined the forty MFCC vectors and twelve STFT vectors to jointly represent individual word utterances. The dataset has been constructed with fifty-two vectors for each utterance. The vectors are then used to construct the accented ASR using the machine learning methods described in the other experiments.

3.5 Experiment with Tempogram Features

Tempograms can be used to characterize tempo variations and local pulses in speech signals. Tempograms indicate the variations in the pulse strength over the time pulses of the audio data. These are normally used to represent the rhythmic patterns in speech signals. Since the accent information can be contained in the rhythmic patterns,

tempogram corresponds to the accent vectors from the speech signals. Seven hundred and sixty-five features are extracted from every speech signal, and the dataset has been constructed for each speech signal. These vectors are then used to repeat the same experiments discussed earlier.

3.6 Experiment with Tempogram, STFT and MFCC

Here, the authors combined all the vectors obtained from MFCC, STFT, and tempogram to construct the accented ASR. Hence every signal is vectorized into 817 vectors. The dataset contains all eight hundred and seventeen vectors for every speech signal. These vectors are hence used to construct the accented speech models following the same procedure in the above experiments.

3.7 Experiment with Tempogram and MFCC

Here, the combined features of tempogram and MFCC are used for word-level vectorization. Every speech signal is vectorized into eight hundred and five vectors, and hence the dataset has been constructed. These then underwent similar procedures as in the earlier experiments.

3.8 Experiment Using MFCC, STFT, Mel Spectrogram, Spectral Roll off, Root Mean Square and Tempogram

Here, the authors have tried to extract the maximum features for every accented speech signal. Here, the authors took account of forty MFCC features, twelve STFT vectors, one hundred and twenty-eight Mel Spectrogram features, two hundred and fifty-six spectral roll-off features, three hundred and twenty-nine root mean squared vectors, and seven hundred and sixty-five tempogram vectors. A total of thousand and ninety-seven vectors are used to represent each speech signal in the dataset. This dataset is used to model the accented ASR using the LSTM-RNN methods.

4 Experimental Results

Seven different experiments were conducted in this study to investigate the better vectorization procedure for accented Malayalam speech. All experiments used

Table 2 The WER percentage for accented machine learning models

Feature engineering	MLP (%)	Decision tree (%)	SVM (%)	RFC (%)	KNN (%)	SGD (%)	Ensembled (%)
MFCC	0.52	47.85	39.72	19.52	18.31	68.74	18.31
STFT	62.69	65	69.78	55.96	55	99	56.48
Tempogram	28	66	70	58	60	84.81	56.31
MFCC + STFT	5.88	46.12	40.25	19.35	18.31	73.06	56.48
MFCC + Tempogram	0	52.85	36.44	35.92	16.59	76.86	19.69
MFCC + STFT + Tempogram	0.5	45	30.71	15.67	17.23	75.35	17.12

different approaches for the vectorization of the accented data. The first six experiments used the respective vectors to construct the accent ASR using various machine learning methods, whereas the last experiment involved the deep learning approach (Table 2).

The experiment produced better results when the MFCC, STFT, and tempogram features are combined to model the accented data. It is noted that SGD did not perform well in any of the experiments, and hence it can be concluded that the method is not ideal for modeling the accented Malayalam speech (Fig. 2).

The seventh experiment was conducted using thousand and ninety-seven vectors from the accented data using LSTM-RNN. LSTM-RNNs are neural networks that can be used to model sequential data. The labels in the experiments are one-hot encoded and are fed into the network for model construction. Here in this experiment, the model was trained for 98,000 steps and obtained a training accuracy of ninety-five percent and a test accuracy of 82% over 98,000 steps. The model produced a WER of 5% during the training phase and 18% during the validation period. The model was trained for 3500 epochs with a batch size of 20. The training loss of the model declined to 0.24% at step 98,000 which was 0.5 when the training began (Fig. 3).

5 Conclusion and Future Scope

The authors investigated the feature extraction process of accented speech for the Malayalam language. The authors initially constructed a dataset from scratch and vectorized the recordings at the word level. Seven different approaches were adopted for the feature engineering process. Experiments were conducted for every approach of feature engineering by modeling the accented speech using different machine learning approaches. A deep learning approach was also adopted in the final experiment where the maximum vectors have been considered. The experiment that used maximum vectors generated better results both in the case of machine learning and

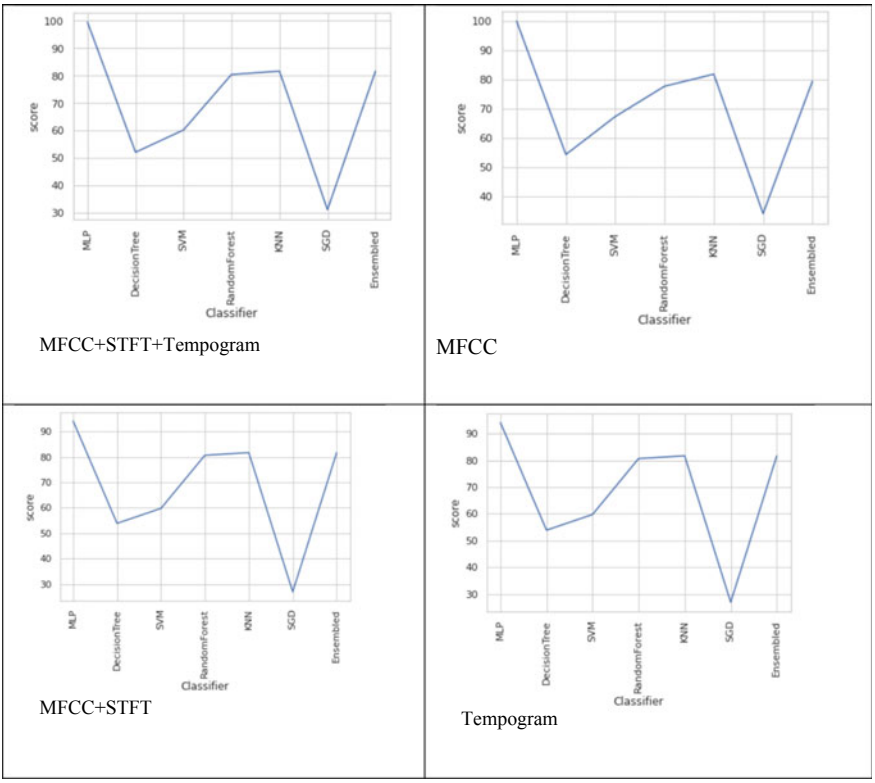


Fig. 2 Performance evaluation of experiments in terms of accuracy



Fig. 3 Performance evaluation for LSTM RNN

deep learning scenarios. There are certain methods like SGD that proved to be not fit for modeling the dataset we have constructed.

The authors could successfully propose a method that can be adopted for the feature engineering process for the accented speech data. The experiments with multiple feature extraction techniques gave better results than many other baseline methods.

References

1. Deng K, Cao S, Ma L (2021) Improving accent identification and accented speech recognition under a framework of self-supervised learning. arXiv:2109.07349. https://doi.org/10.48550/arXiv.2109.07349
2. Huang H, Xiang X, Yang Y, Ma, Qian Y (2021) AISpeech-SJTU accent identification system for the accented english speech recognition challenge. In: ICASSP 2021—2021 IEEE international conference on acoustics, speech and signal processing (ICASSP), pp 6254–6258. https://doi.org/10.1109/ICASSP39728.2021.9414292
3. Na H-J, Park J-S (2021) Accented speech recognition based on end-to-end domain adversarial training of neural networks. Appl Sci 11(18):8412. https://doi.org/10.3390/app11188412
4. Das N, Bodapati S, Sunkara M, Srinivasan S, Chau DH Best of both worlds: robust accented speech recognition with adversarial transfer learning. arXiv:2103.05834. https://doi.org/10.48550/arxiv.2103.05834
5. Chen Y-C, Yang Z, Yeh C-F, Jain M, Seltzer ML (2020) Aipnet: generative adversarial pre-training of accent-invariant networks for end-to-end speech recognition. In: ICASSP 2020—2020 IEEE international conference on acoustics, speech, and signal processing (ICASSP), pp 6979–6983. https://doi.org/10.1109/ICASSP40776.2020.9053098
6. Li S, Ouyang B, Liao D, Xia S, Li L, Hong Q (2021) End-To-end multi-accent speech recognition with unsupervised accent modelling. In: ICASSP 2021—2021 IEEE international conference on acoustics, speech and signal processing (ICASSP). 1EU1jBxj8nKfvCaAzdeq1yafPEGrimcg8k
7. Hossan MA, Memon S, Gregory MA (2010) A novel approach for MFCC feature extraction. In: 2010 4th international conference on signal processing and communication systems: n. pag. Web
8. Sahidullah M, Saha G (2009) On the use of distributed dct in speaker identification. In: 2009 annual IEEE India conference (INDICON), pp 1–4
9. Liu H, Li L, Ma J (2016) Rolling bearing fault diagnosis based on STFT-deep learning and sound signals. Shock Vib 2016, Article ID 6127479:12. https://doi.org/10.1155/2016/6127479

Machine Learning Based Fruit Detection System



Krishnapriya Ajit and S. Sofana Reka

Abstract Technology has improved our lives in all areas including agriculture. New and efficient techniques for evaluating fruits and vegetables in a garden have been rapidly increasing in the agricultural field. Real-time fruit detection is highly effective in monitoring fruits and vegetables in the garden remotely. This work proposes a scheme for real-time fruit detection from the real-time video or images using Convolutional Neural Network (CNN). The CNN algorithm we use in this project is YOLOv3 (You Look Only Once) algorithm. YOLO is one of the fastest real-time object detection algorithms with a reasonably good accuracy. In this paper, we prepare the dataset of fruit images, pretrain the convolutional network and then detect them from real-time camera images or video. The training and detection were done on NVIDIA GeForce GPU for faster processing of image data. The result would be a framed label for each detected fruit in the video or image.

Keywords Embedded systems · Convolutional neural network (CNN) · YOLOv3 · Real-time detection

1 Introduction

The intervention of technology in any field can improve the efficiency and productivity of it. New and improved techniques have been incorporated in the agriculture sector too. Real time and early detection of fruits and vegetables help in the harvesting in large orchards and gardens. This project aims at detecting fruits from gardens and fields with the help of embedded systems. Real-time detection is a more efficient way to estimate the supplies and manage them. This work uses real-time camera images

K. Ajit
School of Electronics Engineering, Vellore Institute of Technology, Chennai, India

S. Sofana Reka (✉)
Centre for Smart Grid Technologies, School of Electronics Engineering, Vellore Institute of Technology, Chennai, India
e-mail: chocos.sofana@gmail.com

© The Author(s), under exclusive license to Springer Nature Singapore Pte Ltd. 2023

S. Chinara et al. (eds.), *Advances in Distributed Computing and Machine Learning*, Lecture Notes in Networks and Systems 660,
https://doi.org/10.1007/978-981-99-1203-2_22

to detect fruit images with the help of the YOLO algorithm. Traditional detection methods used intensive hardware setup and was difficult for infield installation. This method is simpler, compact and the detection can be done remotely by installing cameras at proper positions in the fields. In horticulture, though a lot of technological developments have been arising, handpicking is the most common way of sorting fruits while harvesting. Detecting the different fruits in a garden with different varieties of fruits and vegetables can be a challenging task since the ripening and maturing of different crops have different seasons and timelines. Sorting and classifying are a manual and tiring job for large scale fruit vendors also. In this work mainly projects on establishing the identification of discriminative features and in accordance with that classifying the fruits.

1.1 Related Work

Cohen et al. [1] collected images with the help of a standard CCD camera. Using these images an algorithm was designed for quantifying the number of apples on trees. This algorithm showed an accuracy of 85% in detecting and finding the location of the apples in the images. The combination of color and texture analysis with shape analysis resulted in this high detection rate. An automatic recognition vision system was developed by Zhao et al. [2] to assist an apple harvesting robot. Machine vision systems are used to detect viably and locate the apples. The work was implemented in 4 stages based on vision system setup and image acquisition using robot vision, Image processing, Apple automatic recognition based on SVM and Field testing. The pre-processing of the images of the fruits obtained using CCD camera was done by the vector median filter. The field testing resulted in an accuracy of approximately 89%.

Gongal et al. [3] also used the machine vision system by setting up a huge hardware installation for locating and recognizing apples that are not fully visible and also bunches of apples with great precision. Artificial lighting and structural installations helped in enhancing the environment of imaging. The technique involved in detecting the apples that are not fully visible and also bunches of apples clearly was Circular Hough Transform (CHT). During the field testing, approximately 90% apple identification accuracy was obtained.

Other works include sorting and classifying fruits with the help of Artificial Neural Networks and SVM Classifiers [7] and also by involving physical sensors and machine vision with the help of image processing [6].

2 Background Details

This section describes the background details of this project such as the concept of real-time object detection, the deep learning strategies involved and the details of the CNN algorithms used.

2.1 Real-Time Object Detection

Humans can identify and detect objects in their line of vision in a span of milliseconds. This is real-time object detection. Machines are trained with various algorithms and technologies to do the same as human vision. So, we use Real-time Object detection algorithms. Object detection can be object localization or recognition based on the number of images to be detected. In our project the network is trained to detect and classify fruits from the real-time vision.

2.2 Convolutional Neural Network (CNN)

Convolutional Neural Networks (CNNs) are feed-forward artificial neural networks. CNNs have layered architecture, namely, the convolution layer, pooling layer, fully connected layers and soft-max layer. CNN learns and extract features automatically through a backpropagation algorithm. CNN has recently achieved expert-level performances in various fields, including the agriculture sector.

2.3 YOLOv3 Algorithm

YOLO is the shortened form of You Only Look Once. The version used in our project is YOLOv3. It is an algorithm that detects particular objects from inputs images, videos and live feeds. YOLO network was particularly designed for detecting real-time objects quickly and accurately. YOLO is a single fully convolutional neural network(F- CNN) that takes a raw image as input and gives bounding boxes with related classes of recognized objects inside the presented scene as output. The deep learning libraries required for the implementation of this algorithm are the Keras or OpenCV.

2.4 Architecture of YOLOv3

The YOLO first takes an input image and divides it into smaller grids. After classifying the whole input image into smaller grids of uniform size, it employs classification and localization techniques. Then it detects the bounding box coordinates for each of the grids. Class probabilities can be learnt for the objects recognized by this method. In YOLO, the neural network is applied to the entire image to detect the bounding boxes and the probability of detection for each region. The regions with high probability of detection are assumed to be the objects identified. YOLO is faster as it scans the image only once when compared to other algorithms which performs multiple scans. That is why it is called 'You Look Only Once'.

Darknet is a 53-layer network trained on Imagenet. YOLOv3 is an adaption of the Darknet. YOLO, in its version 3 has added features for improving the training and increasing the performance. The added features include multi-scale predictions and other minor techniques.

The addition of an extra 53 layers on top of the Darknet layer results in a fully convolutional YOLO v3 architecture with 106 layers. 1×1 detection kernels on feature maps of three different sizes are applied at three different places in the network for detection. The shape of the detection kernel is given by the equation:

$$1 \times 1 \times (P \times (5 + Q))$$

where, P: number of bounding boxes that can be predicted by a cell on the feature map,

Q: number of classes.

There are 4 bounding box attributes and one object confidence which adds up to 5 as represented in the equation. The classification loss for each label is calculated using binary cross-entropy. Logistic regression is used for predicting the object confidence and class prediction (Fig. 1).

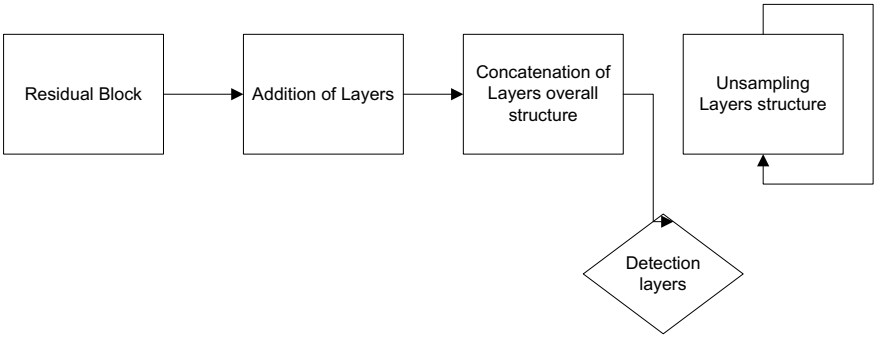


Fig. 1 Architecture of YOLOv3

3 Proposed Methodology

In the first process, a dataset was prepared from raw images obtained from the internet. In the second step, the algorithm for classification YOLO was customized accord- ing to the prepared dataset. The third step involves generating configuration and weight files. Next step is to train the YOLO using the prepared dataset for the detection and classification of various fruits.The proposed model is exhibited as shown in Fig. 2 explaining the process.

At first raw images of various fruits were downloaded from the internet. Photographic images as well as clipart images were used. Each fruit was labelled and framed in all the images using the LabelImg software. LabelImg is an open-source tool used for graphically labelling images written in Python. The labelled images are saved along with a file containing the class information and annotations in YOLO format. The prepared images and its yolo labelled files are zipped and uploaded to Google drive to use in Google Colab. The uploaded zip files from Google Drive are mounted in Colab. The darknet is then cloned into the Colab for customization. The Darknet is configured by enabling the GPU and OpenCV. NVIDIA CUDA DNN is also enabled for GPU accelerated training of the network. The training is done in NVIDIA GPU for faster processing. After configuring the Darknet is compiled.The .cfg files are configured by altering the values of the following parameters according to our dataset. The pretrained weights for the convolutional layers along with the configured .cfg file and the names files are downloaded and the network is trained with the custom dataset prepared. The fruit detection program is written in Python language and is detected using the webcam real-time (Table 1).

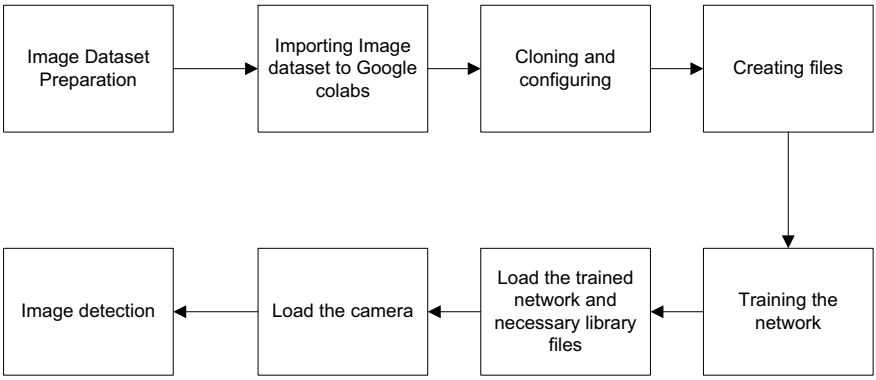


Fig. 2 Proposed model

Table 1 Yolov3 configurations

YOLOv3 configurations	
Parameters	Value
Batches	64
Subdivisions	16
Classes	8
Max_batches	16,000
Filters	39

4 Results and Discussions

The dataset consists of images of 8 fruits viz., Apple, Banana, Grapes, Mango, Orange, Pineapple, Strawberry and Watermelon, divided into 8 classes. Each class consists of around 230 images comprising a total of 1716 images used for training the network. Later 655 images from different classes are used for testing and validation (Fig. 3).

The experiments were performed in a PC with Intel(R) Core (TM) CPU at 1.6 GHz. The tool used for dataset preparation was LabelImg. The customisation, preprocessing and training the CNN is done on Google Colab using the NVIDIA GeForce MX130 GPU. The image detection was done on the Jupyter Notebook IDE.The program was coded in Python. The necessary library files and the trained network are loaded and then the camera is enabled. The webcam turns on and the images are detected from the webcam feed and the fruit images are identified in a frame along with the classification accuracy and the fruit name (Fig. 4).

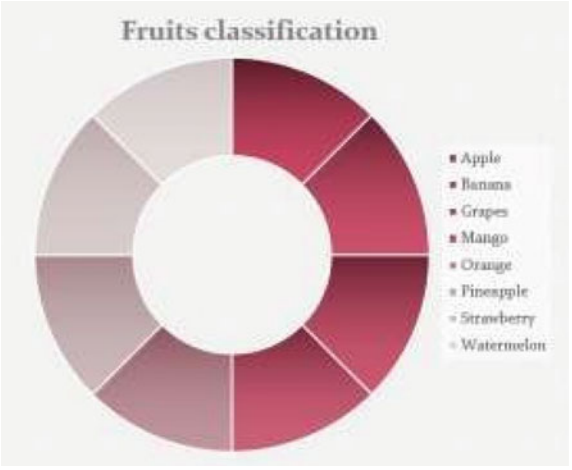


Fig. 3 Fruits classification

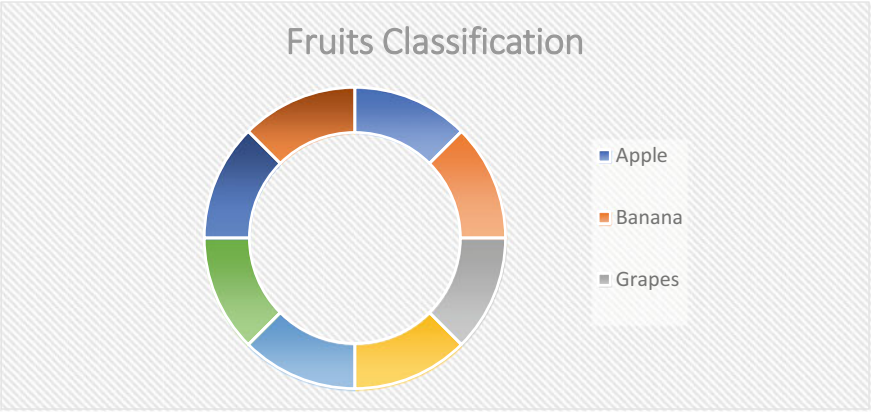


Fig. 4 Implementation results

5 Conclusion

In this paper, the main focus was on real-time detection and classification of the fruits into 8 categories namely, Apple, Banana, Grapes, Mango, Orange, Pineapple, Strawberry and Watermelon based on the prepared custom dataset. The dataset was prepared from raw images from the internet and the YOLO network was trained using the prepared dataset. The proposed approach has achieved an average accuracy of 85–98%. The results were detected slower but with fair accuracy. The future work aims at improving the detecting speed of the images and the accuracy. The present work detects images from real-time webcam feed which can be modified in future by including the inputs from the image file and video file other than the webcam feed. The network can be further trained for raw and ripened images of any particular fruit for improving the harvesting quality. The estimation of the quantity of the fruits detected can also be developed as a next step.

References

1. Cohen O, Linker R, Naor A (2011) Estimation of the number of apples in color images recorded in orchards. In: Proceedings of the international conference computer and computing technologies in agriculture, pp 630–642
2. Ji W, Zhao D, Cheng F, Xu B, Zhang Y, Wang J (2012) Automatic recognition vision system guided for apple harvesting robot. Comput Elect Eng 38(5):1186–1195
3. Gongal A, Silwal A, Karkee M, Zhang Q, Lewis K, Amatya S (2016) Apple crop-load estimation with over-the-row machine vision system. Comput Electron Agric 120:26–35
4. Jawale D, Deshmukh M (2017) Real time automatic bruise detection in (Apple) fruits using thermal camera. In: 2017 international conference on communication and signal processing (ICCSP). IEEE, pp 1080–1085

5. Kang H, Chen C (2020) Fast implementation of real-time fruit detection in apple orchards using deep learning. Comput Electron Agric 168:105108
6. Li P, Lee S-H, Hsu H-Y (2011) Review on fruit harvesting method for potential use of automatic fruit harvesting systems. Procedia Eng 23:351–366
7. Abirami L, Bajpai P, Pavithraa S, Adithya M, Kavitha T Automatic fruit classification system using machine learning techniques- a detailed analysis
8. Bulanon DM, Kataoka T, Okamoto H, Hata S (2004) Development of a real-time machine vision system for the apple harvesting robot. In: SICE 2004 annual conference, Sapporo, vol 1, pp 595–598
9. https://towardsdatascience.com/a-comprehensive-guide-to-convolutional-neural-networks-the-eli5-way-3bd2b1164a53
10. Bhadoria RK, Chowdhury C (2021) Healthcare paradigms in the internet of things ecosystem

Post hoc Interpretability: Review on New Frontiers of Interpretable AI



Ashly Ann Jo and Ebin Deni Raj

Abstract In order to make critical decisions, organizations are increasingly using sophisticated black box machine learning algorithms. Post hoc interpretability techniques are a well-liked solution to the issue of transparency in black box machine learning algorithms. These techniques try to explain the internal workings of the underlying machine learning models so that stakeholders can comprehend them. This paper suggests that post hoc interpretability approaches' insights can be used to control black box machine learning. The veracity of these statements is investigated in this article. This paper discusses the post hoc interpretability methods on individual prediction. The paper provides detailed comparison of post hoc interpretability methods, software packages, and interpretability toolkits available in market.

Keywords Post hoc interpretability · Black box · Explainability

1 Introduction

It has been noticed in a small number of commercial AI implementations that their judgments are biased. Recent instances [1] include Apple's credit card's sexism, algorithms that charge higher interest rates to black and Latino borrowers in the mortgage industry, and an organization's preference for treating white patients over black patients. Computer vision algorithms, voice recognition systems, and text classifiers may all be hacked, according to studies. The performance and widespread adoption of AI have been called into doubt as a result of its poor choices and inherent weaknesses. Customers have started to distrust the accuracy and fairness of the judgments because of the obvious AI bias, thus companies need a way to audit these conclusions. The advent of XAI (explainable artificial intelligence) is timely. When

A. Ann Jo (✉) · E. Deni Raj
Indian Institute of Information Technology, Kottayam, India
e-mail: ashlyannjo.phd2112@iiitkottayam.ac.in

E. Deni Raj
e-mail: ebindeniraj@iiitkottayam.ac.in.com

© The Author(s), under exclusive license to Springer Nature Singapore Pte Ltd. 2023

S. Chinara et al. (eds.), *Advances in Distributed Computing and Machine Learning*,
Lecture Notes in Networks and Systems 660,
https://doi.org/10.1007/978-981-99-1203-2_23

it comes to models, XAI isn't able to reveal their inner workings or the factors that may have an impact on a given choice, but it can provide a set of tools and procedures that can assist shed light on these issues.

By constructing more explainable models, while keeping a high degree of performance, explainable AI allows human users to understand and accept model decisions as fair, responsible, and transparent. Several studies have proposed novel Explainable AI (XAI) techniques with the aim of producing model explanations with desirable qualities including explainability, fairness, robustness, and interpretability [6]. Though explanations are moreover evaluated based on their effect on decisions provided by the models. Interpretability focuses on model understanding techniques. The most used traditional model interpretation techniques are linear and non-linear models, visualization, and exploratory data analysis [8]. Interpretable machine learning methods are important during building and operations of machine learning workflow. It paves the way for building an explainable system. Interpretability becomes useful when the model must explain how it comes to the prediction because the correct prediction can only solve the existing original problem. Most problems involving tabular data such as credit card rejection or acceptance, Loan approval, etc., require fully interpretable machine learning models. Humans can only make sense of a model's computation if it's completely interpretable. Both numerical and categorized information may be understood in tabular data. Although tabular data is of great academic and commercial relevance, it has been often ignored since most works concentrate on explaining their predictions. We present a high-level overview of tabular data post hoc interpretability. The paper is organized as follows: Sect. 2 gives a brief introduction of post hoc interpretability on tabular data, the interpretation method, and comparison. Section 3 provides a detailed description of Interpretability AI toolkits. Section 4 focuses on post hoc interpretability of local individual prediction and illustrates with examples. Section 5 discusses several limitations of post hoc interpretability. Finally, Sects. 6 and 7 discuss the future scope of post hoc interpretability and the conclusion of the paper, respectively.

2 Post hoc Interpretability on Tabular Data

Post hoc interpretability refers to the process of deciphering a black box model, for example, neural network or ensemble model, using model interpretability techniques such as feature significance, partial dependency, or LIME after the model has been trained. It applies methods to analyze the trained model and is also applicable to intrinsically interpretable models. Post hoc interpretability is classified into two; individual prediction and entire model prediction. Software for interpreting models that are model-specific is designed to work only with that model or family of models. The effectiveness and functionality of these instruments are highly dependent on a particular model. Model-specific methods function according to the unique architecture of the used machine learning or deep learning model. In this case, the strategies are used overtly for a predetermined model structure. These methods employ a reverse

engineering strategy to reveal the inner workings of a machine learning model, such as a neural network, and explain how the underlying deep learning or machine learning algorithm arrived at the desired conclusion. Using model-specific models has several benefits, including a better understanding of the choice thanks to familiarity with the model's inner workings and the ability to create a more tailored, explainable model. Integrated Gradients, DeepLIFT, TreeSHAP, and LIME+SHAP are just a few of the prevalent model-specific techniques.

Tools that are model-agnostic may be used to any machine learning model, regardless of its complexity. These unbiased strategies typically examine pairs of input and output features. White box methods are those that explicitly take a model into account, whereas black box methods are those that are model-agnostic. Without considering the model's structure, model-agnostic approaches can't be trusted. They function as a black box model and may be used with any machine learning technique. To figure out what's going on, they juggle and tinker with the input data to see how the results change, and then measure the effect that these changes had on the data's overall performance. SHAP and LIME are two well-known model-agnostic methods. Both of these methods are generalizable to any ML model since they both operate on the black box principle of varying inputs to see what happens. They work with any kind of ML algorithm. These model-independent techniques may be used independently of the model training process. As a technique, post hoc interpretation is model-neutral. One of the ways to achieve Post hoc interpretability is to deploy a local proxy model. There are several local interpretable models are available for individual prediction in post hoc explainability. In Sect. 3, we discuss further the local individual interpretation methods and their comparative study.

3 Interpretable AI Toolkits

3.1 AIX360

Open-source toolkit such as AI Explainability360 supports both explainability and interpretability of machine learning models. IBM built AI Explainability 360 using algorithms including case-based reasoning, directly interpretable rules, post hoc local explanations, and global explanations so that it could handle different types of explanations with precision. AIX360 provides an interactive experience with credit scoring application. It differs from other open-source interpretation services in its diversity of methods, emphasis on multi-stakeholder education, and scalability through a common framework. It interacts with two more open-source toolkits from IBM Research launched in 2018 to aid in the development of comprehensive and trustworthy machine learning pipelines; these are AI Fairness 360 and Adversarial Robustness 360. This toolbox is designed to include several interpretation algorithms. The toolkit provides an API and unifies the interface for each algorithm. It is also prepared for customized implementation, so the users can add their own algorithms.

AIX360 contains several algorithms with different goals. Some focus on interpret data or models. Algorithms focus on this model distinguished by the way they interpret and the scope they consider. AIX360 contains the algorithms such as ProtoDash, Teaching explanations for decisions(TED), ProfWeight, etc.

3.2 Dr.Why.AI-Dalex

DALEX is a collection of tools, which aid in understanding how complex models work. DALEX explainers can be used to determine what type of relationship the model is capable of learning and what the model has learned. Dalex is compatible with XGBoost, TensorFlow, and H2O. Python and R can be used to install it.

3.3 InterpretML

Data scientists and academics alike may benefit from the interpretability algorithms included in the open-source Python module InterpretML. This package provides two kinds of interpretation methods such as glass box and black box. Glass-box methods include existing inherently interpretable models, for example, logistic regression, linear regression, decision trees, etc. It can be trained as part of the package, as well as corresponding interpretation tools. While black box models include only model-independent explanatory tools such as LIME and Kernel SHAP that are compatible with models trained outside the package. The glass-box models provide explanations for both local and global scope, while black box models only support local interpretation. In addition, InterpretML has an integrated visualization method that allows users to easily compare different methods. InterpretML package is available at GitHub and Pip.Microsoft's new interpretable model, the Explainable Boosting Machine (EBM), is included in this package as well. Although it has similarities to black box models, EBM may be interpreted by humans. In addition, InterpretML also provides other interpretable models and black box model interpretation tools.

3.4 H2O Driverless AI

H2O Driverless AI is the premium closed-source version. Machine learning interpretability is a strong capability of H2O Driverless AI [24]. As a consequence, the outputs of machine learning models are easier to understand. It includes four graphs that illustrate several approaches to understanding and explaining the models' outputs. K-LIME, Feature Importance, Shapley, Partial Dependence and Decision Tree,

and Plot charts are only a few of the examples. Currently, H2O Driverless AI is supported for experiments such as binary classification, regression, multi-class classification, and time-series predictions.

3.5 Amazon SageMaker Clarify

By identifying possible biases and explaining how machine learning models create predictions, Amazon Sage Maker Clarify [25] aids in the development of better machine learning models. SageMaker Clarify allows users to create trustworthy and interpretable machine learning models thanks to its fairness and explainability. Biases in data collection, model training, and model deployment are all quantified by Amazon SageMaker Clarify. It generates governance reports for distribution to external authorities and internal risk and compliance departments.

3.6 Quantus

Anna Hedstrom et al. [26] create Quantus, a comprehensive Python toolbox that collects, organizes, and describes a variety of metrics recommended for explanatory methods. Quantus provides targeted users, ML, and XAI professionals with a set of over 25 reference metrics for evaluating ML forecast descriptions. It also provides detailed instructions on how to use these measurements. Quantus was developed for quantitative evaluation of explanations (Table 1).

Table 1 Interpretability AI Toolkit for explaining Black box models

Toolkit	Ante hoc	Post hoc	Local individual interpretation	Local entire-model interpretation	Quantitative metrics
AIX360	✓	✓	✓	✓	✓
Dalex	×	✓	✓	✓	×
H2O	✓	✓	✓	✓	×
InterpretML	✓	✓	✓	✓	×
Amazon sage-maker clarify	×	✓	✓	✓	✓
Quantus	×	✓	✓	✓	✓

4 Post hoc Interpretability Local Individual Predictions

Local interpretation methods are used to explain individual predictions. Local interpretation is defined as the ability of conditional interaction between dependent and independent variable with respect to a single prediction. Local interpretation could possibly be achieved by approximating the behavior of complex model with the help of single input using as interpretable surrogate model. The detailed taxonomy of individual prediction on post hoc interpretability is given below (Fig. 1).

4.1 LIME

Ribero et al. [9] suggest that Interpretable models like local surrogate models are used to describe particular predictions made by black box machines. Models for learning Local surrogate models are used to enlighten on the specific predictions made by black box machine learning models. Surrogate models are taught to approximate the predictions of the underlying black box model locally as opposed to globally, resulting in Local Interpretable Model-Agnostic Explanations (LIME) [9]. LIME computes the impact on predictions of providing the machine learning model with

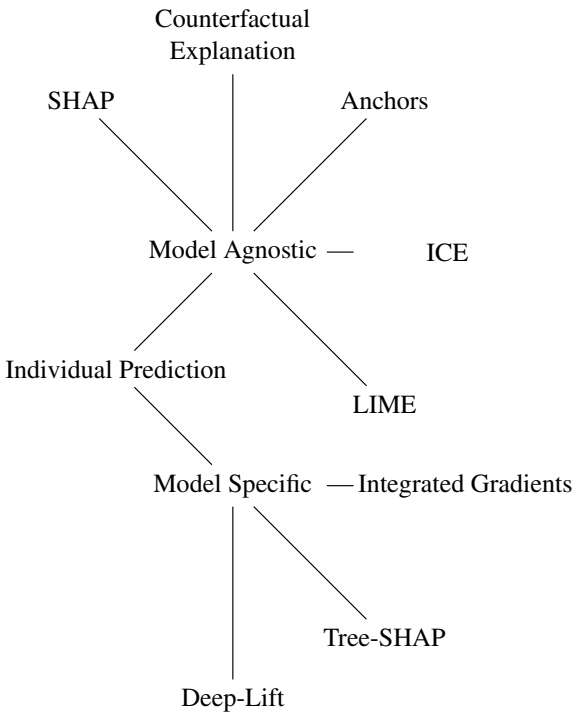


Fig. 1 Taxonomy of the Post hoc interpretability local individual predictions

slightly different input data. To do this, it generates a new dataset comprised of shuffled observations around the observation of interest and the related predictions of the black box model. To explain the black box model at this particular location, LIME utilizes this additional data to train an interpretable model, such as a tree or linear model. It is possible to implement LIME in Python with the help of the lime library, and in R with the help of the lime package and the iml package. These programs may be set up and used with little effort. LIME generates fresh samples for tabular data by randomly selecting from a normal distribution with feature-specific means and variances. Each instance is represented by a row in the table, and each characteristic is represented by a column. In LIME, samples are chosen from the middle of the training data, which may be somewhat troublesome. However, this improves the likelihood that LIME will be able to pick up on at least some reason for why the predictions made by some of the sample points diverge from the data point of interest. LIME explanations are different from feature importance. In LIME for each individual instance that passes to the model and for all the predictions of the model, it performs sensitivity analysis. This is performed to understand how sensitive is the prediction with regards to each feature of this particular instance. The objective function of LIME is given below:

$$\text{LIMEexp}(x) = \arg\min_{g \in G} L(f, g, \pi_x) + \Omega(g) \tag{1}$$

where explanation model of instance x is the model g that minimizes loss L. L measures how close the explanation is to the prediction of the original model f, while the model complexity Ω is kept low. Ω is based on the number of features. For each given instance x, G is the set of all viable explanations, andπ_x determines the size of the surrounding region. The purpose of this work is to use interpretability to learn how a model determines whether or not a certain individual suffers from cardiovascular disease. For that purpose used a heart disease dataset from four databases such as Long Beach V, Switzerland, Hungary, and Cleveland. It has 76 properties, however, only 14 are being used in this study. Patient cardiac disease status is the dependent variable of interest. If the target variable is integer-valued 0, then the patient has no disease, and if it is 1, then the person is suffering from heart disease.

LIME's output is depicted in Fig. 2. The LIME algorithm has helped us understand the numerical characteristics that determine whether the patient has heart disease or not. Positive is represented in orange color, it indicates whether the person is suffering from heart disease, while negative (blue) indicates the patient has no heart disease. It is simple to interpret the weights by applying them to the prediction probabilities. The bars represent the weight for each feature and the corresponding value for the test slice. As shown by the results, the model has made the prediction based on the person's maximum heart rate achieved (thal), chest pain (cp), or number of major vessels (0–3) colored by flouroscopy(ca). Hence, the patient is more likely to have heart disease and the model has made the predictions based on these variables. The

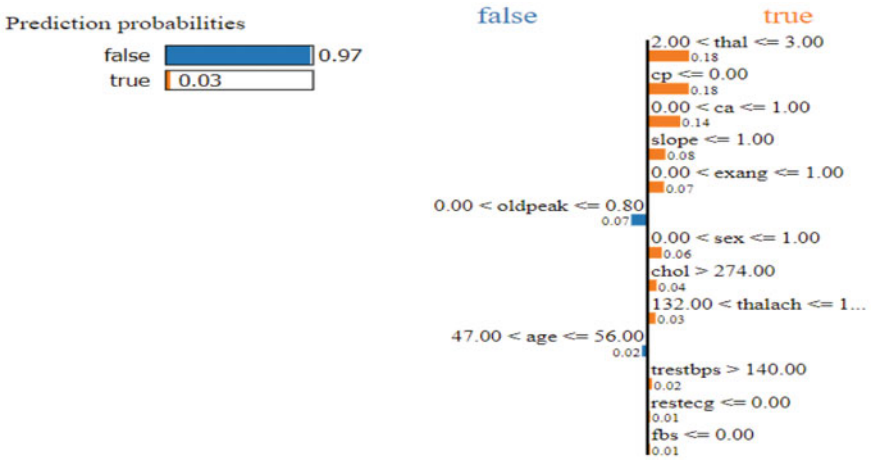


Fig. 2 LIME explanation on heart disease dataset

prediction clearly explains that age is not a factor for having heart diseases. An erroneous forecast (heart disease = 0) has a slightly greater chance than a successful prediction (heart disease = 1).

4.2 Individual Conditional Expectation (ICE)

The Individual Conditional Expectation (ICE) plot displays a single line for each occurrence to demonstrate how a prediction changes as a feature shifts. Partial Dependence Plots (PDP) of the average effect of a characteristic are a global method because they focus on the overall average rather than a specific instance. Goldstein et al. [11] state that the equivalent of a single data instance PDP is called an Individual Conditional Expectation (ICE). The ICE plot provides a separate visualization of the predictive dependencies on the characteristics of each instance, with one row per instance compared to the entire row of the partial dependency plot. Partial dependency charts may be more difficult to interpret than individual conditional expectation curves. The ICE curve, in contrast to the partial dependence diagram, may show irregularities. Both R and Python provide support for ICE charts. The R packages include iml, ICEbox, and pdp.Scikit-learn in Python includes partial dependence plots (Fig. 3).

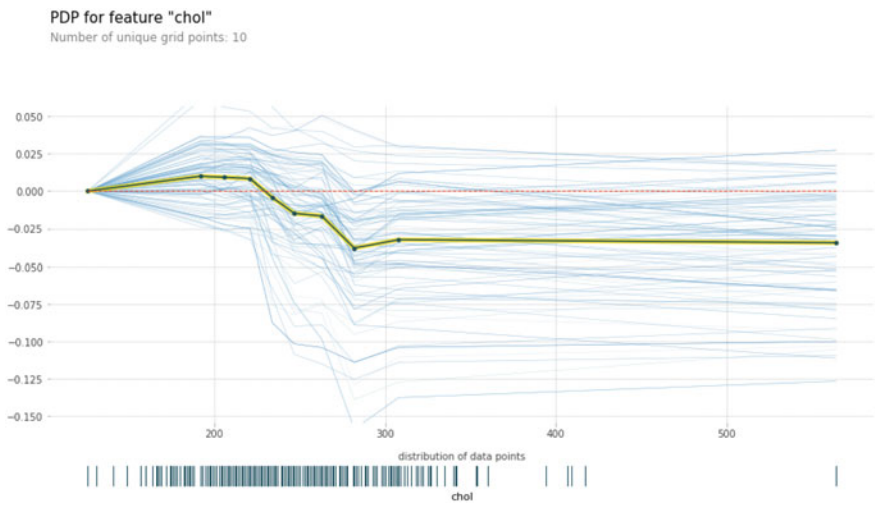


Fig. 3 Combined ICE plot for the "chol" variable in the heart disease dataset

4.3 Counterfactual Explanations

A counterfactual explanation of a prediction details how even a tiny change in the feature values might change the forecast into a desired outcome. Counterfactual explanations are sometimes also called contrastive explanations. There are model-specific counterfactual explanation techniques as well as model-agnostic techniques [18], but this paper emphasizes model-agnostic techniques that just use the model inputs and outputs and not the internal structure of specific models. The counterfactual explanation is human-friendly because it is selective, contrary to the current instance. That is, it usually focuses on a small number of characteristic changes. However, the counterfact suffers from the Rashomon effect.

A counterfactual explanation is an alternative data point that is close to the original but has a different prediction for a particular data point and model. To what extent does this assist in addressing questions regarding the adjustments that need to be made to cross the decision boundary? Each counterfact tells a different explanation about how a particular result was achieved. One counterfact may say that it changes one instance, while the other counterfact may say that it keeps the same and changes another instance. There is an inconsistency here. Reporting all possible counterfactual explanations or using criteria to assess and pick the best counterfactual explanation will help with this multiple-truth dilemma. After the counterfact is applied to the instance's feature values, the forecast is updated to reflect the new feature values. Although it generates a new instance, the counterfactual technique may be summarized by indicating which feature values have changed. The counterfactual approach may be used even if you don't have data or a model at your disposal. The



Fig. 4 SHAP values to explain the predicted heart disease probabilities The predicted probability is equal to 0.88. The red features, like thal and cp increase the probability of having a heart disease, while thalach feature decreases the probability

counterfactual explanation establishes a compromise between explaining the model's predictions and preserving the interests of the model owner (Fig. 4).

4.4 Anchors

Ribeiro et al. [10]'s anchors are the antithesis of counterfactual. By determining a decision rule that may safely anchor a prediction, the anchors technique provides an explanation for the results of any black box classification model. A rule serves to secure a forecast when variations in other feature values have no impact on it. To reduce the amount of model calls while still allowing for escape from local optima, anchors use methods from reinforcement learning in combination with a graph search algorithm. When it comes to explaining the results of black box machine learning models, the anchoring method employs a perturbation-based technique to provide regional justifications for the results. The resultant explanations are written in the form of straightforward IF–THEN rules called anchors, as opposed to LIME's more complex surrogate models. Anchors incorporate the idea of coverage, which describes which further, perhaps unknown, instances the rule applies to. This makes the rules reusable. Compared to LIME, the anchoring method offers a number of benefits. Anchors' principles are intuitive, and the algorithm's output is clear. Formally speaking, an anchor is

$$\mathbb{E}_{\mathcal{D}_x(z|A)}[1_{\hat{f}(x)=\hat{f}(z)}] \geq \tau, A(x) = 1 \tag{2}$$

Here, x stands for the specific case being discussed. The symbol A stands for the anchor or ensuing rule, whereas f represents the underlying categorization scheme.

Also, by including coverage, anchors may be sub-settable and convey a sense of significance. Second, when model predictions around an instance are non-linear or otherwise complicated, the anchoring technique may help. Compared to fitting surrogate models, the strategy is less likely to under-fit the model since it employs reinforcement learning approaches. It involves the fields of pattern recognition and data mining (Table 2).

Table 2 Comparative study of individual post hoc interpretability methods

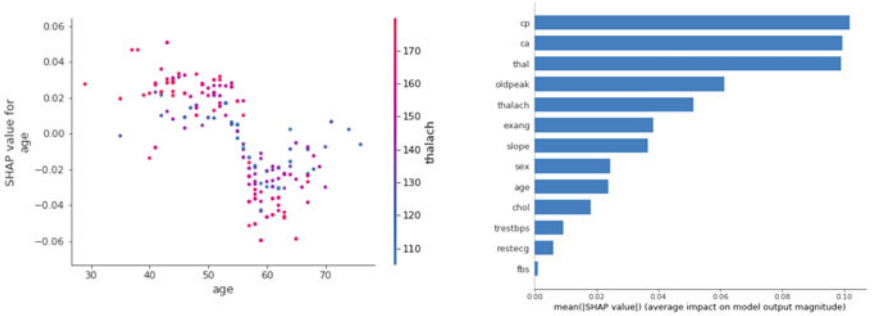
Post hoc interpretability methods	Perturbation based	Explanation types			
		Numerical	Rule-based	Textual	Visual
LIME	✓	✓	×	×	✓
ICE	✓	×	×	×	✓
Counterfactual Explanations	×	×	×	✓	×
Anchors	✓	✓	✓	✓	✓
SHAP	✓	✓	×	×	✓

4.5 SHAP

Lundberg [19] developed a technique called SHapley Additive exPlanations(SHAP) to provide justification for each forecast. SHAP uses the optimum Shapley values according to game theory. Unlike other methods like LIME, it provides the contribution of each feature value while also providing the right size of these contributions, making it a great tool for analyzing individual predictions. The theoretical groundwork for SHAP may be traced back to game theory. All possible feature values get an equal share of the forecast. When we compare the forecast with the typical prediction, we get contrasting interpretations. The value of SHAP links the LIME and Shapley systems. This is a significant aid in comprehending the two approaches. It helps bring the area of interpretable machine learning together.

The purpose of SHAP values is to provide an explanation for a model's output as the cumulative result of the impacts of each feature added through conditional expectation. By assigning each characteristic a value in SHAP, much like a player in a game, you can determine how much each feature contributes to the black box model. Shapley values are most intuitive when comparing the correlation's performance with and without a single participant. In this approach, we may learn what role this player had in the overall success of the team. The Shapley value of a prediction tells us how evenly the various inputs contribute to that prediction. SHAP also gets global interpretation through aggregating over these individual predictions. For tree-based models, SHAP has a quick implementation. The fast computation allows for the computation of the many Shapley values required for global model interpretations. A SHAP dependence plot depicts the relationship between the feature and its impact on the SHAP outcome. The most important variables are listed in descending order on a Feature importance plot. The top variables have a strong predictive potential since they contribute more to the model than that of the bottom ones (Fig. 5).

The summary plot above shows the different SHAP values for high or low feature values. The "cp" feature has low feature value and high SHAP value, which means a higher probability of having a heart disease (Fig. 6).



(a) SHAP Dependency Plot

(b) SHAP Feature Importance

Fig. 5 Explanation of feature importance based on the SHAPley values

Table 3 Accuracy of classification model with SHAP

Classification model	Accuracy-Train	Recall-train	Accuracy-test	Recall-test
Logistic regression	0.86	0.91	0.78	0.87
Decision tree	1.0	1.0	0.98	0.97
Random forest	0.95	0.97	0.87	0.94

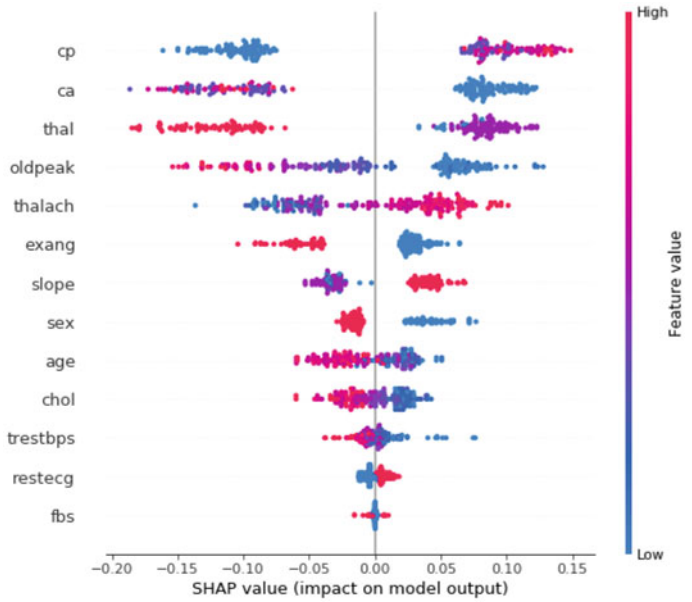


Fig. 6 SHAP summary plot

5 Post hoc Explanation Vulnerabilities

Instability: Methods that don't rely on a particular model to explain data tend to be more unstable than their gradient-based counterparts.

Adversarial attacks: Even though modern artificial intelligence is incredibly powerful, one of the limitations of artificial intelligence is that sometimes it can be fooled. In particular, AI systems are sometimes susceptible to adversarial attacks. An adversary may construct an adversarial classifier that mimics the original classifier's behavior on the input data points by distinguishing between data points from the input distribution and instances created by perturbation. On the other hand unperturbed cases, it acts normally (seems impartial and fair), but, on the disturbed ones, it acts arbitrarily differently, successfully tricking LIME or SHAP into giving harmless explanations [16].

Manipulative: Explanation maps can be replaced with any target map [18]. This is achieved by adding a very little disturbance to the input, one that is just barely perceptible. This perturbation has no effect on the neural networks output. This finding may impact negatively if a doctor, wants to rely on interpretable explanation map in the clinical decision-making process.

Unjustified Counterfactual Explanations: Based on ground-truth labeled data, Laugel et al. [19] propose an interactive foundational element for more relevant counterfactual explanations, which aids in the generation of better explanations. They develop a test to highlight the risk of undesirable counterfactual examples trying to interfere with the generation of counterfactual explanations, and they adapt it to a wide range of datasets and classifiers, proving that the threat of producing inappropriate counterfactual examples is high.

Faithfulness: The explanations given by explainable machine learning techniques do not accurately reflect the results of the original models computations [20]. Sometimes Explanations may be false. They are unable to accurately duplicate the original model. If the explanation accurately described what the original model computed, it would be identical to the original model, removing the need for the initial model in the first place. A flawed explanation model reduces faith in the explanation.

6 Future Scope

Further study of the interpretability of the model to gain insight into the faithfulness of the model will improve the predictive model for high-risk decisions. Current analysis tools do not provide reliable indicators of whether a model is correct or not. A study evaluating whether the predictions of deep neural networks can be interpreted by humans. In addition, future research should focus on exposing the flaws of currently accepted post hoc explanations and offering solutions to these problems. To move the discipline forward, we need a better understanding of how

various post hoc interpretation techniques might be used in various situations. In addition to traditional classification techniques, examining post hoc interpretation of complex machine learning tasks can be a good starting point for researchers.

7 Conclusion

The development of AI is opening up new possibilities for enhancing people's lives all across the world, in fields like business, healthcare, and education. It is also bringing up novel concerns regarding the most effective ways to include fairness, accountability, and transparency in these systems. This paper provided a basic overview of machine learning interpretability, as well as a summary of the taxonomy for interpretability. Also, it made a point about the significance and need of Interpretability in various sectors of society and discuss various methods of post hoc interpretability on tabular data. The detailed comparative study of advanced Interpretability AI Toolkits was discussed in the paper along with the vulnerabilities of post hoc interpretability methods.

References

1. Murdoch WJ, Singh C, Kumbier K, Abbasi-Asl R, Yu B (2019) Definitions, methods, and applications in interpretable machine learning. Proc Natl Acad Sci 116(44):22071–22080
2. Lipton ZC (2018) The mythos of model interpretability: in machine learning, the concept of interpretability is both important and slippery. Queue 16(3):31–57
3. Jesus S, Belém C, Balayan V, Bento J, Saleiro P, Bizarro P, Gama J (2021) How can I choose an explainer? an application-grounded evaluation of post-hoc explanations. In: Proceedings of the 2021 ACM conference on fairness, accountability, and transparency. pp 805–815
4. Guidotti R, Monreale A, Ruggieri S, Turini F, Giannotti F, Pedreschi D (2018) A survey of methods for explaining black box models. ACM Comput Surv (CSUR) 51(5):1–42
5. Doshi-Velez F, Kim B (2017) Towards a rigorous science of interpretable machine learning. arXiv preprint arXiv:1702.08608
6. Arrieta AB, Díaz-Rodríguez N, Del Ser J, Bennetot A, Tabik S, Barbado A, García S et al (2020) Explainable artificial intelligence (XAI): concepts, taxonomies, opportunities and challenges toward responsible AI. Inf Fusion 58:82–115
7. De T, Giri P, Mevawala A, Nemani R, Deo A (2020) Explainable AI: a hybrid approach to generate human-interpretable explanation for deep learning prediction. Procedia Comput Sci 168:40–48
8. Kasza J, Wolfe R (2014) Interpretation of commonly used statistical regression models. Respirol 19(1):14–21
9. Mood C (2010) Logistic regression: why we cannot do what we think we can do, and what we can do about it. Eur Sociol Rev 26(1):67–82
10. Lee H, Kim S (2016) Black-box classifier interpretation using decision tree and fuzzy logic-based classifier implementation. Int J Fuzzy Log Intell Syst 16(1):27–35
11. Seni G, Elder JF (2010) Rule ensembles and interpretation statistics. In: Ensemble methods in data mining. Springer, Cham, pp 88–107
12. Ting SL, Ip WH, Albert HC Tsang (2011) Is naive bayes a good classifier for document classification. Int J Softw Eng Its Appl 5(3):37–46

13. Wallace E, Feng S, Boyd-Graber J (2018) Interpreting neural networks with nearest neighbors. arXiv preprint arXiv:1809.02847
14. Ribeiro MT, Singh S, Guestrin C (2016) "Why should i trust you?" explaining the predictions of any classifier. In: Proceedings of the 22nd ACM SIGKDD international conference on knowledge discovery and data mining. pp 1135–1144
15. Ribeiro MT, Singh S, Guestrin C (2018) Anchors: high-precision model-agnostic explanations. In: Proceedings of the AAAI conference on artificial intelligence, vol 32, no 1
16. Goldstein A, Kapelner A, Bleich J, Pitkin E (2015) Peeking inside the black box: visualizing statistical learning with plots of individual conditional expectation. J Comput Graph Stat 24(1):44–65
17. Wachter Sandra, Mittelstadt Brent, Russell Chris (2017) Counterfactual explanations without opening the black box: automated decisions and the GDPR. Harv. JL & Tech. 31:841
18. Karimi AH, Barthe G, Balle B, Valera I (2020) Model-agnostic counterfactual explanations for consequential decisions. In: International conference on artificial intelligence and statistics. PMLR, pp 895–905
19. Lundberg SM, Lee SI (2017) A unified approach to interpreting model predictions. In: Advances in neural information processing systems 30
20. Arya V, Bellamy RKE, Chen PY, Dhurandhar A, Hind M, Hoffman SC, Stephanie Houde et al (2020) AI explainability 360: an extensible toolkit for understanding data and machine learning models. J Mach Learn Res 21(130):1–6
21. Baniecki H, Kretowicz W, Piatyszek P, Wisniewski J, Biecek P (2021) dalex: responsible machine learning with interactive explainability and fairness in python. J Mach Learn Res 22(1):9759–9765
22. Kaur H, Nori H, Jenkins S, Caruana R, Wallach H, Wortman Vaughan J (2020) Interpreting interpretability: understanding data scientists' use of interpretability tools for machine learning. In: Proceedings of the 2020 CHI conference on human factors in computing systems. pp 1–14
23. Nori H, Jenkins S, Koch P, Caruana R (2019) Interpretml: a unified framework for machine learning interpretability. arXiv preprint arXiv:1909.09223
24. Li Y, Tian S, Huang Y, Dong W (2021) Driverless artificial intelligence framework for the identification of malignant pleural effusion. Transl Oncol 14(1):100896
25. Hardt M, Chen X, Cheng X, Donini M, Gelman J, Gollaprolu S, He J, Larroy P, Liu X, McCarthy N, Rathi A, Rees S, Siva A, Tsai E, Vasist K, Yilmaz P, Zafar MB, Das S, Haas K, Hill T, Kenthapadi K (2021) Amazon sagemaker clarify: machine learning bias detection and explainability in the cloud. In: Proceedings of the 27th ACM SIGKDD conference on knowledge discovery & data mining (KDD '21). Association for Computing Machinery, New York, NY, USA, pp 2974–2983
26. Hedström A, Weber L, Bareeva D, Motzkus F, Samek W, Lapuschkin S, Höhne MMC (2022) Quantus: an explainable AI toolkit for responsible evaluation of neural network explanations. arXiv preprint arXiv:2202.06861
27. Dieber J, Kirrane S (2020) Why model why? assessing the strengths and limitations of LIME. arXiv preprint arXiv:2012.00093
28. Garreau D, Luxburg U (2020) Explaining the explainer: a first theoretical analysis of LIME. In: International conference on artificial intelligence and statistics. PMLR, pp 1287–1296
29. Främling K, Westberg M, Jullum M, Madhikermi M, Malhi A (2021) Comparison of contextual importance and utility with lime and Shapley values. In: International workshop on explainable, transparent autonomous agents and multi-agent systems. Springer, Cham, pp 39–54
30. Slack D, Hilgard S, Jia E, Singh S, Lakkaraju H (2020) Fooling lime and shap: adversarial attacks on post hoc explanation methods. In: In Proceedings of the AAAI/ACM conference on AI, ethics, and society. pp 180–186
31. Rudin Cynthia, Chen Chaofan, Chen Zhi, Huang Haiyang, Semenova Lesia, Zhong Chudi (2022) Interpretable machine learning: fundamental principles and 10 grand challenges. Stat Surv 16:1–85
32. Dombrowski AK, Alber M, Anders C, Ackermann M, Müller KR, Kessel P (2019) Explanations can be manipulated and geometry is to blame. In: Advances in neural information processing systems 32

33. Laugel T, Lesot M-J, Marsala C, Renard X, Detyniecki M (2019) The dangers of post-hoc interpretability: unjustified counterfactual explanations. arXiv preprint arXiv:1907.09294
34. Rudin C (2019) Stop explaining black box machine learning models for high stakes decisions and use interpretable models instead. Nat Mach Intell 1(5):206–215

SRGAN with 3D CNN Model for Video Stabilization



Sunil Kumar Kumawat and Mantosh Biswas

Abstract The Video Stabilization (VS) has been an energetic place of study for many years. Video stabilization is a technique that improves the video's quality, eliminating or reducing undesired and unwanted camera motions. Non-professionals taking videos will have unintended consequences, like visual blurring, image distortion, etc. Therefore, many researchers study these shortcomings to raise the caliber of video. In this paper, a model for smoothing down jerky videos is put forward. Applying a 3D convolution to both the spatial and temporal domains, we proposed a model 3D convolutional generative adversarial networks GAN with Gaussian Mixture, we reduce spatiotemporal data for unstable background subtraction. Our model can simultaneously collect the spatiotemporal information that is stored in nearby frames, directly enhancing the performance of video stabilization. The input video is recognized and processed before being optimized and stabilized using a generator in the GAN (generative adversarial network) architecture, which also employs a content loss in addition to an adversarial loss for effective adversarial training. The Gaussian distribution model is revised for dynamic changes to determine the background process. In the foreground, detection removes moving substances from frames. Background and foreground are fundamental problems of video study. Our method produces both visually and quantitatively improved results, for KITTI data set was used to compare peak signal-to-noise ratio (PSNR) and structural index similarity (SSIM). This model produced stable output video and successfully eliminated distortion from videos recorded under various conditions.

Keywords Convolutional neural networks · Spatio-temporal learning · Adversarial learning · Video stabilization

S. K. Kumawat (✉) · M. Biswas
National Institute of Technology, Kurukshetra, India
e-mail: kumawatsunil9494@gmail.com

M. Biswas
e-mail: mantoshbiswas@nitkkr.ac.in

© The Author(s), under exclusive license to Springer Nature Singapore Pte Ltd. 2023

S. Chinara et al. (eds.), *Advances in Distributed Computing and Machine Learning*,
Lecture Notes in Networks and Systems 660,
https://doi.org/10.1007/978-981-99-1203-2_24

1 Introduction

Videos were taken with handheld cameras sometimes have undesired blurs brought on by camera shake [1] or moving objects in the background [2, 3]. Video stabilization attempts to restore crisp frames from the input video by erasing such unwanted blurring. In the practical areas of computer vision and image processing, this is an active research area. Numerous significant disciplines, including 3D reconstruction [4], Simultaneous localization and mapping [5], and tracking [6], make use of video stabilization. Contrary to single picture deblurring, video stabilization has recently received less attention than it once did. Additionally, video stabilization is more difficult, in part because it is unclear how to represent and make use of the intrinsic temporal dynamics present among continuous video frames. The typically used performance metric, pixel-wise residual error, which is frequently assessed by PSNR, is dubious since it does not account for visual perceptions of how realistic or crisp a restored picture is [7, 8]. To enhance the performance of video stabilization, we want to use the adversarial learning technique's most recent advancements in this paper. Finding a practical method to effectively collect the Spatio-temporal information present in adjacent image frames is a significant difficulty for video stabilization. Denoising and deblurring of images and videos have lately benefited greatly from the use of deep learning-based approaches. However, previous deep learning techniques have mostly relied on 2D convolutions, primarily for computational reasons. However, it is not natural to use 2D convolutions to capture spatial and temporal joint information, which is essential in a 3-D feature space. In this paper, we propose a deep neural network-based model called SRGAN With 3D CNN model, which employs deep residual learning and 3D (volumetric) convolutional layers, it is possible to learn feature representations that span both pictures planes and temporal frames. The usual pixel-wise PSNR and SSIM measure, as mentioned above, is thought to be adequate for image/video stabilization. In this case, the generative network is the aforementioned SRGAN, which recovers crisp pictures and can distinguish between restored and authentically sharp images. We introduce the phrases "content loss" and "adversarial loss," which make up a training loss together. The adversarial loss encourages a more realistic-looking (and hence sharper) picture, while the content loss is employed to respect the pixel-wise measurement. SRGAN training with 3D CNN We extract crisp video frames from an unstable input video sequence in an end-to-end way. The following are the contributions of this work: We propose a model called SRGAN with 3D CNN, which uses 3D convolutions in a deep residual network to collect joint Spatio-temporal data. We create a generative adversarial network containing adversarial losses as well as the content. It learns adversarial and restores video frames that appear more realistic. Studies using the Blurred KITTI dataset, a common benchmark dataset, demonstrate the effectiveness and superior performance of the proposed network SRGAN with 3D CNN.

2 Related Work

Many approaches have been proposed for image/video stabilization. The conventional, physically-based video stabilization techniques and the more contemporary, deep learning-based techniques are both summarized in this section. The majority of currently used 2D physically-based approaches for video stabilization. All of the techniques listed below use 2D feature tracking. The approach for feature track smoothing and steady frame creation mostly accounts for the discrepancy. Using the non-metric IBR technique, Christopher Buehler et al. [9] re-render the frames at smoothed camera locations. The original frames are warped using straightforward 2D full-frame transformations by Michael Let al. and [10] Yasuyuki Matsushita et al. [11]. The frames are warped by Shuaicheng Liu et al. [12] using a grid, and the surrounding feature tracks are smoothed. For smoothing the feature tracks, Matthias Grundmann et al. [13] L1 optimum camera path was suggested. Eigen-trajectories are extracted and smoothed by Feng Liu et al. [14]. The feature track smoothing is constrained by Goldstein and Fattal's[15] use of epipolar geometry. Miao Wang et al. [16] apply just 2D restrictions while maintaining the relative location of feature points. In addition to these 2D physically-based techniques, Sen-Zhe Xu et al. [17] method reconstruct the 3D locations of the feature points and cameras before smoothing the camera trajectory and reprojecting the feature points to new camera positions. Brandon M. Smith et al. [18] and Jian Sun [19] also employ 3D information, although they need light field cameras and depth cameras, respectively. Later works employ optical flow to pixel-level smooth motion. Iterative Jacobi-based optimization is used by SteadyFlow [20] to smooth the motion vector changes on each pixel. The optical flow is used by Jiyang Yu [21] to track the mobility of the pixels. Instead of explicitly solving for the warp field, they optimize the neural network weights that produce it. For every new video, their optimization must be carried out again. Deep learning is being used recently to stabilize videos. The adversarial network is used by Sen-Zhe Xu et al. [17] to produce a target picture that directs the frame warping. A two-branch Siamese network is used by Miao Wang et al. [16] to build a grid that warps the video frames. These networks accept color frames as input and are trained using the stable and unstable video pairings included in the DeepStab dataset [19]. Video stabilization is now possible in almost real-time thanks to deep learning techniques. These works' visual outcomes fall short of those produced by more conventional techniques. There are two possible explanations for deep learning's subpar video stabilization performance. The first issue is the spatial transformation of the video stabilization. Rich texture information can be found in the color photos, but the inter-frame spatial relationship is still ambiguous. ResNet50 is used directly by Miao Wang et al. [16] without taking into account spatial modification. The adversarial network's spatial transformer modules were introduced by Sen-Zhe Xu et al. [17], however, it is challenging to train a single network to infer the spatial transformation of numerous frames just from color frames. Second, there is not enough data in the training dataset.

3 Proposed Model

The proposed method has six stages. The first stage is input video, the second stage is extracting current frames from the video, the third stage is the video stabilization model, the fourth stage is the updated background model, the fifth stage is related to background and foreground frames, and the final stage is the video output.

3.1 Video Input

Real-time video is given as input to the model. The input video contains slow-motion changes.

3.2 Extract Current Frames

In this step, ground truth video and corresponding blurred videos are given as input for training the model. Current frames are extracted using the OpenCV library inbuilt method 'VideoCapture()' from input video and passed to the video stabilization model. This step generates frames for the ground truth videos and corresponding blurred videos.

3.3 Video Stabilization Model

In this step, randomized cropped frames in a batch size of 5 are given as input to train the model. The objective of video stabilization is to remove undesired blurs and recover good quality video frames from the input blurred video frames. Super Image resolution models are capable of capturing the perceptual detail in images with 2D convolution. We can extend the same logic for video frames by using 3D convolution as it is capable of capturing spatial information along with time dimension. We proposed a modified model with ResNet by replacing 2D convolution with convolutions in a 3D manner. The network is capable of exploiting the 3D spatial information and can retain extracted feature predictions which result in stabilized improved videos. Super-resolution generative adversarial network SRGAN [22] has to train generative models. The output of one convolutional layer connected to the input of another convolutional layer many layers later is called a residual link as shown in Fig. 3. We construct a CNN model as our adversarial training model, using the generator network model. It contains 64 convolutional layers (Figs. 1 and 2).

These layers enhance the accuracy and performance of results. The basic objective is to gradually learn more complicated characteristics by adding additional layers.

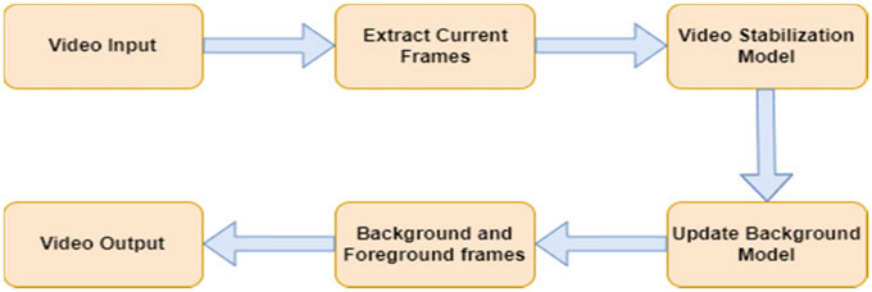


Fig. 1 Proposed model

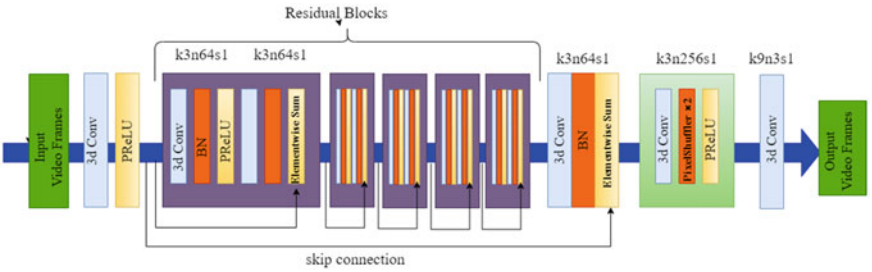


Fig. 2 The modified architecture of generator SRGAN

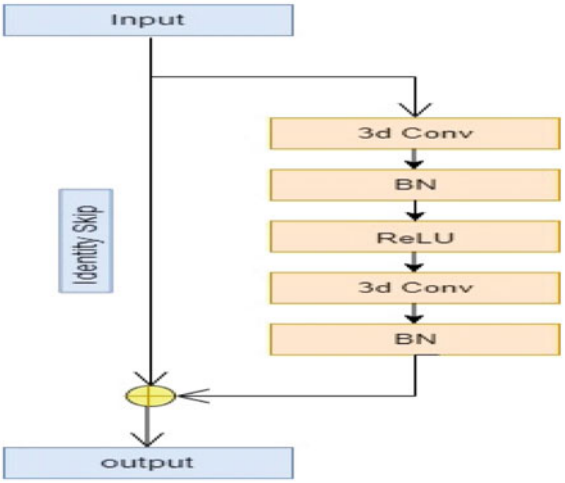


Fig. 3 Residual block with 3D CNN

Learning to recognize edges makes up the first tier. Learning to recognize similarities is the second layer. Learning to detect objects is the third layer. The most complex problem is solved in this manner. We replace 2D-CNN with 3D-CNN to remove temporal changes, removes spatial and temporal features. The existence of skip layers in a neural network, as seen in the connection in Fig. 3, characterizes a residual block [23]. Each layer in the ResNet is trained concurrently, and the input to each layer depends on the one before it.

Two 3D convolutional layers with the same number of output channels make up the residual block. Video to reflect the motion deviation in the temporal dimension in various frames. For deep residual networks, the convolution part shows spatial and temporal dimensions for deblurring and video stabilization. By convolving 3D kernels and filters using a matrix created from many nearby frames, we carry out the 3D convolution. Through feature maps in the convolution layers, the dynamic fluctuations are captured. These variants can be used to further recover crisp frames and models for video steady development. Model is trained by minimizing the loss function. The trained model takes real-time video frames as input and generates the stabilized video frames which are converted into the video.

3.4 Update Background Model

We use the Gaussian mixture model for background subtraction. The input of the Gaussian mixture model uses the output from the video stabilization model. In our model, every pixel in a frame is considered a distribution. Each pixel value in video frames is considered as a mixture of Gaussian distribution (K-distributions) shown in Fig. 4. The Gaussian mixture model studies the values of a specific pixel in the pixel process. This pixel process is considered a sequence of pixels. We consider scalars and vectors to be gray values and color images. The mean and variance of Gaussian distribution depend on each pixel value of consecutive video frames. If the pixel value at a particular location in a frame is not changing with time can considered a background pixel as it has low variance. An initially low prior weight and high variance are used if the current pixel value does not fit within 2.5 standard deviations of distribution and the current pixel value is replaced with the mean of the distribution. The background contains non-moving objects and the foreground contains moving objects which can be used to track objects. This step generates the video frames in which static pixel values are replaced by the mean value of the distribution.

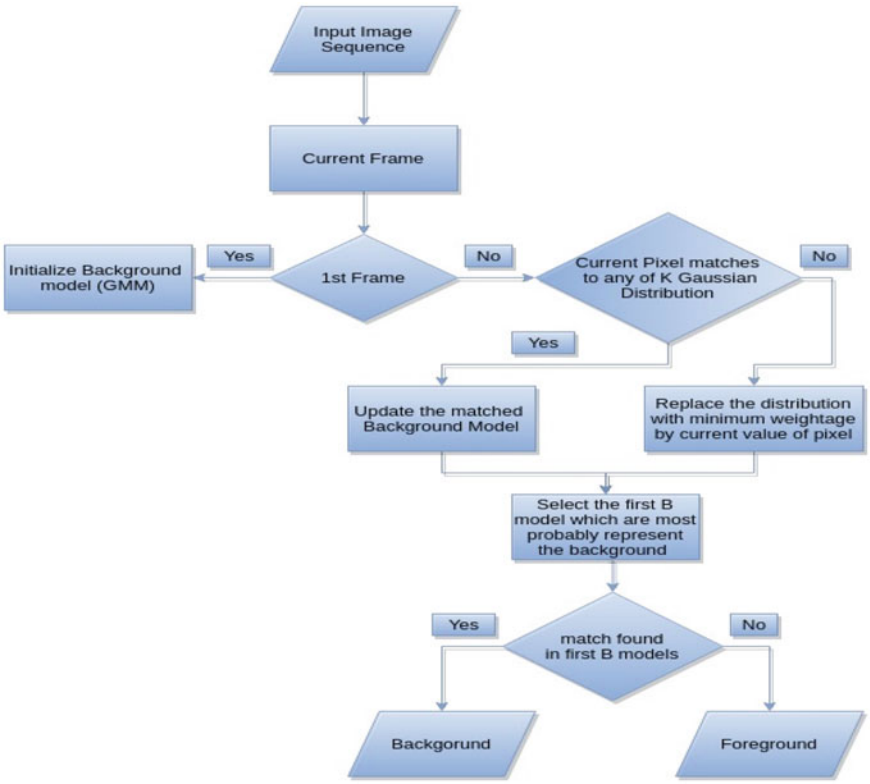


Fig. 4 Flow chart for background and foreground frames

3.5 Background and Foreground Frames

In this step, updated video frames generated by the previous step are taken as input. For real-time tracking, only moving objects are required in video frames. This is achieved by separating background and foreground frames. The Gaussian mixer model produces background frames as output shown in Fig. 4. The foreground is achieved from frames masking the background. By using noise removal filters (like the Kalman filter), we improve the frame quality. This step generates the background mask frames as output.

3.6 Video Output

We convert background and foreground frames into output video using OpenCV inbuilt methods. Two separate videos are generated for foreground and background.



Fig. 5 Input video frame **a** Playground, **b** Car, **c** Music, and **d** Truck

Real-time input videos usually are not stable in nature as they contain blur, noise, and slow-moving objects. Video stabilization objective is achieved by 3D convolution modeling which preserves perceptual information and at the same time removes blur and noise. Slow-moving objects and lighting changes are removed by GMM modeling. Finally, we get stabilized videos with foreground and background separated which can be further used for object tracking more accurately as the output video does not contain slow-moving objects and noise is reduced.

4 Experiment Result

In this chapter, we discuss the dataset and training details that are used in our model. Also, analysis of our result in quantitate and image form, result compare with existing methods. We used SRGAN with 3D CNN for video stabilization. Our purpose is to

Table 1 Result of PSNR value for video stabilization by Choi et al. [25], Mildenhall et al.[26], Mildenhall et al. [27], Aliev et al. [28], Riegler et al. [29], Liu et al. [30], and Proposed

S. No	Method	Truck	Train	Music	Playground
1	Choi et al. [25]	14.99	11.81	9.66	16.34
2	Mildenhall et al. [26]	10.66	8.88	8.98	13.27
3	Mildenhall et al. [27]	19.47	13.16	15.99	21.16
4	Aliev et al.[28]	20.32	18.08	12.35	23.03
5	Riegler et al. [29]	22.21	17.46	17.13	22.32
6	Liu et al. [30]	23.25	18.76	17.43	23.20
7	Proposed	18.91	16.26	15.71	20.49

Table 2 Result of SSIM value for video stabilization by Choi et al. [25], Mildenhall et al. [26], Mildenhall et al.[27], Aliev et al. [28], Riegler et al. [29], Liu et al. [30], and Proposed

S. No	Method	Truck	Train	Music	Playground
1	Choi al. [25]	0.563	0.454	0.473	0.610
2	Mildenhall et al. [26]	0.432	0.356	0.427	0.517
3	Mildenhall et al. [27]	0.690	0.532	0.691	0.734
4	Aliev et al. [28]	0.822	0.801	0.716	0.876
5	Riegler et al. [29]	0.875	0.759	0.785	0.849
6	Liu et al. [30]	0.882	0.778	0.788	0.854
7	Proposed	0.593	0.50	0.52	0.62

optimize the video quality by removing temporary changes from input frames shown in Fig. 4 and the ground truth of the video is shown in Fig. 6. We use, the loss function computed using adversarial loss and content loss. We calculate the mean square error (MSE) during training shown in Fig. 8. To compare video stability shown in Table 1, we use the peak- signal-to-noise ratio (PSNR). PSRN is the measurement between the maximum possible value (power) of a video frame and the power of distorting noise that affects the quality of video frames. The structural similarity index measure (SSIM) is used for measuring the similarity between two images shown in Table 2. We use a system with 4 GB RAM and colab pro (16 GB RAM) GPU for training purposes.

4.1 Dataset

KITTI dataset was developed by Geiger et al. using their self-directed stage [24]. The KITTI dataset has many subsections for different types of video like audio equivalent identical, optical flow approximation, tracking, and 3D object detection. In this dataset, double datasets consider one is quantitative and the other qualitative.



Fig. 6 Stabilized ground truth frame **a** Playground, **b** Car, **c** Music, and **d** Truck

We train the network using the KITTI dataset. In this training dataset 54 unstable and stable video pairs. Dataset video clips are 25 s or maybe longer. The dataset video is divided into frames then it has more than 10,000 training models. We use **SRGAN** for the training of the model on the dataset.

4.2 Parameter Setting

3D-CNN SRGAN is the use of two $3 \times 3 \times 3$ convolutional layers and several residual blocks. The spatial size of the feature maps in our model remains constant, and this architecture was inspired by the SRGAN with 2D Convolutional Neural Network. In five successive frames, we employ input. Used 3D convolutions with a kernel size of $3 \times 3 \times 3$, here $3 \times 3 =$ spatial size, and the temporal dimension for the last 3 is used. To reduce the temporal dimensions use the size of convolution kernels is $3 \times 3 \times 1$,

we employ larger convolution kernels. Layer stride and padding are set to one. We use a standard deviation of 0.15 and 0.5 mean at the time of training to initialize weights. In each iteration, update weights following the learning of a mini-batch of size 5. We crop a 96 × 96 patch at any position of an image (1280 × 720). By cropping the image we increase the number of samples. They trained at 0.5×10^{-4} a learning rate. The content loss determines the learning rate. If training loss does not enhance performance, we lower the learning rate 10^{-5}. The no of residual block in SRGAN can be set to 12. The MSE loss is optimized using the Adam optimizer. We used 50 epochs to optimize the model. We used cuda to train the model, due to time constraints the model is trained only for 50 epochs. The MSE loss can be further minimized if we train the model for 100 or more epochs. The number of distributions is used from 3 to 5. All new pixel values and existing K Gaussian distributions are matched with X_t, K considers up to a match is established. Standard deviations of a distribution consider a pixel value within 2.5. The value 2.5 is effective for per pixel distribution. The threshold value is very convenient for different areas. If the current pixel is matched with any K distribution. Distribution is substituted by the least probable distribution. This distribution takes the recent data as a low prior value also the mean value. Weight is considered at the starting high variance. To take weights approximation then weights are renormalized. For unmatched distributions μ and σ parameters remain the same. If distribution parameters are matched, then new values are informed. For background Model Estimation parameter initialization, we used the below mentioned parameter values for GMM model training and background subtraction, k = no of distribution in GMM = 4, T = the minimum part of frames that reflected as background = 0.95, $1/\alpha$ = learning rate, $\alpha = 0.05$, Initial value of mean–variance $(\sigma^2) = 100(highvalue)$. For each pixel, the Gaussians mixture model parameters have changed. In GMM there are a few parameters that need to be tuned, learning rate ($1/\alpha$) and threshold value T (How much portion of the frame belongs to the background). We observed that more than 90% portion of the frame belongs to the background and after a few seconds of video, the model starts to learn quickly.

4.3 Discussion

In this paper, we conduct experiments to demonstrate the effectiveness of the proposed model on the task of video stabilization. We can observe from the above table 1 and table 2 our approach performs much better than Choi et al., Mildenhall et al. methods. In Scase of better performance, we can complete the calculation in the early stage by increasing the layers. Setup cost and time consumption are lower compared to other methods Mildenhall et al., Aliev et al., Riegler et al., Liu et al., but we increase the number of layers in our model then take more time for training, becomes more suitable for video stabilization but our approach able to find the solution for video stabilization.

The input video frames are shown in Fig. 6, we take input unstable video into different categories, (a) Playground, (b) Car, (c) Music, and (d) Truck. Blurred and unstable input video frames of different videos are used to train the video stabilization model. The ground truth of the video is shown in Fig. 7, (a) Playground, (b) Car, (c) Music, and (d) Truck. In Fig. 8, the input video frames of GMM (a), and the output is shown in the image (b) foreground image and (c) background image. Shows moving objects are removed from the input video frames at the same time slow-moving objects like trees show motion is retained.

Loss Functions. Our goal is to get the stabilized video frames from a blurred video. In GAN-based models, the loss function is generally calculated using content loss and adversarial loss. We used loss functions during training, two-loss functions are Content Loss and Adversarial Loss.

In Fig. 8 we show the Training Loss and MSE loss of the 100 epoch's graph. In the graph, we take MSE loss on Y-axis and epochs on X-axis. If we increase the number of epochs then reduce MSE loss. If the learning rate is decreased model takes more time for training. During the training time, we balance the different loss functions, using the weight fusion fashion are combined in a loss function. The content and

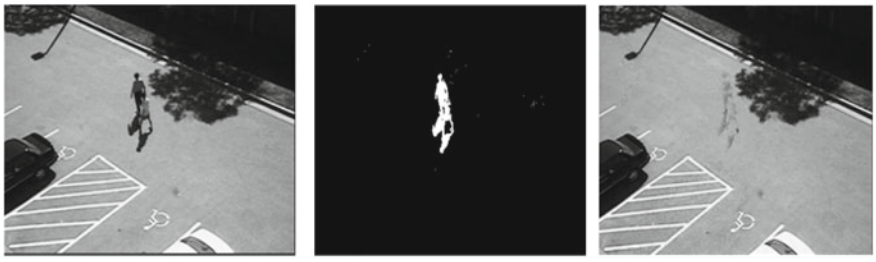


Fig. 7 **a** Input video frame, **b** foreground output image, and **c** background output image

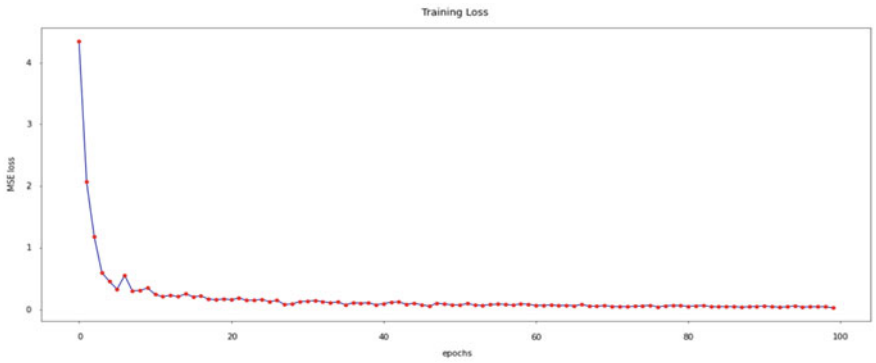


Fig. 8 MSE loss graph of 100 epochs of training loss

adversarial losses for balancing to custom a hyper-parameter to reduce the finishing loss. In training explore dissimilar standards α from 0 to 0.1. At $\alpha = 0$, working content loss. In this situation, 3D CNN SRGAN degrades to ResNet. When the value α increases, adversarial loss becomes more significant. The peak signal-to-noise ratio (PSNR) captures the perceptually related details in videos, such as corners, high texture detail, sharp points, and edges. Though minimizing pixel-wise Mean Square Error (MSE) maximizes the PSNR as it is calculated based on pixel-wise differences between the restored video frames and the ground truth video frames. We get a better PSNR value using fine-tuning our model. The 3D CNN- SRGAN has a high Peak signal-to-noise ratio value SRGAN use at the start. The functions combine with weight fusion fashion.

5 Conclusion

In this paper, we demonstrated that blurring in real-time video frames can be removed with video stabilization models. We proposed a modified video stabilization model with ResNet by replacing 2D convolution with 3D convolution. We use adversarial training and spatiotemporal learning to recover clear and accurate video frames for video stabilization. We specifically suggested new network models based on deep residual neural networks and 3D convolutional. We showed that SRGAN with 3D CNN can better capture Spatio-temporal characteristics, resulting in superior blur reduction. Our model includes both adversarial loss and content loss, which are complementary to one another and enable the model to produce aesthetically realistic video. The experimental outperforms current state-of-the-art video stabilizing techniques. The model is capable of exploiting spatial information and can retain extracted features which result in stabilized improved videos.

References

1. Kang, Sing Bing, “Automatic removal of chromatic aberration from a single image”, IEEE Conference on Computer Vision and Pattern Recognition, pp.1–8, 2007.
2. Sun, Jian, Wenfei Cao, Zongben Xu, and Jean Ponce, “Learning a convolutional neural network for non-uniform motion blur removal”, IEEE Conference on Computer Vision and Pattern Recognition, pp. 769–777, 2015.
3. Shi, Jianping, Li Xu, and Jiaya Jia, “Just noticeable defocus blur detection and estimation”, IEEE Conference on Computer Vision and Pattern Recognition, pp. 657–665, 2015.
4. Seok Lee, Hee, and Kuoung Mu Lee, “Dense 3d reconstruction from severely blurred images using a single moving camera”, IEEE Conference on Computer Vision and Pattern Recognition, pp. 273–280, 2013.

5. Lee, Hee Seok, Junghyun Kwon, and Kyoung Mu Lee, "Simultaneous localization, mapping and deblurring", IEEE International Conference on Computer Vision, pp. 1203–1210. 2011.
6. Jin, Hailin, Paolo Favaro, and Roberto Cipolla, "Visual tracking in the presence of motion blur", The IEEE Conference on Computer Vision and Pattern Recognition, pp. 18–25, 2005.
7. Wang Z, Simoncelli EP, Bovik AC (2003) Multiscale structural similarity for image quality assessment. Asilomar Conference on Signals, Systems and Computers 2:1398–1402
8. Wang Z, Bovik AC, Sheikh HR, Simoncelli EP (2004) Image quality assessment: from error visibility to structural similarity. IEEE Trans Image Process 13(4):600–612
9. Christopher Buehler, Michael Bosse, and Leonard McMillan, "Non-metric image-based rendering for video stabilization", IEEE CVPR, Vol. 2, pp. II-II, 2001.
10. Michael L. Gleicher and Feng Liu., "Re-cinematography: Improving the camerawork of casual video", ACM Trans. Multimedia Comput. Commun. Appl., vol. 5, no. 1, pp. 1–28, 2008.
11. Yasuyuki Matsushita, EyalOfek, Weina Ge, Xiaoou Tang, and Heung-Yeung Shum, "Full-frame video stabilization with motion inpainting", *IEEE Transactions* on *Pattern Analysis* and *Machine Intelligence,* vol. 28, no. 1, pp. 7, pp. 1150–1163, 2006.
12. Shuaicheng Liu, Lu Yuan, Ping Tan, and Jian Sun, "Bundled camera paths for video stabilization", ACM
13. Trans. Graph., vol. 32, no. 4, pp. 1–10, 2013.
14. Matthias Grundmann, VivekKwatra, and Irfan Essa, "Autodirected video stabilization with robust l1 optimal camera paths", IEEE CVPR, pp. 225–232, 2011.
15. Liu F, Gleicher M (2009) HailinJin, and AseemAgarwala, "Content-preserving warps for 3D video stabilization." ACM Trans Graph. 28(3):1–9
16. Amit Goldstein and RaananFattal (2012) Video stabilization using epipolar geometry. ACM Trans. Graph. 31(5):1–10
17. Miao Wang, Guo-Ye Yang, Jin-Kun Lin, Song-Hai Zhang, Ariel Shamir, Shao-Ping Lu, and Shi-Min Hu., "Deep online video stabilization with multi-grid warping transformation learning", IEEE Transactions on Image Processing, vol. 28, no. 5, pp.2283–2292, 2018.
18. Sen-Zhe Xu, Jun Hu, Miao Wang, Tai-Jiang Mu, and ShiMin Hu., "Deep video stabilization using adversarial networks", Computer Graphics Forum, vol. 37, no. 7, pp. 267–276, 2018.
19. Brandon M. Smith, Li Zhang, HailinJin, and Aseem Agarwala, "Light field video stabilization", IEEE ICCV, pp. 341–348, 2009.
20. Jian Sun, "Video stabilization with a depth camera", IEEE CVPR, 2012.
21. Shuaicheng Liu, Lu Yuan, Ping Tan, and Jian Sun, "Steadyflow: Spatially smooth optical flow for video stabilization", IEEE CVPR, pp. 4209–4216, 2014.
22. Jiyang Yu and Ravi Ramamoorthi, "Robust video stabilization by optimization in cnn weight space", IEEE CVPR, pp. 3800–3808, 2019.
23. Ledig, Christian, Lucas Theis, Ferenc Huszár, Jose Caballero, Andrew Cunningham, Alejandro Acosta, Andrew Aitken, "Photo-realistic single image super-resolution using a generative adversarial network", Conference on computer vision and pattern recognition, pp. 4681–4690. 2017.
24. Zhang, Yulun, Yapeng Tian, Yu Kong, Bineng Zhong, and Yun Fu, "Residual dense network for image super-resolution," conference on computer vision and pattern recognition, pp. 2472–2481, 2018.
25. Geiger A, Lenz P, Stiller C, Urtasun R (2013) Vision meets robotics: The kitti dataset. The International Journal of Robotics Research 32(11):1231–1237
26. Choi, Inchang, Orazio Gallo, and Alejandro Troccoli, Min H, Kim, and Jan Kautz., "Extreme view synthesis", International Conference on Computer Vision, 2019.
27. Mildenhall B, Srinivasan PP, Ortiz-Cayon R (2019) Nima Khademi Kalantari Ravi Ramamoorthi, Ren Ng, and Abhishek Kar, "Local light field fusion: Practical view synthesis with prescriptive sampling guidelines." ACM Transactions on Graphics 38(4):1–14
28. Mildenhall B, Srinivasan PP, Tancik M, Barron JT, Ramamoorthi R, Ng R (2020) Nerf: Representing scenes as neural radiance fields for view synthesis. European conference on computer vision 30:405–421

29. Aliev K-A, Sevastopolsky A, Kolos M, Ulyanov D, Lempitsky V (2020) Neural point-based graphics. In European Conference on Computer Vision 30:696–712
30. Riegler G, Koltun V (2020) Free view synthesis. European Conference on Computer Vision 30:623–640
31. Liu, Yu-Lun, Wei-Sheng Lai, Ming-Hsuan Yang, Yung-Yu Chuang, and Jia-Bin Huang, "Hybrid neural fusion for full-frame video stabilization", IEEE/CVF International Conference on Computer Vision, pp. 2299–2308, 2021.

Finding the Source of a Tweet and Analyzing the Sentiment of the User from h(is)er Tweet History



Subhadip Mondal, Bilas Ghosh, Uday Dey, Antara Pal, and Alok Ranjan Pal

Abstract Spreading a rumor or hoax or any false news in social media has a great impact on today's society. This report focuses on this particular issue for Twitter platform. This type of incidents could be handled mainly in three steps, first- identification of the initiator of the fake news in that particular platform; second- after identification of the initiator, it is needed to track h(is)er earlier activities in that platform; and finally-sentiment of that user should be analyzed based on h(is)er previous posts. In this work, these three tasks have been implemented in three phases. In the first phase, the initiator of a re-tweeted link is found by crawling down the path and the profile of the initiating user is identified. In the second phase, the tweet history of that user is retrieved from h(is)er twitter handle. In the third phase, the sentiment of the user is analyzed from the retrieved tweets using a knowledge based approach. Experimentally, the analysis is done on four types of profiles, as- sports, politics, travel and movie. During the execution, 100 re-tweeted links, 25 from each category were tested. The reference sense inventory was prepared by the help of an online newspaper archive. The challenges and the pitfalls of this work are explained in detail at the end of this paper.

Keywords Sentiment analysis · Twitter data · Initiator identification

S. Mondal (✉) · B. Ghosh · U. Dey · A. R. Pal
College of Engineering and Management, Kolaghat, India
e-mail: mondalsubhadip212@gmail.com

B. Ghosh
e-mail: bilasghosh5664@gmail.com

U. Dey
e-mail: udayaustin316@gmail.com

A. R. Pal
e-mail: chhaandasik@gmail.com

A. Pal
National Institute of Technology, Durgapur, India
e-mail: antarapal22@gmail.com

© The Author(s), under exclusive license to Springer Nature Singapore Pte Ltd. 2023

S. Chinara et al. (eds.), *Advances in Distributed Computing and Machine Learning*,
Lecture Notes in Networks and Systems 660,
https://doi.org/10.1007/978-981-99-1203-2_25

1 Introduction

The evolution of social media has been fueled by the human impulse to communicate with each other and obviously it became possible due to the advancement in digital technology. It is a story about establishing and nurturing personal connections at a scale what follows is an examination of the origins of social media, its relatively rapid growth as a sociological and commercial force, and the change it has brought to the marketing world.

Twitter is one of the biggest social media networks for sharing authentic news and contents, but now-a-days fake news spreads faster than the true news on twitter. Studies found that 70% of re-tweets in twitter comprise fake news, hoax, threats, and most importantly un-controlled re-tweets are being done by internet bots to spread fake news. As of July 2021, Twitter is ranked as the 16th most popular social network worldwide (the rank is done based on the number of monthly active users). To put that in perspective, Twitter has 8.85% of the world's overall social media users. It is estimated that more than 4.48 billion people use social media at least once monthly. Twitter is predominantly used for obtaining the news (around 48% of all kinds of news sources). Other commonly reported usage includes entertainment, staying in touch with friends and family, and following corporate accounts.

But, spreading of the fake news or hatred in this platform is a matter of deep concern now-a-days. People use this platform for different malicious activities. The proposed work focuses on this particular issue. The very first thing is to identify the initiator of the fake news. In this work, it is performed by crawling down the path of re-tweeted message. Second task is to track the earlier activities of the user. Gathering this information cites the mentality of the user. And, third is to analyze the sentiment of the user using some algorithm. In the proposed work, this task is performed using a knowledge based approach with the help of a newspaper archive and an online semantic dictionary.

This work may help in reducing the spread of fake news, hoax, threats and bot generated re-tweets. In another way, by analyzing the sentiment of a user, that can be utilized for any useful purpose as well like promotion of any commercial product, educational resource or any relevant business purposes.

2 Survey

Before the experiment, a survey is carried out in this domain which is described below:

In 2020, Omar Adwan et al. [1] worked on sentiment analysis on twitter and other social media data. The authors tackled the problem of analyzing users' tweets in terms of thoughts, interests and opinions in a variety of contexts and domains. The authors combined the machine learning and lexicon-based approaches with theories and technologies from cognitive science, semantic web and big data. Such analysis

becomes valuable for several researchers and applications that require understanding people's view about a particular topic or event.

In 2022, Richa Sharma et al. [2] worked on the application of sentiment analysis on twitter data to extract the users' sentiments. They have observed that Tweeter Analyzer is capable of finding out the top ten trending hash-tags used by the users at any point of time and plotting them against their frequency using a bar graph. Their proposed model is an extension to the present model with additional functionalities of tracking users' sentiment and optimizing the processing power. According to their survey, machine learning techniques are more efficient than the symbolic techniques for analyzing users' sentiments.

In 2019, Dijana Kosmajac and Vlado Keselj [3] used a set of feature extraction and transformation methods in conjunction with ensemble classifiers for the PAN Author Profiling task. To check whether a user is a bot or a human being, the authors used the fingerprint and statistical diversity measures and for gender identification, the authors used a set of text statistics as well as syntactic information and raw words. They observed that the automated accounts have less diverse behavior than the genuine user accounts and these measures are more efficient in the fraud account detection task without diving into language-specific analyses.

In 2020, Shanaz Lathif [4] proposed a work on sentiment analysis based on the twitter data which could be used for News Recommendation. They observed that some people use twitter as a medium for real-time information propagation. Therefore, it acts as an entry point to the news world for many online news readers. Therefore, users' twitter activities could be used for news recommendation in the online platform as well as for different promotional activities. They focused on a few other factors as well, like trending news stories, preference of searching, profession of the user, community the person h(sh)e belongs to, etc.

E. Cambria et al. [5] carried out a work on cognition-driven sentiment analysis by the help of SenticNet which is a publicly available semantic resource for concept-level sentiment analysis. They observed that the concept-level sentiment analysis is more effective than the graph-mining and dimensionality-reduction based techniques. As the SenticNet is populated with 30,000 multi-word expressions, it can establish a deeper relation among the wide varieties of knowledge representations.

Pratap Nair et al. [6] attempted on a very useful topic from the social point of view. The authors analyzed the facebook posts of a user and analyzed h(is)er sentiment based on those data. According to their experiment, if any suspicious matter is observed in the user's posts, a notification will be sent to the user's friends about h(is)er mental status.

Eliaçık and Erdogan [7] carried out a research work on sentiment analysis which was mainly focusing on the financial community on Twitter. They proposed a model which added new user metrics to the traditional Naive Bayes based sentiment analysis model and after that the model analyzed the correlation between the mood of the financial community and the behavior of the stock exchange. The authors used the Spearman's rank correlation coefficient (SRCC) method in their work. They showed that their proposed sentiment analysis method (SRCC value 0.5634) results

in an effective positive correlation between stock market behavior and the sentiment polarity of the financial community.

There are few other relevant research works in this domain focusing on sentiment analysis of the users using different approaches [8–14].

3 Resources Used

3.1 Preparing a Knowledge Base

The innate sense of a text is retrieved by the help of a knowledge base. In this experiment, a newspaper archive has been created as a knowledge base. The newspaper is a rich source of categorized texts. Therefore, in this experiment an archive is created using the online repository of the *Times of India* newspaper. The multiple levels of indices of the texts in this archive are as follow:

Level 1: Politics; Level 2: Sports, Level 2.1: Cricket, Level 2.2: Football; Level 3: Movie; Level 4: Lifestyle, Level 4.1: Fashion, Level 4.2: Cooking, Level 4.3: Travel, Level 4.4: Interior-decoration; Level 5: Business; Level 6: Tech-update; Level 7: Job; Level 8: Astrology.

This machine readable archive has been used intensively for sense retrieval.

3.2 Lemmatization

The collected tweets and the texts of the sense inventory are lemmatized using the "nltk.stem" package available in "Python" software.

3.3 English WordNet

The English WordNet [15] has been used in this experiment as an online semantic dictionary. This dictionary contains several semantic information of a word. In this experiment, contexts of the texts are expanded by considering the synonymous words of every word. These synonymous words have been retrieved programmatically from the WordNet.

4 Proposed Approach

The proposed work is carried out in three phases: first, the initiator of a re-tweet (no matter, how many times the tweet is re-tweeted) is identified; second, the tweet history of the user is collected from the twitter handle of that user; and third, the sentiment of the initiator is analyzed using a knowledge based approach based on h(is)er last 450* tweets on the twitter handle.

First phase: Finding the initiator of a re-tweet.

Algorithm_1: Finding_the_Initiator.
Input: Tweet link.
Output: Initiator of the tweet.

Step1: Check whether the link is a genuine twitter link or not.

Step2: Establish the connection with the twitter API.

Step3: If the link is authentic, collect all the data from the account using its *id.*

Step4: Retrieve the ancestor of each tweet/re-tweet using some web-applications.

Step5: If any ancestor is available, repeat Step3 and Step4 until it returns null.

Step6: Null ancestor of a link signifies the source tweet.

Step7: Stop.

After identifying the initiator of the re-tweet in phase-1, the account *id* of that user is passed to the second phase for retrieving h(is)er tweet history.

Second phase: Retrieving the tweet history from the twitter handle.

Algorithm_2: Finding_Tweet_History.
Input: Source tweet.
Output: Tweet history of the user from the twitter handle.

Step1: Track the user name of an account by the help of its *id*.

Step2: Retrieve the most recent tweets* from the twitter handle of that user.

Step3: Identify the information part of the data.

Step4: Store the information section for further processing.

Step5: Stop.

After retrieving the tweet history of the user, that data is passed to the third phase for sentiment analysis of that user.

[*There's a limit on how much data can be collected from a twitter API within a span of time. It is called rate limit. Every API uses this restriction to prevent excess use of resources by a user or an internet bot. In twitter API, the data searching limits are: for twitter API v1.0, it is 180 search/15 min; and for twitter API v2.0, it is 450 search/15 min.]

Third phase: Sentiment analysis using knowledge based approach.

The sentiment analysis part has been developed as an extension of the earlier experiment of the scholars [16]. First, both the texts (tweets and the texts of the newspaper archive) are passed through the preprocessing steps, like normalization and lemmatization. Next, the content words (Noun, Verb, Adjective and Adverb) are identified in both the texts. In the earlier experiment, it was observed that finding the lexical similarity between a pair of regular texts is not sufficient for evaluating the context similarity. That is why, in this experiment, contexts of both the texts are expanded by retrieving the synonymous words of the content words from the English WordNet. The lexical similarities between the context-expanded texts result in better accuracy than the earlier experiment. Finally, Bootstrapping has been implemented to enrich the data sets. In every test case, the content words of the sense derived tweet are re-inserted in the corresponding knowledge base which has a direct impact on the next executions.

Algorithm_3: Sentiment_Analysis_using_ Knowledge_Base.
Input: Tweet history of the user.
Output: Sentiment of the user.

Step1: Normalize the collected tweets through a series of pre-processing steps, like-removal of different punctuation symbols, removal of uneven spaces, etc.

Step2: Normalize the texts of the newspaper archive.

Step3: Lemmatize the texts of the tweets and the sense inventory (knowledge base).

Step4: Identify the content words (noun, verb, adjective and adverb) of both the texts.

Step5: Collect the most frequent words from each category (sports, politics, travel and movie) of the archive for context expansion.

Step6: Retrieve the synsets of the words (tweets and the archive) from the English WordNet for context expansion.

Step7: Find the lexical overlap between a tweet and each of the sense inventories individually.

Step8: The sense inventory, having the maximum number of common words represents the sense of the tweet.

Step9: Implement Bootstrapping for performance enhancement.

Step10: Stop.

The above mentioned three algorithms demonstrate the overall work.

5 Result and Corresponding Evaluation

The model is tested on 100 re-tweet links. For experiment purposes, these links have been selected from some predefined categories, as- movie, travel, politics and sports.

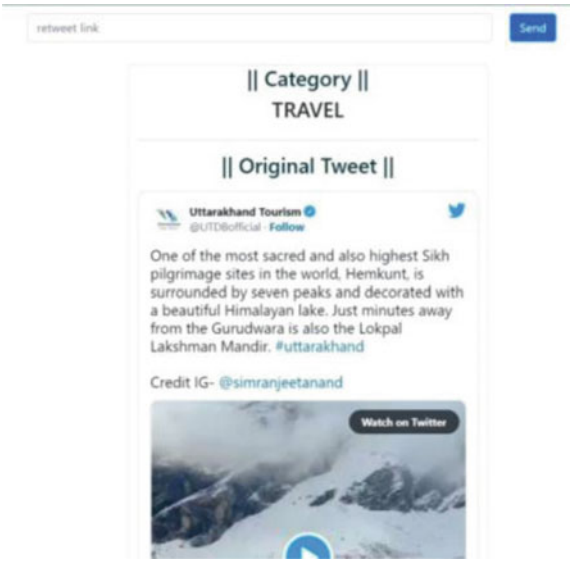


Fig. 1 Screenshot of the output of the travel related re-tweeted link

First, the re-tweet link is given in the text box and submitted. Next, the initiator of the re-tweet is found out by the help of Algorithm_1. After that, the tweet history of the initiator is retrieved from h(is)er twitter handle by using the Algorithm_2. Finally, with the help of Algorithm_3, sentiment of the initiator is derived.

Execution of one sample re-tweet from travel category is presented below (Fig. 1).

(a) Test 1:

https://twitter.com/crabsnil2/status/1537984323403427840?s=20&t=vLWyjLb22HlgHV1Nb_k6sg

Here, the initiator of the above mentioned tweet is "Uttarakhand Tourism" and the sentiment of the account holder according to its last 450 tweets is of Travel category (Fig. 2).

(b) Test 2:

https://twitter.com/crabsnil2/status/1465960573296779265?s=20&t=TbmsTCBD8HQ3qEr434ZNDw

If a user tweets on miscellaneous topics then the system could not retrieve a steady sentiment of the user. For example, Virat Kohli tweets mostly on promotion and other non-sports related data. Therefore, the system could not retrieve the sentiment from his tweet history.

Overall performance: Among the 100 test samples, all the initiating users were identified correctly, but the sentiments of the 18 users were not evaluated correctly due to the miscellaneous posts in their handles.

Comparison with the other works: The works discussed in the survey section incorporated different methodologies. Few of them are developed using machine

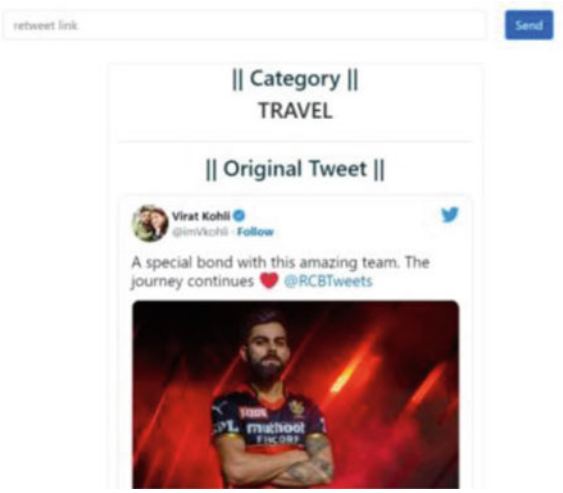


Fig. 2 Screenshot of a wrong output

learning based approaches, lexicon-based approaches etc. [1, 2, 11, 12]; some of them used feature extraction and syntactic information based strategy [3, 4, 7, 10, 13, 14]. But, these works did not consider the semantics of the texts which is an important feature for the natural texts. Cambria et al. [5] focused on the semantic analysis using a semantic dictionary. But, the work does not emphasize the senses of the meaningful words (decision making words) which is a crucial factor in the sense retrieval task.

The proposed work focuses on the key factors of the sentiment analysis task, such as- identification of the key decision making words, analyzing the expanded contexts of the texts to tackle the natural texts, and enhancing the performance of the system by incorporating the Bootstrapping strategy. These features have made the proposed system robust and capable of handling natural texts (Table 1).

6 Challenges

While developing this project, a lot of challenges have been encountered.

(a) **Understanding the Rate Limit of Twitter API**

To prevent internet bots and random users from sending uncontrolled requests to twitterAPI, a mechanism called "limit control" is incorporated by twitter. It defines a particular time, in which a certain amount of requests could be accepted by a developer account for accessing the resources. If excessive requests are made from an account, then twitter gives a warning and eventually bans the account. As this experiment is carried out by a human being, not a robot, therefore this problem has not been encountered.

Table 1 A few test samples and the resolved innate senses

Test samples	Topic
https://twitter.com/ForbesInspector/status/1582107133939961857?s=20&t=ro8OqpgIN8txbyYCRmg0IA	Travel
https://twitter.com/NatGeoTravel/status/1582919721342894081?s=20&t=ro8OqpgIN8txbyYCRmg0IA	Travel
https://twitter.com/NatGeoTravel/status/1582779220903616517?s=20&t=ro8OqpgIN8txbyYCRmg0IA	Movie
https://twitter.com/hp_tourism/status/1561195507170492417?s=20&t=ro8OqpgIN8txbyYCRmg0IA	Travel
https://twitter.com/TourismSikkim/status/1574586207064858625?s=20&t=ro8OqpgIN8txbyYCRmg0IA	Travel
https://twitter.com/TourismSikkim/status/1572586082654646276?s=20&t=ro8OqpgIN8txbyYCRmg0IA	Travel
https://twitter.com/ChrisMasonBBC/status/1582836978139410432?s=20&t=ro8OqpgIN8txbyYCRmg0IA	Politics
https://twitter.com/MamataOfficial/status/1580905845243908096?s=20&t=ro8OqpgIN8txbyYCRmg0IA	Politics
https://twitter.com/RahulGandhi/status/1582714217693876224?s=20&t=ro8OqpgIN8txbyYCRmg0IA	Politics
https://twitter.com/ArvindKejriwal/status/1582706646064975873?s=20&t=ro8OqpgIN8txbyYCRmg0IA	Movie
https://twitter.com/nprpolitics/status/1582935903361646592?s=20&t=ro8OqpgIN8txbyYCRmg0IA	Politics
https://twitter.com/CNNPolitics/status/1582885082402287616?s=20&t=ro8OqpgIN8txbyYCRmg0IA	Politics
https://twitter.com/ErosNow/status/1582755958287015936?s=20&t=ro8OqpgIN8txbyYCRmg0IA	Movie
https://twitter.com/yrf/status/1582944700683755521?s=20&t=ro8OqpgIN8txbyYCRmg0IA	Movie
https://twitter.com/yrf/status/1582946533510549504?s=20&t=ro8OqpgIN8txbyYCRmg0IA	Movie
https://twitter.com/filmfare/status/1582972207374163968?s=20&t=ro8OqpgIN8txbyYCRmg0IA	Movie
https://twitter.com/AnushkaSharma/status/1573255628327653377?s=20&t=Jp5sKeQQQOtaBW1UYIKOKA	Movie
https://twitter.com/AnushkaSharma/status/1564217554994106371?s=20&t=Jp5sKeQQQOtaBW1UYIKOKA	Sports
https://twitter.com/SkySportsNews/status/1582854169224171520?s=20&t=Jp5sKeQQQOtaBW1UYIKOKA	Sports
https://twitter.com/SkySports/status/1582819263030910976?s=20&t=Jp5sKeQQQOtaBW1UYIKOKA	Sports

(b) **Limiting the recursion**

The tweeted data provided by the twitter API consists of many different parts, one of them is the data that defines its previous tweet or the current one. The data behind the tweeted link includes all its current details and the link of its parent tweet. If the parent tweet does not exist, it holds its current link again. Through recursion, when the parent tweet is reached from a re-tweeted post, the recursion becomes infinite. To solve this problem, the recursion needs to be stopped carefully. In this experiment, this issue has been taken care of by setting a base condition for the recursion process.

(c) **Dependency on captions**

Since the analysis solely depends on the captions written by the user in the tweet, if a movie actor does repeated promotions which include captions that are not related to movies, then the data collected from the profile will have less number of matches with the movie related data set. Hence, it will have a very slim chance for that user to be categorized under the movie class. In this experiment, this situation happened several times (refer Sect. 5, test 2). As these experiments have been carried out on natural data, therefore these issues persist in the execution process and leave an impact on the performance of the system.

(d) **Variable size of data set**

If the sizes of the reference files are different, then the similarity finding task becomes biased, as the larger data set results in more common words. To avoid this circumstance, uniformity of the data sets has been incorporated by considering the mostly used 100 words for sense evaluation.

7 Conclusion and Future Scope of Work

The proposed work mainly focuses on sentiment analysis of a user using a knowledge based approach based on h(is)er activities in the Tweeter platform. Although there are several techniques for solving this problem, this paper emphasizes analyzing the context of a text through semantic analysis instead of syntax analysis. As the natural languages are creative in nature, any fixed rule is not sufficient to deal with all the variants of an expression. The English WordNet plays a great role in this respect. As a knowledge base newspaper archive has been used. The structured representation of data in the archive resolves the context of the text. The work might be extended further by analyzing the semantic information from the WordNet upto multiple levels.

References

1. Adwan O, Al-Tawil M, Huneiti A, Shahin R, Zayed A, Al-Dibsi R (2020) Twitter sentiment analysis approaches: a survey. Int J Emerg Technol Learn 15:79

2. Sharma A, Kedar A, Jadhav H, Gunjan, Ospanova A (2022) A survey on sentiment analysis of twitter using machine learning. Int J Innov Res Sci Eng Technol 11:3721–3725
3. Dijana K, Vlado K. (2019) Twitter user profiling: bot and gender identification notebook for PAN at CLEF 2019. In: LEF (2019)
4. Lathif S (2020) The potential of using twitter as a data source for user profiling in news recommendation. Int J Innov Res Comput Commun Eng 8
5. Cambria E, Olsher D, Rajagopal D (2014) SenticNet 3: a common and common-sense knowledge base for cognition-driven sentiment analysis. Proc Natl Conf Artif Intell 2:1515–1521
6. Nair P, Shaikh MM, Shinde VK (2020) Sentimental analysis based on user behavior on social media platforms. Int J Innov Res Comput Commun Eng 8(7)
7. Eliaçık AB, Erdogan N (2015) User-weighted sentiment analysis for financial community on Twitter
8. El Rahman SA, AlOtaibi FA, AlShehri WA (2019) Sentiment analysis of twitter data. In: International conference on computer and information sciences (ICCIS), pp 1–4
9. Nilpao P, Nanta N, Nopparuj S, Promsuk N (2022) Development of the recommended coffee shops application based twitter sentiment analysis. In: 37th international technical conference on circuits/systems, computers and communications (ITC-CSCC), pp 305–308
10. Arif SA, Hossain TB (2021) Opinion mining of customer reviews using supervised learning algorithms. In: 5th international conference on electrical information and communication technology (EICT), pp 1–6
11. Pandya V, Somthankar A, Shrivastava SS, Patil M (2021) Twitter sentiment analysis using machine learning and deep learning techniques. In: 2nd international conference on communication, computing and industry 4.0 (C2I4), pp 1–5
12. Sai Kumar TS, Pandian KA, Aara ST, Pandian KN (2021) A reliable technique for sentiment analysis on tweets via machine learning and BERT. In: Asian Conference on Innovation in Technology (ASIANCON), pp 1–5
13. Agarshana K, Karan P, Leo Celestine S, Vasantha VK (2021) Naive bayes algorithm for sentiment analysis on twitter. In: International conference on system, computation, automation and networking (ICSCAN), pp 1–4
14. Singh J, Tripathi P (2021) Sentiment analysis of Twitter data by making use of SVM, random forest and decision tree algorithm. In: 10th IEEE international conference on communication systems and network technologies (CSNT), pp 193–198
15. https://wordnet.princeton.edu/. Accessed July 2022
16. Pal A, Pal AR (2021) User profiling based on tweeter data using wordnet and news paper archive. In: 2021 international conference on advances in electrical, computing, communication and sustainable technologies (ICAECT), 2021, pp 1–4

Bitcoin Price Prediction by Applying Machine Learning Approaches



Debachudamani Prusti, Asis Kumar Tripathy, Rahul Sahu, and Santanu Kumar Rath

Abstract In the present-day market, the use of various cryptocurrencies is getting popular, and out of them, Bitcoin is very much accepted by the user. It is an application of blockchain that is based on decentralized peer-to-peer technology that leads to the development of cryptocurrency. As it is highly volatile, there is a necessity for the proper prediction of the price of Bitcoin for the users. Existing studies indicate a good number of articles on the price prediction of Bitcoin with higher accuracy using various methodologies. This study aims to extend research work based on effective as well as efficient models to predict Bitcoin prices using other machine learning and deep learning techniques. Various machine learning models such as Facebook's prophet, Linear regression, Autoregressive Integrated Moving Average (ARIMA), random forest, support vector regression (SVR), and deep learning model long short-term memory (LSTM) have been considered in this study for effective price prediction of Bitcoin. Later, the performance of various techniques is critically assessed using several standard parameters.

Keywords Bitcoin · Blockchain · ARIMA · LSTM · Prophet · Linear regression · SVR

1 Introduction

Cryptocurrencies have been a point of attraction for investors and traders. There are over 18,000 cryptocurrencies in existence as of March 2022. However, most of the currencies have negligible volume and value whereas some others have higher popularity. Bitcoin is a highly recognized cryptocurrency that carries the highest price as well as volume among other cryptocurrencies. Drastic fluctuations in its price have

D. Prusti (✉) · R. Sahu · S. K. Rath
Department of CSE, NIT Rourkela India, Rourkela, India
e-mail: debaprusti@gmail.com

A. K. Tripathy
School of Information Technology and Engineering, VIT Vellore India, Katpadi, India

© The Author(s), under exclusive license to Springer Nature Singapore Pte Ltd. 2023

S. Chinara et al. (eds.), *Advances in Distributed Computing and Machine Learning*,
Lecture Notes in Networks and Systems 660,
https://doi.org/10.1007/978-981-99-1203-2_26

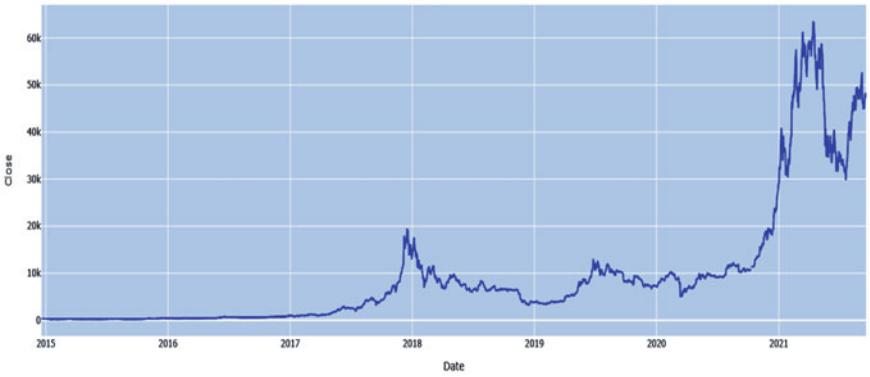


Fig. 1 Fluctuation in the closing price of Bitcoin over the years

been seen in the past few years. In 2008, having zero value Bitcoin observed an all-time high, i.e., 65,000 USD in February, April, and November 2021. From Fig. 1, the fluctuation in Bitcoin price prediction may be observed.

1.1 Cryptocurrency

The world is moving toward digital currency from the physical form of currency. Cryptocurrency is the digital form of currency that exists in the form of a complex equation [1, 2]. The concept of Cryptography is involved in the creation of cryptocurrency and securing the transaction. Cryptocurrencies are not controlled by any central or regulating authority or it can be said that they are free from any control by financial institutions. Cryptocurrencies are run on a distributed public ledger, i.e., Blockchain.

1.2 Blockchain

Blockchain is based on decentralized peer-to-peer technology [3]. It is also known as distributed ledger technology. When a transaction has been performed a block is created for each of these transactions and these blocks hold the transaction details along with encrypted transaction details from the previous block. These blocks are linked to each other as each of them takes references from the previous block. Once the data is validated inside a block by the network, the block is closed and connected to an existing chain of blocks. Each block is associated with the exact timestamp when added to the existing chain of blocks. This chain of blocks or records is called a ledger, and it is distributed to every node of the network. The distributed ledger

is immutable; hence, information inside the block cannot be altered, destroyed, or deleted. Thus, Blockchain guarantees the security and fidelity of the data and claims the trust without the need of any third party.

1.3 Theoretical Background of Bitcoin

Bitcoin was founded in 2008 by an anonymous author named Satoshi Nakamoto, who uploaded an academic white paper titled "Bitcoin: A Peer-to-Peer Electronic Cash System", where the very first time the idea of electronic currency was discussed [4]. Bitcoin provides the solution for online payments and offers lower transaction fees than any other existing online payment method.

When a Bitcoin is transferred from one party to another, this transaction needs to be validated before the corresponding block is added to the existing Blockchain. This mechanism is known as proof-of-work, where miners compete to find a nonce (a little random value) that goes to SHA-256 (a hashing algorithm) to generate a desired target hash value to verify the correctness and truthfulness of the block that requires a tremendous amount of processing power [5]. The equation is such that it is hard to solve but straightforward to verify. So, the security of Bitcoin comes from its creative use of hashing (SHA-256) as well as proof-of-work. As a reward for solving the proof-of-work, equation miners get some fraction of the newly created Bitcoin.

When a new block is created, it is distributed to every node present in the network. Initially, a copy of the ledger is available with all the decentralized nodes as shown in Fig. 2 in the network, where the verified blocks are added. The unverified blocks are dropped from the ledger. There is no centralized network as shown in Fig. 3. If more than half of the nodes approve a new block, then it will be added to the existing blockchain; otherwise, it will be rejected. So, to modify a block successfully control over more than 50% of the nodes in the network is to be accessed. This demonstrates the immutability of the blockchain and the trustworthiness of the Bitcoin.

Conventional techniques that have been used in the past research work for cryptocurrency price prediction are Linear Regression, Polynomial Regression, Autoregressive Integrated Moving Average (ARIMA), Random Forest (RF), Long Short-Term Memory (LSTM), Support Vector Regression (SVR), Multilayer Perceptron,

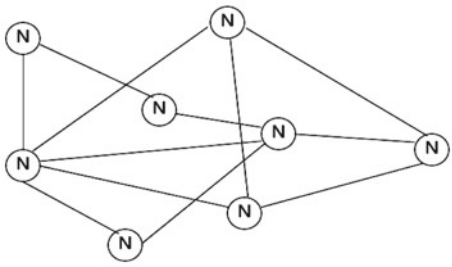


Fig. 2 Decentralized network

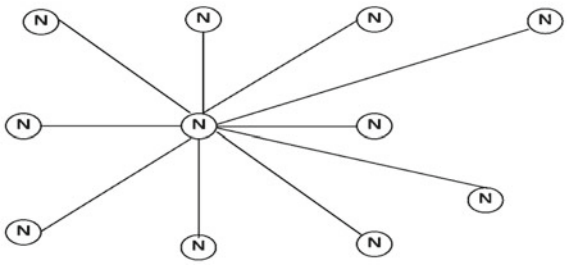


Fig. 3 Centralized network

etc. In this proposed research, the objective is to apply different machine learning models in making predictions and critically assess their performance.

2 Literature Study

A good number of researchers and experts have carried out the study in the area of Bitcoin price prediction. Mallqui et al. have studied different feature selection techniques to predict the maximum and minimum price of Bitcoin effectively [5]. They have considered different learning techniques such as Artificial Neural Network (ANN), Support Vector Machine (SVM), and ensemble methods to compare the price prediction. Chen et al. have discussed a set of high-dimensional features such as trading and market, gold prices, and market attention. They have used statistical methods such as logistic regression (LR) and linear discriminant analysis (LDA) with high-dimensional features [6].

Patel et al. [7] proposed a combined approach based on Gated Recurring Unit (GRU) and LSTM and they observed that the proposed scheme predicts the result accurately and this can be applied to various cryptocurrencies' price prediction [7]. Vijh et al. have applied Random Forest Regression and Artificial Neural Network for the prediction of the next-day closing price for five stocks of different sectors and calculated different performance evaluation metrics [8]. Henrique et al. [9] have discussed about Support Vector Regression (SVR) for price prediction of stocks from different sectors. They have taken daily price and up-to-minute frequencies. Further, the model is compared to the Random Walk model proposed by the efficient market hypothesis (EMH) [9].

Kristjanpoller and Marcel Minutolo [10] have proposed a hybrid Artificial Neural Network-Generalized AutoRegressive Conditional Heteroskedasticity (ANN-GARCH) model for the price prediction of Bitcoin that is mostly accepted coin and has the largest market capitalization [10]. Wu et al. [11] have proposed a new forecasting framework for price prediction of the Bitcoin cryptocurrency using LSTM. They applied conventional LSTM and LSTM with AR (2) by taking the daily price of Bitcoin [11]. Sin and Wang proposed a method that discovers the relationship

between features of Bitcoin and changes in the price of Bitcoin. They applied an Artificial Neural ensemble approach for the next-day price prediction [12]. Albariqi and Winarko have discussed baseline, neural network models, for Bitcoin price prediction for a long time frame as well as a short time frame. Multilayer Perceptron (MLP) and Recurrent Neural Network (RNN) are Baseline models [13].

3 Proposed Methodology

In this study, various techniques such as ARIMA, Linear Regression, SVR, Prophet, LSTM, and Random Forest Regression have been considered because these algorithms carry a good track record for prediction. These techniques are well recognized and work well on the dataset, containing price as a feature. Satisfactory results are observed for prediction using these machine learning techniques.

3.1 Collected Data

In this research work, two datasets d_1 and d_2 as shown in Tables 1 and 2 are considered for Bitcoin price prediction and both are obtained from Yahoo finance (www.finance.yahoo.com). Yahoo finance provides a wide range of information about different stocks and cryptocurrencies such as Bitcoin, Ethereum, and Dogecoin. Both the datasets contain seven features such as date, open, close, adjusted close, high, low, and volume. The first dataset d_1 contains daily Bitcoin prices from 01–01–2015 to 31–12–2018 having 1461 records and another dataset d_2 contains daily price data from 01–01–2019 to 17–02–2022 having 1144 records.

The closing price has been considered for prediction because investors make trading using the closing price. To carry out the research, the datasets d_1 and d_2 are divided into 80 and 20% ratios for training and testing, respectively, because this ratio was presenting the best result. Functions performed on the datasets vary according to the models applied. In the preprocessing part, missing values in the datasets are replaced by the mean values.

Table 1 Various feature attributes of dataset d_1

Date	Open	High	Low	Close	Volume
2015–01–01	320.434998	320.434998	314.002991	314.248993	8,036,550
2015–01–02	314.079010	315.838989	313.565002	315.032013	7,860,650
2015–01–03	314.846008	315.149994	281.082001	281.082001	33,054,400
2015–01–04	281.145996	287.230011	257.612000	264.195007	556,229,100
2015–01–05	265.084015	278.341003	265.084015	274.473999	43,962,800

Table 2 Various feature attributes of datasets d_2

Date	Open	High	Low	Close	Volume
2019–01–01	3746.71338	3850.91382	3707.23120	3843.52002	4,324,200,990
2019–01–02	3849.21631	3947.98120	3817.40942	3943.40942	5,244,856,836
2019–01–03	3931.04858	3935.68506	3826.22290	3836.74121	4,530,215,219
2019–01–04	3832.04004	3865.93457	3783.85376	3857.71753	4,847,965,467
2019–01–05	3851.97388	3904.90308	3836.90015	3845.19458	5,137,609,824

3.2 Data Pre-processing

Pre-processing is an essential step in the application of machine learning. In this research, datasets obtained from www.finance.yahoo.com contain some null valued rows that might affect the learning of the model so to deal with it null valued data is replaced with the mean of the remaining data. According to different learning models, different preprocessing techniques are required. For example, it is required to carry out stationary testing (a preprocessing technique) on the dataset before applying the ARIMA model.

3.3 Soft Computing Techniques Applied to the Prediction

In this section, an insight into background details of different machine learning algorithms is discussed and performance evaluation parameters are described in detail.

3.3.1 Linear Regression

Linear regression is a supervised learning algorithm that tries to obtain the relationship between the dependent variable (y) and one or more than one independent variable (x) as shown in Fig. 4. In linear regression, the objective is to fit the sloped straight line onto the data point. Mathematically, it is presented using the following formula:

$$Y = m_0 + m_1 X + E \tag{1}$$

where Y is the dependent variable, X is the independent variable, m_0 is the intercept of the line, m_1 linear regression coefficient, and E is the random error.

When applying linear regression, the intention is to find the line that best fits the data point so that the error term is as less as possible. Assigning different values to the regression coefficient m_1 will provide different slope to slope to line [14].

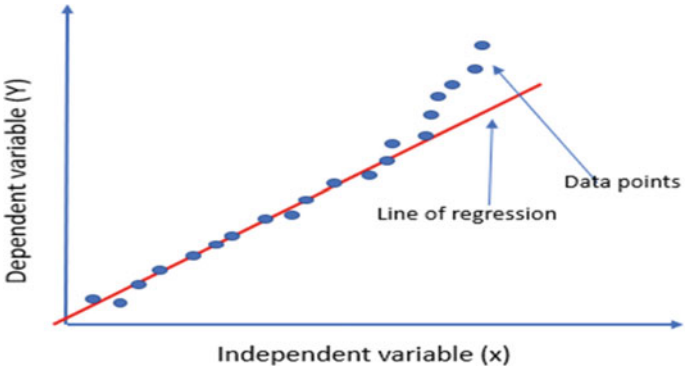


Fig. 4 Linear regression with independent and dependent variables

3.3.2 Autoregressive Integrated Moving Average (ARIMA)

ARIMA is a statistical model that is widely used for time series forecasting. Time series means a change in data points occurs concerning time. This model helps to predict future data on the basis of past data. ARIMA is a combination of Autoregressive (AR) and Moving Average (MA) [14]. It depicts three components of the model.

- *AR: Autoregression (p)*: This model denotes the dependency of an observation on its past observation, i.e., some value of lagged observations.
- *MA: Moving Average (q)*: This model depicts the linear dependency between an observation and residue error when a moving average model is applied to the lagged observation.
- *I: Integrated (d)*: The use of differencing of raw observations at a different time.

An *Autoregressive model* can be mathematically shown as

$$Y_t = m + n_1 Y_{t-1} + n_2 Y_{t-2} + \cdots + n_p Y_{t-p} + E_1 \tag{2}$$

where $Y_{t\text{-}1}$ is lag_1 of the series. Y_t is a stationary variable and m is a constant that is an intercept term. n_1 is the coefficient of lag_1 and E is the residue.

A *Moving average model* can be mathematically shown as

$$Y_t = \mu + E_t + \theta_1 E_{t-1} + \theta_2 E_{t-2} + \cdots + \theta_q E_{t-q} \tag{3}$$

where error terms are the error of autoregressive models of respective lags. Where μ term is the expectation of Y_t, the θ terms are weights associated with current and prior values, and E is the residue.

An ARIMA model is a combination of both the AR and MA models. If Eqs. 2 and 3 are combined, it will demonstrate the final equation of the ARIMA model.

$$Y_t = m + n_1 Y_{t-1} + n_2 Y_{t-2} + \cdots + n_p Y_{t-p} + \theta_1 E_{t-1} + \theta_2 E_{t-2} + \cdots + \theta_q E_{t-q} + E_t \tag{4}$$

where $n_i \neq 0$, $\vartheta_i \neq 0$, and $\sigma^2 > 0$. In this, p is considered as the order of AR, and q is considered as the order of q. The integration step is responsible to handle the non-stationary data. In this step, differencing is performed for the desired time to convert the non-stationary data to the stationary form. The general form of ARIMA can be represented as ARIMA (p, d, q).

ARIMA model verifies whether the data is stationary or not by performing the Dickey-Fuller test. If data is not stationary, then different techniques (log transformation and degree of differencing) are applied to make it stationary. It is a necessary step to make data stationary so that trends and seasonal structure could not affect the regression model. By selecting the ARIMA parameters p, d, q, the model can be configured to perform a specific function such as ARMA and even a simple AR, I, or MA model. 0 value of a parameter denotes not to use that element of the model [14].

3.3.3 Support Vector Regression (SVR)

A Support vector machine (SVM) is widely used in classification problems. It is also used for regression problems. This type of model is known as Support vector regression (SVR). Support vector regression follows the same principle as SVM.

In SVR, the intention is to choose the best-fit hyperplane on the data points. Based on the kernel function, the hyperplane changes from a straight line to a curved line as shown in Fig. 5. Popular kernel functions such as linear, RBF, polynomial, and sigmoid can be applied to make the prediction [15].

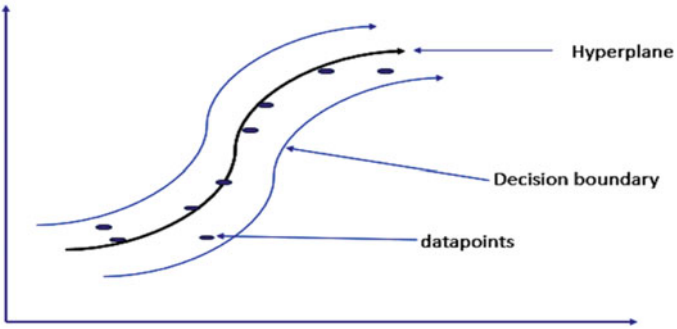


Fig. 5 SVR with the maximized hyperplane

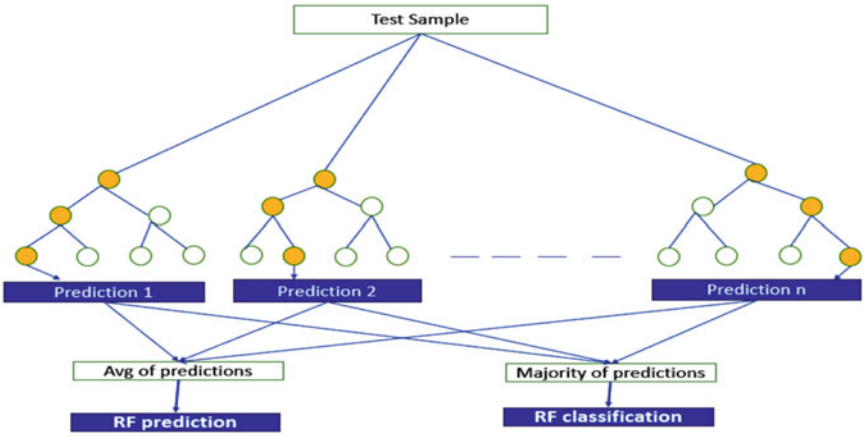


Fig. 6 Random forest for class prediction

3.3.4 Random Forest

Random Forest can be used for regression as well as classification problems as shown in Fig. 6 [16, 17]. Randomly generated decision trees are the building blocks of the random forest. Random forest takes n number of the random records from the dataset and generates an individual decision tree for each sample and each decision will produce some output then an average of the output is taken to make the final prediction [8].

3.3.5 Facebook's Prophet

A prophet is the decomposable additive model developed by the core data science researchers' team of Facebook in 2017, for univariate (one variable) time series forecasting. It provides a good accuracy result for time series prediction where nonlinear trends fit with daily, weekly, and yearly seasonality in addition to the holiday effect. Prophet model with three components seasonality, trend, and holidays can be written as the following equation [18].

$$y(t) = g(t) + s(t) + h(t) + \varepsilon_t \tag{5}$$

where $y(t)$: additive regression model, $s(t)$: seasonality, $g(t)$: trend, and ε_t: error. Parameters used by the Prophet model are simple to tune. Prophet attempts to fit many linear and nonlinear functions of time as components.

3.3.6 Long Short-Term Memory

LSTM is an advanced RNN that has the capability of handling the vanishing gradient problem that occurs in RNN as shown in Fig. 7. In a traditional neural network, all the inputs and outputs are loosely coupled which means they show less dependency. But in the case when the current output depends on the previous one then it is required to remember the output from the previous steps; thus, RNN came into existence. RNN resolves this issue by using hidden layers because these layers are responsible for remembering the information. RNN contains the *memory* to remember the information that is coming from previous steps. The same function is performed repeatedly for all the inputs that's why it is called a "recurrent" Network [11]. RNN cannot remember the long-term dependencies due to this gradient problem. LSTM is designed in such a way that long-term dependencies are not present. The high-level architecture of the LSTM consists of three gates. Each gate has its function to perform.

The *forget gate* is responsible to determine whether the information is relevant and should be remembered or irrelevant and must be forgotten. It gives the output between 0 and 1. Where 0 means completely forget the information and 1 means keep the information. The *memory gate* is responsible to store new data and update the data stored in a cell. The *output layer* is responsible to provide the output from the current timestamp to the next timestamp. This layer yields the output that depends on the information newly added, updated information, and the current cell state [14].

3.4 Performance Evaluation

To assess the performance of the models, different evaluation matrices are used. These matrices are listed as follows:

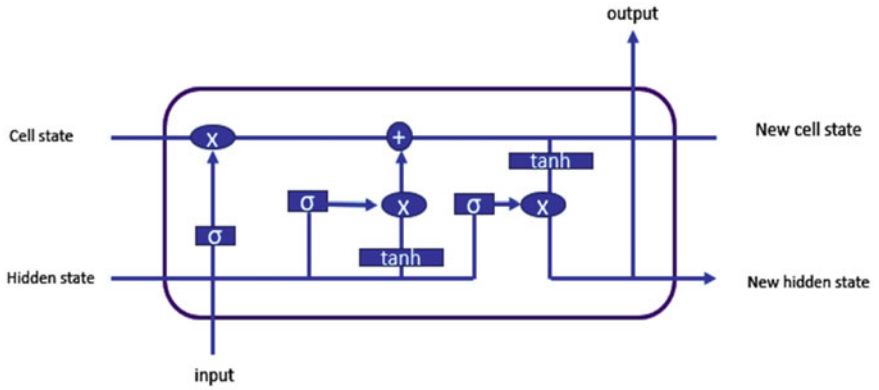


Fig. 7 LSTM Architecture

3.4.1 Mean Absolute Percentage Error (MAPE)

It is the most commonly used metric to measure the accuracy of forecasting. It is the mean of the individually calculated absolute percentage error. MAPE is easy to calculate and easy to interpret. The formula to calculate MAPE is as follows:

$$MAPE = \left(\frac{1}{n}\right) \sum_{1}^{n} \left(\frac{|real\ value - predicted\ value|}{|real value|} \right) * 100 \tag{6}$$

3.4.2 Mean Squared Error (MSE)

It is represented as an average of the squared difference between the actual value and the predicted value. MSE can be represented mathematically as follows:

$$MSE = \frac{1}{n} \sum_{1}^{n} (real\ value - predicted\ value)^2 \tag{7}$$

3.4.3 Root Mean Squared Error (RMSE)

It is simply the root of mean squared error that can be written as follows:

$$RMSE = \sqrt{\frac{\sum_{1}^{n} (real\ value - predicted\ value)^2}{n}} \tag{8}$$

3.4.4 R^2

It is a statistical measure that depicts how effectively a regression model fits on data points. 1 is the ideal value for r^2. A value close to r^2 depicts the effective fitting of the model. It is a derived metric which means it can be calculated using other metrics.

$$R^2 = 1 - \frac{SS_{\text{residue}}}{SS_{\text{total}}} \tag{9}$$

where $\mathbf{SS}_{\text{total}}$: Total sum of square and $\mathbf{SS}_{\text{residue}}$: Residual sum of square.

4 Experimental Result Discussion

In this research work, it is intended to critically assess the result obtained by applying various machine learning techniques such as linear regression, ARIMA, SVR, LSTM, random forest regression, and Prophet.

4.1 Methodology Used

The above techniques are applied to two datasets (d_1 and d_2) of Bitcoin from different time frames. Different performance evaluation metrics are empirically assessed for each model and analyzed for comparative study. Before applying the model as shown in Fig. 8, preprocessing has been carried out to replace null values from the dataset. According to different learning models, different preprocessing techniques are required. For example, stationary testing (a preprocessing technique) needs to be carried out on the dataset before applying ARIMA model. If the dataset is not stationary, then various techniques such as differencing and transformation are applied to make the dataset stationary. In LSTM, data is converted into a 3-D array shape as LSTM needs input in the form of a 3-D array.

In this research work, six different forecast models are presented. Thus, experiments have been performed on both datasets for all six machine models such as Linear Regression, Support Vector Regression, Long Short-Term Memory, Random Forest, ARIMA, and Prophet. All the scripts are written in Python language. As mentioned earlier, train and test set ratio is considered 80 and 20%, respectively. First, all the above models are applied to dataset d_1 and then dataset d_2, and a comparative analysis has been carried out for both the datasets.

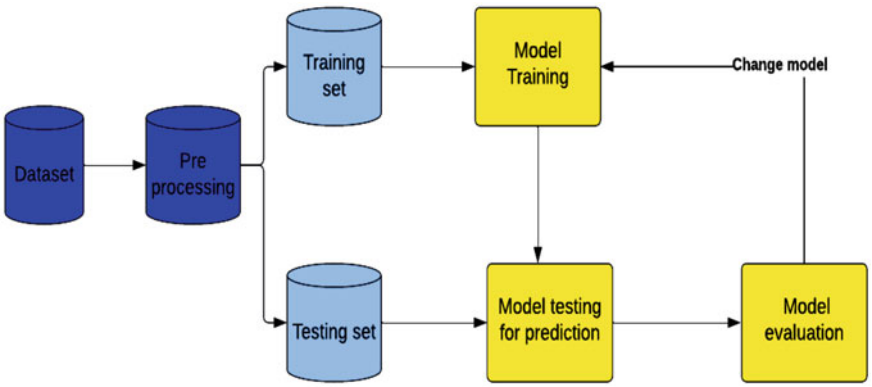


Fig. 8 A general methodology for machine learning model

Table 3 Result analysis for dataset d_1 and d_2 with different performance metrics

Model name	Dataset (d_1)	Dataset (d_2)
Linear regression	MAPE–170.99	MAPE–65.980
	R-Squared–0.55	R-Squared–0.73
ARIMA	MAPE–3.627	MAPE–2.55
	MSE–135.69	MSE–106.23
	RMSE–164.390	RMSE–154.36
SVR (Linear Kernel)	MAPE–80.80	MAPE–8.82
SVR (polynomial kernel)	MAPE–147.20	MAPE–59.20
SVR (RBF kernel)	MAPE–24.10	MAPE–25.38
LSTM		
Epochs 60	MAPE–5.25	MAPE–5.12
Epochs 100	MAPE–3.67	MAPE–3.321
Epochs 150	MAPE–3.887	MAPE–3.66
Epochs 200	MAPE–4.86	MAPE–4.56
Prophet	MAPE–9.90	MAPE–6.03
	RMSE–207.31	RMSE–174.16
Random forest regression	MAPE–27.69	MAPE–8.17
	R–Squared–0.79	R–Squared–0.82

4.2 Result Analysis

In this research work, an investigation is conducted on the prediction of Bitcoin value based on the closing price. Six machine learning techniques are accomplished, one at a time successively, on both the datasets (d_1 and d_2) individually as shown in Table 3. From the analysis of the results, it is observed that for dataset d_2, better prediction accuracy is obtained compared to dataset d_1. This unfavorable effect for the first dataset d_1 is observed because of the abrupt increment and decrement in the Bitcoin price during the time frame of this dataset. For the second dataset, the results are pretty satisfying because the price changes observed are not immediate. From the detailed analysis of the evaluation matrices, it is discovered that among all the applied models, LSTM and ARIMA are providing better results. Mean Absolute Percent Error obtained for ARIMA is 2.55% for dataset d_2 and 3.62% for dataset d_1.

For the LSTM model, different prediction results for different epochs have been obtained, and its best result has been witnessed for 100 epochs; because for this number of epochs, the model is trained perfectly without any overfitting and underfitting. Although other models such as random forest regression and Prophet come up with a satisfactory outcome. MAPE for Prophet is observed 9.90% and 6.03% for datasets d_1 and d_2, respectively, which is quite impressive. These models can apply to other cryptocurrencies as well as for different stocks for price prediction.

5 Conclusion and Future Work

In this research work, it is intended to apply various techniques for price prediction of Bitcoin, that have been applied such as Linear Regression, SVR, ARIMA, Prophet, Random Forest, and LSTM on the Bitcoin cryptocurrency, and their performance is critically assessed using performance metrics such as MAPE, R^2, MSE, and RMSE. Detailed result analysis is also presented in the result analysis section. From the analysis of the results, it is observed that for dataset d_2 better prediction accuracy is obtained compared to dataset d_1. This unfavorable effect is reflected in the first dataset d_1 because of the abrupt increment and decrement in the Bitcoin price during the time frame of this dataset. It is further observed that among all models LSTM and ARIMA are providing better results. These techniques can be further applied to various cryptocurrencies and stock market price predictions. Also, the Hybrid approach of these models can explore new possibilities for future research work.

References

1. Antonopoulos AM (2014) Mastering Bitcoin: unlocking digital cryptocurrencies. O'Reilly Media Inc
2. Baur AW, Bühler J, Bick M, Bonorden CS (2015) Cryptocurrencies as a disruption? empirical findings on user adoption and future potential of bitcoin and co. In: Conference on e-business, e-services and e-society. Springer, Cham, pp 63–80
3. Dai F, Shi Y, Meng N, Wei L, Ye Z (2017) From bitcoin to cybersecurity: a comparative study of blockchain application and security issues. In: 2017 4th international conference on systems and informatics (ICSAI). IEEE, pp 975–979
4. Nakamoto S (2008) Bitcoin: A peer-to-peer electronic cash system. Decent Bus Rev 21260
5. Mallqui Dennys CA, Ricardo ASF (2019) Predicting the direction, maximum, minimum and closing prices of daily Bitcoin exchange rate using machine learning techniques. Appl Soft Comput 75:596–606
6. Chen Z, Li C, Sun W (2020) Bitcoin price prediction using machine learning: an approach to sample dimension engineering. J Comput Appl Math 365:112395
7. Patel MM, Tanwar S, Gupta R, Kumar N (2020) A deep learning-based cryptocurrency price prediction scheme for Financial Institutions. J Inf Secur Appl 55:102583
8. Vijh M, Chandola D, Tikkiwal VA, Kumar A (2020) Stock closing price prediction using machine learning techniques. Procedia Comput Sci 167:599–606
9. Henrique BM, Sobreiro VA, Kimura H (2018) Stock price prediction using support vector regression on daily and up to the minute prices. J Financ Data Sci 4(3):183–201
10. Kristjanpoller W, Minutolo MC (2018) A hybrid volatility forecasting framework integrating GARCH, artificial neural network, technical analysis, and principal components analysis. Expert Syst Appl 109:1–11
11. Wu C-H, Lu C-C, Ma Y-F, Lu R-S (2018) A new forecasting framework for Bitcoin Price with LSTM. In: 2018 IEEE international conference on data mining workshops (ICDMW)
12. Sin E, Wang L (2017) Bitcoin price prediction using ensembles of Neural Networks. In: 2017 13th international conference on natural computation, fuzzy systems and knowledge discovery (ICNC-FSKD)
13. Albariqi R, Winarko E (2020) Prediction of bitcoin price change using neural networks. In: 2020 international conference on smart technology and applications (ICoSTA)

14. Siami-Namini S, Tavakoli N, Siami Namin A (2018) A comparison of ARIMA and LSTM in forecasting time series. In: 2018 17th IEEE international conference on machine learning and applications (ICMLA). IEEE, pp 1394–1401
15. Smola AJ, Schölkopf B (2004) A tutorial on support vector regression. Stat Comput 14(3):199–222
16. Kumar M, Thenmozhi M (2006) Forecasting stock index movement: a comparison of support vector machines and random forest. In: Indian institute of capital markets 9th capital markets conference paper
17. Prediction of stock prices using random forest and support vector machines. Int J Recent Technol Eng 8(4):473–477
18. Yenidogan I, Cayir A, Kozan O, Dag T, Arslan C (2018) Bitcoin forecasting using Arima and prophet. In: 2018 3rd international conference on computer science and engineering (UBMK)

Social Engineering Attack Detection Using Machine Learning



Kesari Sathvik, Pranav Gupta, Saipranav Syam Sitra, N. Subhashini, and S. Muthulakshmi

Abstract Since the 1990s online explosion, data security has been everyone's top priority while using the internet. It is crucial to take the proper precautions both at the organizational and individual levels to protect sensitive data from attackers who use social engineering flaws and human fallibility in addition to technological weaknesses. One of the prevalent methods is phishing where malicious emails are used in order to carry out these attacks. The combined use of Machine Learning (ML) algorithms and Natural Language Processing (NLP) provides a means to develop a system that can not only detect spam emails but also categorize emails by scanning their content and extracting information. In this research work, the authors have compared various ML algorithms such as Logistic Regression, Naive Bayes, Decision Trees, Gradient Booster, XG Boost, Random Forest, etc., and various NLP techniques to determine the best-performing model for such a system and implemented the same to develop a complete email security system that can help counter these prevalent social engineering attacks. The system architecture and methodology for source validation, spam detection, content scan, URL extraction and model analysis using various performance metrics like performance accuracy and execution time have been discussed in detail. The authors have also discussed how this approach can be used to develop a resource-efficient, yet accurate model which can be fine-tuned according to the needs of the organizations.

Keywords Machine learning · Natural language processing · Spam detection · Email classification

K. Sathvik · P. Gupta · S. S. Sitra · N. Subhashini (✉) · S. Muthulakshmi
Vellore Institute of Technology, Chennai 600127, India
e-mail: subhashini.n@vit.ac.in

© The Author(s), under exclusive license to Springer Nature Singapore Pte Ltd. 2023

S. Chinara et al. (eds.), *Advances in Distributed Computing and Machine Learning*,
Lecture Notes in Networks and Systems 660,
https://doi.org/10.1007/978-981-99-1203-2_27

1 Introduction

With the advancement in technology, the use of the internet increases exponentially, and with it, use of email for the purpose of exchanging information and communication has also increased, it has become second nature to most people. Email is considered as the most simple and cheap to use and also the quickest method for communication and sharing information worldwide. As email is known for simplicity, it also becomes a source of target for private and sensitive information collection, endangering them to different types of attacks. Spam mails are mails which the user is not interested to receive and considers it as a waste of time and resource. Sometimes, these spam mails can have secret contents that may be malicious and can steal information without anyone noticing and also lead to breaches in the host security system. In recent statistics, 40% of all emails are spam, which is about 15.4 billion emails per day and that costs internet users about $355 million per year. For example, if you received 100 emails in a day out of which spam mails are 70 and ham includes only 30 emails [1]. So, it takes time to identify the ham or important emails from it, which irritates the user. Email users receive hundreds of spam emails per day with a unique address and new content which are generated automatically by robot software. Attackers focus on techniques for attracting users towards providing the individual's information online through such mails. The information supplied to these phished emails may lead to leakage of private information and identity theft. Many email monitoring systems use keyword-based rules and content-based spam detection which is not a good approach for spam detection as these systems are accessing the content of URLs [2, 3].

This project mainly aims for designing an accurate and reliable system which detects if the particular mail consists of any malicious links or spam contents and classifies the mail accordingly into spam, malicious and safe based on the email source, content and the urls present in it. Systems that can detect social engineering attacks and classify emails are often considered as costly high-end products to protect the information security of organizations. The proposed system mainly consists of 4 functional blocks namely URL extractor, Malicious URL detector, Email source validator, and Spam Detection. URL extractor retrieves all URLs present in an email and provides them to the URL Classifier block which classifies the email as malicious or safe. Spam detection classifies email into spam or not spam based on the email body or content. Email Source Validator is an additional layer or feature which classifies mail based on the sender's email id. These block systems are developed individually using Natural Language Processing (NLP) techniques and Machine Learning models such as Logistic Regression, Naive Bayes, Decision Trees, Gradient Booster, Random Forest etc., and are integrated to form a final system. Although many of these classification-oriented techniques and ML algorithms are available, it is important to analyze and understand which of them is most suited to the task.

We proposed a complete system to monitor email for spam detection. The existing email monitoring systems have features like spam detection using email's content, malicious URL detection, etc. [4, 9]. But there is not any system consisting of image

text extraction, URL extraction, spam detection, URL classification together. We added all these features together to improve the accuracy of the system. Along with this we added a novel feature of source email validation which is the first layer of this email monitoring system. The advantage of the source email validation feature is to improve execution time. All these features combined together in our project make email classification more accurate.

Further, the paper is structured as follows. Section 2 describes the related works. Section 3 provides the description of the dataset and Sect. 4 provides details on the implementation methodology. Section 5 provides information on the software and the libraries used. Section 6 discusses the results and Sect. 7 concludes the paper and provides directions for future work.

2 Literature Survey

There are many algorithms available for email content-based spam detection. Natural Language Processing based on the Random Forest approach (NLP-RF) method helps in detecting spam emails with ease. Data is preprocessed by removing the numbers, punctuations and stop words and converting the text into lower-case letters, and then tokenized [4]. ML algorithms such as Support Vector Machine (SVM), ANN, J48, and Naïve Bayes are used for email spam classification and filtering. SVM provides the highest accuracy of 93.91% [5].

Emails with attachments like images may contain malicious content. There are many classification techniques to extract printed text from images. OCR is one of the techniques which helps to extract text from the images [6]. To identify whether the given content is spam or not, Bayesian classifier and Support Vector Machine (SVM) are used.

In [7] the authors determine the performance of the three methods of web scraping i.e., HTML DOM, XPath and Regular Expression (Regex). The results show that HTML DOM method, and Xpath take more memory as compared to web scraping with the regex. While Regex and XPath methods requires more amount of time and the smallest data consumption compared to HTML DOM.

Emails containing malicious URLs are considered as spam. In [8] the author uses the lexical features of the URLs to improve the performance of the classifiers to detect malicious web sites. Gradient boosting classifiers and random forest models are used to create an URL classifier using URL string attributes as features. Random forest model gave the highest accuracy of 98.6%. In [9] authors discussed about the detection of malicious URLs as a binary classification problem and analyzed different well-known classifiers, such as Support Vector Machines, Multi-Layer Perceptron, Naïve Bayes, Random Forest, Decision Trees, and k-Nearest Neighbors. Adding to that, they used a public dataset that comprises of 2.4 million URLs and 3.2 million features. In this, Random Forest and Multi-Layer Perceptron are the two classifiers which provides the highest accuracy among all.

Email Validation is a method to check whether an email address is valid or not. In [10] the author demonstrated a survey of different ML techniques that are existing such as K-Nearest Neighbor, Naive Bayes, Bayes Additive Regression, SVM, KNN Tree. This paper also presents a case-based spam filtering method to detect whether the sender's email is malicious or not. This idea was used for sender's email validation.

The proposed study in [11] uses the existing machine learning algorithms like Naive Bayes, SVM, CNN, and LSTM to detect and classify email content. According to findings, the LSTM model proves to be the best among all the other models with a highest accuracy of 98.4%.

In [12] the author presents URL detection with the use of three techniques for text feature extraction count vectorizer, hashing vectorizer-IDF vectorizer, and then built a phishing website detection model with the use of four ML classifiers Logistic Regression, K-NN, Decision Tree, Random Forest. We are also using this method at the initial step. The ML model with hash vectorizer and random forest gave an accuracy of 97.5%.

Lexical features of URL [13] like URL length, domain, symbol count, top level domain, etc. to propose ML model for malicious URL detection. They achieved accuracy of 99% by using random forest with an SD filter. The idea of URL properties is also used in implementation. A detection model was designed based on the dynamic convolutional neural network [14]. The k-max-pooling-layer replaces the pooling layer which is adapted according to the URL length. The results achieved a highest accuracy of 98% when the word embedding is performed based on character embedding.

Malicious URLs are the biggest threat to cyber security. J48 Decision Tree, Random Forest, Lazy Classifier, Bayes-Net are some classifiers used in [15]. The results shown using performance metrics TPR, FPR, Precision, Recall and F Measure and random forest showed the highest accuracy compared to other classifiers.

3 Dataset Description

For this research, 5 datasets are used for implementing the classifier. URL Classifier is trained upon 4 datasets:

1. Benign dataset—Dataset consists of 35378 URL which are categorized as safe. One column of zero's is added which specifies that the URLs are safe.
2. Malware dataset—This dataset consists of 11566 URLs which are classified as malicious URLs. One column of one's is added which specifies that the URLs are malicious.
3. Malicious and benign URLs dataset—This dataset consists of 450176 URLs of both safe and malicious category.

4. Malicious URLs dataset—This dataset has 641119 URLs categorized into safe, defacement, phishing and malware. All safe URLs are marked as 0 and other category URLs are marked as 1 specifying as safe and malicious respectively.

Spam detection model is trained upon "Spam email detection dataset" which consists of about 5697 unique email contents, which is classified as spam and not spam.

4 Implementation

The proposed system consists of 3 main subsystems namely URL Extractor, URL Classifier and Spam detection as shown in Fig. 1. Email source validator acts as an extra security layer, by classifying the email only based on the email id of the sender. The API classifies the given email id as safe or not safe. If any image is present, the content of the image is extracted and added along with the email content Then, if safe, Email content is passed to the URL extractor, extracts any URL present in the content. If URL is present, the URLs are provided to URL Classifier which categorizes URL into malicious or not. Mail will be classified as malicious, if any URL is found to be malicious. If all the URLs are classified as safe, the email content is then passed through spam detector. If no URL is present, the content is directly passed to spam detection which classifies the mail as spam or safe.

Email Source Validator was implemented as an additional security layer which classifies the email based on the sender's email id. This is implemented with the use of third-party API—Emailable. The email id to be validated is passed through the API. The API platform classifies the email id as 'deliverable', 'risky', 'undeliverable' and 'unknown'. Sometimes, email is presented in the form of an image. So, it becomes mandatory to check and extract information from the image for classifying that email. Pytesseract library is used for detecting text in the image.

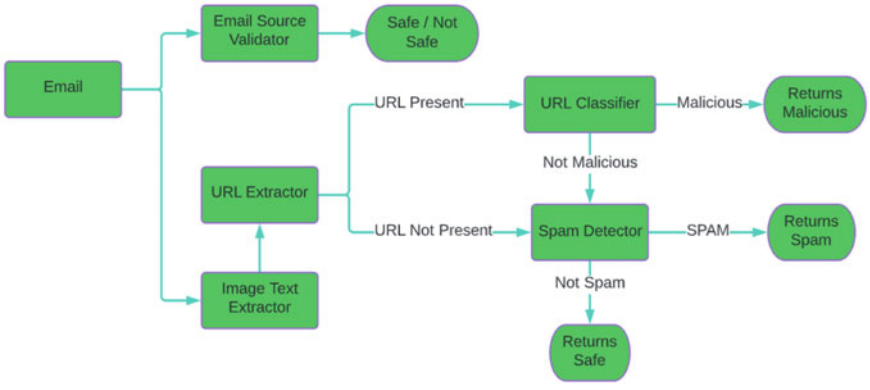


Fig. 1 Block diagram of proposed system

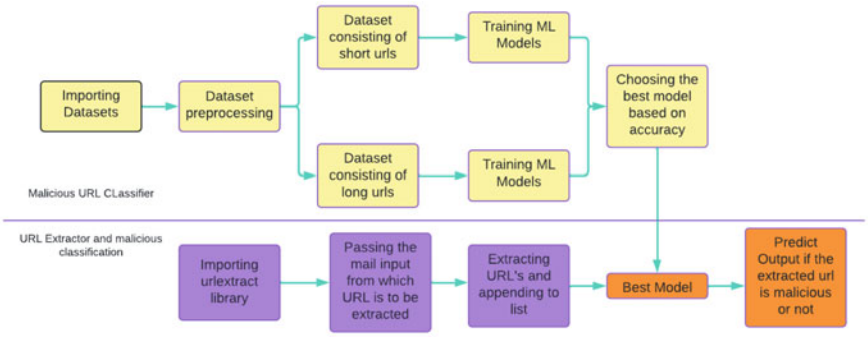


Fig. 2 Workflow diagram of URL classifier

URL Extractor block make use of the python library 'urlextract'. This library extracts URL from a string by searching for an occurrence of top-level domain (TLD). If any TLD is found in the string, it parses the string in both the direction from that position and searches for any stop character for detecting the boundaries of an URL.

The work flow for proposed URL Classifier system is shown in Fig. 2, which is designed based on the lexical features of an URL. The URL Classifier extracts lexical features such as domain length, entropy, protocol used, URL length, URL Path length, Character length of domain, Top level Domain, number of digits, count of each special character ('&', '#', '%', '/', '.') from an URL, checks if the domain consists of any port number, inspects URL for presence of any IP address and words such as 'client', 'admin', 'server' and 'login'.

After extracting the lexical features, the average of URL length is calculated and the final dataset is divided into 2 datasets—Short and Long URLs. Short URL dataset consists of approximately 687,352 URLs whose length is less than that of average URL length and remaining 460,959 URLs are categorized into Long URL dataset. 5 basic Machine Learning Algorithms Namely Logistic regression, Bernoulli Naïve bayes, Decision Tree, Gradient Booster and Random Forest are used in this system. ML Models for both short and long URL datasets are trained and implemented separately. Random forest performed the best for both long and short URLs making it the best model. The URLs extracted from the email are provided as input into the random forest trained model for classification.

Spam detection system is implemented using Natural Language Processing (NLP) Algorithm. The process of NLP involves converting string into machine understandable language so that data can be processed further. Firstly, using the 'nltk' library in python, the stopwords are removed. Stopwords are unimportant or commonly used words which don't add much important meaning to the content. Work flow for this block system is shown in Fig. 3.

Two pre-processing algorithms namely Tokenizer and Tfidvectorizer are used for dataset pre-processing and are compared against each other based on Machine learning model performances trained separately for both of these algorithms. Four

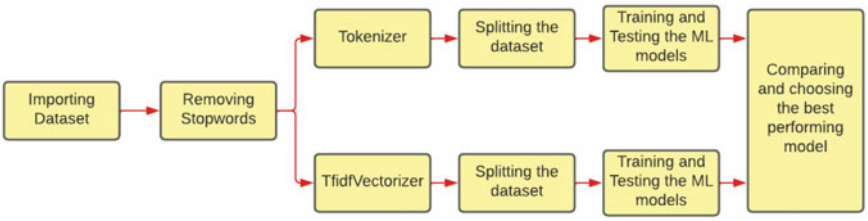


Fig. 3 Work flow for spam detection

ML models are used in this system namely Support Vector Machine, K-Neighbors Classifier, Multinomial Naïve bayes and Random Forest for comparison.

5 Software and Libraries Used

The software and the libraries used are tabulated below in Table 1.

6 Results and Discussions

Machine Learning model's performance is generally evaluated based on the model's accuracy. Accuracy is defined as the percentage of correctly predicted occurrence by total prediction occurrences. Accuracy is one of the factors that can be considered for evaluation as it alone doesn't guarantee the same performance on another set of occurrences. Sometimes, all the models perform with equal or negligible differences of accuracy. In this situation, it's wise to test the models with other parameters for best performance consideration.

As of the second parameter, Execution time is calculated for each model. Here, Execution time is defined as the time taken by the model for training and predicting the testing data. Models which have best accuracy performance but also takes more time, becomes inefficient for implementation in real world. So, the best model would be defined as the optimized model with high accuracy with least time taken.

Table 2 provides the comparison between machine learning models based on the accuracy performance of a model on the dataset. Random Forest proved to be the best model for both short and long URLs as it yielded the highest accuracy of 95 and 99% respectively. Random Forest bears the best accuracy as the outputs of many decision trees are usually averaged.

Table 3 provides the comparison between machine Learning Models when the dataset is preprocessed using Tokenizer and Tfidfvectorizer algorithms. From the below observation, it can be concluded that Tfidfvectorizer is proved superior to the

Table 1 Softwares and libraries used

Google Colab	Platform in which the whole coding process of the project is executed
Emailable API	Third party API which classifies an email id as deliverable or not
Emailable	A library used for connecting colab with emailable API with help of the key
Urlextract	This library is used for extracting URLs present in the string or mail content
Pandas	Used in reading and processing datasets
Urlparse	It breaks the url into 6 different components
Train_Test_Split	It splits the dataset into training and testing sections
BernoulliNB	Naive Bayes classifier for multivariate Bernoulli models
Tree	Used in implementing decision tree machine learning model
LogisticRegression	Library for implementing logistic regression ML model
GradientBoostingClassifier	Implements gradient boosting algorithm
RandomForestClassifier	Library for using Random Forest ML model
Pickle	Stores objects in a byte stream file
Nltk	For applying the concept of natural language processing
Tokenizer	Splits the complete text into small words by removing symbols, stop words, etc.
TfidVectorizer	A text vectorizer which transforms the text into a usable vector
Pad_sequences	Used to pad all the sequences to the same length
SVC	Implements support vector classification algorithm
MultinomialNB	An approach used to solve NLP problems by constructing classifiers e.g. counting words in text classification
KNeighboursClassifier	A supervised machine learning technique helps in classification and regression
String	Used in applying inbuilt methods for processing a string
Pytesseract	It is an optical character recognition (OCR) tool using which text can be extracted from images
PIL	Python image library is best for image processing

Table 2 Accuracy performance of ML models for URL classification

Machine learning models	Short URLs (%)	Long URLs (%)
Logistic regression	87	90
Bernoulli Naive Bayes	74	77
Decision tree	94	98
Gradient booster	91	95
Random forest	95	99

Table 3 Accuracy Performance Comparison of ML models for different preprocessing methods

Machine learning models	Tokenizer (%)	Tfidfvectorizer (%)
Support vector machine	76.4	99.3
K-Neighbors	78	97.49
Multinomial Naive Bayes	57.4	98.74
Random forest	87.29	98.39

Table 4 Time performance comparison of ML models

Machine learning models	Time taken (in seconds)
Support vector machine	5.66
K-Neighbors	0.57
Multinomial Naive Bayes	0.01
Random forest	1.38

tokenizer pre-processing algorithm. We can see the amount of performance difference that can be observed by just changing the preprocessing algorithm.

As we can observe that all Machine Learning models provide the same amount of accuracy with negligible difference. So, to find the best model, time taken for the model to train and test is tabulated as shown in Table 4. We can see that the best accuracy model, Support Vector Machine, is slower by 5 times than the best time performer, Multinomial Naive Bayes. So, we can conclude that Multinomial Naive Bayes is the optimum model compared to Support Vector Machine.

The proposed system classifies email into 4 categories namely—'Validation Failed', 'Malicious URL Detected', 'Spam' and 'Safe'. Email is classified as 'Validation Failed' if the email id is categorized as risky or non-deliverable by the third-party API. If validation is successful, and any of the extracted URL is classified as 'malicious', then system classifies the email as 'Malicious URL Detected'. If all the extracted URL are classified as 'Safe' or no URL is detected, then if the contents are detected as spam, then the email is classified as 'Email Detected is Spam'. If the email is classified as safe in all of the subsystems, then the model returns 'Email is safe'.

7 Conclusion and Future Work

In this research work, a method to monitor and classify an email based on various means like on the basis of its text, URLs present in it, images attached to it and sender's email address was demonstrated. Generally phishing attacks are performed by sending spam/malicious email. So, this system is good enough to detect these kinds of attacks by checking different attributes of email. This type of classifier is important for an organization's growth, as it enables more security for the private information.

It is important to develop an algorithm for email id validation instead of relying on a 3rd party API as the organization data may be prone to third party API and the terms and agreements needs to be monitored continuously for maintaining the integrity with the third party. A dataset can be maintained which keeps updating the ids with its classification based on the AI model or user feedback.

URL extraction can be extended to detect hyperlinks from an email. An advanced Image detection can be used for extracting texts more accurately.

URL classification accuracy can be improvised if URL host-based features or content in URL can be extracted from the available URL. Advanced ML concepts like neural network, deep learning etc. can be implemented and tested for accuracy.

References

1. Pashiri RT, Rostami Y, Mahrami M, Spam detection through feature selection using artificial neural network and sine–cosine algorithm. Math Sci. https://doi.org/10.1007/s40096-020-00327-8
2. Dada EG, Bassi JS (2019) Machine learning for email spam filtering: review, approaches and open research problems, vol 5, Issue 6, e01802. ISSN 2405–8440, https://doi.org/10.1016/j.heliyon.2019.e01802
3. Khonji M, Iraqi Y, Jones A (2013) Enhancing phishing e-mail classifiers: a lexical url analysis approach. Int J Inf Secur Res (IJISR) 3(1)
4. Alanazi R, Taloba A (2021) Detection of email spam using natural language processing based random forest approach. Research square, information system department. Assiut University. https://doi.org/10.21203/rs.3.rs-921426/v1
5. Jazzar M, Yousef RF, Eleyan D (2021) Evaluation of machine learning techniques for email spam classification. Int J Educ Manag Eng (IJEME) 11(4):35–42. https://doi.org/10.5815/ijeme.2021.04.04
6. Khakurel N, Bhagat (2019) Natural language processing technique for image spam detection. In: 2nd international Himalayan conference on advanced engineering and ICT—convergence 2019 (ICAEIC-2019), vol 2, no 1, pp 22–30. https://doi.org/10.1109/ICDAR.2013.140
7. Gunawan R, Rahmatulloh A, Darmawan I, Firdaus F (2019) Comparison of web scraping techniques: regular expression, HTML DOM and Xpath. In: Proceedings of the 2018 international conference on industrial enterprise and system engineering (ICOIESE 2018), Atlantis Highlights in Engineering (AHE), vol 2, pp 282–287. https://doi.org/10.2991/icoiese-18.2019.50
8. Catak FO, Şahinbaş, Dörtkardeş V (2020) Malicious URL detection using machine learning. Artif Intell Paradig Smart Cyber-Phys Syst 160–180. https://doi.org/10.4018/978-1-7998-5101-1.ch008. https://doi.org/10.4018/978-1-7998-5101-1.ch008
9. Vanhoenshoven F, Nápoles G, Falcon R, Vanhoof K, Köppen M (2016) Detecting malicious URLs using machine learning techniques. IEEE Symp Ser Comput Intell (SSCI) 2016:1–8. https://doi.org/10.1109/SSCI.2016.7850079
10. Hanif, Bhuiyan H, Ashiquzzaman A, Juthi T, Biswas S, Ara J (2018) A survey of existing e-mail spam filtering methods considering machine learning techniques. Glob J Comput Sci Technol 1(2):Version 1.0, 21–28. Online ISSN: 0975–4172 & Print ISSN: 0975–4350: 0975–4172 & Print ISSN: 0975–4350
11. Siddique ZB, Khan MA, Din IU, Almogren A, Mohiuddin I, Nazir S (2021) Machine learning-based detection of spam emails, vol 2021. Scientific Programming Hindawi publications, Article ID 6508784, 11 pages. https://doi.org/10.1155/2021/6508784

12. Lakshmanarao A, Babu MR, Bala Krishna MM (2021) Malicious URL detection using NLP, machine learning and FLASK. In: 2021 international conference on innovative computing, intelligent communication and smart electrical systems (ICSES), pp 1–4. https://doi.org/10.1109/ICSES52305.2021.9633889
13. Mamun M, Rathore, Muhammad, Lashkari H, Arash, Stakhanova, Natalia, Ghorbani A (2016) Detecting malicious URLs using lexical analysis, vol 9955, pp 467–482. https://doi.org/10.1007/978-3-319-46298-1_30
14. Wang Z, Ren X, Li S, Wang B, Zhang J, Yang T (2021) A malicious URL detection model based on convolutional neural network. Commun Secur Soc-Oriented Cyber Spaces Article ID 5518528. https://doi.org/10.1155/2021/5518528
15. Kapil D, Bansal A, Anupriya, Mehra N, Joshi A (2019) Machine learning based malicious URL detection. Int J Eng Adv Technol (IJEAT) 8(4S). ISSN: 2249–8958

Corn Yield Prediction Using Crop Growth and Machine Learning Models



Audrey B. Moswa, Patrick Killeen, Iluju Kiringa, and Tet Yeap

Abstract The advancement of Internet of Things technology has created a plethora of new applications and a growing number of devices connected to the internet. Among these developments emerged the novel concept of smart farming, where sensor nodes are used in farms to help farmers acquire a deeper insight into the environmental factors affecting productivity with the help of machine learning (ML) models and mechanistic crop growth models (MCGM). We introduce an MCGM to (a) predict crop growth and subsequent yield, subject to weather, soil parameters, crop characteristics, and management practices, and (b) measure the influence of nitrogen on yield throughout the growing season. We trained ML models to improve the yield prediction of the MCGM using the historical data from the state of Iowa (US) and found that the multilayer perceptron (MLP) produced the best results. As such, we chose the MLP to maximize corn yield prediction. The experiment was performed using stochastic gradient descent (SGD) and adaptive moment estimation (Adam) optimizers. The experiment results revealed that the SGD optimizer and the dataset with the scenario of unchanged parameters provided the highest crop yield prediction compared to the MCGM.

Keywords Smart farming · Precision agriculture · Machine learning · Crop growth model · Yield prediction

1 Introduction

The agricultural sector faces food scarcity and insufficient cost-effectiveness due to the global increase of population. Smart technologies can be used to address this challenge, including the Internet of Things (IoT), a network of systems designed to communicate, perform distributed sensing, and compute in collaboration with

A. B. Moswa (✉) · P. Killeen · I. Kiringa · T. Yeap
School of Electrical Engineering and Computer Science, University of Ottawa,
75 Laurier Ave E, Ottawa, Ontario K1N 6N5, Canada
e-mail: amosw052@uottawa.ca

© The Author(s), under exclusive license to Springer Nature Singapore Pte Ltd. 2023

S. Chinara et al. (eds.), *Advances in Distributed Computing and Machine Learning*,
Lecture Notes in Networks and Systems 660,
https://doi.org/10.1007/978-981-99-1203-2_28

other devices in real time [31]. Other technologies include crop models for cropping systems and prediction applications, such as mechanistic and machine learning (ML) models [8]. When combined, ML and mechanistic crop growth models (MCGM) are expected to allow precision agriculture (PA), an application of IoT, to deliver high operational efficiency, maximize crop growth and yield, reduce production costs, and efficiently utilize inputs (e.g., water or fertilizers) [20, 31]. In past years, the increase in maize yields was associated with increased amounts of nitrogen (N) fertilizer application to crops. Nitrogen plays a crucial role in crop metabolism, and an adequate supply of N is critical to increasing crop yield.

The present work performs corn yield prediction experiments, identifies the better-performing ML algorithms, and proposes an MCGM that predicts corn growth and yield. In addition, the present work suggests ML models for optimizing dataset parameters from the MCGM and is structured as follows: Sect. 2 details the necessary background relating to IoT, IoT architectures, PA, and smart farming (SF); Sect. 3 explains the methodology applied for yield prediction using the MCGM and ML; Sect. 4 presents experiments and results; and Sect. 5 concludes our work and discusses future work.

2 Background

The **Internet of Things** (IoT) has found its application in many domains including agriculture. The main advantages of using IoT in agriculture are achieving higher crop yields and lower costs. Recently, the number of network-connected IoT devices has increased significantly; as a result, cloud computing models have become limited in their ability to manage the produced data and meet the real-time requirements of IoT applications. Big data involves the collection of massive amounts of heterogeneous data from multiple distributed data sources [13], and in PA, big data is used to improve productivity and supply chain management [31]. Fog computing aims to move processing abilities from the cloud closer to the network edge to reduce computational loads and minimize latency [21], while edge computing focuses less on the infrastructure side and more on the things/device side [4]; that is, fog computing nodes are slightly further away from the data sources/things than edge computing nodes [21]. A gateway is an intermediate component that operates as a communication medium between sensor nodes and the cloud [5]. It is responsible for gathering and aggregating heterogeneous sensor data and performing lightweight data processing and storage. An **IoT architecture** requires (a) concurrent data collection to support, analyze, and control data from various sources; (b) good connectivity and communication between devices [32]; (c) robust protocols between sensors/actuators and the cloud; (d) availability; and (e) high-quality service [12]. By reviewing Killeen et al. [17], Stamatescu et al. [26], and Yang et al. [33], an SF IoT architecture can be divided into (a) three physical layers, namely: cloud computing, fog computing, and edge computing (or field devices) [7]; and (b) three logical layers: perception (lower layer), middleware (middle layer), and application (upper layer). The percep-

tion layer is close to the data sources, supports real-time data analytics, abstracts both the edge and fog, and contains various resource-constrained nodes (e.g., actuators). Data from in-field sensors is sent (e.g., over Zigbee or Wi-Fi) to microcontrollers to be read and transmitted to a local gateway [27]. A gateway is deployed on every farm and sends aggregated data to the cloud (e.g., over LTE or Fiber) through the middleware layer (via MQTT). The middleware layer abstracts both the fog and cloud and performs more heavy-duty processing, storage, and analytics for decision-making. The decisions and other results are then sent (via HTTP) to the application layer. The application layer offers the full resources of the cloud, provides an interface to the entire PA system, and offers PA services to the farmers (e.g., yield prediction) [5]. **Precision agriculture** (PA) is a farm management method that uses information technology and specialized equipment to enhance productivity and decision-making in agriculture [31]. PA's main objective is to support farmers in managing their business by optimizing inputs and reducing the environmental impacts to ensure profitability, sustainability, and environmental protection [12]. IoT implementation in PA requires sensor integration, automatic control, information processing, and network connection. PA uses ML and data analytics to explore data, enable corrective actions, and achieve operational effectiveness [11]. Unlike PA, **smart farming** (SF) is the efficient use of information and communication technology to manage and optimize complex farming systems. A variety of data is collected from smart machines located in the field and is processed and analyzed to provide farmers with access to insights to help decision-making by rendering agriculture more connected and intelligent [31]. SF also reduces the use of critical resources and improves productivity while lowering farming costs [14]. Significant research has been conducted to leverage **machine learning** (ML) in agriculture to acquire and assess data, achieve more precise predictions, and manage crop components [10]. The monitoring of crop growth and yield prediction is one of the most critical processes to ensure food availability and promote sustainable agriculture. **Crop growth models** are used to understand the crop production process, analyze strategies, and manage agricultural systems [1, 29]. Estimating agricultural output based on weather and soil conditions as well as crop management is a key purpose of crop simulation models [19]. The objective of a crop growth model is to determine whether the model's complexity is suited to a specific problem and whether the model has been tested in diverse environments. Whisler et al. [29] present empirical models as a unique function to evaluate the components involved in the process, but this is insufficient to express the research results. Crop models are limited in their improvement of crop systems' performance and environmental impact assessment. An example of a crop growth model is the Decision Support System for Agrotechnology Transfer (DSSAT) model [16]. **Important parameters** in crop growth models include (a) total available water, (b) field capacity, (c) wilting point [2], and (d) readily available water [3]. Generally, crops do not reach their potential growth and yield due to a variety of **environmental limitations**. The crop growth models evaluate the risks of the stresses generated by several factors such as water and nutrient content, temperature, aeration, and radiation. Drought stress is a factor that limits biomass production in proportion to transpiration reduction [25]. Water stress (which is estimated by considering supply and demand [2])

is primarily responsible for significantly impacting the plant's phonological aspect, and decreases grain yield due to significant reduction of (a) total leaf area, (b) leaf expansion, and (c) biomass, and due to the increase of senescence rates [22].

3 Methodology

This section presents MCGM and ML models and shows how the combination of the two assists in predicting and optimizing crop growth and yield.

3.1 Mechanistic Crop Growth Model

The MCGM is a mathematical description of all physiological processes throughout the growing season. It entails a dynamic simulation process of an entire crop by predicting the growth of crops' components and considers field farming interrelationships. Yearly crops grow from the date of planting to harvest time or grow until total heat units (HU) are equal to the potential HUs of the crops.

$$HU_{(i)} = \frac{(T_{max,(i)} - T_{min,(i)})}{2} - T_b, j \tag{1}$$

where $HU_{(i)}$, $T_{max,(i)}$, and $T_{min,(i)}$ represent HU values, maximum temperature (°C), and minimum temperature (°C) on day i, respectively, and $T_{b,j}$ is the base temperature specific to the crop in question, which is j (growth does not occur at or below T_b). The base temperature most commonly used for all phenological phases is 8 °C, except for silking [15]. The crop's phenological progression is dependent on daily HU accumulation. The following equation represents the relevant computation:

$$AHU_{(i+1)} = AHU_{(i)} + HU_{(i)} \tag{2}$$

where $AHU_{(i+1)}$ is the accumulated HUs on day $i + 1$. The leaf number is used as a measure of development rates and is influenced by the soil temperature and photoperiod.

$$A_{(i)} = AM \cdot e^{[-0.0344 \cdot (LN_{(i)} - LNM)^2 + 0.000731 \cdot (LN_{(i)} - LNM)^3]} \tag{3}$$

where $A_{(i)}$ is the area of an individual leaf, LN is the number of leaves, AM is the area of the largest leaf, and LNM is the leaf number with the largest area from the total number of leaves initiated. Dwyer et al. [9] suggest an exponential relationship that computes the *fraction of the total leaf that was senesced*, denoted as *FAS*, increased with HUs from emergence. In the majority of crops, the leaf area index (LAI) starts at zero or close to that. It then exponentially increases in the stage of early vegetative

growth. *LAI* is calculated as the difference between plant leaf area (*PLA*) and *FAS* multiplied by the population of plants.

$$LAI_{(i)} = \begin{cases} \text{Plant Population} \cdot (PLA - PLA \cdot FAS_{(i)}) & \text{if } LAI > 0 \\ 0 & \text{Otherwise} \end{cases} \tag{4}$$

Potential Growth

Using Beer's law equation, energy interception is approximated as a function of solar radiation and the *LAI* of the crop.

$$PHS_{(i)} = 0.5 \cdot RA_{(i)}[1 - e^{(-0.4 \cdot LAI_{(i)})}] \tag{5}$$

where $PHS_{(i)}$ is the radiation use efficiency obtained from photosynthesis on day i, *RA* is solar radiation, and 0.5 is the ratio between $PHS_{(i)}$ and $RA_{(i)}$. The possible daily increase in biomass can be approximated as the product of intercepted energy and a crop parameter for transforming energy into biomass [18].

$$BM_{(i+1)} = BM_{(i)} + PHS_{(i)} \tag{6}$$

where $BM_{(i)}$ is the daily potential accumulation in biomass and $BM_{(i+1)}$ is the daily potential increase in biomass on day $i + 1$.

$$HI_{(i+1)} = \begin{cases} 0 & \text{At the planting stage} \\ HI_{(i)} + 0.015 & \text{Otherwise} \\ 0.5 & \text{33 days after planting stage} \end{cases} \tag{7}$$

where $HI_{(i)}$ is the harvest index and $HI_{(i+1)}$ is the harvest index on day $i + 1$. The crop yield is estimated through an integrated calculation of the potential biomass and harvest index concept.

$$GRAIN_{(i)} = HI_{(i)} \cdot BM_{(i)} \tag{8}$$

where $GRAIN_{(i)}$ is crop yield.

Water Availability and Nitrogen

Soil Water Balance Soil water inputs could be from rainfall and irrigation, and could be removed by soil evaporation and crop transpiration. There are two different layers of soil water: the top layer of soil water ($SW1$) and the entire soil water layer (SW).

$$SW1_{(i)} = \begin{cases} SW1_{(i-1)} + RF_{(i)} + Irr_{(i)} & SW1 < MSW1 \\ 19.5 & \text{Otherwise} \end{cases} \tag{9}$$

$$SW_{(i)} = \begin{cases} SW_{(i-1)} + RF_{(i)} + Irr_{(i)} & SW < MSW \\ 135 & \text{Otherwise} \end{cases} \tag{10}$$

where $MSW1$ is the maximum (total) soil water in the top layer, MSW is the maximum soil water in the entire soil water, RF is the rainfall, and Irr is the irrigation. The soil water is lost due to soil evaporation (SEP) and crop transpiration (TR). There are two stages for the assessment of soil evaporation. In stage I, the SEP is evaluated using the FAO Penman–Monteith equation [2].

In stage II, SEP is triggered after the topsoil layer has dried or when the transpirable soil of the entire soil is low [3]. TR is calculated from the PHS as well as vapor pressure deficit and can be affected if the transpirable soil water (TSW) on the top volume reduces. The daily TSW in both volumes is updated by considering the daily evaporation and crop transpiration.

$$SW1_{(i+1)} = \begin{cases} SW1_{(i)} - TR_{(i)} - SEP_{(i)} & SW1 > 0 \\ 0 & \text{Otherwise} \end{cases} \tag{11}$$

$$SW_{(i+1)} = \begin{cases} SW_{(i)} - TR_{(i)} - SEP_{(i)} & SW > 0 \\ 0 & \text{Otherwise} \end{cases} \tag{12}$$

The fraction of TSW ($FTSW$) for total soil volume is computed as follows:

$$FTSW_{(i+1)} = \frac{SW_{(i+1)}}{135} \tag{13}$$

Crop Nitrogen (N) Uptake Nitrogen levels in corn range between 2.76 and 3.5%, while a level less than 2.25% N is considered insufficient. Nitrogen in crops is determined using the supply–demand method, but relatively constrains the variation in N concentrations in plant tissues. Williams et al. [30], Sinclair et al. [23], and Bennett et al. [6] use an alternative approach for nitrogen supply calculation to the crop without considering a pre-established demand function. The characteristic of this approach is that the nitrogen uptake is determined independently, and the physiological activity is estimated by taking into account the resultant N levels in the crop tissue. Sinclair et al. [23] show a linear function between the accumulated heat unit (AHU) and the N uptake for maize. The total potential N uptake (PNU) in maize is estimated by considering the daily N uptake potential rate (NUP) and the HU. When the mineral N is insufficient in the soil, the daily N uptake (NU) is equal to the amount of N available in the soil for crop uptake [23].

$$\alpha = PNU \cdot (5.24 \cdot 10^{-8}),\ \beta = AHU^{1.58} \cdot e^{[-(AHU/958)^{2.58}]}$$
$$NUP_{(i)} = \begin{cases} HU_{(i)} \cdot \{\alpha \cdot \beta\} & \text{SoilN} > 1 mgN/LH_2O \\ \text{SoilN} & \text{Otherwise} \end{cases} \tag{14}$$

N uptake depends on the $FTSW$; whenever $FTSW$ decreases to levels below a certain threshold, the value of NU considerably decreases.

$$NU_{(i)} = NUP_{(i)}/(1 + 9 \cdot e^{(-15.3 \cdot FTSW_{(i)})}) \tag{15}$$

The daily total amount of N accumulated in the grain ($GRAINN$) is estimated as the sum of NU that is transferred from the quantity of vegetative N available for translocation to seed (TVN).

$$GRAINN_{(i+1)} = GRAINN_{(i)} + NU_{(i)} + \frac{TVN \cdot HU_{(i)}}{1150} \tag{16}$$

Since the grain biomass and N accumulation are estimated separately, the grain N concentration simulation also varied; however, two constraints are set on grain N concentration (GN) that might affect grain biomass and grain N accumulation. The same applies to leaf nitrogen; if GN is less than the minimum GN (MGN), grain development will be inhibited. The total grain N is estimated as follows:

$$GN_{(i)} = \frac{GRAINN_{(i)}}{GRAIN_{(i)}} \tag{17}$$

If the grain N concentration reaches the maximum GN, N transfer to seeds will be restricted. Consequently, the restricted N transfer will lead to a significant decrease in N loss in the leaves and stems.

$$GRAINN_{(i+1)} = GRAINN_{(i)} - \frac{TVN \cdot HU_{(i)}}{3 \cdot 1150}. \tag{18}$$

3.2 Machine Learning Models

We combined ML and crop growth model approaches to get better performance results, as illustrated in Fig. 1. We chose regression ML models to forecast crop output. The ML models used are the random forest regressor (RFR) and artificial neural networks (ANN), where for ANN, the input layer consists of the soil and weather variables to predict the crop yield. In this case, we used the forward multilayer perceptron (MLP) to generate and improve results. The algorithm is trained using the backpropagation of error to adjust the weights and reduce the errors, thereby achieving non-linear regression. The training data was divided into train and validation

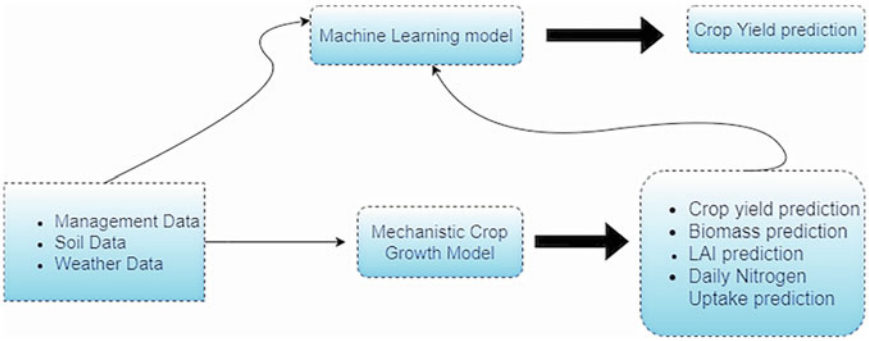


Fig. 1 Theoretical framework

subsets; the latter was utilized to construct the models. We used principal component analysis (PCA) to reduce the feature set size. We used 10-fold cross-validation for hyperparameter selection and model calibration.

Dataset

The historical data of corn yield was obtained from the USDA-NASS website for the year 1982 for a state in the US. The features used in this study include (a) **plant population**, acquired from the USDA-NASS website; (b) five **weather** variables, accumulated daily from Daymet [28]; (c) various **soil** parameters, acquired from the Web Soil Survey [24]; (d) **nitrogen** fertilizer application rates; and (e) annual **corn yield** obtained from USDA-NASS.

4 Results

These experiments first simulated the MCGM to predict corn yield based on historical data. We varied the value of the N fertilizer and selected the value that maximized yield. Next, we evaluated the performance of the ML models using the following performance metrics: RMSE, RRMSE, MAE, and R^2. Lastly, we optimized the model using the ML algorithm found to produce the least prediction error to maximize corn yield in the state of Iowa.

4.1 Nitrogen Application

We evaluated the models with different N input values to estimate the impact of nitrogen on yield prediction and found the following:

- **Low level of N**: When applying 11.6 g N/m^2 of N, we get 565.26 g N/m^2 of grain yield, the accumulation of biomass is low, and *LAI* is reduced.
- **Intermediate level of N**: When applying 27.6 g N/m^2 of N, we get 782.72 g N/m^2 of grain yield. The accumulation of biomass and *LAI* are moderated.
- **High level of N**: When applying 40.1 g N/m^2 of N, we get 712.44 g N/m^2 of grain yield. We can notice that as the level of N is high, the grain yield and accumulation of biomass are reduced, and the *LAI* becomes very high. This also impacts the environment.

The highest grain yield of 782.72 g N/m^2 was produced when we applied the intermediate level of N, 27.6 g N/m^2 fertilizer. The length of the grain growth period and leaf number remained unchanged. Higher N levels increased the *LAI*, *LFN*, *GRAINN*, and *GN* but decreased the grain yield.

4.2 ML Model Hyperparameter Configuration and Performance

Initially, we selected 10 trees for RFR to evaluate the model's performance. The MLP was trained with 20 nodes, 1 hidden layer, a learning rate of 0.01, and a rectified linear unit (ReLu) activation function, and we used a stochastic gradient-based optimizer. The results are summarized in Table 1.

4.3 Model Optimization

We applied MLP methods of optimization to maximize crop yield using the N fertilizer amount of 27.6 g N m^{-2} in the state of Iowa.

Table 1 Performance metric evaluation of the models for Iowa

Machine learning model	Error measures			
	RMSE (g/m^2)	RRMSE (%)	MAE (g/m^2)	R^2
Multilayer perceptron	0.2423	0.4252	0.1586	0.7453
Random forest	0.2470	0.4336	0.1327	0.7351

Normalization

After assessing the ideal ML model, MLP, it is necessary to optimize the factors that affect crop growth in order to maximize production. MLP data preparation involved utilizing processes such as normalization to rescale input/output variables before training the MLP model. The dataset was rescaled in a range of [0, 1] for the input value, and the output value was divided by 1000.

Parameter Selection

We selected different parameters to optimize the model.

We chose the ReLu activation function, and the following optimizers were chosen:

- Adaptive moment estimation (Adam) Optimizer: $\alpha = 0.001$, $\beta_1 = 0.9$, $\beta_2 = 0.999$, and $\epsilon = 10^{-8}$.
- Stochastic Gradient Descent (SGD) Optimizer: $\alpha = 0.01$, decay = 0.0, momentum = 0.7, and nesterov = False.

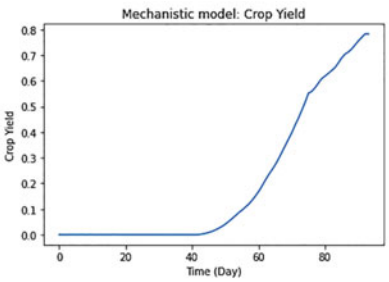
Model Simulation Scenarios

To perform the experiment, we executed different scenarios for the Adam and SGD optimizers to determine which one provides better optimization and grain yield maximization. We first started with the actual dataset, adjusted parameters individually, and then adjusted all parameters simultaneously. Table 2 shows the optimization results with Adam and SGD optimizers for each scenario.

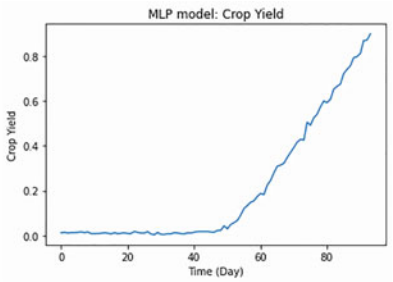
The experiment results demonstrated that the highest grain yield 896 g/m^2 occurred when using the SGD optimizer with the dataset in the unchanged parameters scenario. Figures 2a and b compare the grain yield obtained using the MCGM with the optimized grain yield using the MLP model. In conclusion, the optimization of the model using the MLP-SGD optimizer improved the yield prediction and provided a better result than the MCGM.

Table 2 Results of yield optimization with Adam and SGD optimizers

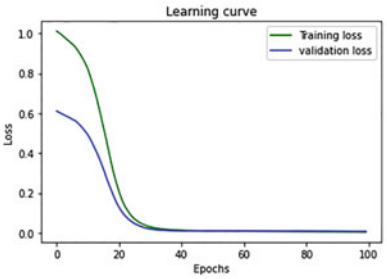
Scenario	Adam (g/m^2)	SGD (g/m^2)
Actual dataset/Unchanged parameters	865	896
Solar radiation adjusted	800	793
Precipitation adjusted	807	821
Minimum temperature adjusted	791	813
Maximum temperature adjusted	812	826
All selected parameters	820	888



(a) Yield from mechanistic model



(b) Optimized grain yield



(c) Learning curves

Fig. 2 Experimental results

Learning Curve

Figure 2c shows a good fit of the learning curves since the plot of training loss decreases to the point of stability, and the validation loss decreases to the point of stability and has a small gap with the training loss. It shows that the training and validation datasets are suitably representative. The learning performance has been done with the Epoch = 100. The mean squared error (MSE) of training loss = 0.006 and validation loss = 0.007.

5 Conclusion

We introduced an MCGM to predict corn growth and yield, and we trained ML models and selected the best one to combine with the MCGM. The MCGM was built on parameters such as weather, soil management, and plant population data that reflect the physiological process of the growing season. We applied three different nitrogen levels and observed that they significantly impacted yield prediction. Historical data and the MCGM's output were used as input to the ML algorithms. We evaluated the MLP and RFR models using RMSE, RRMSE, MAE, and R^2. We performed various scenarios and applied two MLP optimization algorithms, Adam and SGD,

to improve the MCGM yield prediction, thus maximizing the corn yield. Results showed that the combination of MCGM and MLP improved corn yield prediction. The highest grain yield occurred when using an SGD optimizer and the dataset with unchanged parameters scenario. In the future, we plan to work on four different schemes, namely: (a) investigate the feasibility of performing further data analytics on edge devices; (b) run experiments on the Area X.O (a local smart farm in Ottawa, Ontario, Canada) datasets; (c) involve various parameters such as crop disease, water salinity, and pest control, in the corn growth model and combine them with multiple ML algorithms to maximize the crop yield; and (d) implement other use cases and optimization algorithms for various parameters involved in the crop growth model.

References

1. de Wit A et al (2019) 25 years of the wofost cropping systems model. Agric Syst 168:154–167. https://doi.org/10.1016/j.agsy.2018.06.018
2. Allen RG, Pereira LS, Raes D, Smith M et al (1998) Crop evapotranspiration-guidelines for computing crop water requirements-FAO irrigation and drainage paper 56. FAO 300(9):D05109
3. Amir J, Sinclair T (1991) A model of water limitation on spring wheat growth and yield. Field Crop Res 28(1):59–69
4. Anawar MR, Wang S, Azam Zia M, Jadoon AK, Akram U, Raza S (2018) Fog computing: an overview of big IoT data analytics. Wirel Commun Mob Comput 2018. https://doi.org/10.1155/2018/7157192
5. Bell C (2016) MySQL for the internet of things, 1st edn. Apress Berkeley
6. Bennett JM, Mutti LSM, Rao PSC, Jones JW (1988) Interactive effects of nitrogen and water stresses on biomass accumulation, nitrogen uptake, and seed yield of maize. Agronomy Physiology Laboratory, University of Florida, Gainesville, FL 32611, USA
7. De Donno M, Tange K, Dragoni N (2019) Foundations and evolution of modern computing paradigms: cloud, IoT, edge, and fog. IEEE Access 7:150936–150948
8. Dumont B, Basso B, Leemans V, Bodson B, Destain JP, Destain MF (2015) A comparison of within-season yield prediction algorithms based on crop model behaviour analysis. Agric For Meteorol 204:10–21
9. Dwyer L, Stewart D (1986) Leaf area development in field-grown maize 1. Agron J 78(2):334–343
10. Elavarasan D, Vincent DR, Sharma V, Zomaya AY, Srinivasan K (2018) Forecasting yield by integrating agrarian factors and machine learning models: a survey. Comput Electron Agric 155:257–282
11. Elijah O, Rahman TA, Orikumhi I, Leow CY, Hindia MN (2018) An overview of internet of things (IoT) and data analytics in agriculture: benefits and challenges. IEEE Internet Things J 5(5):3758–3773
12. Ferrández-Pastor FJ, García-Chamizo JM, Nieto-Hidalgo M, Mora-Martínez J (2018) Precision agriculture design method using a distributed computing architecture on internet of things context. Sens 18(6):1731
13. Hu H, Wen Y, Chua TS, Li X (2014) Toward scalable systems for big data analytics: a technology tutorial. IEEE access 2:652–687
14. IoT applications in agriculture. http://www.iot.qa/2018/01/iot-applications-in-agriculture_23.html. Accessed 13 July 2022
15. Jones CA, Kiniry JR, Dyke P (1986) CERES-maize: a simulation model of maize growth and development

16. Jones JW et al (2003) The DSSAT cropping system model. Modelling cropping systems: science, software and applications. Eur J Agron 18(3):235–265. https://doi.org/10.1016/S1161-0301(02)00107-7
17. Killeen P, Kiringa I, Yeap T (2022) Unsupervised dynamic sensor selection for IoT-based predictive maintenance of a fleet of public transport buses. ACM Trans Internet Things
18. Muchow RC, Sinclair TR, Bennett JM (1990) Temperature and solar radiation effects on potential maize yield across locations. Agron J 82(2):338–343
19. Murthy VRK (2004) Crop growth modeling and its applications in agricultural meteorology. Satell Remote Sens GIS Appl Agric Meteorol 235
20. Rezk NG, Hemdan EED, Attia AF, El-Sayed A, El-Rashidy MA (2020) An efficient iot based smart farming system using machine learning algorithms. Multimed Tools Appl 80:773–797. https://doi.org/10.1007/s11042-020-09740-6
21. Omoniwa B, Hussain R, Javed MA, Bouk SH, Malik SA (2019) Fog/edge computing-based IoT (FECIoT): architecture, applications, and research issues. IEEE Internet Things J 6(3):4118–4149
22. Osakabe Y, Osakabe K, Shinozaki K, Tran LS (2014) Response of plants to water stress. Front Plant Sci 5. https://doi.org/10.3389/fpls.2014.00086
23. Sinclair TR, Muchow RC (1995) Effect of nitrogen supply on maize yield: I. Modeling physiological responses. Agron J 87(4):632–641
24. Soil Survey Staff (1982) Natural resources conservation service. United States Department of Agriculture, Web Soil Survey
25. Song Y, Birch C, Qu S, Dohert A, Hanan J (2010) Analysis and modelling of the effects of water stress on maize growth and yield in dryland conditions. Plant Prod Sci 13(2):199–208. https://doi.org/10.1626/pps.13.199
26. Stamatescu G, Drgana C, Stamatescu I, Ichim L, Popescu D (2019) IoT-enabled distributed data processing for precision agriculture. In: 2019 27th mediterranean conference on control and automation (MED). pp 286–291
27. Tao M, Hong X, Qu C, Zhang J, Wei W (2018) Fast access for ZigBee-enabled IoT devices using raspberry pi. In: 2018 chinese control and decision conference (CCDC). pp 4281–4285. https://doi.org/10.1109/CCDC.2018.8407868
28. Thornton MM, Shrestha R, Wei Y, Thornton PE, Kao S, Wilson BE (2020) Daymet: daily surface weather data on a 1-km grid for North America, version 4. ORNL DAAC, Oak Ridge, Tennessee, USA
29. Whisler FD et al (1986) Crop simulation models in agronomic systems. Advances in agronomy, vol 40, pp 141–208. Academic Press
30. Williams JR, Jones CA, Kiniry JR, Spanel DA (1989) The EPIC crop growth model. Trans ASAE 32(2):497–511
31. Wolfert S, Ge L, Verdouw C, Bogaardt MJ (2017) Big data in smart farming - a review. Agric Syst 153:69–80
32. Wu M, Lu TJ, Ling FY, Sun J, Du HY (2010) Research on the architecture of internet of things. In: 2010 3rd international conference on advanced computer theory and engineering (ICACTE), vol 5, pp V5–484–V5–487
33. Yang Z, Yue Y, Yang Y, Peng Y, Wang X, Liu W (2011) Study and application on the architecture and key technologies for IOT. In: 2011 international conference on multimedia technology. pp 747–751

Deep Learning-Based Cancelable Biometric Recognition Using MobileNetV3Small Model



Shakti Maheta and Manisha

Abstract Cancelable Biometric techniques are used to stop the forgery and database breaches that happened in the traditional biometric system. A deep learning-based MobileNetV3Small model for the generation of cancelable biometrics templates is proposed in this work. Numerous experiments have been accomplished on different gray datasets, i.e., ORL Face and FingerPrint Verification Challenge (FVC2002). For color datasets, experiments are carried out on IIT Delhi iris and AMI Ear datasets. The proposed method is also satisfying the Diversity, Revocable, Non-invertible, and Performance characteristics of cancelable biometric systems. The performance is measured in terms of Accuracy. A comparison in terms of performance between the proposed cancelable biometric system and the traditional biometric system is also presented in the same work. With IIT Delhi iris and AMI ear, color datasets resultant accuracy is 100%. In ORL and Fingerprint gray datasets, the accuracy achieved is 92.50% and 96.87%, respectively. Also, the experimental results show that the proposed method outperforms the other compared methods in terms of performance measures.

Keywords Cancelable biometrics · MobileNetV3Small · Accuracy · Diversity · Revocable · Performance

1 Introduction

A biometric-based system is used to authenticate a person and counteract any potential security concerns. To verify the identification of people seeking their services, many systems require reliable personal authentication procedures, e.g., access control, authentication transactions, etc. Other application areas where authentication

Marwadi University

S. Maheta · Manisha (✉)
Marwadi University, Rajkot, Gujarat, India
e-mail: manisharawat615@gmail.com

© The Author(s), under exclusive license to Springer Nature Singapore Pte Ltd. 2023

S. Chinara et al. (eds.), *Advances in Distributed Computing and Machine Learning*,
Lecture Notes in Networks and Systems 660,
https://doi.org/10.1007/978-981-99-1203-2_29

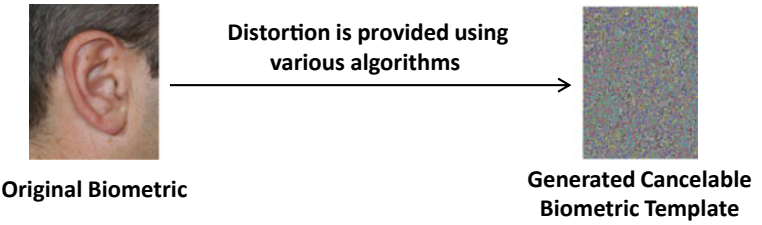


Fig. 1 Cancelable biometric template corresponding to AMI ear image

requires can be automatic teller machines (ATMs) in which user has to present his/her ATM card with a password or key by which access is provided to the user. If the combination of user card and password is wrong then the user is denied by the system. Nowadays, various electronic gadgets, e.g., mobile phones and laptops are also using biometric credentials. Before using the electronic gadgets, user has to present his/her face, iris, and fingerprint biometrics to make an entry into the respective device. In the lack of reliable authentication protocols, these systems are vulnerable to impostor techniques. These solutions are made to ensure that the given services are only accessible by the authenticated users and no one else. *Cancelable Biometric* is used to impose security in the existing conventional biometric system. In other words, traditional biometric system suffers from security and privacy attacks. Due to the storage of original features and images in the database, the probability of forgery and compromisation of original features from database is always at high risk. In cancelable biometrics, the distorted or deformed features are recorded in the database instead of the original features as shown in Fig. 1. The cancelable biometric system possesses four characteristics, i.e., (i) Diversity, (ii) Non-invertible, (iii) Revocable, and (iv) Performance. A complete guide about cancelable biometric is present in the research works [17, 18]. Recently, to improve the recognition accuracy in cancelable biometric-based system, few researchers have proposed deep learning-based techniques. As this is a newer area of investigation in the domain of cancelable biometric, few research works have been discussed in the next section.

2 Related Work

The research work proposed by Phornchaicharoen et al. [2] has used transfer learning for extracting the face biometric features. Further, a Multilayer Perceptron (MLP) model for image classification. Deep learning requires a huge amount of data. When the data size is small it would not give an accurate result. The proposed work is divided into three steps, i.e., (i) Face Detection, (ii) Feature Extraction, and (iii) Face Recognition. In face detection phase, different facial regions, i.e., eyes, nose, and mouth are detected. For feature extraction phase, transfer learning is used for extracting the different features from different facial regions. Finally, facial recog-

nition phase consists of the matching between the query and the stored features. The results are obtained on the YouTube Face and Extended Cohn-Kanade Datasets. The result outcomes are presented in terms of entropy, accuracy, and computation time. The matching score will determine if the system permits the queried user or not. A convolutional neural network (CNN)-based cancelable biometric system is present in the research work [1]. The proposed technique first applies max-out units which further using weighted concatenation. It combines the discriminative features of two modalities. The integrated representation of periocular and iris biometrics is improved by simultaneously refining convolutional filter parameters and fusion weights. For experimental work, the CASIA-Iris-Mobile-V1.0 dataset is used. The research work [3] proposed a method which extracts the important features from the images and performs tasks like cropping the image, rotation strategies, and simplification by using CNN. It classifies the facial expression recognition methods using image sequence and static image. The research work performed by Talreja et al. [4] integrates a deep hashing technique with fusion architecture and generates a binary multimodal which is robust. It is a hybrid architecture which combines the cancelable biometrics along with secure sketch architecture and then it will be combined with the deep hashing framework. The method is used for multiple biometrics or multimodal biometric system. It utilizes the hashing framework for secure multibiometric systems which used feature-level fusion. In the research work [6], a multimodal unified template technique is proposed. The method ensures security, privacy as well as high performance for the generated cancelable features which uses deep neural network. It is basically a graph-based fusion method. The proposed method uses adaptive graph-based fusion technique which uses mixed features of iris and periocular biometrics. For features extraction process, Residual Networks (ResNet) model has been used. Firstly, seperate features from the iris and periocular biometrics have been extracted. Next, Random projection has been performed on these selected features separately. Further, Deep Feature Unification (DFU) is processed which consists of Normalized Graph Construction and Optimal Graph Construction methods which are applied to the extracted features separately. In normalized graph construction, the normalization of query features is performed. Further, the optimal features from normalized features are obtained. Finally, the unification process is completed by integrating the best features from iris and periocular. The templates retrieved after this unification process are cancelable templates which are further used for authorization purpose. In the research work proposed by Abdellatef et al. [21], cancelable iris and face templates are generated for an individual. This work is based on the CNN model along with bio-convolution method. Firstly, CNN model takes biometric image as an input and then the feature extraction is performed. After extracting important features, classification of the images will be done using Support Vector Machine (SVM). Further, bio-convolving method is used for generating cancelable biometric template. For experiments, four different datasets have been used. On the LFW dataset, 99.15% accuracy is achieved, while, for FERET, accuracy is 98.35%. On the IITD iris dataset, accuracy is 97.89% and the third version of CASIA Iris, accuracy is 95.48%.

3 Proposed Methodology

This research work includes the novel algorithm for the generation of cancelable biometric template using deep learning approach. In this work, we have used MobileNetV3Small model with pre- and post-processing steps. A depiction of these steps is also provided in Fig. 2. At first, *data cleaning* and *data splitting* process is applied to the datasets as a part pre-processing steps. The removal of noise or duplicated data from the dataset is known as the *Data Cleaning*, while data splitting divides the dataset into train and test datasets. For the proposed work, a 70:30 ratio is used as a data splitting. Further MobileNetV3Small model is used in this work, a detailed explanation of this model is given later in the subsection. For generating the cancelable biometric templates for various datasets, *Gaussian-based Random Projection* is used. Further, Random Forest [10] is used as classifier in this work. For measuring the performance of the proposed work, accuracy metric with confusion matrix is used.

- **Input Data**: The input to this step is the gray and color datasets. In this proposed work, we have used different datasets like AMI Ear, IIT Delhi iris, FVC2002, and ORL dataset is used.
- **Data Cleaning**: Data cleaning is generally associated by removing the duplicate and incorrect data or noise from the datasets.
- **Data Splitting**: Data splitting is dividing the data into subparts. Here, the data is split into two parts, i.e., Training and Testing. In the proposed work, 70% of

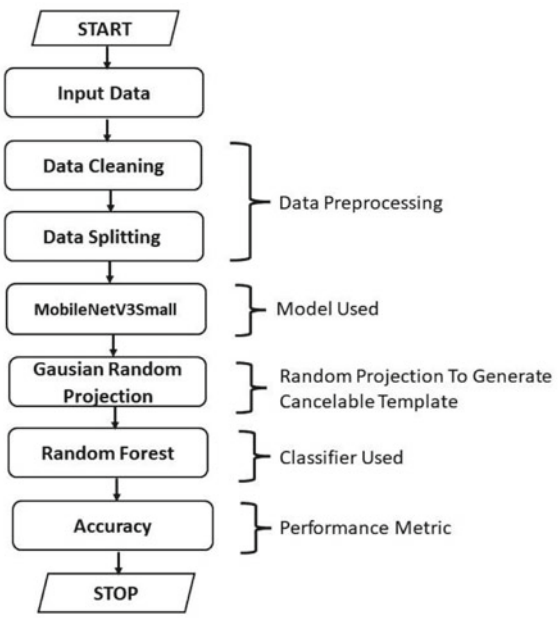


Fig. 2 Step-by-step detail of the proposed methodology

the total data is used for training purpose, while 30% of the total data is used for testing purpose. And based on that the accuracy will be calculated.

- **Model**: The proposed method is using the model MobileNetV3Small because of some advantages of the model like, Accuracy, Efficiency, Easy to use, Lightweight, etc. Along with the model we are using Random Forest Classifier to classify the data which gives the most accurate results.
- **Cancelable Biometric Template Generation Using Random Projection**: Random projection is a method used in mathematics and statistics to reduce the dimensionality of a collection of Euclidean space-based points. When compared to other approaches, random projection methods are renowned for their strength, simplicity, and low error rates. Random projection effectively preserves distances, according to experimental findings, although empirical evidence is scant. In the proposed work, Gaussian Random Projection has been used.
- **Accuracy as a Performance Metric**: The performance of the proposed work has been measured using accuracy metric. This shows how many genuine users are recognized as genuine and imposters have been restricted in the system.

3.1 MobileNet

With improved classification accuracy and fewer parameters, MobileNet is a compact deep neural network [12]. In order to further reduce the number of network parameters and improve classification accuracy, MobileNet uses dense blocks from DenseNets as mentioned by Zhang et al. [13]. In Dense-MobileNet models, convolution layers that are the same size as the input feature maps are employed as dense blocks and dense connections are carried out within the dense blocks [14]. In order to build a high number of feature maps with fewer convolution cores and continually employ the features, the new network structure may fully utilize the output feature maps produced by the preceding convolution layers in dense blocks. The network can further lower the parameters and calculation costs by choosing a low growth rate. Dense1-MobileNet [7] and Dense2-MobileNet [7] are two Dense-MobileNet models that have been developed. Studies reveal that Dense2-MobileNet can perform recognition tasks more accurately than MobileNet while using less compute power and parameters. Deep neural networks such as convolution neural networks (CNN) [15] are frequently employed in picture classification. It extracts picture characteristics through several convolution layers. The use of neural networks for mobile terminals has grown more widespread as a result of the growth in the amount of picture data that mobile devices are processing. It is challenging to adapt these networks to mobile devices since they require advanced hardware support and huge computing.

3.2 *Versions of MobileNet*

Till date, three versions of MobileNet are available: (i) *MobileNetV1*, (ii) *MobileNetV2*, and (iii) *MobileNetV3*. Further, MobileNetV3 is categorized into two parts, namely: (i) *MobileNetV3Large* and (ii) *MobileNetV3Small*. concepts in the deep learning field [7]. In 2017, the first version of MobileNetV1 has launched. **MobileNetV1** model is built on TensorFlow which aims to increase the accuracy. The main two key objectives of its launch are to reduce the model size which in turn reduces the time and space complexity of the model. The main application area of MobileNetV1 is the computer vision which may be further extended to segmentation, embeddings, detection, and classification. This model is built on the foundation of compact deep neural networks using depth-wise separable convolutions. Each input channel in MobileNetV1 receives a single filter. The outputs of the depth-wise convolution are combined using 11 convolutions after the pointwise convolution. This is divided into two layers by the depth-wise separable convolution, i.e., (i) a layer for combining and (ii) a layer for filtering. The TensorFlow-Slim Image Classification Library contains the initial MobileNetV1 implementation. As new mobile apps were developed utilizing this new paradigm, fresh suggestions for enhancing the overall architecture surfaced. **MobileNetV2** is launched in year 2018, which includes the categorization, object identification, and semantic segmentation in its earlier version. The new keypoint in the architecture of the MobileNetV2 model is that (i) layer architecture is linear and (ii) links between the layered architecture are fast, which results in increased accuracy in comparison to the existing model. A new version which launches in the year 2019 is known as **MobileNetsV3**. The main application area of MobileNetV3 is its use in Automatic Machine Learning (AutoML). In this proposed work, we have worked on MobileNetV3Small. Next, a detailed explanation of this model is presented (Fig. 3).

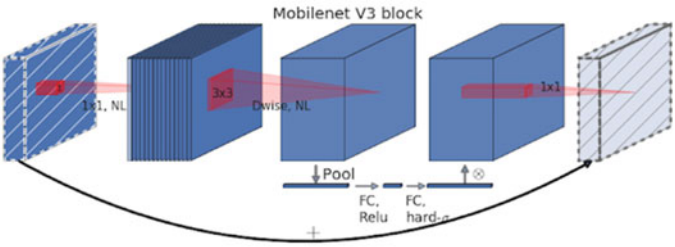


Fig. 3 Inside working of the MobileNetV3 model architecture [8]

3.3 MobileNetV3Small

MobileNetV3Small [9] is the latest version from the MobileNetV3 series. This mainly includes two AutoML approaches, namely: (i) MnasNet [16] and (ii) NetAdapt [19]. This version also includes the *Squeeze and excitation block*. MnasNet is one of the types of Convolution Neural Network (CNN), that is optimized for the use in mobile devices. However, *optimization* is achieved by decreasing the *latency time* from the earlier version. The term latency means the time taken to go from one layer to another layer. The model also uses NetAdapt to fine-tune the design of the model by continuously cutting the under-utilized activation channels. The main goal to used sequeeze and excitation block is to enhance the quality of the features that are represented by the network. In other words, it can be said that this block is used to show the relationships between the channels of the network's convolutional features by suppressing the less informative features. The main difference between MobileNetV3 and MobileNetV2 is that by using MnasNet, NetAdapt, and Sequeeze & excitation block, MobileNetV3 version reduces the latency time by approx. 25% in object identification field by comparison to the earlier version.

3.4 Gaussian Random Projection

In random projection, firstly, the various biometric features are extracted using various algorithms. Further, a random matrix is generated by using various methods, i.e., uniform distribution or normalized distribution. Next, the extracted features are projected on this random matrix which generates the transformed features. In Gaussian Random Projection, this projection matrix is obtained using Gaussian distribution in which the mean value is zero. Further, sector random projection, sparse random projection, and dynamic random projections are the various alternative methods for the generation of random matrix. By using any of the above-discussed matrices, distorted or cancelable biometric templates can be generated.

3.5 Random Forest

The foundation of the random forest is the decision tree. In decision tree, internal nodes are used to represent the condition or test on attributes. However, output and class labels are denoted on different branches and leaf nodes, respectively. The basic idea of random forest is to combine different decision trees where each tree is used to give the classification for different categories. The category which is having more number of vote is selected by the forest. Random forest is also used to make the

regression in which the average or mean of all the trees is calculated. In other words, it can be said that random forest is easy to use, effective, accurate, versatile, and more user-friendly [9, 10].

4 Experimental Results and Discussion

Numerous experiments have been carried out on various datasets. For gray datasets, FVC2002 and ORL face have been used, while color experimentations have been performed on AMI ear and IIT Delhi Iris datasets. Experiments have been performed on Intel i7, 10th-generation processor-based system which also consists of 16GB RAM and Window 10 operating system. Further, simulation work has been performed using Python 3.7.14. The results achieved on various datasets have been depicted in Table 1. It can be seen from the table that training and testing results for gray datasets have a gap in terms of accuracy, while, for color datasets, accuracy is 100% in both the cases. In other words, for color datasets, the cancelable biometric-based recognition system accuracy is same in comparison to the traditional biometric recognition system. A comparison in terms of recognition accuracy among both cancelable and traditional biometric systems is depicted in Table 2. And, proposed work is also compared with the state-of-the-art methods in terms of accuracy in Table 3. The results in the table also depicted that the proposed method is good in terms of performance accuracy than others.

Table 1 Performance in terms of Accuracy on Different gray and color Datasets

Dataset	Training accuracy on cancelable template (%)	Testing accuracy on cancelable template (%)
ORL Face (Gray)	100	92.50
FVC2002 (Gray)	100	96.87
IIT Delhi (Color)	100	100
AMI Ear (Color)	100	100

Table 2 Comparison in terms of recognition accuracy by cancelable biometric and traditional biometric-based recognition system on various gray and color datasets

Dataset	Training accuracy on original images (%)	Testing accuracy on original images (%)	Training accuracy with cancelable biometric (%)	Testing accuracy with cancelable biometric (%)
ORL	100	96.67	100	92.50
FVC2002	100	98.95	100	96.87
IIT Delhi	100	100	100	100
AMI Ear	100	100	100	100

Table 3 Accuracy comparison with state-of-the-art methods

Method	Highest accuracy (%)
Walid et al. [5]	92.58
Essam et al. [11]	95.42
Veeru et al. [4]	96.33
Jin et al. [3]	97.38
Phornchaincharoen et al. [2]	99.93
Proposed method	100

5 Conclusion and Future Work

The proposed research work presents a novel deep learning-based MobileNetV3Small model is used in the cancelable biometric recognition domain. Numerous experiments are carried out on various gray and color datasets whereas accuracy is used to gauge performance. The results obtained from various experiments show that, on color datasets, training and testing accuracies are better and same, while, on gray datasets, the gap between both accuracies is present. Also, in comparison to traditional biometric system, a cancelable biometric system also exhibits same accuracy for color datasets, while, for gray datasets, accuracy is to be improved. Similarly, the proposed method also performed well in comparison to other methods. Further, we will attempt to improve the accuracy on gray datasets and test the model on more available datasets in future work.

Acknowledgements One of the authors, Dr. Manisha, is thankful to Marwadi University, Rajkot, India, for providing financial support towards this research work (Sanction No. MU/R&D/21-22/MRP/FTO1 dated 05-02-2022).

References

1. Zhang Q, Li H, Sun Z, Tan T (2018) Deep feature fusion for iris and periocular biometrics on mobile devices. IEEE Trans Inf Forensics Secur 13(11):2897–2912
2. Phornchaicharoen A, Padungweang P (2019) Face recognition using transferred deep learning for feature extraction. In: 2019 joint international conference on digital arts, media and technology with ECTI northern section conference on electrical, electronics, computer and telecommunications engineering (ECTI DAMT-NCON). IEEE, pp 304–309
3. Li K, Jin Y, Akram MW, Han R, Chen J (2020) Facial expression recognition with convolutional neural networks via a new face cropping and rotation strategy. Vis Comput 36(2):391–404
4. Talreja V, Valenti MC, Nasrabadi NM (2020) Deep hashing for secure multimodal biometrics. IEEE Trans Inf Forensics Secur 16:1306–1321
5. El-Shafai W, Mohamed FAHE, Elkamchouchi HM, Abd-Elnaby M, Elshafee A (2021) Efficient and secure cancelable biometric authentication framework based on genetic encryption algorithm. IEEE Access 9:77675–77692

6. Walia GS, Aggarwal K, Singh K, Singh K (2020) Design and analysis of adaptive graph based cancelable multi-biometrics approach. IEEE Trans Dependable Secur Comput
7. Wang W, Li Y, Zou T, Wang X, You J, Luo Y (2020) A novel image classification approach via dense-MobileNet models. Mob Inf Syst 2020
8. Howard A, Sandler M, Chu G, Chen LC, Chen B, Tan M, Wang W, Zhu Y, Pang R, Vasudevan V, Le QV, Adam, H (2019) Searching for mobilenetv3. In: Proceedings of the IEEE/CVF international conference on computer vision, pp 1314–1324
9. Kumar S, Sahoo G (2017) A random forest classifier based on genetic algorithm for cardiovascular diseases diagnosis (research note). Int J Eng 30(11):1723–1729
10. Breiman L (2001) Random forests. Mach Learn 45(1):5–32
11. Abdellatef E, Ismail NA, Abd Elrahman SES, Ismail KN, Rihan M, El-Samie A, Fathi E (2020) Cancelable multi-biometric recognition system based on deep learning. Vis Comput 36(6):1097–1109
12. Qin Z, Zhang Z, Chen X, Wang C, Peng Y (2018) Fd-mobilenet: improved mobilenet with a fast downsampling strategy. In: 2018 25th IEEE international conference on image processing (ICIP). IEEE, pp 1363–1367
13. Zhang K, Guo Y, Wang X, Yuan J, Ding Q (2019) Multiple feature reweight densenet for image classification. IEEE Access 7:9872–9880
14. Trockman A, Kolter JZ (2021) Orthogonalizing convolutional layers with the cayley transform. arXiv preprint arXiv:2104.07167
15. Gu J, Wang Z, Kuen J, Ma L, Shahroudy A, Shuai B, Chen T (2018) Recent advances in convolutional neural networks. Pattern Recognit 77:354–377
16. Shah P, El-Sharkawy M (2020) A-MnasNet: augmented MnasNet for computer vision. In: IEEE 63rd international midwest symposium on circuits and systems (MWSCAS) 2020. IEEE, pp 1044–1047
17. Manisha, Kumar N (2020) Cancelable biometrics: a comprehensive survey. Artif Intell Rev 53(5):3403–3446
18. Rathgeb C, Uhl A (2011) A survey on biometric cryptosystems and cancelable biometrics. EURASIP J Inf Secur 2011(1):1–25
19. Yang TJ, Howard A, Chen B, Zhang X, Go A, Sandler M, Adam H (2018) Netadapt: platform-aware neural network adaptation for mobile applications. In: Proceedings of the European conference on computer vision (ECCV). pp 285–300
20. Pillai JK, Patel VM, Chellappa R, Ratha NK, Secure and robust iris recognition using random projections and sparse representations. IEEE Trans Pattern Anal Mach Intell 33(9):1877–1893
21. Abdellatef E, Soliman RF, Omran EM, Ismail NA, Abd Elrahman SE, Ismail KN, El-Samie FEA (2022) Cancelable face and iris recognition system based on deep learning. Opt Quantum Electron 54(11):1–21

Performance-Based Evaluation for Detection and Classification of Breast Cancer in Mammograms



Dakshya Prasad Pati and Sucheta Panda

Abstract The breast cancer is considered as one of the deadly disease since last three decades. Among several types of cancer, breast cancer causes death of millions of women in every year. The effective death rate of breast cancer is still in increasing order irrespective of having assistance from highly developed computational tools and techniques. Sometimes, noise or micro-calcification in mammary gland in breast misleads the detection procedure. In this research, the enhanced mammogram images are segmented and the intensity of segmentation is compared using both conventional and optimal threshold techniques to identify the affected breast tissue or dense lesion patch on breast. The performance of both threshold segmentation technique is evaluated by comparing the obtained results using some segmentation score parameters as accuracy, DICE, sensitivity specificity, and Matthews correlation coefficient (MCC). The performance analysis recommends that optimum threshold is better than conventional threshold segmentation method, which can be implemented for computer aided detection (CAD) for detection abnormal patches in low contrast mammogram images toward classification of normal breast to benign or malignant types.

Keywords Threshold · CAD · Specificity · MCC · DICE

1 Introduction

Breast cancer is a common death cause among women next to the lung cancer, skin cancer. The death rate in women is about 100 times that in men due to late detection. Since the 1980s, mammography technique has been used for early detection of breast abnormalities in medical imaging using automated CAD [3, 5] system. In this

D. P. Pati (✉) · S. Panda
Department of Computer Application, Veer Surendra Sai University of Technology, Burla 768018, Odisha, India
e-mail: dakshyaprasad@yahoo.com

S. Panda
e-mail: suchetapanda_mca@vssut.ac.in

© The Author(s), under exclusive license to Springer Nature Singapore Pte Ltd. 2023

S. Chinara et al. (eds.), *Advances in Distributed Computing and Machine Learning*, Lecture Notes in Networks and Systems 660,
https://doi.org/10.1007/978-981-99-1203-2_30

technique, a low intensity X-ray system was used for screening of the breast tissue at different angles to detect and diagnosis of breast disease in women. The detection and prediction of breast abnormality depends upon the efficient segmentation technique like thresholding, region growing [4], watershed transform, etc. The accuracy of segmentation score affect due to noise on image during image acquisition which require additional enhancement pre-processing methods [8, 9].

In some mammogram images, both normal and malignant [6, 12] are associated with dense tissue. So that contrast of both cases is not possible only through applying thresholding image segmentation technique [4, 11]. Therefore better understanding of information is required for detection of massive region of cancerous tissue in breast which help to identify the tumor cells as well as its region analysis by segmentation. Many sophisticated computational techniques are used for segmentation of mammogram, still no one of the solutions is able to satisfy the prognosis criteria including cancerous lesion detection successfully [2, 7].

In this paper, we compare conventional thresholding image segmentation technique with optimal thresholding image segmentation technique by considering parameter-like accuracy, sensitivity, specificity, DICE, etc.

2 Materials and Methodologies

2.1 Datasets

For the investigation purpose of the current work, a set of selective mammogram image samples were collected from the popular mini-Mammographic Image Analysis Society (MIAS) database of mammograms (Ref. Table 1 and Fig. 1). The images selected have a uniform resolution of 10254 × 1024 pixels. All the selected images were analyzed for finding breast tissue abnormalities in the mammogram images [1, 8–10].

Table 1 Image type with observe abnormality

Image no.	Type	Abnormality
Img1	Benign	Dense-glandular
Img2	Benign	fatty
Img3	Benign	Dense-glandular
Img4	Malignant	fatty
Img5	Malignant	Dense-glandular
Img6	Malignant	Fatty-glandular

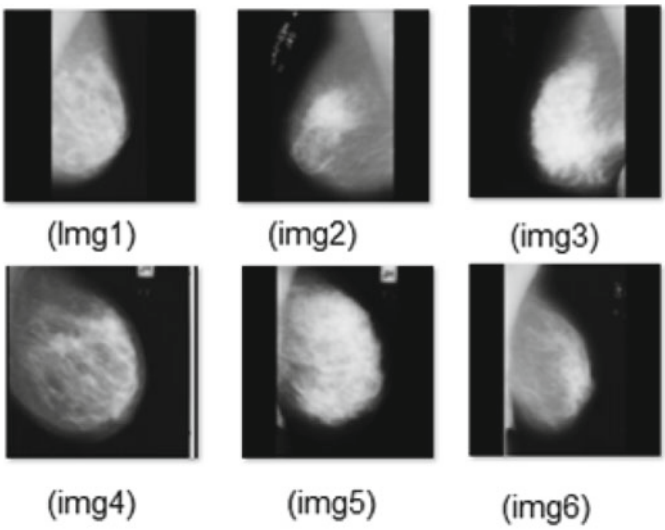


Fig. 1 Original images (Benign: Img1 to Img3, Malignant: Img4 to Img6)

2.2 Methodology

The analysis algorithm for the current work was developed using the MATLAB framework to investigate breast tissue abnormality in mammogram images. In the proposed computational model, first the selected mammogram images of 1024×1024 pixels were imported for analysis. Then, in the second phase, the imported images are compressed to 300×300 pixels for convenience of computational analysis. In the third phase, the noise content of the images obtained in the second phase was decreased by applying an average filter. The fourth phase converts the filtered images obtained from the third phase into two separate classes of grayscale images with 8-bit and 16-bit respectively. In the fifth phase, the different classes of images obtained in the fourth phase are subjected to two different thresholding segmentation methods (conventional and optimal). In the sixth and final phase, the output from fifth phase is subjected to analysis for better segmentation performance during the early detection of abnormality in breast tissues of mammogram images.

Thresholding Method Thresholding is a simple image segmentation technique based on intensity or brightness, which classifies the normal and abnormal tissue types in a mammogram image. A particular threshold value is considered for finding the separation between the black (background) region and white (foreground) regions of the mammogram images. If the threshold value is greater than the normal intensity value of the pixel of a mammogram image, it is considered a foreground part, otherwise it is a background part.

A threshold (T) value may be global (conventional) or optimum used for different segmentation purposes. The global threshold value is unable to separate both the foreground and background regions in low contrast mammogram images. Before the use of thresholding techniques for low contrast images, enhancement in the intensity level of pixel is essential [1, 8–10], which occurs through the addition of extra intensity, resulting in greater brightness values that help in marking the non-overlapping peaks for clarity of observation and understanding. This process results in better performance of specific image processing frameworks. However, our proposed approach aims at performance analysis of conventional and optimum thresholding techniques for detecting abnormalities in the breast tissue from mammogram images.

Conventional Thresholding Technique The segmentation task in image processing is the initial step for image pre-processing for useful information extraction leading to further analysis. Segmentation frameworks can be broadly classified into two types, edge and region-based frameworks. As a segmentation technique, conventional thresholding is used in several areas of application such as pattern detection, text digitization, and intelligent vision. Conventional thresholding as a pre-processing technique is widely used for image segmentation for effective image processing and analysis such as feature detection and analysis. This approach searches for T value in a segmented image that minimizes the intra-class discrepancies and provides better performance when the original image has distinct background and foreground peaks in the histogram. The T value of an image is established by traversing the entire range of the pixel values to find the minimum intra-class variance. The larger variance value of the foreground or background determines the T value. In the current work, the images are segmented by considering both minimum and maximum values of T based on histogram values (Figs. 2, 3, 4, 5 and 6). To find the appropriate T value of different masked segmented images (Fig. 3) is generated.

Optimum Thesholding Segmentation Technique The optimal thresholding is determined by a weighted histogram of the image with the sum of two or more

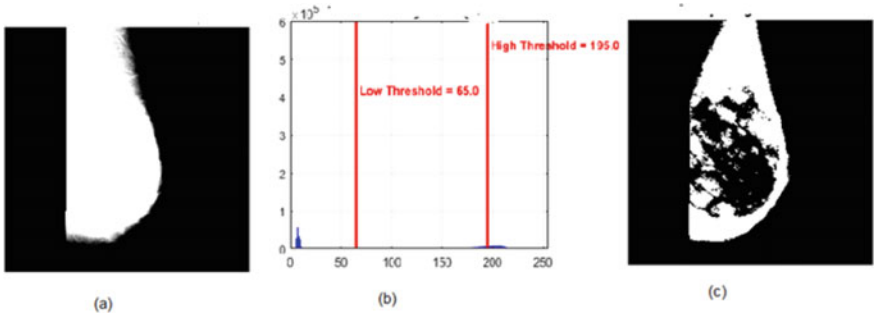


Fig. 2 The conventional thresholding segmentation technique results in benign integer type mammogram image. **a** Original gray scale image. **b** Histogram of original image. **c** Binary image or "Mask"

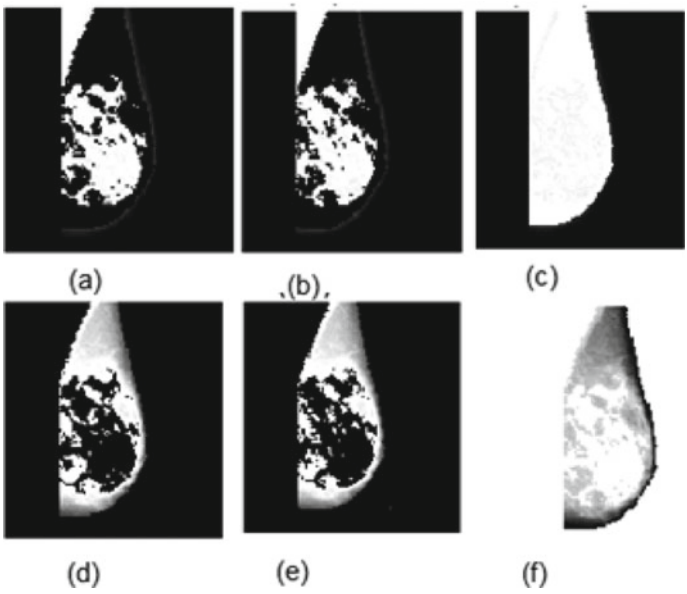


Fig. 3 Conventional thresholding technique results using different masking technique (Integer Type Benign Image) **a** Zero value inside the mask. **b** Min value (0, 0) inside the mask. **c** Max value (195) inside the mask. **d** Zero value outside the mask. **e** Min value (0, 0) outside the mask. **f** Max value (195) outside the mask

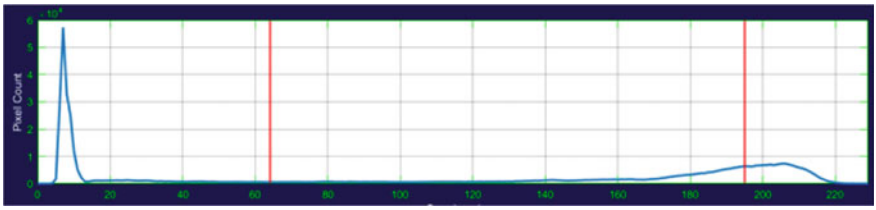


Fig. 4 Plotted graph between pixel count verses gray level (integer type image)

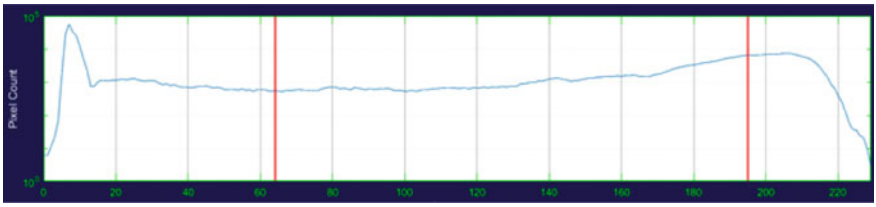


Fig. 5 Plotted graph between log pixel count verses gray level (integer type image)

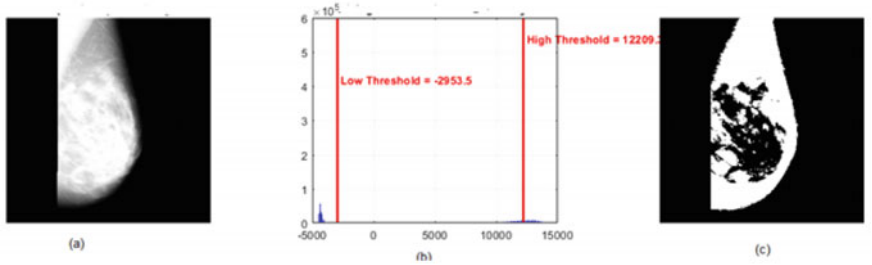


Fig. 6 Conventional thresholding segmentation technique results in benign floating type mammogram image. **a** Original gray scale image. **b** Histogram of original image. **c** Binary image or "Mask"

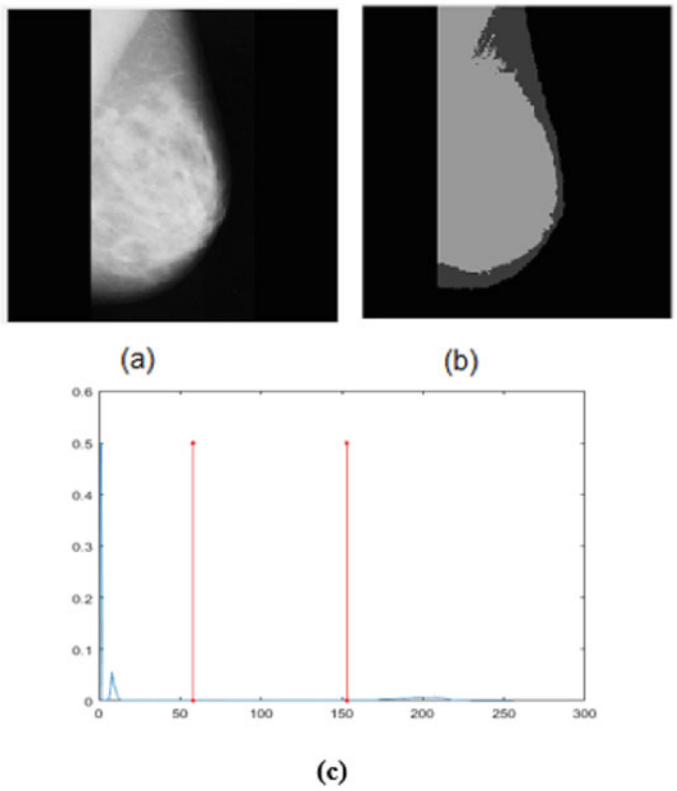


Fig. 7 Optimal thresholding segmentation technique results on benign type mammogram image. **a** Original gray scale image. **b** Segmented image. **c** Histogram of original image

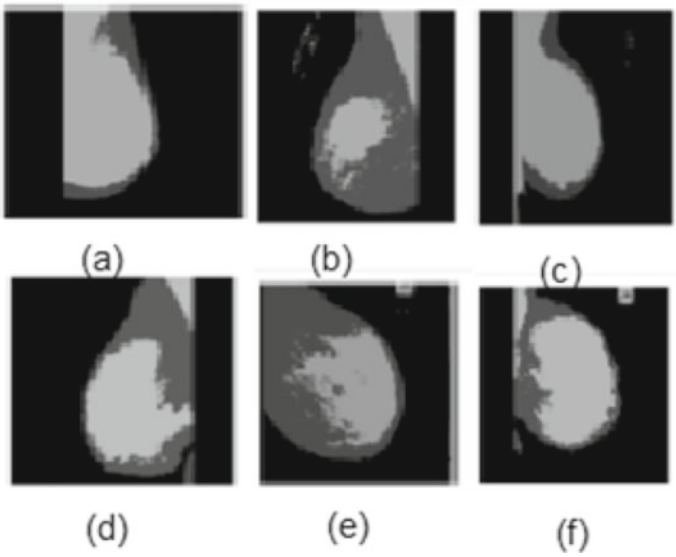


Fig. 8 Optimal thresholding segmented images (Benign Type: **a**–**c**, Malignant Type: **d**–**f**)

probability densities. The optimum thresholding technique produces better results than the conventional thresholding technique (Ref. Figs. 7 and 8).

The probability distribution of Gray Level histogram ('P_i') on image is defined as

$$P_i = \frac{n_i}{N} \tag{1}$$

where n_i = No. of pixel N = Total no. of pixel in the image.

3 Results and Discussion

In the current work, the popular programming tool, MATLAB is used to implement the computational analysis framework. It is a widely used tool in the image processing domain. The experiment was conducted on six mammogram images taken from the mini-Mammogram Image Analysis Society (MIAS) database.

3.1 Performance Metrics and Results

The simulated results are evaluated for better performance using a set of image quality measures like Accuracy, Sensitivity, Specificity, Precision, F-Measure, Matthews Correlation Coefficient (MCC), Dice Coefficient (Dice), Jaccard Coefficient (Jaccard).

The Accuracy measure represents how well the segmentation output matches with human perception. It can be calculated using the following equation:

$$Accuracy = \frac{(TP + TN)}{(FN + FP + TP + TN)} \tag{2}$$

The sensitivity measure denotes the percentage of the correctly predicted and classified positive class. It is also termed as Recall. It can be calculated using the following equation:

$$Sensitivity = \frac{TP}{(TP + FN)} \tag{3}$$

Specificity calculates the percentage of the correctly predicted and classified negative class. This is also termed as True Negative Rate. It can be calculated using the following equation:

$$Specificity = \frac{TN}{(TN + FP)} \tag{4}$$

Precision denotes the probability of getting the valid results. In other words, it represents the quality of the predictions made. It can be calculated using the following equation:

$$Precision = \frac{TP}{(TP + FP)} \tag{5}$$

F-measure is used to signify the accuracy of a test as the harmonic mean (average) of the precision and recall. It can be calculated using the following equation:

$$Fmeasure = \frac{2 \times TP}{2 \times TP + FP + FN} \tag{6}$$

DICE coefficient measures the number of correctly classified pixels as abnormal tissue, which can either be benign or penalize the incorrect classification (FP or FN).

$$DICE = \frac{2 \times TP}{2 \times TP + FP + FN} \tag{7}$$

Jaccard coefficient is used to determine how similar sample sets of images are. Jaccard index is zero if the two sets are disjoint, i.e., they have no common members, and is one if they are identical.

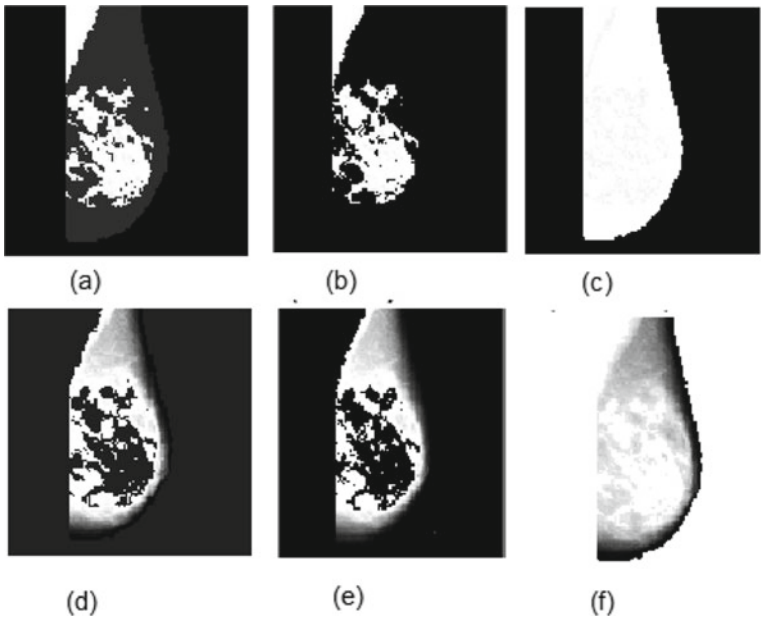


Fig. 9 Conventional thresholding technique results using different masking techniques (Floating Type Benign Image). **a** Zero value inside the mask. **b** Min value (−5000.0) inside the mask. **c** Max value (15000.0) inside the mask. **d** Zero value outside the mask. **e** Min value (−5000.0) outside the mask. **f** Max value (15000.0) outside the mask

$$Jaccard = \frac{Dice}{(2 - Dice)} \tag{8}$$

MCC is a balanced approach to find the correlation coefficient between expected target and predicted result. It varies in the range of −1 and +1. MCC is 0 when the prediction appears random with respect to the actual values.

$$MCC = \frac{(TP \times TN - FP \times FN)}{2 \times TP + FP + FN} \tag{9}$$

The final results of both types of threshold segmentation techniques (conventional and optimal) are shown in Figs. 12 and 8. The different masking and processing of sample images using conventional segmentation methods are shown in Figs. 2, 3, 4, 5, 6, 7, 8, 9, 10, and 11 for both integer and floating types of images (benign or malignant).

The outcome of the current research work can be outlined as follows.

1. The performance of both types of segmentation techniques are compared and analyzed using various parameters based on segmentation scores as shown in Tables 2 and 3.

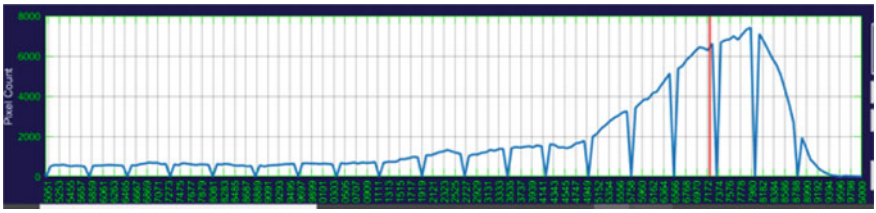


Fig. 10 Plotted Graph between Pixel count verses Gray level (Floating Type Image)

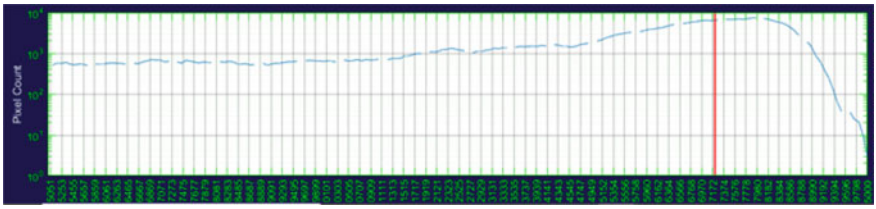


Fig. 11 Plotted Graph between Log Pixel count verses Gray level (Floating Type Image)

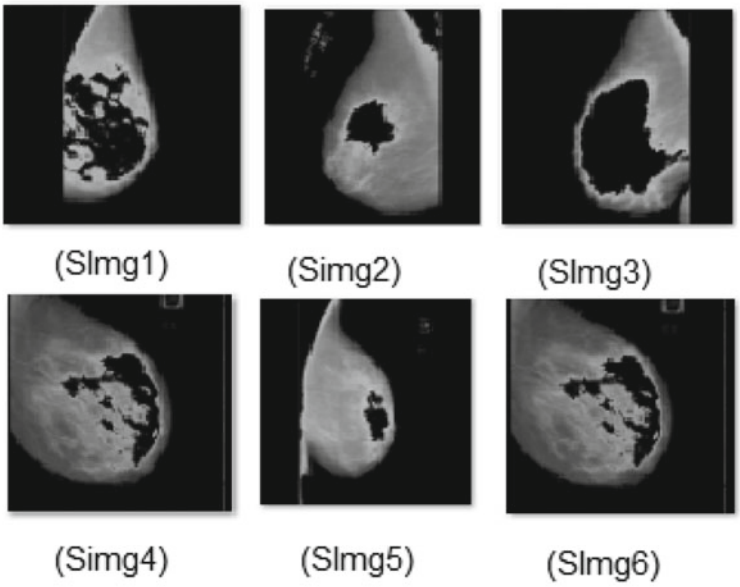


Fig. 12 Conventional thresholding segmented images (Benign Type: SImg1 to SImg3, Malignant Type: SImg4 to SImg6)

Table 2 Segmented image score results of conventional threshold segmentation technique

Si. No.	Accuracy	Dice	F-measure	Jaccard	Precision	Sensitivity	Specificity	MCC
Img1	0.7225	0.6101	0.6101	0.4564	0.6167	0.5738	0.7018	0.4684
Img2	0.7608	0.6531	0.6531	0.4448	0.6421	0.6651	0.627	0.4044
Img3	0.7519	0.5161	0.5161	0.4623	0.5226	0.5857	0.6914	0.5703
Img4	0.7448	0.6516	0.6516	0.4118	0.6024	0.5141	0.6667	0.5141
Img5	0.73594	0.6511	0.6511	0.5428	0.7899	0.5197	0.5914	0.6993
Img6	0.78278	0.6636	0.6636	0.3222	0.6112	0.5586	0.6491	0.5876

Table 3 Segmented image score results of optimal threshold segmentation technique

Si. No.	Accuracy	Dice	F-measure	Jaccard	Precision	Sensitivity	Specificity	MCC
Img1	0.8725	0.71	0.71	0.5589	0.9562	0.5736	0.9894	0.6746
Img2	0.9396	0.92	0.92	0.8553	0.9762	0.8735	0.9553	0.9762
Img3	0.9393	0.88	0.88	0.7980	0.9678	0.8198	0.9887	0.8519
Img4	0.8293	0.70	0.70	0.5492	0.9102	0.5807	0.9680	0.6266
Img5	0.8732	0.86	0.86	0.7569	0.9167	0.8128	0.9302	0.7500
Img6	0.8278	0.31	0.31	0.1889	0.4149	0.2576	0.8129	0.0806

2. The performance parameter accuracy values indicate that the optimal threshold technique (0.93) is better than the conventional technique (0.78) used for the CAD system.
3. The high value of specificity confirms that minimal FP detection (optimal thresh: 0.9894).
4. For dice index score, optimal threshold technique value is 0.88 which better than conventional technique.

4 Conclusions

In our current work, performance comparison of conventional and optimal segmentation techniques is performed. The outcome of segmentation score followed by qualitative analysis recommends that the optimal threshold segmentation technique is better than the conventional threshold segmentation technique but is not sufficient to analyze the region of interest (ROI) in mammogram images along with useful feature extraction. Future research work is needed in developing advanced sophisticated computational methods for detecting abnormality in the breast tissue by segmenting mammogram images at an early stage.

References

1. Al-Najdawi N, Biltawi M, Tedmori S (2015) Mammogram image visual enhancement, mass segmentation and classification. Appl Soft Comput 35:175–185
2. Burke HB, Goodman PH, Rosen DB, Henson DE, Weinstein JN, Harrell FE Jr, Marks JR, Winchester DP, Bostwick DG (1997) Artificial neural networks improve the accuracy of cancer survival prediction. Cancer 79(4):857–862
3. Dheeba J, Singh NA, Selvi ST (2014) Computer-aided detection of breast cancer on mammograms: A swarm intelligence optimized wavelet neural network approach. J Biomed Inform 49:45–52
4. Jaber IM, Jeberson W, Bajaj H (2006) Improved region growing based breast cancer image segmentation. Int J Comput Appl 975:888
5. Meenalosini S, Janet J (2012) Computer aided diagnosis of malignancy in mammograms. Eur J Sci Res 72(3):360–368
6. Moradmand H, Setayeshi S, Karimian AR, Sirous M, Akbari ME (2012) Comparing the performance of image enhancement methods to detect microcalcification clusters in digital mammography. Iran J Cancer Prevent 5(2):61
7. Pandey B, Jain T, Kothari V, Grover T (2012) Evolutionary modular neural network approach for breast cancer diagnosis. Int J Comput Sci Issues 9(1):219–225
8. Pati DP, Panda S (2018) Extraction of features from breast cancer mammogram image using some image processing techniques. In: 2018 fifth international conference on parallel, distributed and grid computing (PDGC). IEEE, pp 528–533
9. Pati DP, Panda S (2020) Feature extraction and enhancement of breast cancer mammogram noisy image using image processing. In: 2020 international conference on computer science, engineering and applications (ICCSEA). IEEE, pp 1–5
10. Pradeep N, Girisha H, Karibasappa K (2012) Segmentation and feature extraction of tumors from digital mammograms. Comput Eng Intell Syst 3(4):37–46
11. Ramaprabha T (2023) A comparative study on the methods used for the detection of breast cancer. Int J Recent Innov Trends Comput Commun 5(9):143–147
12. Sandhya G, Vasumathi D, Raju G (2015) Classification of mammogram images for detection of breast cancer. Int J Recent Innov Trends Comput Commun 17(2):11–17

Predictive Maintenance of NASA Turbofan Engines Using Traditional and Ensemble Machine Learning Techniques



Dangeti Saivenkat Ajay, Sneegdh Krishnna, and Kavita Jhajharia

Abstract Predictive maintenance can be understood as applying ML & DL techniques for the maintenance of equipment in industries. Nowadays AI is applied in every other industry and it makes our life smarter and easier. Predictive maintenance techniques predict beforehand when the breakdown/failure is going to occur thereby increasing the safety. The importance of predictive maintenance is gradually being realized. The aviation industry is one such area where safety is of paramount importance. This work focuses on calculating the remaining useful life (RUL) of turbofan jet engines which is an application of predictive maintenance in aviation. Estimating an equipment's or system's remaining useful life (RUL) is regarded as one of the complex tasks in predictive maintenance. In this paper, we discuss the importance of predictive maintenance, and its implication in aviation, elaborate on the dataset, and the model chosen, perform a comparative study of different models, and investigate their shortcomings. The model was constructed based on regression algorithms like random forest (RF), linear regression, XgBoost & K-nearest neighbors (KNN). The results obtained were compared & evaluated with the help of regression parameters like r2score and root mean squared error (RMSE). XgBoost was the best performing algorithm with RMSE of 27.7277.

Keywords Prognostics · Predictive maintenance · Remaining useful life · Ensemble modeling

1 Introduction

Predictive maintenance has gained prominence in the last couple of years. This rapid adoption is because it allows to account for maintenance in advance and plan up a logistic process that will lead to a smoother transition for change of machinery from a faulty one to a fully operational one.

D. S. Ajay · S. Krishnna · K. Jhajharia (✉)
Manipal University Jaipur, Jaipur 303007, India
e-mail: Kavita.chaudhary@outlook.com

© The Author(s), under exclusive license to Springer Nature Singapore Pte Ltd. 2023

S. Chinara et al. (eds.), *Advances in Distributed Computing and Machine Learning*,
Lecture Notes in Networks and Systems 660,
https://doi.org/10.1007/978-981-99-1203-2_31

As we emerge into Industry 4.0, i.e., the fourth industrial revolution, AI techniques are increasingly being adopted across all the areas. Predictive maintenance techniques using machine learning has become one of the important use cases of AI. Nowadays we have sensors and IoT devices fitted in equipment in the factories for condition monitoring. These sensors capture a lot of data and machine learning techniques can be applied to this data. This provides a more reliable approach as an adaptation of such techniques saves a lot of time and energy and results in lower maintenance costs, higher safety, reduction in unscheduled downtime, and improved decision making. PdM pipelines are now being used in aircraft engines (aviation), power plants, automobiles, transportation, oil and gas, medical equipment, and even the software industry [2].

Maintenance procedures are of various types, and can be broadly categorized as follows:

- *Corrective or Reactive maintenance*, where equipment is inspected into once the failure has occurred.
- *Preventive maintenance or Scheduled maintenance*, which is performed periodically in a planned schedule. But it often leads to over-maintenance leading to a rise in operating costs.
- *Predictive maintenance* techniques under which we monitor some equipment during its operation, allowing for maintenance to be performed only when needed.

Predictive Maintenance will allow for identifying unscheduled downtime or breakdown beforehand, optimizing maintenance by preventing over & under maintenance thereby streamlining the use of equipment [3]. In the case of aircraft engines, estimating RUL (Remaining Useful life) will enhance the safety, improve the airworthiness, provide considerable cost savings. The main challenge in Predictive Maintenance is acquiring quality fault progression data, it is expensive and time-taking. Due to this such datasets are quite sporadic in nature. In most cases, such datasets are synthetic—they were made in lab for study purposes, and even the NASA dataset used here is synthetic. Datasets, where sensor readings are recorded until they naturally fail, are extremely rare.

For our research work, we decided to focus on Predictive maintenance application in aviation, where safety is of paramount importance and there is no scope for error [4]. Today, all modern aircraft use turbofan engines, which produce thrust using jet core efflux and bypass air. An aircraft engine is a complex system and therefore estimating an equipment's or system's remaining useful life (RUL) is regarded as one of the complex tasks in predictive maintenance. Prognostics and Predictive maintenance go hand in hand. Prognostics is a discipline that focuses on identifying & predicting the time beforehand when equipment will no longer perform its intended function. Estimation of Remaining Useful Life (RUL) is a part of prognostics [5]. The remaining useful life (RUL) estimates are in units of time (e.g., hours or cycles). Machine Learning, a subset of AI, is a powerful tool for developing intelligent algorithms. ML approaches can handle multivariate data and extract meaningful

relationships within data in complex environments. But the techniques used before or during the training of the model judge the performance of the application.

Therefore, this work is a relative study of classical machine learning algorithms applied to the turbofan engine dataset [6]. We calculated the remaining useful life (RUL) of a jet engine based on the NASA turbofan dataset. A regression approach was followed, and a comparative study was performed where multiple models were constructed based on regression algorithms like RF, linear regression, XgBoost & KNN. These different models are compared, and their shortcomings are discussed. The results obtained are evaluated with the help of regression parameters like r2score and RMSE for comparing the prediction accuracy. The rest of the paper is organized in the following manner. Section 2 discusses the methodology followed, Sect. 3 illustrates the results and discussion while Sect. 4 details the conclusion and future scope.

2 Methods and Materials

2.1 Dataset

We chose NASA turbofan dataset, which serves as a baseline for calculating the remaining useful life (RUL) in turbofan jet engines. It was taken from the Prognostics Data Repository of NASA [1] and includes run-to-failure sensor readings of degrading turbofan engines. The NASA turbofan dataset has multiple distinct characteristics that make it apt for this study [7]. The dataset is frequently used for prognostics research.

There are multiple turbofan engines of the same type with each engine having different degree of initial wear & tear and manufacturing conditions. The initial wear was assumed to be normal, and the engine is assumed to operate normally at the beginning. The dataset has four sets of training and test sets and is contaminated with noise. We have chosen the second dataset since it has more data compared to the counterparts. The engine develops a fault at some point in time as it operates. The fault grows in magnitude until system failure in the training set while in the test set, the data ends prior to machine failure. Each row corresponds to a single operational cycle. There are 26 columns in the dataset, and more details can be found in the Table 1. Each engine has 21 sensors collecting different readings related to engine state at runtime and with three operational settings to change the performance of the engine.

Table 1 Sample dataset

Index	Features	Type	Description
1	unit_nr	Integer	Unique engine identifier
2	Time cycles	Integer	Current cycle (a measure of time)
3	Setting_1	Float	Engine setting 1
4	Setting_2	Float	Engine setting 2
5	Setting_3	Float	Engine setting 3
6	s_1	Float	Sensor reading 1
7	s_2	Float	Sensor reading 2

Table 2 R-squared error and Root mean squared error for different algorithms

Linear regression				
	Train		Test	
	R2	RMSE	R2	RMSE
Base model	0.650	40.9187	0.6445	32.0611
Clipped at 125 RUL	0.755	20.6210	0.6512	31.760
With feature engineering	0.754	20.6245	0.6510	31.770
XGBoost regression				
	Train		Test	
	R2	RMSE	R2	RMSE
Base model	0.7924	31.5171	0.7153	28.694
Clipped at 125 RUL	0.8831	14.2429	0.7341	27.7277
With feature engineering	0.8831	14.2429	0.7341	27.7277
Random forest				
	Train		Test	
	R2	RMSE	R2	RMSE
Base model	0.9599	13.8522	0.7167	28.6200
Clipped at 125 RUL	0.9772	6.2863	0.7284	28.0265
With feature engineering	0.9771	6.3049	0.7270	28.0943
KNN				
	Train		Test	
	R2	RMSE	R2	RMSE
Base fit	0.77680	32.683	0.6537	31.64533
Clipped @ 125	0.8699	15.0271	0.7026	29.3232
With feature engineering	0.8699	15.0271	0.7026	29.3232

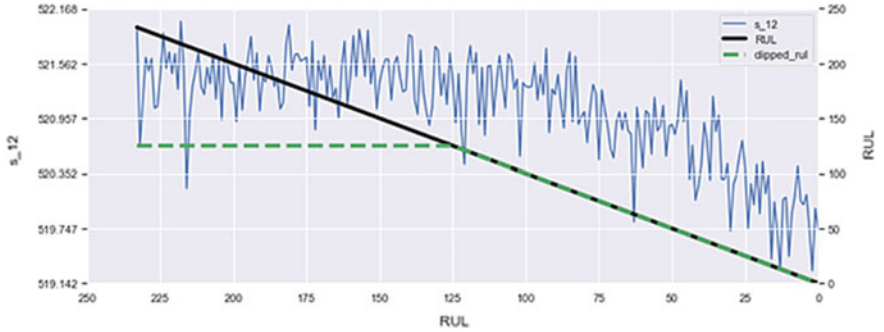


Fig. 1 Plot showing few features do not vary linearly with RUL

2.2 Prediction Methodology

The present work is an attempt to compare the performance of various algorithms on the NASA turbofan dataset and broaden the scope of machine learning applied to RUL prognosis [8]. First, the remaining useful life (RUL) of each instance in the training and test sets was calculated. Then, the preprocessing methods were applied to attain data in a suitable form before applying an algorithm. Finally, the training data is fed to the algorithm to make a model. Python programming environment was used to develop and test the models. The variations are plotted using python libraries like seaborn and matplotlib. The aptness of the model was verified with the test set. The results obtained were compared and model evaluation was done using metrics such as r2score and RMSE. This is repeated multiple times with different approaches (for instance, excluding few features) before collating the results.

Initially, all the features were fit without altering the feature set. The baseline scores were noted and then the target (RUL) was clipped at 125. This is because plotting RUL shows that it does not decline linearly with time, it's in fact constant initially and then begins to decline as the engine develops faults [9]. Figure 1 indicates the same, sensor 12 reading does not fluctuate linearly with RUL. Doing this fetched a good increase in train scores. Finally, key features were picked (for instance, using the feature importance attribute of random-forest or XgBoost) and were fit, eliminating possible noise introduced by less important features.

2.3 Algorithms

Machine learning models in Predictive maintenance can be categorized into supervised and unsupervised approaches. Based on the data and labels, it was decided to go with the supervised learning model. Supervised learning models can further be divided into the Regression and Classification approaches. Classification models predict the possibility of a breakdown in the next n-steps while with the Regression

models we can predict the time left before the next failure, which is the remaining useful life (RUL) [10]. In this paper, four regression algorithms were applied, and a comparative study was done to ascertain the best-performing model.

Linear Regression. It is one of the basic algorithms where the function establishes the best fit line between the dependent and the independent variables such that the line has the least sum of squares or errors.

Random Forest (RF). Random Forest is a frequently used algorithm for predictive maintenance techniques. Its simplicity coupled with resistance to overfitting makes it an attractive choice.

XgBoost. It stands for Extreme Gradient Boosting and is one of the most advanced algorithms. It provides a parallel tree boosting and can be used for both regression and classification tasks.

K-Nearest Neighbor (KNN). KNN classifies the objects based on some nearest K value with respect to features.

2.4 Performance Metrics

Two performance metrics were used to evaluate the models—(1) Root Mean Squared Error (RMSE), and (2) R squared (R^2) (%). Root mean squared error (RMSE) can be understood as the measure of how well a regression line fits the data points. RMSE is determined by taking the square root of the residuals' mean. The lower the root mean squared error, the better is the model.

A statistical indicator of the variance in the dependent variable that can be predicted from the independent variable is the r-squared (r2 score). It is used for explanation of variability of one factor that can be caused by its relationship to another factor. RMSE & R-squared can be calculated as follows:

$$RMSE = \sqrt{MSE} = \sqrt{\frac{1}{N}\sum_{i=1}^{N}\left(y_1 - \hat{y}\right)^2} \qquad R^2 = 1 - \frac{\sum_{i=1}^{N}\left(y_1 - \hat{y}_i\right)^2}{\sum_{i=1}^{N}\left(y_1 - \bar{y}_i\right)^2}$$

where,

y_i—*predicted value of* y_i.
$\bar{y}$—*mean value of* y_i.
y—*actual RUL.*

Figure 2 depicts that the dataset is riddled with outliers. This possibly explains why algorithms overfit before hyperparameter tuning. Algorithms like Random Forest still overfit if the dataset is festered with outliers, which was found to be true in this case.

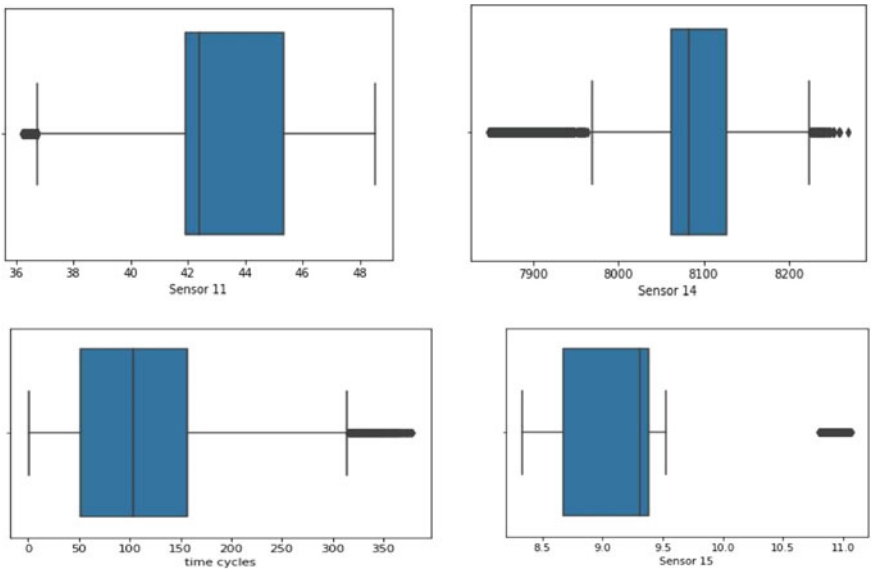


Fig. 2 Box plots of a features indicating dataset is polluted with outliers

3 Result and Discussion

There were four datasets out of which the second dataset had more data (50 k rows) while the other datasets had approximately 20 k rows. Generally, more the data is preferred assuming it is of good quality, so the second dataset was used to train and test the model. The data was collected from 260 engines and each engine had 21 sensor readings. The run-to-failure data is generated from numerous engines of a similar type, and each row refers to a distinct cycle [11]. Table 1 includes the features and their definitions. Unit_nr refers to each machine's individual number, and cycle is the engine's operational lifespan.

In the second dataset, all features have variance which is not the case with other datasets, meaning all features of second dataset might have useful information for RUL prediction. Figure 3 shows that the frequency distribution of lifespans of different engines is right skewed with more engines failing on or before their 200th cycle (Fig. 4).

Plotting actual vs predicted RUL values in Fig. 5 shows all the algorithms used were good at predicting lower RUL limits and struggle to predict at higher limits. Figure 6 bolsters this observation further; all predictions stagnate over RUL of 125. This most possibly is due to the clipping performed during preprocessing to achieve better scores.

To get the RUL of a particular row (for the engine), we calculated the number of cycles the engine ran before failing and then subtracted the cycle number from the

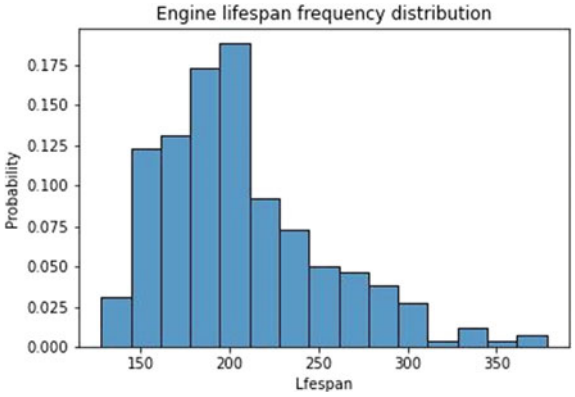


Fig. 3 Engine lifespan Frequency distribution

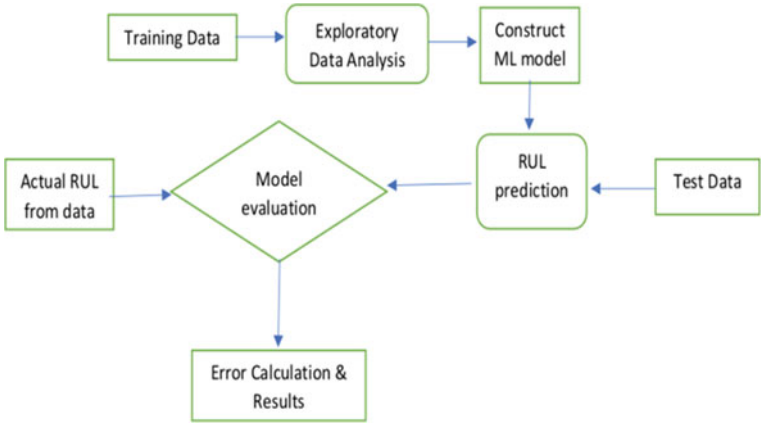
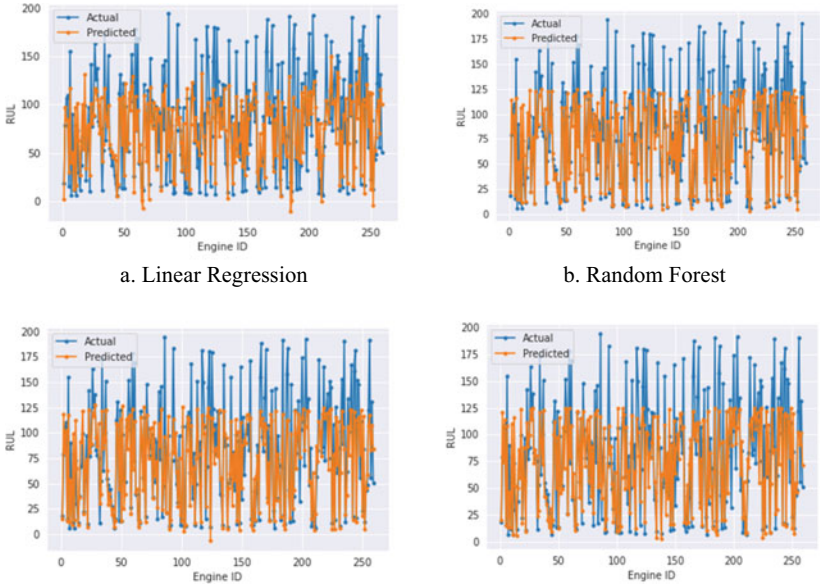


Fig. 4 Flowchart of research methodology

lifespan (which we calculated earlier) for each row. It is seldom possible to calculate RUL with complete accuracy. R2-score and RMSE were used as performance metrics. R2-score indicates how much variance the model can explain while RMSE was used as there was a chance of undermining mean error. To counter that, we square errors to turn negative ones into positive ones, sum them up and then take root to come back to the original unit [12].

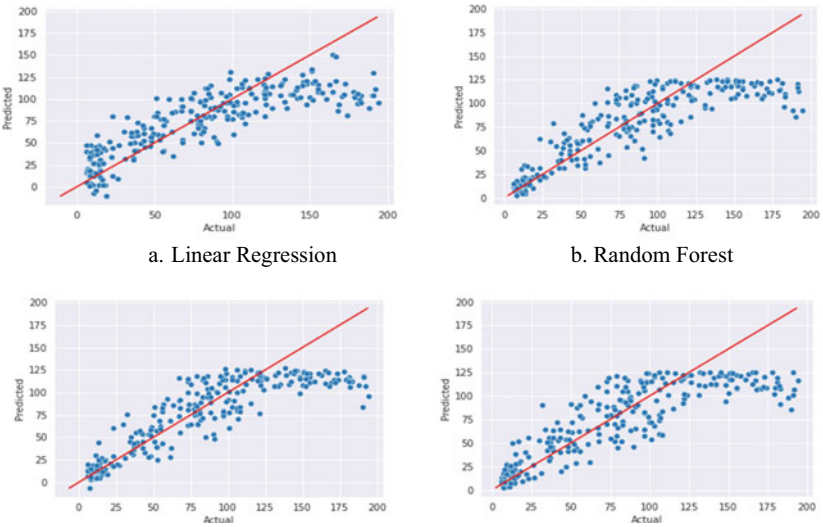
As per our hypothesis, RF was overfitting every time [13]. Contrary to popular belief, random forest is still susceptible to outliers, many features (few ranking top in random forest feature importance list) in the dataset have outliers [14]. A possible explanation why XgBoost does not overfit could be that they are fundamentally different from RF in how they make predictions, i.e., boosting. Hyperparameter



a. Linear Regression b. Random Forest

c. XgBoost d. KNN

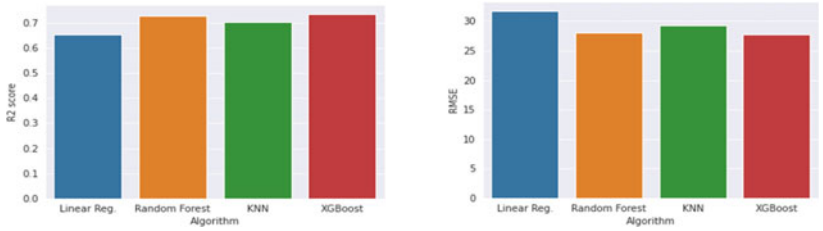
Fig. 5 Visualization of machine ID and remaining useful life (Actual RUL vs. Predicted RUL) of dataset2



a. Linear Regression b. Random Forest

c. XgBoost d. KNN

Fig. 6 Visualization of remaining useful life (Actual RUL vs. Predicted RUL)



a. R2 comparison bar graph b. RMSE comparison bar graph

Fig. 7 Comparison of R-squared & root mean squared error of RULs

tuning gave negligible gains in scores which proves that the feature set given was not rich in the first place.

In our analysis, we emphasized the RMSE scores. The RMSE and r2 score values can be found in Table 2. XGBoost was found to be the best performing model with a root mean squared error of 27.7277.

4 Conclusion and Future Work

In this paper a comparative study of different algorithms used in predictive maintenance using ML was performed. We calculated the Remaining useful life (RUL) of aircraft jet engines which is an application of predictive maintenance techniques in aviation. Linear Regression, Random Forest, KNN, and XgBoost were used to calculate RUL and their performance was compared based on RMSE and the R2 measures. Figure 7 shows a comparative plot of results using the metrics defined. XGBoost Regression was found to have the highest accuracy, due the robust ensemble technique used in the algorithm. The remaining useful life prediction of jet engines will help optimize the maintenance for the airlines [15]. The future scope of this work is to try the time series and the classification approach [16]. Deep learning architecture may also be explored to get fewer false scenarios and better results.

References

1. https://www.kaggle.com/behrad3d/nasa-cmaps
2. Costello JJA, West GM, McArthur SDJ (2017) Machine learning model for event-based prognostics in gas circulator condition monitoring. IEEE Trans Reliab. https://doi.org/10.1109/TR.2017.2727489
3. Phuc D, Levrat E, Voisin A, Iung B (2012) Remaining useful life (RUL) based maintenance decision making for deteriorating systems. IFAC Proc Vol (IFACPapersOnline). https://doi.org/10.3182/20121122-2-ES-4026.00029

4. Mathew V, Toby T, Singh V, Rao BM, Kumar MG (2017) Prediction of remaining useful lifetime (RUL) of turbofan engine using machine learning. IEEE Int Conf Cir Syst (ICCS) 2017:306–311. https://doi.org/10.1109/ICCS1.2017.8326010
5. Saxena KG, Simon D, Eklund N (2008) Damage propagation modeling for aircraft engine run-to-failure simulation. In: 2008 international conference on prognostics and health management, pp 1–9. : https://doi.org/10.1109/PHM.2008.4711414.S. Vollert and A. Theissler, "Challenges of machine learning-based RUL prognosis:
6. Thyago PC, Fabrízzio AAMN, Soares RV, Roberto da PF, João PB, Symone GSA (2019) A systematic literature review of machine learning methods applied to predictive maintenance. Comput Ind Eng 137:106024. ISSN03608352, https://doi.org/10.1016/j.cie.2019.106024
7. Vollert S, Theissler A (2021) Challenges of machine learning-based RUL prognosis: A review on NASA's C-MAPSS data set. In: 2021 26th IEEE international conference on emerging technologies and factory automation (ETFA), pp 1–8. https://doi.org/10.1109/ETFA45728.2021.9613682
8. Shah E, Madhani N, Ghatak A, Ajith KA (2022) Comparative study on estimation of remaining useful life of turbofan engines using machine learning algorithms. In: Reddy VS, Prasad VK, Mallikarjuna RDN, Satapathy SC (eds) Intelligent systems and sustainable computing. Smart innovation, systems and technologies, vol 289. Springer, Singapore. https://doi.org/10.1007/978-981-19-0011-2_42
9. Yurek OE, Birant D (2019) Remaining useful life estimation for predictive maintenance using feature engineering. Innov Intell Syst Appl Conf (ASYU) 2019:1–5. https://doi.org/10.1109/ASYU48272.2019.8946397
10. Yang C, Chen Q, Yang Y, Jiang N (2016) Developing predictive models for time to failure estimation. In: Proceedings of the 2016 IEEE 20th international conference on computer supported cooperation work in design
11. Saranya E, Sivakumar PB (2020) Data-Driven prognostics for run-to-failure data employing machine learning models. Int Conf Invent Comput Technol (ICICT) 2020:528–533. https://doi.org/10.1109/ICICT48043.2020.9112411
12. Sankararaman S, Goebel K (2013) Why is the remaining useful life prediction uncertain? PHM 2013—proceedings of the annual conference of the prognostics and health management society 2013:337–349
13. Behera S, Choubey A, Kanani CS, Patel YS, Misra R, Sillitti A (2019) Ensemble trees learning based improved predictive maintenance using IIoT for turbofan engines. In: Proceedings of the 34th ACM/SIGAPP symposium on applied computing
14. Eker O, Camci F, Jennions IK (2012) Major challenges in prognostics: Study on Benchmarking Prognostics Datasets
15. Samaranayake P, Kiridena S (2012) Aircraft maintenance planning and scheduling: an integrated framework. J Qual Maint Eng 18(4):432–453
16. Amruthnath N, Gupta T (2018) A research study on unsupervised machine learning algorithms for early fault detection in predictive maintenance. In: 2018 5th international conference on industrial engineering and applications (ICIEA), pp 355–361. https://doi.org/10.1109/IEA.2018.8387124

Stacking a Novel Human Emotion Recognition Model Using Facial Features



Vikram Singh and Kuldeep Singh

Abstract Emotion recognition is one of the computationally complex tasks with a spectrum of real-world applications/scenarios. Though, it's a multifaceted task to detect emotions accurately using a monolithic learning model based on a single data source. In the recent decade, the spectrum of approaches and techniques were designed to achieve efficient and accurate human emotion detection keeping the emphasis on the modality of input data and harnessing the capacity of heterogeneous learning models and methods. In generic settings, image data object asserts potential quality features related to human emotions. Therefore, primarily image-based emotion recognition models, i.e., MobileNetV2, ResNet121 and VGG-16 adopted over the FER2013 dataset. In this, a stacked framework for facial emotion recognition is proposed with a feature to steer implicit data argumentation parameters. Interestingly it outperforms the CNN (AlexNet model) and an efficient R-CNN model with an improved recognition rate of 3.1%.

Keywords Data argumentation · Deep learning · Human emotion recognition

1 Introduction

The recognition of human emotion is a fundamental task in several real-world application scenarios. Though, it's a multifaceted task in data-driven scenarios to accurately outline human emotion over a set of data objects, on single data modality or multiple data modality. A potentially successful strategy is expected to consider multiple factors, e.g., *face detection, face element observation, facial expressions, speech analysis, behavi*or *(gesture/posture) or physiological signals* and many more for completing the recognition task [1].

V. Singh (✉) · K. Singh
National Institute of Technology, Kurukshetra 136119, HR, India
e-mail: viks@nitkkr.ac.in

K. Singh
e-mail: kuldeep_32013111@nitkkr.ac.in

© The Author(s), under exclusive license to Springer Nature Singapore Pte Ltd. 2023

S. Chinara et al. (eds.), *Advances in Distributed Computing and Machine Learning*,
Lecture Notes in Networks and Systems 660,
https://doi.org/10.1007/978-981-99-1203-2_32

In recent years, emotive analytics emerges as an interesting research area of work, with a blend of psychology and technology related efforts [2, 3]. Though numerous strategies are designed and experimented for the emotion recognition task over a visual data modality and thus arguably reductive, many facial expression detection tools lump human emotion into 7 main categories: *Happy, Sadness, Anger, Fear, Surprise, Neutral, and Disgust*, as shown in Fig. 1. With only focus visual data modality or to facial emotion detection algorithm, detecting faces within a photo or video and sense expressions by analyzing the relationship between points on the face, based on curated databases compiled in academic environments [4–6].

The key motivation of the proposed model for the facial recognition is to supplement facial-feature based emotion recognition. In the view of range of existing models, selecting an appropriate model is a tedious task. We have adapted to lightweight models, i.e., have a smaller number of training parameters and serves the purpose of emotion recognition as well as created our own custom models to improve the accuracy. In this process, identification of potential feature sets, combination of different layers, choices of different datasets and choice of different base models will have a direct impact on the combined results are key challenges.

An adaptive data argumentation plays a pivotal role in proposed facial based emotion recognition approach, as dealing with data objects' corresponding features may lead to a tedious task. The argumentation of feature space during the preprocessing, e.g., feature annotation and training, leads to a smoother learning phase in the recognition model, specifically in deep learning models.

In the paper, we have experimented on the various approaches of facial-features based emotion recognitions. The experimental observations of the designed approach

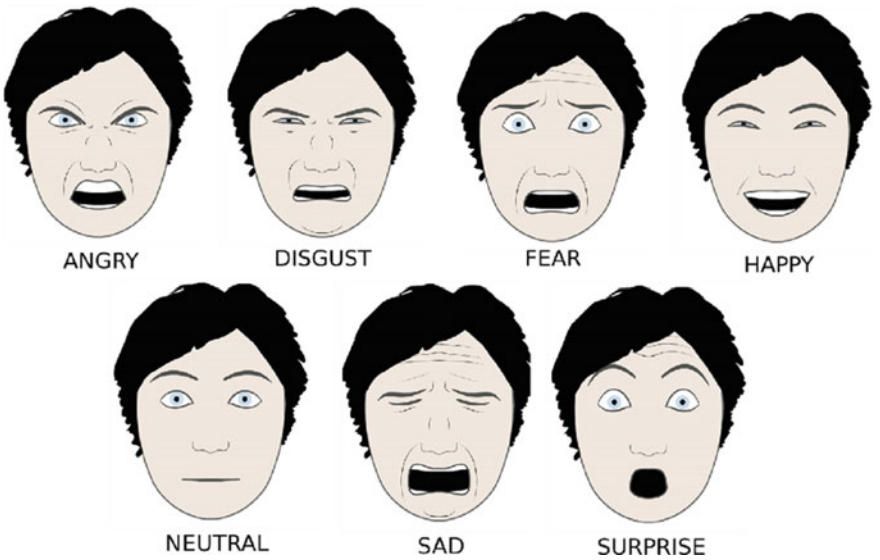


Fig. 1 A set of facial image-object with implicit human emotions [23]

over popular datasets, i.e., *FER2013* for training, validation and testing our model. Further, a three-level cross validation on *FER2013* dataset, as *Train, Test and Validation* is adapted. With achieved accuracy of about 64.3%, surpassed existing models with an accuracy of about 61.2%, as good as human accuracy which is about 60–70%.

The adaptive data argumentation is fundamentally introduced to minimize the implicit imbalance. Since *CNN* are used in our case, less data result in over-fitting of *CNN* model, so to tackle overfitting, *Dropout* and *Kernel Regularization* for facial model are also used. The key strategy to train our model is Transfer learning and some sort of *Stacking* technique.

1.1 Design Challenges and Research Questions (RQs)

Emotion recognition is a tedious task, even among the emergence of deep learning based algorithms. As most of the existing research efforts are limited on two design fronts, firstly either they are based on the assumption that increasing the number of layers will also increase the recognition rate which is somewhat incorrect and secondly they are unable to deal with multi-modal data scenarios. The conceptual reason for the first argument could be existing on a fact that due to increase in the depth of model there is a certain point up to which the recognition rate increase further there will be no change in it. The soft clarification for the second reason could be a lesser number of experimentations for the facial emotion recognition scenario.

In view of the above design issues and overall motivation of the proposed work is to overcome the limitation of a facial based emotion recognition model. The following research questions (RQ) are formalized to conduct the design and overall analysis:

RQ-I: Which deep learning techniques may be kept in a suite to create an extensive framework for '*Human Emotion recognition*'?

RQ-II: How much stacking must be given to *Facial-features*, in order to get better results for human emotion recognition model?

RQ-III: How data arbitration plays a pivotal role in facial based learning models' data scenario?

The key motivation of the factor of consideration in this work is a fascination about how machines or programs recognize human emotions, '*How by using a set of camera or datasets machine learn to recognize corresponding to happy, sad and other emotions as well*'. In this work, a new custom model for facial emotion recognition is developed, and some pre-existing models to make further enhancements in the recognition accuracy as well. Emotion recognition itself is a research problem whether it's a facial emotion or speech emotion or any other modality. For classification problems, there are lots of predefined models in *keras* library which serve the purpose but for every use case, these predefined models need to be changed.

1.2 Contribution and Outline

The key contribution of the paper is a novel model of the human emotion model. In this paper, the following has been contributed. A learning model for the recognition task with Stacking of Layers is a complex computational situation. As within learning techniques is a tedious task because the choices to be made are a trial-and-error process. Wrong choice of the layer lead to very low recognition rate. The main problem with this task is there is no correct direction that which layer should be put after another which is a quite challenging task. In this work, a new stacking arrangement is contributed after MobileNetV2 as a base model. Dataset Formation: In our work new datasets formation involves, a modified *FER2013* for facial emotion recognition is adapted. Various Argumentation parameters are used to achieve the following task.

The paper is organized as: Sect. 1 gives some introduction about the emotion recognition field and some research questions which has been identified. Section 2 Introduced with current research work done in the field of facial emotion recognition and some widely used methods for facial emotion recognition. Section 3 Described the proposed approach i.e., how an appropriate stacking can affect the recognition rate. Section 4 Described all the experimental settings, how to steer data and all the comparison of models and with other research work also. Section 5 Described conclusion and future scope of the work.

2 Related Work

Convolution Neural Network (CNN) has shown great potential in image processing when first arrived in late 1990. Akhdand [1] used 8 pretrained deep convolution neural network (DCNN) models and used transfer learning to avoid training from starch i.e., freezing all the layers except the last block and used 10 cross validation and famous datasets like *KDRF* and *JAFFE* used for model evaluation [2]. In this paper, author adopted the use of *VGG* with regularization and used *sgd* as an optimizer and other optimization methods and used the saliency map for visualization [3]. In this paper, instead of using deep dense network, it uses deep sparse network and inception layers and used 7 publicly available Dataset [4]. In this author uses *SWATS* (Switching from *Adam* to *SGD*) here Adam is Adaptive moment estimation and SGD as Stochastic Gradient Descent i.e., during the middle of training it can take advantage of fast convergence of *Adam* at the beginning but later we can use *sgd* so that model generalizes well [5].

Author used *AFF-Wild* 2 dataset and train *CNN* in this model and then test it on *FER2013* [6]. Amil, Charles and Ferhat uses Transfer learning, Data Argumentation, Class weighting, and Auxiliary Data and also used ensemble with soft voting to archive an accuracy of 75.8%. In this paper [7] authors added a Local Normalization Process between *CNN* layers that enable to detect smiling and recognize facial

expressions. In this paper [8] Gupta and L.K. combine spatial and temporal features which are available in video because of this aggregation between features in a layer, it reduces overfitting problem in *CNN* models. Avagyan and Aneesh [9] proposed a similar architecture as VGG they doubled the conv layers as goes deeper in architecture and used different data argumentation techniques [10]. Shervin, Mehdi and Amirali proposed an attentional based technique which uses end to end *CNN* that focuses on important parts of the face using a Localization network and achieves significant improvements [11].

Barsoumn acquired noisy labels from crowd-sourcing and used a taggers technique to Relabel the dataset and used cost function for their *DCNN* [12]. Want et al. proposed a network which self-cures itself and reduce the uncertainty efficiently and prevents deep networks to overfit and two uncertainty from different aspects are [13] *Relabel* mechanism for lowest ranked groups [14] During mini batch ranking regularization is used. Liu et al. trained the three separate *CNNs* and ensemble them to improve performance. Their best single-network accuracy is 62.44% [15]. After that, Liu et al. used ensemble of 3 *CNNs* and improved their accuracy by 2.6%.

Different Research has been carried out on optimization algorithms which are used during training Process. There is no document or guideline which tells which optimizer needs to be used. By choosing suitable optimization algorithm for the use case will improve model performance significantly. Most commonly used optimizers are *SGD* which updates the parameters on the basis of single data point [16], other is Adam which is a combination of *AdaGrad* and *RMSProp* [17].

Dropouts, Data argumentation and *Regularization* are used to reduce over-fitting and avoid vanishing gradient problem [18]. Different pooling methods are used for better generalization such as max pooling, and average pooling [19].

Due to various variations, detecting the presence of a human face in a photograph or video is a difficult and complex procedure, such as varying sizes, angles, and positions of the human face, which is the present cause of variation, other factors that contribute to difference include beard, hair, cosmetics, and spectacles [20, 21].

Based on decades of study, face recognition techniques are classified into four types: knowledge-based techniques, feature invariant approaches, template-based approaches, and appearance-based approaches. Feature based invariant approaches simply describe the facial features as well as texture, color etc. Template Based Features describe face templates of type predefined and deformable type and at last appearance-based approaches simply uses *SVM*, *HMM*, etc.

2.1 Facial Emotion Recognition Methods

In this part, we will quickly cover each of the four techniques for face detection and emotion recognition. The first category of the work is under *Knowledge-based Approach*, where the model solely depends upon the geometry of face and the most commonly relative distance and positions are calculated for facial features, after applying these rules on images faces are detected and after that if incorrect items are

detected then are trimmed using a verification process and difficult task to convert these approaches into strict rules. Extending this approach is nearly impossible for all cases so this is limited to some uses [20, 22].

Feature Invariant Approach. In this technique, facial features are highly recognized and then classified together depending on face geometry. However, this strategy is not suited for noise and illuminations since they significantly affect or weaken the feature boundaries, whereas sensitivity has no influence on this approach, making it a superior answer for the facial detection problem. However, skin color has lately been used by researchers to identify traits. There have also been approaches developed that incorporate numerous facial traits.

Template-based Approach. In this method, a window is utilized to compare to the original face, and the standard pattern is used to identify the existence of a human face within that window. This solution appears easy, however, the scale necessary for the image is a disadvantage, and it is incapable of dealing with human facial variances [20].

Appearance-based Approach. The human face is considered in terms of pixel intensities. In this technique, training is more difficult and time-consuming, neural networks are utilized instead of traditional ways for collecting complicated face patterns from facial photos. The network is trained using both supervised and unsupervised methods. *SVM*, *Eigen Faces*, Distribution-based techniques, *Nave Bayes classifiers*, and *HMM* are employed in addition to neural networks [20]. SVM is classified using *hyperplane*s. Eigen faces use an *eigenspace* decomposition-based probabilistic visual learning technique. The Nave Bayes method gives a more accurate assessment of conditional density function in face subregions. As opposed to template and appearance-based techniques, HMM does not need alignment.

3 Proposed Approach

In facial emotion recognition, a transfer learning strategy is used in which pretrained models are used and according to our need, we modify models and added multiple layers to improve the performance.

Network Architecture for Modified MobileNetV2. The proposed architecture for the emotional classifier involves a number of layers, as each computational layer in the model adds a depth to the delivery of the overall computational task. If these layers are increased the overall capacity of the model will increase, though unnecessarily increasing layers will cause the model to overfit and put too many neurons in a single layer. These settings further enhance the overall cost of a model instead of separating it out which is computationally efficient.

Figure 2 illustrates the proposed MobileNetV2 based network for the multilevel emotion recognizer. The proposed strategy has an input shape of $224 \times 224 \times 3$ that is resized all the dataset images to $224 \times 224 \times 3$ and added over proposed 7 layers at last.

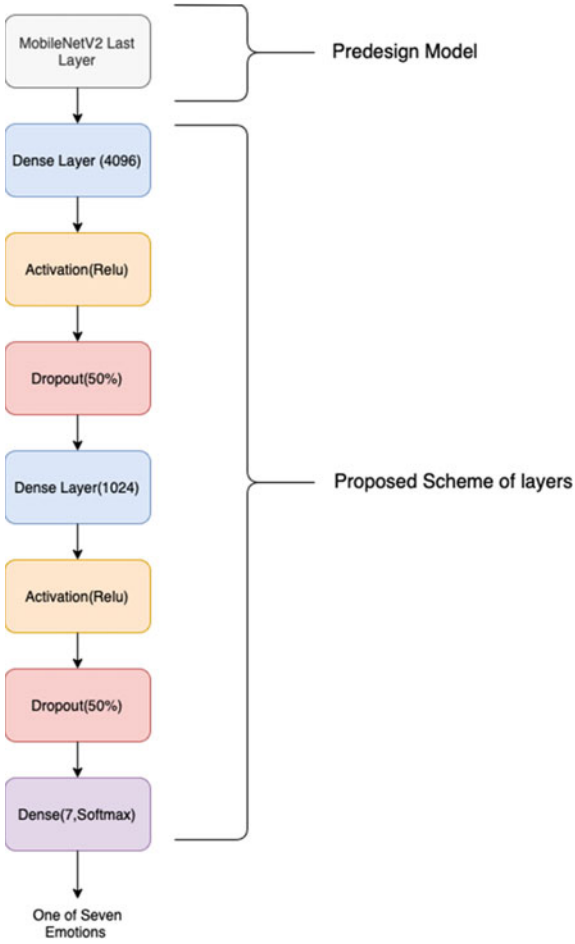


Fig. 2 MobileNetV2 with Proposed Scheme of Layers

In the proposed architecture, a Dense layer is used to acquire all the information from the previous layer and then performed activation so that overall values squash between *(0,x)* where *x* is a value greater than 0, which is *ReLu* functionality. Interesting point to note here is that from all the 3 models *MobileNetV2* used *ReLU6* as the underlying architecture but we used *ReLu* as common *Activation* in our model.

The proposed model has not changed the underlying architecture to match the proposed, because it will change the overall architecture of the predefined model and we have not adapted the *ReLu6* because other models used the underlying architecture as *ReLu* itself. Since layers can cause the overfitting issue so we used the random dropouts of 10%, 20% and 50% to reduce the overfitting and used *SoftMax* as the classification layer.

4 Experimental Analysis

4.1 Data Settings

All studies were carried out on a computer equipped with an Intel i5-8300H 2.5 GHz processor and 16 GB of RAM, with no additional devices such as GPU. *TensorFlow* and *OpenCV* for pertained models and dataset preprocessing.

The primary source of data is taken from *Facial Emotion Recognition* dataset (*FER2013*) with located images crawled over Google search in an uncontrolled environment. The dominant features on these images set on over 07 emotions with each image size of 48×48. The key challenge, here is observed during the data preparation, is data imbalance, e.g., dataset is quite an imbalance for '*Happy*' emotion has lots of images as compared to '*Disgust*' emotion, with a limited image. Figure 3 (*left* part) illustrated the overall imbalance of the data objects, for each training and test image set.

A traditional data argumentation helps in these situations, as for each class imbalance new images can be generated. We placed data objects into three sets: train, test and validation set. The dataset is now balanced for each emotion class, and the proposed hypothesis, as shown above in Fig. 3 (right) part.

The preparation of data for above listed models on images i.e., new sample of images are generated and image normalization and reshape has been done to match input shape of $224 \times 224 \times 3$ because base input of all 3 models expects this input. Batch size of 128 has been adopted for all the models and *Learning rate* has been set to 0.01 and also used l_2 type *kernel regularizes* and *Dropout* to reduce the overfitting to some extent. Since we need to classify multiple emotions then adapted loss is *Sparse Categorical Cross Entropy.*

Two optimizers are used in our experiment *Adaptive momentum Estimation (Adam)* and *Stochastic Gradient Descent (SGD)* in which Adam gives us better performance during model convergence performance. *ReLu* has been adapted as activation function and for classification layer *softmax* activation is used. *Hyperparameters* are listed in Table 1.

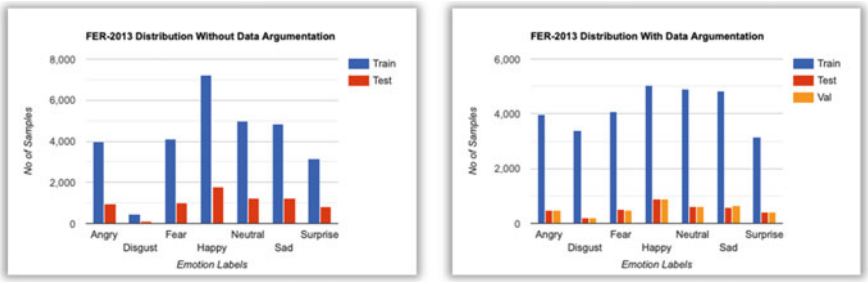
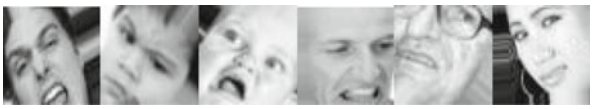


Fig. 3 (left) Data Instances Imbalance Data and (right) with Data Argumentations

Table 1 (a) List of hyper parameters (b) Data argumentation parameters

Parameter name	Value	Argumentation type	Value
Input shape	224 × 224 × 3	Fill mode	Nearest
Batch size	128	Width shift	0.21
Epoch	100	Height shift	0.2
Learning rate	0.01	Shear	0.2
Optimizer	Adam, SGD	Zoom	0.21
Kernel regularizes	l2(0.01)	Horizontal flip	TRUE
Loss	Sparse CC entropy	Rotation	45°
Activation	Classification layer (Softamax), Relu		
Dropout	20, 50%		

Since *FER2013* dataset is imbalanced i.e., dataset contain only 486 *Disgust* images and over 7000 images for Happy so use *data argumentation* technique such as *Width Shift, Height Shift, Shear Range, Zoom, Horizontal Flip, Fill mode and Rotation,* has Generated 7 new images from single image in *disgust* emotion with following parameters and deleted some images from the dataset to make this category balance with new images some of the samples are shown below.



'*Disgust*' Argumentation images

4.2 Human Emotion Recognition Accuracy Evaluation

In facial emotion work three models are used as emotion classifiers. The key performance criterion observed is its effectiveness on the estimation of emotion. The measurement precision is adapted with a definition in *Eq.* 1 The proposed model precision of formalized as its capacity on precisely identifying the emotions and their labels:

$$P = True\ Positive/(True\ Positive + False\ Positive) \tag{1}$$

The measure is *Recall* and adapted definition is.

$$R = True\ Positive/(True\ Positive + False\ Negative) \tag{2}$$

The measure of f1-score is the harmonic mean of both *precision* and *recall* values for a model. The f1-score indicates the overall accuracy of a system and is formalized as *Eq.* 3, as:

$$F1 = 2 * (Precision \times Recall)/(Precision + Recall) \tag{3}$$

The performance analysis reports of listed models are in the form of classification reports. Further classification report is placed into three-measures: *precision, recall, and F1-score.* The accuracy of correctly outlining the fundamental emotions, these levels of accuracy are illustrated over *Confusion metric*; depicts the overall accuracy of the system, as for each emotion we can estimate and analyze.

MobileNetV2: MobileNetV2. It uses the same concept as *ResNet* uses inverted residual uses skip connection and linear bottleneck and has fewer parameters as low as 2.63 million and uses *RELU6* because it increases robustness when dealing with low-precision accuracy network architecture.

The effective's analysis of the designed models, primarily based on fundamental measures and traditional definitions. As we can see this model has other two models in terms of classification '*disgust*' as well as '*happy*' emotion. Because of the underlying structure in which it uses skip connection and linear bottleneck to reduce *Vanishing Gradient Problem* and overfitting and also assures higher layer gets the same weights if weights are almost zero or modified weights and in this depth of model also increases with some dropout so that this property will be maintained throughout.

Modified *MobileNetV2* has required around 100 h of training so that model converges completely, as shown in Fig. 4. During the training phase model is tested after every epoch on validation test and after the completion of the training phase model is tested upon test set correct validation of recognition rate. For evaluation purposes *Precision, Recall, F1-Score* are used.

So the Overall Accuracy boiled down to 64.2% for the test set and for the validation set accuracy is around 63.9%. Confusion matrix for the same is shown in which it is observed that the happy emotion is the most correctly recognized emotion.

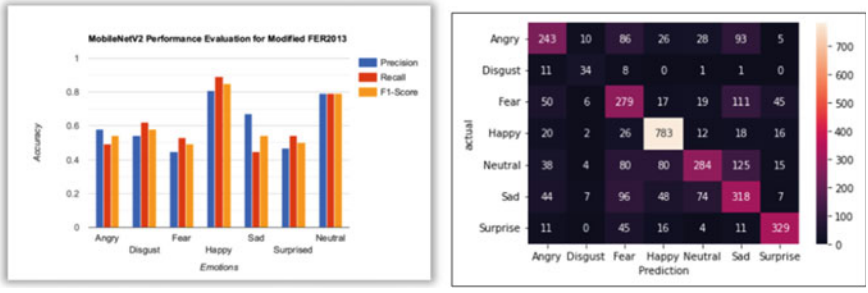


Fig. 4 Performance evaluation chart for *MobileNetV2*



Fig. 5 Performance evaluation and chart for *ResNet50*

Residual Network (ResNet50). Traditionally, a residual network (ResNet50) contains a 50-layered network and is capable of tackling the old rooted problem of *vanishing gradient* with notions of skip/jump connections. The effect of *vanishing gradient* significantly minimizes the weights to zero and limits its learning possibility. A traditional *ResNet50* ensure that higher layer must learn at least as much as lower layers architecture and have 23 million parameters.

The accuracy analysis of the model asserts that it works better for the '*happy*' and poor for the '*disgust*', though overall *ResNet50* is better than *VGG-16*. Primarily due to the underlying structure of *ResNet50* which used skip connection to avoid *Vanishing Gradient problem* and deliver higher recognition accuracy, as shown in Fig. 5.

Modified ResNet50 has required around 135 h of training so that model converges completely. During the training phase model is tested after every epoch on validation test and after the completion of the training phase model is tested upon test set correct validation of the recognition rate. For evaluation purposes, *Precision, Recall, and F1-Score* are used.

So the overall accuracy boiled down to 59.4% for test set and for validation set accuracy is around 58.7%. Confusion matrix for the same is shown in which it is observed that happy emotion is the most correctly recognized emotion.

VGG-16. *VGG* model contains 16 layers in whole network, where predefined input size of 224×224. Every layer uses filters of size 3*3 and a stride of size 1 and padding of *size 1* as well respectively and *MaxPool* size of 2 and a stride of having 2 steps, this model are having 138 million parameters which are very huge and computationally costly model architecture.

Modified *VGG-16* has required around 170 h of training so that model converges completely. *VGG* architecture is very dense in nature due to which it requires more time as compared to other models. During the training phase model is tested after every epoch on validation test and after the completion of the training phase model is tested upon test set correct validation of recognition rate. For evaluation purposes *Precision, Recall, and F1-Score* are used.

Figure 6, confusion metrics reveal the accurate classification of '*happy*' and poor performance for '*disgust*' recognition. The '*disgust*' emotion is a complex situation

due to its proximity with '*angry'* emotion. Further, '*Neutral*' has also been misclassified with '*Sad*', as it's difficult to tell whether a person is sad or neutral via a physical observation and due to the dense network of *VGG-16* that causes overfitting. We have dealt with the placement of additional architecture at the end additional parameters have been increased to 139 M which is the major cause of overfitting because amount of data is less for a huge number of parameters.

So the overall accuracy boiled down to 53.5% for test set and for validation set accuracy is around 52.9%. Confusion matrix for the same is shown and can be observed that happy emotion is the most correctly recognized emotion.

Figure 7 illustrates an overall comparison among four models for the human emotion recognition on standard datasets. Most of the models perform better on '*Happy*' emotions and poorly works for '*Angry*' emotion. Similarly, moderate on '*Neutral*' emotion.

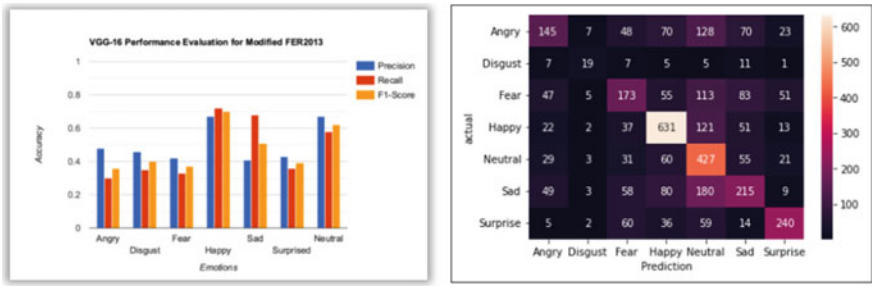


Fig. 6 Performance evaluation chart for modified *VGG-16*

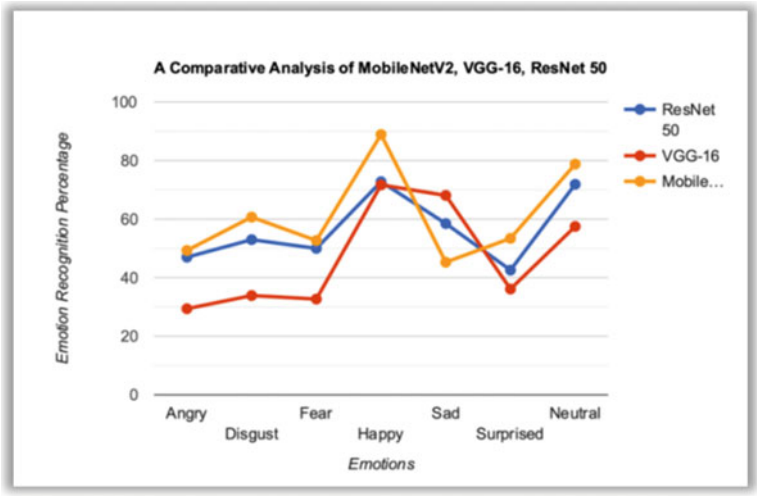


Fig. 7 Comparative analysis of MobileNetV2, VGG-16, ResNet50

Table 2 Comparative analysis of *MobileNetV2, VGG-16, ResNet50* with existing approaches

Sr no	Method	Recognition rate (%)
1	Modified MobileNet V2	**64.2**
2	Modified ResNet50	**59.4**
3	Modified VGG-16	**53.5**
4	CNN (AlexNet) [67]	61.1
5	Net B [66]	60.91
6	Net B_DAL [66]	58.33
7	Net B_DAL_MSE [66]	58.15
8	Fast R-CNN(VGG-16)	30.19

Common observation in Table 2, we potentially identify in all three models is, the most correctly classified emotion is happy because a large number of images are available for the model to train and the most incorrectly classified emotion is disgust because of a smaller number of images are available for model to train, in our architecture classification of disgust improve to some extent. All three models use different underlying working but have a common algorithm adopted i.e., CNN.

5 Conclusion and Future Work

This work aims to design a new architecture and overcome the emotion recognition, with an objective to accurately identify 07 human emotions. The proposed learning model combines facial-features. In facial-domain model is based on *FER2013* dataset with custom layers embedded into a predefined model. Here the proposed work surpassed a *Fast R-CNN* with an increase of accuracy of around 3.1% with our addition layers in *MobileNetV2*.

Based on the conceptual work conducted and experimental analysis there are few potential research directions and issues have been identified. Summarize as below.

(i) Data Argumentation is one of the key preprocessing activities within a learning based model. During conceptual design, it has been widely realised that due to scarcity of data in *FER2013* dataset the *SMOTE* could be utilized to generate synthetic data samples.
(ii) Current proposed work may be extended to accurately estimate the intensity levels within a specific human emotion to highlight the secondary emotions e.g., for *Happy* emotion intensity levels could be *Ecstatic*, *Serenity* and *Joy*.
(iii) Dynamic Filtering can be used while training models i.e. one can define their own filters and use a dynamic approach so that filters will change accordingly.
(iv) For human emotion recognition based scenarios, a comprehensive dataset with potential feature set could be a great advantage for training and testing the

model, for this a synthetic dataset can be created in order to achieve a higher recognition rate.

References

1. Akhand MAH, Roy S, Siddique N, Kamal MAS, Shimamura T (2021) Facial emotion recognition using transfer learning in the deep CNN. Electronics 10(9):1036
2. Mollahosseini A, Chan D, Mahoor MH (2016) Going deeper in facial expression recognition using deep neural networks. In: 2016 IEEE Winter conference on applications of computer vision (WACV). IEEE, pp 1–10
3. Keskar NS, Socher R (2017) Improving generalization performance by switching from adam to sgd. arXiv:1712.07628
4. Anas H, Rehman B, Ong WH (2020) Deep convolutional neural network based facial expression recognition in the wild. arXiv:2010.01301
5. Khanzada A, Bai C, Celepcikay FT (2020) Facial expression recognition with deep learning. arXiv:2004.11823
6. Ivanovsky L, Khryashchev V, Lebedev A, Kosterin I (2017) Facial expression recognition algorithm based on deep convolution neural network. In: 2017 21st Conference of Open Innovations Association (FRUCT) IEEE, pp 141–147
7. Gupta R, Vishwamitra LK (2021) Facial expression recognition from videos using CNN and feature aggregation. Mater Today: Proc
8. Sinha A, Aneesh RP (2019) Real time facial emotion recognition using deep learning. Int J Innov aImplem Eng 1 (2019).
9. Minaee S, Minaei M, Abdolrashidi A (2021) Deep-emotion: facial expression recognition using attentional convolutional network. Sensors 21(9):3046
10. Barsoum E, Zhang C, Ferrer CC, Zhang Z (2016) Training deep networks for facial expression recognition with crowd-sourced label distribution. In: Proceedings of the 18th ACM international conference on multimodal interaction, pp 279–283
11. Liu W, Zheng W, Lu B (2016) Emotion recognition using multimodal deep learning. Neural Inf Proc 521–529
12. Liu K, Zhang M, Pan Z (2016) Facial expression recognition with CNN ensemble. In: Proceedings—2016 international conference on cyberworlds, CW 2016
13. Sun R (2019) Optimization for deep learning: theory and algorithms
14. Sebe N, Cohen I, Gevers T, Huang T (2005) Multimodal approaches for emotion recognition: a survey. Int Imaging VI
15. Kingma DP, Ba JL (2015) Adam: a method for stochastic optimization. In: 3rd international conference on learning representations, ICLR 2015—conference track proceedings
16. Han B, Sim J, Adam H (2017) BranchOut: regularization for online ensemble tracking with convolutional neural networks. In: Proceedings—30th IEEE conference on computer vision and pattern recognition, CVPR 2017, 2017, vol 2017-January. https://doi.org/10.1109/CVPR.2017.63
17. Giusti A, Cireşan DC, Masci J, Gambardella LM, Schmidhuber J (2013) Fast image scanning with deep max-pooling convolutional neural networks. In: 2013 IEEE international conference on image processing, ICIP 2013—Proceedings. https://doi.org/10.1109/ICIP.2013.6738831
18. Dahl GE, Sainath TN, Hinton GE (2013) Improving deep neural networks for LVCSR using rectified linear units and dropout. In: ICASSP, IEEE international conference on acoustics, speech and signal processing—proceedings. https://doi.org/10.1109/ICASSP.2013.6639346
19. Chin WS, Zhuang Y, Juan YC, Lin CJ (2015) A learning-rate schedule for stochastic gradient methods to matrix factorization. In: Lecture notes in computer science (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics), vol 9077. https://doi.org/10.1007/978-3-319–18038–0_35

20. Lisetti CL, Rumelhart DE, Facial expression recognition using a neural network
21. Pham T, Worring M, Face detection methods: a critical evaluation. ISIS Technical
22. Ma Y (2011) Number local binary pattern: an extended local binary pattern. Wavelet Anal Pattern Recognit 10–13
23. Schultz T (2010) Facial expression recognition using surface electromyography

RecommenDiet: A System to Recommend a Dietary Regimen Using Facial Features



Dipti Pawade, Jill Shah, Esha Gupta, Jaykumar Panchal, and Ritik Shah

Abstract There is a steady rise in health-related issues in today's world. Due to the busy routine people are disinclined towards keeping check on weight, height and BMI values regularly. In addition to that unhealthy food habits, consumption of junk food and blindly following the diet routine leads to the major health issues. Thus, getting personalized dietary regimen on one click can solve the issue to certain extent. This motivated us to develop an application which can predict the height, weight and Body Mass Index (BMI) from the user's photograph and further use this information for recommending the diet plan. Here FaceNet model is used to get the facial features on which Multilinear Regression is implemented to predict the height, weight and BMI values. Only BMI is not enough to calculate the calorie consumption as the physical exercise done by the user plays important role in that. Thus BMR (Basal Metabolic Rate) is calculated gender wise and based on its daily calorie consumption is predicted which is further divided into four course meals. Finally, KNN algorithm is used to generate the weekly diet plan as per the food preferences given by the user.

Keywords Facial feature extraction · BMI estimation · FaceNet model · Dietary regimen · Diet plan · Machine learning

D. Pawade · J. Shah · E. Gupta · J. Panchal · R. Shah (✉)
Department of Information Technology, K. J. Somaiya College of Engineering, Vidyavihar, Mumbai, India
e-mail: ritik.ds@somaiya.edu

D. Pawade
e-mail: diptipawade@somaiya.edu

J. Shah
e-mail: jill25@somaiya.edu

E. Gupta
e-mail: esha.gupta@somaiya.edu

J. Panchal
e-mail: jaykumar.mp@somaiya.edu

© The Author(s), under exclusive license to Springer Nature Singapore Pte Ltd. 2023

S. Chinara et al. (eds.), *Advances in Distributed Computing and Machine Learning*, Lecture Notes in Networks and Systems 660,
https://doi.org/10.1007/978-981-99-1203-2_33

1 Introduction

Nowadays, there has been a rise in the number of diseases and fitness problems. People frequently visit doctors and dieticians in order to get diagnosed with various health related problems, prospect of obesity, etc. People have started looking for a healthy lifestyle. This can be done by proper workout and healthy diet. However, working out may not be possible for everyone. They can, however, adhere to a diet. Unfortunately, most people don't know how much and when to eat and what to avoid while following a healthy routine so we need something to assist us like a diet recommendation system based on some metric that involves height and weight of the person. For deciding the diet plan, as per their preference, people consider the general notion for proportion of nutrition and choose the items accordingly. However, diet plans differ for each person and dependent on their level of activity and BMR. The BMR is based on one's height weight and who are differently abled or knowing the current height and weight may not be possible. This motivated us to take the challenge to develop an application that enhances the user experience by taking a picture of the user and estimate the height, age which varies person to person. Thus, everyone needs a customized diet as per their physique and activity type. There are few solutions available for the diet plan generation like www.mydietmealplan.com, https://planmeal.com, https://www.strongrfastr.com/diet-meal-planner etc. However, the issue is that they require the user's height and weight as input. This may not be known to people. Weight and height are used to calculate BMR based on which calory consumption breakup and accordingly generate the detailed diet plan based on the dietary preferences.

The paper is systematized such that Sect. 2 covers the literature survey, methodology is explained in Sect. 3. Section 4 gives the Results and Sect. 5 explains the conclusion and elaborates on the future scope.

2 Literature Survey

The faces of humans have a variety of indicators, including age, identity, personality traits, expression, gender, and so on, that can be studied and used for various applications. Researchers have come up with the thought of linking face features to body weight in many aspects. For height and weight prediction from facial features many researchers have correlated it with BMI prediction or classification into BMI categories.

Bolukbas et al. [1] experimented with Support Vector Machine (SVM), Random Forest method and Gradient Boost method and then compared the results for BMI prediction. These models are trained using the differential regression technique and found that Gradient Boosted regressor showed the best results. Min Jiang et al. [2] used VGG-Face, LightCNN-29 and Centerloss for feature extraction and support vector regression (SVR) is used for BMI prediction. VGG-Face and Arcface perform

well among the techniques. Fook et al. [3] used Active Shaped Model for facial feature extraction. They compare three classifiers, viz. Artificial Neural Network (ANN), SVM and K Nearest Neighbors (KNN) and concluded that ANN has given the best results. Kocabey et al. [4] proposed BMI prediction method comprising of deep feature extraction and regression model training. They have employed two well-known deep models for feature extraction: one trained with general object classification (i.e. VGG-Net) and the other trained on a face recognition task. They used epsilon support vector regression models for BMI regression because of their robust generalization characteristics. When presented with a pair of profile pictures, the tool performs as well as humans in determining the more overweight person. Dantcheva et al. [5] predicted the height, weight and BMI using ResNet-50 model. The authors [6] computed the facial fatness based on the distance and ratios between the facial landmarks and implemented regression function to calculate the BMI. Siddiqui et al. [7] put forth weight monitoring application where facial features are investigated using VGG19, ResNet50, DenseNet, MobileNet, and lightCNN. It has been observed that ResNet50 and DenseNet showed better performance over other Convolutional Neural Networks (CNN). From the literature survey we found that regression method is widely used to predict the BMI and ultimately height and weight.

The other part of our research is to recommend the diet plan. The authors [8] identified three typical food recommender system methods: collaborative filtering recommender system (CFRS), knowledge-based recommender system (KBRS), and context-aware recommender system (CARS). Wellness recommender systems assist users in finding and adapting customized wellness therapies that are tailored to their specific needs. Artificial intelligence and semantic web approaches were used in food recommendation systems. Another study [9] used expert information to create a diet recommendation system for chronic diseases. The recommendation system in this work is built using an ontology, decision trees, and Jena. Based on the user's own situations, the algorithm will determine whether the user has a balanced diet. The system infers the relationships between foods and patients using the JENA inference engine, and then recommends a suitable diet for patients. To provide a more accurate nutritional suggestion, the dietary recommendation system involves knowledge as well as the user's personal information. The author [10] put forth a recommender system that anticipates users' preferences and then suggests new items to them based on those predictions using health psychology theory. The authors [11] discussed a recommender system that can analyze dietary intake using a validated Food Frequency Questionnaire (FFQ) and provide consumers with accurate tailored nutrition advice. Their major objective is to provide individualized online nutritional recommendations that take into account an individual user's tastes, population data, and expert knowledge to improve the diet quality at population level. Yum-me [12] is another personalized nutrient-based meal recommender system that uses a visual quiz-based user interface to create food preference profiling and then projects the learnt profile into the domain of nutritionally adequate meal options. Some authors [13] put forth a cloud-based food recommendation system that provides nutritional advice based on users' pathology data. The selection of diet patterns is done in real time-based nutritional needs. The authors used the ant colony method and claim

that increasing the number of ants results in satisfactory accuracy. The authors [14] have used the fuzzy model that tells the relation between food calorie value and the profile of the user. Age, sex, height, weight are taken as inputs to calculate BMI and BMR. A system was created that recommends a diet plan to the users using their preferences. Another application [15] of tracks the user's weight, BMI, and daily calorie consumption, recommend the meal based on the user's previous meals and preferences and also suggest the nearby the restaurant's name, address, cost, restaurant images, etc. who serves those items. The author [16], tried to provide efficient calorie intake tracking systems for diet plan recommendation. The system not only take care of proper calorie consumption but also consider appropriate combination of proteins, fats, and carbohydrates for it. The authors [17] propose a case-based approach to diet recommendations. The project's goal is to create an IT system that filters the available food goods by taking into account information from health profiles of clients or patients. They used open-source tools such as BIKE and WEKA for implementing the suggested approach. This paper [18] proposes a system that uses image processing and Convolutional Neural Networks. The objective of the system is to recommend users with a diet based on the input facial images of the feature. The first step is the detection of face from input image and extract the features. Data is taken from dietitians on every feature alongside its corresponding nutritional value e.g.: proteins, carbohydrates, etc. Then the food items are mapped to nutrients so that the predictions account for balancing the total nutritional value of the diet. A feedback mechanism can also be included to improve the system.

From the literature survey, it has been observed that there is a gap in providing the diet plan recommendation system that accept minimum input from user and consider physical activity type as well as dietary preference for diet plan generation specifically for Indian cuisine. This motivated us to come up with system that and provide efficient solution to this problem.

3 Methodology Overview

On signing-in, the user will be asked to upload their face image and along with that the user will have to provide details like age, gender, dietary preference like vegetarian and non-vegetarian. Also, activity type of an individual is very important to consider so the preferences are taken as no exercise, low exercise, moderate exercise and hard exercise. The dietary preference will be used to recommend the food items in the diet plan and activity type is used as one of the parameters to estimate the total calorie count. The rest of the system has two main modules: (1) Height and weight prediction and (2) Calories Consumption based diet plan generation.

The first module accepts the face image as an input, apply FaceNet module to extract the features and perform regression to predict the height, weight and BMI of the person. Diet plan generation module calculates the BMR based on height, weight, age and gender, estimates the calories requirement and recommends the weekly diet

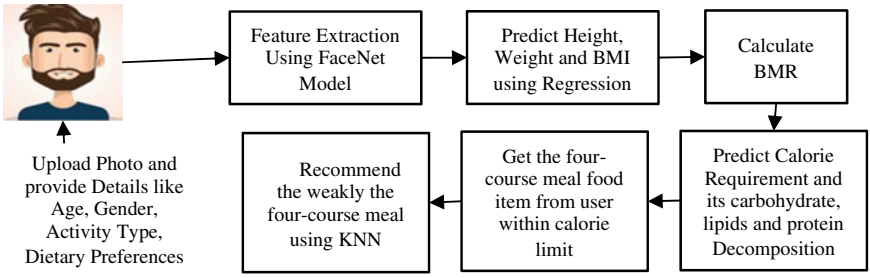


Fig. 1 Overview of the proposed system

plan based on the food preferences. Figure 1 gives the overview of the system. These modules are discussed in the subsequent sub-sections.

1. Height and Weight Prediction

For recommending diet plan, it is important to consider the calorie requirement of an individual and calorie requirement is based on one's BMI which is based on height and weight. So, the first task is to predict the height and weight of an individual. This section discusses about the process for the estimation of height, weight and BMI from the face image of an individual. We have explored different methods to get the best possible methodology for estimating height and weight. For the experimentation FIWBMI [2] and VIP [5] datasets is used. The VIP dataset contains 1026 facial images, specifically 513 female and 513 male images. The dataset contains a csv file with image name, height, weight and the BMI values and a folder with all the images. The FIWBMI dataset comprising of 8370 face images of each individual has 1 to 4 images. The annotation of each image consists of 5 parts: the first part denotes the individual, the second part denotes different images from the same individual, the third part denotes the body weight in kg, the fourth part denotes the height in meters, the fifth part denotes gender (true is female, false is male). Rescaling of dataset is done to help treat all images in the same manner. Each dataset is split into 80:20 for the training and testing sets respectively. These datasets underwent various pre-processing steps. For the FIWBMI dataset, a csv file was created by extracting the height, weight, and name values of each image. After this, the training is done to extract the facial feature vector for each image in datasets. If the user uploads an image that contains a group of people, or an image that is not clear, a warning will be shown advising the user to upload a clear face image, considering this is an outlier condition. Since images that do not contain a proper face, the feature vector obtained has zero values which helps in this validation. Three methods were used to extract facial features, but the best one is used in actual implementation. The three methods used were using CNN model, using OpenCV and lastly using a pretrained model called FaceNet. Efficient results were obtained using the FaceNet model.

Figure 2 depict the FaceNet model architecture. This gave a 128-feature feature vector as shown in Fig. 3 which was used for prediction. For the estimation of height, weight and BMI, the Machine Learning techniques like regression and classification

were tested. Many algorithms like SVR with various kernel values, Lasso Regression, Random Forest, Multiple Linear Regression, and Ridge Regression were tried for regression and SVM with different kernel values, Naive Bayes, Logistic Regression, Stochastic Gradient Descent, K-Nearest Neighbours (KNN), Random Forest and Decision Tree were tried for classification were tried. The best results obtained for prediction were using Linear regression. Hence, the process used in the web app is to find the facial feature vector using a FaceNet model and then applying linear regression to predict the height, weight and BMI. Height and weight values are used further to calculate the BMR.

2. Calorie Consumption Based Diet Plan Generation

The system uses height and weight values predicted in the previous module as an input and calculates the BMR [19]. Apart from height and weight, the system needs the age and gender of the user so that information is fetched from the database directly. Figure 4 shows the pseudo code for BMR calculation.

The next step is to calculate the calorie requirement of the user. Generally, the calorie requirement is directly based on the BMR with respect to the activities of the user. For example, if two people may have the same BMR but one's lifestyle is active and other's is sedentary or no exercise then definitely the calorie requirement of the first one will be greater than the second. Once the daily total calorie requirement of the user is calculated then it divided into four meals i.e., breakfast, lunch, snacks and

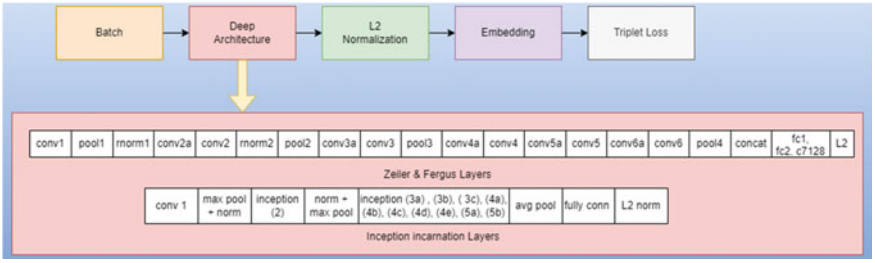
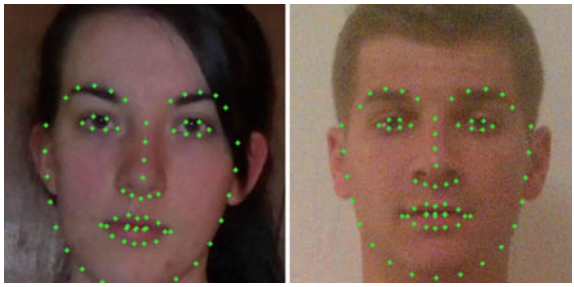


Fig. 2 FaceNet architecture

Fig. 3 Facial landmarks



```
Pseudocode for calculating BMR:
    Input:    H ← Body height, W ← Body weight, A ← Age, G ← Gender
    Output: BMR ← Basal Metabolic Rate
    IF G == "Male" THEN BMR = 88.362 + (13.397 * W) + (4.799 * H * 100) - (5.677 * A);
    IF G == "Female" THEN BMR = 447.593 + (9.247 * W) + (3.098 * H) - (4.330 * A);
```

Fig. 4 Pseudocode for calculating BMR

```
Pseudocode for calculating calories for each meal:
Input: H←Height, W←Body weight, A←Age, G←Gender, AT←Activity Type, MT←Meal Type
Output: cals ← Calories for each meal (breakfast, lunch, snacks, dinner)

    BMR = Calculate_BMR(G, H, W, SA)   // Basal Metabolic Rate
    Typedefenum {"No Exercise", "Light Exercise", "Moderate Exercise", "Hard Exercise"} AT;
    Typedefenum {"Breakfast", "Lunch", "Snacks", "Dinner"} MT;

    Case AT of
          1: {IF AT == 1 THEN TC = BMR * 1.2;} Break;  // Total Calories (TC)
          2: {IF AT == 2 THEN TC = BMR * 1.375;} Break;
          3: {IF AT == 3 THEN TC = BMR * 1.55;} Break;
          4: {IF AT == 4 THEN TC = BMR * 1.725;} Break;
     Case MT of
          1: {IF MT == 1 THEN cals = 0.30 * TC;} Break;
          1: {IF MT == 2 THEN cals = 0.38 * TC;} Break;
          1: {IF MT == 3 THEN cals = 0.07 * TC;} Break;
          1: {IF MT == 4 THEN cals = 0.25 * TC;} Break;
```

Fig. 5 Pseudocode for calculating calories for each meal

dinner [20, 21]. Figure 5 gives the pseudo code to calculate total calorie requirement and its decomposition in four meals.

Once the calorie decomposition for each meal is ready then next step is to recommend the diet plan. For diet plan recommendation, the biggest challenge was to get the appropriate dataset with Indian Food. There are certain datasets like USDA dataset, COVID-19 Healthy Diet Dataset, etc. which includes the food names and their calorie count but they all contain continental cuisine. There are some datasets like Indian Food 101 dataset, 6000 + Indian Food Recipes Dataset which contains the Indian food item and their ingredients. However, we needed a dataset of Indian food items that included carbohydrate, proteins, fats, and calorie information. So, we stripped the names of food items from these datasets and their carbohydrates, proteins, fats and calorie specification is taken using the Nutritionix API. Our dataset contains a total 745 food items with their nutritional information like calories, fats, proteins, carbohydrates, the serving size quantity and are labelled as veg and non-veg.

Figure 6 demonstrates the weekly diet plan generation process. For the first day users need to choose their preferred food items for each meal according to the required calories. For this, the user will click on the Add Item button, then search for a specific food item present in the database. If the calories of that food item exceed the required calories for that particular meal, then the user will be displayed a relevant error message. This way users need to choose the food items for a 4-course meal on the first day. Now, the food items chosen by the user for Day 1 will be used to recommend

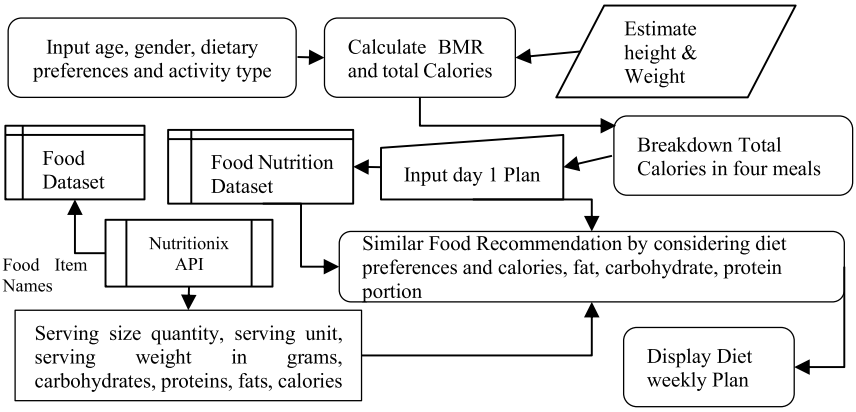


Fig. 6 Weekly diet plan generation

similar food items for the rest 6 days of the week. For that, the K Nearest Neighbors (KNN) algorithm is used. The parameters of KNN algorithm are 7 k_neighbors, ball_tree algorithm and metric as Euclidean distance. The KNN is done with respect to the calories, proteins, carbohydrates and fats of the user selected food items to find other food items with similar nutritional value in the dataset. Similar food items will be then added to the diet plan for each day from Day 2 to Day 7 according to the meal type. The user will be shown a page to view the Diet Plan for the whole week where using tabs the diet plan for each day can be viewed. The user can view the whole week's diet plan in one table where they can also save it as a pdf.

4 Results and Discussion

Table 1, covers a case study where a sample image is provided to the system and how system process that input and gives the end result is demonstrated. As discussed in the methodology section, VIP and FIWBMI datasets are used to estimate the height, weight and BMI values. For the testing purpose various regression algorithms are applied on these datasets and Mean Squared Error (MSE) and Root Mean Squared Error (RMSE) values are used as a metric to evaluate the performance of the system. Figures 8 and 9 gives MSE and RMSE values for different regression methods on VIP and FIWBMI dataset respectively. From these figures, it can be clearly seen that The FaceNet model with Multiple Linear Regression method gives the Lowest MSE and RMSE values for both the dataset and hence proved to be the most efficient way to predict height, weight and BMI (Fig. 7).

In the diet plan generation module, KNN is used to predict the other dishes to include in a diet plan. The MSE and RMSE obtained for similar recommendation of food items using KNN is 18.086 and 4.252 respectively.

Table 1 Case study

Step 1: Fig. 7 shows the sample image uploaded and following other details Age → 25 years, Gender →Male, Activity Type →Light Exercise Dietary Preference →Vegetarian	–

Predicted values by the system are:
Height →1.87 m Weight → 80.955 kg BMI → 23.064 kg/m2
BMI Category →Normal BMR →1928.404135 Total Calories for a day → 2651.56
Calorie Breakdown into for meat: Breakfast →795.3 cals, Lunch →1007.38 cals, Snacks →185.57 cals, Dinner →662.7 cals

Following is the sample recommended plan

–	Breakfast	Lunch	Snacks	Dinner
Day 1	Poha, Vanilla Ice Cream, Vegetable Jalfrezi	Chapati, Aloo Gobi, Kakinada, Khaja, Mawa Bati, Split Pea Soup	Tea, Apple	Pulav, Kakinada, Khaja Choco Icecream
Day 2	Quninoa Chorafali, Dosa	Rice, Sheer Korma, Kaju Katli, Eggplant	Fara Cantaloupe	Sheer Korma, Kaju Katli, Rice
Day 3	Poha, Vanilla IceCream, Vegetable Jalfrezi	Chapati, Aloo Gobi, Bhindi Masala, Mawa Bati, Split Pea Soup	Tea, Apple Vinegar	Strawberry Icecream, Popcorn
Day 4	Keerai Masiyal Aloo Methi Papad	Rajma Chaval Strawberry Icecream Kakinada Khaja Bhindi Masala, White Rice	Watercress Stems, French-Fries	Aloo Gobi, Chevdo, Rajma, Roti
Day 5	Keerai Poriyal, French Fries, Kachori	Upma, Mishti Chholar, Dal, Mawa Bati, Kakinada Khaja Black Rice	Green Tea Bombil Fry	Mishti Chholar Dal, Mawa Bati, Til Pitha
Day 6	Sweet Potatoes, Mexican Rice, Heated Peas	Lassi, Steamed Peas, Pindi Chana, Turiya Patra, Paratha	Celery, Dark Chocolates	Fresh, Steamed Peas Pindi Chana Kalakand
Day 7	Grape Juice, Masala Dosa	Pulav Pav Bhaji, Patra	Coffee Iceberg	Karela Bharta, Thepala



Fig. 7 User input

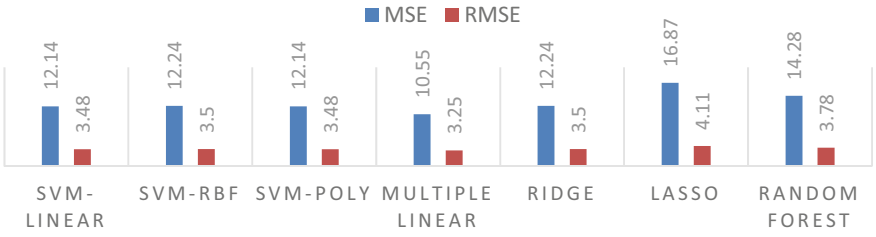


Fig. 8 Regression performance evaluation on VIP dataset

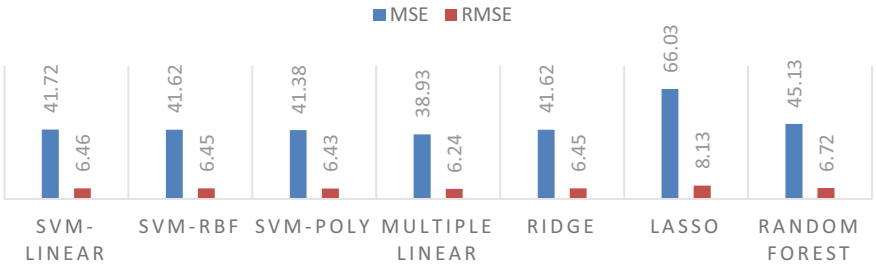


Fig. 9 Regression performance evaluation on FIWBMI dataset

5 Conclusion

In this paper, we have discussed personalized diet plan recommendation using the facial image. The system extracts facial features from an individual's face image, which are then used to estimate height, weight, and BMI. For the facial features extraction various methods were tested and the best results were obtained when FaceNet model was used. Furthermore, regression was used to estimate height, weight, and BMI. The regression metric chosen was RMSE. The lowest RMSE was observed using Multiple linear regression. VIP dataset had an RMSE of 3.25, while FIWBMI dataset had an RMSE of 6.2. The estimated height and weight were used

to calculate the BMR which in turn was used to calculate the total required calories in a day for the individual. The user's preferred foods are taken into consideration, and then similar foods are suggested to generate a diet plan for the entire week.

Due to few entries in the food dataset, very limited and repeated food items are included in the diet plan. Thus, the dataset of Indian food items can be enriched to have a large variety of items. Currently the application consider standard serving size for food. In future system can be upgraded to work with the customised serving size too. Additionally, the user image can be taken using a camera directly. This will lead to capturing of the most recent picture of the user and can give best results. Also, the age and gender prediction can be applied on that photo itself so that the user does not have to input the age and gender.

References

1. Bolukbas G, Başaran E, Kamaşak ME (2019) BMI prediction from face images
2. Jiang M, Shang Y, Guo G (2019) On visual BMI analysis from facial images. Image Vis Comput 89
3. Fook CY, Chin LC, Vijean V, Teen LW, Ali H, Nasir ASA (2020) Investigation on body mass index prediction from face images
4. Kocabey EC, Ofli M, Aytar F, Mar´ın Y, Torralba J, Weber A, Ingmar (2017) Face-to-BMI: using computer vision to infer body mass index on social media
5. Dantcheva A, Bremond F, Bilinski P (2018) Show me your face and I will tell you your height, weight and body mass index
6. Barr M, Guo G, Colby S, Olfert M (2018) Detecting body mass index from a facial photograph in lifestyle intervention
7. Siddiqui H, Rattani A, Kisku DR, Dean T (2020) Al-based BMI inference from facial images: an application to weight monitoring
8. Norouzi S, Nematy M, Zabolinezhad H, Sistani S, Etminani K (2017) Food recommender systems for diabetic patients: a narrative review
9. Chen R, Huang C, Ting Y (2017) A chronic disease diet recommendation system based on domain ontology and decision tree
10. Trang Tran T, Atas M, Felfernig A, Stettinger M (2017) An overview of recommender systems in the healthy food domain
11. Franco R (2017) Online recommender system for personalized nutrition advice
12. Yang L, Hsieh C, Yang H, Pollak J, Dell N, Belongie S, Cole C, Estrin D (2017) Yum-Me: a personalized nutrient-based meal recommender system
13. Rehman F, Khalid O, Haq N, Khan A, Bilal K, Madani S (2017) Diet-Right: a smart food recommendation system
14. Hussain MA et al (2018) Income based food list recommendation for rural people using fuzzy logic
15. Carvalho M, Kotian P, George H, Pawade D, Dalvi A, Siddavatam I (2021) Implementation of smart diet assistance application
16. Shrimal M, Khavnekar M, Thorat S, Deone J (2021) Nutriflow: A diet recommendation system
17. Kovásznai G (2011) Developing an expert system for diet recommendation
18. Chheda T, Bhinderwala H, Acharya P, Nimkar AV (2019) Diet and health advisory mechanism from facial features using machine learning
19. Calories: Recommended intake, burning calories, tips, and daily needs, Medicalnewstoday.com (2018). https://www.medicalnewstoday.com/articles/245588#daily_needs

20. Meal Calorie Calculator, Omnicalculator.com. https://www.omnicalculator.com/health/meal-calorie
21. Meal Calorie Calculator, Calculator-online.net. https://www.calculators.tech/meal-calorie-calculator

Image Transformation Based Detection of Breast Cancer Using Thermograms



Vartika Mishra, Shibashis Sahu, Subhendu Rath, and Santanu Kumar Rath

Abstract In today's health sector scenario, it is availed that breast cancer is the major cause of death among urban women. Every third woman is suspected to be a victim of breast cancer. The detection of tumor at an early stage can prevail a better chance of survival. Among different modalities, thermography is observed to be an early detector of tumor. It records the temperature of the surface and thus it helps in identifying the tumor due to the indication of hot colors identified in thermography. In this study, we propose an approach for detection of breast cancer using thermograms based on image transformation methods. The input images are considered from the database available at DMR, Visual Labs, Brazil. The texture variations are better captured through the different transform methods. Thus, to capture the variation in the texture caused by the tumor, the images are transformed using the three different image transformations methods viz. Wavelet transform, Curvelet transform and Lifting Scheme based Wavelet transform. Then features are extracted from the transformed images and ranked using different techniques. The final set of features are classified using different classifiers viz. Support Vector Machine, K-Nearest Neighbor, Naïve Bayes, Random Forest, and Multi-Layer Perceptron. Among the different classifiers, random forest gives the best accuracy with lifting scheme-based wavelet transform images of 93.89%.

Keywords Breast cancer · Wavelet transform · Curvelet transform · Lifting scheme transform · Ranking methods · Classification

V. Mishra (✉) · S. K. Rath
NIT Rourkela, Rourkela 769008, OR, India
e-mail: vartikamishra151@gmail.com

S. K. Rath
e-mail: skrath@nitrkl.ac.in

S. Sahu
Department of Engineering, Leoforce India Pvt. Ltd., Hyderabad, India
e-mail: shibusahu30@gmail.com

S. Rath
VCU, Richmond, VA 23219, USA
e-mail: subhendu.rath@vcuhealth.org

© The Author(s), under exclusive license to Springer Nature Singapore Pte Ltd. 2023

S. Chinara et al. (eds.), *Advances in Distributed Computing and Machine Learning*,
Lecture Notes in Networks and Systems 660,
https://doi.org/10.1007/978-981-99-1203-2_34

1 Introduction

Among two hundred different types of cancer, breast cancer is observed to be one of the most direful diseases that has an alarming toll of deaths recorded worldwide. Studies have observed that 23% of total cancer cases and 14% of the cancer deaths are diagnosed in the urban women worldwide [1]. This rate of deaths can decrease in number if the treatment is taken at an early stage.

However, many different modalities have been taken into consideration for the detection of breast cancer such as Ultrasound, Mammography, Magnetic Resonance Imaging (MRI,) X-Ray and Thermography etc. These modalities have helped in reducing the mortality rate from 30–70% by assisting the radiologists and physicians in diagnosing the abnormalities [2]. In Mammography structural imaging modality, X-rays exposed are used for image screening where the breast are compressed resulting in poor performance among young women. It is observed that tumor of average size 1.66 cm goes undetected while screening by using mammography [3, 4].

Infrared Thermography is non-ionizing, non-contact and radiation free technique. It detects the thermal pattern of human body, and as human body is thermally symmetric so, any variation or asymmetric of temperature can signify the presence of unhealthy cells. Asymmetric nature of left and right part of breast can give hint for abnormality as this abnormality side will lead to higher temperature when compared to the normal cells [5]. Presence of cancerous cell leads to higher metabolic activity which in turn increases the temperature as compared to the normal cell [6].

In this work, the role of thermography image transformation is emphasized for the detection of breast cancer between healthy and unhealthy subjects. So, the breast thermograms are transformed using the three different image transformations methods, viz. Wavelet, Curvelet and Lifting Scheme wavelet transforms. The different texture features such as Gray Level Co-occurrence Matrix (GLCM), Gray Level Run Length Matrix (GLRLM), Neighborhood Gray Tone Difference Matrix (NGTDM), Gray Level Dependence Matrix (GLDM) and Gray Level Size Zone Matrix (GLSZM) are extracted from these transformed images. Further, the features are ranked using five different ranking methods to select the common features and then different classifiers are applied for comparison of performance accuracy obtained for classifying healthy and unhealthy breast thermograms.

The paper is organized as follows: Sect. 2 discuss the related work followed by Sect. 3 gives the dataset description. Section 4 explains the methodology proposed for this work. Section 4 analyses results for the proposed approach and Sect. 6 concludes the whole work.

2 Related Work

In breast cancer detection, thermography was observed to be limited in the initial years due to low sensitivity, unable to differentiate between inflammatory hot spots and real cancerous tumors. Over these years of research, these limitations are somehow overcome by advanced feature extraction techniques. This section represents a few kinds of literature that focus on the feature extraction methods for the breast thermograms based on a variety of features and their analysis.

In [7], by applying wavelet transform statistical features are extracted. Further for classification, ANN is applied for classifying healthy and unhealthy breast thermogram. Pseudo colored thermogram images were used in their work. In [8], different features such as energy and amplitude are extracted by applying the Gabor wavelet transform method. It was observed that normal and abnormal tissues showed a maximum of 6% difference in average energy value at an orientation of at 150°.

In [9], temperature values are extracted as a measure with eight level of orientations by standard deviation. The performance of Support Vector machine (SVM) has shown efficient results in sensitivity and specificity rate. In [10], different texture and statistical features are extracted in the curvelet domain for classifying the breast thermograms as healthy and unhealthy breast. In [11], different features based on run length features are extracted by separation of left and right breast regions. These extracted features from region of interest are used for the classification using support vector machine technique.

3 Dataset Description

In this work, the dataset comprises of total thermograms of 56 subjects each having 20 positional temperature matrices. This dataset is available at a public repository at Database for Mastology Research (DMR) of UFF, Brazil [12]. The breast thermograms are captured the thermal camera FLIR SC-620 with a spatial resolution of 640×480. Out of 56 subjects, 37 are unhealthy and 19 are healthy.

4 Proposed Methodology for Detection of Breast Cancer

In this work, breast thermograms are obtained by transforming the temperature matrix into images. Gray level images are well scrutinized with the help of the transform methods. The different transforms are well-known multiresolution analysis tool, which decomposes source images into different resolutions. The information of an image at different resolution levels, usually represents different physical structures present in the image. Thus, on this obtained image wavelet, curvelet, and lifting transform are applied followed by different texture feature extraction methods viz.

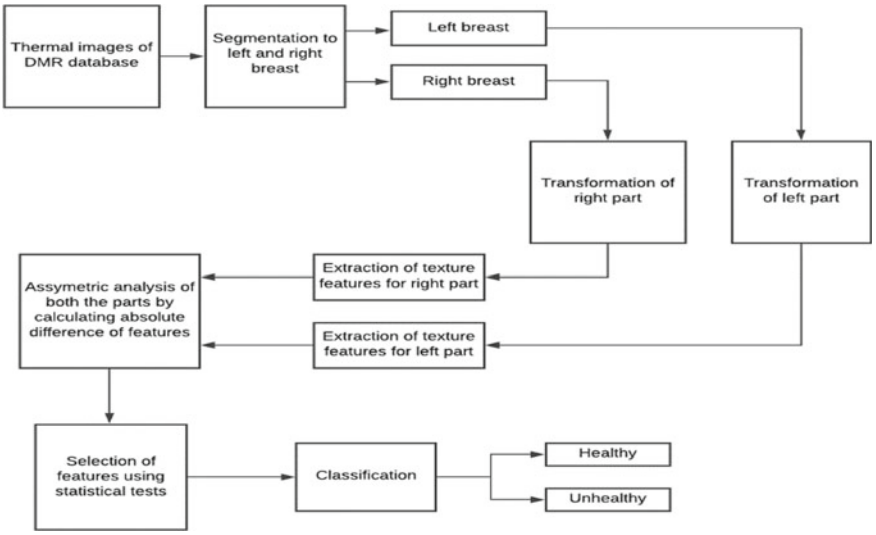


Fig. 1 Block diagram representation of our proposed work

GLCM, GLSZM, GLRLM, NGTDM, GLDM as the statistical features describe very precisely the characteristic of a distribution, if each patch represents a random distribution. After extracting the features from thermograms, ranking methods are applied to select the most distinctive features as they help to discriminate between the significant and non-significant features. These distinct features obtained are then classified for healthy and unhealthy subjects as shown in Fig. 1.

4.1 Image Preprocessing

The breast cancer detection between healthy and unhealthy breast thermograms is carried out by transforming the images obtained from the temperature matrix available online at DMR lab [12, 13]. The images are further segmented between left and right breast using the manual segmentation method as shown in Fig. 2.

4.2 Image Transformation

Image transformation is a function that maps set of input values to corresponding output values according to the operation performed. In this work, Wavelet Transform, Curvelet transform and Lifting scheme-based wavelet transform techniques are applied for image transformation. Texture of the abnormal image vary rapidly due to presence of tumor cells, which can be better captured using these techniques.

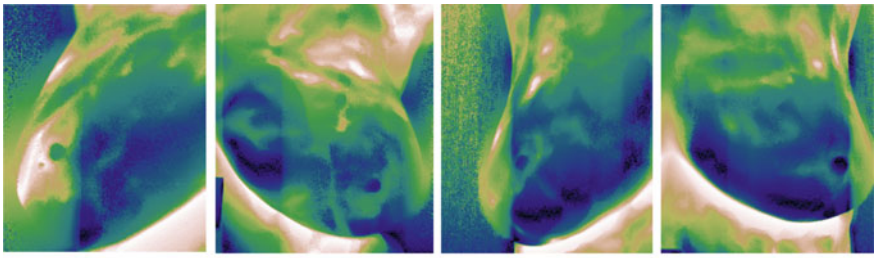


Fig. 2 Segmented left and right part thermograms of **a** healthy and **b** unhealthy

a. Wavelet Transform

The thermal emission of cancerous cell leads to the temperature variation which can be detected using wavelet transform. Wavelet transform of an image; f is defined as [14]:

$$W_{\varphi}(j_0, k, l) = \frac{1}{\sqrt{MN}} \sum_{m=0}^{M-1} \sum_{n=0}^{N-1} f(m, n) \times \varphi_{j_0,k,l}(m, n) \tag{1}$$

$$W_{\psi}^{i}(j, k, l) = \frac{1}{\sqrt{MN}} \sum_{m=0}^{M-1} \sum_{n=0}^{N-1} f(m, n) \times \psi_{j,k,l}^{i}(m, n) \tag{2}$$

where,

- i = {H, V, D} represents directional index Horizontal, Vertical and Diagonal,
- j_0 = scale, M = number of rows, N = number of columns,
- $W_{\varphi}(j_0, k, l)$ = Approximation coefficient of $f(m, n)$ at scale j_0,
- $W_{\psi}^{i}(j, k, l)$ = Detailed coefficient at ith direction with $j \geq j_0$,
- φ and ψ are scaling and wavelet functions, respectively.

2-D analysis uses the two-dimensional convolution function where scaling function is represented as ϕ and corresponding wavelet function is represented as ψ and are applied first in horizontal direction then in vertical direction.

The two-dimensional discrete wavelet transform is applied along horizontal direction then along vertical direction of the image. Figure 3 shows the steps for DWT transform. Here, 2D DWT transforms the Approximation coefficient at scale j + 1 to produce the Approximation and Detailed Coefficients at scale level j. First rows are convoluted with $h_{\varphi}(-l)$ and $h_{\psi}(-l)$ and columns are down sampled and we get two sub-bands whose resolution along the horizontal direction is reduced by a factor of two. Then columns are convoluted and rows are down sampled and we get four sub-band images whose both resolutions along the vertical and the horizontal is reduced by two [20]. They are LL, LH, HL and HH sub-bands, where H signifies High-pass filter and L signifies Low-pass filter. For next level of decomposition,

wavelet transformation is applied to approximation coefficient of current level. In our work, we have used two level of decomposition as shown in Fig. 3. DWT transformed Approximation Coefficient images using daubechies-1 family are shown in Fig. 4. using level 1 transformation.

b. Curvelet Transform

The image is discretized using a wrapping algorithm. This transformation technique consists of transforming image to form a set of scales and orientations which are wrapped around the origin. The approximate scale and orientation are represented

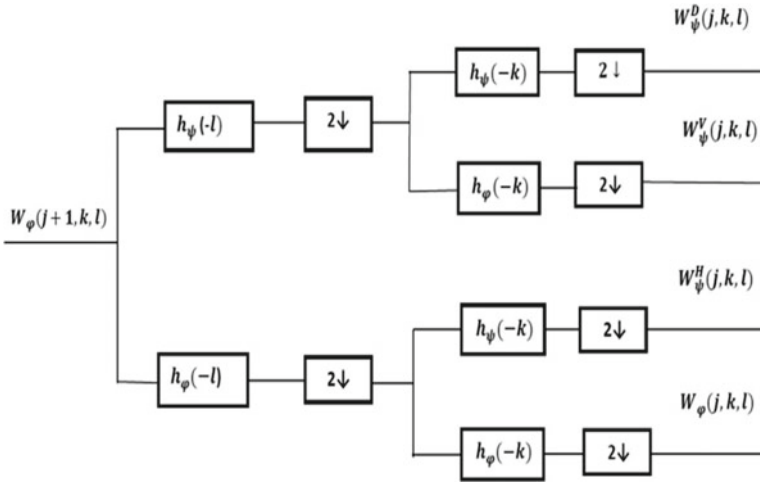


Fig. 3 2D-DWT filtering using scaling and wavelet functions

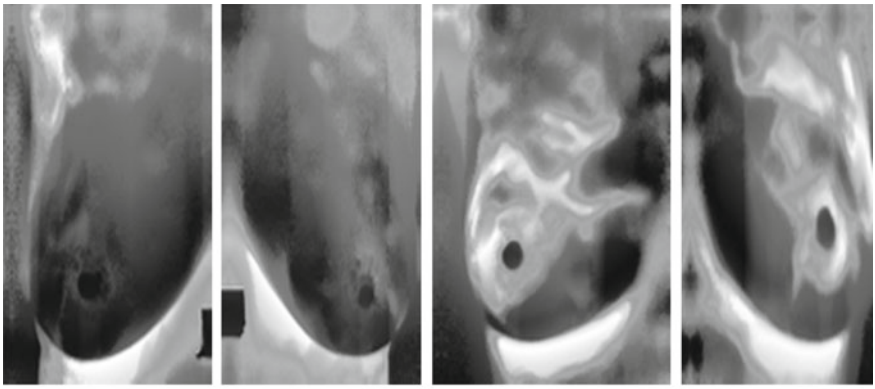


Fig. 4 Wavelet transformed images with level 1 decomposition for **a** healthy and **b** unhealthy patient

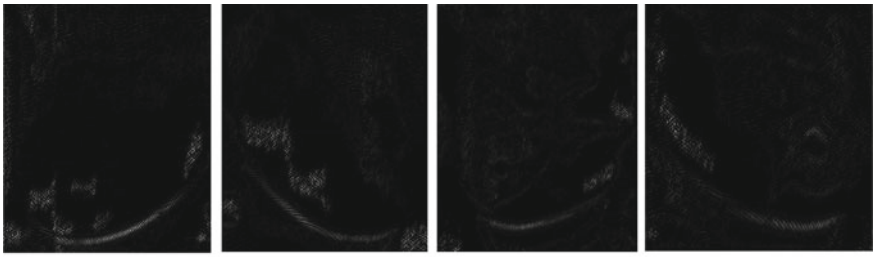


Fig. 5 Curvelet transform at scale 2 and orientation 16 for **a** healthy and **b** unhealthy breast thermograms

as wedge. Parameters that are involved in curvelet transformation are resolution and orientation. Image is transformed into sub-images using different scale and orientation as shown in Fig. 5. Increasing the number of scales and orientations may result in reduction of information and increase in sharpness of the image [15]. Image is transformed to 8 orientations in scale 1 and 16 orientations to scale 2. Images generated by fast discrete curvelet transform (FDCT) are shown in Fig. 5.

c. Lifting Scheme Based Wavelet Transform (LWT)

Lifting scheme on DWT forms the new generation technique for wavelet transformation [16]. This lifting step module consists of three major steps viz. splitting, lifting which consists of predict and update and last step is normalization or scaling. In splitting, input one dimensional signal is split to even and odd component using Lazy lifting scheme. For one dimensional input I(k), even component is calculated as $E(k) = I(2 * k)$ and odd component is computed as $O(k) = I(2 * k + 1)$. Next comes the lifting step, which consists of one or more update and/or predict operations depending on the family of wavelet. In predict step, predict filtering function P() is applied on even coefficient to compute the odd coefficient while in update step, an update filtering function $U()$ is applied on odd coefficient to predict new even coefficient. Finally in scaling step, two coefficients N_L and N_H are applied on even and odd coefficients respectively to produce coefficients corresponding to low-pass and high-pass filter. For normalized transform, N_L and N_H are multiplicative inverse of each other.

This technique is faster as compared to wavelet transform and is easier to implement. It uses in-place memory for transformation which reduces the memory usage. Figure 6 shows the images obtained by applying LWT method.

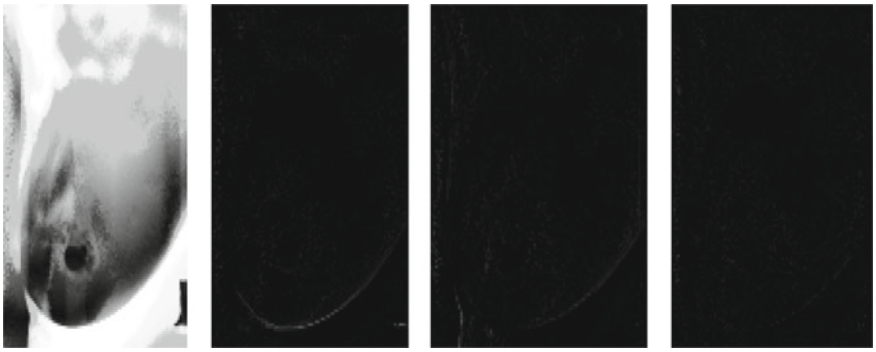


Fig. 6 LWT; level-1; db-1 (LL, LH, HL, HH)

4.3 Feature Extraction

Texture Feature signifies the measure of relationship between the pixels in each area. In this work, second order statistical feature extraction techniques viz. GLCM, GLSZM, GLRLM, NGTDM and GLDM signifies the properties of two or more pixels at a location are applied for the extraction of the features [16–20].

4.4 Feature Ranking

Different statistical tests are applied to infer whether a feature is prominent in discriminating between normal and abnormal cells. Considering Null Hypothesis and Alternate Hypothesis as a measure to differentiate, where Null Hypothesis (H_0) for this work defines: if both the samples are similar whereas Alternate Hypothesis (H_a) means both the samples are statistically different. Hypothesis test is used by comparing the p-value with the significance level ($\boldsymbol{\alpha}$). If $\boldsymbol{p - value \geq \alpha}$ then, accept Null Hypothesis. In this work, we are considering confidence level of 99.5%, which means α in our case is 0.005 [24]. Here, we are comparing two groups that are: features related to healthy and unhealthy breast thermograms. Different statistical tests viz. Student's test, Welch's test, Mann–Whitney test, Kruskal Wallis test and Z-test are performed [20–23].

5 Results

In this work, the breast thermal images from DMR dataset are segmented into left and right part. Images are transformed using wavelet transform as shown in Fig. 1. with one level of decomposition using Daubechies-1 (Db-1) family and level 1 approximation coefficient are used for further work. In curvelet transform, 8 orientations are taken for scale 1 and 16 orientations at scale 2. This curvelet transform produces one approximate coefficient at scale 0, eight detailed coefficients at scale 1 and sixteen detailed coefficients at scale 2 and for further work, coefficients of scale 2 and 16th orientation are considered. In Lifting Scheme based Wavelet transform, images are transformed at level one decomposition using Daubechies-1 (Db-1) family and approximation coefficient at level one is considered for the extraction of features.

For healthy and unhealthy breast thermograms, texture features of both right and left breast are extracted using GLCM, GLSZM, GLDM, NGTDM and GLRLM. Total 75 features are extracted. For comparison of bilateral symmetry, we subtracted feature set of left and right part and took the absolute value to form one single feature set for feature ranking and classification purpose. Common features are selected by considering significance level (α) of 0.005 for Mann–Whitney U test, Student's t-test, Welch's t-test, Kruskal–Wallis H test and Z-test. Dataset is split into 7:3 ratio for training and testing purpose. For k-NN classifier, we used k-fold cross validation of tenfold to find the optimal k value. Four different kernels viz. Polynomial, Linear, Sigmoid and Radial Basis Function (RBF) are used for SVM classifier. In MLP, activation function of rectified linear transformation function (ReLu) is used in hidden layer of 512 neurons and sigmoid function is used in output layer with one neuron. The performance of different classifiers with different performance metrics are shown in Tables 1, 2 and 3 for three different methods applied viz. Wavelet transform, Curvelet transform and Lifting Scheme based Wavelet transform.

For Wavelet Transform, images are transformed using Daubechies-1 of wavelet family. After applying feature ranking methods and by considering significance level of 0.005, the number of features is reduced to 31. $K = 3$ is chosen for k-NN classifier. For Polynomial kernel, a degree of 3 is used. Random Forest ensemble with 330

Table 1 Comparison of performance of classifiers for wavelet transform (%)

Classifiers	Accuracy	Precision	Recall	F1-Score
Naïve Bayes	67.2	50.42	98.0	67.04
k-NN	46.11	53.73	41.67	48.62
SVM (rbf)	87.87	86.54	75.0	80.36
SVM (linear)	84.44	83.33	66.67	74.07
SVM (polynomial)	75.00	85.71	62.00	44.44
SVM (sigmoid)	66.67	50.00	61.67	55.23
Random forest	89.23	96.0	82.26	88.89
MLP	72.78	61.22	50.00	55.04

Table 2 Comparison of performance of classifiers for curvelet transform (%)

Classifiers	Accuracy	Precision	Recall	F1-Score
Naïve Bayes	65.55	48.27	46.67	47.46
k-NN	74.44	70.59	40.00	51.06
SVM (rbf)	66.11	57.06	43.33	35.78
SVM (linear)	67.22	51.85	53.84	32.18
SVM (polynomial)	62.77	44.67	54.26	42.36
SVM (sigmoid)	57.78	38.23	43.33	40.62
Random forest	46.11	48.64	52.46	38.49
MLP	70	65	51.67	42.5

Table 3 Comparison of performance of classifiers for lifting scheme-based wavelet transform (%)

Classifiers	Accuracy	Precision	Recall	F1-Score
Naïve Bayes	62.78	47.2	96.46	63.78
k-NN	63.89	43.59	58.78	34.34
SVM (rbf)	86.11	72.73	93.24	81.75
SVM (linear)	88.33	74.68	98.33	84.89
SVM (polynomial)	74.44	73.33	36.67	48.89
SVM (sigmoid)	61.67	43.66	51.67	47.33
Random forest	93.89	88.89	93.51	91.06
MLP	63.89	54.29	41.67	52.99

Decision Tree and Gini Index split is used for finding the best split. After applying all the different classifiers as mentioned above, it is observed that Random Forest gives the highest accuracy, precision, and F1-Score as 89.23, 96.0 and 88.89%. However, Naïve Bayes gives the highest recall, as 98%.

For Curvelet Transform, Images at scale 2 and orientation 16 are used for classification. After applying feature ranking and by considering significance level of 0.005, the number of features is reduced to 22. $K = 3$ is chosen for k-NN classifier. For Polynomial kernel, a degree of 3 is used. Random Forest ensemble with 160 Decision Tree and Entropy split is used for finding best split. After applying all the different classifiers as mentioned above, it is found that k-nearest neighbor gives highest accuracy of 74.44%.

For Lifting Scheme Based Wavelet Transform, images are transformed using Daubechies-1 of wavelet family. After applying feature ranking methods and by considering significance level of 0.005, the number of features is reduced to 44. $K = 9$ is chosen for k-NN classifier. For Polynomial kernel, a degree of 3 is used. Random Forest ensemble with 320 Decision Tree and Gini Index split is used for finding the best split. After applying all the different techniques, it is found that Random Forest gives the highest accuracy, precision and F1-score as 93.89%, 88.89%

and 91.06. However, SVM (linear) gives the highest recall as 98.33%. Thus, it has been demonstrated that Lifting scheme-based texture features can effectively better represent breast surface temperature variations in an unhealthy breast.

It is be observed that from the above mentioned three tables, among all the three transform methods, lifting scheme transform method outperforms the other two methods i.e., wavelet transform and curvelet transform.

6 Conclusion

In this work, we have proposed a technique for the detection of breast cancer by transforming the images using wavelet, curvelet and lifting scheme transformation methods and extracted different texture features. Among the three mentioned image transformation methods, we observed that Lifting scheme-based wavelet transform using Daubechies-1 (Db-1) improves the accuracy when compared with the other two transform methods. Further, future work can be extended in attaining better accuracy by applying hybrid methods may be applied.

References

1. Jemal A, Bray F, Center MM, Ferlay J, Ward E, Forman D (2011) Global cancer statistics. Cancer J Clin 61(2):69–90
2. Schneider M, Yaffe M (2000) Better detection: improving our chances, Digital Mammography. In: 5th international workshop on digital mammography
3. Keyserlingk JR, Ahlgren PD, Yu E, Belliveau B (1998) Infrared imaging of breast: initial reappraisal using high-resolution digital technology in 100 successive cases of stage I and II breast cancer. Breast J 4:241–251
4. Ng EYK, Ung LN (2001) Statistical analysis of healthy and malignant breast thermography. J Med Eng Technol 25:253–263
5. Qi H, Kuruganti PT, Snyder WE (2012) Detecting breast cancer from thermal infrared images by asymmetry analysis. Med Med Res
6. Gogoi UR, Majumdar G, Bhowmik MK, Ghosh AK, Bhattacharjee D (2015) Breast abnormality detection through statistical feature analysis using infrared thermograms. In: 2015 international symposium on advanced computing and communication (ISACC). IEEE, pp 258–265
7. Pramanik S, Bhattacharjee D, Nasipuri M (2015) Wavelet based thermogram analysis for breast cancer detection. In: International symposium on advanced computing and communication (ISACC) Silchar, India. IEEE, pp 205–212
8. Suganthi SS, Swaminathan R (2014) Analysis of breast thermograms using gabor wavelet anisotropy index. J Med Syst 38(9):101
9. Borchartt TB, Martins A, Lima RCF (2011) Thermal feature analysis to aid on breast disease diagnosis. In: Proceedings of the 21st Brazilian congress of mechanical engineering RN, Brazil
10. Francis SV, Sasikala M, Saranya S (2014) Detection of breast abnormality from thermograms using Curvelet transform based feature extraction. J Med Syst 38(4):1–9
11. Tavakol EM, Sadri S, Ng EYK (2010) Application of K- and Fuzzy c-means for color segmentation of thermal infrared breast images. J Med Syst 34(1):35–42
12. PROENG dataset. http://visual.ic.uff.br/en/proeng/thiagoelias/

13. Mishra V, Rath SK (2021) Detection of breast cancer tumours based on feature reduction and classification of thermograms. Quant InfraRed Thermogr J 18(5):300–313
14. Sathish D, Kamath S, Prasad K, Kadavigere R (2019) Role of normalization of breast thermogram images and automatic classification of breast cancer. Vis Comput 35(1):57–70
15. Prakash O, Park CM, Khare A, Jeon M, Gwak J (2019) Multiscale fusion of multimodal medical images using lifting scheme based biorthogonal wavelet transform. Optik 995–1014
16. Aditya CSK, Hani'ah M, Bintana RR, Suciati N (2015) Batik classification using neural network with gray level co-occurence matrix and statistical color feature extraction. In: 2015 international conference on information & communication technology and systems (ICTS). IEEE, pp 163–168
17. Thibault G, Angulo J, Meyer F (2013) Advanced statistical matrices for texture characterization: application to cell classification. IEEE Trans Biomed Eng 61(3):630–637
18. Loh HH, Leu JG, Luo RC (1988) The analysis of natural textures using run length features. IEEE Trans Ind Electron 35(2):323–328
19. Amadasun M, King R (1989) Textural features corresponding to textural properties. IEEE Trans Syst Man Cybern 19(5):1264–1274
20. Ruxton GD (2006) The unequal variance t-test is an underused alternative to Student's t-test and the Mann–Whitney U test. Behav Ecol 17:688–690
21. Derrick B, Toher D, White P (2016) Why Welch's test is Type I error robust. Quant Methods Psychol 12(1):30–38
22. Ostertagova E, Ostertag O, Kováč J (2014) Methodology and application of the Kruskal-Wallis test. Appl Mech Mater 611:115–120
23. Lin CC, Mudholkar GS (1980) A simple test for normality against asymmetric alternatives. Biometrika 67(2):455–461
24. Gogoi UR, Bhowmik MK, Ghosh AK, Bhattacharjee D, Majumdar G (2017) Discriminative feature selection for breast abnormalitydetection and accurate classification of thermograms. In: 2017 international conference on innovations in electronics, signal processing and communication (IESC). IEEE, pp 39–44

Vehicle Re-identification Using Convolutional Neural Networks

Nirmal Kedkar, Kotla Karthik Reddy, Hritwik Arya, Chinnahalli K Sunil, and Nagamma Patil

Abstract Vehicle re-identification is the process of matching automobiles from one place on the road (one field of vision) to the next. Important traffic characteristics like the trip duration, travel time variability, section density, and partial dynamic origin/destination needs may be acquired by performing vehicle re-identification. However, doing so without using number plates has become challenging since cars experience substantial variations in attitude, angle of view, light, and other factors, all of which have a major influence on vehicle identification performance. To increase each model's representation ability as much as feasible, we apply a variety of strategies that will bring a major change like using filter grafting, semi-supervised learning, and multi-loss. The tests presented in this paper show that such strategies are successful in addressing challenges within this space.

Keywords Vehicle · Re-identification · Traffic · Smart city

1 Introduction

It is obvious that traffic is a major issue in India: the average vehicle speed in India is as low as 24 kmph [1, 2]. That is a no-no for both consumers and city planners. It's inconvenient, and it contributes to excessive air and noise pollution. As a result, most countries are heavily investing in Intelligent Transportation Systems (ITS). However, for such ITS systems, identifying the vehicles involved in these cities

N. Kedkar (✉) · K. Karthik Reddy · H. Arya · C. K. Sunil · N. Patil
Department of Information Technology, National Institute of Technology Karnataka, Surathkal, India
e-mail: nirmalhk7@gmail.com

K. Karthik Reddy
e-mail: karthikreddykotla.181it230@nitk.edu.in

N. Patil
e-mail: nagammapatil@nitk.edu.in

© The Author(s), under exclusive license to Springer Nature Singapore Pte Ltd. 2023

S. Chinara et al. (eds.), *Advances in Distributed Computing and Machine Learning*, Lecture Notes in Networks and Systems 660,
https://doi.org/10.1007/978-981-99-1203-2_35

becomes critical. Thus it is best to avoid using number plates because they can be tampered with and require cameras at a very specific angle to work properly. Even so, the reliance on license plates is far too great: it may sound like a great solution in theory, but we believe it makes little sense in practice.

In computer vision, vehicle recognition is a crucial problem in which we will need to identify a particular car using many photos or videos of the camera, especially when the license plate information is unavailable [3, 4]. When it comes to smart city design, such responsibilities take center stage: smart city concepts include ITS, which needs vehicle identification. The system, for example, may follow the target vehicle's progress and identify traffic abnormalities. Most recent vehicle Re-ID research has centered on deep learning approaches, which have done brilliantly on benchmarks like Veri-776 and VehicleID [5].

While there are sensor-based approaches for Vehicle Re-ID, such as magnetic sensors, inductive loops, infrared, ultrasonic, microwave, or piezoelectric sensors [6, 7], or a combination of them all, they are not widely used. These systems may protect the privacy of the traveling public (as it is very hard to trace vehicle signatures to individual vehicles), but such systems may require a capital expenditure: either in the form of sensors under the street or in the environment, or within the vehicle itself [8]. These systems are unable to monitor several lanes and are also speed-restricted, resulting in various signatures based on the speed at which the vehicle passes. Finally, sensor-based approaches are unable to offer information regarding vehicle characteristics such as color, length, and kind [9, 10].

As a result, we think vehicle-based Re-ID makes more sense: using many cameras, we will need to detect a specified car with a view that is not overlapping without any use of license plates. Because number-plate reading cameras are high-resolution specialized machines that are solely deployed to capture speeding, this is a more difficult assignment than number-plate recognition. These cameras are not like CCTV or regular security cameras in terms of attributes. It makes more sense to run an identifying system without scanning the number plates with smart traffic systems in mind. Large camera networks like this are installed in parks, universities, streets, and other public spaces, and it makes more sense to run an identifying system without scanning license plates.

Vision-based vehicle-Re-ID systems, either alone or in conjunction with a sensor-based system, can be used for a variety of applications, including: Automatic toll collection, Suspicious Vehicle Search, Vehicle tracking, and Parking lot system.

2 Literature Survey

2.1 Related Work

The majority of past object recognition research has focused on either a person or a human face. Both have long been hot issues in the computer vision community, and they may be stated as a unified problem: given a probe picture and a gallery of many

candidates, we must determine which one in the gallery is the same item as the probe image. Vehicle model classification and vehicle model verification are two of the most closely connected challenges that target vehicles. However, due to the nature of our work, all of the approaches presented in the research can only determine the vehicle model level rather than determining if the two cars are identical. As a result, person re-identification is our most closely connected issue.

Recent vision-based vehicle Re-ID approaches also use handcrafted features over a large-scale dataset to learn a discriminative representation. Liu et al. [11, 12] gives a multi-view dataset which of great quality. So this is the initial benchmark for the given problem of vehicle re-identification. Another efficient method for learning the important representation is metric learning; Bai et al. [13] introduce some triplet embedding which is sensitive to groups, so as to increase the model efficiency for the variance of the intra-class. And Kumar et al. [14, 15] give a method for the evaluation of the triplet loss, which is calculated on the re-id of the vehicles. We also find the work done by [16, 17], which handles orientation of vehicle images, [18] which does a great job of pose estimation, and [19–22] which combines pose estimation and makes it much more efficient.

2.2 Motivation

While there are sensor-based ways for Vehicle Re-ID, such as magnetic sensors, inductive loops, infrared, ultrasonic, microwave, or piezoelectric sensors, or a combination of these technologies, there are also sensor-based approaches for Vehicle Re-ID.

While these technologies may preserve the privacy of the traveling public, they are not without flaws (as it is very hard to trace vehicle signatures to individual vehicles). Such systems may need an investment in the form of sensors installed beneath the roadway, in the surroundings, or within the car itself. These technologies aren't capable of monitoring numerous lanes, either. These sensors are also speed-constrained: depending on how rapidly the vehicle passed by, they may produce various signals. Finally, sensor-based approaches are unable to offer information regarding vehicle characteristics such as color, length, and kind.

2.3 Problem Statement

Build a vision-based vehicle Re-ID system for smart city applications that can verify whether two vehicles are the same or not without looking at their number plates: i.e., with their color, model, shape, etc., by training multiple Convolutional Neural Networks.

Objectives There are several suggested vehicle databases, but re-identification of the vehicles got a lot of interest in the current days. The existing vehicle re-identification effort is mostly done on the level of characteristics and how much information can be gleaned from them. As CNN has progressed in previous years, the re-identification of vehicles has improved dramatically.

The broad objectives of this work are listed below.

- Re-id of the vehicle by training multiple CNNs.
- Training the CNN model using methods like multi-loss, filter grafting, etc.
- To give the test set images with some fake label.
- Improving the result by using some post-processing techniques.

3 Methodology

3.1 Datasets

One of the most extensive vehicle Re-ID databases is CityFlow [23, 24]. The dataset comprises 56,280 bounding boxes from 666 vehicle identities, with the training set consisting of 36935 bounding boxes from 333 object identities and the test set consisting of 18290 boxes that are bounding from the remaining identities, which are 333.

There are 1,052 and 18,290 photos in the normal probe and gallery sets, respectively. The synthetic dataset was created using VehicleX, a freely downloadable 3D engine. In all, detailed labels have been added to 192000 photos of 1360 automobiles. We chose 50 vehicle identification as our self-val set from a total of 3502 photos in CityFlow.

All of the photos on the left make up the training set. As a consequence, the self-training set for our vehicle Re-ID model has 225623 photos of 1645 vehicle ids. In addition, the pictures in the test set are given 330 ids after semi-supervised learning, resulting in a training set of 1975 ids.

3.2 Proposed Model

The basic architecture of our model is shown in Fig. 1. So basically, when the model is training, the images of the train set with the label ID are sent into the model, which acts as a backbone and this backbone generates the 2048d features along with the label ID, which is predicted. Here two networks are being trained with the same structure parallelly, which performs filter grafting. For the optimization of the model, we apply two different loss functions one is the hard triplet loss and the other one is cross-entropy loss. The first one, which is the hard triplet loss will be increasing

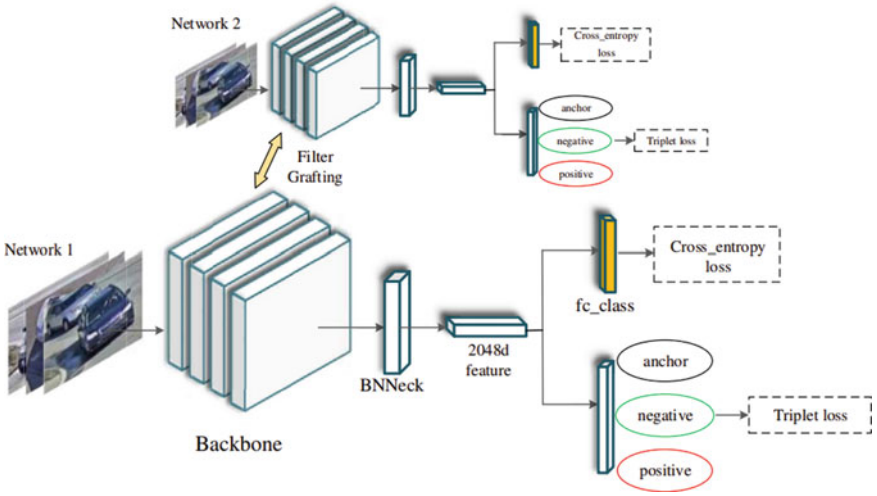


Fig. 1 Overview of the training framework

the distance of the features between the two label samples if they are different, and it also decreases the distance between the two label samples having the same label.

The cross-entropy loss is the second one. When given a set of training pictures, yi represents the ground label ID, pij represents the prediction ID logits of the ith image on the jth class, and N is the total number of images on which it is trained in a single batch.

And then, both the losses are combined and the final formulae to calculate are given as

$$\mathrm{L} = \mathrm{Lcross} + \alpha * \mathrm{Lhardtriplets}$$

After the backbone layer, we use the BNNeck, which is used for the addition of batch normalization.

So now, for the framework which we have now, we will be adding some other modules which are very useful for improving the performance of the single model [17, 25]. The first one which we will be adding is the MGN module. When given an input sample, it divides that sample horizontally into many other parts. It then calculates loss for the each part separately. The second one we will be adding is the self-attention constraint where which makes the network give higher importance to some of the subtleties. The other one which is added is the squeeze and excitation block, which will help in enhancing the representation power of the features.

While the testing is happening, the network which is trained will be extracting the generated 2048d features from each of the input images. The features which are collected from the different networks will be merged at one place for the re-identification of the vehicles.

3.3 Impact of Filter Grafting

Filter grafting shown in Fig. 2 is a learning paradigm that aims to increase the representation capability of deep neural networks. It is also used to improve the representation power by reactivating invalid filters during the training phase. So if the filter weights are valid for the other networks, those filter weights will be grafted to the non-valid filters in its own network. So like, this exchange of the filter weights will be happening between the networks. Many of the networks are brought together and grafted, increasing the self-network's progress. The framework behind the filter grafting is shown in Fig. 2. The given network in Fig. 2 where multiple processes will be started parallelly, which are denoted as M1 and M2. So when the model is trained in each epoch, we will be grafting the best filter weight of the process M1 to the invalid filter of the process M2. This grafting does not occur in the filter level, it actually occurs in the convolutional level, so what we do is we graft every filter weight of a certain layer of process M1 to process M2 in the same layer. So this happens the same from process M2 to process M1, grafting every filter weight of a certain layer of process M2 to process M1 in the same layer.

On the SE-ResNet152, we conducted some tests to validate the efficiency of filter grafting. We explore two grafting models, each of which improves mAP by roughly 1% over the baseline. Because we have two networks trained independently during training, we will only use one for grafting during the testing phase.

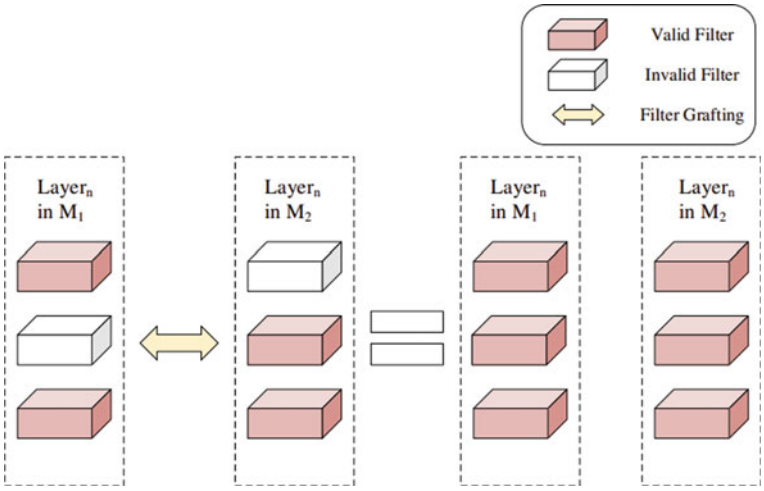


Fig. 2 Filter grafting

3.4 Semi-supervised Learning

For the images of the test set, the labels are not given, so we will be building a model which is semi-supervised. This method helps in annotating the images of the test set with the fake label. So now, this test set can be used as an add-on to the images of the training set.

The steps for annotating the test set images are

- First, we will be using the images of the training to train the vehicle re-id model.
- Next, for the test set images, we will be extracting the features using the already trained vehicle re-identification model.
- Following that, we'll cluster the test set's photos based on the characteristics of the testing images and then give a bogus label as the ID to each cluster.
- The set of tests with bogus labels will now be combined with the photos from the training set that we currently have to create a new training set of images. We will retrain the vehicle re-id models using this new set of training when the new training set has been produced.

Here the test set contains 333 Id. So, the k-means clustering algorithm is used for clustering the test set images, and the clustering center is given as 333.

3.5 Post-processing

So for the results which we got from the training of many single models, we will be using multiple techniques for the post-processing as shown in Fig. 3. Some of them are listed below.

Center Crop: There will generally be some redundancy for the query and gallery of the test set. We will be using the crop on the center for extracting the features of the test set; this crop is to change the image so that it will be focused more on the vehicle only, and this will also decrease the impact of the background, which is surrounding the vehicle. **Image Flipping:** We will be giving the image of the vehicle and the flip of the image of the vehicle in the horizontal direction when

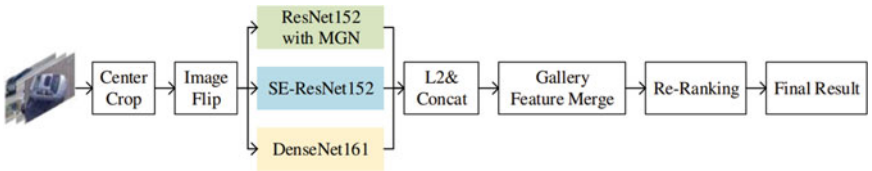


Fig. 3 Overview of the testing framework

the model is in the forward pass of the training, and we will be averaging both the values of the features as a single one. **Model Ensemble:** This Model Ensemble is the approach in machine learning which combines multiple models in the process of the prediction. It is very effective in the process of detection and classification and also in the Re-identification tasks. Many of the problems will have their own ensemble methods. So for this model, we have tried many of the ensemble models and finally found out that expanding the dimensions of the features by the feature concatenating is a lot better than the method to calculate the features average of many models. After training the above models, we will extract the feature vectors for any image, and we will concatenate the feature vectors as the representation of a final feature for that particular image. So now the features of the vehicles which are extracted by each of the models will be concatenated. It is also seen that the performance of the models increased by so much when compared to the single model in the next experiments. **Query Expansion:** This expansion of the query is also one of the re-ranking approaches. The recall rate of the retrieval will be increased with the help of this expansion of the query. More improvement of this method can be observed when the accuracy of the k-top is large. So once the retrieval for the first time is done, the query and the top k features of the images will then be taken in the set of galleries, and they will be averaged as the query features; after that, we will be doing the retrieval will be done for the second time, and this same process will be repeated for n times which is a changeable parameter. **Gallery Feature Merge:** The test set that we got will also have the id information of the track. This contains 800 tracks of the vehicles gallery, and every track will contain many images of the same vehicle. This can be used as information which is given before only. For this information which is given, the post-processing is done by taking the average of the K features of the picture for each track as the features of those pictures of that particular track. The constraint here is the K which can be adjusted where this T can be taken as the pictures that are of this track. This method can be used for many re-identification tasks, which will provide the information on the track of the video. **Re-ranking:** For this Re-ranking, we will be using the K-reciprocal encoding approach [26]. This is one of the very effective re-ranking approaches. One of the assumptions which are taken in this is that if the image which is returned is ordered in the k nearest neighborhood of the given probe then we can consider it as a match which is true (Figs. 4 and 5).

4 Results and Analysis

4.1 Results

Also note that the numbers above each image are references to their filenames within the dataset: the same numbers can also be found within the output file, a screenshot of which is attached below.



Fig. 4 Sample output 1



Fig. 5 Sample output 2

4.2 Model Performance

We use the output file generated on testing (as seen in Fig. 6) to calculate the performance of our model.

Table 1 shows the performance of our model. The model achieved a mean average precision score of around 75.6, which is better than our base paper. The average

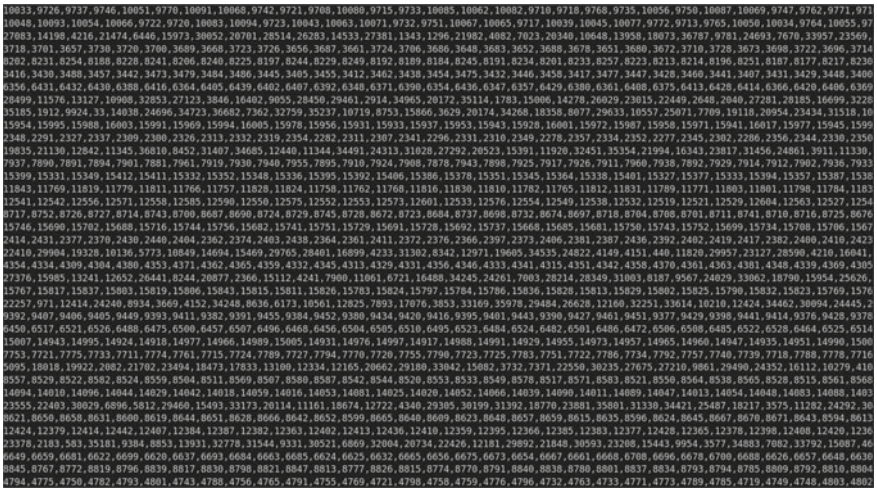


Fig. 6 Sample output file

Table 1 Results for vehicle re-identification

Measure	Scores
Mean average precision	75.6
Average accuracy	80.952
Average F-measure	0.6698

Table 2 Comparison of the results

Model	mAP scores
Base paper	73.22
Our model (with grafting)	75.6

accuracy of the model is 80.95, and the average F-measure of the model is 0.6698. As this is the object identification problem, we will be using the mAP as the comparison parameter. The mean of the average precision will be calculated using the recall values. This will be calculated by calculating the mean AP of all the classes. This mean average precision will also be used in evaluating the object detection models like the fast R-CNN, and many others.

Table 2 shows how our model performed when compared with the base model. The model achieved a mean average precision score of around 75.6, which is better than our base paper. Our base paper had a mean average precision of 73.22; we used filter grafting for the model proposed in the base paper and improved the performance of the model. Using filter grafting shares the best model weights between the models, which will help in increasing the performance of the model.

5 Conclusion and Future Work

We developed a vision-based car re-identification method. Our car re-identification approach is built on using filter grafting, multi-loss, and other methods to train multiple CNNs. All of these strategies will significantly improve each network's ability to be represented.

We also used filter grafting, which shares the model weights between the different models, which will help in improving the efficiency of the model [27]. This filter grafting has a major part in how the model performs. The semi-supervised method is used for annotating the test set images with fictitious labels, and these labeled images will be merged with the original images so as to improve the performance of the model. There are many other post-processing methods added to the model to improve the result; some of them are the cropping of the center, flipping of the image, and an ensemble of the model [28]. Even the re-ranking is also applied to the model, which is one of the key steps for improvement in the performance. In the future, we can also work to extend the concept to bikes: a concept like a vehicle Re-ID hasn't been tried on motorcycles yet.

References

1. How fast does traffic move in your city? by the Hindu https://www.thehindu.com/opinion/op-ed/how-fast-does-traffic-move-in-your-city/article25570594.ece
2. Luo P, Loy CC, Tang X, Yang L, Luo P, Loy CC, Tang X (2015) A large-scale car dataset for fine-grained categorization and verification. In: CVPR
3. Ma BP, Su Y, Jurie F (2012) BiCov: a novel image representation for person re-identification and face verification. In: BMVC
4. Manmatha R, Wu CY, Smola AJ, Krahenbuhl P (2017) Sampling matters in deep embedding learning. In: CVPR
5. Ma B, Su Y, Jurie F, Ma B, Su Y, Jurie F (2012) Local descriptors encoded by fisher vectors for person re-identification. In: ECCV workshops
6. Klein LA, Kelley MR, Mills MK (1997) Evaluation of overhead and in-ground vehicle detector technologies for traffic flow measurement. J Test Eval 25:205–214
7. Davies P (1986) Vehicle detection and classification. Information Technology Applications in Transport, 11–40
8. A survey of advances in vision-based vehicle re-identification by Sultan Daud Khana and Habib Ullah
9. Mishchuk A, Mishkin D, Radenovic F, Matas J (2017) Working hard to know your neighbor's margins: local descriptor learning loss. In: NIPS
10. Mnih V, Heess N, Graves A, Kavukcuoglu K (2014) Recurrent models of visual attention. In: NIPS
11. Liu X, Liu W, Ma H, Fu H (2016) Large-scale vehicle reidentification in urban surveillance videos. In: 2016 IEEE international conference on multimedia and expo (ICME), pp 1–6
12. Liu X, Liu W, Mei T, Ma H (2017) PROVID: progressive and multimodal vehicle reidentification for large-scale urban surveillance. TMM 20(3):645–658
13. Bai Y, Lou Y, Gao F, Wang S, Wu Y, Duan LY (2018) Group sensitive triplet embedding for vehicle reidentification. TMM 20(9):2385–2399
14. Kumar R, Weill E, Aghdasi F, Sriram P (2020) A Strong and efficient baseline for vehicle re-identification using deep triplet embedding. J Artif Intell Soft Comput Res 27–45

15. Schroff F, Kalenichenko D, Philbin J (2015) FaceNet: a unified embedding for face recognition and clustering. In: CVPR
16. Wang Z, Tang L, Liu X, Yao Z, Yi S, Shao J, Yan J, Wang S, Li H, Wang X (2023) Orientation invariant feature embedding and spatial temporal regularization for vehicle re-identification
17. Shen L, Lin Z, Huang Q (2016) Relay backpropagation for effective learning of deep convolutional neural networks. In: ECCV
18. Prokaj J, Medioni G (2023) 3-D model based vehicle recognition
19. Zheng L, Huang Y, Lu H, Yang Y (2017) Pose-invariant embedding for deep person re-identification
20. Jose C, Fleuret F (2016) Scalable metric learning via weighted approximate rank component analysis. In: European conference on computer vision
21. Zheng T, Naphade M, Birchfifield S, Tremblay J, Hodge W, Kumar R, Wang S, Yang X (2019) Pamtri: pose-aware multi-task learning for vehicle re-identification using highly randomized synthetic data. In: Proceedings of the IEEE international conference on computer vision, pp 211–220
22. Zhou Y, Shao L (2018) Aware attentive multi-view inference for vehicle re-identification. In: Proceedings of the IEEE conference on computer vision and pattern recognition, pp 6489–6498
23. Tang Z, Naphade M, Liu M-Y, Yang X, Birchfield S, Wang S, Kumar R, Anastasiu D, Hwang J-N (2019) Cityflow: A city-scale benchmark for multi-target multi-camera vehicle tracking and re-identification. In: Proceedings of the IEEE conference on computer vision and pattern recognition, pp 8797–8806
24. Kostinger M, Hirzer M, Wohlhart P, Roth PM, Bischof H (2012) Large scale metric learning from equivalence constraints. In: IEEE conference on computer vision and pattern recognition, pp 2288–2295
25. Shen Y, Xiao T, Li H, Yi S, Wang X (2017) Learning deep neural networks for vehicle re-ID with visual-spatiotemporal path proposals. In: ICCV
26. Chen P, Bai X, Liu W (2014) Vehicle color recognition on urban road by feature context. IEEE Trans Intell Transp Syst 15:2340–2346
27. Li W, Zhao R, Xiao T, Wang X (2014) Deepreid: deep filter pairing neural network for person re-identification. In: Proceedings of the IEEE conference on computer vision and pattern recognition, pp 152–159
28. Krizhevsky A, Sutskever I, Hinton GE (2012) Imagenet classification with deep convolutional neural networks. In: Advances in neural information processing systems, pp 1097–1105

Evaluation of Federated Learning Strategies on Industrial Time Series Fault Classification



Baratam Prathap Kumar, Sameer Chouksey, Madapu Amarlingam, and S. Ashok

Abstract Federated Learning (FL) can enable industrial devices to collaboratively learn a shared Machine Learning (ML) model while keeping all the training data on the device itself. The FL inherently solves the data privacy issues where as it also reduces the cloud data hosting cost and communication overhead. However, existing FL algorithms and frameworks are generically designed and not evaluated against the industrial applications. In case of industrial applications, the data generated from assets and machines are mostly non-independent and Identically Distributed (non-IID) which demands for an investigation and evaluation on the existing FL algorithms against such non-IID industrial data. This paper presents a survey of FL frameworks, algorithms, and optimizers, and evaluates their performance against the industrial data. The paper shares the results of extensive numerical experiments conducted with the edge device, Raspberry pi. The data set used in this study is benchmarking fault classification industrial data, called PRONTO data set, for both IID and non-IID cases. With the experimental results, the paper demonstrates that the adaptive optimization-based FL algorithm, `FedAdam` outperforms other algorithms on an industrial time series fault classification application.

Keywords Federated learning · Federated optimization · Non-IID data distribution

S. Chouksey (✉) · M. Amarlingam
ABB Global Industries and Services Private Limited, Bengaluru, India
e-mail: sameer.chouksey@in.abb.com

M. Amarlingam
e-mail: madapu.amarlingam@in.abb.com

B. Prathap Kumar · S. Ashok
Depatment of Electrical Engineering, National Institute of Technology Calicut, Kerala, India
e-mail: m200192ee@nitc.ac.in

S. Ashok
e-mail: ashoks@nitc.ac.in

© The Author(s), under exclusive license to Springer Nature Singapore Pte Ltd. 2023

S. Chinara et al. (eds.), *Advances in Distributed Computing and Machine Learning*,
Lecture Notes in Networks and Systems 660,
https://doi.org/10.1007/978-981-99-1203-2_36

1 Introduction

Various operating machinery and automation systems are common in industrial production systems and process sectors. The key to frictionless manufacturing and competitive product cost is high availability and quick reconfiguration of each running equipment [1]. Condition monitoring, anomaly detection, fault prediction, and fault categorization are frequently achieved based on machine learning (ML) models deployed to edge devices to assure high availability of each machine, e.g., reporting abnormalities in production [2, 3]. The performance of the ML models depends on the size and quality of the data, i.e., the data should contain all the scenarios considered to train a model and sufficient data for each scenario [4]. Often in industrial applications, the data availability is limited for individual assets or machines [5]. Increasing training data might be realized by sharing data with an external industry partner. However, this approach is often critical as confidential industrial information may get compromised.

A collaborative learning mechanism that allows users to collectively benefit by sharing trained ML models from siloed data, without collecting the data centrally, was introduced by McMahan et al., in [6]. This approach is called Federated learning(FL), which is a privacy-preserving distributed learning technique. In this technique, a federation of devices (FL clients) solves an ML problem by sharing the locally trained ML model weights to a central server, where the server aggregates all the model weights from clients and sends them back, the same is visualized in Fig. 1. As data are not leaving the client, with only model updates being communicated to the server, it preserves the data privacy and eliminates the necessity for the server to access the raw data for training the model. Based on the type of clients participating, the existing FL algorithms can be classified into two classes: cross-device and cross-silo [7]. If the clients are different types of devices such as edge devices [8] and mobile devices [6], it is called as cross-device FL. On the other side when the clients are the same type of assets or edge devices in process plants with similar data sets contributing to creating a collaborative global model, it is known as cross-silo FL. In this paper, we consider cross-silo scenario.

Often industrial applications data are non-IID [9]. Vanilla FL algorithm `FedAvg` [6] does not address the non-IID data scenario. Studies like `FedAvgM` [10] and `QFedAvg` [12] have proposed FL algorithms for non-IID data scenario. In[11], Sashank et al. proposed adaptive federated optimizers such as `FedAdam`, `FedYogi` and `FedAdagrad` to address the challenges with non-IID data. Paper [13] presented a study with different FL algorithms that works in non-IID data scenario. However, all these approaches lack on experimental studies with realistic industrial application data set and scenarios of non-IID data.

The article [14] provided an overview of different FL frameworks, and FL protocols. However, they did not evaluate these algorithms with industrial application data set. In [15], the authors evaluated existing FL frameworks. However, they did not consider different FL algorithms for evaluation, which is more important for industrial applications as data from industries often follows non-IID distribution.

This paper presents a comparison of performance of different FL aggregation algorithms and recommends the best-suited algorithms for industrial applications such as time series data classification. We break the barrier of experiments on non-IID data scenarios by using realistic industrial application data with useful non-IID scenarios. These experiments are conducted by implementing FL algorithms on edge devices. This paper finds out the optimal values of Hyperparamters in FedOpt (`FedAdagrad`, `FedYogi`, `FedAdam`), `FedAvgM`, `QFedAvg`. And also presents a review of the state-of-the-art FL frameworks that can be used for the implementation of FL algorithms.

This paper is organized as follows: Sect. 2, discusses the overview of different FL frameworks and their comparison and overview of FL strategies. Section 3, talks about the non-IID cases, data set details, experiments, and evaluation of FedOpt, QFedAvg, and FedAvgM. Section 4 concludes the paper.

2 Overview of Existing FL Frameworks and Aggregation Strategies

Consider a setup with K industrial plants (clients) with a centralized server or cloud, as shown in Fig. 1. Suppose the training data available at the kth client is $\{(\mathbf{x}_s, \mathbf{y}_s), \forall s \in \mathcal{S}_k\}$. Generally, data at different plants are mutually disjoint and non-empty sets $\mathcal{S}_k$ such that the overall data from all the plants can be written as $\mathcal{S} = \bigcup_{k=1}^{K} \mathcal{S}_k$ and $\mathcal{S}_k \cap \mathcal{S}_l = \emptyset$ for $k \neq l$. Each plant trains a local model parameters at the kth client by minimizing the loss function $F_k(\boldsymbol{\Theta}) = \frac{1}{|\mathcal{S}_k|} \sum_{s \in \mathcal{S}_k} f_s(\boldsymbol{\Theta})$, where

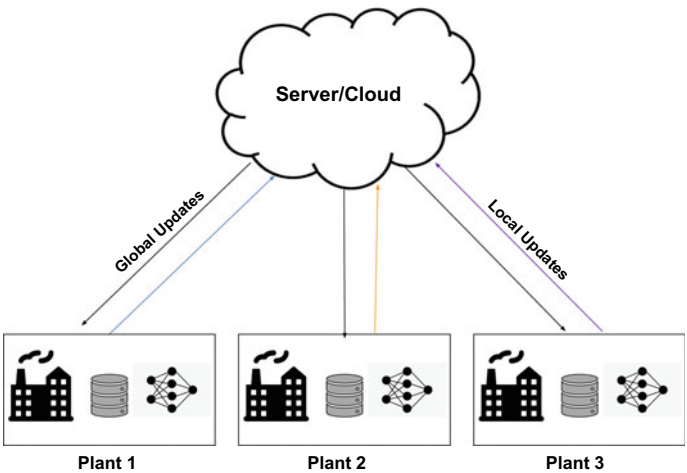


Fig. 1 Federated learning for industrial application

$\boldsymbol{\Theta}$ represents model parameters. The gradients computed at the clients based on their local data can be written as $\nabla F_k(\boldsymbol{\Theta}) = \frac{1}{|\mathcal{S}_k|} \sum_{s \in \mathcal{S}_k} \nabla f_s(\boldsymbol{\Theta})$.

As the data sets are disjoint, the learning at each plant is limited to corresponding local data. To train a collaborated robust model from disjoint data sets of different plants, FL computes the gradient of $f(\boldsymbol{\Theta})$ with respect to $\boldsymbol{\Theta}$ as

$$\nabla f(\boldsymbol{\Theta}) = \frac{1}{|\mathcal{S}|} \sum_{s \in \mathcal{S}} \nabla f_s(\boldsymbol{\Theta}) = \sum_{k=1}^{K} \frac{|\mathcal{S}_k|}{|\mathcal{S}|} \nabla F_k(\boldsymbol{\Theta}),$$

where $\nabla F_k(\boldsymbol{\Theta})$ denotes the gradients computed at the client's based on their local data. In reality the gradients from the clients can be aggregated at the server to compute the global consensus gradient $\nabla f(\boldsymbol{\Theta})$. Further, the global update of $\boldsymbol{\Theta}$ can be computed by aggregating the local updates from the clients as

$$\text{Local training:} \quad \boldsymbol{\Theta}_k \leftarrow \boldsymbol{\Theta}_k - \eta_l \nabla F_k \tag{1}$$

$$\text{Aggregation:} \quad \boldsymbol{\Theta} \leftarrow \sum_{k=1}^{K} \frac{|\mathcal{S}_k|}{|\mathcal{S}|} \boldsymbol{\Theta}_k. \tag{2}$$

In other words, each client computes the current model parameters with its local data, and the server then takes a weighted average of all the client's model parameters and sends it back to the clients.

In the following section, we describe the existing frameworks to realize different FL algorithms.

2.1 Federated Learning Frameworks

Tensor Flow Federated (TFF): TFF is an open-source platform [16] for decentralized deep learning. It uses gRPC (Google Remote Procedure Call) protocol for communication. Currently, TFF supports only simulation mode. Note that simulation mode means the framework supports the FL implementation on local PC whereas federated mode means the FL framework supports implementation in client-server architecture, where devices can act as clients and server.

Flower: Flower [17] provides higher level abstractions and a stable, language and ML framework-agnostic implementation of the key components of an FL system, allowing researchers to rapidly explore and implement new ideas on top of a dependable stack. It uses RPC Protocol for communicating between server and clients. It supports simulation mode as well as federated mode [18, 19].

FATE: Webank's AI section launched FATE [20], an open-source platform to provide a safe computational foundation for a federated AI ecosystem that supports model training from horizontal (homogeneous) and vertical (heterogeneous) data

Table 1 Overview of existing Federated learning frameworks and their features

Features	TFF	Flower	FATE	Pysyft	Xaynet
OS	Mac,Linux	Mac,Linux, windows,Android	Mac,Linux	Mac,Linux, Windows, Android	Mac, Windows, Linux
Settings	Cross-silo	Cross-silo and Cross-device	Cross-silo	Cross-silo and Cross-device	Cross-device
Scalability	No	Tested with 15M clients	Tested with few clients	Tested with few clients	Tested with few clients
ML Framework Agnostic	No	Yes	Yes	Yes	Yes
Platform independent	No	Yes	No	No	Yes
Mode	Simulation	Simulation and federated	Simulation and federated	Simulation and federated(Under development)	Simulation and federated
Protocol	gRPC	gRPC/RPC	gRPC	SPDZ	PET

partitions. For interaction, FATE uses gRPC technology with a data format in the form of proto-objects. FATE can be implemented in both simulation mode (standalone deployment) and federated mode (cluster deployment).

Pysyft: PySyft [21], is an MIT-licensed open-source Python platform for safe and private deep learning. PySyft is a privacy-focused service that is part of the OpenMined ecosystem (a private corporation). Pysyft uses SPDZ (pronounced as Speedz) protocol which uses encrypted computation. Currently, it supports simulation mode and doesn't support federated mode.

Xaynet: Xaynet [22] is another open-source FL framework, which is specifically designed for privacy and protection and scripted in Rust programming language. In this framework, they have created some Python bindings (Python SDK) in order to use Xaynet in Python. In this framework, PET (Privacy Enhancement Technology) is used as a communication protocol to ensure privacy between parties. In PET protocol, model updates are masked so that no sensitive information is shared between the coordinator (server) and participants (clients). Table 1 summarizes all the frameworks and their features.

Based on a summary of different frameworks from Table 1, we decided to use Flower FL framework for our experiments, which is ML framework agnostic, platform-independent, and scalable.

Algorithm 1: Federated Learning: different algorithms

Input: K: Number of clients, C: Number of communication rounds, E: Number of local epochs, η, η_l: Learning rates and β: momentum parameter.

Output: $\boldsymbol{\Theta}$
Server executes;
$\boldsymbol{\Theta}_0$: Initial model parameters;
for *each round* $c = 1, 2, \cdots C$ **do**
  for *each client* $k = 1, 2, \cdots K$ **do**
    $\boldsymbol{\Theta}_c^k \leftarrow$ **ClientModel**(k,$\boldsymbol{\Theta}_0$);
  end
  $\boldsymbol{\Theta}_c \leftarrow \sum_{k=1}^{K} \frac{|\mathcal{S}_k|}{|\mathcal{S}|} \boldsymbol{\Theta}_c^k$;
  if ***FedAvg*** **then**
    Return $\boldsymbol{\Theta}_c$
  end
  if ***FedOpt*** **then**
    Initialize $\beta_1, \beta_2 \in (0, 1)$ and τ, m_0, v_0;
    $m_t = \beta_1 m_{t-1} + (1 - \beta_1)\boldsymbol{\Theta}_c$;
    $v_t = v_{t-1} + \boldsymbol{\Theta}_c^2$ (**FedAdagrad**);
    $v_t = v_{t-1} - (1 - \beta_2)\boldsymbol{\Theta}_c^2 \text{sign}(v_{t-1} - \boldsymbol{\Theta}_c^2)$ (**FedYogi**);
    $v_t = \beta_2 v_{t-1} + (1 - \beta_2)\boldsymbol{\Theta}_c^2$ (**FedAdam**);
    Return $\boldsymbol{\Theta}_c = \boldsymbol{\Theta}_{c-1} + \eta \frac{m_t}{\sqrt{v_t} + \tau}$
  end
  if ***FedAvgM*** **then**
    $\mathbf{v}_t \leftarrow \beta \mathbf{v}_{t-1} + \boldsymbol{\Theta}_{c-1}$;
    $\boldsymbol{\Theta}_c \leftarrow \beta \boldsymbol{\Theta}_{c-1} - \mathbf{v}_t$
  end
  if ***QFedAvg*** **then**
    Return $\boldsymbol{\Theta}_c = \boldsymbol{\Theta}_{c-1} - \frac{\sum_{k=1}^{K} \boldsymbol{\Delta}_k^c}{\sum_{k=1}^{K} \boldsymbol{H}_k^c}$
  end
end
ClientModel(k,$\boldsymbol{\Theta}_c$);
for $e = 1, 2, \cdots E$ **do**
  $B \leftarrow$ (split $\mathcal{S}_k$ into batches of size B);
  for $b \in B$ **do**
    $\boldsymbol{\Theta}_c^k \leftarrow \boldsymbol{\Theta}_c^k - \eta_l . \nabla F_k(\mathbf{y}, \hat{\mathbf{y}}; \boldsymbol{\Theta}_c, \mathbf{b})$
  end
end
if ***FedAvg*** **then**
  Return to Server $\boldsymbol{\Theta}_c^k$;
end
if ***FedOpt*** **then**
  Return to Server $(\boldsymbol{\Theta}_c^k - \boldsymbol{\Theta}_{c-1})$ and s_k;
end
if ***QFedAvg*** **then**
  Initialize $q = 0.001$;
  for $e = 1, 2, \cdots E$ **do**
    $B \leftarrow$ (split $\mathcal{S}_k$ into batches of size B);
    for $b \in B$ **do**
      $\boldsymbol{\Theta}_k^c \leftarrow L(\boldsymbol{\Theta}_{c-1} - \boldsymbol{\Theta}_c^k)$;
      $\boldsymbol{\Delta}_k^c \leftarrow F_k{}^q(\mathbf{y}, \hat{\mathbf{y}}; \boldsymbol{\Theta}_c, \mathbf{b})\boldsymbol{\Theta}_k^c$;
      $\boldsymbol{H}_k^c \leftarrow q F_k{}^{q-1}(\mathbf{y}, \hat{\mathbf{y}}; \boldsymbol{\Theta}_c, \mathbf{b})\|\boldsymbol{\Theta}_k^c\|^2 + L F_k{}^q(\mathbf{y}, \hat{\mathbf{y}}; \boldsymbol{\Theta}_c, \mathbf{b})$
    end
  end
  Return to Server $\boldsymbol{\Delta}_c^k, \boldsymbol{H}_k^c$;
end

2.2 FL Algorithms

We considered popular FL algorithms, `FedAdaGrad`, `FedYogi`, `FedAdam`, `QFedAvg`, and `FedAvgM`, which works with non-IID data along with vanilla `FedAvg` for evaluation. Pseudo-code of all considered FL algorithms is shown in Algorithm 1. Algorithm 1 has two parts, one is server execution and another one is client model. We have written pseudo-code as blocks and mentioned the related part for different algorithms. Please note that in `QFedAvg`, the cost function F^q is different from others, we suggest the reader to go through the paper [12] for more information.

Optimization Techniques(FedOpt) in FL Standard optimization methods, such as distributed SGD, are not suitable in FL when it deals with non-IID data. To overcome, [11] proposed to use adaptive optimizers (e.g., Adam, Yogi) with `FedAvg` called as `FedAdaGrad`, `FedYogi`, and `FedAdam`. These algorithms make learning rates adaptive, this makes the FL converge faster in a lower number of communication rounds. The degree of adaptivity, or how well the algorithms can adapt, determines `FedYogi` and `FedAdam`'s performance. The parameter τ determines the degree of adaptivity of the algorithms, with smaller τ values indicating greater adaptivity. Pseudo-code for these algorithms is presented in Algorithm 1. If we choose FedOpt, server executes lines from 3 to 8 and 12 to 18 from Algorithm 1. Client executes lines 29 to 34 and 38 to 40.

`QFedAvg` Although the accuracy may be high on an average, there is no guaranteed good accuracy for individual devices in the network. This is due to heterogeneity in federated networks as well as both in terms of size and distribution, model performance can vary drastically. Due to variation across data in different clients, the model accuracy is quite poor for some clients. `QFedAvg` [12] encourages fairness in FL, i.e., enforces the accuracy parity between clients and minimizes an aggregate re-weighted loss parameterized by q such that the devices with higher loss are given higher relative weight. Pseudo-code for `QFedAvg` is described in Algorithm 1. If we choose `QFedAvg` from Algorithm 1, then server executes lines from 3 to 8 and 24 to 26. Client executes lines from lines 41 to 51.

`FedAvgM` Using the idea of momentum in conjunction with SGD has shown to be quite effective in terms of speeding up model training by accumulating the gradient history over time to attenuate oscillations. This appears to be especially important in FL, where participants may have a sparse distribution of data and just a small number of labels. `FedAvgM` [10] adds the momentum at the server for the vanilla `FedAvg`. Vanilla `FedAvg` updates the weights via $\boldsymbol{\Theta}_{c+1} \leftarrow \boldsymbol{\Theta}_c - \nabla\boldsymbol{\Theta}_c$, where $\nabla\boldsymbol{\Theta}_c = \sum_{k=1}^{K} \frac{|\mathcal{S}_k|}{|\mathcal{S}|} \nabla\boldsymbol{\Theta}_c^k$ (s_k is the number of examples, $\nabla\boldsymbol{\Theta}_c^k$ is the weight update from kth client, and $\mathcal{S} = \sum_{k=1}^{K} n_k$. To add momentum at the server, `FedAvgM` instead computes $\mathbf{v}_t \leftarrow \beta\mathbf{v}_{t-1} + \boldsymbol{\Theta}_{c-1}$, and update the model with $\boldsymbol{\Theta}_c \leftarrow \beta\boldsymbol{\Theta}_{c-1} - \mathbf{v}_t$. Pseudo-code for `FedAvgM` is described in Algorithm 1. If we choose `FedAvgM` from Algorithm 1, then server executes lines from 3 to 8 and 20 to 23. Client executes lines from lines 28 to 34.

3 Experiments and Evaluation with Industrial Data Set

In this section, we describe the considered data set and elaborate on the FL algorithms used with the data set. We tuned related hyperparameters in all FL algorithms to achieve the best performance.

3.1 Data Preparation

We used PRONTO [23] data set for experimentation. The data sets are suitable for fault detection, fault identification, fault classification, fault severity evaluation, and monitoring of fault evolution. In our experimentation, we use the data for fault classification. The total data $\{\mathbf{x}, \mathbf{y}\}$ samples are of a size 45420. There are total 5 classes in the data set which are normal (11899 samples), slugging(4843 samples), air blockage (8880 samples), air leakage (8279 samples), and diverted flow(11519 samples). There are total 29 features (including alarm data). We considered a window size of 10 with a stride of 1 throughout our experiments.

In practice, getting publicly available real-time non-IID data for FL experimentation is difficult due to data privacy concerns. In [24], the authors suggested a way to convert IID data to non-IID data distribution of 5 types: (1) label distribution skew (i.e., $P(\mathbf{y}_i)$ is different among clients) means all clients will have the same labels but different distribution of labels; (2) feature distribution skew (i.e., $P(\mathbf{x}_i)$ is different among clients) means features distribution is different; (3) same label but different features (i.e., $P(\mathbf{x}_i|\mathbf{y}_i)$ is different among clients); (4) same features but different labels (i.e., $P(\mathbf{y}_i|\mathbf{x}_i)$ is different among client); (5) quantity skew (i.e., $P(\mathbf{x}_i, \mathbf{y}_i)$ is same but the amount of data is different among clients). Case 3 is related to vertical FL, as our focus is on horizontal FL, we do not consider it for our experimentation. We also do not consider case 2 and 4 for experimentation, as they are not applicable for most of the FL applications [24]. In our experimentation we considered cases (1) and (5) combined and homogeneous partitions (IID scenario) also and shown in Table 2. The notation in the table: Scenario (Sc), Client-1 (Cl-1), Normal (Nor.), Slugging (Slug.), Blockage (Block.), Leakage (leak.), Diverted flow (Div. Fl,), Numbera of samples (No. Samp.), Categories (Cat.), Homogenous partitions (Homogen. part.) and Label distribution skew & Quantity skew (LD & QS).

3.2 Lab Setup

To investigate the effectiveness of existing FL algorithms on IID and non-IID data settings we conducted experiments on realistic time series fault classification data set PRONTO [23]. We have implemented the Algorithm 1 using Flower framework. We setup the experiments using edge devices Raspberry pi4 model B [25] (quad core 64-

Table 2 Different data scenarios

Sc	Clients	Nor.	Slug.	Air Block.	Air leak.	Div. Fl.	No. Samp.	Cat.
Sc1	Cl-1	4724	1925	3548	3299	4599	18095	*Homogen. part.
	Cl-2	4723	1926	3548	3300	4600	18097	
Sc2	Cl-1	9000	3000	96	599	199	12894	*LD & QS
	Cl-2	447	851	7000	6000	9000	23298	



Fig. 2 Lab setup for federated learning algorithms evaluation

bit processor with 8GB RAM), where we used two edges as clients and one used as a server. The scripting of the algorithm is done using Jupyter notebook. The Lab setup is shown in Fig. 2. F_1 score and accuracy are considered as performance evaluation metrics. For experimentation, we considered a batch size of 32, and the number of local epochs is equal to 10. We executed our experiments for 100 communication rounds for each FL algorithm in both IID and non-IID data scenarios.

Neural Network Model We use two convolutional neural network (CNN) layers with two pooling layers. Each convolutional layer has 6 filters of size 7 and an average pooling size of 3. For the second convolutional layer, we have used 12 filters of size 7. This selection of filters is found by trial and experiment and based on the best training results. We have considered a categorical cross-entropy loss for training the network. We considered the data input shape as 10x17, where only 17 features were considered by excluding alarm data.

3.3 Experiments and Observations

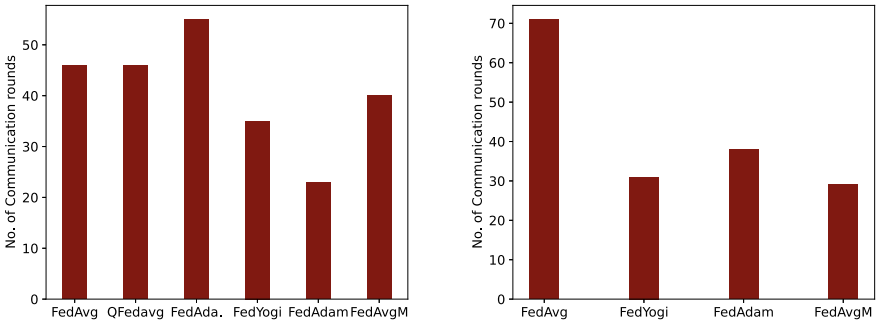
Hyperparameter Tuning For finding the best hyperparameters we use grid search. For `FedAdam`, `FedAdagrad`, `FedYogi` server learning rate η parameter takes the best values from set $\{10^{-1}, 10^{-0.5}, 10^{0}, 10^{0.5}\}$, $\{10^{-2}, 10^{-1.5}, 10^{-1}, 10^{-0.5}\}$ and $\{10^{-2.5}, 10^{-2}, 10^{-1.5}, 10^{-1}\}$, respectively.Client learning rate η_l is fixed as 0.01 for all FedOpt algorithms. For `FedAvgM`, the best momentum parameter β is selected from {0, 0.9, 0.97, 0.99} based on performance and server learning rate is set to 1.For `QFedAvg`, we selected best values for q and q-learning rate from {0.0001, 0.001, 0.2, 5, 10} and $\{10^{-1}, 10^{-2}, 0.2, 0.3\}$, respectively. In FedOpt algorithms, best adaptivity parameter τ is selected from $\{1e^{-4}, 1e^{-9}\}$. For `FedAdam` and `FedYogi` we set momentum parameters β_1, β_2 as 0.9, 0.99 respectively. From the experiments, we observed that the optimal values of η, η_l, τ are 10^{-1}, 10^{-2}, 10^{-4} respectively for all FedOpt algorithms. In case of `FedAvgM` optimal value of β is 0.9.

Results Discussion Results from all the experiments are tabulated in Table 3. In case of scenario-1, almost all FL algorithms including `FedAvg`, achieved good performance as it is homogeneous partition, i.e., IID data scenario. `FedYogi` and `FedAdam` outperformed among other strategies with F_1 scores of 0.8971 and 0.8962, respectively. In case of scenario-2, `FedAvg` not performed well as it is non-IID scenario but other FedOpt strategies performed well. `FedAdam` outperformed among other strategies with F_1 score 0.8874, loss 0.8921 and accuracy 0.8874.

We also conducted experiments for finding the number of required communication rounds to achieve a targeted F_1 score of 0.7. The results are illustrated in Fig. 3. From Fig. 3, one can observe that, in scenario-1, `FedAdam` could achieve targeted F_1 in 23 communication rounds, whereas Vanila `FedAvg` took 46 communication rounds. In scenario-2, other FL algorithms `FedYogi` (31 com. rounds), `FedAdam` (38 com.

Table 3 Experiments and observations

Scenarios	FL algorithm	F_1 score	loss	Accu.
Scenario-1	`FedAvg`	0.8018	0.5336	0.8059
	`QFedAvg`	0.8005	0.5349	0.8045
	`FedAdagrad`	0.6498	0.7059	0.6741
	`FedYogi`	**0.8971**	**0.2764**	**0.9003**
	`FedAdam`	**0.8962**	**0.2839**	**0.8989**
	`FedAvgM`	0.8852	0.3258	0.8872
Scenario-2	`FedAvg`	0.5772	0.8109	0.6173
	`QFedAvg`	0.5637	0.8272	0.6211
	`FedAdagrad`	0.6428	0.8121	0.669
	`FedYogi`	0.8609	0.3634	0.8698
	`FedAdam`	**0.8874**	**0.8921**	**0.8874**
	`FedAvgM`	0.8716	0.3387	0.8795



(a) Scenario-1 (b) Scenario-2

Fig. 3 Number of communications required to achieve target F_1 score $= 0.7$

rounds), `FedAvgM` (29 com. rounds) required very less number of communication rounds compared to vanilla `FedAvg` (71 com. rounds) to achieve the target F_1 score. `FedAdagrad` and `QFedAvg` could not achieve the targeted F_1 score in considered number of communication rounds (100).

The results of these numerical experimental indicate that `FedAdam` could achieve the best performance in terms of F_1 score, and accuracy in both IID and non-IID scenarios. Also, `FedAdam` could achieve a targeted F_1 score with a fewer communication rounds. Hence, one can conclude that `FedAdam` FL algorithm suits best for industrial time series fault classification application.

4 Conclusion

To find the best performing FL algorithm for industrial applications such as time series data classification, we studied different FL algorithms and FL frameworks (used for implementation of FL algorithms) specifically with non-IID data. We found that Flower framework which is ML framework agnostic, scalable, and platform-independent is the most suited framework for industrial edge applications. Through extensive real-time experiments on the PRONTO heterogeneous benchmark dataset, we demonstrated that `FedAdam` performs best against metrics of F_1 score and accuracy. Also, we not only evaluated these FL algorithms but also shared the most optimal values of Hyperparameters in FedOpt (`FedAdagrad`, `FedYogi`, `FedAdam`), `FedAvgM`, `QFedAvg`. Also, results indicated that `FedAdam` could achieve faster convergence in both IID and non-IID data scenarios.

References

1. Ge Z, Song Z, Ding SX, Huang B (2017) Data mining and analytics in the process industry: the role of machine learning. IEEE Access 5:20 590–20 616
2. Candanedo IS, Nieves EH, Gonzalez SR, Martın MTS, Briones AG (2018) Machine learning predictive model for industry 4.0. In: Uden L, Hadzima B, Ting I-H (2018) Knowledge management in organizations. Springer International Publishing, Cham, pp 501–510
3. Bagavathiappan S, Lahiri B, Saravanan T, Philip J, Jayakumar T (2013) Infrared thermography for condition monitoring-a review. Infrared Phys Technol 60:35–55
4. Abt S, Baier H (2014) Are we missing labels? a study of the availability of ground-truth in network security research. In: 2014 third international workshop on building analysis datasets and gathering experience returns for security (BADGERS). IEEE, pp 40–55
5. Diez-Olivan A, Del Ser J, Galar D, Sierra B (2019) Data fusion and machine learning for industrial prognosis: trends and perspectives towards industry 4.0. Inf Fusion 50:92–111
6. Moore ME, Ramage D, Hampson S, Arcas BAY (2017) Communication efficient learning of deep networks from decentralized data. In: -2017-Proceedings of the 20th international conference on artificial intelligence and statistics (AISTATS) 2017. PMLR, Fort Lauderdale, Florida, USA, pp 1273–1282. Accessed from 20–22 Apr 2017
7. Yang Q, Liu Y, Chen T, Tong Y (2019) Federated machine learning: concept and applications. ACM Trans Intell Syst Technol 10(2)
8. Nguyen V-D, Sharma SK, Vu TX, Chatzinotas S, Ottersten B (2021) Efficient federated learning algorithm for resource allocation in wireless IoT networks. IEEE Internet Things J 8(5):3394–3409
9. Hiessl T, Rezapour Lakani S, Kemnitz J, Schall D, Schulte S (2021) Cohort-based federated learning services for industrial collaboration on the edge
10. Hsu T-MH, Qi H, Brown M (2019) Measuring the effects of non-identical data distribution for federated visual classification. arXiv:1909.06335
11. Reddi SJ, Charles Z, Zaheer M, Garrett Z, Rush K, Konecný J, Kumar S, McMahan HB (2021)Adaptive federated optimization. In: ICLR
12. Li T, Sanjabi M (2020) Fair resource allocation in federated learning. In: ICLR
13. Li Q, Diao Y, Chen Q, He B (2021) Federated learning on non-iid data silos: an experimental study. arXiv:2102.02079
14. Aledhari M, Razzak R, Parizi RM, Saeed F (2020) Federated learning: a survey on enabling technologies, protocols, and applications. IEEE Access 8:140699–140725
15. Kholod I, Yanaki E, Fomichev D, Shalugin E, Novikova E, Filippov E, Nordlund M (2020) Open-source federated learning frameworks for IoT: a comparative review and analysis. Sensors 21(1):167
16. TensorFlow federated: machine learning on decentralized data. https://www.tensorflow.org/federated
17. Flower Framework. https://flower.dev/
18. Beutel DJ, Topal T, Mathur A, Qiu X, Parcollet T, Lane N (2020) Flower: a friendly federated learning research framework. vol. abs/ arXiv:2007.14390
19. Mathur A, Beutel Pedro Porto Buarque de Gusmao DJ, Fernandez-Marques J, Topal T, Qiu X, Parcollet T, Gao Y, Lane ND (2021) On-device federated learning with flower. arXiv:2104.03042v1 [cs.LG]
20. An industrial grade federated learning framework. https://fate.fedai.org
21. let's solve privacy. https://blog.openmined.org/tag/pysyft/
22. Xaynet framework. https://www.xaynet.dev/
23. PRONTO heterogeneous benchmark data set. https://zenodo.org/record/1341583#.Yl0r3NpBw2w
24. Kairouz P, McMahan HB, Avent B, Bellet A, Bennis M, Bhagoji AN, Bonawitz K, Charles Z, Cormode G, Cummings R et al (2019) Advances and open problems in federated learning. arXiv:1912.04977
25. https://www.raspberrypi.com/products/raspberry-pi-4-model-b/

Optimized Algorithms for Quantum Machine Learning Circuits

Lavanya Palani, Swati Singh, Balaji Rajendran, B. S. Bindhumadhava, and S. D. Sudarsan

Abstract Machine learning models for quantum computers are more powerful in providing faster computation, and better generalization with fewer data achieving potential quantum advantage. This paper focuses on the most important quantum machine learning algorithms for the quantum environments. Quantum machine learning is applied to the data generated from quantum experiments and is capable of learning some properties from quantum states under certain conditions. Quantum machine learning algorithms map the input dataset to a higher dimensional feature space using kernel functions. This paper also highlights the learning algorithm on quantum state which are proven for their efficiency using multiple copies of quantum states. Further, the paper explains the techniques that measure the loss function to optimize the learning algorithm and minimize the loss function.

Keywords Quantum machine learning · Feature map · Quantum optimization · Support vector machine · Quantum kernel · Loss function

http://www.cdac.in.

L. Palani (✉) · S. Singh · B. Rajendran · B. S. Bindhumadhava · S. D. Sudarsan
Cybersecurity Group, Centre for Development of Advanced Computing, Bengaluru, India
e-mail: lavanya@cdac.in

S. Singh
e-mail: swatisingh@cdac.in

B. Rajendran
e-mail: balaji@cdac.in

B. S. Bindhumadhava
e-mail: bindhu@cdac.in

S. D. Sudarsan
e-mail: sds@cdac.in

© The Author(s), under exclusive license to Springer Nature Singapore Pte Ltd. 2023

S. Chinara et al. (eds.), *Advances in Distributed Computing and Machine Learning*,
Lecture Notes in Networks and Systems 660,
https://doi.org/10.1007/978-981-99-1203-2_37

1 Introduction

In quantum computing, all information is encoded in quantum bits or qubits which is the basic unit of quantum information and also a variant of the classical bit. Using qubits and quantum operations, quantum computers are more powerful in improving the generalization of machine learning algorithms. Certain problems which are hard to solve by classical computers can be easily predicted by quantum computers. Quantum classifiers help in solving classification problem which can be extended further as variational quantum classifiers which use quantum circuits for classification. Parameterized quantum circuits allow the transformation of quantum states by changing the parameters on which measurement operations are applied to measure the qubit state.

2 Classification

Classical data are fed into the quantum machine and convert them into quantum states with quantum feature map and measure the output.

Supervised Binary Classification With new measurements, features in two-dimensional space can be classified by labeling the new data. The training set T is the vector in feature space [11] and the predefined map maps to $+1$ or -1 class. Test class also exists with the labels. Now classification is performed by finding the map that takes the training set and test set which agrees well with the unknown map c determining all true labels. With SVM, the idea is to draw a line between the two classes, and identify the support vectors that maximize the distance between the two classes [20].

Kernelized SVM For nonlinear feature transformation, we have to map from the original feature space into a higher dimensional feature space and replace the product of w and pi(x) with the kernel which introduces a function that maps the data from the original space to higher dimensional space. By rewriting the SVM, which depends explicitly only on the kernel values so that the computation of new feature vectors directly can be avoided which is time and resource-consuming.

If the original data is in two-dimensional space and when mapping it to three-dimensional space, an ordinary plane can be used to divide the two classes, and for more than two dimensions a hyperplane can be used.

Quantum Feature Maps and Kernels The general task of machine learning is to find and study patterns in data. Many machine learning [3] algorithms map the input dataset to a higher dimensional feature space using a kernel function [8].

With finite data[1], the kernel function is represented as a matrix [17]. If there exists a structure in the sample which cannot be linearly separated then in three dimensions, the data is separable by a hyperplane but in two dimensions the hyperplane is nonlinear as shown in Fig. 1.

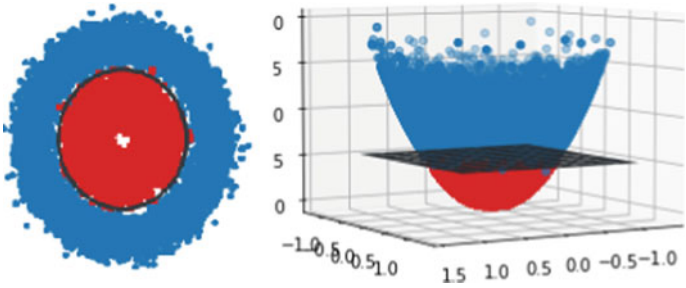


Fig. 1 Classification on dataset not linearly separable in two-dimensional space [16]

3 Different Approaches to Quantum Machine Learning

In quantum machine learning [23], the quantum feature map maps to a classical feature vector to a quantum Hilbert space [22]. A quantum feature map is a parameterized quantum circuit that is hard to stimulate and can be implemented by short-depth circuits on near-term quantum devices [26] which is a PauliFeatureMap. Quantum feature map with depth d contains layers of Hadamard gates interleaved with entangling blocks. In two dimensions [16], the ZZFeatureMap of two features has complicated class boundaries (Table 1). Support vector classification is a kernel machine learning algorithm that estimates the quantum kernel matrix representing the feature map as a parameterized quantum circuit. Each row and column of the kernel matrix represents the transition amplitude of a single data point with all other data points in a dataset. The quantum kernel matrix is symmetric and has a unit diagonal.

Quantum Support Vector Classification: Support vectors are the decision function for a linearly separable problem on the class boundaries. The nearest training data points to the separating hyperplane in each class is a support vector.

Quantum Kernel Alignment: Quantum feature maps can have variational parameters [10] that can be optimized using a technique called kernel alignment. Quantum kernels can also be used for other machine learning tasks, other than classification. Quantum metric learning is a technique that enables effective quantum kernel alignment [18].

Table 1 Classical ML versus Quantum ML

Classical versus quantum	Classical data	Quantum data
Algorithm runs on classical computer	Classical ML	ML on experimental data
Algorithm runs on Quantum computer	*QML*	Quantum–Quantum

Amplitude Encoding: Consider the set of data in quantum datasets, where each point xi is represented as a vector with coefficients vi,

$$|data\rangle \propto \sum_{l} \sum_{k} v_k^l |l\rangle |k\rangle$$

consider one of the states and represent that in a state vector by encoding the coefficient in the amplitude, further the entire dataset can be encoded into a single state by performing superposition of all amplitudes and state vectors.

Quantum Linear Solvers: This proves that the entire dataset contains potentially many data points that can be represented by one single quantum point. The results conclude two issues:

(i) reading out samples requires exponential run time and
(ii) working with a quantum state does not have any direct methodology to read the individual amplitudes, which can be done through samples leading to exponential overhead. Limitations of quantum linear system solvers include the relationship between the condition matrix A and the dimensions of A [16]. There are massive conditions on matrix coefficients and further, the read-in and read-out require exponential resources.

QSVM: When we consider quantum linear algebra, it is an extremely powerful formalizer and there are cases that cannot be simulated classically. There are different techniques, minimization procedures, and differential equations that can be mapped into linear systems, one of which is the Quantum Support Vector Machine (QSVM). The existing quantum algorithm for data feeding states that least square minimization can be mapped onto quantum linear systems and quantum linear systems can be solved through Hamiltonian simulation, through this series of mapping we can develop a quantum algorithm with exponential speedup under some assumptions for least square problems as shown in Fig. 2. The support vector machine can be mapped on the least square minimization problem with different reductions and mapping of one problem to another. A quantum support vector machine is a linear system problem that will provide exponential speed up considering certain assumptions.

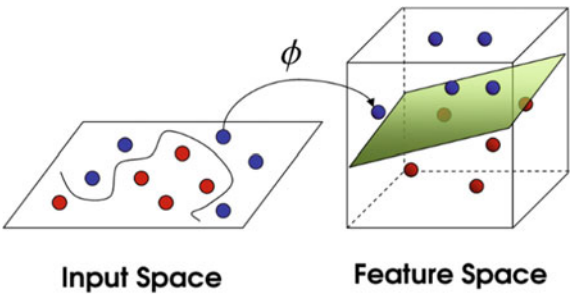


Fig. 2 Coefficient values estimated using least squares [11]

4 Dequantization

On the classical counterpart, the assumptions are powerful that problems are framed for a classical algorithm that does not output the entire solution vector, but the quantum algorithm only samples from it. The classical algorithm typically does not have any constraints about lowering matrices or the constraints on sparsity are much milder in the quantum case, so the output is much more from the classical than the quantum algorithm. The data provided in the quantum algorithm is in superposition which is an extra assumption that provides an unfair advantage to the quantum algorithm. All these algorithms can be effectively dequantized which points that while comparing a quantum machine learning algorithm with a classical machine learning algorithm and with a focus on quantum advantage, we have to match the preparation assumptions carefully between these two different models if we assume QML algorithms have to access some quantum states then the classical model should also be able to sample from this state. It's very hard to construct a classical model that would match the assumptions of a classical model, we first assume that the classical model has access to the data structure that allows sampling from the inputs. While working on the low-rank approximation of the matrix, so the classical will not store the entire matrix [17] from memory under this assumption there are some randomized algorithms that will have the complexity of poly logarithmic in the dimension [16]. For quantizing several other QMLs like quantum principal component analysis, QSVM, and quantum linear system solvers that work with classical data are fed through QRAM using highly constrained cases of quantum linear algebra.

5 Polynomial Speedups

Exponential speedups are very rare in quantum computing, polynomial speedups, or quadratic speedups for searching in an unstructured database through Grover search and this technique can also be generalized to quantum amplitude estimation [21] or amplitude amplification using machine learning algorithms like nearest-neighbor methods and Bayesian network. These techniques were giving asymptotic speed up when implemented in fault-tolerant quantum computers that have clock speeds much slower than traditional computers and many of these algorithms have a large constant overhead. Estimating overheads for quantum optimization and graph algorithm for quadratic speedups, both state that the perspective should be of very large instances to show a quantum algorithm performing similarly to a classical algorithm.

6 Learning Theory

Complexity theory specifically shows how to connect machine learning [6] tasks with complexity results that can or cannot be done with classical and quantum computer efficiently. In the theory of learning, complexity results show which functions or classes of probability distributions can be learnt in a polynomial time. They are rigorous lower and upper bounds considering the number of samples that can be learnt from complex class [28]. For practical applications, one of the big downsides of learning theory [12] is that the problems considered for experiments are generic in nature, for example, general unitary quantum states [15] which are highly structured. These types of probability functions will separate quantum and classical techniques, but they are not the real-world problems in practice. In learning theory [12], one of the fundamental results is no-free lunch theorem which connects learning with optimization.

6.1 No-Free Lunch Theorem

The no-free lunch theorem tells us that when performance is considered over averages of all possible problem instances and of training sets then the performance will only depend on the size of the training set and not on the optimization procedure. Unknown function $f : X \to Y$, Hypothesis $h : X \to Y$ given a subset of $X \times Y$, Risk $R_f(h) := P[f(x) \neq h(x)]$. This provides an understanding that with a given small training set and consider only small subset of training dataset can make classification difficult and unclear and the hypothesis may fail leading to further challenges where considering all instances of training data is not possible in real time classification problems. Quantum no-free lunch theorem can be extended by considering the input and output that are quantum states, training set: input and output are quantum states and the goal is to learn a unitary $U : H_x \to H_y$ which can be extended with different dimensions. But assume when input and output states have same dimensions then the learning function is unitary which leads to complications, but guarantees the function will be one to one. Also, results show that considering examples that can be entangled with some reference system in the limit of maximal entanglement will need fewer different states. This points to some of the difficulties that need to be considered when extending classical results in the quantum realm, as in the classical states how many copies of data needed to learn is not considered. But in the quantum case, the sample complexity and how many copies need from each state can really change the performance significantly.

6.2 PAC Learning

It is always difficult to learn completely but what is required is confidence over the learning performed, this is stated by Probably Approximately Correct (PAC) Learning where approximate learning is used. PAC learning is a well-known framework which goals to learn some functions from samples and the results need not be always correct, but the results should be approximately correct with high probability. To generalize the hypothesis, and test it on unknown examples, the results are bounded with approximation or generalization error [28]. This framework provides a concrete theoretical tool for connecting complexity and quantum machine learning [23] where the focus is not on the performance of the unknown data but explicitly on the generalization error. Results accomplished in the PAC learning space concrete the learning possibilities of classical and quantum environment. The results state that (i) problems could be difficult for a classical learner but can perform efficiently with quantum computer. (ii) Outputs from quantum circuits connected to circuit sampling or connect with discrete logarithmic problem are hard for classical computers. (iii) Solving discrete logarithmic problem by breaking encryptions can be done efficiently with finite data.

7 Learning Quantum States

As an extension of PAC learning, it's known that learning the quantum state is hard. To learn the quantum states for a well-known problem, the amplitudes and the elements of the density matrix are required that can be achieved by tomography [7] which is exponentially in the number of qubits. Sometimes, amplitudes may not be the concern but interest could be in other properties of quantum states where tomography may not perform efficiently leading to much more efficient algorithms. There are known algorithms that are proven efficient in the number of sampling which uses entanglement by looking at multiple copies simultaneously.

7.1 Classical Shadows

Classical shadows are the framework that does not give an exact description of quantum states but can reproduce classical data that is efficiently estimated and also does not require exponential memory. Learning with unentangled measurements is the ability to create entanglement between different copies of the system is useful but not always achievable in the quantum experimental setting. To learn some unknown state from the circuits with multiple rows, the measurement of only one single row copy at that time is required. This is called single-copy or non-entangled measurements. It is always useful to have multiple copies of quantum state at the same time

so that some quantum circuits and entanglement can be introduced on it before measurement as stated by the optimal tomography protocols. Quantum system performs some random unitary[15] and measures the states on different bases and remembers the outcomes. These outcomes are used to represent classical representation. Shadow description is used for predicting properties of quantum states. There are many applications of quantum information processing with possible properties such as quantum fidelity, entanglement witness, entanglement entropy, two-point correlations, Hamiltonian, and local observables. Learning with shadow data requires training data from quantum experiments or simulations in the form of classical shadows. Using classical shadow data, the quantum states can be predicted and classified.

7.2 Hamiltonian Learning

An interesting framework for learning dynamics considering many properties of quantum states. If some dynamics happen between qubit and entanglement, by observing the system and from the dynamics, it is possible to estimate the Hamiltonian by observing some interactions between individual spins. This type of problem is called Hamiltonian learning where quantum systems are accessed and its Hamiltonian are estimated. Hamiltonian can be learnt from a dynamical process, we can also do from the thermal state. Given a Hamiltonian and temperature, gives the Gibbs state corresponding to the temperature. Given some copies of Fibbs states either for high- or low-temperature observation is made on how difficult is to learn the Hamiltonian and how many measurements are required to perform the learning task. Only a polynomial number of local measurements on the thermal state of a quantum state are necessary and sufficient for accurately learning its Hamiltonian. Future interesting experiments: With devices like NISQ, noisy intermediate scale quantum, interesting experiments are generated that show the states a quantum algorithm can learn, what data can be processed classically and what problems can be solved in a faster way. With very limited quantum processing power for QML algorithms. Quantum computer performs some quantum operations which is noisy and this quantum device-generated data [1] is fed into a classical computer and measured directly to perform machine learning on classical data coming from quantum experiments.

7.3 Variational Quantum Circuits as QML Model

Quantum algorithm that runs on near-term devices uses a popular class of algorithm called variational quantum circuits [25]. Some parameterized quantum circuits have some fixed operations and some can be tuned to different parameters to measure some subset of qubits. Qubits to be measured are considered as visible qubits and the rest are ancillary qubits labelled as hidden which is similar to neural network. In

quantum neural network, these qubits can be trained and use to map inputs to output which does the prediction of properties or can classify different inputs.

In QML model [10], all the qubits are not measurable, only one qubit might be measured depending on the loss function which is considered. Variation framework is very general to build all types of states and perform measurements at the end. But measuring qubits is performed before noise destroys the states. Binary classifier is a type of variational circuit with input data, applying a parameterized quantum circuit on it. The goal of the training is to take inputs from one class and observe the output of measured qubit and for the other remaining classes the output is always zero. All qubit with one will be hidden qubits irrespective of objective function does correct classification or not.

Another type of variational circuit is Convolutional Quantum Neural Network (CQNN) which is a quantum circuit of logarithmic depth with two different types of layers. In convolutional layer, quantum operators are applied and pooling layer measurement of other qubits is performed and, to the remaining qubit, we will apply gates that will be conditional on the measuring outcomes. This step is performed logarithmic number of times until the number of qubits is reduced to some constant number. QNN [27] is based on tensor networks where full tomography [7] on the state can be done to reduce to one qubit and process it. Another model is deep QNN also called dissipative QNN where all the units in the QNN [27] are assigned different qubits, as we move from one layer to other, some new qubits are introduced followed by tracing the remaining ones. The process is dissipative, as the measurement of qubits is done from the output layer.

8 Quantum Optimization Algorithm and Loss Function

In several fields, many problems can be categorized as optimization problems as they provide complex decision-making with well-defined strategies which are to be minimized or maximized. Hence, it is also termed an objective function. Ising Hamiltonian does the automatic conversion of problems to required representations using optimization algorithms like QAOA, Grover Adaptive search [4], and classical solvers. These problems are termed as searching for an optimal solution in a finite or countably infinite set of potential solutions. In QNN, one or more qubits are measured and with the outcome, decision is made on how well the QNN [27] model performs. The function that defines how well the model performs is the cost or loss or objective function in the quantum environment. Single qubit gate does not provide any advantage over classical computer, so mostly two qubit gates are used in QNN. The goodness of the QNN is measured by the fidelity between the output and what is expected. With generative models, the outcome would be large quantum state. The problem of using infidelity for training is that when we are in one state and want to learn from other random states, the infidelity between them will be close to one for the majority of different states. So, the overlap between two randomly chosen state in large Hilbert space [22] is very small and difficult to train. Hence, energy

penalty is assigned to penalize the states. The variation algorithm will minimize the energy of the system. Quantum relative entropy helps to create quantum analog of KI Divergence, but they cannot be estimated very efficiently as computing its gradient [14] is difficult. There are some cases where we cannot estimate the gradient which is not in a closed form and have hidden units. In QML, there are several techniques that will directly measure the objective function and compute the derivatives using finite methods. Using kernel methods [18], quantum states are mapped to some higher dimension space and perform classification in that space, these are the feature map. Using feature map, a quantum learner can recognise classically intractable complex patterns such as discrete log performing efficient learning with Quantum kernels.

9 Conclusion and Future Work

QML is more efficient when used for an abstract mathematical problem where classification is required so that the data [1] coming from quantum process of quantum machine learning [3] can extract valuable information which runs exponentially. Quantum machine learning [10] algorithms are advantageous when used under a very structured problem so that they can speed up which also emphasizes that classical machine learning is efficient on problems which are uncertain. In the current scenario, only few proofs are available to show where the quantum computers work better and which quantum algorithm can do exponentially more quickly say when a mathematical approach of traversing a messier graph. Also when working on a high-dimensional feature say protein folding experimenting to find the mathematical structure itself is a hard problem and quantum computer will be at the top in proving its efficiency.

References

1. Huang HY, Broughton M, Mohseni M et al (2021) Power of data in quantum machine learning. Nat Commun 12:2631. https://doi.org/10.1038/s41467-021-22539-9
2. Martín-Guerrero JD, Lamata L (2022) Quantum machine learning: a tutorial, neurocomputing, vol 470, pp 457–461. ISSN 0925-2312, https://doi.org/10.1016/j.neucom.2021.02.102
3. Pushpak SN, Jain S (2021) An introduction to quantum machine learning techniques. In: 2021 9th international conference on reliability, infocom technologies and optimization (trends and future directions) (ICRITO), pp 1–6. https://doi.org/10.1109/ICRITO51393.2021.9596240
4. Grover LK (1996) A fast quantum mechanical algorithm for database search. In: Proceedings of twenty-eighth annual ACM symposium on theory of computing
5. Caro MC, Huang HY, Cerezo M et al (2022) Generalization in quantum machine learning from few training data. Nat Commun 13:4919. https://doi.org/10.1038/s41467-022-32550-3
6. Mishra N, Kapil M, Rakesh H, Anand A, Mishra N, Warke A, Sarkar S, Dutta S, Gupta S, Dash A, Gharat R, Chatterjee Y, Roy S, Raj S, Jain V, Bagaria S, Chaudhary S, Singh V, Maji R, Panigrahi P (2019) Quantum machine learning: a review and current status. https://doi.org/10.13140/RG.2.2.22824.72964

7. Kieferova M, Wiebe N (2016) Tomography and generative data modeling via quantum Boltzmann training. PRX Quantum
8. Schuld M, Killoran N (2022) Is quantum advantage the right goal for quantum machine learning?. PRX quantum. https://doi.org/10.1103/PRXQuantum.3.030101
9. Schölkopf B et al (2002) Learning with kernels: support vector machines, regularization, optimization, and beyond. https://mitpress.mit.edu/books/learning-kernels
10. Schuld M, Sinayskiy & Petruccione F (2015) An introduction to quantum machine learning. Contemp Phys 56:172–185
11. Havlek V et al (2019) Supervised learning with quantum-enhanced feature spaces. Nature 567:209–212
12. Arunachalam S, de Wolf R (2017) A survey of quantum learning theory. https://arxiv.org/abs/1701.06806
13. Schuld M, Sinayskiy I, Petruccione F (2016) Prediction by linear regression on a quantum computer. Phys Rev A 94:022342
14. Rebentrost P, Schuld M, Petruccione F, Lloyd S (2016) Quantum gradient descent and Newton's method for constrained polynomial optimization. https://arxiv.org/abs/1612.01789
15. Bisio A, Chiribella G, D'Ariano GM, Facchini S, Perinotti P (2010) Optimal quantum learning of a unitary transformation. Phys Rev A 81:032324
16. Lau H-K, Pooser R, Siopsis G, Weedbrook C (2017) Quantum machine learning over infinite dimensions. Phys Rev Lett 118:080501
17. Wossnig L, Zhao Z, Prakash A (2017) A quantum linear system algorithm for dense matrices. https://arxiv.org/abs/1704.06174
18. Chatterjee R, Yu T (2016) Generalized coherent states, reproducing kernels, and quantum support vector machines. https://arxiv.org/abs/1612.03713
19. Alvarez-Rodriguez U, Lamata L, Escandell-Montero P, Martín-Guerrero JD, Solano E (2016) Quantum machine learning without measurements. https://arxiv.org/abs/1612.05535
20. Schuld M, Fingerhuth M, Petruccione F (2017) Quantum machine learning with small-scale devices: implementing a distance-based classifier with a quantum interference circuit. https://arxiv.org/abs/1703.10793
21. Wan KH, Dahlsten O, Kristjánsson H, Gardner R, Kim MS (2016) Quantum generalisation of feedforward neural networks. https://arxiv.org/abs/1612.01045
22. Schuld M, Killoran N (2019) Quantum machine learning in feature Hilbert spaces. Phys Rev Lett 122:040504
23. Biamonte J (2017) Quantum machine learning. Nature 549:195–202
24. Schuld M, Sinayskiy I, Petruccione F (2014) The quest for a quantum neural network. Quantum Inf Process 13:2567–2586
25. Cerezo M (2021) Variational quantum algorithms. Nat Rev Phys 3:625–644
26. Farhi E, Neven H (2018) Classification with quantum neural networks on near term processors. arXiv:1802.06002
27. Abbas A (2021) The power of quantum neural networks. Nat Comput Sci 1:403–409
28. Huang HY, Kueng R, Preskill J (2021) Information-theoretic bounds on quantum advantage in machine learning. Phys Rev Lett 126:190505

Prediction of SOH and RUL for Lithium-Ion Batteries Using Regression Method with Feature of Indirect Related to SOH (FIRSOH) and Linear Time Series Model

Aradhna Patel and Shivam Patel

Abstract Lithium-ion battery state of health (SOH) and residual usable life (RUL) are two significant parameters that are typically estimated using battery capacity. The capacity of lithium-ion batteries for online applications can be measured indirectly. In this study, voltage, current, and temperature curves during charging and discharging of lithium-ion batteries—which react to the process of battery capacity degradation—are used to extract Feature of indirect related to SOH (FIRSOH). The Linear Ridge Regression (LRR) method is used for SOH prediction. Then, taking into account that SOH and RUL have a particular mapping relationship, five FIRSOHs and the current SOH value are used to forecast RUL of lithium-ion batteries using the linear time series (LTS) model. The outcomes demonstrate that the suggested approach has a high level of prediction accuracy.

Keywords SOH · RUL · Li Ion battery · LRR.LTS

1 Introduction

Electric cars (EVs), consumer gadgets, and even spacecraft all rely heavily on lithium-ion batteries [1–3]. Lithium-ion batteries' dependability and safety are thus a major issue when it comes to actual applications. With an increase in service life, batteries' performance steadily declines, which may not only impair the regular operation of electrical equipment but can have serious implications [4]. For instance, in recent years there have been explosions of battery energy storage boxes in some power plants, spontaneous combustion of electric vehicles, and the cell explosion of the

A. Patel (✉)
MNNIT Allahabad, Prayagraj, India
e-mail: aradhnaen1009@gmail.com

S. Patel
Galgotias University, Greater Noida, India

© The Author(s), under exclusive license to Springer Nature Singapore Pte Ltd. 2023

S. Chinara et al. (eds.), *Advances in Distributed Computing and Machine Learning*,
Lecture Notes in Networks and Systems 660,
https://doi.org/10.1007/978-981-99-1203-2_38

Samsung NOTE7 [5]. SOH and RUL prediction of lithium-ion batteries has emerged as a hot topic and difficult topic in the prognostics and health management (PHM) of electronics in an effort to prevent such mishaps.

The battery management system (BMS) is actually created for a variety of instruments to guarantee secure operation. The primary roles of BMS in modern practice are SOH determination and RUL prediction. Utilizing data from online measurements, such as current, voltage, and temperature, they must be calculated. Model-based approaches and data-driven approaches make up the bulk of the currently used lithium-ion battery SOH and RUL prediction techniques [6].

The simplest Thevenin model with a single RC branch is provided in Reference [7], and all of the model's parameters are fixed. Many internal battery properties as well as resistance ageing parameters are included in comparable circuit models used to calculate battery ageing. It is necessary to identify parameters using a big and varied data set that was gathered through time-consuming tests.

However, it is challenging to identify the key factors involved in the model-based strategy due to the limited understanding of the capacity degradation mechanism of lithium-ion batteries. Through parameter identification, it can be challenging to determine some side reaction parameters that follow the primary reaction. Additionally, model-based techniques perform poorly in real-time.

Data-driven approaches have been getting more and more attention as machine learning and artificial intelligence have developed quickly [8, 9]. Data-driven methods are nonparametric in comparison to model-based methods and do not fully take into account electrochemical principles. As a result, several mapping and regression algorithms are used to construct degradation models for lithium-ion batteries.

Time series analysis, artificial neural networks (ANN), relevance vector machines (RVM), Gaussian process regression (GPR), and others are examples of existing methodologies. By using particle swarm optimization, online predictions were made by [10] using an enhanced autoregressive (AR) model via particle swarm optimization (PSO). To anticipate SOH and RUL, some of the approaches mentioned above typically use capacity degradation series or impedence.

However, it is challenging to do online measurements using the capacity fade data to estimate SOH and RUL since measuring the impedance and resistance takes time [11].

In order to replace capacity data, several researchers have used indirect features. These, such as current, voltage, and temperature, among others, can be conveniently measured in real-time and online.

The fundamental contribution of this research is that using Feature of indirect related to SOH (FIRSOH) and the LRR model, the suggested method can forecast the short-term SOH of lithium-ion batteries. The first suggestion made in this work is to extract indirect health indicators from data that may be gathered by regular sensors, such as voltage, current, and temperature curves, in order to lower the cost of the prediction approach and increase the forecast accuracy. The LRR model is then designed to forecast the short-term SOH of lithium-ion batteries using FIRSOHs that have a good correlation with the capacity deterioration curve as high-dimensional

input. Finally, three FIRSOHs, the current SOH value, and the mapping relationship between SOH and RUL are used to forecast RUL of lithium-ion batteries using the LTS model.

The structure of the essay is as follows. In Sect. 2, a basic introduction to FIRSOH selection and extraction is given. LRR prediction technique and SOH is discussed in Sect. 3. Section 4 reports the RUL simulation results using LTS method. Finally, Sect. 5 presents the conclusion.

2 Experimental Information

The open data set of the NASA Ames Prognostics Center of Excellence (PCoE) provided the lithium-ion batteries data set used in this study [12]. Four lithium-ion batteries—Nos. 5, 6, 7, and 18—from an experimental set of lithium-ion battery data are employed. At room temperature (24 °C), they are charged and discharged, and the impedance is measured under various working conditions. There were two stages to the charging process. The battery voltage was first charged at 1.5 A of constant current until it reached 4.2 V. The battery voltage was maintained at 4.2 V and the current was decreased to 20 mA in the second step using constant voltage charging. The process of discharging was carried out at a constant current of 2 A until the voltages of Nos. 5, 6, 7, and 18 fell to, respectively, 2.7, 2.5, 2.2, and 2.5 V.

2.1 Extraction of Feature of Indirect Related to SOH (FIRSOH)

Lithium-ion battery capacity is often determined by measuring the internal resistance of the battery to reflect the specific capacity [13]. This is due to the fact that as battery

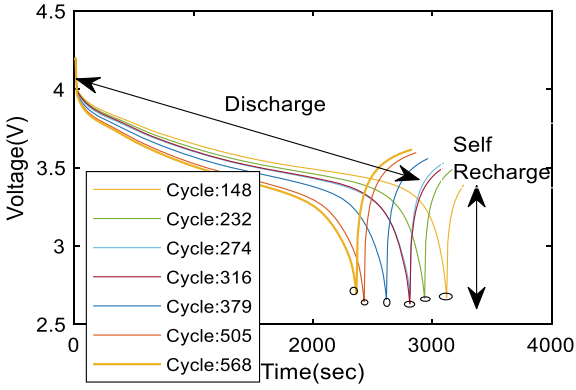


Fig. 1 Discharge voltage plot for B0005 Li Ion battery

capacity and life are lost, the impedance of the battery rises. However, impedance measurement using an impedance meter is difficult and time-consuming [14, 15]. In order to measure some indirect parameters that are simple to get using common sensors, it is fairer to estimate the SOH and RUL of lithium-ion batteries [16].

In this study, the charge and discharge voltage, current, and temperature curves are used to extract FIRSOH that indicate the lithium-ion batteries' capacity. Figure 1 displays the discharge process voltage of lithium-ion batteries with various cycles. It is evident, when the no. of cycles increases, the voltage discharge length is increases, similarly from Figs. 2, and 3 show that battery reaches maximum temperature in lesser time and duration of constant discharge current also reduces as no of cycles increases. On the basis of these analysis, we have selected five features indirectly related to battery capacity fade. The details of FIRSOH are given below:

- It is clear that as cycle numbers rise, less time is required to reach the lowest discharge point (FIRSOH 1).

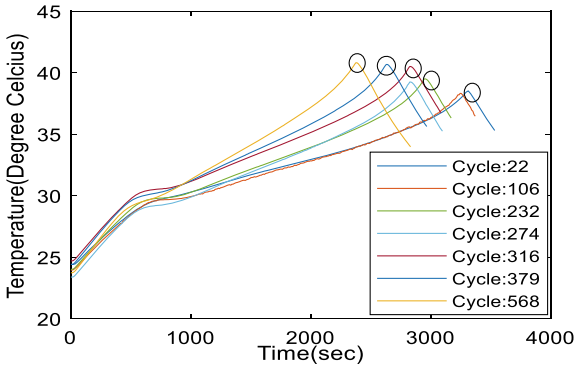


Fig. 2 Discharge temperature plot for B0005 Li Ion battery

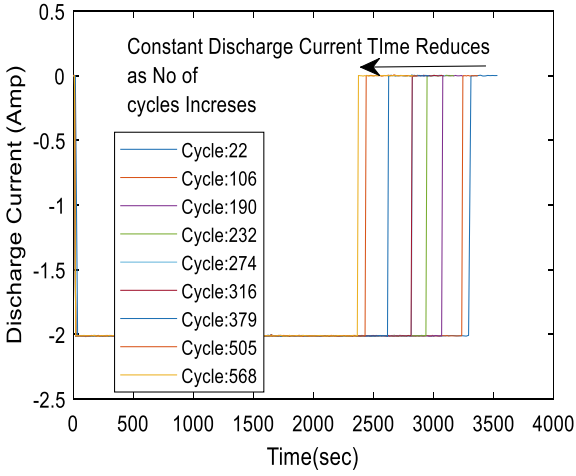


Fig. 3 Discharge Current plot for B0005 Li Ion battery

- The duration shortens as the number of cycles increases when the battery temperature reaches its peak point, known as FIRSOH 2.
- The corresponding time related to FIRSOH 1 and FIRSOH 2 are FIRSOH 3 and FIRSOH 4 respectively.
- The last feature selected is FIRSOH 5 as, the voltage corresponding to last sample of discharge cycle.

3 Prediction of SOH Using LRR Method

In this case study, battery's data (B0005) is utilized for prediction of the SOH when the no. of cycles increases. We have selected five FIRSOHs. LRR is a statistic method which is generally used for prediction. The normalized data set have (X, Y). Here, we have used 60 percent data $(X_{Trainig}, Y_{Trainig})$ for training and 40% data for (X_{Test}, Y_{Test}) testing. The Predicted SOH is represented by $S\hat{O}H$. Formula of SOH is calculation from capacity is given as:

$$SOH = \frac{Capacity \times 100}{Initial\ Capacity} \tag{1}$$

Figure 4 shows the predicted SOH using LRR method. The predicted SOH is very close to original battery B0005 data. The root means square error (RMSE) and mean absolute percentage error (MAPE) of SOH, are shown in Table 1 for all lithium-ion batteries.

$$MAPE = \frac{100\%}{n} \sum_{i=1}^{n} \left| \frac{S\hat{O}H(i) - SOH(i)}{SOH(i)} \right| \tag{2}$$

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^{n} \left(S\hat{O}H(i) - SOH(i) \right)^2} \tag{3}$$

4 RUL Prediction Using LTS Method

Lithium-ion batteries' RUL and SOH have a specific mapping relationship, and SOH can be mapped using FIRSOHs. As a result, the RUL of lithium-ion batteries is predicted using the LTS model using five FIRSOHs and the current SOH values. The lithium-ion battery end-of-life (EOL) cycle is set in accordance with the cycle point that corresponds to the battery capacity deterioration at the failure threshold. The battery B0005 have a capacity failure threshold of 1.38 Ah. Battery's EOL cycle is 129. The efficacy of proposed LTS method for EOL estimation is reflected from

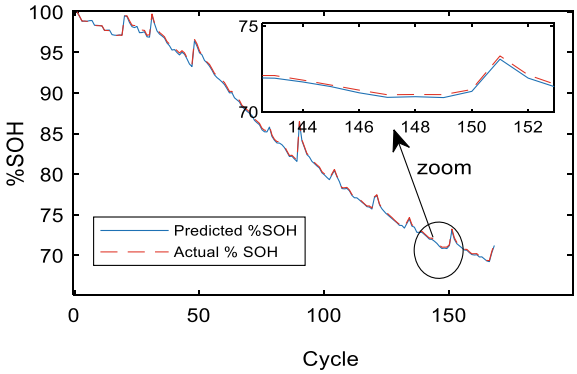


Fig. 4 Estimated SOH for B0005 Li Ion battery using LRR method

Table 1 RMSE and MAPE in SOH prediction for three Li Ion batteries

Li Ion battery	RMSE (Root Mean Square Error)	MAPE
B0005	0.022	2.2
B0006	0.026	2.6
B0007	0.038	3.8
B0018	0.027	2.7

simulation result. Figure 5 shows that estimated life cycle of the battery B0005 using LTS method.

Properties: Description: "ARIMA (15,1,1) Model (Gaussian Distribution)".

Distribution: Name = "Gaussian".

P: 16, D: 1, Q: 1, Constant: −0.0023938.

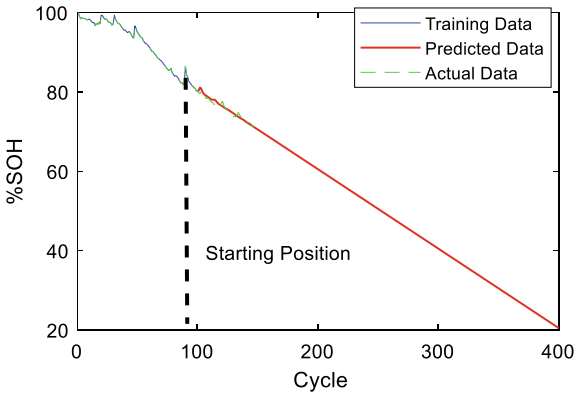


Fig. 5 Predicted RUL for B0005 Li Ion battery using lTS method

AR: {0.33541 0.0205154 −0.0781077 −0.0165676 −0.020057 −0.00953649 0.00405635 −0.00423132 −0.0568304 0.0343355 0.0847939 0.17043 0.00453367 −0.266526 0.165811} at lags [1 2 3 4 5 6 7 8 9 10 11 12 13 14 15].

MA: {−0.5728} at lag [1]

Variance: 0.000199184,

Elapsed time is 5.507231 s.

Model (Gaussian **Distribution**):

	Value	StandardError	TStatistic	PValue
Constant	-0.0023938	0.0080337	-0.29797	0.76573
AR{1}	0.33541	1.4359	0.23358	0.81531
AR{2}	0.020515	0.39921	0.05139	0.95901
AR{3}	-0.078108	0.26138	-0.29883	0.76507
AR{4}	-0.016568	0.3455	-0.047952	0.96175
AR{5}	-0.020057	0.27884	-0.071931	0.94266
AR{6}	-0.0095365	0.28975	-0.032912	0.97374
AR{7}	-0.0040564	0.29255	-0.013865	0.98894
AR{8}	-0.0042313	0.24031	-0.017608	0.98595
AR{9}	-0.05683	0.25365	-0.22405	0.82272
AR{10}	0.034336	0.2709	0.12675	0.89914
AR{11}	0.084794	0.19301	0.43932	0.66043
AR{12}	0.17043	0.20743	0.82161	0.4113
AR{13}	0.0045337	0.35523	0.012763	0.98982
AR{14}	-0.26653	0.30374	-0.87749	0.38022
AR{15}	0.16581	0.24818	0.6681	0.50407
MA{1}	-0.5728	1.4274	-0.4013	0.6882
Variance	0.00019918	2.3831e-05	8.3581	6.3746e-17

5 Conclusion

Li Ion batteries electrical parameters are dynamics in nature. They are continuously changing with time. So, model-based technique is difficult to use for SOH and RUL estimation. A chemical reaction is continuously occurring inside the LI Ion battery, as number of charge/ discharge cycle increases the health of battery degrades and capacity reduces. Here we used, a unique LRR with FIRSOHs-based SOH and RUL prediction scheme for lithium-ion batteries. First, FIRSOHs that replace the battery capacity are derived using voltage, temperature, and current data that have been gathered using various common sensors. The experimental findings demonstrate that the strategy put forth in this study is precise and efficient in predicting SOH and

RUL, and that it can greatly boost prediction performance. Experiments for lithium-ion batteries under various operating situations, such as various charging currents, changes in ambient temperature, various discharge voltages, etc., were prepared to confirm the adaptability of the suggested method.

References

1. Lu L, Han X, Li J, Hua J, Ouyang M (2013) A review on the key issues for lithium-ion battery management in electric vehicles. J Power Sour 226:272–288
2. Takami N, Inagaki H, Tatebayashi Y, Saruwatari H, Honda K, Egusa S (2013) High-power and long-life lithium-ion batteries using lithium titanium oxide anode for automotive and stationary power applications. J Power Sour 244:469–475
3. Hu X, Zou C, Zhang C, Li Y (2017) Technological developments in batteries: a survey of principal roles, types, and management needs. IEEE Power Energy Mag. 15:20–31
4. Doughty DH, Roth EP (2012) A general discussion of Li Ion battery safety. Interf Mag. 21:37–44
5. Wang Q, Mao B, Stoliarov SI, Sun J (2019) A review of lithium-ion battery failure mechanisms and fire prevention strategies. Prog Energy Combust Sci 73:95–131
6. Su C, Chen HJ (2017) A review on prognostics approaches for remaining useful life of lithium-ion battery. IOP Conf Ser Earth Environ Sci 93:012040
7. Liu C, Wang Y, Chen Z (2019) Degradation model and cycle life prediction for lithium-ion battery used in hybrid energy storage system. Energy 166:796–806
8. Wang Y, Yang D, Zhang X, Chen Z (2016) Probability based remaining capacity estimation using data-driven and neural network model. J Power Sour 315:199–208
9. Li Y, Liu K, Foley AM, Berecibar M, Nanini-Maury E, van Mierlo J, Hoster HE (2019) Data-driven health estimation and lifetime prediction of lithium-ion batteries: a review. Renew Sustain Energy Rev 113:109254
10. Long B, Xian W, Jiang L, Liu Z (2013) An improved autoregressive model by particle swarm optimization for prognostics of lithium-ion batteries. Microelectron Reliab 53:821–831
11. Yu J (2018) State of health prediction of lithium-ion batteries: multiscale logic regression and Gaussian process regression ensemble. Reliab Eng Syst Saf 174:82–95. [CrossRef]
12. Saha B; Goebel K (2019) Battery Data Set. https://ti.arc.nasa.gov/tech/dash/groups/pcoe/prognostic-data-repository/. Aaccessed 10 May 2019
13. Razavi-Far R, Chakrabarti S, Saif M (2016) Multi-step-ahead prediction techniques for Lithium-ion batteries condition prognosis. In Proceedings of the IEEE international conference on systems, man, and cybernetics, SMC 2016–conference proceedings, Budapest, Hungary, 9–12 Oc 2016; pp 4675–4680
14. Coleman M, Kwan LC, Chunbo Z, Hurley WG (2007) State-of-Charge determination from EMF voltage estimation: using impedance, terminal voltage, and current for lead-acid and lithium-ion batteries. IEEE Trans Ind Electron 54:2550–2557
15. Sun YH, Jou HL, Wu JC (2011) Aging estimation method for lead-acid battery. IEEE Trans Energy Convers 26:264–271
16. Liu D, Zhou J, Liao H, Peng Y, Peng X (2015) A health indicator extraction and optimization framework for lithium-ion battery degradation modeling and prognostics. IEEE Trans Syst Man Cybern Syst 45:915–928

Chatbot for Mental Health Diagnosis Using NLP and Deep Learning



Neel Ghoshal, Vaibhav Bhartia, B. K. Tripathy, and A. Tripathy

Abstract In the discussion of health, human rights, and equality, mental disability and mental health care have been overlooked. This is puzzling considering that 8% of the world's population suffers from mental impairments, which are widespread. A scalable option that offers an interactive way to engage consumers in behavioral health interventions powered by artificial intelligence might be chatbots. Although several chatbots have showed early efficacy results that are encouraging, there is little data on how people really utilize these chatbots. Understanding chatbot usage trends for mental health issues such as depression, anxiety, stress, etc. provides a critical first step in enhancing chatbot design and revealing the advantages and disadvantages of the chatbots. In this paper a customized chatbot framework is proposed with a blended neural network design. The dataset used is completely scraped and prepared manually to inculcate the various mental health diseases and the appropriate responses provided by professionals. The dataset upon preprocessing undergoes various state of the art deep learning models such as Logistic Regression, Decision Tree, Random Forest, and Naive Bayes. Performance of the proposed mechanism is further compared with the simulated outcomes of the other existing models. Nevertheless to mention, the results show promising rate of efficiency.

Keywords Chatbot · Mental health · NLP · Deep learning · Soft computing

N. Ghoshal · V. Bhartia
SCOPE, Vellore Institute of Technology, Vellore 632014, TN, India

B. K. Tripathy (✉)
SITE, Vellore Institute of Technology, Vellore 632014, TN, India
e-mail: tripathybk@vit.ac.in

A. Tripathy
Carnegie Mellon University, Pittsburgh, PA 15213, USA

© The Author(s), under exclusive license to Springer Nature Singapore Pte Ltd. 2023

S. Chinara et al. (eds.), *Advances in Distributed Computing and Machine Learning*,
Lecture Notes in Networks and Systems 660,
https://doi.org/10.1007/978-981-99-1203-2_39

1 Introduction

Nobody wants to talk about the axiomatic problem in the world that is mental health [1]. There is hardly any public discussion about how to prevent or cure mental illness in India [2], despite the country being on the cusp of a crisis. Al-most little action is being taken on the scale necessary to address the rising number of individuals with mental health difficulties.

There is a significant gap between the support that is readily accessible, easily affordable, and the treatment that should be offered. Even in wealthy countries, the ratio of psychiatrists, psychologists, psychiatric social workers, and mental health nurses to patients is one to 10,000 [3]. The system's flaw ensures that the majority of those with mental health problems never get the assistance they require. Numerous digital interfaces are emerging as practical supplemental services to fill some of the needs of artificial intelligence-based solutions that offer help and frequently some form of companionship. These solutions were developed in close collaboration with healthcare professionals. Additionally, it may lower the price of psychiatric diagnosis and care. When it comes to psychiatric diseases, most people have experienced the stigma that pervades our society and frequently prevents proper treatment. Using spoken, written, and visual languages, chat-bots are NLP-based frameworks that communicate with human users [4]. Chat-bots, created expressly to communicate with persons with mental health issues, have the potential to be helpful resources. On multiple platforms, a chatbot can imitate a conversation using text, audio, and even video. While some chatbots use human interface, others are totally automated. To enable these bots to be compatible with the complexity of human communication and the capacity to recognize cultural subtleties, the AI frameworks need to be trained with a lot of data [5].Chatbots can offer company, assistance, and treatment, which significantly lessens the workload for therapists. It presents itself as a practical choice for those who struggle with accessibility and cost.

2 Related Work

Tech companies around the world are combining the power of artificial intelligence with the portability of smartphones to create chatbots designed to assist patients with mental health issues. These conversational agents assist patients while maintaining a high level of privacy and anonymity.

While some chatbots, like Ellie [6], can detect minor changes in our facial expressions, speech speeds, or length of pauses and create a diagnosis accordingly, Woebot is a completely automated conversational robot that treats depression and anxiety using a digital version of cognitive behavior therapy (CBT) [7]. The patient is given the option to meet with a real therapist if a serious issue is found, and pertinent hotline numbers are offered [8].

A chatbot that can speak in the voice of the departed person can help a person who is grieving after the loss of a close relative [9]. Messages are delivered over time that aid the recipient in erasing the trauma brought on by an unexpected loss and the lack of closure. In a fantastic advertisement for the Bixby Voice Assistant [10], Samsung's #VoiceForever, this idea has been thoroughly explored. The commercial features a mother's voice working in tandem with a voice assistant to assist a young girl in coming to terms with the death of her mother [11].

The cognitive-behavioral therapy model (CBT) serves as the foundation for chatbots that address mental health [12]. A step-by-step software manual or chatbot that employs CBT can help people examine and alter their mental patterns by using structured activities [13]. The management of mental health issues can benefit from prompt chatbot interventions with patients [14]. This is mostly accomplished by encouraging patients to transform their negative ideas into positive ones by utilizing clinical skills and natural language processing. For the end-user, this serves to provide a cathartic and healing experience. The Silicon Valley start-up X2AI created a chatbot named Karim that speaks Arabic and assists Syrian refugees with their mental health difficulties [15].

In [16], Cameron et al. have created a chatbot interface, wherein the chatbot initiates a conversation by asking how the user is feeling, and the user can respond using any one of the 'emojis' provided. Then, the user can pick from a set of issues that they may be facing and according to that, tips are given as a response.

The use of neural networks for cohesive detection of specific and technical mental health categorizations isn't provided.

We have use an artificial neural network system for the conversational chatbot to generate and carry forward textual conversations according to emotion and context of user, and parallel to that a systemic classification model runs on the user's responses to detect what problems that the user is facing and generate appropriate responses for that. In [17], Grové C has created a conversational chatbot for user interaction, wherein intent of the user is appropriated using machine learning, and appropriate responses are given out according to this. A risk categorization is done according to level of risk the youth may be in, and a trigger to alarm system is present for high risk situations. Some more application developments using chatbots are found in [18–21].

A categorization model to detect the specific mental health problem the user may be facing isn't present. We have developed a cohesive mental health and empathetic conversational chatbot, along with this we also categorize the specific and medical term of mental health issue that the user is facing using a classification model and give appropriate responses according to this as well.

2.1 Proposed Method

The proposed method emphasizes on mainly three parts: Namely (a) Getting user input as input data through a conversational chatbot model. (b) Classification of

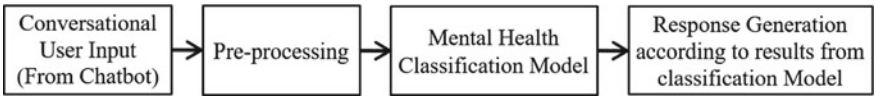


Fig. 1 General overview of the proposed method

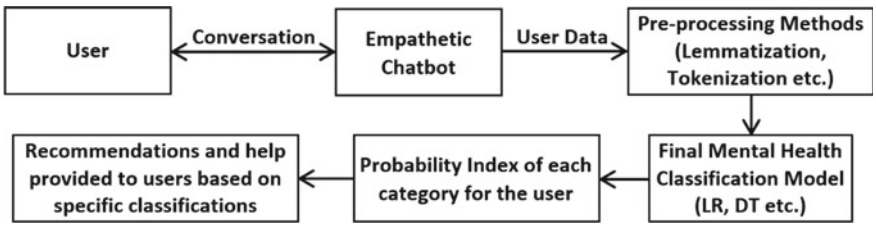


Fig. 2 Architecture of the proposed framework

various mental health diseases based on the previous input using state-of-the-art classification models. (c) Generating appropriate responses for the user based on the output of the previous models as shown in Fig. 1. In general, chatbots either directly classify the diseases without conversing with the user or just has normal conversation without helping the user with its issues. The idea behind the proposed work is creating a well-balanced chatbot which gives the user solutions to its mental health issues while conversing with the user to give them a platform the user can only get while conversing with a professional. Considering the lack of data in this domain, the dataset was scraped and manually extracted from Reddit as well as various surveys and websites present on the internet between patients and therapists. Mental health being a sensitive topic, the responses generated were thoroughly surveyed and validated by professionals to maintain a user-friendly environment while conversing with the chatbot. The architecture of the proposed framework is shown in Fig. 2. The proposed method is divided into three components as:

- Conversational Model
- Classification Model
- Response Generation.

2.2 Conversational Model

A specific and empathetic chatbot is created using mental health and therapist related data, which allows seamless and on topic conversations with user entailed, the chatbot

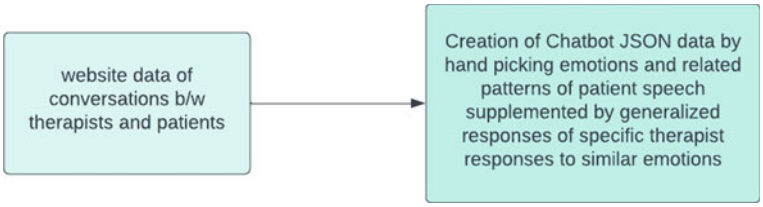


Fig. 3 Chatbot dataset

allows a healthy and beneficial conversation with a potential patient while simultaneously gathering the user data for further processing and classification. The parts of the framework are as follows:

- Dataset Description
- Model.

Dataset Description. For the dataset, we have scraped authentic conversation data between therapists and patients publicly available websites. The data is then categorized manually to tag specific emotions and their specific contexts to allow for proper response generation and efficient training. The responses encoded to be given out for each emotion tag is curated to be a general and appropriate response, gathered from authentic therapist responses. The Dataset contains patterns, specific emotion tags specific to them and appropriate responses for them, encoded in JSON format. The tags contained are: "greetings", "goodbye", "self-esteem-A", "self-esteem-B", "relationship-a", "relationship-b", "angermanagement-a", "angermanagement-b", "domesticviolence", "griefandloss", "substanceabuse-a", "substanceabuse-b", "familyconflict The conversation dataset diagram is shown in Fig. 3.

Model. For training and creation of conversational chatbot model, we have used an Artificial Neural Network structure. An ANN is a collection of layers each having a specific number of nodes, each ascertaining to a mathematical function/formula allowing for cohesive training for creating models. The model consists of an input layer, 1 hidden layer, and an output layer along with Dropout Regularization for the layers. We have also used regularization to tackle the problem of overfitting and underfitting while training the data. For regularization, we have used Dropout regularization which allows for random disablement of some nodes in the respective layers allowing for a more distributed and proper training mechanism. Dropout allows for more efficient training process due to the removal of noise carried on in the layers while training.

2.3 Classification Model

Classification of various mental health diseases such as depression, stress, anxiety, bipolar disorder, and personality disorder is done using state-of-the-art models such as Logistic Regression (LR), Decision Tree (DT), Random Forest (RF), and Multinomial Naïve Bayes (NB). The dataset is extracted from Reddit and is preprocessed using various preprocessing techniques. The stages of the framework are:

- Dataset Description
- Data Cleaning and Exploration
- Models.

Dataset Description. The data is collected by scraping Reddit using Reddit API. It contains information regarding 5 diseases such as depression, stress, anxiety, bipolar disorder, and personality disorder. It mainly has 3 columns: Namely, title, post text and target value. It contains 1818 documents(rows) for each said disease with the three features(columns) mentioned. The data was collected from the filtered subsection of the disease subreddit which are the most viable according to the users (hot).The dataset was split into training and testing in the ratio 4:1. The classification model dataset is shown in Fig. 4.

Data Cleaning and Exploration. The dataset being directly scraped and extracted from Reddit needs to undergo various preprocessing techniques. The data is thoroughly cleaned to remove any kind of noise in the dataset. Duplicity is avoided to remove any noise which could hurt the classification models. Null values occurred in the text feature; this occurs when a user on Reddit decides to use only the title feature. Since the title feature are an important part of the dataset, the null values were not dropped instead were filled with unique text. Furthermore, the data was visualized into a word cloud with the top 100 words in all 5 subreddits to give a better understanding of the dataset (Fig. 5).

Models. The classification of the diseases was done using various machine learning classifiers. Count vectorizer was used for feature engineering to convert the collection of text document i.e., the rows of the dataset into a matrix containing token counts. Various other feature engineering techniques were used such as stop word removal, stripping accents along with a n-gram model with a fixed threshold frequency for the document. The subsequent classification models that were used are:

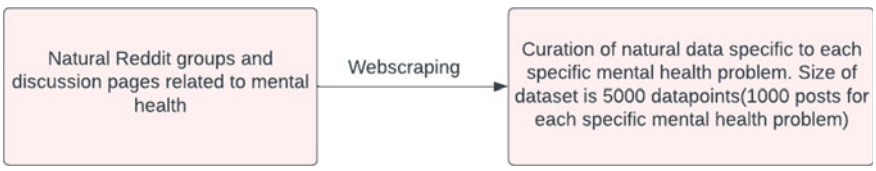


Fig. 4 Classification dataset

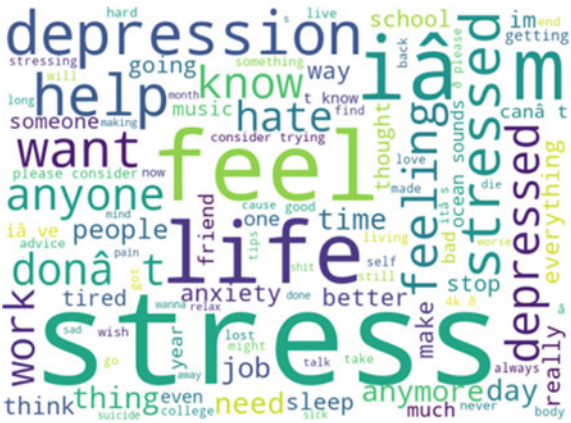


Fig. 5 Sample word cloud

Logistic Regression. Logistic Regression is used for the prediction of a binary event occurring. Two LR models were performed with different parameters. The first model and second model both had class_weight as balanced, warm_state as true, and solver as liblinear. The first model had 'C' value set as 1 with penalty set as 'l1' whereas the second model had 'C' value set as 2.5 with penalty set as 'l2'. The first model showed better results in comparison to the second model.

Decision Tree. Decision Tree is a classifier that enables you to make decisions according to some process. Like LR, two DT models were performed with the common parameter max_depth. The distinction between the two models was the criterion, with one model focusing on gini and the other on entropy. Both the models showed almost identical results.

Random Forest. Random forest is a collection of decision trees known as "forest" which used for both classification and regression. The two models which were performed had max_depth, bootstrap, warm_start, max_features, n_estimators and as the common features. Similar to the DT, the distinction between the two models was the criterion. The second model showed better results than the first model.

Multinomial Naïve Bayes. Multinomial Naïve Bayes (NB) is a classifier used for classification of datasets with discrete features. The two variations of the models had fit_prior hyperparameter as the distinction where it was set as "true" for the first model and "false" for the second model. The second model showed comprehensively better results than the first model.

2.4 Response Generation

While conversing with the user, the conversational model gives out specific, friendly and knowledgeable responses. These responses are specific to the emotion and

Table 1 Performance metrics obtained for the models used

	Train Accuracy	Test Accuracy
LR	96.2121	88.6486
DT	85.1731	86.1621
RF	93.3982	89.4054
NB	88.3116	88.1081

context the user is speaking in. The response dataset is curated from and vetted by therapists dealing with these specific problems and allows for a friendly and medical environment during the conversation. When the conversation reaches it's end, the classification model is applied upon the users input and according to the specific mental health problems the user may be facing, informative and precautionary responses are given to users.

3 Experimental Analysis

This part of the paper deals with the overall performance of the different classifiers in the terms of metrics. The results obtained from each model has been determined systemically using GridSearchCV. Since, two models were performed for each classifier with different parameter, the model with the better results are taken for visualization.

The accuracy obtained for the models performed are presented in Table 1. The confusion matrixes obtained are presented in Fig. 6. The comparison among various classifiers based on accuracy is presented in Fig. 7. Random Forest classifier has performed the best among the classification models with an accuracy of 89.40%.

4 Discussion

The conversational chatbot provides for a empathetic and dynamic conversation environment allowing for seamless exchange of conversation between the entities. It asks and answers appropriate and leading questions to and from the user and allows for proper data gathering for diagnosis, while simultaneously conversing with the user in a safe environment. Among the classification models the Random Forest classifier has performed the best with the accuracy of 89.40%. Random Forest works best here due to its inherent mechanisms for working with missing datapoints in the dataset and for the variance and lack of standardization of the data used in the model. Logistic Regression works well on the data because it assumes that only highly meaningful features are included, and that data is linearly separable. Naïve Bayes works properly because it assumes independence of all variables and classes, and normally requires relatively lesser training data for better outputs. As the data is independent and highly

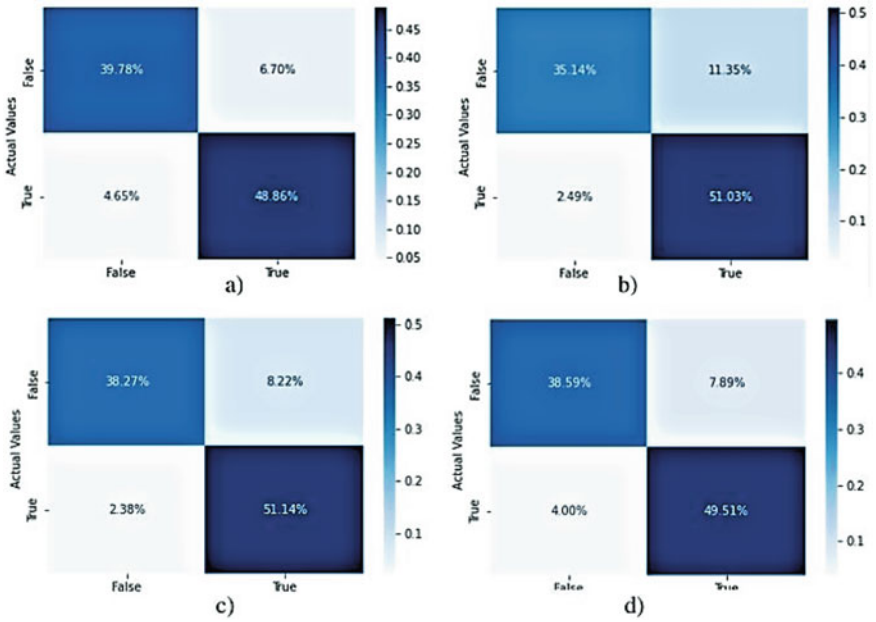


Fig. 6 Confusion matrices for LR, DT, RF, and NB in **a**, **b**, **c** and **d** respectively

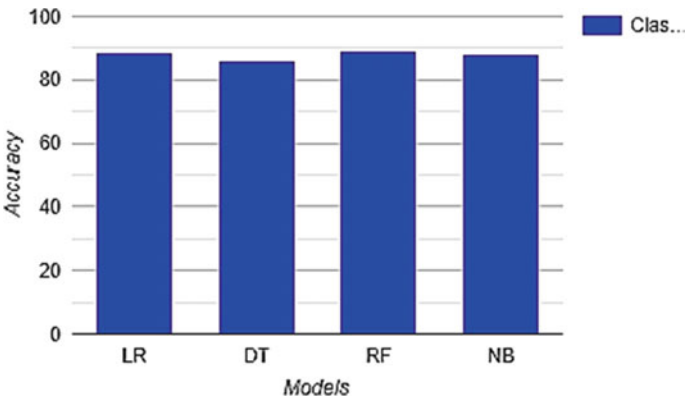


Fig. 7 Comparison graph of various models based on test accuracy

varied, Decision Tree model doesn't work properly here as all terms are assumed to interact and independency of variables isn't tackled in the mechanism. All the models performed have approximately the same accuracy with slight deviations due to the above-mentioned reasons. The application for this comprehensive model is manyfold and can be used in multiple areas such as an application software, for research surveys and studies, as a medical tool for therapists and professionals etc.

5 Conclusion and Future Scope

In conclusion, we have created a multifunctioning and cohesive system for detection, prevention and support related to available mental health diseases under study. We have created a chatbot for conversing appropriately with the user in a very specific and medical manner, and enabled classification of the user's input data on whether they may be affected by a mental health disease or not. Regarding the future of the paper, the following recommendations can be made (a) The chatbot model can be trained using NLU(Natural Language Understanding) Mechanisms to generate automatic responses instead of generalized ones, the model can be added with more conversation data to enable longer conversations. (b) Time varying analysis can also be used when conversing with and obtaining data from the user. The user data can be analyzed across time periods and time variance can be taken as a factor while training and creation of classification model. (c) Training data can be scraped in a corresponding manner. (d) A Dynamic classification model can be created which will automatically extract the reddit data and feed it in the training systems allowing for a continuously updated model. (e) More Reddit features, like upvotes, downvotes, shares, awards etc. can be also taken into consideration while training the data.

References

1. World Health Organization (2005) Department of Mental Health, Substance Abuse, World Health Organization, World Health Organization. Department of Mental Health, Substance Abuse. Mental Health, World Health Organization. Mental Health Evidence, and Research Team. Mental health atlas 2005. World Health Organization
2. Reddy V (2019) Mental health issues and challenges in India: a review. Int J Soc Sci Manag Entrepr (IJSSME) 3(2)
3. Katschnig H (2010) Are psychiatrists an endangered species? Observations on internal and external challenges to the profession. World Psychiatr 9(1):21
4. Adamopoulou E, Lefteris M (2020) Chatbots: history, technology, and applications. Mach Learn Appl 2:100006
5. Brandtzaeg PB, Følstad A (2017) Why people use chatbots. In: International conference on internet science. Springer, Cham, pp 377–392
6. Kim H, Hyejin Y, Dongkwang S, Jang HL (2022) Design principles and architecture of a second language learning chatbot. Lang Learn Technol 26(1):1–18
7. Fitzpatrick KK, Alison D, Molly V (2017) Delivering cognitive behavior therapy to young adults with symptoms of depression and anxiety using a fully automated conversational agent (Woebot): a randomized controlled trial. JMIR Mental Health 4(2):e7785
8. Darcy A, Aaron B, Emil C, Jade D, Kim G, Timothy YM, Paul W, Athena R (2022) Anatomy of a Woebot® (WB001): agent guided CBT for women with postpartum depression. Expert Rev Med Devices 19(4):287–301
9. Tracey P, Mo S, Chris H (2021) Applying NLP to build a cold reading chatbot. In: 2021 international symposium on electrical, electronics and information engineering, pp. 77–80
10. Nobles AL, Eric CL, Theodore LC, Zhu SH, Steffanie AS, John WA (2020) Responses to addiction help-seeking from Alexa, Siri, Google Assistant, Cortana, and Bixby intelligent virtual assistants. NPJ Digit Med 3(1):1–3

11. Narmadha V, Ajay Krishnan JU, Praveen Kumar R, Ram Kumar R (2019) Telepathic virtual assistant. In: 2019 3rd international conference ICCCT. IEEE, pp 321–325
12. Fenn K, Majella B (2013) The key principles of cognitive behavioural therapy. InnovAiT 6(9):579–585
13. Sulaiman S, Marzita M, Rohaizah AW, Nur A, Alisa NA (2022) Anxiety assistance mobile apps chatbot using cognitive behavioural therapy. Int J Artif Intell 9(1):17–23
14. Gabrielli S, Silvia R, Sara C, Valeria D (2020) A chatbot-based coaching intervention for adolescents to promote life skills: pilot study. JMIR Hum Fact 7(1):e16762
15. Sekkat K, Nassim EH, Siham EA, Asmaa C (2021) Designing a chatbot for young diabetics: Unprecedented experiment at the endocrinology service of the casablanca university hospital. In: Endocrine abstracts, vol 73. Bioscientifica
16. Cameron G, Cameron D, Megaw G et al, Towards a chatbot for digital counselling. https://doi.org/10.14236/ewic/HCI2017.24
17. Grové C (2021) Co-developing a mental health and wellbeing chatbot with and for young people. Front Psychiatr 11:606041. https://doi.org/10.3389/fpsyt.2020.606041
18. Verma S, Singh M, Tiwari I, Tripathy BK (2022) An Approach to Medical Diagnosis Using Smart Chatbot. In: Das AK et al (eds) Proceedings of CIPR, CIPR 2022, LNNS 480, pp 1–14
19. Srividya V, Tripathy BK, Akhtar N, Katiyar A (2021) AgentG: an engaging bot to chat for e-commerce lovers. In: Tripathy A, Sarkar M, Sahoo J, Li KC, Chinara S (eds) Advances in distributed computing and machine learning. LNNS, vol 127. Springer, Singapore. https://doi.org/10.1007/978-981-15-4218-3_27
20. Srividya V, Tripathy BK, Akhtar N, Katiyar4 A (2020) AgentG: a user friendly and engaging bot to chat for e-commerce lovers. Comput Rev J 7:7–19
21. Bhattacharya S, Kushwaha A, Banu SK, Tripathy BK (2022) Customizing backend logic using a chatbot. In: Pandit M, Gaur MK, Rana PS, Tiwari A (eds) Artificial intelligence and sustainable computing. algorithms for intelligent systems. Springer, Singapore. https://doi.org/10.1007/978-981-19-1653-3_43

SincSquareNet: Deep Neural Network-Based Speaker Identification for Raw Speech



Banala Saritha, K. Anish Monsley, Rabul Hussain Laskar, and Madhuchhanda Choudhury

Abstract Biometrics, forensics, and access control systems such as speaker identification systems all benefit from advances in machine learning and deep learning. A distinctive convolutional neural network is SincNet architecture that performs speaker identification by feeding one-dimensional raw audio into the first convolutional layer, which is made up of parameterized sinc filters. We proposed a SincSquareNet for speaker identification in this paper, in which a CNN effectively learns tailored triangular bandpass filters using trainable sinc-squared functions. Only the low and high cutoff frequencies of bandpass filters are directly learned from data, unlike typical CNNs, which are able to learn the features of each filter. The proposed model is tested on the LibriSpeech dataset, and it outperforms SincNet in terms of performance. According to experimental results, identification accuracy is increased by 3.5%, while validation loss is reduced by 12% when compared to SincNet.

Keywords Speaker identification · Convolutional neural networks · Triangular band pass filters

1 Introduction

In the fields of biometric authentication, forensics, and access control systems, as well as speech identification, speaker identification is a very active research area. Speaker identification is the technique of distinguishing an unknown speaker from a set of known speakers [1]. Deep learning-based speaker identification has gradually

B. Saritha (✉) · K. Anish Monsley · R. Hussain Laskar · M. Choudhury
National Institute of Technology Silchar, Assam 788010, India
e-mail: banalasarita@gmail.com

R. Hussain Laskar
e-mail: rhlaskar@ece.nits.ac.in

M. Choudhury
e-mail: m_chhanda@ece.nits.ac.in

© The Author(s), under exclusive license to Springer Nature Singapore Pte Ltd. 2023

S. Chinara et al. (eds.), *Advances in Distributed Computing and Machine Learning*,
Lecture Notes in Networks and Systems 660,
https://doi.org/10.1007/978-981-99-1203-2_40

overtaken i-vector-based speaker identification. Even though many state-of-the-art systems still use conventional features like filterbank and Mel-Frequency Cepstral Coefficients (MFCC), directly feeding a CNN with raw audio samples is becoming more prevalent. Convolutional Neural Networks (CNNs) use raw waveforms to learn low-level speech representations [2]. CNNs are the most often used framework for interpreting raw speech, because of shared weights, filters, and pooling techniques enable reliable and consistent interpretations.

Deep learning approaches, despite having a higher computing cost for training, produce better results than standard methods. The SincNet [3] model is a viable deep learning-based method for speaker identification. The first convolutional layer accepts one-dimensional raw audio samples as input and convolves with a collection of parameterized sinc functions that provide bandpass filters to extract basic low-level features. The only filter parameters learned from the data are the lower and higher cutoff frequencies. The deeper layers of the network then process the low-level features further.

In this study, we proposed SincSquareNet, shown in Fig. 1, as a new method for speaker identification that is heavily influenced by the SincNet design. On the Librispeech datasets [4], speaker identification is performed under realistic conditions with test utterances duration of about 2–6 s. The proposed SincSquareNet performs competitively with SincNet and converges faster, provides greater accuracy, and is better interpretable than a traditional CNN.

The paper is organized into various sections. The SincSquareNet architecture is briefly described in Sect. 2. Related work is presented in Sect. 3. The experimental design is explained in Sect. 4, along with the results. We reached our conclusions in Sect. 5 at the end.

2 The Sincsquarenet Architecture

A standard CNN's initial convolutional layer takes one-dimensional raw audio as input and convolves it with a set of finite impulse response filters [5]. The following are the definitions for each convolution:

$$y[n] = x[n] * h\,[n] = \sum_{m=0}^{K-1} x[m]h[n-m] \tag{1}$$

where x[n] denotes a speech signal chunk, h[n] indicates the filter of K-length, and the filtered output is provided by y[n]. In the case of typical CNNs, each filter has K elements that are learnt through data. On the other hand, the proposed SincSquareNet executes convolution with a predetermined function t that is only dependent on a few variables that can be learned θ, as illustrated by the equation below:

$$y[n] = x[n] * t\,[n, \theta] \tag{2}$$

To define t in a manner that can be employed with a triangular bandpass filter bank is a feasible solution that is motivated by digital signal processing filtering. The magnitude of a generic bandpass filter can be expressed as the difference between two low-pass filters in the frequency domain:

$$T\left[f, f_1, f_2\right] = tri(f/2f_2) - tri(f/2f_1) \tag{3}$$

where f_1 and f_2 are the learned lower and higher cutoff frequencies, tri (.) is the triangular function in the magnitude frequency domain. Below is the reference function when employing the inverse Fourier transform and switching to the temporal domain [5].

$$t\left[n, f_1, f_2\right] = 2f_2 sinc^2\left(2\pi f_2 n\right) - 2f_1 sinc^2\left(2\pi f_1 n\right) \tag{4}$$

where the sincsquare function is defined as $sinc^2(x) = sin^2(x)/(x^2)$.

The cutoff frequencies f_1, f_2 can be chosen randomly in the range [0,f(s)/2] where f(s) is the input signal's sampling frequency. Filters can also be initialized with the mel-scale filter banks cutoff frequencies, which have the benefit of explicitly assigning extra filters in the lower portion of the spectrum, where several critical hints about the speaker's identity can obtain. The following parameters are put into the preceding equation to achieve $f_1 \geq 0$ and $f_2 \geq f_1$.

$$\bar{f}_1 = |f_1| \tag{5}$$

$$\bar{f}_2 = f_1 + |f_2 - f_1| \tag{6}$$

$$f_2^{abs} = f_1 + |f_2 - f_1| \tag{7}$$

There are no bounds on frequency f_2 that restrict it to be smaller than the Nyquist frequency. This constraint is handled in a natural way during training. Also, the convolutional layer can adjust these frequencies before applying them to further standard layers. Furthermore, at this level, the gain of every filter is not learned. We can see from the above equations that SincLayer aimed to learn the cutoff frequencies. The gains for each sinc filter are then learned by applying appropriate weights to convolutional and fully connected layers. Hamming windowing $w_H(n)$ softened the sharp discontinuities in the sinc filter by minimizing passband ripples [6]. As a result, we have

$$t_{wH}\left[n, \bar{f}_1, \bar{f}_2\right] = t\left[n, f_1, f_2\right].w_H[n] \tag{8}$$

where the Hamming window of length K is defined as

$$w_H[n] = 0.54 - 0.46cos(2\pi n/K) \tag{9}$$

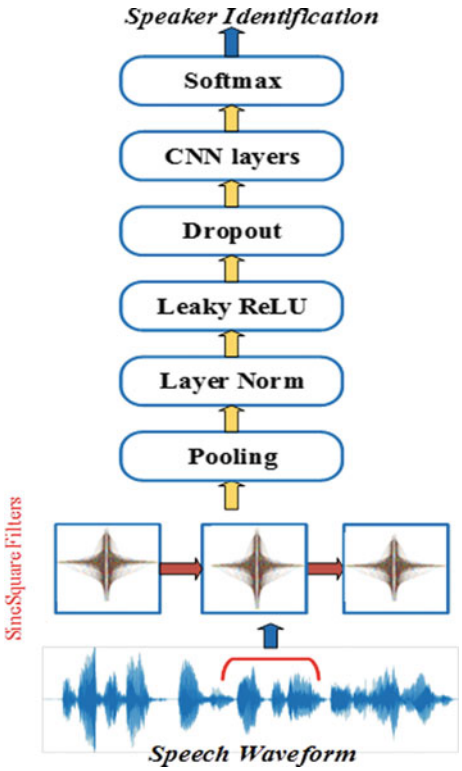


Fig. 1 Architecture of SincSquareNet

Figure 2 shows examples of learned filters by standard CNN, ConstantSincSquareNet, and the SincsquareNet training on the Librispeech dataset. The figures show that the standard CNN doesn't train filters with a specific frequency response. The frequency response of filters is drawn at a frequency 0 Hz–4 kHz. In a few instances, the frequency response appears chaotic in the initial CNN filter, whereas, in other instances, the third CNN filter assumes multi-band patterns. SincsquareNet, on the other hand, is intended mainly to design triangular bandpass filters, leading to a more effective filter bank. Due to advantages like quick convergence, adaptability, compact architecture (few parameters), and computing economy (symmetric sincsquare functions), SincsquareNet is attractive for speaker modeling.

3 Related Work

Recent research has examined how low-level speech representations can be used to analyze speech using CNNs. The majority of earlier approaches make use of spectrogram characteristics [7–9]. The construction of spectrograms still demands

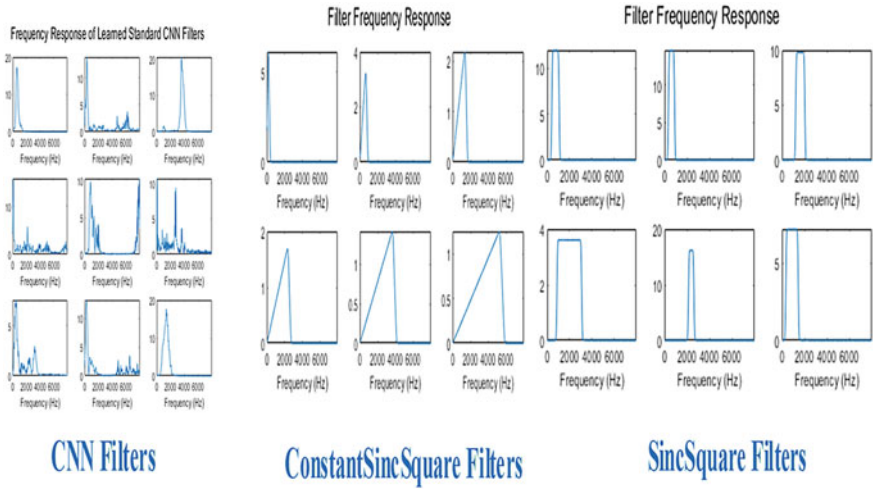


Fig. 2 Illustration of learned filters by standard CNN, ConstantSincSquare, and SincSquare filters

careful tweaking of several essential hyper-parameters, despite preserving more content than conventional features. In light of this, a much more emerging technique is to acquire features directly from raw speech waveform. Parallel to SincNet, a few earlier studies suggested imposing restrictions on the CNN filters, such as requiring filters to operate only on particular bands [9, 10]. The proposed work was motivated by the filtering technique used in signal processing and the SincNet concept. This work demonstrates the efficacy of the proposed sincsquare filters for processing raw speech waveforms for the first time. The filters trained by SincSquareNet are ideally suited for the speaker identification task.

4 Experimental Setup & Results

4.1 Dataset

The proposed method SincSquareNet is evaluated using the well-known Librispeech dataset, which features 21933 samples from 2484 classes/unique speakers and around 1000 h hours of reading English speech at a sampling rate of 16 kHz. We utilized the identical data that the authors used in their experiment [3, 11]. Internal silences of more than 125 milliseconds in Librispeech utterances were separated into multiple chunks. Preprocessing is performed by normalizing the amplitude, and non-speech durations were excluded from the beginning and end of each sentence.

4.2 Experimental Configurations

Each speech sentence was segmented into 200 ms slices (with a 10 milliseconds overlap) and provided to the SincsquareNet architecture as input. The initial convolutional layer includes 80 filters with a length(L) of 251 samples to perform sincsquare-based convolutions. Following that, the architecture utilizes two conventional convolutional layers, each with 60 filters of length 5. For both the input samples and all convolutional layers, layer normalization [12] was applied (including the SincsquareNet input layer). Following that, three fully connected layers with a total of 2048 neurons were added and normalized using batch normalization [13]. Leaky-ReLU non-linearities are used in all hidden layers.

The sincsquare layer parameters are defined using mel-scale cutoff frequencies, and the popular "Glorot" initialization [14] was utilized to build the remainder of the model. A softmax classifier was used to generate frame-level speaker classification, which provided posterior probabilities for the chosen speakers. The frame predictions were averaged, and the speaker whose speech has the best average posterior was chosen to produce a sentence-level categorization. Training was done using the Adam optimizer, using learning rates of $lr = 0{:}001$, $\alpha = 0.95$, $\epsilon = 10^{-5}$, and 128 mini-batches. All of the architecture's hyper-parameters were adjusted on Librispeech. This research was conducted using MATLAB on a workstation configured with an Intel Xeon Gold 6254 processor, 128 GB of RAM, and an NVIDIA Quadro P5000 graphics card (16 GB GPU).

Three convolutional neural networks (CNNs) are trained in this study to accomplish speaker identification and analyze the findings. Except for the first convolutional layer in each, the architectures of the three CNNs are identical.

4.3 Standard CNN

In the first design, first convolutional layer is a "normal" convolutional layer created with convolution2dLayer. The direct connection between the input waveform and a convolutional layer that attempts to learn features and is initiated randomly extract information from the raw audio samples.

4.4 ConstantSincSquareNet

As in the second design, a constant sincsquare filter bank that is constructed using a custom layer serves as the first convolutional layer. The raw waveform is combined with a set of sincsquare functions with specified widths. Bandpass triangular filters are a time-domain linear pair of two sincsquare filters. On the mel scale, these bandpass filters' frequencies are equally spaced.

4.5 SincSquareNet

The first convolutional layer of the third design is a trainable sincsquare filter bank, which is implemented using a customized layer. SincsquareNet is the name given to this architecture. The network convolved the input waveform with a collection of sincsquare functions. At the same time, the network learns the parameters, filter frequencies, and filter bandwidths. During training, the SincsquareNet architecture tunes the parameters of the sincsquare functions. In this paper, the three suggested designs are trained, and their effectiveness is evaluated on Librispeech. Table 1 provides a summary of the frame accuracy for these architectures.

Table 1 shows that SincSquareNet outperforms ConstantSquareSincNet and traditional CNN in terms of accuracy during validation, training, and testing. At the same time, the training time is comparable with the others.

Figures 3 and 4 present a bar graph of the values from Table 1 for easy analysis. The final validation accuracy, average training accuracy, and testing accuracy are illustrated in Fig. 3. All accuracy is expressed as a percentage. Figure 4 compares the training loss to the final validation loss.

The training loss and the final validation loss are compared in Fig. 4. SincSquareNet's final validation loss is 0.7602 compared to ConstantSincSquareNet's (0.8466) and Standard CNN architecture's (0.9572), indicating a loss reduction of around 20%.

Figure 5 depicts the frame-level accuracy of the test set for 15 epochs. This plot determines how effectively the networks train as the number of epochs rises. Standard CNN exhibited an average testing accuracy of 73.58% in identifying the speaker, while ConstantSincSquareNet and SincSquareNet both outperformed SincNet by 2.4% and 3.3%, respectively. These details are listed in Table 2.

Table 1 Comparative analysis of three proposed architectures. Bold font indicates best result

Parameters	Standard CNN	ConstantSinc-SquareNet	SincSquareNet
FinalValidationAccuracy(%)	73.58	77.83	**81.69**
FinalValidationLoss	0.9572	0.8466	**0.7602**
AverageTrainingAccuracy(%)	75.78	79.96	**82.23**
AverageTrainingLoss	0.8491	0.7581	**0.6732**
TestingAccuracy(%)	73.58	77.83	**81.69**
Total Time for Training	445 min	412 min	**467 min**

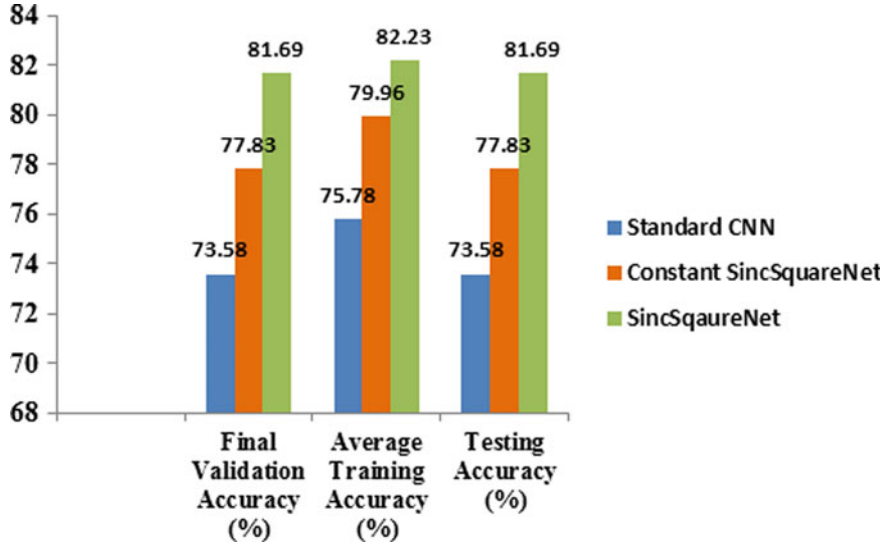


Fig. 3 Comparison of accuracy value with proposed architectures

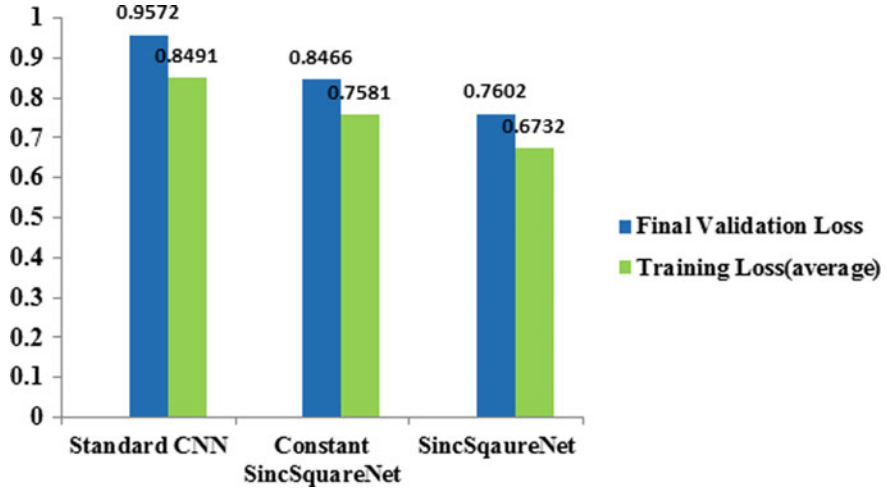


Fig. 4 Training loss and validation loss comparison for the proposed architectures

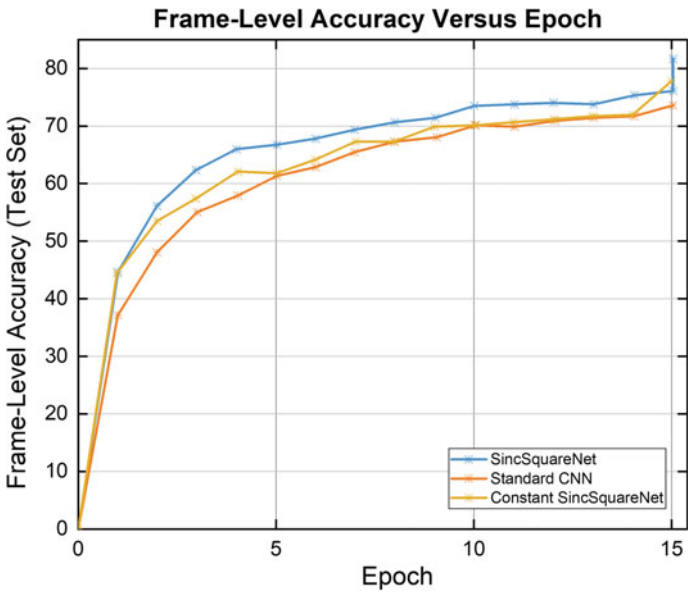


Fig. 5 Comparison of proposed architectures in terms of accuracy

Table 2 Comparative analysis of three proposed architectures. Bold font indicates best result

S.No	Network type	Accuracy
1	Standard CNN	73.58
2	ConstantSincNet/ConstantSincSquareNet	75.45/**77.83**
3	SincNet/SincSquareNet	78.39/**81.69**

5 Conclusion

In this research, we introduced SincSquareNet, a neural architecture for efficiently processing the speech signal to identify the speaker. Our method, which was motivated by the filtering technique used in signal processing, constrained the filter patterns by appropriate parameter characterization. The SincSquareNet proposed in this research encourages the development of deep convolutional layers to learn configurable triangular bandpass filters using trainable sincsquare functions. It directly learns the low and high cutoff frequencies of each triangular bandpass filter, unlike typical CNNs. SincSquareNet outperforms conventional CNN and SincNet. The Librispeech dataset was used to evaluate this work, and it is clear that compared to traditional CNN, it improved Speaker Identification accuracy by 9% and diminished validation loss by 20%. According to the results, identification accuracy is enhanced

by 3.5%, while validation loss dropped by 12% compared to SincNet. The objective of the subsequent study was to assess SincSquareNet's performance on the VoxCeleb database and also investigate the effectiveness of various windows and frameshifts on the SincSquareNet layer.

References

1. Laskar MA, Laskar RH (2019) Integrating DNN-HMM technique with hierarchical multi-layer acoustic model for text-dependent speaker verification. Circuits Syst Signal Process 38(8):3548–3572 Aug
2. Jung J, Heo H, Yang I, Yoon S, Shim H, Yu H (2018) D-vector based speaker verification system using Raw Waveform CNN
3. Ravanelli M, Bengio Y (2018) Speaker recognition from raw waveform with SincNet
4. Panayotov V, Chen G, Povey D, Khudanpur S (2015) Librispeech: an ASR corpus based on public domain audio books. In: ICASSP, international conference on acoustics, speech, & signal processing - Proceedings, vol 2015, pp 5206–5210
5. Rabiner LR, Schafer RW (2007) The essence of knowledge introduction to digital speech processing introduction to digital speech processing foundations and trends® in signal processing introduction to digital speech processing, vol 1, Issue 12
6. Saritha B, Shome N, Laskar RH, Choudhury M (2022) Enhancement in speaker recognition using sincnet through optimal window and frame shift, pp 1–6
7. Muckenhirn H, -Doss MM, Marcel S (2023) Towards directly modeling raw speech signal for speaker verification using CNNS
8. Meng H, Yan T, Yuan F, Wei H (2019) Speech emotion recognition from 3D Log-Mel spectrograms with deep learning network. IEEE Access 7:125868–125881
9. Sainath TN, Weiss RJ, Senior A, Wilson KW (2023) Learning the speech front-end with raw waveform CLDNNs
10. Yu H, Tan ZH, Zhang Y, Ma Z, Guo J (2017) DNN filter bank cepstral coefficients for spoofing detection. IEEE Access 5:4779–4787
11. Ravanelli M, Bengio Y (2018) Interpretable convolutional filters with SincNet. no. Nips
12. Ba JL, Kiros JR, Hinton GE (2016) Layer normalization
13. Ioffe S, Szegedy C (2015) Batch normalization: accelerating deep network training by reducing internal covariate shift. In: 32nd international conference on machine learning ICML 2015, vol 1, pp 448–456
14. Glorot X, Bengio Y (2010) Understanding the difficulty of training deep feedforward neural networks. J Mach Learn Res 9:249–256

RSSI-Based Hybrid Approach for Range-Free Localization Using SA-PSO Optimization



Maheshwari Niranjan and Buddha Singh

Abstract For many Wireless Sensor Network (WSN) applications, the localization of sensor nodes has become crucial. Among all the range-based positioning approaches, received signal strength indication (RSSI) could provide accurate location while preserving hardware, but it is susceptible to background noise. Furthermore, the fundamental range-free positioning approach known as the classic distance vector hop (DV-Hop), is straightforward, affordable, and unaffected by environmental variables, but it performs poorly in the absence of Nodes with Known Position (NKP). This work introduces SAPSODV-Hop, a revolutionary node localization technique based on "Simulated Annealing" (SA) and "Particle Swarm Optimization" (PSO). In this approach, "RSSI" based distance value is used at one hop (within the communication ring) of NKP. To further refine hop size, the weight factor is introduced. The simulation results confirm that, when compared to the existing localization schemes, our proposed approach reduces localization error (LE) and increases localization accuracy (LA).

Keywords WSN · Localization · DV-Hop · SA · PSO · LE

1 Introduction

To boost WSN performance, we require efficient localization methods. Using the Global Positioning System (GPS), the sensor nodes can be precisely localized [1], but due to external hardware, high cost and indoor localization makes this method less efficient in large scale WSNs. The sensor nodes randomly distributed in WSN, and small number of NKPs, which are GPS-equipped, can retrace their positions. The additional nodes, known as Nodes with Unknown Position (NUP), do not,

M. Niranjan (✉) · B. Singh
SC&SS, Jawaharlal Nehru University, New Delhi 110067, India
e-mail: niranjan.maheshwari7@gmail.com

B. Singh
e-mail: b.singh.jnu@gmail.com

© The Author(s), under exclusive license to Springer Nature Singapore Pte Ltd. 2023

S. Chinara et al. (eds.), *Advances in Distributed Computing and Machine Learning*,
Lecture Notes in Networks and Systems 660,
https://doi.org/10.1007/978-981-99-1203-2_41

however, capture their positions. The NKP typically assists those NUP by utilizing connectivity information between nodes and also uses multi-hop routing to exchange messages to locate themselves. There are two node positioning technique: range free and range based. The "RSSI" technique comes under the category of range-based positioning approach [3]. Due to its inexpensive price [4, 6], RSSI is a more preferable option, and it has been extensively employed for device-free sensor node localization. However, "RSSI" is susceptible to external noise [7] and could result in a reduction in localization precision. The latter, however, is easier and less expensive because it merely requires connectivity [8]. A few of the simple, heuristic range-free localization methods that are now available could be applied in a distributed setting. Furthermore, due to its ease of implementation, the traditional range-free positioning approach, known as "DV-Hop" [9], is a desirable option in a setting with low hardware assistance. Several methods were presented by combining "RSSI" and "DV-Hop" to carry out the positioning for NUP, utilizing both the "range-based" and "range-free" approaches [5–7, 9]. The localization inaccuracy of the NUP and NKP could be effectively decreased in this method. Nevertheless, there are some circumstances in which the coordinate calculation may still be inaccurate [11].

2 Related Work

This paper primarily focuses on the "DV-Hop" algorithm, a well-liked positioning algorithm among the scholars. The success of this approach can be attributed to its straightforward structure, strong coverage, affordability, and low energy consumption in WSN systems with limited resources. However, this technique has a few drawbacks, including poor localization accuracy and a significant localization inaccuracy. This large localization error is primarily due to inaccurate distance calculations between the NUP and NKP. In DV-Hop, distance is determined by multiplying the NKP hop-size value by the number of hops between NUP and NKPs. The nodes communication rings are used to determine this hop count number, and the hop size is estimated by averaging the actual distances and hop counts between the NKPs. This hop-size parameter is incorrect and causes a significant amount of distance calculation mistakes. On the other side, the major difficult problem in the "DV-Hop" approach is to optimize the predicted location of NUP by choosing the appropriate optimization technique. Researchers have presented numerous enhanced versions of the real "DV-Hop" algorithm by solving the aforementioned problems. The main focus in many of them was to enhance the hop-size calculation method and the selection of best optimization strategy to increase the localization accuracy.

2.1 Traditional "DV-Hop" Approach

The "DV-Hop" is utilized as a common "range-free" positioning approach. The distance between the NUP and the NKPs is calculated by multiplying the average hop size by the hop count, and the crucial aspect of the "DV-Hop" scheme is that the positions of the NUP are estimated using the tri-literation/multi-literation approaches [16]. The minimal hop count for an NKP is selected by each NUP, and the NKP then calculate the moderate distance per hop by using the minimal hop value and the distances measured between itself and all other NKPs [17]. Although environmental elements like landscape and climate have no impact on the range measuring method of "DV-Hop", the range measurement inaccuracy is significant if the distribution of the sensor nodes inside a WSN is unequal. Assuming, that a_1, a_2, and a_3 are NKPs, it is necessary to determine the locations of the NUP u_1, u_2, u_3, and u_4. These NKP actual distances are 40, 40, and 50, are known. Then, the hopsize for each NKP a_1, a_2, a_3 is 11.25, 10, and 11.43 respectively. Due to each NKP flooding its average hop distance throughout the network, Only the first number is provided to the NUP as the average hop size. Therefore, NUP u_1, there is just one hop from a_1 to u_1, thus u_1 simply records the average. The average hop size of a_1's, 11.25, is the hop distance. The distances to the other NKPs are then estimated by u_1 as follows (Fig. 1):

$$\begin{aligned} &\text{distance from } u_1 \text{to} a_1 \text{is } 11.25 \times 1 = 11.25, \\ &\text{distance from } u_1 \text{to} a_2 \text{is } 11.25 \times 4 = 45, \\ &\text{distance from} u_1 \text{to} a_3 \text{is } 11.25 \times 2 = 22.5. \end{aligned} \tag{1}$$

Given that the 17 is the exact distance between u1 and a1, not 11.25, Using the DV-Hop technique, it is clear that the range measurement inaccuracy is substantial. The considerable range inaccuracy will further raise the final position inaccuracy when using the triangle positioning technique or the greatest likelihood estimate strategy.

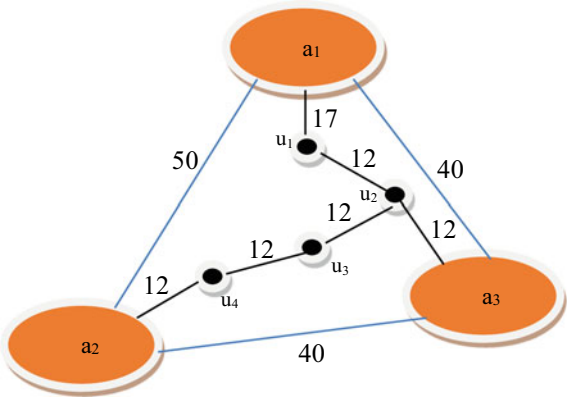


Fig. 1 Range error measurement

2.2 RSSI

RSSI is a viable alternative in WSN due to its simpler implementation and lack of additional hardware requirements [18]. However, noise and obstructions can readily alter RSSI, which can result in considerable estimation mistakes. In addition, in an uncertain environment, two sensor nodes can receive different RSSIs coming from the same source. There are now three categories for the RSSI signal propagation model for WSN: the free space model, the two-ray ground model and the log-normal shadow model [19]. The final signal transmission model is a more general one that accounts for signal intensity fading and is applicable in both indoor and outdoor settings. The previous two models apply to some exceptional instances. Then, using [20], it is possible to describe the received signal strength of sensor nodes for the log-normal shadow model.

$$\text{RSS}\ (d)(\ \text{dBm})\ = P_{TR} - P_{loss}(d_0) - 10\,\tau\,\log 10\frac{d}{d_0} + X_\sigma, \tag{2}$$

where, d represents the distance between the transmitter and receiver nodes, P_{TR} represents the transmitting power, The RSS value of sensors at d distance is represented by RSS(d), where d_0 is the reference distance. $P_{loss}(d_0)$ represents the path loss of the d_0; τ represents the loss exponent and X_σ represents the noise in RSSI.

3 Improved Hybrid Optimization Technique (SAPSO)

3.1 Simulated Annealing (SA) Algorithm

The SA is a well-liked physics-based meta-heuristic algorithm for global optimization mimicked by the annealing of metals. The basis of this algorithm is to use the Metropolis procedure to accept a higher energy state with probability. In this scheme, there are predefined parameters like initial temperature (T_0), cooling strategy ($T_t = \propto^t T_0$), and radius of neighboring states [12]. In this scheme, there is a single random starting point that can move downward or upward in the search space to find an appropriate neighboring point looking for a better solution. The final estimated point is greatly dependent on the randomly chosen initial point. Thereafter temperature reduction strategy is used to find the next approximated point around the neighborhood of the current point whose fitness is better than the current point. There is various temperature cooling strategy available in the literature, but the commonly used cooling function is $T_t = \propto^t T_0$ where T_0 is an initial temperature, $\propto^t$ is a positive constant usually taken between $0.8 \leq \propto \leq 0.9$, t is an iteration number, and T_t is a temperature in iteration no. t. SA is a very efficient approach to balance exploration/exploitation using a temperature reduction strategy. This means at the beginning of

the search temperature is high so this can provide better exploration and as the iteration goes on the temperature will be reduced, therefore at a low temperature, this can provide better exploitation for the already explored region at a high temperature.

3.2 The PSO Algorithm

The PSO is well-liked metaheuristic optimization method that was motivated by the amicable (social) behavior of a swarm of birds and schooling fish. This is first proposed by Eberhart and Kenned in [13]. This comes under the category of well-liked meta-heuristics algorithms and covers a vast set of optimization problems. As stated above, this is inspired by the grouping of birds moving together in the sky for finding food. While moving they have a group leader to accomplish their migration in a different direction to explore local as well as global space (to balance exploration/exploitation). PSO is well-liked by the researchers because of several benefits, if it is well tuned it has an acceptable computational cost to find a suitable global optimal solution, and also the computational cost of this algorithm is very less as it requires less mathematical computation [14].

PSO comprises a flock of n contender (candidate) solutions called particles, which explore a d-dimensional search space of the global solution. A particle p occupies position X_{pd} and velocity V_{pd} in the Dth dimension of the global search space, $1 \leq p \leq n$ and $1 \leq D \leq d$. In this scheme, each particle evaluates its fitness value (cost) using the fitness function which is based on the optimization problem. The fitness function $f(p_1, p_2, p_3, \ldots p_n)$ where $f : \mathbb{R}^d \rightarrow \mathbb{R}$ maybe minimize or maximize depending on the problem domain. In PSO, at iteration number one, the initial population vector is generated with uniformly distributed random positions $P_p(t)$ where, t is the time step, and $\left(N_p\right)$ is the number of particles hereafter. Formula (3) is used to locate the particles in the state-space at every time step:

$$P_p(t) = P_p(t-1) + V_p(t) \tag{3}$$

where, $P_p(t)$ is the updated position of the particle which depends on the velocity of the particle varies with t.

Formula (4) is used to determine the velocity of the particles w.r.t to each time step:

$$V_p(t) = w \cdot V_p(t-1) + C_1 \cdot R_1(k) \cdot \left(P_L - P_p(t-1)\right) + C_2 \cdot R_2(k) \cdot \left(P_G - P_p(t-1)\right) \tag{4}$$

where, w = inertia weight (to control the velocity changes smoothly) $R_1(k)$ & $R_2(k)$ = uniformly distributed random no. in [0, 1] P_L & P_G = local and global best solutions of the particles C_1 & C_2 = constant to alter the convergence rate If the objective function value of the updated position is less, means if $f\left(P_p(t)\right) < f(P_L)$,

then $P_L = P_p(t)$ and $f\left(P_p(t)\right) < f(P_G)$, then $P_G = P_p(t)$ in local and global search spaces of the particles.

The strategy used to explore the new promising zones in the state space to find the best solution is called exploration. On the other hand, exploring all the newly founded promising zones in depth by local best particle using their best experience to get the best solution in the local area and update these local best solutions to get the best global solution is termed an exploitation mechanism in the PSO algorithm. In PSO, to get the best possible solution based on local best P_L and global best P_G search the convergence criteria can be the number of iterations, stagnation of results, or scheduled iterative time.

4 Proposed SAPSODV-Hop

This section serves as a demonstration of the SAPSODV-Hop, a revolutionary method of DV-Hop positioning with RSSI. The localization accuracy is increased using RSSI, and if RSSI's noise sensitivity could be decreased, the accuracy could be increased even more. In the meantime, we also describe the SAPSODV-Hop positioning algorithm based on "DV-Hop" with RSSI for the sake of comparison.

4.1 SAPSO-Based DV-Hop Positioning with RSSI

The DV-Hop and RSSI are incorporated into the suggested strategy to decrease distance measuring error without the use of any additional hardware. Utilizing the SAPSO Optimization approach, the coordinates of NUP are determined after obtaining the distance from all NKP to NUP.

(1) Each NKP broadcasts to every surrounding node a beacon information and an RSSI packet. The beacon information contains the NKP's node id, position coordinates (x_i, y_i), an initialized hop number $hops_i$, and an initialized cumulative RSSI distance d_{rssi}. The beacon message's format can then be stated as {id, (x_i, y_i), $hops_i$, d_{rssi}}. After receiving the broadcast, each neighboring node applies Eq. (5), which changes the $hops_i$ and d_{rssi} values, to continue broadcasting the revised NKP message to the other neighbor sensor nodes.

$$\begin{cases} hops_i = hop_i + 1 \\ d_{rssi} = d_{rssi} + d_{hop} \end{cases} \tag{5}$$

where d_{hop} denotes the predicted distance after being converted by this nearby node's RSSI value. Then, d_{hop} can be defined by:

$$d_{hop} = 10^{\frac{p_{tr} - p_{loss}(d_0) - rssi'}{10\tau}} \times d_0 \tag{6}$$

where $rssi^{'}$ is the average *rssi* value of this neighboring sensor.

(2) Once, the lowest hop count of the NKP and the associated cumulative RSSI distances from the NKP to other NKPs are determined, it should be easy to calculate an average "hopsize" and "RSSI" range of first hop. The NKP i then determines the "hopsize" written as HS_{avg} and the moderate RSSI distance written as $ARDPH_i$ of each hop as:

$$HS_i = \left(\sum_{i \neq j}^{n} \sqrt{\left(x_i - x_j\right)^2 + \left(y_i - y_j\right)^2} \right) \cdot \left(\sum_{i \neq j}^{n} h_{ij} \right)^{-1} \tag{7}$$

$$ARDPH_i = \left(\sum_{I \neq J}^{N} d_{rssi_{ij}} \right) \cdot \left(\sum_{i \neq j}^{n} h_{ij} \right)^{-1} \tag{8}$$

where $\left(x_j, y_j\right)$ and (x_i, y_i) represents the coordinates of NKP j and NKP i, respectively. Additionally, h_{ij} represents minimum hop value between NKP i and j; $d_{rssi\,ij}$ represents the "RSSI" accumulated distance between NKP i and j. Here, n represents total NKPs. Each NKP communicates its information "hopsize" value and average hop RSSI distance in the network. The NUP saves the moderate hop RSSI distance and average "hopsize" that it receives from a certain NKP and omits any other information after that.

The localization process is significantly influenced by neighboring NKPs; the inaccuracy in position estimation is directly inversely proportional to the distance between the NUP and the NKP. In light of the weights provided in [21], In this work novel weight factor formula is given in Eq. (9):

$$w_i = \left(\frac{\sum_{j=1}^{n} hopc_{uj}}{n \times hopc_{ui}} \right)^{\frac{R}{HS_{ui}}} \tag{9}$$

HS_{ui} is as given by Eq. (7). Each NKP's influence in relation to its nearby neighbors is determined by the weight's base. The term utilized in the exponent offers additional improvement by increasing the influence of nearby NKPs and decreasing the influence of distant NKPs.

(3) The adjustment factor β is determined by dividing the $d_{rssi\,ij}$ by the moderate RSSI distance of each hop $ARDPH_i$. Then, using Eq. (10), one may determine the distance between the NKP i and NUP j:

$$\begin{cases} \beta = \dfrac{d_{rssi_{ij}}}{ARDPH_i} \\ D_{ij} = \beta \times HS_i \end{cases} \tag{10}$$

(4) Finally, this study employs the SAPSO to get the coordinates of each NUP after estimating the distances between each NUP and the NKPs.

4.2 Optimized Node Localization Process Using SAPSO

Once the proposed procedure has produced the calculated distances. Thereafter, the objective function for SAPSODV-Hop is defined in Eq. (11) using the average squared inaccuracies of the estimated distances,

$$f(x_u, y_u) = \sum_{i=1}^{n} * w_i \left(\sqrt{(x_u - x_\mathrm{i})^2 + (y_u - y_\mathrm{i})^2} - d_{iu} \right)^2 \tag{11}$$

where, (x_i, y_i) is the location of NKP i, (x_u, y_u) is the location of NUP u, d_{iu} is the estimated distances, and N_k is the connected NKP count.

5 Simulation Parameter and Experimental Environment

Simulation Parameters:

Moderate LE

LE is the inaccuracy that was made when estimating the actual position of a NUP. In this work, the impact of the total number of NUPs, NKPs, communication range and deployment area on the localization inaccuracies is evaluated. The moderate localization inaccuracy is given as:

$$\mathrm{LE} = \frac{\sum_{I=A+1}^{N} \sqrt{\left(x_u^{est} - x_u^{act}\right)^2 + \left(x_u^{est} - x_u^{act}\right)^2}}{(N - A) * R} \tag{12}$$

where, (x_u^{est}, y_u^{est}) and (x_u^{act}, y_u^{act}) is the estimated position and true position of the NUP . The total sensors are represented by N. A is the NKP. R is the communication radius.

Experimental Results

This section analyses LE and LA in a simulation environment to show the efficacy of the suggested approach (SAPSODV-Hop). To get at the results, the proposed and other compared algorithms are implemented in MATLAB 2019b. The proposed SAPSODV-Hop approach is matched with the "DV-Hop" and PSO "DV-Hop" algo-

rithms for performance verification. For comparison, the simulations of the proposed SAPSODV-Hop and other algorithms, each ran 50 times, were taken into account. All experimental data have been adjusted to account for the 100 m × 100 m squared region. All the nodes are homogeneous.

• **Experiment 1**: Influence of R on LE and LA.

(i) LE versus R

The impact of R on LE is shown in Fig. 2. The constant value of total node count is 100, the constant value of NKP percent is 30% has been taken into consideration, and the communication radius varies from 20 to 50 m. The simulated results of the proposed SAPSODV-Hop are compared with DV-Hop and PSODV-Hop. The results demonstrate that the LE of the proposed SAPSODV-Hop is smaller than other compared algorithms.

(ii) LA versus R

The impact of R on LA is shown in Fig. 3. The constant value of total node count is 100, the constant value of NKP percent is 30% has been taken into consideration, and the communication radius varies from 20 to 50 m. The simulated results of the proposed SAPSODV-Hop are compared with DV-Hop and PSODV-Hop. The results demonstrate that the LA of the proposed SAPSODV-Hop is greater than other compared algorithms.

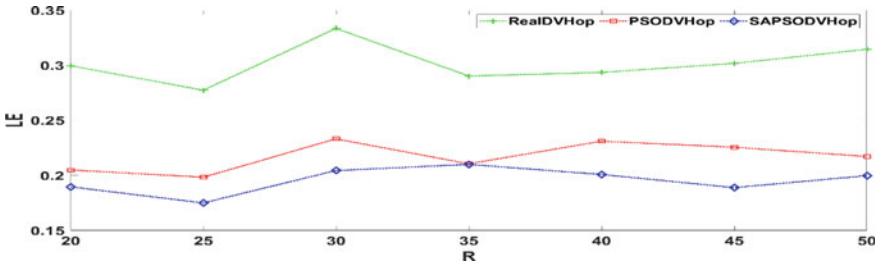


Fig. 2 LE versus R

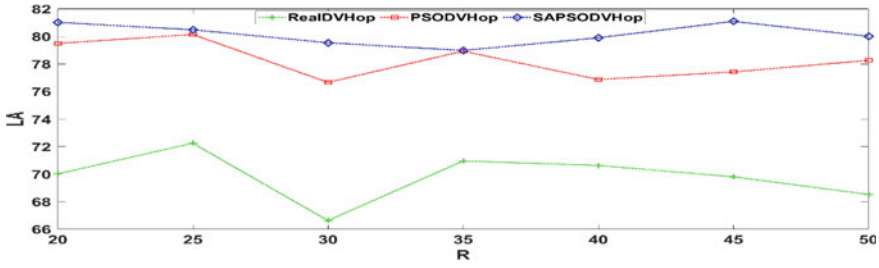


Fig. 3 LA versus R

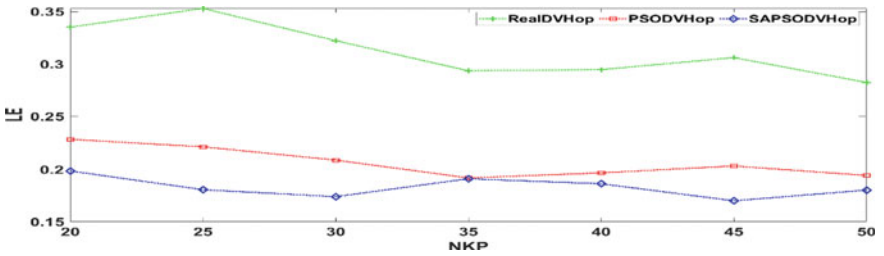


Fig. 4 LE versus NKP

• **Experiment 2**: Influence of NKP percent on LE and LA.

(i) LE versus NKP

The impact of NKP on LE is shown in Fig. 4. The constant value of total node count is 100, the constant value of R is 35 m has been taken into consideration and the percent of NKPs varies from 15 to 45. The simulated results of the proposed SAPSODV-Hop are compared with DV-Hop and PSODV-Hop. The results demonstrate that the LE of the proposed SAPSODV-Hop is smaller than other compared algorithms.

(ii) LA versus NKP

The impact of NKP on LA is shown in Fig. 5. The constant value of total node count is 100, the constant value of R is 35 m has been taken into consideration and the percent of NKPs varies from 15 to 45 m. The simulated results of the proposed SAPSODV-Hop are compared with DV-Hop and PSODV-Hop. The results demonstrate that the LA of the proposed SAPSODV-Hop is greater than other compared algorithms.

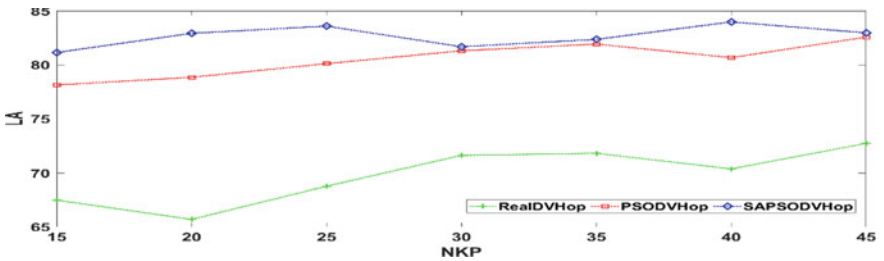


Fig. 5 LA versus NKP

Table 1 Parameter settings of SAPSODV-Hop

Parameter	Value	Parameter	Value
Sensing field area	100×100 m^2	Number of generations	50
Total run	50	Size of the population (NP)	20
Percentage of NKPs	15, 20, 25, 30, 35, 40, 45%	Inertia weight	w = 0.8
Communication range R	20, 25, 30, 35, 40, 45 m, 50 m	Acceleration coefficient C1 & C2	1.5
Network topology	Random	Initial temp	T0 = 0.025
Dimension of problem	D = 2	Temp. reduction rate	Alpha = 0.99

6 Conclusion

This study offers an improvement to the range-free "DV-Hop" method. The location of NKPs is estimated using the proposed SAPSODV-Hop localization method, The proposed SAPSODV-Hop has three phases, the first phase is similar to the original "DV-Hop", and in the second phase, "RSSI" based distance value is used at one hop (within the communication ring) of NKP to determine the improved hopsize. To further refine hop size, the weight factor is introduced. The results of the simulation show that the SAPSODV-Hop strategy surpasses the other "DV-Hop" methods in terms of LE and LA.

References

1. Bulusu N, Heidemann J, Estrin D (2000) GPS-less low-cost outdoor localization for very small devices. IEEE Pers Commun 7(5):28–34
2. Yick J, Mukherjee B, Ghosal D (2008) Wirel Sens Netw Surv Comput Netw 52(12):2292–2330
3. Akyildiz IF, Su W, Sankarasubramaniam Y, Cayirci E (2002) Wireless sensor networks: a survey. Comput Netw 38(4):393–422
4. Čapkun S, Hamdi M, Hubaux JP (2002) GPS-free positioning in mobile ad hoc networks. Clust Comput 5(2):157–167
5. Wang J, Gao Q, Yu Y, Cheng P, Wu L, Wang H (2012) Robust device-free wireless localization based on differential RSS measurements. IEEE Trans Industr Electron 60(12):5943–5952
6. Patwari N, Ash JN, Kyperountas S, Hero AO, Moses RL, Correal NS (2005) Cooperative Localization in Wireless Sensor Networks. IEEE Signal Process Mag 22:54–69
7. Xie H, Li W, Li S, Xu B (2016) An improved DV-Hop localization algorithm based on RSSI auxiliary ranging. In: 2016 35th Chinese Control Conference (CCC). IEEE, July 2016, pp 8319–8324
8. Golestanian M, Poellabauer C (2016) Localization in heterogeneous wireless sensor networks using elliptical range estimation. In: 2016 international conference on computing, networking, and communications (ICNC). IEEE, Feb 2016, pp 1–7
9. Niculescu D, Nath B (2001) Ad hoc positioning system (APS). In: GLOBECOM'01. IEEE global telecommunications conference (Cat No 01CH37270), vol 5. IEEE, pp 2926–2931

10. Niculescu D, Nath B (2003) DV-based positioning in ad hoc networks. Telecommun Syst 22(1):267–280
11. Chen Y, Li X, Ding Y, Xu J, Liu Z (2018) An improved DV-Hop localization algorithm for wireless sensor networks. In: 2018 13th IEEE conference on industrial electronics and applications (ICIEA). IEEE, May 2018, pp 1831–1836
12. Javidrad F, Nazari M (2017) A new hybrid particle swarm and simulated annealing stochastic optimization method. Appl Soft Comput 60:634–654
13. Kennedy J, Eberhart R (1995) Particle swarm optimization. In: Proceedings of ICNN'95-international conference on neural networks, vol 4. IEEE, pp 1942–1948, Nov, 1995
14. Kulkarni RV, Venayagamoorthy GK (2010) Particle swarm optimization in wireless sensor networks: a brief survey. In: IEEE transactions on systems, man, and cybernetics, part C (Applications and Reviews), vol 41(2), pp 262–267
15. Singh SP, Sharma SC (2018) A PSO-based improved localization algorithm for wireless sensor network. Wirel Pers Commun 98(1):487–503
16. Man DP, Qin GD, Yang W, Wang W, Xuan SC (2014) Improved DV-HOP algorithm for enhancing localization accuracy in WSN. Appl Mech Mater 543:3256–3259
17. Cheikhrouhou O, Bhatti M, Alroobaea GR (2018) A hybrid DV-hop algorithm using RSSI for localization in large-scale wireless sensor networks. Sensors 18(5):1469
18. Tomic S, Beko M, Dinis R (2014) Distributed RSS-based localization in wireless sensor networks based on second-order cone programming. Sensors 14(10):18410–18432
19. Benkic K, Malajner M, Planinsic P, Cucej Z (2008) Using RSSI value for distance estimation in wireless sensor networks based on ZigBee. In: Proceedings of the 15th international conference on systems, signals and image processing, Bratislava, Slovakia, 25–28 June 2008; pp 303–306
20. Yang X, Pan W (2012) DV-Hop localization algorithm for wireless sensor networks based on RSSI ratio improving. Transducer Microsyst Technol 32:126–128
21. Zhang B, Ji M, Shan L (2012) A weighted centroid localization algorithm based on DV-hop for wireless sensor network. In: 2012 8th international conference on wireless communications, networking and mobile computing. IEEE, pp 1–5, Sept 2012

Multimodal Paddy Leaf Diseases Detection Using Feature Extraction and Machine Learning Techniques



P. Kaviya and B. Selvakumar

Abstract Agriculture is a major determinant of a country's economic development, and it is the primary source of income for many Indian farmers. One of the most often cultivated crops is paddy. However, paddy cultivation is affected by changing environmental conditions and is subjected to suffer diverse diseases. Majority of these diseases initially affects the plant's leaves, but eventually it propagates to the entire paddy crop, impacting the quality and quantity of product yield. By accurate and timely diagnosis of those diseases and disease-causing pests, it is possible to assist farmers in opting for suitable plant treatment and thereby reducing economic loss while improving yield. Healthy and diseased leaves are identified and classified using their morphological features like color, texture, and shape of the leaves. In this scenario, feature extraction is an essential and the most important step for leaf disease identification. To do so, the feature extraction methods like GLCM, GLDS, GLRLM, and LBP are used in this study to extract features from the images collected from the Kaggle rice disease dataset. Besides, SVM, Decision Tree, KNN, and Naive Bayes classifiers are employed to classify the features obtained by those approaches to determine the disease. To detect paddy leaf diseases, the most successful feature extraction technique and classification algorithm has been demonstrated.

Keywords Paddy leaf disease · Feature extraction · Machine learning · GLCM · GLDS

P. Kaviya (✉)
Kamaraj College of Engineering and Technology, Vellakulam 625701, TN, India
e-mail: kaviyaap@gmail.com

B. Selvakumar
Mepco Schlenk Engineering College, Sivakasi 626005, TN, India
e-mail: selvakumar.b@mepcoeng.ac.in

© The Author(s), under exclusive license to Springer Nature Singapore Pte Ltd. 2023

S. Chinara et al. (eds.), *Advances in Distributed Computing and Machine Learning*,
Lecture Notes in Networks and Systems 660,
https://doi.org/10.1007/978-981-99-1203-2_42

1 Introduction

Agriculture is critical to any country's economic development. One such crop that is of crucial importance in agriculture is paddy. Since rice is the primary source of food for many Asian people (90% of the world's rice is produced and consumed in Asia [12]), it is essential to ensure the disease-free and high productivity rate of paddy. Besides, India's population is expanding at a rate of 1.0% each year [18] and is a warning sign to conserve the quality of paddy cultivation to keep up with the country's growing population. It is evident from statistical analysis that, farmers lose an average of 37% of their rice yield due to pests and crop illnesses [13]. It is imperative to note that rice productivity is slowed for a variety of causes. One such cause that affects paddy cultivation is derailing climatic conditions.

Leaf blast, bacterial leaf blight, tungro, sheath blight, sheath rot, brown spot, false smut, leaf streak, and grain discoloration are the most common rice plant diseases [20]. Plants infected with disease produce less rice and rice of poor quality. According to the International Rice Research Institutes' report, 70% of crop production is affected due to bacterial leaf blight disease [6]. Also [7] states that brown spot disease affects rice productivity up to 45%. Diseases can wreak havoc on rice production if they aren't treated quickly.

In response to such condition, proper mitigation, or curation after detecting disease condition and monitoring the infection stage is the need of hour. In general, manually detecting and recognizing diseases with the naked eye is quite difficult. It is also time demanding. One of the most difficult tasks in agricultural development is detecting rice plant illnesses and disease-inducing pests at right time. As a result, the most intriguing and compelling topic in agro-informatics is a automatic plant disease identification system. A fast and accurate leaf disease recognition system is required to boost rice yield, also care must be done to ascertain the severity of the sickness. To distinguish healthy leaves from diseased leaves, knowledge on the features of a natural paddy leaf is critical. Diseases are categorized only based on their change in features. In this regard, feature extraction is essential for image classification. Machine learning and deep learning approaches are recently used to improve image categorization accuracy. As mentioned above in this paper the frequently occurring paddy leaf diseases like BrownSpot, Hispa, and LeafBlast, are the focus of this research.

In this paper, a paddy leaf disease detection model is proposed using the Kaggle rice disease dataset. The leaf features from pre-processed leaf images are extracted using several feature extraction algorithms. The collected features are fed into a variety of machine learning classifiers to detect leaf diseases. Besides, in this study, a comparative analysis is done to determine the best combination of feature extraction algorithm and machine learning technique to detect the disease of interest. Various feature extraction algorithms and machine learning techniques to identify and categorize plant leaf diseases in previous work are addressed in Sect. 2. Section 3 discusses the proposed methodology, which includes the development of machine learning

models using various feature extraction techniques to classify paddy leaf diseases. Section 4 provides a comparative analysis of built models on the Kaggle rice disease dataset. Finally, Sect. 5 concludes the work with possible future directions.

2 Related Works

One of the most important information repositories for detecting and classifying plant types and illnesses is leaf images. In today's research world, machine learning and deep learning techniques are primarily used to monitor crop productivity and crop-associated diseases. Various authors are working on feature extraction, detection, and categorization of plant leaf diseases [9, 16].

Zhong et al. [21] have combined a deep convolution network DenseNet-121 with a cross-entropy loss function to identify apple leaf diseases. Few-Shot Learning (FSL) algorithms were introduced by Argueso et al. [3] for plant leaf disease classification using Inception-v3 with the PlantVillage dataset. Deep learning algorithms (Inception-v3, VGG-16, and VGG-19) and machine learning (Random Forest (RF), Support Vector Machine (SVM), and Stochastic Gradient Descent (SGD)) were compared by Sujatha et al. [17] to detect citrus plant diseases.

Agarwal et al. [1] have used a simplified convolutional neural network to detect diseases in tomato crop using PlantVillage dataset. Mwebaze et al. [11] have classified diseases in cassava plants using Rotated BRIEF (ORB), Color and Oriented FAST feature extraction algorithms with machine learning techniques like linear support vector, K-Nearest Neighbor (KNN), and extremely randomized trees. Tulshan et al. [19] have performed image segmentation, disease-feature extraction from the segmented images, type of plant disease classification using k-mean clustering, Gray-Level Co-occurrence Matrices (GLCMs), and k-Nearest Neighbor (KNN) classifier, respectively.

Sethy et al. [15] have proposed a deep feature plus SVM model to segregate four different rice leaf diseases like bacterial light, brown spot, blast, and tungro using 5932 diseased rice leaf images. Rahman et al. [14] have demonstrated deep learning-based approaches for diagnosing diseases and causative pests on rice plant images. Azim et al. [4] have used k-mean clustering for image segmentation, GLCM for feature extraction, and Extreme gradient Boosting (XGBoost) classifier to detect leaf diseases in rice plants.

Alfarisy et al. [2] have classified paddy leaf diseases and their causative pests using fine-tuned CaffeNet—an open-source deep learning framework. Zhou et al. [22] have combined K-Means clustering algorithm (FCM-KM) with the R-CNN algorithm to distinguish rice diseases on 3010 images. Mainak Deb et al. [5] have concentrated on paddy disease identification using five different disease classification datasets by various CNN models that included MobileNet V2, Inception-V3, ResNet-18, Alex Net, and VGG-16. Among the CNN models used, their experimental study indicated that Inception-V3 outperformed with an accuracy of 96.23%.

Mashoor et al. [10] have developed a deep learning model, MASK Recurrent Convolutional Neural Network (Mask RCNN), and Efficient Net B3 Convolution Neural Network for detecting the disease-affected regions and disease classification, respectively. Islam et al. [8] have compared four deep learning algorithms like VGG-19, Inception-Resnet-V2, ResNet-101, and Xception for the detection of paddy leaf disease. Among them, Inception-ResNet-V2 was outperformed with an accuracy of 92.68%.

In the proposed work, a paddy leaf disease detection model is constructed using various feature extraction techniques and machine learning classifiers on the Kaggle rice disease dataset to efficiently detect the diseases like BrownSpot, Hispa, and LeafBlast.

3 Materials and Methods

A paddy leaf disease detection model is designed and trained on the rice disease dataset extracted from Kaggle. The features are extracted using several feature extraction algorithms after the leaf images have been pre-processed. The collected features are then fed into several machine learning classifiers to detect leaf diseases. Figure 1 depicts the proposed system's design.

The construction of paddy leaf disease detection model includes data collection, pre-processing, feature extraction, and rice leaf disease classification

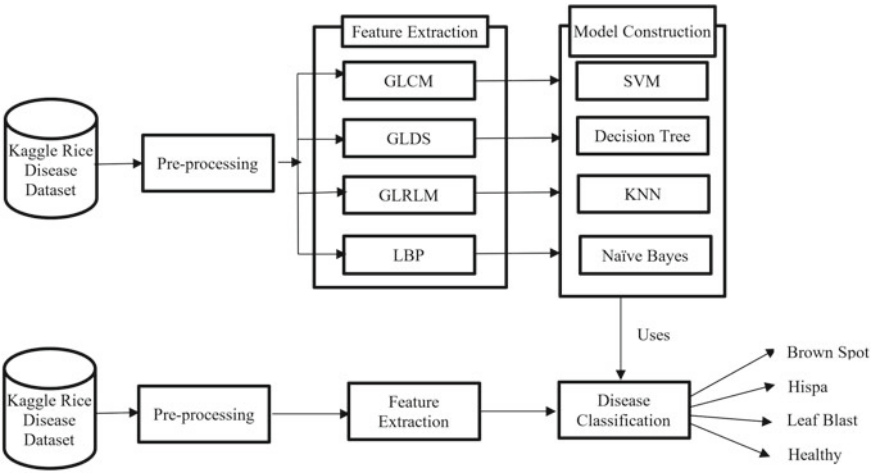


Fig. 1 Proposed system architecture

3.1 Dataset

The collected rice disease dataset from Kaggle consists of 2,092 images. The images are categorized into three diseased classes and one healthy leaf class according to the feature analysis. The three diseased classes are BrownSpot, Hispa, and LeafBlast. Table 1 describes the number of images used to train and test the model. The sample healthy and diseased images are given in Fig. 2.

3.2 Pre-processing

Pre-processing is used for image enhancement and noise removal which is an essential step for feature extraction once image acquisition is completed. Pre-processing is done in two steps; they are grayscale conversion and image enhancement. The original images in the dataset were of different sizes and it is resized into same size of 224x224. Then the RGB images are converted into grayscale images and have been given as an input for feature extraction.

3.3 Feature Extraction

Feature extraction approaches seek to identify the most important aspects that can be used to describe an image with fewer parameters. Various textural-level feature

Table 1 Kaggle rice disease dataset

S. No	Disease class	No. of training samples	No. of testing samples
1.	BrownSpot	400	123
2.	Healthy	400	123
3.	Hispa	400	123
4.	LeafBlast	400	123

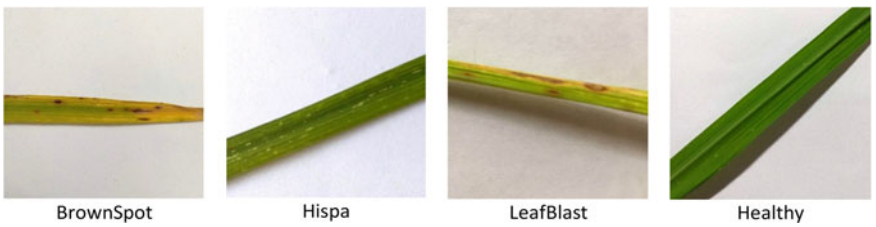


Fig. 2 Sample images from dataset

extraction strategies are utilized in this study to extract critical features from rice leaf disease images for machine learning algorithms to classify the disease. They are extracted using the GLCM, Gray-Level Difference Statistics (GLDS), Gray-Level Run Length Matrix (GLRLM), and Local Binary Pattern (LBP).

3.3.1 Gray-Level Co-occurrence Matrix (GLCM)

GLCM is a typical textural feature extraction technique that uses the second-order join conditional probability between two pixels to extract properties like contrast, correlation, and variance from the gray-level spatial dependency matrix.

3.3.2 Gray-Level Difference Statistics (GLDS)

GLDS extracts five texture measures like homogeneity, contrast, entropy, and mean by using first-order statistics of regional property values, based on absolute differences between any two pixels of proximity.

3.3.3 Gray-Level Run Length Matrix (GLRLM)

GLRLM extract features like gray level and run length distribution. From the gray-level run length matrices, minimum gray level is emphasized, where gray-level run is a set of continual collinear image points with same gray-level value.

3.3.4 Local Binary Pattern (LBP)

LBP is used in texture classification and segmentation. LBP calculates the pixel value by measuring and comparing the magnitude of threshold value with the proximal pixels with center pixel.

Table 2 describes the equations used to extract the various features from paddy leaf.

3.4 Paddy Leaf Disease Classification

The collected features are then employed to the classifiers such as Support Vector Machine (SVM), Decision Tree, K-Nearest Neighbor (KNN), and Naive Bayes to further classify rice illnesses. SVM is a fast supervised classification algorithm that finds the hyperplane that separates the different classes. By finding information entropy, decision tree classifier constructs an appropriate tree from a given learning

Table 2 Feature selection methods

S. no	Feature extraction method	Feature name and equation		
1.	GLCM	Angular Second Moment (ASM) $\sum_{i=0}^{N-1}\sum_{j=0}^{N-1} p(i,j)^2$	Contrast $\sum_{i=0}^{N-1} n^2 \left\{\sum_{i=0\|i-j\|=n)}^{N-1}\sum_{j=0}^{N-1} p(i,j)\right\}$	Correlation $\frac{\sum_{i=0}^{N-1}\sum_{j=0}^{N-1}(ij)p(i,j)-\mu_x\mu_y}{\sigma_x\sigma_y}$
		Homogeneity $\sum_{i=0}^{N-1}\sum_{j=0}^{N-1}\frac{p(i,j)}{1+\|i-j\|}$	Variance $\sum_{i=0}^{N-1}\sum_{j=0}^{N-1}(i-\mu)^2 p(i,j)$	Sum Average $\sum_{k=1}^{2N-1} k p_{x+y}(k)$
		Sum Variance $\sum_{k=1}^{2N-1}(i-\mu_{x-y})^2 p_{x+y}(k)$	Sum Entropy $-\sum_{k=1}^{2N-1} p_{x+y}(k)\ln\left[p_{x+y}(k)\right]$	Entropy $-\sum_{i=0}^{N-1}\sum_{j=0}^{N-1} p(i,j)\log[p(i,j)]$
		Difference Variance $\sum_{k=0}^{N-1}(k-\mu_{x-y})^2 p_{x-y}(k)$	Difference Entropy $-\sum_{k=0}^{N-1} p_{x-y}(ik)\log\left[p_{x-y}(k)\right]$	Correlation Measures $\frac{HXY-HXY1}{max\{HX,HY\}}$
		Maximal Correlation Coefficient $\frac{HXY-HXY1}{max\{HX,HY\}}$		
2.	GLDS	Homogeneity $\sum_i \frac{p_\delta(i)}{i^2+1}$	Contrast $\sum_i i^2 p_\delta(i)$	Angular Second Moment $\sum_i p_\delta(i)^2$
		Entropy $-\sum_i p_\delta(i)\log[p_\delta(i)]$	Mean $\sum_i i p_\delta(i)$	
3.	GLRLM	Short Run Emphasis $\frac{\sum_{i=0}^{N_g-1}\sum_{j=0}^{N_r-1}\frac{p_\theta(i,j)}{j^2}}{N_z}$	Long Run Emphasis $\frac{\sum_{i=0}^{N_g-1}\sum_{j=0}^{N_r-1} j^2 p_\theta(i,j)}{N_z}$	Gray-Level Distribution $\frac{\sum_{i=0}^{N_g-1}\left(\sum_{j=0}^{N_r-1} p_\theta(i,j)\right)^2}{N_z}$
		Run Length Distribution $\frac{\sum_{j=0}^{N_g-1}\left(\sum_{i=0}^{N_r-1} p_\theta(i,j)\right)^2}{N_z}$	Run Percentage $\frac{N_z}{N_p}$	Low Gray-Level Run Emphasis $\frac{\sum_{i=0}^{N_g-1}\sum_{j=0}^{N_r-1}\frac{p_\theta(i,j)}{i^2}}{N_z}$
		High Gray-Level Run Emphasis $\frac{\sum_{i=0}^{N_g-1}\sum_{j=0}^{N_r-1} p_\theta(i,j)i^2}{N_z}$	Short Low Gray-Level Emphasis $\frac{\sum_{i=0}^{N_g-1}\sum_{j=0}^{N_r-1}\frac{p_\theta(i,j)}{i^2 j^2}}{N_z}$	Short Run High Gray-Level Emphasis $\frac{\sum_{i=0}^{N_g-1}\sum_{j=0}^{N_r-1}\frac{p_\theta(i,j)}{j^2}}{N_z}$
		Long Run Low Gray-Level Emphasis $\frac{\sum_{i=0}^{N_g-1}\sum_{j=0}^{N_r-1}\frac{p_\theta(i,j)}{i^2}}{N_z}$	Long Run High Gray-Level Emphasis $\frac{\sum_{i=0}^{N_g-1}\sum_{j=0}^{N_r-1} p_\theta(i,j)i^2 j^2}{N_z}$	
4.	LBP	$LBP_{P,R}(x)=\begin{cases}\sum_{p=0}^{P-1} s(g_p-g_c), if\, U(x)\le 2\\ P+1, Otherwise\end{cases}$		

set class instances. Each data attribute can be utilized to make a choice that divides the data into smaller chunks. The decision is made based on the attribute with the best information gain ratio.

To categorize data, naive Bayes employs the Bayesian theorem, which describes a set of variables and their probabilistic independencies. In KNN, an instance of feature is classified based on most of its neighbors, with the instance of feature being allocated to the most common class among its k-nearest neighbors. The extracted features from GLCM, GLDS, GLRLM, and LBP are fed into all the classifiers separately and, finally, the paddy leaf diseases are classified as BrownSpot, Hispa, LeafBlast, and Healthy by employing the above-mentioned techniques.

4 Results and Discussion

In this work, a paddy leaf disease detection model is built to classify the diseases on an extracted dataset containing 2,092 images from the Kaggle rice disease dataset. BrownSpot, Hispa, LeafBlast, and Healthy are the four types of images in the dataset. Table 1 shows the train:test split ratio that was used in the experiments. To diagnose paddy leaf illnesses, a comparative analysis is performed utilizing feature extraction techniques like GLCM, GLDS, GLRLM, and LBP, as well as machine learning algorithms such as SVM, Decision Tree, KNN, and Naive Bayes. The performance of feature extraction techniques and machine learning classifiers is compared in Fig. 3. When the feature matrix generated by the GLRLM method is classified by the KNN algorithm, it outperforms in detecting paddy leaf illnesses than other algorithms. The feature matrices created by the GLRLM algorithm are classed by SVM, yielding the second best result. The LBP algorithm yields lower outcomes when it comes to feature extraction.

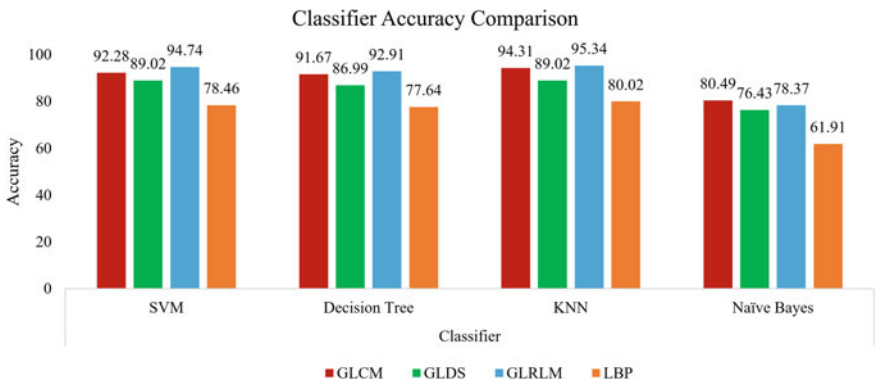


Fig. 3 Comparative analysis of rice leaf disease detection model

5 Conclusion

Using various feature extraction and machine learning methodologies, a comparative analysis is performed to detect and categorize paddy leaf illnesses. Images are scaled and turned to grayscale for use in the research. GLCM, GLDS, GLRLM, and LBP generate feature extraction matrices, which are then classified using SVM, Decision Tree, KNN, and Naive Bayes. GLRLM's feature matrix findings are found to be more accurate than other feature extraction techniques at detecting diseases. Other feature extraction techniques outperform LBP algorithm. KNN classifier detects paddy leaf illnesses more effectively than other machine learning classifiers. With a success rate of 95.34%, the combination of GLRLM and KNN was found to be the most successful in paddy leaf disease detection. From the extended literature survey, it is explicit that, this comparative paddy leaf detection approach with the different combinations of feature extraction algorithms and machine learning techniques is unique and a newer way in digital platform. This demonstrates that this work can be an important element in the roadmap to the future research world.

References

1. Agarwal M, Gupta SK, Biswas KK (2020) Development of efficient CNN model for Tomato crop disease identification. Sustain Comput: Inf Syst 28:100407. https://doi.org/10.1016/j.suscom.2020.100407
2. Alfarisy AA, Chen Q, Guo M (2018) Deep learning based classification for paddy pests and diseases recognition. In: Proceedings of 2018 international conference on mathematics and artificial intelligence. https://doi.org/10.1145/3208788.3208795
3. Argüeso D, Picon A, Irusta U, Medela A, San-Emeterio MG, Bereciartua A, Alvarez-Gila A (2020) Few-Shot Learning approach for plant disease classification using images taken in the field. Comput Electron Agric 175:105542. https://doi.org/10.1016/j.compag.2020.105542
4. Azim MA, Islam MK, Rahman MdM, Jahan F (2021) An effective feature extraction method for rice leaf disease classification. TELKOMNIKA (Telecommunication Computing Electronics and Control) 19(2):463. https://doi.org/10.12928/telkomnika.v19i2.16488
5. Deb M, Dhal KG, Mondal R, Gálvez J (2021) Paddy disease classification study: a deep convolutional neural network approach. In: Optical memory and neural networks, vol 30(4). Allerton Press, pp 338–357. https://doi.org/10.3103/s1060992x2104007x
6. International Rice Research Institute. Bacterial blight-IRRI Rice Knowledge Bank. Rice Knowledge Bank. http://www.knowledgebank.irri.org/training/fact-sheets/pest-management/diseases/item/bacterial-blight?category_id=326. Accessed Oct 2021
7. International Rice Research Institute. Bacterial blight-IRRI Rice Knowledge Bank. Rice Knowledge Bank. http://www.knowledgebank.irri.org/training/fact-sheets/pest-management/diseases/item/brown-spot. Accessed Oct 2021
8. Islam MdA, Nymur Md, Shamsojjaman M, Hasan S, Shahadat Md, Khatun T (2021) An automated convolutional neural network based approach for paddy leaf disease detection. Int J Adv Comput Sci Appl 12(1). The Science and Information Organization. DOIurl10.14569/ijacsa.2021.0120134
9. Li L, Zhang S, Wang B (2021) Plant disease detection and classification by deep learning-a review. In: IEEE Access. 9, 56683–56698. Institute of Electrical and Electronics Engineers (IEEE). https://doi.org/10.1109/access.2021.3069646

10. Mashroor F, Ishrak IF, Alvee SM, Jahan A, Islam Suvon MN, Siddique S (2021) Rice paddy disease detection and disease affected area segmentation using convolutional neural networks. In: TENCON 2021–2021 IEEE region 10 conference (TENCON). IEEE. https://doi.org/10.1109/tencon54134.2021.9707192
11. Mwebaze E, Owomugisha G (2016) Machine learning for plant disease incidence and severity measurements from leaf images. In: 15th IEEE international conference on machine learning and applications (ICMLA). https://doi.org/10.1109/ICMLA.2016.0034
12. Papademetriou MK (2021) Rice production in the Asia-pacific region: issues and perspectives. http://www.fao.org/docrep/003/x6905e/x6905e04.htm. Accessed Oct 2021
13. Pests and diseases. IRRI Rice Knowledge Bank. http://www.knowledgebank.irri.org/step-by-stepproduction/growth/pests-and-disease. Accessed Oct 2021
14. Rahman CR, Arko PS, Ali ME, Iqbal Khan MA, Apon SH, Nowrin F, Wasif A (2020) Identification and recognition of rice diseases and pests using convolutional neural networks. Biosyst Eng 194:112–120. https://doi.org/10.1016/j.biosystemseng.2020.03.020
15. Sethy PK, Barpanda NK, Rath AK, Behera SK (2020) Deep feature based rice leaf disease identification using support vector machine. Comput Electron Agric 175:105527. https://doi.org/10.1016/j.compag.2020.105527
16. Shrivastava VK, Pradhan MK (2020) Rice plant disease classification using color features: a machine learning paradigm. J Plant Pathol 103(1):17–26. Springer Science and Business Media LLC. https://doi.org/10.1007/s42161-020-00683-3
17. Sujatha R, Chatterjee JM, Jhanjhi N, Brohi SN (2021) Performance of deep learning versus machine learning in plant leaf disease detection. Microproc Microsyst 80:103615. https://doi.org/10.1016/j.micpro.2020.103615
18. TNAU AGRITECH PORTAL. Crop Protection. https://agritech.tnau.ac.in/crop_protection/crop_prot_crop%20diseases_cereals_paddy.html. Accessed Oct 2021
19. Tulshan AS, Raul N (2019) Plant leaf disease detection using machine learning. In: 10th international conference on computing, communication and networking technologies (ICCCNT). https://doi.org/10.1109/icccnt45670.2019.8944556
20. Worldometers (2020). https://www.worldometers.info/world-population/india-population/. Accessed Oct 2021
21. Zhong Y, Zhao M (2020) Research on deep learning in apple leaf disease recognition. Comput Electron Agric 168:105146. https://doi.org/10.1016/j.compag.2019.105146
22. Zhou G, Zhang W, Chen A, He M, Ma X (2019) Rapid detection of rice disease based on FCM-KM and faster R-CNN fusion. IEEE Access 7:143190–143206. https://doi.org/10.1109/ACCESS.2019.2943454

Quad Mount Fabricated Deep Fully Connected Neural Network Based Logistic Pricing Prediction



M. Shyamala Devi, Penikalapati Sai Akash Chowdary, Muddangula Krishna Sandeep, and Yeluri Praveen

Abstract In order to enhance customer satisfaction, logistics management is a component of the supply chain process that organizes, carries out, and regulates the effective flow and storage of goods, services, and related information from the point of source to the site of consumption. Due to a number of factors, including liberalization, weaker state interference of transportation, alterations in customer behavior, technological innovations, business increasing power, and globalization of trading, determining the precise supply chain pricing remains a difficult problem. With this impetus, this paper seeks to predict the logistic pricing by proposing a "Quad Mount Fabricated Deep Fully Connected Neural Network (QMF-DFCNN)" that uses the supply chain pricing dataset retrieved from the KAGGLE machine-learning repository. The supply chain pricing dataset contains 32 features with 10,037 logistic details, which had been processed with incomplete values. The Quad Mount Fabricated Deep Fully Connected Neural Network initiates by performing exploratory data analysis and the data division is done at 80:20. The coding is implemented with Python through an Nvidia V100 GPU workstation with 30 training iterations subjected to a batch size of 64. The dataset has been subjected to all the regressors to examine the performance of logistic pricing prediction. Execution results portray that the Gradient boost regressor is with high RSquared Value. The dataset was exposed to a gradient boost regressor to extract the six important features. The gradient boost feature extracted dataset applied to a Deep Fully Connected Neural Network had designed with single input having six features, an output layer with pricing attribute and four dense layers with 10 nodes each. Experimental results show that the proposed Quad Mount Fabricated Deep Fully Connected Neural Network shows a minimum Mean Squared Error of 19.2135 and a maximum RSquared error of 0.984 when analyzed with all other regression models.

Keywords DFCNN · MSE · GPU · Regressor · RSquared error

M. S. Devi (✉) · P. S. A. Chowdary · M. K. Sandeep · Y. Praveen
Computer Science & Engineering, Vel Tech Rangarajan Dr. Sagunthala R&D Institute of Science and Technology, Chennai, Tamilnadu, India
e-mail: shyamalapmr@gmail.com

© The Author(s), under exclusive license to Springer Nature Singapore Pte Ltd. 2023

S. Chinara et al. (eds.), *Advances in Distributed Computing and Machine Learning*, Lecture Notes in Networks and Systems 660,
https://doi.org/10.1007/978-981-99-1203-2_43

1 Introduction

All costs associated with business logistics are referred to as logistical costs. They, therefore, concern the costs associated with shipping, moving, and storing products. However, poor preparation during decision-making frequently leads to exorbitant prices. Strong industry rivalry combined with escalating travel expenses emphasises the significance of effective freight and logistics pricing policies. Variations of a dynamic pricing scheme are typically among the finest solutions for designing pricing strategies with adaptability to suit the needs of various cost-based scenarios and individual clients. The variances in the delivery area or location, the product, rising or falling consumption, and shifting economic circumstances are most frequently reflected in the logistic costing fluctuations.

Freight and logistics companies are able to adjust rates within the variable pricing space in order to meet revenue objectives. The prediction and optimization applications of machine-learning technology could be employed in transdisciplinary applications. The paper is further organized to deal with review in Sect. 2 followed by Research methodology under Sect. 3. Section four deals with Implementation Setup and Result discussion ending with a conclusion in Sect. 5.

2 Literature Review

Two deep In order to create the lists for customized trip package recommendations, this research suggests a cocktail technique. To further capture the latent links between the vacationers in each tour company, this study extends the TAST model to the tourist-relation-area-season topic (TRAST) model. Finally, they assess the cocktail recommendation approach, and the TAST model using data from actual vacation packages. Experimental findings demonstrate that the cocktail strategy is significantly more successful than typical prediction models for capturing the distinctive properties of the statistics [1]. In addition to the fundamental features, the features extraction segment further analyzes mobile user behavior and suggests a few unique feature scenarios. These scenarios include the group-based rating, transitioning rate, centralized percentage, re-buy behaviors, and others that have significantly improved the model in practice. unique model training for various user-item pair patterns, such as the forecasting of sales the following day and re-purchase patterns [2].

In the retail market, the difficulty of customers to make rapid, firm judgments when faced with a broad variety of product selections is referred to as indecisiveness. Indecision has been studied in a variety of disciplines, including sociology and economics. This research typically relies on irrational consumer questionnaire responses with some personally created queries [3]. The limitations of present recommendation techniques are also discussed in this study, along with potential additions that could enhance recommendation capabilities and broaden the variety of uses for recommender systems. The improvements in understanding of users and objects, the

inclusion of relevant data in the recommendation process, support for multicriteria ratings, and the availability of more adaptable and unobtrusive suggestion kinds are only a few of these developments [4]. By utilizing complementary and mutually beneficial data related to consumers' registration behaviors (such as reservation message, duration, and geography), this study successfully addresses data operability while delivering suggestions [5]. Boosting involves giving points that prior forecasters mispredicted extra weight. A weighted vote is ultimately taken for the forecast. In bagging, each tree is independently built using a bootstrap sample of the data set, and subsequent forests do not rely on previous trees [6].

One of the new concerns in the field of classifiers is the suggestion of logistic products. The most popular collaborative filtering algorithms are typically difficult to utilize for promoting travel-related products. To address these challenges, we provide a novel rating system for logistic-related products based on topic sequential patterns [7]. Projected selection may be represented in one method by a two-dimensional variable. Another approach is to employ a Gaussian prior to simulating user cost preference while taking into account the uncertainty of the cost that a user can afford. They create various expense matrix factorization modelling by including the cost data into the probabilistic matrix factorization using these two methods of reflecting consumer cost preference [8]. The outcomes of the case study demonstrate that the ten new indexes pertaining to the bridge and tunnel increase the model's forecast precision. The CNN methodology is also better suited to resolving nonlinear expense forecasting is a strongly nonlinear problem that can be predicted more accurately than with traditional artificial neural network and regression analysis techniques [9]. A highway project's cost is highly correlated with its total size, structural attributes, economic development, and other factors like supplies, labor rates, etc. A significant amount of research has investigated the engineering affecting elements [10]. Even while thrombolysis identified to reduce impairment and increase health outcomes in patients with a diagnosis, some people continue to suffer detrimental effects. Consequently, it might be beneficial when making healthcare decisions to be able to predict how patients with myocardial infarction would react to systematic rejuvenation [11]. The database is able to critically examine learning using the performance curve and operation characteristics as performance indicators for machine-learning techniques. When assessing machine-learning methods on a single data point, the paper's conclusion advises using AUC rather than overall accuracy [12].

As a result, the data, algorithms, and models used in predictive analytics will be presented in this article together with their effectiveness and supply–chain management outcome taxonomy, which contains all the elements required for a supply chain to be implemented effectively. This taxonomy enables manufacturers to obtain a thorough grasp of these complex circumstances and better manage the supply chain management activities, as well as giving scientists a stepping-stone to produce more valuable publications in the future [13]. The current study bases its evaluation of BDA capacity as a business transformation strategy for enhancing sustainable supply chain performance on the dynamic capability theory. We conducted a poll of mining executives in South Africa's developing economy, and we got 520 legitimate responses. To examine the data, we employed partial least squares structural equation modelling.

The results demonstrate that management competencies for big data analytics have a strong and significant impact on the creation of innovative green products and the success of sustainable supply chains [14]. This study does not claim to be exhaustive; rather, it developed comprehensive literature and case studies in an effort to advance the conversation by using a conceptual framework for examining the connections between production chain risks and digitalization [15].

3 Research Methodology

Figure 1 depicts the general research design of the proposed work. Figures 2 and 3 depict the architecture of the Quad Mount Fabricated Deep Fully Connected Neural Network model that has been suggested.

The main contribution of this paper is to design the Quad Mount Fabricated Deep Fully Connected Neural Network. The supply chain pricing dataset contains 32 features with 10,037 logistic details which are used for logistic pricing prediction as in Eq. (1) where S denotes the supply chain dataset independent variables and P denotes the target logistic pricing.

$$S = \{[s1, s2, s3, \ldots\ldots\ldots\ldots, x31], [P]\} \tag{1}$$

The dataset has been processed with missing values computation. Inferential statistical analysis has been performed to see how the target correlates with the data. Equation (2) explains how to estimate incomplete data after encrypting qualitative

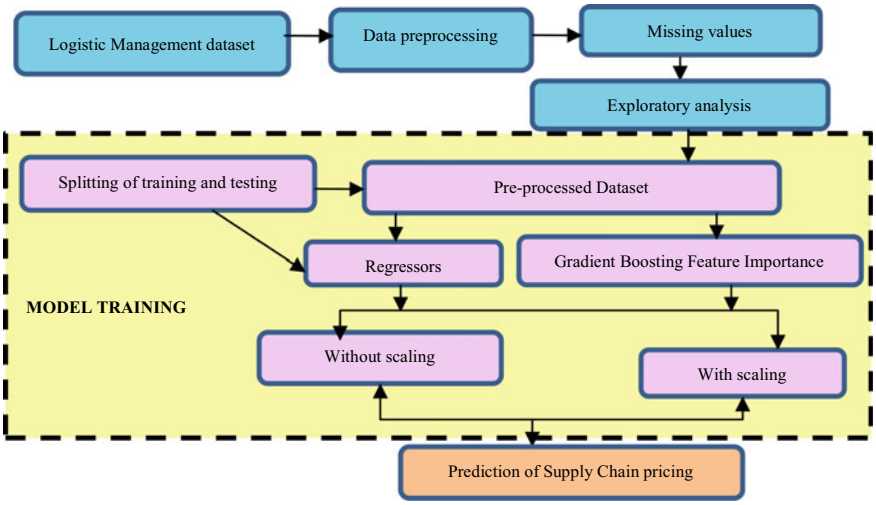


Fig. 1 Research methodology

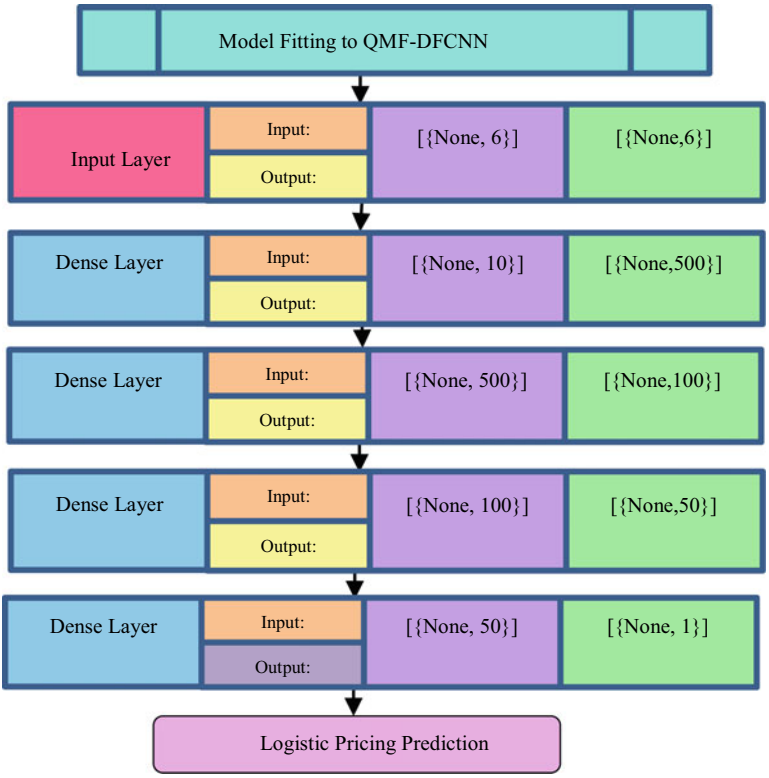


Fig. 2 Model fitting design of QMF-DFCNN

units in the dataset by computing the mean of each input data information for that specific attribute.

$$sR_{ij} = \frac{1}{n} \sum_{k=1}^{m} (sR_{ij})^k \tag{2}$$

Equation (3) shows how to process and analyze the supply chain pricing dataset including null values estimate, and scaling.

$$S' = \frac{1}{X} \sum_{s=1}^{31} S'_s \tag{3}$$

where S' is the processed dataset with complete values and estimation of S from the completed dataset $s = 1, 2, \ldots, 31$ independent features. The average of the estimated variance is used to quantify the imputation variance of attributes. For S'_s from $s = 1, 2, \ldots., 31$ of a complete dataset is shown in (4).

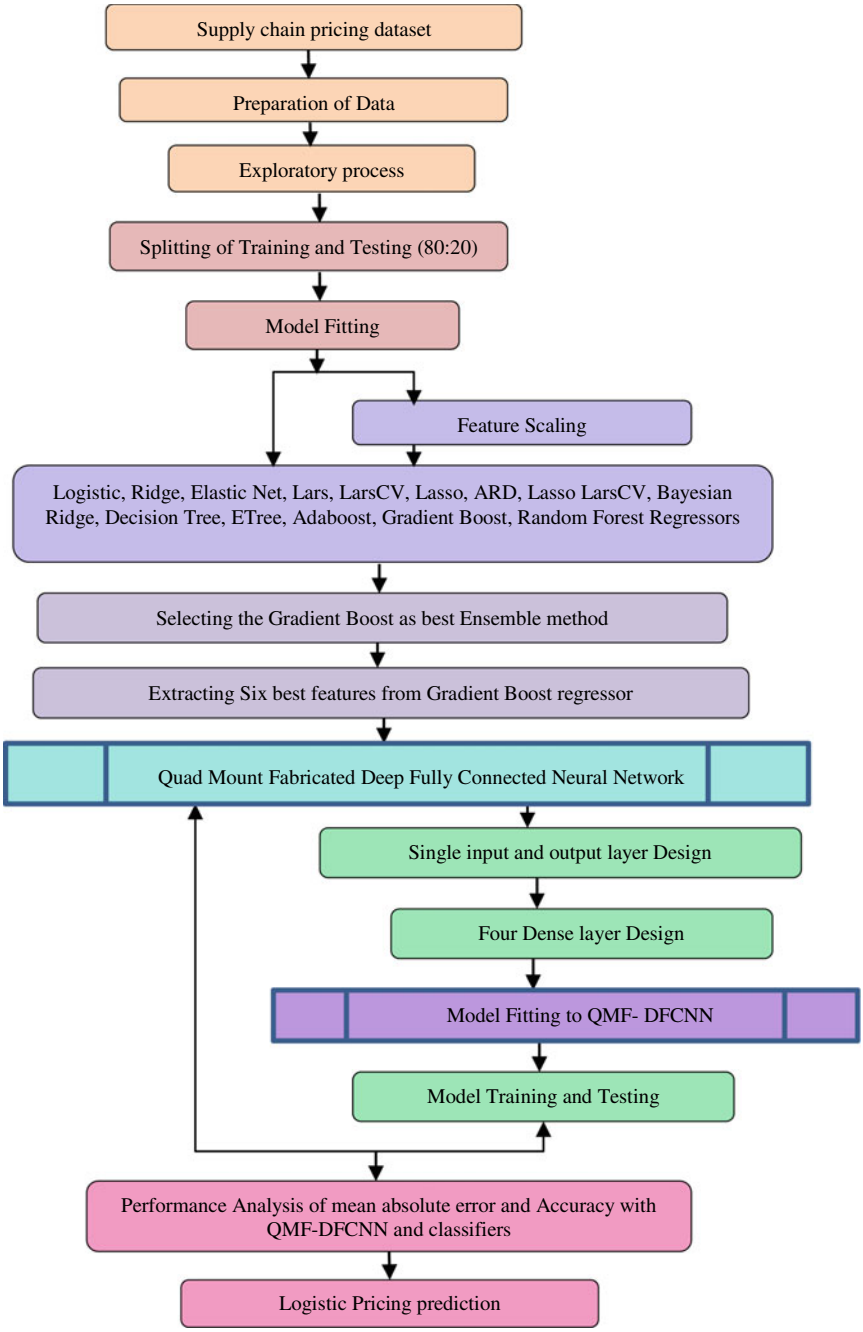


Fig. 3 QMF-DFCNN architecture

$$SW' = \frac{1}{S}\sum_{s=1}^{31} S'_s = \frac{1}{X}\sum_{s=1}^{31} var(S'_s) \tag{4}$$

where SW' is the imputed dataset with no missing values. Between the value "$BV alue$" of the dataset feature variance is calculated with the following (5).

$$BV alue = SW' - \frac{1}{S-1}\sum_{s=1}^{31}(S'_s - var(S_s)) \tag{5}$$

The final data with no missing values having the total variance computed by Eq. (6).

$$EndDataset = SW' + \left(\frac{s+1}{s}\right) \times BV alue \tag{6}$$

Dataset applied to Logistic, Ridge, Elastic Net, Lars, ARD, LarsCV, Lasso, Lasso LarsCV, Bayesian Ridge, Decision Tree, ETree, Adaboost, Gradient Boost and Random Forest Regressors to analyze the performance of logistic price prediction. The dataset was exposed to a gradient boost regressor to extract six important features as in Eqs. (7), (8) and (9) where $\propto$ denotes the learning rate and $Resi_s^p$ denotes residuals.

$$P_s^p = P_s^p + \propto * \frac{\partial \sum_{s=1}^{31}(P_s - P_s^p)^2}{\partial P_s^p} \tag{7}$$

$$Resi_s^p = \left[\frac{\partial \sum_{s=1}^{31}(P_s - P_s^p)^2}{\partial P_s^p}\right]_{Fe(S)=Fe_{s-1}(S)} \tag{8}$$

$$Fe_6(S) = \sum_{s=1}^{6} Fe_{s-1}(S) + P_s(P, Resi_{s-1}) + Resi_s \tag{9}$$

where denotes the top six features from the gradient boost regressor. The gradient boost feature extracted dataset applied to Quad Mount Fabricated Deep Fully Connected Neural Network as in Eqs. (13), (14), (15) and (16) which had designed with single input having six features, an output layer with pricing attribute and four dense layers with 10 nodes each. The input layer contains six as packprice, Unitmeasure, dosageform, product, dosage, brand and site as in Fig. 4.

$$DFCNN = InLayer + 4Dense + \text{OutLayer} \tag{10}$$

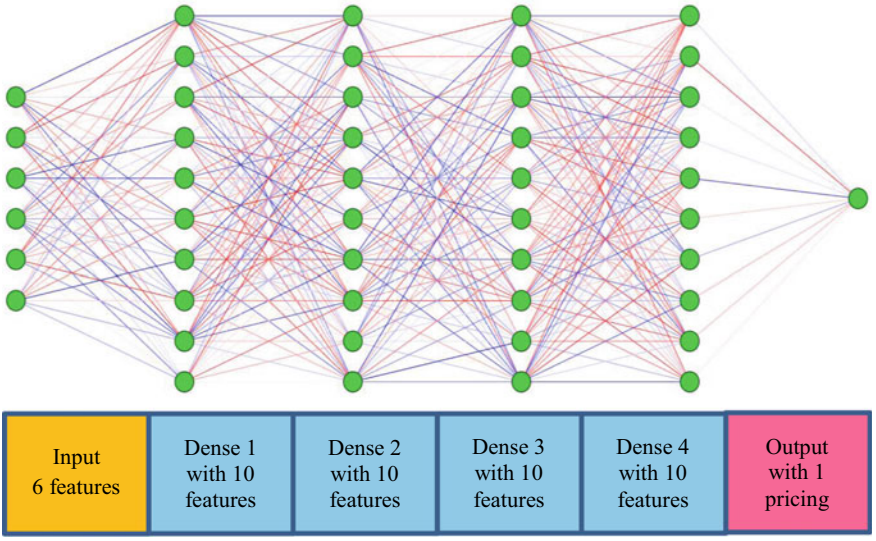


Fig. 4 Layer Design of QMF-DFCNN

$$InLayer_j^{[i]} = \sum_{i=1}^{1} In_{j,i}^{i} \tag{11}$$

$$Dense_j^{[i]} = \sum_{i=1}^{4} Hid_{j,i}^{i} H_i^{i+1} \tag{12}$$

$$OutLayer_j^{[i]} = \sum_{i=1}^{1} Out_{j,i}^{i} \tag{13}$$

The dataset is fitted with proposed QMF-DFCNN and all regressors as in Eq. (14) and the performance is analyzed as in Eqs. (15) and (16).

$$QMFDFCNN = Model\left\{ \begin{matrix} QMFDFCNN(X), \\ Regressors \end{matrix} \right\} \tag{14}$$

The performance metrics utilized for analysis are mean absolute error and Accuracy as shown in Eqs. (15) and (16).

$$MSE = \frac{1}{n}\sum\nolimits_{i=1}^{n} \left| y_{actual} - y_{target} \right|^2 \tag{15}$$

$$RSquared = 1 - \frac{\sum_{s=1}^{n} \left(\widehat{P}_s - P_s \right)^2}{\sum_{s=1}^{n} \left(P_s - \overline{P}_s \right)^2} \tag{16}$$

4 Implementation Setup and Results

The supply chain pricing dataset contains 32 features with 10,037 logistic details which are used for logistic pricing prediction as in Fig. 5. Around an NVidia Tesla V100 Graphics card server workstation, Python was utilized for executing with a total sampling size of 64 and 30 training iterations. Dataset was applied to all Regressors to analyze the performance of logistic price prediction as shown in Tables 1 and 2. The dataset was exposed to a gradient boost regressor to extract six important features as shown in Fig. 5. The gradient boost feature extracted dataset applied to Quad Mount Fabricated Deep Fully Connected Neural Network and all other regressors to analyze the RSquared value and MSE as shown in Table 3 and Fig. 6.

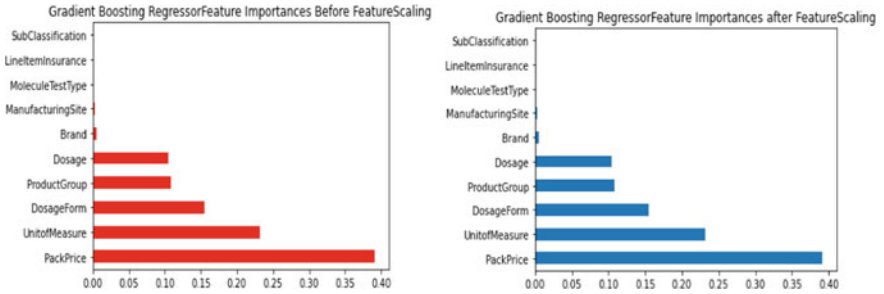


Fig. 5 Top Six Features of Gradient Boost Regressor prior to scaling

Table 1 Metrics of supply chain Dataset prior to scaling

Regressors	EVS	MSE	MAE	RSCORE	Runtime
Linear	0.87214	20.684513	9.82955	0.87214	0.07963
Ridge	0.48990	82.680291	18.94051	0.48891	0.00798
ElasticNet	0.85156	24.014106	10.94664	0.85156	0.08458
Lars	0.82449	28.392886	11.70449	0.82449	0.01562
LarsCV	0.48990	82.80291	18.94051	0.48891	0.00798
Lasso	0.80411	31.624856	11.81200	0.80408	0.01097
LassoLarsCV	0.80411	31.594856	11.81200	0.80408	0.01097
BayesianRidge	0.87234	20.152438	9.83312	0.87234	0.01562
ARD	0.86770	21.4.03153	10.09172	0.86770	0.02713
DTree	0.85791	22.990700	10.21728	0.85788	0.01498
ExtraTree	0.82449	28.392886	11.70449	0.82449	0.01562
Adaboost	0.82449	28.292886	11.70449	0.82449	0.01562
GradientBoost	0.89635	15.909412	1.17142	0.89635	3.16422
RandomForest	0.75046	40.373103	12.05098	0.75043	1.42076

Table 2 Metrics of supply chain dataset prior to scaling

Regressors	EVS	MSE	MAE	RSCORE	Runtime
Linear	0.87214	20.684513	9.82955	0.87214	0.21000
Ridge	0.87213	20.486085	9.82941	0.87213	0.11000
ElasticNet	0.80411	31.694938	11.81200	0.80408	0.14000
Lars	0.82449	28.392886	11.70449	0.82449	0.01562
LarsCV	0.86440	21.938461	10.00649	0.86439	0.12237
Lasso	0.85791	22.990699	10.21727	0.85788	0.01563
LassoLarsCV	0.87177	20.744740	9.83118	0.87177	0.14710
BayesianRidge	0.87209	20.492949	9.82896	0.87209	0.01562
ARD	0.85791	22.390699	10.21727	0.85788	0.01563
DTree	0.80411	31.594856	11.81200	0.80408	0.10024
ExtraTree	0.86185	13.421365	0.39592	0.86183	0.03987
Adaboost	0.84375	21.322127	13.22043	0.86820	2.40376
GradientBoost	0.89635	15.709441	1.17142	0.89635	0.07688
RandomForest	0.75046	40.273103	12.05098	0.75043	1.43943

Table 3 Metrics of supply chain dataset later scaling

Regressors	MSE	RSCORE
Linear	20.684513	0.87214
Ridge	20.486085	0.87213
ElasticNet	31.694938	0.80408
Lars	28.392886	0.82449
LarsCV	21.938461	0.86439
Lasso	22.990699	0.85788
LassoLarsCV	20.744740	0.87177
BayesianRidge	20.492949	0.87209
ARD	22.390699	0.85788
DTree	31.594856	0.80408
ExtraTree	13.421365	0.86183
Adaboost	21.322127	0.86820
GradientBoost	15.709441	0.89635
RandomForest	40.273103	0.75043
Proposed QMF-DFCNN	**9.887654**	**0.98395**

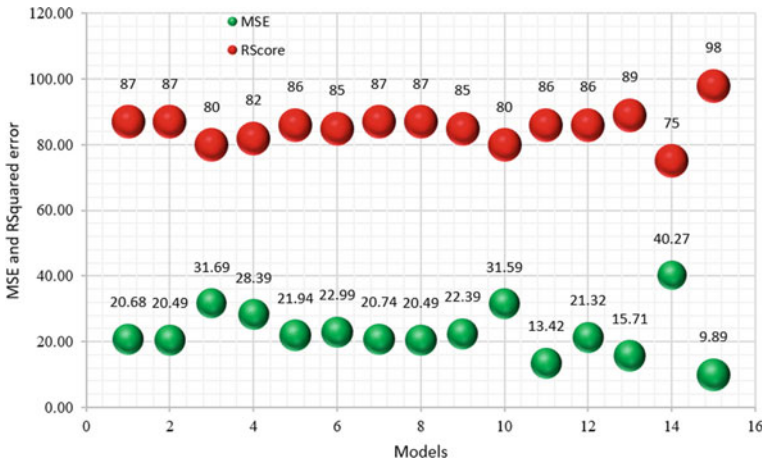


Fig. 6 Metrics analysis of QMF-CNN and regressors

5 Conclusion

This paper examines how well the Quad Mount Fabricated Deep Fully Connected Neural Network model predicts the logistic supply pricing with the least mean squared error and high RSquared value. The top six features from the gradient boost regressor are fitted to the proposed DFCNN model. QMF-DFCNN is proposed with single input and output layer, 4 dense layers and dataset is fitted with proposed model and all other regressors to analyze the performance. Implementation results reveal that proposed Quad Mount Fabricated Deep Fully Connected Neural Network model shows minimum mean squared error of 9.8876544 and a maximum RSquared value of 0.98395 when compared with all other regressor models.

References

1. Liu Q, Chen E, Xiong H, Ge Y, Li Z, Wu X (2014) A cocktail approach for travel package recommendation. IEEE Trans Knowl Data Eng 26(2):278–293
2. Li D, Zhao G, Wang Z, Ma W, Liu Y (2015) A method of purchase prediction based on user behavior log. In: The proceedings of IEEE international conference on data mining workshop (ICDMW), pp 1031–1039
3. Liu Q, Zeng X, Zhu H, Chen E, Xiong H, Xie X (2015) Mining indecisiveness in customer behaviors. In: The proceedings of IEEE international conference on data mining (ICDM), pp 281–290
4. Adomavicius G, Tuzhilin A (2005) Toward the Next generation of recommender systems: a survey of the state-of-the-art and possible extensions. IEEE Trans Knowl Data Eng 6(1):734–749
5. Yin H, Cui B, Zhou X, Wang W, Huang Z, Sadiq S (2016) Joint modeling of user check-in behaviors for real-time point-of-interest recommendation. ACM Trans Inf Syst 35(2):11

6. Liaw A, Wiener M (2002) Classification and regression by random-forest. R News 2(3):18–22
7. Zhu G, Cao J, Li C, Wu Z (2017) A recommendation engine for travel products based on topic sequential patterns. Multimed Tools Appl 76(16):17595–17612
8. Ge Y, Xiong H, Tuzhilin A, Liu Q (2014) Cost-aware collaborative filtering for travel tour recommendations. ACM Trans Inf Syst 32(1):4
9. Xue X, Jia Y, Tang Y (2020) Expressway project cost estimation with a convolutional neural network model. IEEE Access 20:217848–217866
10. Herbsman Z (1986) Model for forecasting highway construction cost. Transp Res Rec 1056:47–54
11. Cheng CA, Lin YC, Chiu HW (2014) Prediction of the prognosis of ischemic stroke patients after intravenous thrombolysis using artificial neural networks. Stud Health Technol Inf 202:115–118
12. Bradley AAP (1997) The use of the area under the ROC curve in the evaluation of machine learning algorithms. Pattern Recognit 30(7):1145–1159
13. Pham XV, Maag A, Senthilananthan S, Bhuiyan M (2020) Predictive analysis of the supply chain management using machine learning approaches: review and taxonomy. In: The proceedings of international conference on innovative technologies in intelligent systems and industrial applications, pp 1–19
14. Bag S, Wood LC, Xu L, Dhamija P, Kayikci Y (2020) Big data analytics as an operational excellence approach to enhance sustainable supply chain performance. Resour Conserv Recycl 153(1):1–10
15. Ivanov D, Dolgui A, Sokolov B (2019) The impact of digital technology and Industry 4.0 on the ripple effect and supply chain risk analytics. Int J Prod Res 53(3):829–846

Machine Learning and Deep Learning Models for Vegetable Leaf Image Classification Based on Data Augmentation



Chitranjan Kumar and Vipin Kumar

Abstract Vegetables are an essential diet for improving nutritional levels in daily living. Therefore, maintaining quality in line with their variety of vegetables is necessary to get a higher yield. Vegetables provide a lot of nutrients, vitamins, and calcium. It is beneficial to our bodies in a variety of ways. To learn about vegetables, first, identify and validate their categorization. As a result, the leaf of the vegetables is used to identify them. This work gathers 25 types of vegetable leaves, totaling 7226 RGB images. After applying data augmentation techniques, images increase from 7226 to 21219 RGB images. In data augmentation techniques, we found that the combination of rotation, flipping, shift, and zoom techniques significantly improves the performance of the Deep learning model. These samples go through the original & augmentation data of the DL & ML model process and train various models. The DL model (resnext50) has the best performance in terms of classification accuracies. The highest test accuracy was 93.08% on original data and 94.89% on augmented data. In comparison, the performance of machine learning models (MLP) is low, where it has been improved from 90.68% on the original data to 92.78% on the augmented data. Using data augmentation in the experiment discovered that data augmentation helps to improve the accuracy of the Resnext50 94.89% approaches.

Keywords Classification · Vegetable plant leaf · Image preprocessing · RGB image · Data augmentation · Deep learning · Machine learning · Computer vision

1 Introduction

Agriculture is critical to our survival and significantly impacts our lives. Vegetables are essential in our lives because they provide a source of energy and help us overcome health issues [1]. The author generated a dataset of 25 classes, each with a minimum

C. Kumar · V. Kumar (✉)
Department of Computer Science and Information Technology, Mahatma Gandhi Central University, Motihari, Bihar, India
e-mail: rt.vipink@gmail.com

© The Author(s), under exclusive license to Springer Nature Singapore Pte Ltd. 2023

S. Chinara et al. (eds.), *Advances in Distributed Computing and Machine Learning*, Lecture Notes in Networks and Systems 660,
https://doi.org/10.1007/978-981-99-1203-2_44

of 250 photos. Images from diverse fields were taken and improved on the system. Each picture is 250 × 250 pixels in size. The photographs of leaves were collected at the marketplace, the farming field, and the Krishi Vigyan Kendra. The colourful images were taken with a 16 MP camera phone. These outcomes were kept as a dataset, and classifiers were employed. A batch image cropper is used to crop, and they are all resized to 250 × 250. Then, to attain the required categorization results, multiple models are applied.

Deep learning and machine learning model are used to identify images [2]. Many leaf characteristics are examined to forecast the vegetable category, including form, size, colour, edges, and veining. Vegetable leaf samples from 25 distinct categories are obtained from various locations, comprising 7226 RGB images. After applying data augmentation techniques, the number of images increases from 7226 to 21219 RGB images. More intricate methods have also been proposed, and various data augmentation techniques have been applied to a vegetable plant leaf dataset [3]. These samples go through the ML model process and train multiple models. Models such as K-Nearest Neighbours (KNN), Linear Regression (L.R.), Decision Tree (D.T.), Support Vector Machine (SVM), Naïve Bayes (N.B.), and Multilayer Perceptron (MLP) are used [4]. And DL models, such as Resnet, Alexnet, Vgg, Squeezenet, Googlenet [5], Shufflenet, Resnext50, and Densenet, are being used to implement the vegetable plant leave classification. After training the models, models are evaluated through Training Accuracy, Training loss, Validation Accuracy, Validation loss, and accuracy [6–9].

2 Literature Review

In this paper [10], Paul Shekonya Kanda, Kewen Xia, and Olanrewaju Hazzan Sansui discussed three technologies for developing a high-accuracy model. The first is a Conditional Generative Adversarial Network, which generates synthetic data; the second is a Convolutional Neural Network, which extracts species features; and the third is a Logistic Regression classifier, which achieves averaging accuracies of 96.1% for about eight datasets, while 99.0–100% on some individual datasets. In another paper [11], Marwan Adnan Jasim and Jamal Mustafa AL-Tuwaijari used the Convolutional Neural Networks (CNN) method to detect plant leaf diseases. They used datasets of leaves from specific plants, such as tomatoes, peppers, and potatoes, because they are common in their native environment. Using the CNN algorithm, they could detect 15 different types of plant leaf diseases. They achieved 98% accuracy, resulting in the accurate and rapid detection of various diseases.

In this, authors [12] have classified 25 categories of herbs. Every category has more than 250 herbal plants' leaves images on it. All the leaf images were collected by the authors manually with the help of a smartphone camera. It is a novel dataset created of its kind by the authors. Then the six classical machine learning algorithms were applied to these 25 categories of datasets to do the training, validation, and testing in the ratio of 6:2:2, respectively. The best classifier is the MLP classifier, which

achieved a test accuracy of 82.51%. In this, authors [13] have classified 25 categories of Flower Plant Leaves. Every category contains more than 250 leaves of images on it. All the leaf images were manually collected by the authors using a mobile camera. Where created of its various kind is novel. Then the six classical machine learning algorithms were applied to the dataset, and the training, validation, and testing is splited in the of 60%, 20%, and 20% respectively, where the best performing classifier is the MLP, which the test accuracy of 89.61% has been achieved.

3 Proposed Methodology

See (Fig. 1).

3.1 Dataset Preparation

Without technical competence, it is impossible to recognize vegetables when the leaves are the essential elements of plants. As a result, finding a way to identify vegetables based on leaf photos utilizing artificial intelligence without technical expertise is critical.

For reading the dataset, an RGB format image is taken and fitted into the model. The picture is resized at the initial step. Images are resized to 250 × 250 pixels. There are 25 vegetable leaf categories with a total of 21219 RGB photos.

Assume I = $\{I_i, y_i\}_{i=1}^{k}$ is a collection of RGB images that have been captured, where I_i denotes that the associated label for each ith image in the dataset $\boldsymbol{y_i} \in Y$, i.e., Y = $\{y_i\}_{i=1}^{k}$ for i = {1,2,3,…..,k} and k represents the number of instances in the dataset. A 2D matrix M $\in \mathbb{R}^{H \times W}$ is been used to represent each sample $I_i \in I$. M_{iJ} denotes the value in the matrix M, i row, and j column. The tensor for deep learning is represented as $T \in \mathbb{R}^{H \times W \times C \times b}$, where H, W, C, *and* b are the height, width, colour channel, and batch size, respectively. The required scaled dataset $\{\mathrm{I}^{\mathrm{t}}{}_i, y_i\}_{i=1}^{k}$ may be constructed using the resized function fx() shown [14] in Eq. 1.

$$I^t = fx(\{I_i, y_i\}_{i=1}^{k}, p \times q) \tag{1}$$

where $p \times q$ is the required resized dimension each image $I_i \in I$.

3.2 Splitting Dataset into the Train, Validation, and Test Set

The dataset must be shuffled in the initial step before the split can be trained, verified, and tested [15]. Let X = $(X_1, \ldots, X_p)$ represents the dataset, while k is the number of samples. It is divided into I^t training, validation, and test datasets, which are denoted

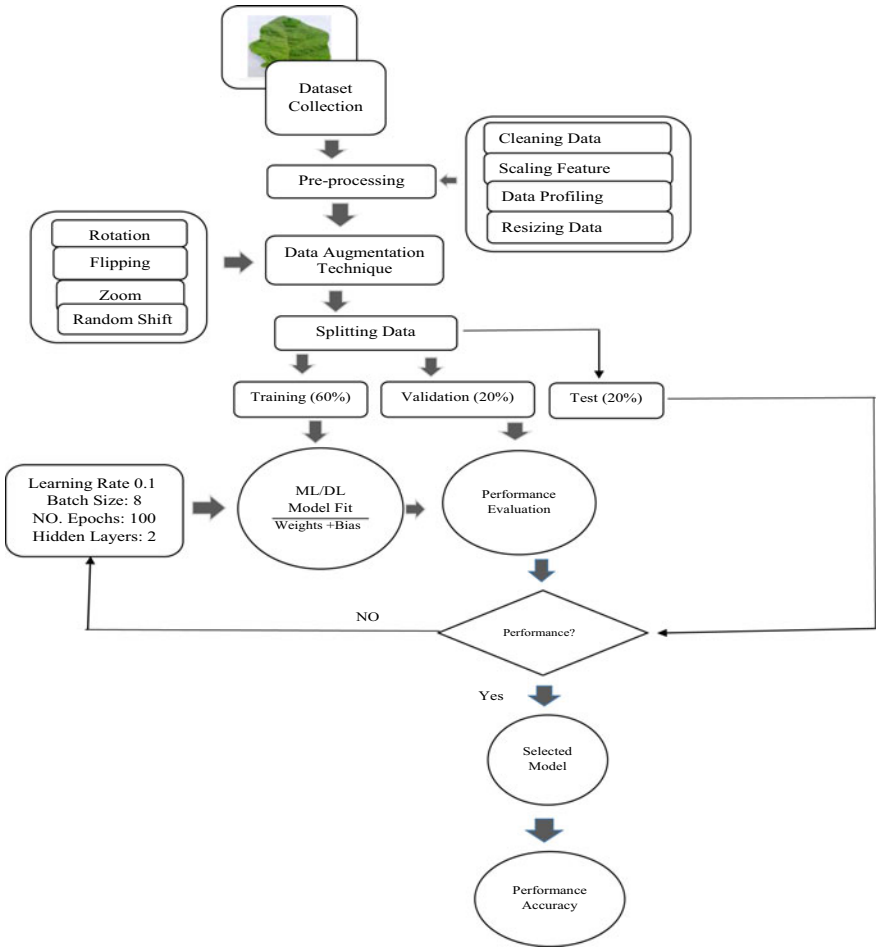


Fig. 1 Flow chart diagram for vegetable leaf image classification using ML/DL Model

as $T_{train} \subset I^t$, $V_{validation} \subset I^t$, and$T_{test} \subset I^t$, where $I^t = T_{train} \cup V_{validation} \cup T_{test}$ and $\{T_{train} \cap V_{validation}\} \cup \{T_{train} \cap T_{test}\} \cup \{V_{validation} \cap T_{test}\} = \emptyset$. . Height (h), width (w), and colour channel (C) of RGB input image are taken into consideration for tensor T. The ith image of the dataset $\{\mathrm{I}^{\mathrm{t}}{}_i \epsilon I^t\}$ which can be represented as$\mathrm{I}^{\mathrm{t}}{}_i = [H, W, C, i]$, where i $\epsilon\{1, 2, 3, \ldots, b\}$ and b is the batch size [16] as shown in Eq. 3.

$$[X_T, Y_T] \tag{2}$$

where X_T is the training part of the data and Y_T is the testing part of the data.

3.3 *Using Deep Learning Model Training and Selections to Analyze Datasets*

A. Optimizing Deep network during training

Batch normalization is required during deep network training in each iteration to ensure efficient convergence of complex deeper networks without overfitting. We can calculate the sample mean and variance of each feature k along the mini-batch axis given a mini-batch x respectively [17, 18].

$$\overline{x}_k = \frac{1}{m}\sum_{i=1}^{m} x_{i,k} \tag{3}$$

$$\sigma_k^2 = \frac{1}{m}\sum_{i=1}^{m}\left(x_{i,k} - \overline{x}_k\right)^2 \tag{4}$$

where m denotes the mini-batch size, we can standardize each feature using these statistics as follows.

$$\overline{x}_k = \frac{x_k - \overline{x}_k}{\sqrt{\sigma_k^2 + \epsilon}} \tag{5}$$

ϵ is a small positive constant used to improve numerical stability.

However, classifying the middle activations reduces the layer's symbolic power. γ and β are two learnable parameters; the scale parameter (γ) and shift parameter (β) is used to maintain the same shape as m for each element in the mini-batch x. Batch normalization (B.N.) appears to stabilize and accelerate training while improving generalization.

$$BN(x_k) = \gamma_k \overline{x}_k + \beta_k \tag{6}$$

The network can regain its original layer representation by setting γ_k to σ_k and β_k to $\overline{x}_k$. So, for a conventional feedforward layer in a neural network. Batch normalization differs between convolutional and fully connected layers. After deploying the batch normalization before the non-linear activation function, the fully connected layer with batch normalization for input x can be written for the output of a neuron y, as shown in the equation.

$$y = \Phi(Wx + b) \tag{7}$$

Batch normalization is done as follows, where W is the weights matrix, b is the bias vector, x is the layer's input, and Φ is an arbitrary activation function. Similarly, batch normalization is applied between the convolution and non-linear activation functions.

$$y = \Phi(BN(Wx)) \tag{8}$$

The bias vector has been eliminated since its influence is neutralized by standardization. Because the network has been normalized, the backpropagation process must be modified to transmit gradients through the mean and variance calculations.

Let $\mathcal{F}$ be a network architecture function with learning rate, hyperparameters, bias, etc. Weights and biases are examples of $\forall f \in \mathcal{F}$ parameters that may be obtained when training on acceptable training datasets. However, finding the ideal function $f^* \in \mathcal{F}$ (with optimal parameters) is essential to obtain the best fit network with the least amount of overfitting/underfitting. The optimum function may be represented as an equation relating to dataset X and labels y.

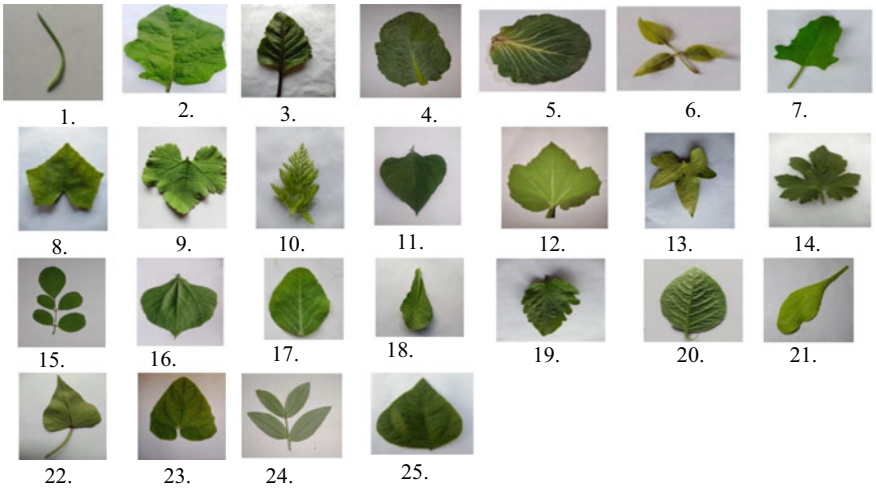
$$f^* = arg \min_f \mu(X, y, f), subject\ to\ f \in \mathcal{F} \tag{9}$$

As a result, f^* is the chosen model for a particular network. A superior architecture $\mathcal{F}'$ is also available as $f^*_{\mathcal{F}} \notin \mathcal{F}$.

4 Experiments

Dataset descriptions: Authors gathered the 25 classes (7226 RGB photos) of vegetable plant leaves photographs, with a minimum of 250 images in each class (Table 1 shows this.) which are 1. (Onion) Allium Cepa:271; 2. (Brinjal) Aubergine:262; 3. (Beetroot) Beta vulgaris:304; 4. (Cauliflower) Brassica Oleracea Var Botrytis:259; 5. (Cabbage) Brassica Oleracea Var Capitate:333; 6. (Sword Bean) Canavala Gladiate:295; 7. (Wild Spinach) Chenopodium:280; 8. (Lvy Gourd) Coccinia Cordifolia:261; 9. (Kumda) Cucurbita Pepo:254;10. (Carrot) Daucus Carota Subsp Sativus:286; 11. (Hyacinth Bean) Lablab Purpures:286; 12. (Pumpkin) Lagenaria Siceraria:315; 13. (Sponge Goud) Luffa Aegyptiaca:259; 14. (Bitter gourd) Momordica Charantia:299; 15. (Drumstick) Oleifera:260; 16. (Lima Bean) Phaseolus Limesis:252; 17. (Green Pea Pisum) sativum:357; 18. (Radish) Raphanus:345; 19. (Tomato) Solanum Lycopersicum:297; 20. (Potato) Solanum Tuberosum:288; 21. (Spinach) Spinacia Oleracea:305; 22. (Samarkand) Sweet potato:337; 23. (Pointed Gourd) T richosanthes Dioica:254; 24. (Horse Beans) Vicia faba:277; and 25. (Yardlong Bean) Vigna Unguicu latassp:319; (S. No., followed by Scientific (Common name) Scientific: The total number of samples in that category).

Table 1 List of all 25 categories of vegetable plants with sample image



5 Result and Analysis

5.1 Description of Results

The classification of several types of vegetable plant leaves images has been performed using DL and ML models. The bar plot shown in Figs. 2 and 3 shows the classification performance based on the comparison of accuracies of original data & augmentation data using machine learning and deep learning models. Figures 4a, b and 5a, b show the characteristics of training accuracy, validation accuracy, and training loss & validation loss, classification of DL models on the Augmentation Dataset corresponding to increasing epochs on the vegetable plant leaf images dataset, where the x-axis and y-axis represent the number of epochs and accuracy or loss respectively of the DL model.

5.2 Analysis of Results

Classification Performance Analysis of DL models: Figure 4a and b shows that the training & validation accuracies have logarithmically increasing nature of line chart on augmented data; they continuously improved their performance until about 9th to 12th epochs. After that, model performance is stabilized by modifications in the succeeding 100th epochs. However, Fig. 5a and b shows that the training and validation losses have a logarithmically decreasing nature of the line chart on augmented data. It shows that the deep learning models become more stable as the

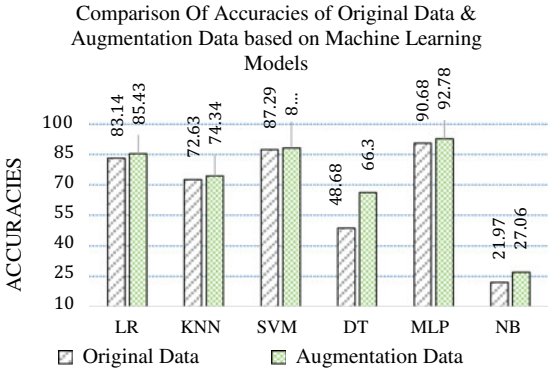


Fig. 2 Bar plot of ML algorithms classification performance based on accuracy measures

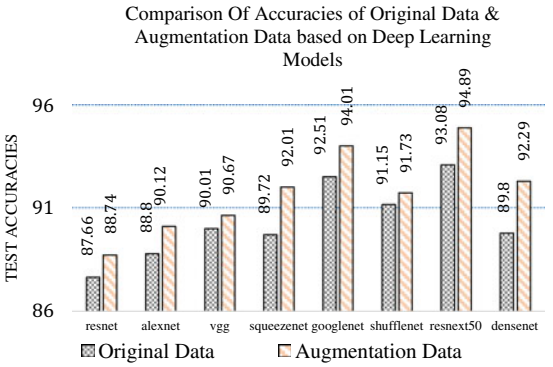


Fig. 3 Bar plot of DL Models classification performance based on accuracy measures

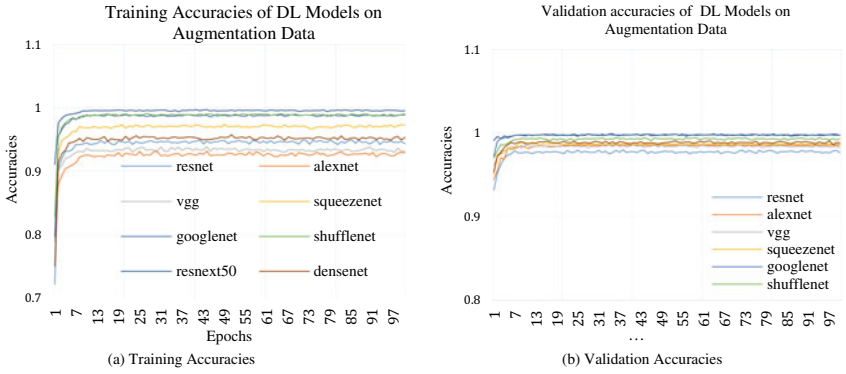
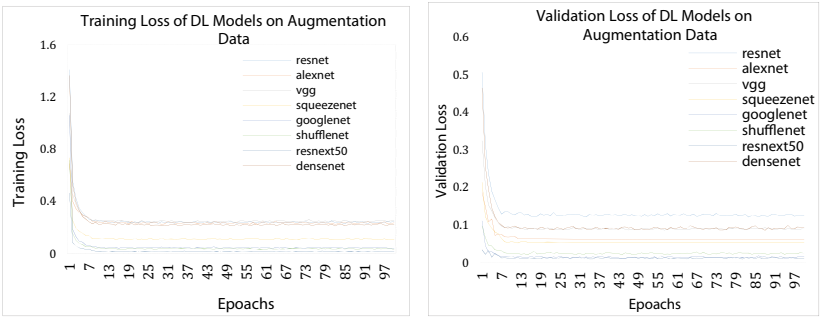


Fig. 4 Classification of accuracies Performance of deep learning model based on augmentation data of vegetable plant leaves



(a) Training Loss (b) Validation Loss

Fig. 5 Classification of loss performance of deep learning model based on augmentation data of vegetable plant leaves

epoch rises. These results show the validation of the dataset's quality. After the model is stable, the Resnext-50 model has the highest test and validation accuracy, while the Resnet model has the lowest.

Classification Comparison of DL Model using a bar plot: As the bar plot in Fig. 3, on the augmentation data, the resnext50 (94.89%) models have higher performance, and the second-highest googlenet (94.01%) model is closest to the resnext50 model than other models, and it is the third highest, on Squeezenet (92.01%), while on the original dataset, Resnext-50 (93.08%), Googlenet (92.51%), and Squeezenet (91.15%) are the first, second, and third highest, respectively. This research suggests that the data augmentation has improved the test accuracy from resnext50 (93.08%) on original data to resnext50 (94.89%) on augmentation data for the DL model, i.e., by 6.92% for Resnext50, 7.49% for Googlenet.

Classification Performance Analysis of ML models: As the bar plot in Fig. 2, by using MLP of Machine learning models, we achieved the highest accuracy of 90.68%, while with SVM (87.29%), we obtained the second-highest, and for L.R. (83.14%) it is the third highest, on the original dataset, while on Augmentation dataset to MLP (92.78%), SVM (88.3%), and L.R. (85.43%) are the first, second, and third highest, respectively, models. These results conclude that the data augmentation has improved the accuracy from 90.68% to 92.78%, i.e., by 9.32% for MLP, 12.71% for SVM, and 16.86% for L.R. So, based on classification accuracy, if we choose Augmentation Data based on Machine Learning Models (MLP), which has the maximum precision, this classifier for the classification of vegetable plants will give the best result compared to other classifiers.

6 Conclusion

This paper classified 25 categories of vegetable plant leaves images. Firstly, the shape feature is extracted to do the classification. Secondly, Vegetable leaf images are magnified by data augmentation techniques. Augmentation data of the DL & ML model have been applied to get the classification report. Based on these results, the analysis has been drawn through a bar plot, training & validation accuracies, training & validation loss, accuracy, and heat map. This study shows that using a deep learning model rather than machine learning techniques significantly improves classification accuracy. The training, validation, and test accuracy on augmented data for the Resnext50 model is the highest, and the training and validation loss is the lowest. The performance of resenext50-94.89% (on the data augmentation) has better classification performance accuracy than the other ML & DL model. As a result, this is the best model overall among all eight DL models for vegetable plant leaf image categorization.

References

1. Liu J, Yang S, Cheng Y, Song Z (2018) Plant leaf classification based on deep learning. In: Chinese automation congress (CAC). IEEE, pp 3165–3169
2. Huang KW, Lin CC, Lee YM, Wu ZX (2019) A deep learning and image recognition system for image recognition. Data Sci Pattern Recognit pp 1–11
3. Perez L, Wang J (2017) The effectiveness of data augmentation in image classification using deep learning. arXiv preprint
4. Orrù PF, Zoccheddu A, Sassu L, Mattia C, Cozza R, Arena S (2020) Machine learning approach using MLP and SVM algorithms for the fault prediction of a centrifugal pump in the oil and gas industry. Sustainability, p 4776
5. Singla A, Yuan L, Ebrahimi T (2016) Food/non-food image classification and food categorization using pre-trained googlenet model. In: Proceedings of the 2nd international workshop on multimedia assisted dietary management, pp 3–11
6. Padmavathi K, Thangadurai K (2016) Implementation of RGB and grayscale images in plant leaves disease detection–comparative study. Indian J Sci Technol 1–6
7. Agarwal A, Sharma P, Alshehri M, Mohamed AA, Alfarraj O (2021) Classification model for accuracy and intrusion detection using machine learning approach. Peer J Comput Sci e 437
8. Bambil D, Pistori H, Bao F, Weber V, Alves FM, Gonçalves EG, de Alencar Figueiredo LF, Abreu UG, Arruda R, Bortolotto IM (2020) Plant species identification using color learning resources, shape, texture, through machine learning and artificial neural networks. Environ Syst Decis 480–484
9. Koklu M, Unlersen MF, Ozkan IA, Aslan MF, Sabanci K (2022) A CNN-SVM study based on selected deep features for grapevine leaves classification. Measurement 110425
10. Kanda PS, Xia K, Sanusi OH (2021) A deep learning-based recognition technique for plant leaf classification. IEEE Access 162590–162613
11. Jasim MA, Al-Tuwaijari JM (2020) Plant leaf diseases detection and classification using image processing and deep learning techniques. In: 2020 international conference on computer science and software engineering (CSASE). IEEE, pp 259–265
12. Kumar G, Kumar V, Hritik AK (2022) Herbal plants leaf image classification using machine learning approach. In: International conference on intelligent systems and smart infrastructure (ICISSI-2022), CRC Press, Taylor & Francis Group

13. Aman BK, Kumar V (2022) Flower leaf classification using machine learning techniques. In: Third international conference on intelligent computing, instrumentation and control technologies (ICICICT-2022). IEEE Explore
14. Brownlee J (2020) Data preparation for machine learning: data cleaning, feature selection, and data transforms in Python. Mach Learn Mastery
15. Tan J, Yang J, Wu S, Chen G, Zhao J (2021) A critical look at the current train/test split in machine learning. arXiv:2106.04525
16. Radiuk PM (2017) Impact of training set batch size on the performance of convolutional neural networks for diverse datasets
17. Arora S, Li Z, Lyu K (2018) Theoretical analysis of auto rate-tuning by batch normalization. arXiv:1812.03981
18. Laurent C, Pereyra G, Brakel P, Zhang Y, Bengio Y (2016) Batch normalized recurrent neural networks. In: IEEE international conference on acoustics, speech and signal processing (ICASSP). IEEE, pp 2657–2661. (Mar 2016).

Deep Fake Generation and Detection



Shourya Chambial, Rishabh Budhia, Tanisha Pandey, B. K. Tripathy, and A. Tripathy

Abstract Nowadays, a lot of fake videos and images are made with the help of various software and new AI technologies but leave few hints of manipulation. There are a lot of unethical ways the videos can be used to threaten, fight, or create panic among people. It is important these days people make sure that such ways are not used to make fake videos. An AI-based technique for the synthesis of human images is called Deep Fake. They are created by combining and superimposing existing videos onto source videos. We will develop a system that uses a convolutional neural network (CNN) and Long Short-Term Memory (LSTM) for the extraction of frame-level features. After this these features are used to train the dataset which learns to classify a fraud and original video. A dataset with thousand videos will be used to detect the originality. With the help of simple architecture, we can achieve this. The main intention of this paper is to detect deep fake videos and eradicate problems like fake news, financial fraud, and malicious hoaxes.

Keywords Fake video · Convolutional neural network · CNN · RNN

1 Introduction

In a nutshell, deep fakes (from "deep reading" and "lies") are made with techniques that can elevate targeted facial images into a source video to make a specific video of a person doing or saying things. The source person does it. This creates a category of deep fakes, which is a face swap. In a broader sense, deep fakes are intricately crafted content that can also fall into two other categories, namely, synchronizing lips with a puppet master. Lip-sync deep fakes tell that videos have been tampered with to make the oral movement consistent with sound recording. Puppet-master deep fakes

S. Chambial · R. Budhia · T. Pandey · B. K. Tripathy (✉)
School of Information Technology and Engineering, VIT, Vellore, TN 632014, India
e-mail: tripathybk@vit.ac.in

A. Tripathy
Carnegie Mellon University, 5000 Forbes Ave, Pittsburgh, PA 15213, USA

© The Author(s), under exclusive license to Springer Nature Singapore Pte Ltd. 2023

S. Chinara et al. (eds.), *Advances in Distributed Computing and Machine Learning*,
Lecture Notes in Networks and Systems 660,
https://doi.org/10.1007/978-981-99-1203-2_45

include animated videos that follow the facial expressions, eye movement, and head of the other person (master) sitting in front of the camera.

While some deep fakes can be created with normal visual effects or computer imagery methods, the latest common basic method of creating deep fake deep learning models such as autoencoders and competing for production networks is widely used in the field of computer vision. These models are used to assess a person's facial expressions and movements and to combine images of another person's face making similar expressions and movements. Deepfake methods generally need large amounts of graphic data to train models to create real photos and videos.

A neural network (NN) contains neurons arranged in vertical columns called layers such that there are connections between neurons of different layers [1]. A NN may contain an input layer one output layer and some intermediate layers called hidden layers. Data are inserted through the input neurons to the constructed neural network. The processing of data mimics the functionalities of the human brain. The NNs with several hidden layers are termed Deep Neural Networks (DNN) [2]. The number of hidden layers is decided by the designer depending on the application.

A convolutional neural network (CNN) is the most commonly used DNN [3]. In CNN, the hidden layers first read the inputs from the first layer and then apply a convolution mathematical operation on the input values instead of the matrix multiplication which is used in the case of basic NN. It uses nonlinear activation functions. There are other variants of DNN like Recurrent Neural Networks (RNN).In RNNs connections between nodes form a directed or undirected graph along a temporal sequence. This allows RNNs to exhibit temporal dynamic behaviour. LSTM is a type of RNN that handles long-term dependencies [4]. LSTM contains feedback connections to learn the entire sequence of data. LSTM has been applied to many fields based on time series data such as classifying, processing, and making predictions. Among all the different techniques used for classifying the videos as fake or real, the one that uses CNN and LSTM has proved to be more accurate.

The rising complexity of smartphone cameras and the widespread availability of high-speed internet connections have expanded the reach of social media and media-sharing portals, making it easier than ever to create and transmit digital videos. Deep learning has become so powerful that as a result of increasing processing power it was considered unachievable only a few years ago. As is the case with any disruptive technology, this has resulted in the emergence of new issues. So-called "Deepfake" generated by adversarial deep generative networks are capable of manipulating video and audio recordings. Spreading the deep fake via social media channels has become more widespread, resulting in spamming and speculating on incorrect information. These forms of deep fake will be heinous and will result in frightening and misleading the general public. Deep fake detection is critical for resolving such a problem. Thus, we present a novel deep learning-based strategy for accurately discriminating between AI-generated false videos (deep fake Movies) and actual videos. It is critical to building technology capable of detecting forgeries so that the deep fake can be discovered and prevented from spreading across the internet.

Our work aims to develop a robust and efficient model to help reduce the menace caused by malicious users who try to take benefit of online and open source images

for unethical activities and try to malign a person's image, also the aim is to reduce the false information spread by these fake videos of which the general public fall prey to.

Section 2describes the existing works related to our work, Sect. 3 presents the proposed methodology including sample dataset, the architectural diagram, flow diagram of the system and explanation of the algorithm proposed. Section 4 comprises of experimental results and the conclusions drawn basing upon these results. Section 5 is devoted to provide outline of some works which can be carried out as further enhancement of this piece of work.

2 Related Work

Two network architectures were provided in [5] for detecting forgeries efficiently and accurately. It is established experimentally that their method has an average detection rate of 95% for Face2Face videos and 98% for Deep fake videos during the real conditions of diffusion. While the present facial image manipulation methods such as DeepFakse, FaceSwap, Face2Face, and FaceSwap show visually impressive results, it was shown in [6] that these fake videos have a chance to be detected by properly trained forgery detectors. They focus on the influence of compression on the detectability of state-of-the-art manipulation methods, by proposing a standardized benchmark used for follow-up work. In [7] considered the ML classifier method XGBoost with summarization by using a confusion matrix. It is shown here that a DNN approach further improves the classification result by precision, recall, and accuracy as compared to the XGBoost. In an attempt [8] shows to improve upon the existing DL methods, which were not clear on the number of layers to be used and which architecture is best for detection. The test analysis of the method proposed shows an accuracy of 96.6% on DF-TIMIT datasets and 84.4% on DFDC datasets. An approach adopted in [9] which applies to large datasets shows an accuracy of 94.21% on the CelebDf Dataset. However, this method is not suitable for videos with fake audio, which was never considered a factor in the videos. A study that compares the existing algorithms till that time was carried out in [10] to understand the effects of DL in Deepfake detection. A study in [11] highlights the problem of forgery and the spread of false information. CNN and GAN are the two main methods used here for comparison [12–14]. A method was designed in [15] to expose Deep Fakes based on the mismatch between directions of different face regions. Further distortion and blurring are required to match and fit the fake face with the source video's background and surrounding context. An algorithm in this direction was proposed in [16] with a successful detection rate of over 90.5%. In [17] Deep fakeStack, a deep ensemble learning technique was presented for detecting manipulated movies, as a solution to the issues provided by Deep fake multimedia. It beats other classifiers in detecting Deep fake, with an accuracy of 99.65% and an AUROC score of 1.0. A temporal aware system was presented in [18] to detect fake videos. This can easily guess if a video has been subject to any manipulation automatically. In [19] a study on

temporal features across video frames was studied and also shallow classifiers for the fake video were detected. A deep fake discovery method based on deep learning was proposed in [20]. A model was developed which extracts various features and variables learned through image analysis. The features learned in this step may be used for further analysis to determine deep fakes. It was observed in [21] that among all the different techniques for the detection of Deep fake video CNN and LSTM have proved to be accurate. The intention was to develop a robust DL-based Deep fake detection method.

A deep fake discovery method based on deep learning was provided in [22]. It scans and analyzes in-depth counterfeit videos and produces a good level of accuracy background.

2.1 Generic Overview

The results of experiments demonstrate that the audio-visual approach is not able to detect face swapping, generated using GANs can match speech to lip movements. So, only image-based approaches are effective to detect Deepfake videos. Only the IQM+SVM system has an equal error rate (EER) of around 8.9% while other methods proposed have shown EERs up to 40%. It means that more advanced techniques need to be discovered to improve further. The inference time of these models is pretty large as these are large models thus a need for smaller models is felt.

The availability of Deep Fakes of high quality with proper classification of original data is lacking. There is a strong need for these to train the supervised model. Another drawback is the incompatibility of the system available with the high-tech detection techniques being used as they require premium graphics cards, high memory, and interfaces that support machine learning packages. The key difference in performance between different learners or classifiers is their model size; Deepfake Stack's model size is very huge, and if it is not carefully designed, it will result in overfitting.

3 Proposed Methodology

3.1 Sample Dataset

FaceForensics ++ is a forensics database that includes the first 1000 video sequences used in four face-off modes: Deepfakes, Face2Face, FaceSwap, and NeuralTextures. The data is taken from 977 YouTube videos, and all videos contain face-to-face tracking without locking which allows automatic interference methods to make a real impression.

3.2 *The Architecture of the Proposed Method*

High level–This is the overview of the system. Here we take both real and fake images, and we generate fake images using a generator and then use a discriminator to discriminate between fake and real images. The high-level diagram is shown in Fig. 1 and Fig. 2.

Low-level–This is a more advanced system design where we design each component in detail. In layman's terms in our project, we will take a dataset, then do pre-processing and then make a model and train it and finally, we will test it and produce the output. The low-level diagram is shown in Figs. 3 and 4.

The dataset is face forensics. We will take the dataset folder, pre-process it and then make a model. The initial state of the model is stored in a pickle file. Then we have a network folder (main working and detection using CNN and LSTM). We

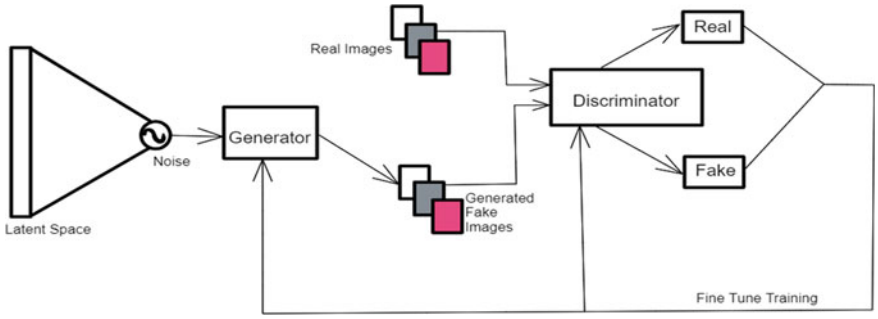


Fig. 1 High-level diagram

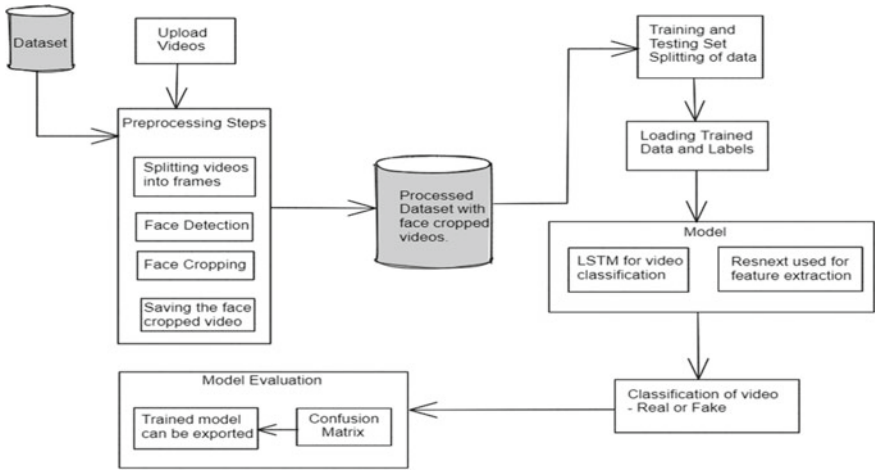


Fig. 2 High-level diagram depicting the flow of events

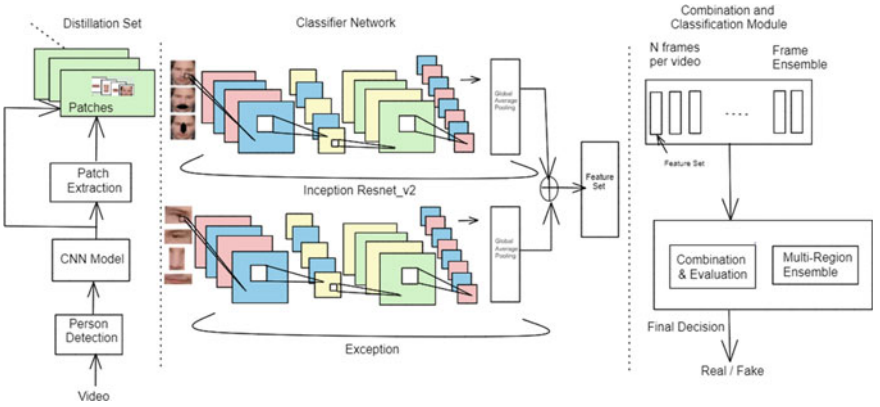


Fig. 3 Low-level diagram

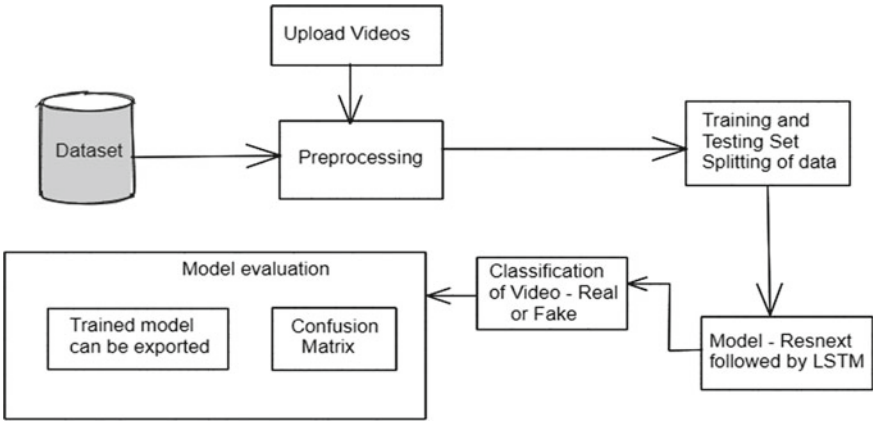


Fig. 4 Low-level diagram depicting the flow of events

involved reference to both xception and models.py. Finally, testing is done by using CNN and producing the output.

3.3 First Order Motion

The whole process is divided into two parts: Motion Extraction and Generation. As a source image entry and driving video is used. Motion extractor uses an autoencoder to get key points and deliver a first-order motion presentation that combines fewer key points and corresponding local changes. This, along with the driving video, is used to produce a dynamic visual flow and a closing map with a dynamic moving

network. Then the output of the dense motion network and the source image using a generator to provide the target image.

3.4 Long Short-Term Memory (LSTM)

The common architecture of LSTM consists of (1) an input gate; (2) forget gate; (3) and an output gate. The cell state is long-term memory that remembers values from previous intervals and stores them in the LSTM cell. First, the input gate is responsible for selecting the values that should enter the cell state. The forget gate is reasonable for determining which information is to forget by applying a sigmoid function, which has a range of [0, 1]. The output gate determines which information in the current time should be considered in the next step.

3.5 Flow Diagram

The flow diagram is shown in Fig. 5.

3.6 Explanation of Algorithm

- The whole algorithm of this model is divided into two modules where we have taken the first part to be the generation of deep fake. And the second part is the detection of deep fake which is done by a self-made model using CNN (Convolutional Neural Network) and LSTM (Long Short-Term Memory).

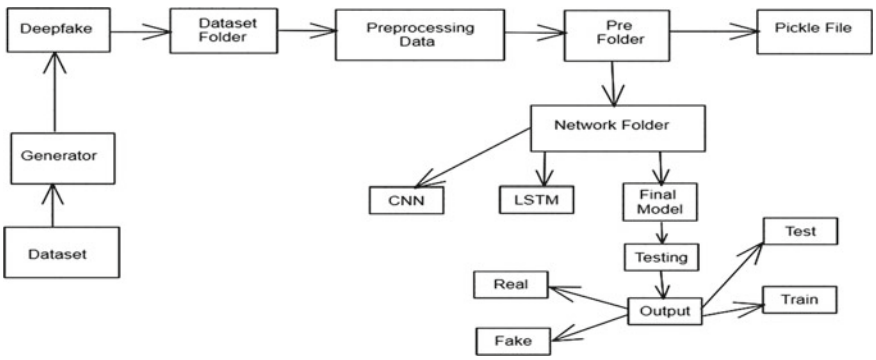


Fig. 5 Flow diagram

- We started with our dataset and imported it into our code. The dataset used in our algorithm can be found in the dataset processing module which is responsible for managing data transforms.
- The network module contains the main working of our algorithm. The CNN implementation is executed in Xception.py. The LSTM implementation can be found in Models.py. The final model is executed using mesonet which contains reference xception and model.
- The pre-module contains a pickle file where it preserves the file after the model is trained. It remembers what the model has learned which saves our time to train the model again and again.
- The detection module is the final part which is the accumulation of all the networks and models.
- Subsequently, training and testing are performed, and the output is generated regarding the similarity score after the detection of deep fake is done on every frame of the dataset.

4 Results and Conclusion

The method described in this paper is a way to identify deep fake pictures utilizing the sign of the source to highlight irregularity inside the manufactured pictures. It depends on the theory that the images distinct by source features can be protected and removed in the wake of going through best-in-class deep fake age processes. The work here presents a smart portrayal of the learning approach, known as pair-wise self-consistency learning (PCL), used for preparing Convolutional Networks to separate the origin highlights and distinguish bogus pictures. It is integrated by a combination approach, called irregularity picture generator (I2G), to result in clear clarified preparing information to PCL.

To obtain the results we are analyzing 30 frames per second to detect whether the video is real or fake. We have taken a dropout value of 0.5 which is a similarity score where below 0.5 is considered a fake video and above 0.5 is considered real.

The evaluation metrics of the Xception model are shown in Table 1 respectively.

Table 1 Evaluation metrics of Xception model

	Xception			
	Precision	Recall	F1-score	support
0 (FAKE)	0.98	0.99	0.98	712
1 (REAL)	0.97	0.91	0.94	185
Accuracy			0.98	897
Macro avg	0.97	0.95	0.96	897
Weighted avg	0.98	0.98	0.98	897

Working of Xception is as follows. Xception is a very efficient deep learning model that depends on the following which includes Depthwise Separable Convolution As in ResNet, there are shortcuts between Convolution blocks. The architecture of Xception is made up of Depthwise Separable Convolution blocks and Maxpooling, all of which are coupled via shortcuts in the same way as ResNet implementations. A Pointwise Convolution does not follow the Depthwise Convolution in Xception; instead, the sequence is inverted. The pseudocode for Xception is as follows. All the necessary layers must be imported. Necessary functions must be written for the Conv-BatchNorm block and SeparableConv- BatchNorm block. For each of the three flows (Entry, Middle, and Exit), write a separate function followed by utilizing these features to create the full model.

The method described in this paper is a way to identify deep fake pictures utilizing the sign of the source to highlight irregularity inside the manufactured pictures. It depends on the hypothesis that the images distinct by source features can be protected and removed in the wake of going through best-in-class deep fake age processes. The work here presents a smart portrayal of the learning approach, called pairwise self-consistency learning (PCL), for preparing ConvNets to separate these source highlights and distinguish deep fake pictures. Itisjoinedbyanotherpicturecombination approach, called irregularity picture generator (I2G), to give clearand clarified preparationinformationtoPCL. Exploratory outcomes on seven well-known datasets show that their models work on finding the middle value of AUC over the condition of the craftsmanship from 96.45% to 98.05% in the in-dataset assessment and from 86.03 to 92.18% in the cross-dataset assessment.

In this research, we have introduced a CNN-based program to automatically detect fake videos. Our experimental results using a large collection of converted videos have shown that using a simple LSTM format we can accurately predict whether a video has ever been deceived or not within 2 s of video data. We believe that our work provides a powerful first line of defense to detect false media created using the tools described in the project. We demonstrate how our system can achieve competitive results in this work while using a simple pipeline design. At the same time, we were also able to determine if the lower part of the video was interrupted or not. In future work, we plan to explore how to increase the strength of our system against deceptive videos using subtle techniques during training. This work serves as an effective initial line of defense in detecting bogus media made with online technologies. In addition, the model can attain competitive output by adopting a pipeline design is also demonstrated. In future, it is possible to use subtle tactics during training to see how the system can be strengthened against false films. The experimental analysis demonstrates that the enhancements have greatly enhanced deepfake detection results with a maximum precision, recall and f1-score of 0.98 respectively. Simultaneously, because video forgery technology and the caliber of video are still in the development phase, it will be possible to ameliorate the proposed model.

5 Future Work

- We can join the generation of video with the detection part where it can directly make use of the generated video.
- We would want the implementation speed to be better with some modifications in the code and with the introduction of a few more methods.
- We would want to develop a user-friendly platform with the integration of our models which can be useful for crime branches of our country for easy detection of deepfakes.

References

1. Tripathy BK, Anuradha J (2015) Soft computing-advances and applications. Cengage Learning publishers, New Delhi. ASIN: 8131526194, ISBN-10: 9788131526194
2. Bhattacharyya S., Snasel V, Hassanian AE, Saha S, Tripathy BK (2020) Deep learning research with engineering applications. De Gruyter Publications. ISBN: 3110670909, 9783110670905. https://doi.org/10.1515/9783110670905
3. Maheswari K, Shaha A, Arya D, Tripathy BK, Rajkumar R (2020) Convolutional neural networks: a bottom-up approach. In: Bhattacharyya S, Hassanian AE, Saha S, Tripathy BK (eds) Deep learning research with engineering applications. De Gruyter Publications, pp 21–50. https://doi.org/10.1515/9783110670905-002
4. Adate A, Tripathy BK (2019) S-LSTM-GAN: shared recurrent neural networks with adversarial training. In: Kulkarni A, Satapathy S, Kang T, Kashan A (eds) Proceedings of the 2nd international conference on data engineering and communication technology. Advances in intelligent systems and computing, vol 828, Springer, Singapore, pp 107–115
5. Afchar D, Nozick V, Yamagishi J, Echizen I (2018) Mesonet: a compact facial video forgery detection network. In: 2018 IEEE international workshop on information forensics and security (WIFS). IEEE, pp 1–7
6. Tejaswini R, Rakesh K, Kanta MVM, Reddy MM, Tejaggna M (2021) Deepfake detection. Iconic Res Eng J 5(1):182–187
7. Kaliyar RK, Goswami A, Narang P (2021) DeepFake: improving fake news detection using tensor decomposition-based deep neural network. J Supercomput 77(2):1015–1037
8. Almars AM (2021) Deepfakes detection techniques using deep learning: a survey. J Comput Commun 9(5):20–35
9. Shende A (2021) Using deep learning to detect deepfake videos. Turkish J Comput Math Educ (TURCOMAT) 12(11):5012–5017
10. Pan D, Sun L, Wang R, Zhang X, Sinnott RO (2020) Deepfake detection through deep learning. In: 2020 IEEE/ACM international conference on big data computing, applications and technologies (BDCAT). IEEE, pp 134–143. https://doi.org/10.1109/BDCAT50828.2020.00001
11. Nasar BF, Sajini T, Lason ER (2020) Deepfake detection in media files-audios, images and videos. In: 2020 IEEE recent advances in intelligent computational systems (RAICS). IEEE, pp 74–79. https://doi.org/10.1109/RAICS51191.2020.9332516
12. Bhardwaj P, Guhan T, Tripathy BK (2021) Computational biology in the lens of CNN, studies in big data. In: Roy SS, Taguchi Y-H (eds) Handbook of machine learning applications for genomics, (Chapter 5), vol 103. ISBN: 978-981-16-9157-7. (496166_1_En)

13. Tripathy BK, Parikh S, Ajay P, Magapu C (2022) Brain MRI segmentation techniques based on CNN and its variants. In: Chaki J (ed) Brain tumor MRI image segmentation using deep learning techniques, (Chapter-10). Elsevier Publications, pp 161–182. https://doi.org/10.1016/B978-0-323-91171-9.00001-6
14. Prabhavathy P, Tripathy BK, Venkatesan M (2022) Analysis of diabetic retinopathy detection techniques using CNN models. In: Mishra S, Tripathy HK, Mallick P, Shaalan K (eds) Augmented intelligence in healthcare: a pragmatic and integrated analysis. Studies in Computational Intelligence, vol 1024. Springer, Singapore. https://doi.org/10.1007/978-981-19-1076-0_6
15. Ivanov NS, Arzhskov AV, Ivanenko VG (2020) Combining deep learning and super-resolution algorithms for deep fake detection. In: 2020 IEEE conference of Russian young researchers in electrical and electronic engineering (EIConRus). IEEE, pp. 326–328. https://doi.org/10.1109/EIConRus49466.2020.9039498
16. Younus MA, Hasan TM (2020) Effective and fast deepfake detection method based on haar wavelet transform. In: 2020 international conference on computer science and software engineering (CSASE). IEEE, pp 186–190
17. Rana MS, Sung AH (2020) Deepfakestack: a deep ensemblebased learning technique for deepfake detection. In: 2020 7th IEEE international conference on cyber security and cloud computing (CSCloud)/2020 6th IEEE international conference on edge computing and scalable cloud (EdgeCom). IEEE, pp 70–75
18. Güera D, Delp EJ (2018) Deepfake video detection using recurrent neural networks. In: 15th IEEE international conference on advanced video and signal based surveillance (AVSS). IEEE, pp 1–6
19. Nguyen TT, Nguyen CM, Nguyen DT, Nguyen DT, Nahavandi S (2019) Deep learning for deepfakes creation and detection: a survey. arXiv:1909.11573
20. Simonyan K, Zisserman A (2015) Very deep convolutional networks for large–scale image recognition. In: ICLR 2015. arXiv:1409.1556v6. [cs.CV]
21. Yang X, Li Y, Lyu S (2019) Exposing deep fakes using inconsistent head poses. In: ICASSP 2019-IEEE international conference on acoustics, speech and signal processing. IEEE, pp 8261–8265
22. Guera D, Delp EJ (2018) Deepfake video detection using recurrent neural networks. In: 15th IEEE international conference on advanced video and signal based surveillance (AVSS), Auckland, New Zealand. IEEE, pp 1–6

Similarity-Based Recommendation System Using K-Medoids Clustering



Aryan Pathare, Burhanuddin Savliwala, Narendra Shekokar, and Aruna Gawade

Abstract Most of the consumer applications of today make use of recommendation systems. From movie recommendations by Netflix to dating recommendations by applications like Tinder, recommendation systems have become ubiquitous in most of today's commercial applications. Most commonly used recommendation techniques require a large amount of data, either as item ratings or ratings given by similar users. This vast data requirement makes it difficult to apply these techniques to cases where the required data is unavailable. Hence, there is a need for recommendation systems that can function with a small amount of data and provide practical recommendations. Empirically it has been found that people with similar attributes are likely to have similar tastes. Clustering can be used to group people with similar attributes. However, such clustering should be performed online as new users are continuously added to the application's user base. This paper proposes a similarity-based recommendation approach that uses an online clustering technique to form clusters of people with similar attributes. Online indicates dynamic data, and re-clustering involves computing clusters for all the points in the system again, which is computationally expensive, contributing to most of the time taken by the algorithm. The proposed approach performs selective re-clustering, thereby enabling faster recommendations and reduced load on servers.

Keywords Recommendation · Clustering · Kmedoids · Similarity

A. Pathare (✉) · B. Savliwala · N. Shekokar · A. Gawade
DJ Sanghvi College Of Engineering Mumbai, Mumbai, India
e-mail: aryanpathare281@gmail.com

B. Savliwala
e-mail: burhanuddinsavliwala.work@gmail.com

N. Shekokar
e-mail: narendra.shekokar@djsce.ac.in

A. Gawade
e-mail: aruna.gawade@djsce.ac.in

© The Author(s), under exclusive license to Springer Nature Singapore Pte Ltd. 2023

S. Chinara et al. (eds.), *Advances in Distributed Computing and Machine Learning*,
Lecture Notes in Networks and Systems 660,
https://doi.org/10.1007/978-981-99-1203-2_46

1 Introduction

Recommendation algorithms are created to personalize the user experience by suggesting items and movies depending on various characteristics. Recommendation systems handle such information by considering only the most relevant attributes from the user's input. These systems forecast the goods consumers will be most interested in and, therefore, likely to buy.

Clustering divides the input points into groups, so the points within the same group are geometrically closer. Depending upon the availability of the data to be clustered, clustering is classified as offline or online clustering. In offline clustering, the entire dataset is available at the time of clustering. Hence, in this case, the appropriate number of required clusters can be determined using the elbow method or other approaches.

Recommendation systems based on clustering must deal with online clustering, as new registrations occur continuously. In contrast to offline clustering, in online clustering, the entire data to be clustered is not available at once; instead, the points are received as a stream. Hence, in this case, determining the ideal number of clusters is more complicated than that in offline clustering.

K-medoids Clustering, an unsupervised clustering algorithm, can be used to cluster unlabelled object points [5]. In K-medoids Clustering, clusters are created based on medoids. For each cluster, the associated medoid is the point located at the center of the cluster. A cluster contains all those points closer to its medoid than the medoids of other clusters.

Clustering-based recommendations boost the diversity, consistency, and reliability of suggestions. In addition, this technique is more adept at dealing with sparse user preference matrices and changing user preferences.

Section 2 provides a summary of the existing work that has been done on the topics of clustering and recommendation systems. An online clustering-based recommendation system is discussed in Section 3, along with explanations of its workflow. Section 4 explains the experimentation process, and the results obtained from this experimentation. Section 5 discusses the conclusion and what future scope it has.

2 Existing Work

The existing publications on the implementation of recommendation systems use various machine learning algorithms. The algorithms mainly aim at linking one entity to another with the maximum possible attribute matching among them (i.e., points falling closer to each other on the graph).

Modern recommender systems get characterized into three broad categories.

- Collaborative Filtering system
- Content-Based system
- Hybrid system

Content-based filtering involves suggesting items similar to what the user had previously preferred. In contrast, in collaborative filtering, users are provided suggestions which were liked by users with similar profiles. However, both existing methods have some limitations preventing their use for specific use cases. So to overcome them, hybrid systems, which combine both methods, are also used [1].

Chen and Tu [2] demonstrates the use of an offline (non-real-time) and an online (real-time) component. The online component puts the data stream in its corresponding density grid and updates the feature vector of the density mesh. Then the offline component automatically adjusts the clusters after each interval. The algorithm makes the first cluster after the first time interval passes. Clusters are refined periodically.

In the implementation of [2], reclustering occurs at regular intervals, irrespective of whether new data points have entered the set or not. Repetitive reclustering happens because the online (data retrieval) and the offline component (computation) work independently; thus, this approach wastes computation power and requires more run time.

The first step involves the collection of reviews given by various users and arranging them in the form of a matrix of user-item ratings. Data is then grouped offline utilizing a fast clustering algorithm (like k-medoids) into predefined numbers groups and persisted in the servers for future recommendations. The second step involves the usage of clusters as neighborhoods.

In [3], a new probability distribution-based K-medoids clustering recommendation algorithm for collaborative filtering (CF) is suggested to fix sparse data problems in CF. It uses scoring information based on Kullback-Leibler (KL) divergence and an anti-symmetric distance measure.

In [4], the recommendation system utilizes three main factors to build the system, including datasets, user-based predictive evaluation, and cosine similarity. This referral system has been used to recommend research articles in their field based on the search queries provided as an input.

Chapter [6] uses CF for recommending movies based on what the user has already rated previously. The rating is given out of 5, where "1" is the worst and "5" is the best. Based on the user's direct reaction to their watched movies, a 2-D matrix is created with row values as the users and column values as the movies. Each value represents the rating given to each movie by the user. Then the algorithm makes use of this matrix and gives recommendations to other users accordingly.

Zisopoulos et al. [7] discusses various approaches for implementing recommendation systems, such as Naive Bayes and decision trees. Although, both algorithms give correct results in specific areas of interest; they cannot simply adapt to new input data. This approach requires heavy computation since it relies on the number of data items that are used to build the model.

Philip et al. [8] implements a CBR for a Digital Library of Research Papers. It works based on user queries. The paper concludes that it works better than a standard search engine that matches only strings. Its drawback is its dependency on user input.

Meteren [9] introduces a recommender system named PRES (Personalized Recommender System). The algorithm then compares user profiles with the existing

documents present in the system's collection. After the comparison, they are ranked based on specific criteria such as importance, uniqueness, closeness, and similarity.

Swathi et al. [10] compares various approaches to implement a content-based recommendation system . Some of them are Bee Colony optimization, Graph-based friend recommendation, Hidden Markov Model implementation, etcetera. Their merits and demerits are explained in a tabular format.

Pazzani and Billsus [11] implements a content-based recommendation system for providing movie suggestions, using techniques like nearest neighbor methods, Naive Bayes, decision trees, and Rocchio's algorithm. The model learns the user's profile from his/her past activities and saves it as a query to retrieve movies for recommendations.

de Campos et al. [12] use a hybrid approach for recommendation systems by building a Bayesian network. The resultant model is versatile and provides good results because of its ability to switch between CF and CB according to the type of data.

Das et al. [13] uses the DBScan algorithm for averaging the user's opinions in the same clusters. Users are clustered based on the genre of the movies they select. The results showcase that this algorithm is scalable due to the division of similar users into clusters instead of computing recommendations for each user separately.

Triyana et al. [14] uses clustering with memory-based methods for the recommendations. The results prove that combining a memory-based approach with clustering improves prediction accuracy, and performing clustering yields results up to 5.5 times quicker than before the clustering was done.

Luong Vuong et al. [15] implements a collaborative filtering similarity using a Cognitive Similarity (CS) based approach. The results show that the performance of CS collaborative filtering is much better than the baseline. The Pearson correlation indicates that the proposed method achieves greater accuracy and refinement.

All algorithms discussed above provide some solution to a particular problem related to the recommendation domain. Some, such as [13, 14], are solving scalability issues via clustering, while some like [15] are improving accuracy by combining various techniques. Nevertheless, the main idea usually revolves around the user's interactions with the system either in the way of providing data [12] or ratings [6] of some kind which aid recommendation systems for recommending the target items. No system aims to solve recommendation problems without user interaction through their registration details alone in a dynamic environment. The proposed algorithm discussed further in this document links data into clusters and recommends based on the closest distance measure, which is fast and seamless and provides results with reasonable accuracy (The practical goodness measure of the recommendation system is subjective to the user himself).

3 Proposed System

This paper proposes an online clustering algorithm for clustering a stream of points received at discrete intervals rather than all at once. This can prove helpful for applications that do not have the entire data set available at the time of clustering and instead need to cluster individual points as and when they are received.

The architecture of the suggested recommendation system is presented in Fig. 1 . First, the user requests the server for recommendations based on their similarities. This request is received by the server and passed on to the clustering controller for further processing.

The clustering controller generates a user vector representing the user as a point in the coordinate space. The user vector encodes the attributes of the user, which are to be used for generating recommendations. Each of these attributes has a preset number of possible values. Using such discretized values for each attribute helps make the input space finite and thereby ensures that after a certain number of user inputs have been seen, the same inputs are repeated, and the need for re-clustering is eliminated.

The clustering controller first accesses the database to check if the user has already been placed in a cluster. If this is the case, items within the assigned cluster are retrieved, sorted by their euclidean distance (Eq. 1) from the user vector, and returned to the user as recommendations.

Euclidean distance between two points is given by–

$$d\,(p, q) = \sqrt{\sum_{i=1}^{n} (p_i - q_i)^2} \tag{1}$$

where, p and q are the points considered and p_i and q_i are the i-th attributes of p and q respectively.

If the user hasn't already been assigned to a cluster, the user vector's euclidean distance from the center of each cluster is calculated and the closest cluster is selected. The silhouette coefficient (Eq. 2) value for the new clustering is now calculated.

The silhouette coefficient for a point can be calculated as -

$$c\,(p) = \frac{b(p) - a(p)}{max(b(p), a(p))} \tag{2}$$

where p is the point in question, a(p) is the average distance between point p and all other objects in its cluster, and b(p) is the minimum average distance from point p to all other clusters except its cluster. The value of silhouette coefficient ranges from -1 to 1, with higher values indicating good clustering.

If the coefficient value is higher than or equal to the set threshold value, the recommendations from the newly assigned cluster are returned. If the silhouette

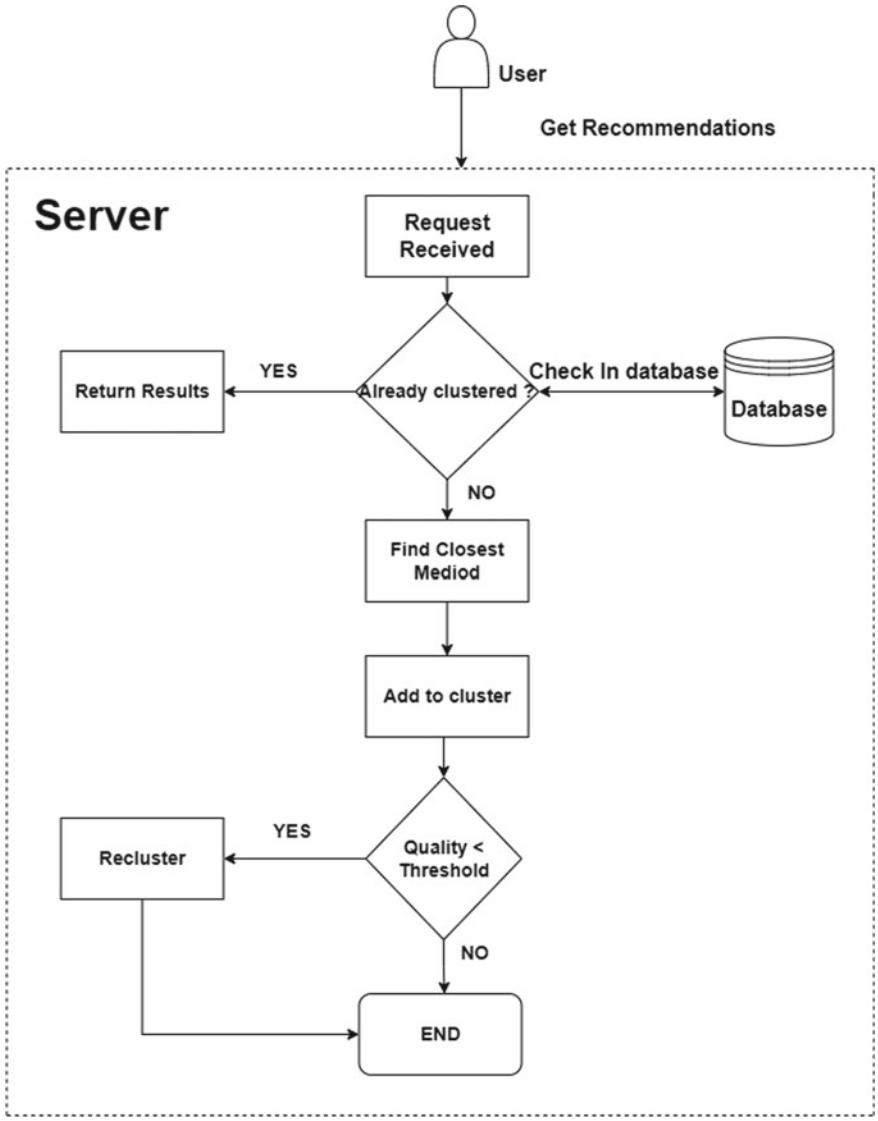


Fig. 1 Architecture diagram

coefficient value falls below the threshold, re-clustering is performed using the K-Medoids clustering algorithm.

Multiple values for the number of clusters (K) are tried for re-clustering. Finally, the clustering corresponding to the K value with the highest silhouette coefficient is selected as the new clustering. The profiles from the new cluster assigned to the user after re-clustering are returned as recommendations.

4 Experimentation and Results

The Python programming language was used for testing. The K-Medoids implementation from the Scikit-Learn machine learning library was used.

4.1 Online Clustering

Clustering was performed ten times on 1000 different points. These points were generated using an isotropic Gaussian blobs function. This function draws points from a special Gaussian Mixture model, such that a point has an equal probability of being sampled from each of the specified numbers of clusters. This helps to ensure that the generated clusters are balanced with respect to the number of points in each cluster.

The points were provided for clustering to the algorithm in sequence during each test iteration. The cluster quality threshold was set to 0.7. During re-clustering, the number of clusters from 2 to 2*k was considered, where k is the current number of clusters.

Figure 2 visualizes the clustering generated by the proposed algorithm. Figure 3 visualizes the clustering generated for the same points using normal K-Medoids clustering.

Table 1 presents the final cluster qualities obtained by clustering the data points using the proposed online clustering algorithm and regular, offline clustering. The clustering quality is measured using the silhouette coefficient. For offline clustering outputs, the K-Medoids algorithm was used. The value of K was varied from 2 to 9, and the results with high quality were selected for reporting.

Table 2 presents the number of clusters generated by the online algorithm against the number of clusters generated by offline K-Medoids clustering.

As is evident from Table 1, in most cases, the proposed system gives similar clustering quality as the regular clustering algorithm. However, in some cases, such as that of datasets 3,4,5,6, it can be observed that the proposed algorithm leads to better clustering quality. By observing Table 2 and comparing Figs. 2 and 3, the reason for this can be determined to be the generation of more clusters, leading to higher separation between the clusters, by the proposed algorithm.

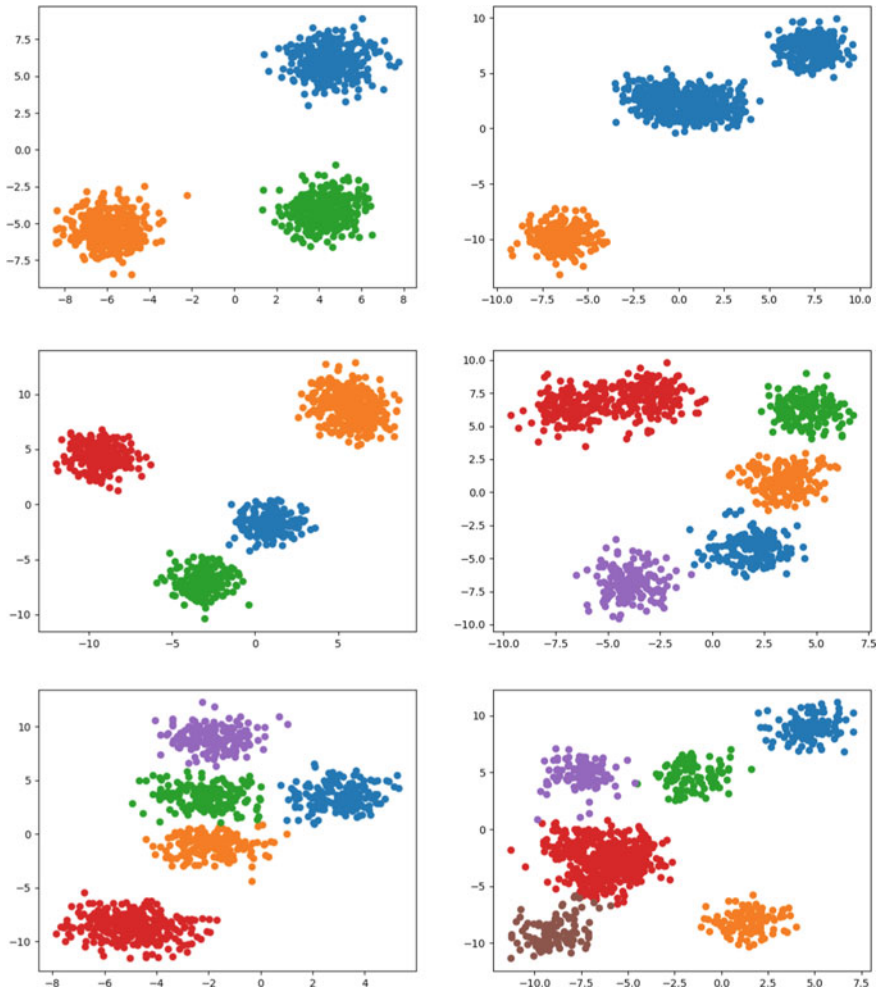


Fig. 2 Online clustering outputs by proposed algorithm for 6 sets of 1000 generated points

The effect of variation of threshold value on the number of times re-clustering occurs is shown in Table 3. Thus, it can be observed that as the threshold value is increased, the re-clustering frequency also increases.

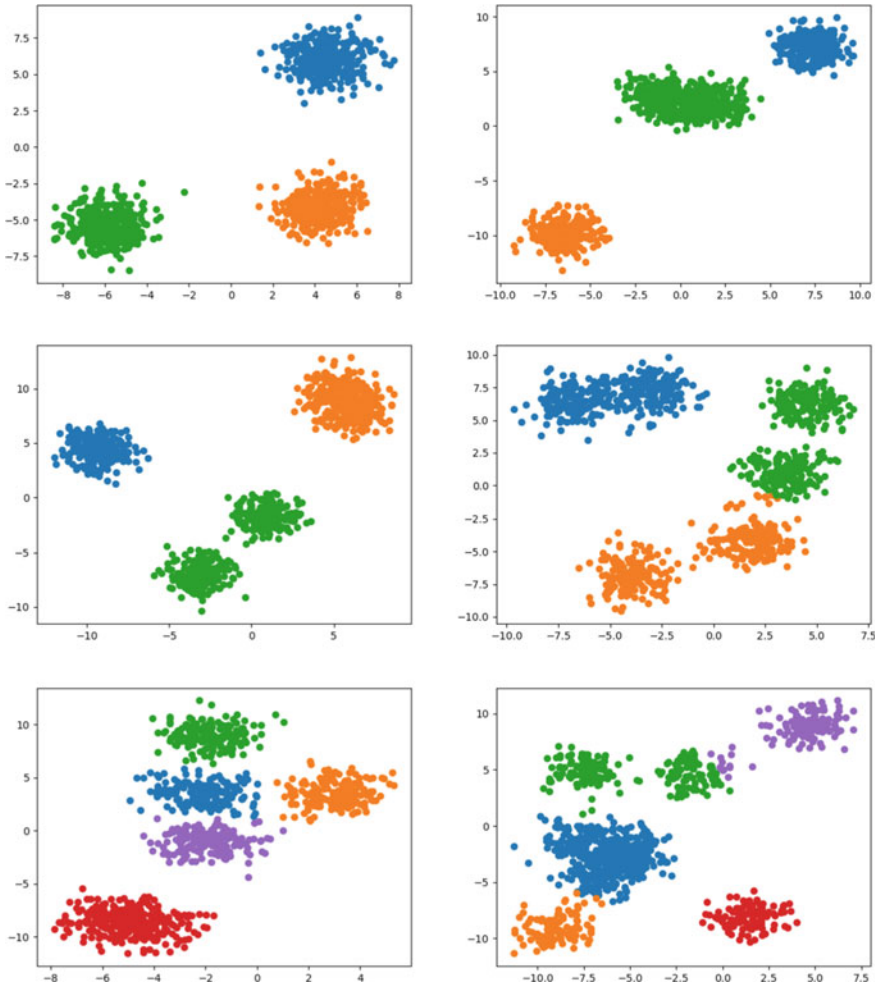


Fig. 3 Offline clustering outputs for the same 6 sets of 1000 generated points

Table 1 The cluster qualities obtained by the proposed online clustering algorithm and normal clustering

Dataset	Offline cluster quality	Online cluster quality
2	0.8242	0.8242
4	0.7763	0.7386
5	0.7701	0.7888
7	0.6151	0.6721
8	0.6689	0.6686
10	0.6124	0.6497

Table 2 cluster count of the proposed online clustering algorithm and normal clustering

Dataset	Offline clusters	Online clusters
1	3	3
2	3	2
3	3	4
4	3	5
5	5	5
6	5	6

Table 3 Reclustering Frequencies for different chosen cluster quality threshold

Threshold	Reclustering count	Silhouette coefficient
0.1	0	0.5341
0.2	1	0.5316
0.3	4	0.4054
0.4	6	0.5341
0.5	25	0.5341
0.6	25	0.6707
0.7	48	0.7410
0.8	998	0.7410
0.9	998	0.7410

4.2 Similarity Based Recommendations

Experimentation was performed, and the obtained results were analyzed to evaluate the efficacy of the proposed similarity-based recommendation system. One thousand points were generated using the isotropic Gaussian blobs function, and online clustering was performed on these as specified in the previous section.

After clustering, ten new points were generated using the isotropic Gaussian blobs function. These points were assigned to the cluster with the closest medoid (according to euclidean distance). Reclustering was performed in case of cluster quality degradation.

Table 4 compares the greatest average cosine similarity between a point and points in another cluster to the average cosine similarity between the point under examination and points in the same cluster.

As can be observed, the points have higher average similarity with other points in the same cluster compared to points in other clusters. Thus, other points in the cluster can be sorted according to their similarity with the queried point and then returned as recommendations ordered by relevance.

Table 4 Average cosine similarities

Point	Assigned cluster	Other cluster
1	0.86065436	0.4262904
2	0.84859488	0.26147972
3	0.65346812	0.33122242
4	0.95744094	0.15305653
5	0.93587811	0.10545943
6	0.47770332	0.33002725
7	0.59342697	0.57251083
8	0.48152841	0.40428508
9	0.50811607	0.35755333
10	0.91060517	0.17941023

5 Conclusion and Future Scope

Thus, the paper concludes that the suggested algorithm can be used for effective online clustering of points and thereby providing similarity-based recommendations to users. Furthermore, based on the observed relation between the threshold taken for ensuring cluster quality and the number of times reclustering occurs, it can be concluded that the number of times clustering occurs increases with the threshold value. This occurs as, in some cases, it is impossible to obtain clusters with quality greater than the threshold; hence, reclustering is performed after adding almost every point.

It can be theorized that as the coordinate space is finite, the user points will start repeating after a time. As a result, the clusters will stabilize since the repeated points will not significantly affect the cluster shapes and sizes. Thus, initially, when the number of points is relatively tiny, reclustering will occur frequently, and as time progresses and the data size increases, the frequency of reclustering will decrease. This is an efficient outcome, as the time complexity of clustering depends on the data size.

The future scope for the system includes research on decaying thresholds to prevent continuous reclustering in case of an unachievable threshold. There is also scope for research on the potential of algorithms other than K-Medoids for clustering. Further research can also be carried out to determine the ideal number of cluster values with which to perform reclustering.

References

1. Sharma M, Mann S (2013) A survey of recommender systems: approaches and limitations. In: ICAECE2013
2. Chen Y, Tu L (2007) Density-based clustering for real-time stream data
3. Deng J, Guo J, Wang Y (2019) A novel K-medoids clustering recommendation algorithm based on probability distribution for collaborative filtering
4. Viswa Murali M, Vishnu TG, Victor N (2019) A collaborative filtering based recommender system for suggesting new trends in any domain of research
5. Al Abid FB (2012) Development of an efficient grid based partitioning around medoids
6. Schafer JB, Frankowski D, Herlocker J, Sen S (2007) Collaborative filtering recommender systems
7. Zisopoulos C, Karagiannidis S, Demirtsoglou G, Antaris S (2008) Content-based recommendation systems
8. Philip S, Shola P, Ovye A (2014) Application of content-based approach in research paper recommendation system for a digital library. Int J Adv Comput Sci Appl 5. https://doi.org/10.14569/IJACSA.2014.051006
9. Meteren RV (2000) Using content-based filtering for recommendation
10. Swathi SR, Devi SG, Joseph DS, Seetha P (2017) Various methods of using content-based filtering algorithm for recommender systems
11. Pazzani MJ, Billsus D (2007) Content-based recommendation systems
12. de Campos LM, Fernández-Luna JM, Huete JF, Rueda-Morales MA (2010) Combining content-based and collaborative recommendations: a hybrid approach based on Bayesian networks. Int J Approx Reason 51(7):785–799. ISSN 0888-613X. https://doi.org/10.1016/j.ijar.2010.04.001
13. Das J, Mukherjee P, Majumder S, Gupta P (2014) Clustering-based recommender system using principles of voting theory. In: Proceedings of 2014 international conference on contemporary computing and informatics, IC3I 2014. https://doi.org/10.1109/IC3I.2014.7019655
14. Widiyaningtyas T, Hidayah I, Adji TB (2021) Recommendation algorithm using clustering-based UPCSim (CB-UPCSim). Computers 10(10):123. https://doi.org/10.3390/computers10100123
15. Nguyen VL, Hong, M-S, Jung J, Sohn B-S (2020) Cognitive similarity-based collaborative filtering recommendation system. Appl Sci 10:4183. https://doi.org/10.3390/app10124183

Towards a General Black-Box Attack on Tabular Datasets



S. Pooja and Gilad Gressel

Abstract Recent studies have shown that machine learning is not secure and is vulnerable to adversarial attacks. While many industries, such as healthcare, finance, and cybersecurity, require tabular datasets, most research in adversarial machine learning has focused on homogeneous datasets (e.g., vision, sound, speech, text). In this work, we will explore a black-box attack designed for tabular datasets. We use the Feature Importance Guided Attack (FIGA) [9], an evasion attack algorithm designed for tabular datasets. Previous work has achieved a 94% success rate using FIGA in a light-grey box setting which assumes having access to the feature representation and summary statistics of the target dataset. These assumptions are unrealistic because it is unlikely to have such prior knowledge about the dataset. We design a black-box attack that requires only querying the target model to learn the labels for a stand-in dataset. Our experiments obtained on average a 70% success rate, with a maximum of 96% success. This demonstrates that FIGA is viable as a black-box attack against tabular datasets.

Keywords Black-box attack · Heterogeneous dataset · FIGA · Evasion attack

1 Introduction

Machine learning is a popular growing technology that enables businesses to automate complex decision-making at high speed and scale [3]. While machine learning has been rapidly adopted into practice in the last decade, the security of machine learning has not been considered [13]. In fact, machine learning models are easily

S. Pooja (✉) · G. Gressel
Center for Cybersecurity Systems and Networks, Amrita Vishwa Vidyapeetham, Amritapuri, India
e-mail: poojasomakumar91@gmail.com

G. Gressel
e-mail: gilad.gressel@am.amrita.edu

© The Author(s), under exclusive license to Springer Nature Singapore Pte Ltd. 2023

S. Chinara et al. (eds.), *Advances in Distributed Computing and Machine Learning*, Lecture Notes in Networks and Systems 660,
https://doi.org/10.1007/978-981-99-1203-2_47

evaded by adversarial perturbations [16]. In this paper, we will focus on adversarial evasion attacks. An adversarial evasion attack is when a malicious user submits a sample with the intention of evading the model. That is, the model will fail to identify the sample correctly. These adversarial samples are created by adding optimized perturbations to the input sample, which causes the model to misclassify them, reducing its performance.

For example, a phishing detection model classifies websites as either phishing or legitimate [8, 11]. An adversary may evade the model by making imperceptible changes to the phishing site causing the model to misclassify the page as a legitimate page.

Adversarial attacks are designed for either heterogeneous or homogeneous dataset attacks [15]. Examples of homogeneous datasets are vision, sound, and text data. In a homogeneous dataset, all the features are semantically identical. For example, all the features of a picture are pixels. In contrast, heterogeneous tabular datasets contain categorical, numerical, and nominal features and often have missing values. The recent research into adversarial attacks is predominantly performed on homogeneous datasets, specifically in computer vision. However, adversarial attacks that perform well on homogeneous datasets cannot be directly applied to heterogeneous datasets because they assume that all features can be equally modified. In a heterogeneous dataset, it is often the case that specific feature columns may be immutable. For example, the URL of a phishing website cannot be changed to mimic the target website's domain name.

Further, many tabular datasets contain features that must remain valid, e.g., a person's birthday cannot be after their death date. For these reasons, we must use specific attack algorithms designed for heterogeneous data to attack tabular datasets.

In the adversarial machine learning literature, three general threat models have been defined based on the knowledge of an adversary about the target (victim) model. There are white-box, grey-box, and black-box attacks [5, 14]. The white-box threat model assumes that the attacker has full knowledge about the target machine learning model. The grey-box setting indicates partial knowledge about the model. In contrast, the black-box attack allows zero knowledge of an attacker regarding the model.

One such tabular attack is the Feature Importance Guided Attack (FIGA), a model agnostic algorithm that considers three factors to establish an adversarial attack. They are the number of features n, the perturbation factor ϵ, and a feature ranking algorithm f_i. FIGA will compute the most important features and add perturbations to them in the direction of the class we are targeting. Gressel et al. have demonstrated that FIGA can achieve a 94% success rate against tabular datasets [9].

However, FIGA was implemented with a light grey-box setting, which assumes that the attacker can access the dataset's feature representation and summary statistics. While this assumption may be realistic for computer vision models where the datasets are primarily open source and shared, this assumption is unrealistic in the tabular dataset domain. For example, the phishing dataset on which FIGA was demonstrated would never be shared publicly. In this work, we perform FIGA in a black-box setting and demonstrate that it can be used as a practical tabular evasion attack.

The challenge of black-box attacks with tabular datasets is that we cannot assume the feature representation for the target model. A defending model would extract features from a phishing website, and the adversary can only guess what those features were. To mimic this guessing game the adversary must play, we conducted experiments with a random sampling of the feature pool to simulate the unknown feature space of the target model. Using a phishing dataset with 348,739 websites, we obtained an average success of over 70% for the black-box attack.

Our contributions are the following:

- We provide a methodology using random sampling which simulates the unknown feature space of a target model for tabular datasets.
- We perform numerous experiments with multiple models and datasets to explore the black box attack.
- We demonstrate that FIGA is successful in a black-box threat model.

2 Related Work

Adversarial examples created to mislead one model can be reused on another model of a different architecture or underlying algorithm, even if the other model is trained with a different training set [18]. This transferability property works only when both models are built to perform the same task. A common approach to black-box attacks is to train a substitute model and craft adversarial examples against the substitute. These adversarial examples are often successful against the victim model. This approach assumes that the feature representation of the substitute and target models are identical. In the case of homogeneous datasets, all features are the same. In contrast to training a substitute model, FIGA only requires the summary statistics of the dataset. However, in tabular datasets, we cannot assume to know the features of the target model, as each model will have a unique set of features designed by the defender. These feature representations are a closely guarded secret.

Weiwei Hu and Ying Tan proposed an algorithm called MalGAN to generate adversarial malware, which can bypass black-box machine learning-based detection models [12]. Their assumption was that the only knowledge the malware author has about a black-box detector is what features it uses, an assumption we do not find reasonable. An adversary may be able to guess at certain features with some accuracy, but surely they will not know exactly all the features used by the target model in the case of malware detection. In our work, we address this by randomly sampling the features for our attack.

Chuan Guo et al. aimed to create a simple method to craft adversarial images by finding a minor disturbance with no gradient information under the black-box attack settings [10]. They believe that the existence of the "output probability" can be a powerful agent to guide the search for adverse images. simBA is applicable to both targeted and untargeted attacks with an improved query efficiency.

Vincent Ballet et al. focus primarily on tabular datasets [4]. They proposed a white-box attack approach named LowProFool. Their objective is to craft adversarial examples such that perturbations applied to a tabular data example are imperceptible to the expert's eye. They hypothesize that important features should not be perturbed because an expert would notice them. Therefore they focus their perturbations on the less important features. We disagree; we believe that using a mimicry approach like FIGA can perturb the important features to be similar to the target class, thus remaining plausible to an expert's review.

Yael Mathov et al. demonstrate that machine learning models trained on heterogeneous tabular data are just as vulnerable to adversarial manipulations as those trained on homogeneous datasets [15]. They presented a general optimization method for identifying and creating adversarial perturbations. They assume a white box scenario where the entire dataset and feature representation is known. In this work, we will also perform an adversarial attack on heterogeneous datasets. However, we will do so without assuming any knowledge of the underlying model and only partial (randomly selected) knowledge of the feature representation in diverse input regions of heterogeneous datasets.

3 Background

3.1 Feature Importance Guided Attack (FIGA)

The FIGA algorithm is a two-step procedure. Step 1 of the algorithm ranks the most significant features and determines the direction that each feature should be perturbed. FIGA performs a perturbation in the direction of the target class. To obtain the direction to perturb the features, we compare the mean feature values of the target class and the mean feature values of the input class. We want to step the input class features in the direction of the target class.

For example, if our input class is phishing websites and our target class is benign websites. To mimic the benign class, we will perturb the phishing features so that each feature is added or subtracted in the direction of the mean feature value of the benign class.

FIGA will perturb the n most important features. We use a feature importance algorithm to rank the features, from which we select the top n features for perturbation. In this work, we use the Gini impurity coefficient; however, Gressel et al. have demonstrated that various feature ranking algorithms are feasible with varying results depending on the model being attacked [9]. These two parameters, n, the number of features, and ϵ, the perturbation factor, can be used to regulate the attack's strength. ϵ is a non-negative floating point number that scales the attack strength. Specifically, when ϵ increases, the perturbation increases, yielding a stronger attack with a greater perturbation. The downside is that the attack is potentially more easily identified (as the sample has been changed more).

In step 2 of the algorithm, the data is transformed into a scaled feature space, and the perturbation is applied. This is done to ensure that all features are perturbed equally on the same scale. After scaling, we perturb the n important features by $\frac{\epsilon}{n} \sum FeatureValues$. ϵ is used to control the size of the perturbation. Since we sum the feature values for each sample and multiply it by $\frac{\epsilon}{n}$ we are perturbing the percentage of each sample with respect to its feature size. Following that, the sample must be inverse transformed back into its original feature space. Finally, we must ensure that all feature values are plausible in the tabular space. If a feature type is discrete, we will clip the resulting attack to be valid. For the full details of this attack, please refer to the full paper [9].

4 Datasets used for the Black-Box FIGA Attack

We use three datasets to test FIGA's effectiveness in black-box conditions. Gressel et al. collected the first dataset (D1) with a selenium-based crawler which contains both benign and phishing data. The phishing data was collected from PhishTank in 2019-2020. The benign data was collected using a URL seed list from Tranco [17] during 2020. In total, 348,739 URLs and their corresponding source code were collected, and 52 features were extracted from the URLs and HTML [9]. We use an additional phishing dataset (D2) containing 10,000 samples and 48 features created by Kang et al. [7]. Finally, we test our method against the Adult dataset (D3), an income dataset with 48,842 samples [2].

5 Methodology

5.1 Threat Model

We assume no knowledge of the target model's underlying algorithm, hyper-parameters, or training set distribution, but we assume partial knowledge of the features through random sampling [1].

A feature space is the set of all possible values that can be extracted from a raw piece of data. The feature representation of the target model is assumed to be known in all previous work on black box attacks. In the case of tabular datasets, this assumption cannot hold. We will assume partial knowledge of the feature representation. We believe this is realistic because most problems will have certain features which are used for that domain. For example in phishing detection, the URL is always encoded into the feature space since it is a strong indicator of the maliciousness of a website. Therefore an attacker can reasonably guess some features that the defending model may use. To simulate this, we randomly sampled the features for each dataset to build the target model.

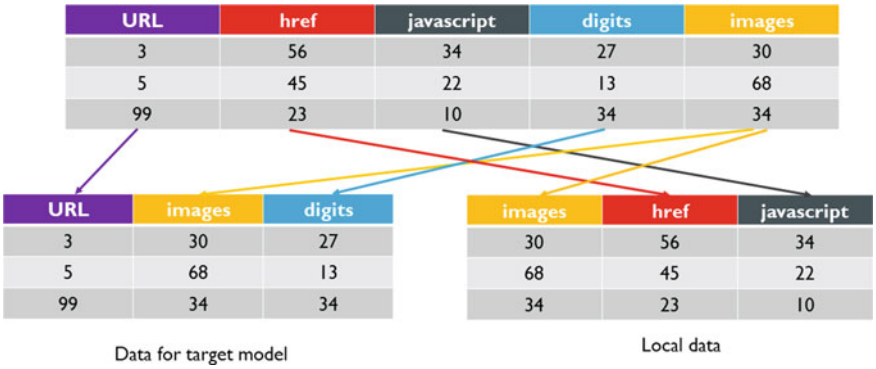


Fig. 1 A visualization of the random sampling process. A superset of all features is sampled without replacement. This allows two separate models to be created, which will have an overlap of some features, but not all

Figure 1 is an illustration that represents the random sampling in feature space, we see that certain features such as "images" may be selected into both sets, but others will not be. For the attack to be successful, the attacker must be able to guess some of the features of the target effectively; the random sampling process captures this uncertainty.

Algorithm 1: Experimental setup for the black-box attack

Input: X, T_m, ϵ, n, f_i
▷ X: data, t_m: target model, ϵ: perturbation factor, n num features, f_i: feat importance algo
Output: X^* ▷ adversarial attack

1: $X_L, X_T \leftarrow sample(X)$ ▷ random sample features from X
2: $(X_{L:train}, X_{L:test}), (X_{T:train}, X_{T:test}) \leftarrow X_L, X_T$ ▷ train test split
3: $T_m \leftarrow T_m(X_{T:train})$ ▷ train target model
4: $l \leftarrow T_m(X_{L:test})$ ▷ query target model
5: $X_{l:test} \leftarrow \text{relabel}(X_{L:test}, l)$ ▷ relabel $X_{L:test}$ with l
6: $X^* \leftarrow FIGA(X_{L:train}, X_{l:test}, \epsilon, n, f_i)$ ▷ FIGA Algorithm [9]
7: **Return:** X^*

5.2 *Experimental Setup*

Algorithm 1 shows the experimental setup used in this work. As an input to our algorithm, we require five elements. X is the data used for the entire experiment. T_m is the target model we will attack. ϵ is the perturbation factor, n is the number of features to perturb, and f_i is the feature importance ranking algorithm. The first step is to randomly sample $s\%$ of the features from X two times and assign them to X_L

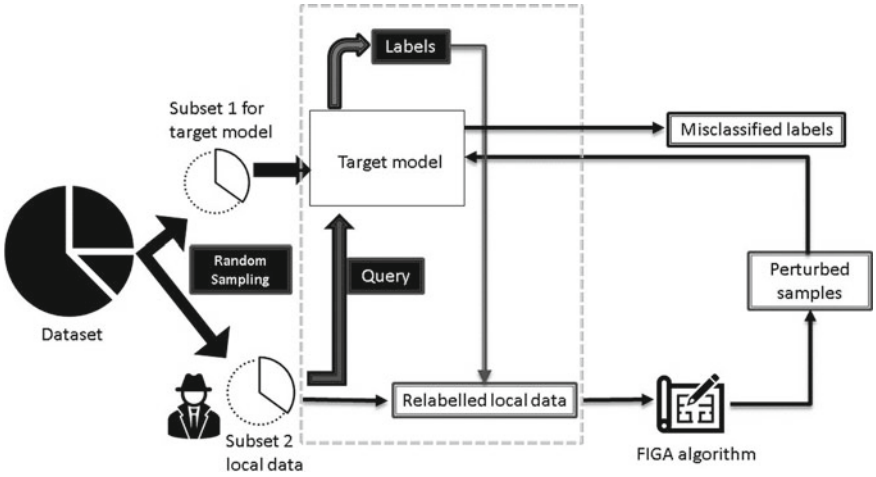


Fig. 2 Experimental setup for black-box attack with FIGA

and X_T. This way, the target model will be trained on an overlapping yet unknown set of features from the local dataset the attack can work with. For example if we sample 30% of the features (without replacement) the resulting two feature sets will have a 30% overlap.

We then split the two datasets into training and testing sets. We train the target model and query it with the local testing data. In order to query it, we transform the feature space of the local data into the same space as the target model; this is akin to sending an actual phishing website to the target model and allowing it to extract its features. We use the labels acquired to relabel our local dataset according to the target model's predictions. We then perform a FIGA attack on the local dataset. The perturbed data X^* is then used to query the target model again, and we record the attack's success rate. A pictorial diagram of this process is presented in Fig. 2.

We experimented with four classifiers as target models: decision trees, random forests,multi-layer perceptron, and gradient-boosted trees. We used Scikit-Learn [19] for the first three and the XGBoost library [6] for the gradient boosted trees. We performed experiments to assess the success rate of our proposed black-box approach on several datasets, including the primary dataset (FIGA dataset) and secondary datasets (Adult dataset and Kang dataset). For each algorithm and dataset, we conducted 100 repeated trials and reported the average findings. This is done to smooth out anomalies due to random feature sampling.

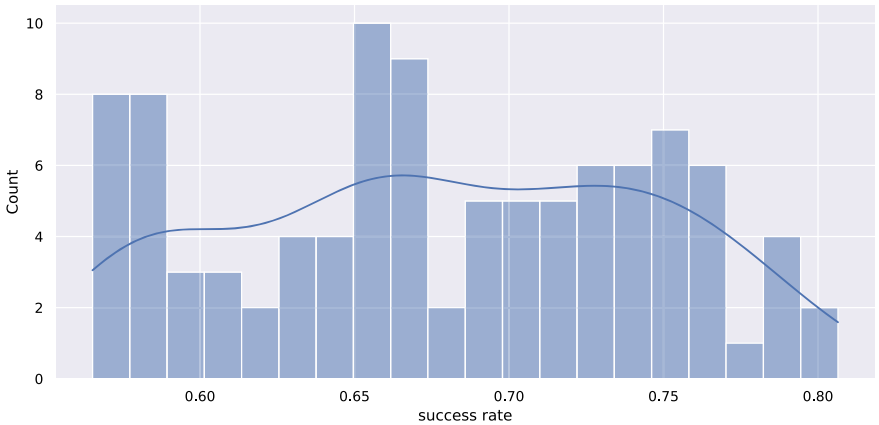


Fig. 3 A histogram of success rates for the decision tree model on 100 repeated experiments. We attacked with n=10 and $\epsilon = 0.02$ using a 50% feature sampling rate

5.3 Tuning FIGA

We use the ideal n, ϵ and f_i found in previous work [9]. In particular, we find that a smaller $\epsilon < 0.04$ yields the best results. In Fig. 3 we can see a histogram of success rates for 100 trials of $n = 10$ and $\epsilon = 0.02$, with a feature sampling rate of 50%.

5.4 Evaluation Metrics

To evaluate the performance of the black-box attack on the target phishing detection model, we considered the recall score and average success rate. The recall score is the number of true positives identified by the detector. We calculate the success rate by comparing the recall score of the model before and after the attack. The success rate is determined by the fall in recall score divided by the total number of true positives. This is a measurement of how many true positives were turned into false negatives.

6 Results

As shown in Table 1, we conducted experiments with different values for the perturbation factor ϵ while using $n = 10$. We attacked all four classifiers for each setting using the primary dataset D1. We report the average success rate and recall scores for 100 trials. It is clear that as epsilon increases, the attack's success rate also increases.

Table 1 Average success rate and recall score of 100 trials for different target phishing detection models on the primary dataset (D1), using a feature sampling rate of 50%

Recall score			
Target model	Before attack	After attack	Success rate (%)
$n = 10, \epsilon = 0.001$			
Decision tree	0.94	0.38	59.6
Random forest	0.95	0.38	60
XG boost	0.96	0.39	60.6
MLP	0.94	0.39	58.5
$n = 10, \epsilon = 0.01$			
Decision tree	0.94	0.33	64.9
Random forest	0.95	0.37	61
XG boost	0.96	0.36	62.5
MLP	0.93	0.37	60.2
$n = 10, \epsilon = 0.02$			
Decision tree	0.94	0.32	66
Random forest	0.95	0.33	65.2
XG boost	0.96	0.34	64.3
MLP	0.93	0.34	63.4
$n = 10, \epsilon = 0.04$			
Decision tree	0.94	0.3	68
Random forest	0.95	0.31	67.3
XG boost	0.96	0.32	65.8
MLP	0.93	0.30	67.7

The maximum success rate we attained in these 400 trials (when epsilon = 0.04) is 76.8%; this is likely due to good luck in the random feature sampling process (in the original research, they were easily able to attain 94% success with full access to the features).

Figure 4 shows the average success rate for each model on different datasets. We used $n = 10$ and $\epsilon = 0.031$ as these demonstrated a good balance of success with a small perturbation amount. The bar chart shows us that success rates generally fluctuate between 40 and 66%. This indicates that the blackbox attack is successful against our primary dataset and generalizes to others as well. Due to space limitations we do not show all results, but our results do indicate that the black box attack is consistent across sampling rates ranging from 0.1 to 0.9. It is able to obtain on average a 70% success rate.

By comparing the success rate of all models with $n = 10$ and $\epsilon = 0.031$, we found that the decision tree classifier resulted in an average success rate of 67% for the primary dataset, 45% for the Adult dataset, and 62% for the Kang dataset.

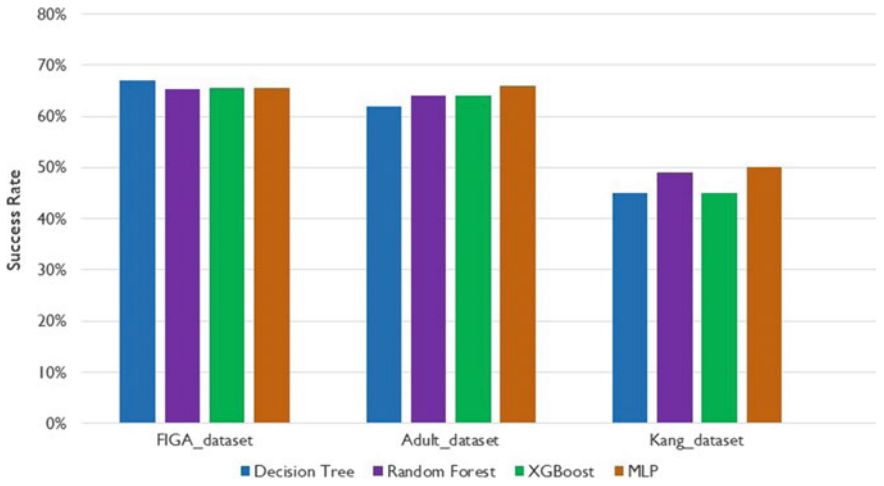


Fig. 4 Average success rate for different datasets using $n = 10$ and $\epsilon = 0.031$, with a feature sampling rate of 50%

7 Conclusion

In this work, we proposed and evaluated a black-box approach using FIGA, an evasion attack algorithm designed for tabular datasets. We demonstrated a black-box attack that only queried the target model to obtain the labels, requiring no prior knowledge of the dataset. FIGA had an average success rate of 70% in the black-box setting, proving that FIGA is a feasible, practical black-box attack on tabular datasets.

References

1. Acharya AS et al (2013) Sampling: why and how of it. In: Indian J Med Spec 4(2):330–333
2. Adult (1996) UCI machine learning repository
3. Aggarwal K et al (2022) Has the future started? The current growth of artificial intelligence, machine learning, and deep learning. Iraqi J Comput Sci Math 3(1):115–123
4. Ballet V et al (2019) Imperceptible adversarial attacks on tabular data. arXiv:1911.03274
5. Brendel W, Rauber J, Bethge M (2017) Decision-based adversarial attacks: reliable attacks against black-box machine learning models. arXiv:1712.04248
6. Chen T, Guestrin C (2016) Xgboost: a scalable tree boosting system. In: Proceedings of the 22nd ACM SIGKDD international conference on knowledge discovery and data mining, pp 785–794
7. Chiew KL et al (2019) A new hybrid ensemble feature selection framework for machine learning-based phishing detection system. Inf Sci 484:153–166
8. Dalvi S, Gressel G, Achuthan K (2019) Tuning the false positive rate/false negative rate with phishing detection models. Int J Eng Adv Technol 9:7–13
9. Gressel G et al (2021) Feature importance guided attack: a model agnostic adversarial attack. arXiv:2106.14815

10. Guo C et al (2019) Simple black-box adversarial attacks. In: International conference on machine learning. PMLR, 2019, pp 2484–2493
11. Harikrishnan NB, Vinayakumar R, Soman KP (2018) A machine learning approach towards phishing email detection. In: Proceedings of the anti phishing pilot at ACM international workshop on security and privacy analytics (IWSPA AP), vol 2013, pp 455–468
12. Hu W, Tan Y (2017) Generating adversarial malware examples for black-box attacks based on GAN. arXiv:1702.05983
13. Kumar RSS (2020) Adversarial machine learning-industry perspectives. In: IEEE security and privacy workshops (SPW). IEEE, pp 69–75
14. Li C et al (2022) An approximated gradient sign method using differential evolution for black-box adversarial attack. IEEE Trans Evol Comput
15. Mathov Y et al (2020) Not all datasets are born equal: on heterogeneous data and adversarial examples. arXiv:2010.03180
16. Nair AB et al (2022) Comparative study of centrality based adversarial attacks on graph convolutional network model for node classification. In: 2022 7th international conference on communication and electronics systems (ICCES), pp. 731–736. https://doi.org/10.1109/ICCES54183.2022.9835948
17. LLC OpenDNS (2016) PhishTank: an anti-phishing site. https://www.phishtank.com
18. Papernot N, McDaniel P, Goodfellow I (2016) Transferability in machine learning: from phenomena to black-box attacks using adversarial samples. arXiv:1605.07277
19. Pedregosa F et al (2011) Scikit-learn: machine learning in python. J Mach Learn Res 12:2825–2830

Multi-task System for Multiple Languages Translation Using Transformers



Bhargava Satya Nunna

Abstract In this work, we explore the problem of the multi-task Neural Machine Translation (NMT) model which can simultaneously translate given sentences from multiple source languages to a single target language. Our solution is supported by a recently proposed transformer model, where we extended the neural machine translation to a multi-task system that shares the common target language translation and separates the modeling of different source language representations. The multi-task system could be helpful in scenarios where insufficient data or imbalanced data exists. As the multi task system shares the parameters of different language pairs, the language pair with inadequate data could utilize the resources from data-abundant language pairs and can improve the translation quality. Our approach is also considerably faster learning when compared to separately trained NMTs.

Keywords Natural language processing · Machine translation · Transformers

1 Introduction

Machine Translation is one of the most vital applications in the Natural Language Processing (NLP) field. Machine Translation (MT) plays a crucial role in improving people's communication with different native languages and helps to share information by eliminating language barriers. There are several machine translation models for language translation tasks where a few years ago, the most commonly used method was statistical machine translation (SMT) [1] that is based on Bayesian systems for the predicting of probabilities of words and phrases in a given sentence, however, SMT has limitations such as considerably low accuracy in translation and much time-consuming method. To avoid such drawbacks recent works propose a Neural

B. S. Nunna (✉)
Department of Computer Science and Systems Engineering, Andhra University, Vishakapatnam, India
e-mail: nbsatya19@gmail.com

© The Author(s), under exclusive license to Springer Nature Singapore Pte Ltd. 2023

S. Chinara et al. (eds.), *Advances in Distributed Computing and Machine Learning*,
Lecture Notes in Networks and Systems 660,
https://doi.org/10.1007/978-981-99-1203-2_48

machine translation (NMT) model with better performance than conventional SMT models while considerably having less computational time.

The NMT model follows a sequence-to-sequence learning approach which is an encoder-decoder model. Encoder-Decoder model in NMT are implemented by advanced neural models, such as Recurrent Neural Networks (RNN) [2], Long Short-Term Memory (LSTM) [3], Gated Recurrent Unit (GRU) [4], Bahdanau et al. Attention-based NMT [5] and Transformer model [6]. Several NMT models were proposed recently for better learning of hidden representations and for improving performance in the translation of various languages. RNN was the most popular approach from past years for successful implementation in Encoder-Decoder models and considerably more progressive than traditionally used SMT but with growing data, even RNN's performance has reached its bottleneck. RNN's & LSTM's too have limitations such as the data passed in a sequential manner where for a large amount of data, the computational time would increase exponentially, and RNN's also wouldn't be able to learn(or keep up) content and context within the longer sequences.

Recently proposed Transformer model [6] has revolutionized the world of sequence-to-sequence modeling by overcoming the limitations of RNN's with the help of self-attention concepts where the transformers were able to handle the long-range dependencies and also support the parallelization unlike RNN's. The Transformer-based NMT models yield promising results when compared to that traditional translational models.

Apart from NMT models, with growing data and numerous languages around the world, it would be a relentless task to generate a translational model for each language pair. To overcome the such issues, motivated by the recently proposed Transformer model [6] and a multi-task learning model proposed by Dong et al. [7], we present a multi-task learning system based on a sequence learning Encoder-Decoder architecture with transformer model to perform a machine translation task from multiple source languages to single target language. In our proposed work, we assume that many languages might be differed lexically but are analogous at the syntactic and semantic levels, where such correlations among the different source languages could be learned by our multi-task system.

In this multi-task framework, every language pair translation is considered as a sub-task in the Transformer model based Encoder-Decoder structure in which the model employ's a separate encoder for each specific source language and shares a common decoder for the same target language where the decoder learns different hidden representations from each encoder.

2 Related Works

Recently many research works have been carried out for improving the performance of NMT and enhancing the translation quality.

For instance, Dong et al. [7] proposed a Neural Machine translation model which can simultaneously translate sentences from given one source language to multiple target languages using Gated Recurrent Neural Network (GRU) and demonstrated that proposed NMT model is able to obtain considerably better translation quality over individually trained model.

Bensalah et al. [8] proposed a hybrid approach using Convolutional Neural Network and a Recurrent Neural Networks with Attention (CRAN) and had improved the Machine Translation of Arabic Language.

Shah et al. [9] have proposed a NMT using LSTM and a attention mechanism for Translation of Indic Languages. The presented model has obtained a BLEU score of 40.33 while translating a language pair of Gujarati to English.

Qi et al. [10] have used to pre-trained word embeddings to enhance the NMT model performance and demonstrated that pre-trained embeddings would be more effective for more similar translation pairs and could obtain upto 20 BLEU points with a proper tuning.

Atrio et al. [11] have performed experiments with Recurrent Neural Networks with a low resource setting and illustrated that smaller batches were favorable along with other regularization's and BLEU scores were improved with latter of 0.30 and 0.04 on 5 k and 160 k sentence pairs dataset respectively.

In the context of RNNs and LSTM models, performance has reached its maximum extent possible. The new self-attention concept presented from Transformer Network has became the most favorable choice for designing a Translational Systems. The NMT's designed with the multi-layer Self Attention blocks have obtained a better translation quality over several several bench-mark datasets. For instance, recently proposed Bidirectional Encoder Representations from Transformers (BERT) model [12] has a variable number of encoder layers and self-attention heads where, the Transformer blocks were organised into bidirectional encoder representations so that the model would have both forward & backward attention in the sequence. BERT has achieved 4.6–7.7% of improvement over various state-of-the-art NMT benchmarks.

3 Background and Methodology

We propose a model which includes recently proposed NMT model, Transformer Neural Network which has eliminated the recurrence and replaced it with a new concept of attention mechanism and procured a better performance when compared to the previous traditional NMT models.

3.1 Transformer Network

The transformer model shown in Fig. 1 is a sequence-to-sequence architecture that contains two primary modules namely, encoder & decoder parts and the model also

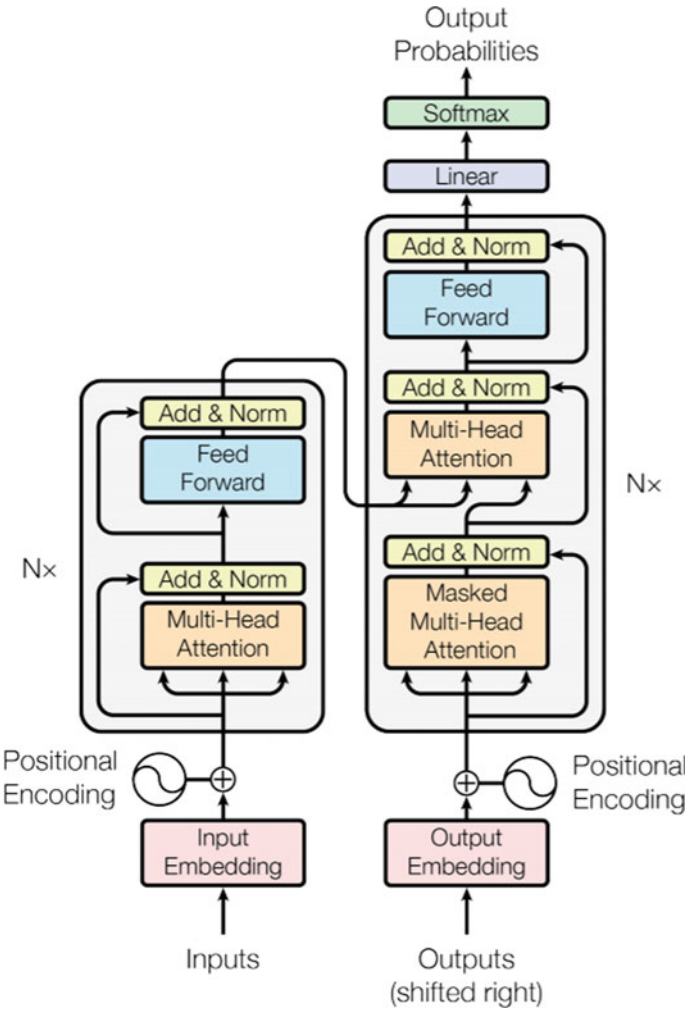


Fig. 1 Illustration of encoder and decoder architecture in the transformer model [6]

consists of Embedding and Positional encoding modules. Here, the received inputs in the encoder & decoder part of the Transformer model were reconstructed into higher vector representation by embedding layer. The embedding outputs from the embedding layer were added with positional information by the positional encoding module. Therefore, the encoded inputs of language pair from the positional encoder are then fed into the encoder and decoder components of the transformer model respectively as shown in Fig. 1. The encoder and decoder modules further consist of a Multi-Headed Self-Attention component, two-layer Normalization, and a Fully-

Connected network at the end. The main components are further explained in the following section.

3.1.1 Positional Encoding

Firstly, the sequential data inputs the embedding module in the Transformer Network and data is converted into embedded tokens with higher dimensional representation. Since the input data is sequential data and Transformer Network includes a feed-forward layer without recurrence-supporting architecture, the model itself doesn't know the position of each token in the given set (or say the order of each word in the input sentence) passing through the encoder & decoder modules of the Transformer model. To overcome the above issue, a Positional Encoding module is introduced after the embedding layer in which it adds the positional information to each token(or word) about its position with respect to others in the set(or sentence). The Positional Encoding module adds the positional information to the given sequence with the following method.

$$PE_{(pos,2i)} = sin\left(\frac{pos}{10000^{2i/d}}\right) \tag{1}$$

$$PE_{(pos,2i+1)} = cos\left(\frac{pos}{10000^{2i/d}}\right) \tag{2}$$

where d represents the embedding dimension of the input sequence, pos indicates the position of word in a sentence and i is a bit position to introduce varying frequencies(or cycles) in sine/cosine functions.

3.1.2 Self-attention

Self-Attention plays a significant role in the transformer model by handling the long-range dependencies in sequential data like text or speech and is also a key factor for supporting parallel learning of sequential data unlike RNN's. Firstly, the input tokens added with positional information are linearly projected into to feature spaces f, g, h to train the attention weights where $f(x) = W_q x,\ g(x) = W_k x\ \ h(x) = W_v x$. Here W_q, W_k and W_v are weight matrices that could be interpreted as the projections of Query, Key and Value in the database retrieval system and x are the input tokens. The attention weights could be calculated by the dot product of the above Key and Query weight matrices such that.

$$Attention(f, g, h) = Softmax\left(\frac{fg^T}{\sqrt{d_k}}\right)h \tag{3}$$

where fg^T represents the attention weight learned by dot-product. Here, f, g, and h have a feature representation with $N \times d_k$ with d_k as hidden dimension shape and

N is the length of given the sequence and further, a Soft-max activation function was applied on the calculated attention weights. And the final attention-based output is generated by multiplying the output from the soft-max function with $h(x)$. The outcomes from the Multi-Head Self Attention module are normalized and further represented with higher dimensions using Feed Forward network.

3.2 Proposed Approach

Inspired by the multi task learning model proposed by [7], we propose a general framework for translating from multiple source languages to a single target language. The proposed model is developed with Transform Neural Network based encoder-decoder model with multiple translation tasks such that each sub-task is a specific translation direction. Different language pairs with a common target language would share the same translation decoder across different language pairs. In this way, we can reduce the training time, computational resources, and have faster learning. The decoder parameters of the model are shared across various language pairs so that model could utilize all of the target language training data across the given language pairs and enhance the target language representation. The sharing of the decoder parameters would be effective when dealing with issues like inadequate data problems where some language pairs with deprived resources can make use of parallel training corpora of remaining language pairs to improve the quality of translation in the multi-task model. Thus the higher dimensional representation of the target language learned by the proposed multi-task model is more stable and has better performance when compared to learning with an individual model. The architecture of the proposed multi-task model is shown in Fig 2. Therefore proposed model is faster and converges to better translation quality when compared to separately trained NMT's by dealing with resource-scarce & resource-rich language pairs (imbalanced data) in a multi-task model.

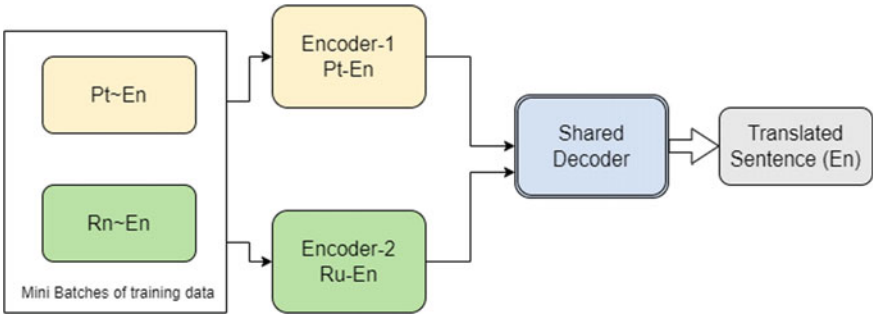


Fig. 2 A multi-task framework with shared decoder for translation of Portuguese to English (Pt–En) & Russian to English (Ru–En) language pairs

We have used standard Cross Entropy loss (or log loss) to calculate the loss of the respective model. A Cross-Entropy (CE) is a measure of the two probability distributions, and the CE is decreased as the predicted probability converges to the ground truth (or label). We could minimize the loss of a model by using appropriate optimization techniques [13]). The cross-entropy loss can be represented with the following equation.

$$\mathcal{L}_{CE}(\ell, t) = -\sum_{k=1}^{K} t_k \log \ell_k \tag{4}$$

where the $\ell \in \mathbb{R}^K$, $t \in \{0, 1\}^K$ (denotes one hot encoding) are the predicted value and target label respectively.

4 Experiments

The proposed architecture has been developed using TensorFlow [14] & NLTK frameworks [15], where we used Cross-Entropy loss function & Adam optimizer [16] for minimizing the overall loss of the model at each epoch.

For the training of the proposed multi-task framework, we used a common corpus of TED talk Datasets which are derived from TED talk transcripts [10] where it has translations between multiple languages. We used Portuguese-to-English (Pt-En) & Russian-to-English (Ru-En) corpus from dataset for our experiments. We used the official validation and test sets of the language pairs for training & evaluating the model. The corpus of $Pt-En$ & $Ru-En$ pairs in the training set are of around the size of 50000 & 45000 respectively. During the training process, firstly we fed the model with a batch of one language pair and the next batch with a different language pair and continued in a similar fashion until the training corpus is finished, we followed this strategy to avoid the problems like over-fitting and make the model robust for translating the language of each pair it learned.

5 Evaluation

We evaluate the performance of our proposed multi-task model from the official test sets of each language pair available in TED talks Datasets. we used the Bilingual Evaluation Understudy (BLEU) [17] metric to measure the performance of multi-task NMT. We evaluate BLEU scores on a Common test set with separately trained NMT models on each language pair and proposed multi-task NMT model to illustrate the validity of the proposed multi-task learning model over the separately trained models. Table 1 consists of the comparative results of the Multi-task NMT model with other approaches, pre-trained word embedding [10] which has also used the same TED talks database.

Table 1 Comparison of BLEU scores of language pairs with proposed multi-task NMT and other approaches

	Pt–En	Ru–En
Qi et al. [10]	30.8	21.1
Single NMT	29.62	17.01
Multi-task NMT	**33.02**	**21.3**

Table 2 Translations of Portuguese and Russian languages with single-task NMT and multi-task NMT

Translation of different languages	
Russian	я бы сотрудничал со своими оппонентами, чтобы стать лучше в том, что я делаю.
Reference English	I would collaborate with my opponents to get better at what I do
Multi-task English	I would have worked with my opponents to become better at what I do
Single-task English	I would have come up with my opponents to become better at what I'm
Portuguese	começamos a falar sobre o público e governo
Reference English	we started talking about the public and government
Multi-task English	we started to talk about the public and the government
Single-task English	we started talking about the audience and government

The BLEU scores for the translation of different language pairs using a transformer based multi-task translation framework were mentioned in Table 1, by considering the BLEU score as the evaluation metric for measuring the performance of the proposed system in translation, we can observe that the proposed multitask framework can efficiently translate multiple languages parallelly and has obtained a propitious performance when compared to separately trained NMT architectures.

Therefore, it is clearly evident from the experimental results that the proposed multi task framework is capable of translating multiple languages simultaneously while maintaining a good translation quality for each language pair and also having a remarkable BLEU scores. The Sample translations of Portuguese and Russian languages with single-task NMT and multi-task NMT models are shown in Table 2.

6 Conclusion

In this work, we explore the problem of how to translate multiple language pairs with different source languages and a common target language using a unified translation model. Our multi-task framework model was developed using a recently proposed transformer model based Encoder-Decoder architecture. Experiments show that for a given parallel training data, the multi-task NMT model is capable of learning different source language representations simultaneously while translating to a common target language. Our proposed system is efficient, faster, and has a better convergence with multi-task learning.

In the future, we would also like to extend our learning framework for translating regional languages and increase the number of language pairs for simultaneous training and also develop a multi-task learning framework using a transformer model for translating language pairs consisting of the same source language with a different target language and improve the translation quality of multiple languages within a unified model.

References

1. Sheridan Peter (1955) Research in language translation on the ibm type 701. IBM Techn Newsl 9:5–24
2. Schuster M, Paliwal KK (1997) Bidirectional recurrent neural networks. IEEE Trans Signal Process 45(11):2673–2681
3. Hochreiter Sepp, Schmidhuber Jürgen (1997) Long short-term memory. Neural Comput 9(8):1735–1780
4. Chung J, Gulcehre C, Cho KH, Bengio Y (2014) Empirical evaluation of gated recurrent neural networks on sequence modeling. arXiv:1412.3555
5. Bahdanau D, Cho K, Bengio Y (2014) Neural machine translation by jointly learning to align and translate. arXiv:1409.0473
6. Vaswani A, Shazeer N, Parmar N, Uszkoreit J, Jones L, Gomez AN, Kaiser Ł, Polosukhin I (2017) Attention is all you need. In: Advances in neural information processing systems, vol 30
7. Dong D, Wu H, He W, Yu D, Wang H (2015) Multi-task learning for multiple language translation. In: Proceedings of the 53rd annual meeting of the association for computational linguistics and the 7th international joint conference on natural language processing, (vol 1: Long Papers), pp 1723–1732
8. Bensalah N, Ayad H, Adib A, Ibn El Farouk A (2022) CRAN: an hybrid CNN-RNN attention-based model for Arabic machine translation. In: Networking, intelligent systems and security. Springer, pp 87–102
9. Shah P, Bakrola V (2019) Neural machine translation system of indic languages-an attention based approach. In: 2019 Second International Conference on Advanced Computational and Communication Paradigms (ICACCP). IEEE, pp 1–5
10. Qi Y, Sachan DS, Felix M, Padmanabhan SJ, Neubig G (2018) When and why are pre-trained word embeddings useful for neural machine translation? arXiv:1804.06323
11. Atrio ÀR, Popescu-Belis A (2022) Small batch sizes improve training of low-resource neural mt. arXiv:2203.10579
12. Devlin J, Chang M-W, Lee K, Toutanova K (2018) Bert: pre-training of deep bidirectional transformers for language understanding. arXiv:1810.04805

13. Ruder S (2016) An overview of gradient descent optimization algorithms. arXiv:1609.04747
14. Abadi M, Agarwal A, Barham P, Brevdo E, Chen Z, Citro C, Corrado GS, Davis A, Dean J, Devin M et al (2016) Tensorflow: large-scale machine learning on heterogeneous distributed systems. arXiv:1603.04467
15. Bird S (2006) Nltk: the natural language toolkit. In: Proceedings of the COLING/ACL 2006 interactive presentation sessions, pp 69–72
16. Kingma DP, Ba J (2014) Adam: a method for stochastic optimization. arXiv:1412.6980
17. Papineni K, Roukos S, Ward T, Zhu W-J (2002) Bleu: a method for automatic evaluation of machine translation. In: Proceedings of the 40th annual meeting of the association for computational linguistics, pp 311–318

Analysis of Various Hyperparameters for the Text Classification in Deep Neural Network



Ochin Sharma

Abstract A branch of artificial intelligence and machine learning called "deep learning" was developed to support machine automation in terms of regression and classification. To create an effective model in deep learning, a variety of variables play the necessary role. These variables include the activation function, the function of loss, the quantity of layers and the number of neurons, among other crucial variables. Due to time restrictions or projects having strict deadlines, it might be difficult and time-consuming to find the ideal working environment by adjusting all these aspects. The investigation of these factors that create the context in which deep neural networks operate is done in this research. The experimental findings will also be examined in order to determine the ideal setting for deep learning text classification. The professionals and researchers would be able to extract.

Keywords Accuracy · Loss function · Industry · Employability · Machine learning

1 Introduction

Deep learning is a recent field that has demonstrated its value with numerous modern technologies, particularly text classification (TC). There are numerous levels of neurons involved in deep learning. The number of neurons in such layers might range from a few to many thousands. The weights allocated to each input are multiplied by the input values, and the resultant is added. The activation function examines this outcome in more detail. This allows a deep learning model to achieve increased precision [1, 2]. Assigning labels to textual units like phrases, queries, paragraphs, and documents is the goal of natural language processing (NLP), also known as text classification or text categorization. It can be used for a variety of things, such as answering questions, sentiment analysis, news classification, spam detection, and user intent classification are a few examples. Text information can

O. Sharma (✉)
Chitkara University Institute of Engineering & Technology, Chitkara University, Punjab, India
e-mail: Ochin.sharma@chitkara.edu.in

© The Author(s), under exclusive license to Springer Nature Singapore Pte Ltd. 2023

S. Chinara et al. (eds.), *Advances in Distributed Computing and Machine Learning*,
Lecture Notes in Networks and Systems 660,
https://doi.org/10.1007/978-981-99-1203-2_49

be found in a variety of places, such as websites, emails, chats, citations, insurance claims, customer reviews, and social media.

Using either manual annotation or machine labelling, text can be classified. Automated text classification is becoming more crucial as data sets in industrial uses grow in size. Two sorts of automatic text classification methods can be distinguished:

- Rules-based procedures
- Machine learning techniques based on data

Rule-based approaches use a set of predefined rules to categorise text into multiple groups; they demand in-depth topic knowledge. Machine learning-based techniques, on the other hand, classify text based on data observations. A machine learning algorithm learns the innate correlations among texts and their labels using instances that have already been tagged as training data. Figure 1 illustrates how deep learning is indeed a topic that both experts and researchers are interested in. In the previous five years, there has been a significant increase in the number of deep learning-related professions. Applications for deep learning include image recognition, voice recognition, computer visualisation, natural language processing (NLP), bioinformatics, advertising, electronic commerce, digital marketing, and robotics [3]. Figure 1 displays the Googling trends in deep learning over the previous five years.

Although there are numerous excellent reviews and textbooks on text classification techniques and applications in general, such as Refs. [4, 20, 21], this survey is special in that it provides a thorough analysis of more than 150 deep learning (DL) models created for a variety of text classification tasks over the past six years, including sentiment analysis, news categorization, topic classification, question answering (QA), and natural language inference (NLI). We classify these works into a number of categories based on their neural network architectures, including attention, Transformers, Capsule Nets, recurrent neural networks (RNNs), and convolutional neural networks (CNNs).

The practise of classifying texts (such as tweets, news articles, and customer reviews) into structured categories is known as text classification (TC). Sentiment analysis, news categorization, and subject classification are examples of typical TC tasks. By enabling DL-based textual classifiers to accept a pair of texts as input, researchers have recently demonstrated that it is beneficial to cast various natural



Fig. 1 Google trend for deep learning over the past five years

language understanding (NLU) tasks as task clustering (TC), such as extractive question answering and natural language inference [5–7]. Along with the opportunities, there are some challenges associated with the area of text classification:

Modern Sets of Data for Complex Tasks: There is still a need for additional datasets for more difficult TC tasks including QA with multi-step thinking, text categorization for multilingual texts, and TC for incredibly lengthy documents even though many large-scale datasets have been gathered for popular TC tasks in recent years.

Building Knowledge through Logical Thinking: Similar to how people use common sense information to do various jobs, DL models that incorporate common sense knowledge have the potential to perform much better. For instance, a QA system with a knowledge base of common sense may respond to inquiries regarding the real world. In cases where there is insufficient information, common sense knowledge is sometimes helpful in finding solutions. Similar to how individuals think, AI systems can make "default" predictions about the unknowns using widely held views about common objects or concepts. Despite the fact that this concept has been looked into for sentiment analysis, much more study is needed to understand how to effectively model and exploit commonsense knowledge in Learning algorithms.

Deep Learning models are interpretable: Although DL models have shown encouraging results on difficult benchmarks, the majority of these algorithms are not comprehensible. Why, for instance, does one model perform better than another algorithm on one dataset but worse on others? What specifically have DL models discovered? What is the smallest possible neural net architecture that can produce a certain level of accuracy on a dataset? A thorough investigation of the underlying behaviour and dynamic of these models is still absent, despite the fact that the focus and self-attention mechanisms help to shed some light on these topics. Deeper models curated for different text analysis scenarios can be created with a better grasp of these models' theoretical underpinnings.

In this paper, In order to do this, a number of experiments have been carried out, including analyses of results using text information (Reuters) with various activation functions, analyses of results using various layer counts, analyses of results using various optimizers, and analyses of results using various loss functions.

2 Review of Literature

Different researchers [5, 6, 8, 9] have explored different activation functions and emphasis that although many new activation functions are being developed the competency of earlier activation functions is also not ruled out.

Optimizers [10, 11] offer a vital role in terms of deep learning model accuracy, for this, a few optimizers are quite important like Adam, SDG. So, these are included in this paper exclusively for experimentation.

Loss functions are used to minimize the errors so that maximum accuracy can be achieved in a deep learning model [7, 12]. A few loss functions that are used and discussed in [7, 12] are used are categorical_crossentropy, mean_squared_error, and mean_squared_logarithmic_error. So, in this paper, these loss functions are included to achieve optimal accuracy. The summary of other important reviews is placed in Table 1.

3 Research Gap

The research gaps identified in the literature are as follows. Since there are numerous optimizers, activation functions, and many other features. In order to identify the desired features to use for text classification, it is crucial to conduct extensive tests.

3.1 Numerous Layers

Choosing the right quantity of layers is crucial to employing deep learning to achieve the best results. Increased levels of layers may be required to address the more difficult problems.

3.2 Numerous Neurons

The amount of neurons that each layer should have is discussed in the next section because a neuron is nothing more than a basic processing unit.

3.3 Activation Function and its Features

Does the estimated processing output from a neuron's processing have value in the final output or will it provide an overall contribution that is false and needs to be discarded? Therefore, the activation function is essential in order to obtain this information. However, there are currently so many activation functions that, even when all other factors are used appropriately, the ideal outcome can still be far away without a thorough exploration.

Table 1 Review Summary of Some Important Studies

S. No	Title	Author	Findings	Knowledge gap	References
1	Deep learning–based text classification: a comprehensive review	Minaee, S., Kalchbrenner, N., Cambria, E., Nikzad, N., Chenaghlu, M., & Gao (2021)	The study explains various deep learning architectures for text classification	What is lacking in an architecture is an explanation of how altering various factors might increase accuracy	[8]
2	Deep Learning for Extreme Multi-label Text Classification	Jingzhou Liu, Jingzhou Liu, Yuexin Wu, Yiming Yang (2017)	XML-CNN model is created for binay and multiclasification environment	Utilised static environment,, could increase the accuracy of the outcome	[5]
3	Deep learning methods for subject text classification of articles	Piotr Semberecki; Henryk Maciejewski (2017)	Discussed the methodologies for auditing textual data	Converted words into vector representations. Worked more on pre-processing of data rather stable the experimental environment	[6]
4	Bag of Tricks for Efficient Text Classification	Bag of Tricks for Efficient Text Classification 2016)	Discussed Deep Learning classification based upon hardware prospective to achieve accuracy	The focus is on hardware prospectives rather improving accuracy by enhancing accuracy in model by using hyperparameters	[9]
5	Mohammed, A., & Kora, R. (2021). An effective ensemble deep learning framework for text classification. Journal of King Saud University-Computer and Information Sciences	AmmarMohammed, Raina Kora (2021)	Discussed outcomes by evaluating how well the suggested ensemble approach performs in comparison to existing cutting-edge ensemble methods	Ensemble approach with comparison using static environments	[10]

3.4 Function Decay

The weights' value is modified by a number given by the decay function, which must be multiplied by the static value of the input.

3.5 Optimization Algorithms

By changing the weight, optimization functions assist in lowering the overall expense of the model. Many optimizers are available, including SGD, Adadelta, Adam, Nadam, and more.

3.6 Numerous Epochs

The quantity of processing epochs and batch size: Epochs are the number of simulations or iterations required to use the same data collection and environment continuously. A smaller number of epochs may produce ineffective results, and picking more epochs would unnecessarily raise the cost of the learning algorithm [7, 12–16].

4 Methodology

The research is being done in the manner shown in Fig. 1.

i. The research question is chosen first.
ii. A comprehensive literature review is carried out.
iii. A small number of the total number of possible parameters (activation functions, optimizers, and loss functions) are chosen for experimentation based on the literature review.
iv. The model was written in Python using TensorFlow and Keras.
v. Based on the analysis of the experimental findings, determine the ideal working hyperparameters for text classification.

5 Examining the Ideal Deep Learning Text Classification Environment

Deep learning has been utilised to do text classification on the Reuters data set. Among the well-known and frequently used datasets for TC [4] is the Reuters–21,578 set. Although there is a lengthy array of activation functions, just a few

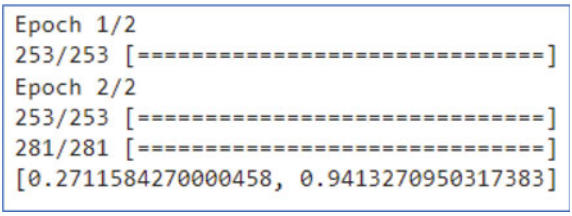


Fig. 2 Accuracy with 'Swish' activation function

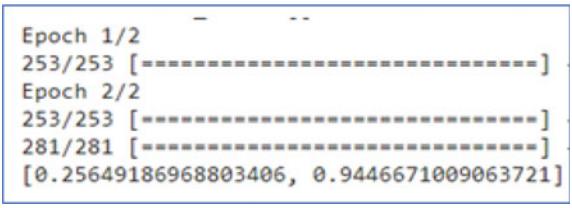


Fig. 3 Accuracy with 'elu + relu' activation function

significant activation functions have indeed been employed in Table 1 to evaluate accuracy [20–22]. The testing is being done based on numerous circumstances, such as examining findings by simulating various activation functions, simulating various layers, simulating various optimizers, and simulating various loss functions (Figs. 2 and 3).

By freezing loss function as categorical cross entropy, batch size by 32 and epochs by 2. The accuracy of various activation function is shown in the Table 2.

By freezing loss function as categorical cross entropy, batch size by 32 and epochs by 2, activation function: first layer Elu function is used and in second layer relu function is used. The accuracy is depicted with different number of layers and accuracy is listed in Table 3.

Table 4 displays the use of different optimizer functions and changes in the accuracy. It has been observed that for a text data set, adam optimizer should be a better choice.

Table 2 Analysing results by simulating upon different activation functions

Activation function at Hidden Layers	RELU	SELU	ELU	SWISH	Elu + relu	Selu + relu
Activation function at Outer Layer	Softmax	Softmax	Softmax	Softmax	Softmax	Softmax
Accuracy	93.20	94.2	94	94.04	94.44	94.19

Table 3 Analysing results by simulating different **Numbers of layers**

No of layer	1	2	3	4
Results	94.41	94.44	92.58	89.33

Table 4 Analysing results by simulating different optimizers

Optimizer	SGD	RMS Prop	Adam	Nadam
Accuracy	65.17	52.05	94.44	89.26

Table 5 Analysing results by simulating different loss functions

Loss function	Categorical cross entropy	Sparse Categorical Crossentropy	Poison loss	Hinge
Accuracy	94.44	50.02	47.20	45.19

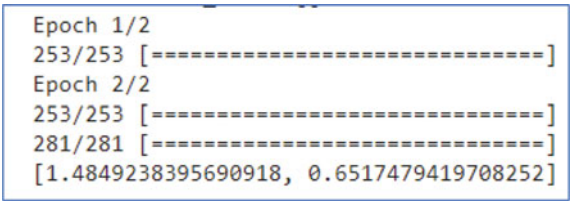


Fig. 4 Result with 'SGD' optimizer

Table 5 displays the analysis with various loss functions and changes noted in accuracy. For text data, it has been found that categorical_crossentropy is the desirable loss function to use. Further, Fig. 3 shows the result of experimentation while using the Elu function. The analysis shows that the accuracy depends upon various parameters and for text data 'elu + relu' activation functions in combination, categorical_crossentropy loss function, adam optimizer with two layers seems sufficient to obtain the adequate results (Fig. 4).

6 Conclusion

The text data set from Reuters has been analysed in the current paper using a variety of hyperparameters. According to the investigation, accuracy is dependent on a variety of factors. For text data, however, 'elu + relu' activation functions combined with 'categorical crossentropy loss function' and 'adam optimizer with two layers' seem to be enough to produce adequate results. Future plans could, however, include simulating our work on different text datasets and experimenting with various activation algorithms and loss functions.

References

1. Kukreja, V., Baliyan, A., Salonki, V., & Kaushal, R. K. (2021, August). Potato blight: deep learning model for binary and multi-classification. In 2021 8th international conference on signal processing and integrated networks (SPIN) (pp. 967–672). IEEE
2. Dhiman, P., Kukreja, V., Manoharan, P., Kaur, A., Kamruzzaman, M. M., Dhaou, I. B., & Iwendi, C. (2022). A novel deep learning model for detection of severity level of the disease in citrus fruits. Electronics, 11(3), 495
3. https://martin-thoma.com/nlp-reuters
4. https://www.tensorflow.org/
5. Liu J, Chang WC, Wu Y, Yang Y (2017) Deep learning for extreme multi-label text classification. In: Proceedings of the 40th international ACM SIGIR conference on research and development in information retrieval, pp 115–124. (Aug 2017)
6. Semberecki P, Maciejewski H (2017) Deep learning methods for subject text classification of articles. In: Federated conference on computer science and information systems (FedCSIS) 2017, pp 357–360. https://doi.org/10.15439/2017F414
7. Gajowniczek K, Liang Y, Friedman T, Ząbkowski T, Van den Broeck G (2020) Semantic and generalized entropy loss functions for semi-supervised deep learning. Entropy 22(3):334
8. Minaee S, Kalchbrenner N, Cambria E, Nikzad N, Chenaghlu M, Gao J (2021) Deep learning–based text classification: a comprehensive review. ACM Comput Surveys (CSUR) 54(3):1–40
9. Joulin A, Grave E, Bojanowski P, Mikolov T (2016) Bag of tricks for efficient text classification. arXiv:1607.01759
10. Mohammed A, Kora R (2021) An effective ensemble deep learning framework for text classification. In: Rasamoelina AD, Adjailia F, Sinčák P (2020) A review of activation function for artificial neural network. In: 2020 IEEE 18th world symposium on applied machine intelligence and informatics (SAMI). IEEE, pp. 281–286. (Jan 2020). (J King Saud Univ Comput Inf Sci)
11. Xu J, Wang X, Feng B, Liu W (2020) Deep multi-metric learning for text-independent speaker verification. Neurocomputing 410:394–400
12. Mutabazi E, Ni J, Tang G, Cao W (2021) A review on medical textual question answering systems based on deep learning approaches. Appl Sci 11(12):5456
13. Mai F, Tian S, Lee C, Ma L (2019) Deep learning models for bankruptcy prediction using textual disclosures. Eur J Oper Res 274(2):743–758
14. Sun T, Vasarhelyi MA (2018) Embracing textual data analytics in auditing with deep learning. Int J Digital Account Res 18
15. Mohan S, Fiorini N, Kim S, Lu Z (2018) A fast deep learning model for textual relevance in biomedical information retrieval. In: Proceedings of the 2018 world wide web conference. pp 77–86. (Apr 2018)
16. Peng S, Cao L, Zhou Y, Ouyang Z, Yang A, Li X, Jia W, Yu S (2021) A survey on deep learning for textual emotion analysis in social networks. Digital Commun Netw
17. Avinash Sharma V (2018) Understanding activation functions in neural networks (Nov 2018). https://medium.com/the-theory-of-everything/understanding-activation-functions-in-neural-networks-9491262884e0
18. Janocha K, Czarnecki WM (2017) On loss functions for deep neural networks in classification. arXiv:1702.05659
19. Minaee S, Kalchbrenner N, Cambria E, Nikzad N, Chenaghlu M, Gao J (2020) Deep learning based text classification: a comprehensive review. arXiv:2004.03705

Analysis and Prediction of Datasets for Deep Learning: A Systematic Review



Vaishnavi J. Deshmukh and Asha Ambhaikar

Abstract As time flows, the amount of data, especially text data, increases exponentially. Along with the data, our understanding of Machine Learning also increases and the computing power enables us to train very complex and large models faster. Fake news has been gathering a lot of attention worldwide recently. The effects can be political, economic, organizational, or even personal. This paper discusses the different analysis of datasets and classifiers approach which is effective for implementation of Deep learning and machine learning in order to solve the problem. Secondary purpose of this analysis in this paper is a fake news detection model that use n-gram analysis and machine learning techniques. We investigate and compare two different features extraction techniques, and three different machine classification datasets provide a mechanism for researchers to address high impact questions that would otherwise be prohibitively expensive and time-consuming to study.

Keywords Deep learning · Machine learning · N-gram · NLP · Text-classification

1 Introduction

The social network generates enormous amounts of information using many social media types. They have offered extremely large amounts of posts, which have caused the social media data on the internet to grow rapidly. When an incident occurs, a lot of individuals use social networks to discuss it online. They look up and debate news stories as part of their everyday practice [1]. Users, however, had to deal with the issue of information overload while searching and retrieving due to the extremely high number of news or posts. People are exposed to fake news, hoaxes, rumours, conspiracy theories, and misleading news through unreliable sources of information.

V. J. Deshmukh (✉) · A. Ambhaikar
Kalinga University, Raipur, Chhattisgarh, India
e-mail: deshmukhvaishnavi572@gmail.com; vjd03@rediffmail.com

© The Author(s), under exclusive license to Springer Nature Singapore Pte Ltd. 2023

S. Chinara et al. (eds.), *Advances in Distributed Computing and Machine Learning*,
Lecture Notes in Networks and Systems 660,
https://doi.org/10.1007/978-981-99-1203-2_50

This study provides these contributions [2]. We analyze multiple datasets for the automated detection of fake information in online news sources. Datasets are pre-processed in order to distinguish between fake and true news [3]. In order to improve a model's capacity for classifying data, approximating it, and making predictions, we then integrate a number of base models and classifiers utilizing ensemble learning. Base classifiers that are used are deep neural networks, k-nearest neighbors (KNN), long short term memory networks (LSTM), Support Vector Machine (SVM), and Naive Bayes [4].

When these classification models are combined, a stronger classifier is created that has the lowest error and the best predictive power of all the models. This kind of strategy is advantageous since it lowers the possibility of a classifier that performs exceptionally poorly by combining many models and taking the average to create a single final model [5]. The objective of this analysis is to determine the applicability of a hybrid machine learning in combination with deep learning techniques to the task to determine the false narrative on social media by implementing the effective analysis.

2 Background

The threat of fake news is expanding in our culture. Its detection is a difficult process that is thought to be significantly more difficult than that of detecting phoney product reviews [6]. Fake news is a serious threat to society and the government as well as to the credibility of some media sources. More than 69% percent of American and Asian adults are said to acquire their news from social media. Furthermore, it is very challenging to determine the reliability of the material in a timely manner due to the volume of information that is distributed and the speed at which it spreads on social networking sites. A significant research issue is the detection of false content in online and offline sources [7].

3 Datasets

The datasets that served as a benchmark for the algorithms are listed in this section. Three publicly available datasets, the Weibo dataset, the Twitter dataset, and the Lier datasets are used for the model training. To the best of our knowledge, these are the only databases that have both image and text information available.

(1) *Weibo dataset*: Weibo dataset in [8], fake news is tracked using the Weibo dataset. This dataset contained actual news from reputable Chinese news organisations like the Xinhua News Agency. The Weibo official gossip debunking system crawled and examined the phoney news from May 2012 to January 2016.

A committee of reliable users is looking into questionable posts, and this mechanism encourages certain users to report communications that seem suspect. For a fair comparison, we use the same pre-processing procedures as [8]. We first eliminate duplicates and poor-quality photos to guarantee the quality of the image. In a ratio of 9:4:6, we divided the dataset into the training, validation, and testing sets.

(2) *Twitter dataset*: This Twitter dataset was made available by MediaEval [2] as part of a contest to find fraudulent material on the social media site. Tweets, photos, and extra social context data make up the dataset. It consists of 17,000 distinct tweets about various incidents. 9,000 false news messages and 6,000 actual news items make up the training set. Here, a growing set was employed as a training set and a test set was used for testing. We exclude tweets with no text or images as well as any social context since we concentrate on multimodal false news [5].

(3) *Liar Dataset*: This dataset from the fact-checking website Politifact.com contains 12.8 K brief statements that have been human-labeled. An editor at Politifact.com assesses the veracity of each claim. The dataset has six fine-grained labels: The dataset has six fine- granulated markers pants- fire, false, slightly true, partial-true, substantially true, and true. The distribution of markers is fairly well- balanced [9]. For our purposes the six fine- granulated markers of the dataset have been collapsed in a double bracket, i.e., marker 1 for fake news and marker 0 for dependable bones. This choice has been made due to double Fake News Dataset point.

Three separate files make up the dataset [10]:

(1) Test Set: 1382 actual news and 1169 false news; (2) Training Set: 5770 real news and 4497 fake news; (3) Validation Set: 1169 phoney news stories and 1382 actual news stories. Because the three subsets are evenly distributed, no oversampling nor under sampling is necessary [11].

Preprocessing means that data set is clearer to our algorithm by removing dummy characters, string, and Impurities. Preprocessing Develop these three steps:

1. Splitting: Distinguish each statement from the next sentences so that you may address it separately.
2. Remove unimportant words from each phrase to stop word removal.
3. Stemming: This process locates each word's genesis.

3.1 Dataset Preprocessing and Model Implementation

One of the crucial jobs in natural language processing is the preparation and cleaning of datasets [3]. It is critical to eliminate irrelevant data. Links, numerals, and other symbolic elements were found in the articles that were utilised but were not necessary for feature analysis. The primary source of information for any NLP work is the statistics of word occurrence in a corpus. Regular Expressions are used to change

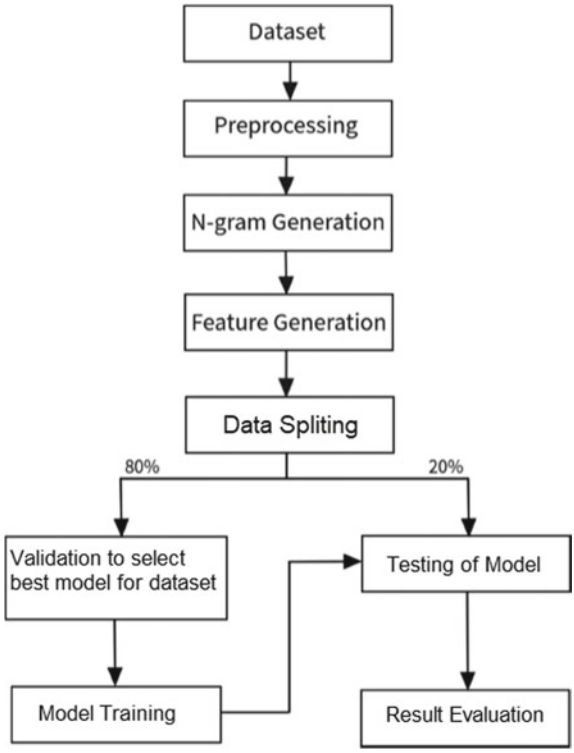


Fig. 1 Classification of dataset

symbolic letters into words, numbers into "numbers," and dates into "dates" [12]. Any source URLs are deleted from the text of the article. After that, the text in the body and headline is tokenized and stemmed. Finally, using the list of tokens, unigrams, bigrams, and trigrams are produced. The various feature extractor modules utilize both these grams and the original text [13] (Fig. 1).

4 Generalized Framework

Each module is described in depth in the sections that follow. Data collecting duties are handled by this module. Social network data, multimedia data, and news data may all be extremely diverse types of data. We compile the news text and any associated materials and images [1] (Fig. 2).

Pre-processing Module: This element is responsible for acquiring the incoming data flow. It carries out procedures for filtering, data aggregation, data cleaning, and data enrichment [14]. Processing module for NLP. It accomplishes the vital task of generating a binary classification of the news articles, i.e., whether they are fake news or trustworthy news. It has two smaller units [15].

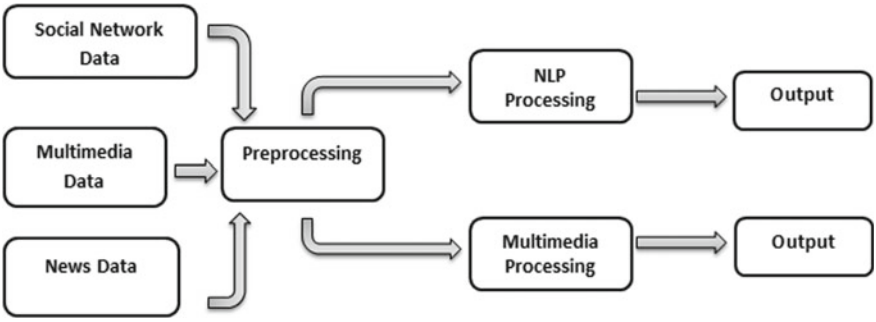


Fig. 2 Generalized framework

After a lengthy process of feature extraction and selection TF-IDF based to minimise the amount of extracted features, the Machine Learning module executes classification using an ad-hoc developed Logistic Regression method. Following a calibration phase for the vocabulary, the Deep Learning module uses the Google Bert algorithm to categories data. In order to evaluate the incoming data more effectively, it additionally executes a binary transformation and perhaps text padding [16].

Multimedia Processing Module: This module is designed for Fake Image Classification utilizing CNN and ELA (Error Level Analysis) Deep Learning algorithms. The objective of the paper is to review the deep learning algorithm on three standard datasets using a novel set of features and statistically validate the results using accuracies and F1 scores [17].

5 Result Analysis

Priyanshi Shah [18] suggested architecture for multimodal false news detection in this part, as shown in Fig. 1. Without taking into account any additional sub-tasks, the fundamental concept underlying our study is to detect bogus news from both modalities of provided tweets separately. Our model was broken down into three primary parts.

The first one is a textual feature extractor that pulls valuable information from textual material using sentiment analysis. The second component is a visual feature extractor that uses segmentation and preprocessing to extract images information from the post [19]. In order to extract the best characteristics, the feature representation from both components was run through a cultural algorithm. The last element is a fake news detector, which employs a classifier to identify false information [20] (Table 1 and Fig. 3).

Table 1 Performance of datasets on different parameters

Dataset	Method	Accuracy	Precision	Recall	F1
Weibo	Cultural algorithm	0.891	0.760	0.767	0.891
Twitter		0.914	0.719	0.789	0.795
Liar		0.873	0.822	0.811	0.866

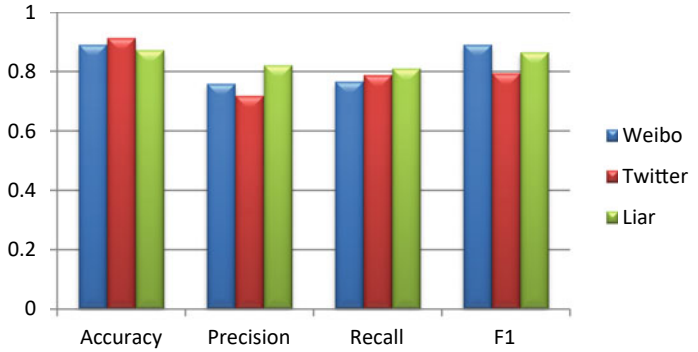


Fig. 3 Result analysis of dataset comparison

6 Conclusion

The analysis is effective for implementation for deep learning techniques are used to address the issue of spotting fake news in text and images. The models were developed using a tagged dataset of true and false news, and they excelled at this task. A larger data set and more intricate methods that explain how different modalities play a crucial part in the identification of fake news can still improve performance.

Future research on the subject will employ deep learning techniques to identify bogus news. Deep learning approaches, which typically produce superior performance results than conventional machine learning techniques, have been used by a number of researchers in this field to report encouraging findings.

References

1. Guo H, Cao J, Zhang Y, Guo J, Li J (2018) Rumor detection with hierarchical social attention network. In: Proceedings of the 27th ACM international conference on information and knowledge management, 2018, pp 943–951
2. Shu K, Sliva A, Wang SH, Tang JL, Liu H (2017) Fake news detection on social media: a data mining perspective. ACM SIGKDD Explor 19(1):22–36
3. Awad NH, Ali MZ, Duwairi RM (2013) Cultural algorithm with improved local search for optimization problems. In: 2013 IEEE congress on evolutionary computation. IEEE, pp 284–291

4. Yeh-Cheng C, Shyhtsun FW (2018) FakeBuster: a robust fake account detection by activity analysis. In: IEEE 9th international symposium on parallel architectures, algorithms and programming. pp 108–110
5. Ahmed H, Traore I, Saad S (2017) Detection of online fake news using N-gram analysis and machine learning techniques. In: Conference paper (October 2017)
6. Shu K, Wang S, Liu H (2017) Exploiting tri-relationship for fake news detection. arXiv:1712.07709
7. Potthast M, Kiesel J, Reinartz K, Bevendorff J, Stein B (2017) A stylometric inquiry into hyperpartisan and fake news. arXiv:1702.05638
8. Ruchansky N, Seo S, Liu Y (2017) CSI: a hybrid deep model for fake news detection. In: Proceedings of 2017 ACM on conference on information and knowledge management, Singapore
9. Granik M, Mesyura V (2017) Fake news detection using naive bayes classifier. http://ieeexplore.ieee.org/document/8100379/
10. The Verge: Your short attention span could help fake news spread (2017). https://www.theverge.com/2017/6/26/15875488/fake-news-viral-hoaxes-botsinformationoverloadtwitter-facebook-social-media. Accessed 16 Aug 2017
11. Qiang C, Michael S, Xiaowei Y, Tiago P (2012) Aiding the detection of fake accounts in large scale social online services. In: 9th USENIX conference on networked systems design and implementation, pp 1–14
12. Mauro C, Radha P, Macro S (2012) Fakebook: detecting fake profiles in online social networks. In: IEEE international conference on advances in social networks analysis and mining, pp 1071–1078
13. Tacchini E, Ballarin G, Vedova MLD, Moret S, de Alfaro L (2017) Some like it hoax: automated fake news detection in social networks. Technical Report UCSC-SOE-17-05, School of Engineering, University of California, Santa Cruz, CA, USA, 2017
14. Parikh SB, Atrey PK (2018) Media-rich fake news detection: a survey. In: 2018 IEEE conference on multimedia information processing and retrieval (MIPR). IEEE, pp 436–441
15. Myo MS, Nyein NM (2018) Fake accounts detection on twitter using blacklist. In: IEEE 17th International conference on computer and information and information science, pp 562–566
16. Waldrop MM (2017) News feature: the genuine problem of fake news. Proc Natl Acad Sci USA 114(48):12631–12634
17. Figueira A, Guimara˜es N, Torgo L (2018) Current state of the art to detect fake news in social media: global trendings and next challenges. In: WEBIST, 2018, pp 332–339
18. Shah P, Kobti Z (2020) Multimodal fake news detection using a Cultural Algorithm with situational and normative knowledge. IEEE Explore. 16 Sept 2020
19. He ZB, Cai ZP, Wang XM (2015) Modeling propagation dynamics and developing optimized countermeasures for rumor spreading in online social networks. In: Proceedings of. IEEE 35th international conference on distributed computing systems, Columbus, OH, USA
20. Guo C, Cao J, Zhang X, Shu K, Yu M (2019) Exploiting emotions for fake news detection on social media. arXiv:1903.01728

Lung Cancer Classification Using Capsule Network: A Novel Approach to Assist Radiologists in Diagnosis



S. NagaMallik Raj, Eali Stephen Neal Joshua, Nakka Thirupathi Rao, and Debnath Bhattacharyya

Abstract Convolution Neural Networks being computationally strong can automatically detect feature maps from the images. Although the performance of CNN is great, there is a loss of valuable information when we are using pooling layers. The concept of capsule networks, which overcomes the loss of information that occurs during pooling operations, is promising in deep learning. The system provides subpar results on benchmark data sets that have complex setups. Congruent with the excellent results achieved by Convolutional Neural Networks (CNNs), namely DeepCaps1, an architecture based on convolutional 3D models and a dynamic routing process. With Deep Caps, we significantly exceed the current state-of-the-art in the capsule network on LuNa-16, while achieving a large percentage decrease in parameter count. In addition, we present a decoder network (independent of class) that enables us to apply loss-reconstruction as a regularization term. Thus, we come to another fascinating attribute of the decoder, that of allowing us to manipulate the image's physical representation determined by its instantiation parameters.

Keywords Capsule networks · Convolutional neural networks · DeepCaps · LUNA16 · CapsNet · CT lung image analysis

S. N. Raj · E. S. N. Joshua (✉) · N. T. Rao
Department of Computer Science and Engineering, Vignan's Institute of Information Technology (A), Visakhapatnam, Andhra Pradesh 430049, India
e-mail: stephen.eali@vignaniit.edu.in

D. Bhattacharyya
Department of Computer Science and Engineering, Koneru Lakshmaiah Education Foundation, Vaddeswaram, Guntur, Andhra Pradesh 522302, India

E. S. N. Joshua
GITAM University, Gandhi Nagar, Rushikonda, Visakhapatnam, Andhra Pradesh 530045, India

© The Author(s), under exclusive license to Springer Nature Singapore Pte Ltd. 2023

S. Chinara et al. (eds.), *Advances in Distributed Computing and Machine Learning*,
Lecture Notes in Networks and Systems 660,
https://doi.org/10.1007/978-981-99-1203-2_51

1 Introduction

Cancer-related deaths worldwide are particularly high due to lung cancer. Cancer screenings with CTs at low doses are becoming increasingly common in the United States, and they may follow elsewhere. Based on a randomized control study examining more than 50,000 high-risk subjects in the United States., the National Lung Screening Trial has shown that using low-dose computed tomography (CT) lowers the risk of lung cancer-related death by 20% in comparison to radiography of the chest. According to 2013 statistics, the United States Public Health Service Task Force recommends screening of low-dose computed tomography for individuals who are at high risk for lung cancer [1], and within the next year, the Centers for Medicare and Medicaid Services will approve Medicare beneficiaries' eligibility for lung cancer screenings through CT scans. There will be millions of CT lung cancer screenings that must be analyzed during this screening program, making it challenging for radiologists to undertake. Therefore, developing algorithms that enhance screening has therefore become a very popular topic of interest. Detecting pulmonary nodules in CT scans of lung cancer is a vital step in the detection of lung cancer and may or may not be an indication of early-stage cancer. In order to achieve this task, many computer-aided detection (CAD) methods have been developed. To determine the effectiveness of these methods, the LUNA16 challenge evaluated automatic nodule detection algorithms based on the LIDC/IDRI data set.

2 Related Works

After COVID, the number of people who have been suffering from lung damage has substantially increased. The patient's chances of surviving the disease are very low if it is not discovered in its early stages. The early detection of lung cancer with screening using low-dose CT is highly effective in lowering lung cancer death rates. The use of artificial intelligence can aid in the diagnosis of this condition earlier through early detection procedures. By using CapsNet, one can retrieve spatial information and more important features to overcome the loss of information that is experienced from pooling operations.

Many computer vision tasks have been better solved using convolutional neural networks than using conventional curated feature-driven models in the last few years. The success of CNNs does not excuse their shortcomings, including pooling invariance and difficulties understanding spatial relationships between features. In order to overcome these limitations, Sabour et al. came up with Capsule Networks [2] on several standard datasets that have demonstrated comparable results to CNNs. To further enhance the capsule networks' performance, it makes sense to try to dig deeper into their architecture. Sabour et al. [2] propose a CapsNet model that consists only of a convolution layer and a capsule layer that is fully connected. The screening process using CapsNet is shown in Fig. 1.

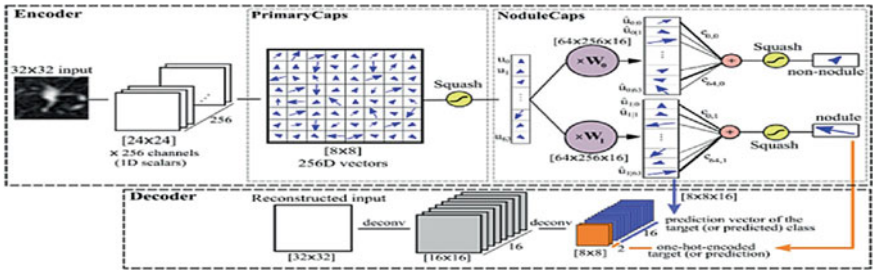


Fig. 1 CapsNet screening process

A deep CapsNet would be limited in its capabilities if such fully connected capsule layers are simply stacked, resulting in a similar architecture to an MLP. Considering that dynamic routing occurs in capsule networks, this approach is extremely computationally demanding; we have to train and infer more layers, so it incurs higher costs. Further, it was recently reported that stacking fully connected layers of the capsule heavily impacts middle layer learning [3]. Furthermore, too many capsules can inhibit learning because the coupling coefficients of multiple capsules are too small, dampening gradient flow. Furthermore, it was observed that correlated units tend to be concentrated in local regions, especially in lower layers [4]. Localized routing may use the above observation to its advantage, yet fully connected capsules cannot implement such localized routing.

Here are some suggestions for overcoming the limitations of stacking capsule layers. In order to minimize the complexity caused by the use of multiple layers demanding dynamic routing, several methods can be employed. Using fewer routing iterations at the beginning allows us to simplify the process without affecting the features since they do not need to be complex. Moreover, the middle layer of the algorithm is inspired by 3D convolution, thereby reducing the number of parameters - thanks to parameter sharing. Fortunately, it is possible to improve gradient flow, which involves skipping connections combined with convolutions, in order to resolve naive stacking's detrimental effects on learning. Furthermore, the deep capsule network should have the ability to handle richer data sets while reducing complexity. The localized routing approach has a better chance of capturing more detailed information compared to fully connected routing.

Sabour et al. [2] reduced overfitting by incorporating reconstruction error (generated by the decoder network) into the regularization procedure. While the emergence of deeper networks requires a stronger regularization than to reduce overfitting, the inherent complexity increase found with increased model depth requires a stronger regularization. Therefore, we propose a decoder independent of class in an effort to improve the regularization. We found that this decoder has the capability of providing control over the learning process as well as the perturbation of instantiation parameters. An instantiation parameter's physical representation cannot be guaranteed with currently available capsules and decoders, which are the same for every class. According to the proposed decoding method, the represented properties

are constant for any instantiation, resulting in a higher degree of control, something that is extremely helpful for both practical and theoretical applications.

Deep networks have gradients that disappear or explode, which is one of the major problems we face. In the midst of many layers, the error signal may dissipate and fade away when approaching the first layers [5, 6], reducing convergence. Numerous models propose using identity connections between layers, including ResNets [1] and Highway Networks [7]. Random depth [8] shortens ResNets by allowing better information and gradient flow by randomly dropping layers during training. A DenseNet is an interconnected network of layers (with corresponding feature-map sizes) that ensures maximum information flow within layers. As a result, each subsequent layer uses input from the previous layer and passes on its feature maps in a feed-forward manner. The early layers are linked by short paths to the latter layers.

Hinton et al. [9] described the idea of grouping the neurons utilizing an agreement between capsules as the basis for routing, Sabour et al. [2] proposed an algorithm for dynamically routing capsules. Network equivalence is achieved via dynamic routing, whereas invariance is only achieved through pooling for CNNs. Hinton et al. [10] used EM routing for assembling meshes from matrix capsules based on pose matrices in addition to dynamic routing. These have been extended numerous times: HitNet [11] implements a hybrid hit-and-miss functionality for augmented data Dilin et al. [12] introduce the divergence of the KL between distributions in order to solve dynamic routing as an optimization problem. In CapsGan [13], capsule networks are used to differentiate GANs, providing better results visually than convolutional GANs. Our work looks at increasing the performance of capsule networks on more complex datasets, rather than going deeper into the algorithms.

On the LUNA16 dataset, SegCaps [14] produced state-of-the-art results by utilizing capsules for image segmentation. The segmented lung images are in Fig. 2 along with original CT of the lungs and masks of the lung area. Based on routing, it is quite similar to ours. The voting system is based on 2D convolution. It uses 2D convolutions to mix the information in each capsule by taking the depth and all the capsules as inputs for the transformation. As a result, 3D-convolution-based routing designs stride along depth to be capsule dimensions, which allows each capsule to be voted separately along the depth dimension.

With multiple layers of capsules, we investigate the possibility of creating deeper networks. As far as we know, this is the first time that time capsule networks have been studied in depth. The capsule network instantiation parameters provide a novel means of representing images with regard to physical factors such as skewness and rotations. The reconstructed image's physical variations will change if you change an instantiation parameter. To date, it has not been determined which parameter causes the reconstructed images to change.

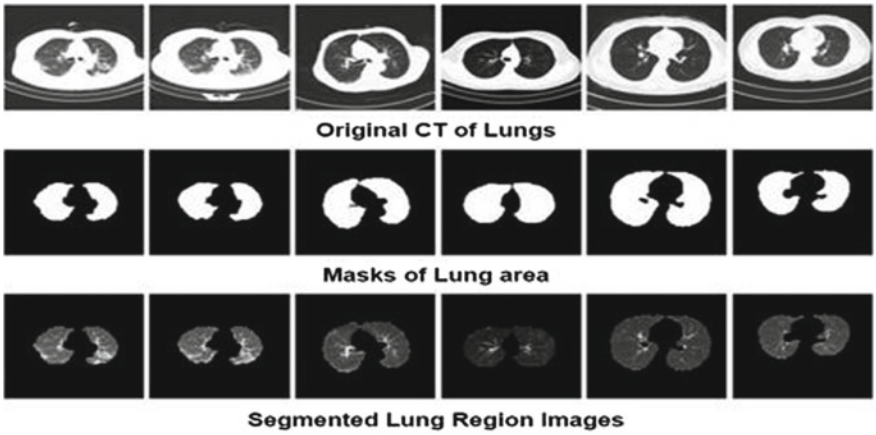


Fig. 2 Showing the segmentation regions of the image

3 Materials and Methods

By constructing deconvolutional layers [15] from DeepCaps instantiation parameters, we construct our decoder network. This decoder captures more spatial relationships than the fully connected layer decoder [2]. The loss function used in this study is binary cross-entropy [16].

As part of the development of DeepCaps, Keras and Tensorflow libraries were used. During our training, we employed Adam's algorithm, which employs a learning rate of 0.001, halved every 20 iterations. In the 7-ensemble model, the weighted average ensembling was used to train on V100 GPUs and GTX-1080.

4 Results and Discussions

DeepCaps is tested by comparing the performance of its architecture with that of the existing capsule networks with a benchmark dataset, LUNA16. As we use the sizes of the original images across our experiments, the 32*32*3 images are resized to 64*64*3.

However, our results are comfortably superior to all existing capsule network models, even though they are slightly less than or in line with the state-of-the-art. According to the implementations of capsule networks with the best results, the model proposed in [2] can be improved by 3.25%. Results from DeepCaps were comparable to those of state-of-the-art software. Based on our analysis, our results nearly surpass those of all other capsule network models on the datasets and achieve near state-of-the-art performance. The confusion matrix obtained from actual and predicted classes

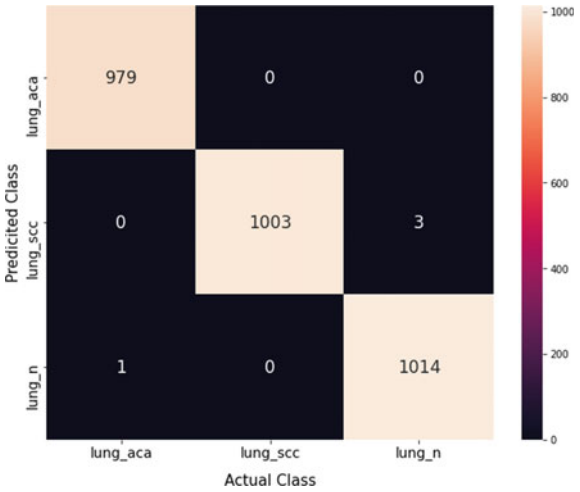


Fig. 3 Confusion matrix

Table 1 Sensitivity and specificity of the three classes of lung cancer

Lung Cancers	Sensitivity	Specificity	False positive rate	False negative rate
Lung adenocarcinoma	1	0.99	0.00	0.00
Lung squamous cell carcinoma	0.99	1	0.00	0.00
Benign lung tissue	0.99	0.99	0.00	0.00

is shown in Fig. 3. The sensitivity and specificity of the three classes of lung cancer along with false positive and false negative rates are mentioned in Table 1.

5 Conclusion

Even though capsule networks are relatively new in the field of medical imaging, they can enhance the detection capabilities of existing systems. We propose a DeepCaps architecture for Capsule Networks that combines 3D convolutions and skip connections. Through skip connections, backpropagation can be achieved among capsule cells. Using fewer layers at the bottom of the network leads to more routing iterations. Using capsule tensors and 3D convolutions, votes for dynamic routing are generated. Each time a group of capsules is grouped together, it can be routed to a higher-level group of capsules. Our study of capsules revealed more about them than Sabour et al. [2]. In addition, we described an intrinsically class-independent decoding network that functions to regularize DeepCaps. For all classes, we found that instantiation parameters learn specific changes based on activity vectors distributed in a

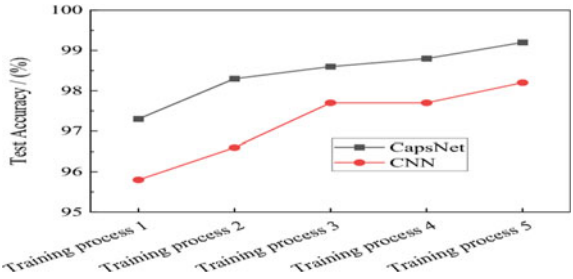


Fig. 4 Comparative analysis of CapsNet and convolutional neural networks

shared space. We believe these results are relevant to data generation applications and CapsNet is far better than Convolutional Neural Networks as shown in Fig. 4.

References

1. Jaiswal A, AbdAlmageed W, Wu Y, Natarajan P (2018) Capsulegan: generative adversarial capsule network. In: ECCV, Munich, Germany
2. LaLonde R, Bagci U (2018) Capsules for object segmentation
3. Zeiler MD, Krishnan D, Taylor GW, Fergus R (2010) Deconvolutional networks. In: CVPR, San Francisco, CA, June 2010
4. Jayasundara V, Jayasekara S, Jayasekara H, Rajasegaran J, Seneviratne S, Rodrigo R (2019) Textcaps: Handwritten character recognition with very small datasets. In: WACV, Waikoloa Village, HI, 2019
5. Wang D, Liu Q (2018) An optimization view on dynamic routing between capsules
6. Kingma DP, Ba J (2015) Adam: a method for stochastic optimization. In: ICLR, San Diego, CA, 2015. Accessed from 21 Nov 2016
7. Doppala BP, NagaMallik Raj S, Stephen Neal Joshua E, Thirupathi Rao N (2021) Automatic determination of harassment in social network using machine learning https://doi.org/10.1007/978-981-16-1773-7_20. www.scopus.com
8. Eali SNJ, Bhattacharyya D, Nakka TR, Hong S (2022) A novel approach in bio-medical image segmentation for analyzing brain cancer images with U-NET semantic segmentation and TPLD models using SVM. Traitement Du Signal 39(2):419–430. https://doi.org/10.18280/ts.390203
9. Eali SNJ, Rao NT, Swathi K, Satyanarayana KV, Bhattacharyya D, Kim T (2018) Simulated studies on the performance of intelligent transportation system using vehicular networks. Int J Grid Distrib Comput 11(4):27–36. https://doi.org/10.14257/ijgdc.2018.11.4.03
10. Joshua ESN, Battacharyya D, Doppala BP, Chakkravarthy M (2022) Extensive statistical analysis on novel coronavirus: towards worldwide health using apache spark. https://doi.org/10.1007/978-3-030-72752-9_8. Accessed from www.scopus.com
11. Joshua ESN, Bhattacharyya D, Chakkravarthy M (2021) Lung nodule semantic segmentation with bi-direction features using U-INET. J Med Pharm Allied Sci 10(5):3494–3499. https://doi.org/10.22270/jmpas.V10I5.1454
12. Joshua ESN, Bhattacharyya D, Chakkravarthy M, Kim H (2021) Lung cancer classification using squeeze and excitation convolutional neural networks with grad cam++ class activation function. Traitement Du Signal 38(4):1103–1112. https://doi.org/10.18280/ts.380421
13. Joshua ESN, Chakkravarthy M, Bhattacharyya D (2021) Lung cancer detection using improvised grad-cam++ with 3D CNN class activation. https://doi.org/10.1007/978-981-16-1773-7_5. Accessed from www.scopus.com

14. Neal Joshua ES, Rao NT, Bhattacharyya D (2022) Managing information security risk and internet of things (IoT) impact on challenges of medicinal problems with complex settings. Multi-chaos, fractal and multi-fractional artificial intelligence of different complex systems, pp 291–310. https://doi.org/10.1016/B978-0-323-90032-4.00007-9. Accessed from www.scopus.com
15. Neal Joshua ES, Thirupathi Rao N, Bhattacharyya D (2022) The use of digital technologies in the response to SARS-2 CoV2–19 in the public health sector. Digital innovation for healthcare in COVID-19 pandemic: strategies and solutions, pp 391–418. https://doi.org/10.1016/B978-0-12-821318-6.00003-7. Accessed from www.scopus.com
16. Rao NT, Neal Joshua ES, Bhattacharyya D (2022) An extensive discussion on utilization of data security and big data models for resolving healthcare problems. Multi-chaos, fractal and multi-fractional artificial intelligence of different complex systems, pp 311–324. https://doi.org/10.1016/B978-0-323-90032-4.00001-8. Accessed from www.scopus.com

Author Index

A
Abhishek Guru, 49
Affan Ahamed, S. MD. K. N. U., 187
Akanksha Tandon, 27
Akshay Kumar, 151
Alam, Md. Rittique, 39
Alekha Kumar Mishra, 163
Alka Dash, 175
Alok Ranjan Pal, 293
Alwabel, Abdulelah, 135
Amlan Sahoo, 175
Anish Monsley, K., 477
Antara Pal, 293
Anuradha Mohanta, 197
Aradhna Patel, 457
Arghya Biswasa, 209
Arikumar, K. S., 89
Aruna Gawade, 545
Aryan Pathare, 545
Asha Ambhaikar, 589
Ashish Kumar Mohanty, 15
Ashly Ann Jo, 261
Ashok, S., 433
Asis Kumar Tripathy, 305
Ayon, Roy D. Gregori, 231

B
Balaji Rajendran, 445
Banala Saritha, 477
Baratam Prathap Kumar, 433
Bhargava Satya Nunna, 569
Bibhudatta Sahoo, 49
Bidisha Bhabani, 1
Bilas Ghosh, 293
Bindhumadhava, B. S., 445
Buddha Singh, 487
Burhanuddin Savliwala, 545

C
Chakrapani Ghadai, 221
Chandrasekhar Rao, D., 15
Chinara, S., 59
Chinmaya Kumar Swain, 49, 135
Chinnahalli K Sunil, 421
Chitranjan Kumar, 521
Chowa, Sumaiya Binte Zilani, 39
Chowdhury, Anupam Hayath, 39

D
Dakshya Prasad Pati, 357
Dangeti Saivenkat Ajay, 369
Debachudamani Prusti, 305
Debnath Bhattacharyya, 597
Derawi, Mohammad, 101
Diksha Sharma, 209
Dipti Patra, 221
Dipti Pawade, 397

E
Eali Stephen Neal Joshua, 597
Ebin Deni Raj, 261
Esha Gupta, 397

G
Gaurav Sa, 209

© The Editor(s) (if applicable) and The Author(s), under exclusive license to Springer Nature Singapore Pte Ltd. 2023

S. Chinara et al. (eds.), *Advances in Distributed Computing and Machine Learning*, Lecture Notes in Networks and Systems 660,
https://doi.org/10.1007/978-981-99-1203-2

Gilad Gressel, 557
Golla Monika Rani, 151

H
Habiba, Umme, 231
Hritwik Arya, 421

J
Jaykumar Panchal, 397
Jill Shah, 397
Judhistir Mahapatro, 1

K
Kabir, S. Rayhan, 39, 231
Kavita Jhajharia, 369
Kaviya, P., 499
Kesari Sathvik, 321
Khan, Sabique Islam, 39
Killeen, Patrick, 333
Kiringa, Iluju, 333
Komeylian, Somayeh, 71, 127
Kotla Karthik Reddy, 421
Krishan Kumar, 101
Krishnapriya Ajit, 253
Kuldeep Singh, 381
Kuldeep Singh Jadon, 111
Kuswiradyo, Primatar, 209

L
Lakhan Dev Sharma, 209
Lavanya Palani, 445

M
Madapu Amarlingam, 433
Madhuchhanda Choudhury, 477
Madhuri Malakar, 1
Maheshwari Niranjan, 487
Manisha, 347
Mantosh Biswas, 277
Manu Elappila, 187
Mayank Dhiman, 111
Mitin, Shomoita Jahid, 231
Mohamed Basheer, K. P., 243
Mohanty, A., 59
Moswa, Audrey B., 333
Muddangula Krishna Sandeep, 509
Muneer, V. K., 243
Muthulakshmi, S., 321

N
NagaMallik Raj, S., 597
Nagamma Patil, 421
Nakka Thirupathi Rao, 597
Narendra Shekokar, 545
Neel Ghoshal, 465
Nidhi Sinha, 163
Nirmal Kedkar, 421
Nishu Gupta, 101
Nitin Gupta, 111

O
Ochin Sharma, 579

P
Paolini, Christopher, 71, 127
Parida, R., 59
Penikalapati Sai Akash Chowdary, 509
Pooja, S., 557
Pradhan, G., 59
Pramod Kumar Sethy, 49
Pranav Gupta, 321
Preeti Routray, 135

R
Rabbi, Md. Sanaullah, 231
Rabul Hussain Laskar, 477
Rahul Sahu, 305
Rajalakshmi Shenbaga Moorthy, 89
Ravichandra Sadam, 151
Rishabh Budhia, 533
Ritik Shah, 397
Rizwana Kallooravi Thandil, 243

S
Sachin Malayath Jose, 187
Sadeq, Muhammad Jafar, 39, 231
Sahaya Beni Prathiba, B., 89
Sahu, K., 59
Saipranav Syam Sitra, 321
Sambit Kumar Mishra, 49, 135
Sameer Chouksey, 433
Sandeep Kumar Sarowa, 101
Sangharatna Godboley, 151
Sanjeev Patel, 27
Santanu Kumar Rath, 305, 409
Sarkar, Mahasweta, 71, 127
Selvakumar, B., 499
Shakti Maheta, 347
Shibashis Sahu, 409
Shivam Patel, 457

Shourya Chambial, 533
Shyamala Devi, M., 509
Sneegdh Krishnna, 369
Sofana Reka, S., 253
Subasish Mohapatra, 175
Subhadarshini Mohanty, 175
Subhadip Mondal, 293
Subham Kumar Sahoo, 49
Subhashini, N., 321
Subhendu Rath, 409
Subrota Kumar Mondal, 175
Sucheta Panda, 357
Sudarsan, S. D., 445
Sudhanshu Kulshresha, 27
Sunil Kumar Kumawat, 277
Suvasini Panigrahi, 197
Swati Singh, 445

T
Tanisha Pandey, 533
Tripathy, A., 465, 533
Tripathy, B. K., 465, 533

U
Uday Dey, 293
Ujjawal Gupta, 111
Umakanta Nanda, 209

V
Vaibhav Bhartia, 465
Vaishnavi J. Deshmukh, 589
Vartika Mishra, 409
Vikram Singh, 381
Vinay Mathew Wilson, 187
Vipin Kumar, 521

Y
Yashwant Kumar, 111
Yeap, Tet, 333
Yeluri Praveen, 509

Z
Zubair Shaban, 101